

从 C++ 到计算系统：第二册

硬件原理、多核并行、高性能计算、同步、无锁结构与分布式计算

CPU Performance Study

本册接续第一册的程序执行基础，系统讲解现代 CPU 硬件、多核心并发、高性能计算、锁与无锁数据结构、并行算法、运行时和分布式计算。AI 模型、算子开发和推理引擎放入第三册。

前言

第一册已经回答了基础问题：一个 C++ 程序如何变成进程、地址空间、二进制接口、汇编指令和可测量的执行现象。它负责打牢源码到二进制、x86-64 汇编、System V ABI、Linux 基础工具、虚拟地址空间和可信 benchmark 的底座。第二册不再把这些内容重新讲一遍，而是把它们当作读者已经掌握的工具：能看汇编，能确认二进制身份，能写基本实验报告，能理解进程和虚拟地址空间的基本边界。

本册继续向下和向外展开，但不提前进入 AI。真正要理解高性能程序，必须先把“计算本身”讲清楚：核心怎样取指、译码、调度和提交，乱序执行为什么能隐藏延迟，分支预测为什么会失败，缓存和 TLB 为什么决定很多程序的上限，多核心之间为什么需要一致性协议，NUMA 为什么让同一块内存存在不同核心看来成本不同。第一册把单个程序放到机器上，第二册追问这台机器怎样扩展为多核心计算系统，再怎样扩展为多进程、多节点和分布式计算系统。

软件层面同样不能跳步。多线程不是简单地把循环切成几份；锁不是只有“慢”一个性质；原子操作不是更快的锁；无锁数据结构也不是不用同步。并行算法要面对数据依赖、负载均衡、局部性、归约顺序和任务粒度。分布式计算更进一步：一旦机器之间只能通过网络通信，延迟、带宽、故障、重试、超时、复制和一致性就会成为计算模型的一部分。

因此第二册的主题是计算系统，而不是某个应用领域。它从硬件原理开始，讲到单机高性能计算、多线程并发、锁与无锁结构、并行算法、运行时系统、异步 I/O 和分布式计算。AI 模型、张量、算子开发和量化推理引擎会放到第三册。这样安排不是延后重点，而是先把地基打牢：没有硬件和并发基础，后续任何 AI 算子优化都会变成经验堆砌。

核心思想

第二册的核心问题是：给定真实硬件、操作系统和网络边界，怎样组织数据、线程、同步、任务和通信，使计算能够正确、可测量、可扩展地运行。

本册仍然沿用第一册的写法：原理先行，代码作为证据；实验必须记录输入、环境、命令和误差；源码阅读抓数据结构、热路径、同步边界和性能瓶颈。不过，本册的证据重心会改变。第一册主要用反汇编、ELF、GDB、`readelf`、`objdump` 和基础 `perfstat` 建立“这段程序真的这样执行”的证据；第二册会更多使用拓扑、PMU、调度事件、锁等待、cache line 迁移、NUMA 放置、队列长度、吞吐曲线和失败注入来解释“系统为什么不能继续扩展”。

读者读完后，应当能解释一个多核程序为什么没有线性加速，能判断锁竞争和 false sharing 的差异，能写出可测试的并行归约、有界队列和环形队列，能读懂 Linux 调度、futex、RCU、内存管理和 perf 相关源码入口，也能把单机计算原则扩展到分布式系统。

怎样阅读本册

本册默认目标平台是本地 Linux 机器。普通 x86-64 Linux 台式机、笔记本或服务器适合完成完整实验；Termux 可以用于阅读、构建小实验和运行基础测试，但 perf、NUMA、线程亲和性、硬件计数器、futex 观察、eBPF 观测和分布式网络实验可能受权限、内核和设备能力限制。所有性能结论都必须写清环境。

阅读本册前，应默认已经完成第一册的最低要求：能把源码编译成二进制并确认产物身份，能读基本 x86-64 汇编，能解释调用约定和栈帧，能写出隔离热路径的 benchmark，能使用 Linux 常见工具观察进程、地址空间和基础性能计数。第二册不会反复解释这些背景；遇到这些内容时，本册只会说明它们怎样服务多核、同步、运行时或分布式问题。

阅读顺序建议严格按章节推进。前四章讲硬件执行模型，这是后面所有内容的解释基础。单机高性能计算部分讲测量、数据布局、SIMD 和典型内核，帮助读者把“程序慢”分解成计算、内存、分支、同步或 I/O 问题。并发部分讲线程、锁、条件变量、原子、内存序和无锁数据结构，重点不是背 API，而是理解共享状态怎样被保护、发布和回收。并行算法和分布式计算部分再把这些原则扩展到任务图、运行时、网络通信、复制和一致性。

本册的贯穿实验叫 Compute Systems Labs。它不追求一开始很大，而是从小而真实的模块开始：顺序归约、并行归约、带锁计数器、原子计数器、有界队列、任务切分和简单 benchmark。进入单机高性能部分后，实验主线会升级为事件流分析引擎：先做独立事件计数，再做会话 scan、滑动窗口、稳定 filter、thread-local histogram、维表 join 和 top-k。这样同一套输入既能训练顺序扫描、缓存局部性和 SIMD，也能训练顺序依赖、乱序缓冲、窗口状态和输出确定性。后续扩写会继续加入线程池、工作窃取、SPSC 环形队列、MPMC 队列、并行扫描、分布式 shuffle、窗口 checkpoint 和故障注入。

矩阵、图遍历和 stencil 会作为专项 kernel lab 出现，但不会替代事件流主线。矩阵适合讲 cache blocking 和 SIMD，图遍历适合讲随机访问和负载倾斜，stencil 适合讲邻域复用；事件流更适合作为整册贯穿项目，因为它能同时承载 parser、I/O、背压、manifest、重试、幂等、窗口和分布式 shuffle。

心智模型

读本册时始终追问四件事：数据在哪里，谁会读写它，硬件会怎样移动它，失败或竞争发生时系统怎样保持正确。

每章至少做一个实验。硬件章节要配合 `lscpu`、`numactl`、`perfstat`、拓扑图和内存访问 microbenchmark；并发章节要写出可重复触发竞争、阻塞、false sharing、CAS 失败或内存序问题的小程序；运行时章节要记录队列长度、任务粒度、worker 空闲时间和背压；分布式章节即使只在本机多进程模拟，也要记录延迟、重试、超时、重复请求和消息顺序。

常见误区

不要把高性能计算理解成“把线程数开大”。线程数只是资源请求，性能来自数据复用、负载均衡、同步控制和测量证据。没有这些证据，加线程常常只是更快地制造争用。

本册的章节之间有明确承接关系。第一部分回答硬件成本从哪里来，第二部分回答怎样把成本变成可测假设，第三部分回答共享内存里怎样保持正确，第四部分回答怎样把多个线程组织成可扩展运行时，第五部分回答当共享内存消失、网络和故障进入模型后，计算原则怎样变化。不要把某章当成孤立技巧阅读；每次做实验，都要把结论回连到这条链上。

全书结构

本册分为五个部分，围绕“计算系统如何正确且高效地扩展”展开。第一册已经承担源码、进程、虚拟地址、汇编、ABI、工具链和基础测量，本册只在需要建立新成本模型时短暂回扣这些内容，不再重复铺陈。这样处理的目的，是把篇幅留给第一册没有充分展开的主题：多核心硬件、缓存一致性、NUMA、SIMD 和内存带宽、Linux 调度和同步、无锁结构、并行运行时、异步 I/O、分布式计算和故障模型。

第一部分是硬件执行模型。它讲核心流水线、乱序执行、分支预测、寄存器重命名、缓存层级、TLB、页表、预取、NUMA、互连和缓存一致性。读者要在这里建立硬件成本模型：一条指令不只是语法，背后有取指、依赖、执行端口、访存、提交和一致性流量。

第二部分是单机高性能计算。第一册已经讲过 benchmark 纪律，所以本部分不再停留在“怎样计时”这类基础问题，而是进入多核进阶测量、Roofline、PMU 归因、cache 和 TLB 事件、数据布局、局部性、SIMD、指令级并行和典型计算内核。目标是把“优化”从口号变成可验证的分析：到底是算力不够、内存带宽不够、cache miss 太多、分支预测差、TLB 覆盖不足、同步等待太长，还是 benchmark 边界仍然写错。

第三部分是多线程、同步与内存模型。它讲线程生命周期、调度、亲和性、锁、条件变量、信号量、原子操作、内存序、false sharing、无锁队列和内存回收。这里的重点是正确性先于性能：如果可见性、互斥、等待谓词、线性化点、进展保证和对对象生命周期没有说清楚，所谓高性能就是不稳定的。本部分给项目留下四个可审查对象：[ThreadPoolPlan/WorkerSnapshot/ThreadTimeline](#) 用来解释线程执行资源，[QueueContract](#) 用来定义等待和关闭，[AtomicProtocolNote](#) 用来审查每个 atomic 的发布关系，[LockFreeLifecycleReport](#) 用来决定无锁结构是否值得进入主路径。

第四部分是并行算法与运行时。归约、扫描、分区、图遍历、任务图、线程池、工作窃取、异步 I/O 和背压，都是把多核硬件变成可用计算能力的桥梁。读者在这里要学会从算法依赖和数据移动角度设计并行程序。

第五部分是分布式计算。它把单机原则扩展到网络和集群：RPC、序列化、时间、故障、分片、复制、一致性、共识、shuffle、检查点和观测性。分布式不是“多几台机器”，而是把通信和失败纳入计算模型。

核心理想

第二册的主线是一条从硬件到集群的链：硬件决定局部成本，多线程决定共享内存内的协作，并行算法决定可扩展形状，分布式系统决定跨机器的通信和故障边界。

和第一册的边界

第一册已经讲清楚的内容，本册只作为前提使用：源码到目标文件和 ELF 的路径，进程和虚拟地址空间的基本概念，x86-64 汇编阅读，System V AMD64 ABI，C++ 与汇编互操作，基础 Linux 命令，基础 benchmark 设计，基础 [perfstat](#) 和 Linux 源码入口。第二册遇到这些内容时，重点不是重复定义，而是解释它们在可扩展计算中的新作用。

例如，第一册讲虚拟地址空间时，目标是让读者理解进程地址、映射和缺页；第二册讲 TLB 和页级性能时，目标是解释页大小、page walk、NUMA 放置和首次触页怎样改变吞吐。第一册讲 [perfstat](#) 时，目标是建立可信测量意识；第二册讲 PMU 时，目标是用事件组合区分前端、后端、内存、同步和调度瓶颈。第一册讲汇编控制流时，目标是读懂分支和跳转；第二册讲分支预测时，目标是理解输入分布、预测器状态和流水线回滚怎样限制单核吞吐。

本册扩展的重点

本册扩展不是简单增加术语，而是围绕可扩展计算的核心矛盾展开。硬件部分要能从 pipeline、ROB、load/store queue、store buffer、cache line、TLB、NUMA 和一致性协议解释成本。同步部分要能从不变量、等待谓词、futex、内存序、线性化点和内存回收解释正确性。运行时部分要能从

任务粒度、局部性、工作窃取、背压和关闭语义解释工程稳定性。分布式部分要能从消息、重试、幂等、复制、共识、shuffle、检查点和观测性解释跨机器计算。

每个章节都要有三个出口：能画出机制图，能写出最小可运行例子，能设计一份可复查实验。只有这样，第二册才不是第一册的复述，而是把第一册的基础能力推进到现代计算系统。

内容完整性和章节密度

第二册按正式书籍标准写作，但篇幅只是结果，不是写作目标。真正的标准是内容完整性和知识密度：每一章都必须把原理、机制、案例、实验和源码阅读边界讲清楚。硬件、多核、运行时、无锁结构和分布式计算的知识链如果需要更多篇幅，就按完整性展开；如果某段内容不能解释机制、判断或实践，就不应因为它增加页面而保留。

各章篇幅不必机械平均。硬件执行、缓存/TLB/NUMA、测量、锁与原子、无锁结构、任务运行时和分布式计算本身更难，应承担更多推导、代码和实验。实验路线和源码索引只能辅助正文，不能替代正文。后续扩展时，每章都要围绕同一条链展开：先讲模型，再讲硬件或系统实现，再讲 C++ 代码形态，再讲测量证据，再讲反例和工程边界。

常见误区

本册不接受重复定义、空泛口号、附录堆表、题目清单或模板化实验报告。若一个段落不能帮助读者理解现代计算系统的机制、判断或实践，就不应进入正文。

章节质量标准

每章必须按“机制、案例、代码、测量、源码、作业”六个维度写作。机制部分要解释底层原理，不只给术语定义；案例部分要从真实问题出发，展示为什么朴素方案不够；代码部分要给 baseline、改进版本和边界处理版本，不能只给最终答案；测量部分要说明实验矩阵、指标、预期现象和失败解释；源码部分要给 Linux 或生产系统的窄入口，并说明读源码要验证什么；作业部分要能训练读者独立复现、修改、测量和写报告。

本册的案例不能停留在“数组求和”“打印线程 ID”这种单点玩具上。简单例子可以作为入口，但必须继续推进到工程问题：线程数增加为什么不线性加速，锁竞争和 false sharing 怎样区分，TLB miss 和 cache miss 怎样分开，任务粒度怎样影响线程池，队列满时背压怎样传导，分布式 shuffle 为什么会被热点 key 拖垮。每个案例都要有问题背景、朴素方案、失败原因、改进方案、代价和实验验证。

大师级深度的写法

本册每个难点都按四层展开。第一层是直觉入口：用一个真实事故、反例或小程序说明读者为什么必须关心这个问题。第二层是原理模型：把语言语义、数据结构、CPU、操作系统和运行时之间的边界画清楚，让读者知道机制为什么存在。第三层是证据闭环：用代码、反汇编、性能计数器、trace、阶段报告或源码路径验证模型，不允许只凭经验口号下结论。第四层是工程判断：说明什么时候采用某个方案，什么时候不采用，代价、风险和结论边界在哪里。

“容易懂”和“有深度”不能互相牺牲。容易懂不是降低难度，而是把复杂机制拆成可跟随的因果链；有深度也不是堆砌术语，而是能解释失败边界、性能拐点和真实系统源码中的取舍。扩写每章时必须同时满足下面的标准：

- 每个核心概念先给可运行或可观察的具体情境，再给定义。
- 每个机制至少配一张图、一个最小代码例子或一个实验矩阵。
- 每个性能结论都要说明输入规模、数据分布、硬件限制和测量边界。
- 每个优化都要保留 reference、正确性 oracle 和失败输入。
- 每个高级主题都要连接到源码阅读、生产库设计或 Compute Systems Engine 的项目交付物。
- 每章末尾要有复查清单、习题、大作业和评分标准，训练读者自己复现和解释。

本册追求的“大师级”不是读者背熟所有 CPU 名词，而是读者能在新机器、新输入、新瓶颈下重新建立模型：先判断语义是否正确，再判断数据怎样移动，再判断硬件和运行时在哪里等待，最后用证据选择工程方案。若一章不能训练这种迁移能力，就还没有达到本册标准。

工业界优秀设计案例

本册必须系统讲解工业界成熟系统中的优秀设计，但不能把工业案例写成项目名罗列。每个工业案例都要回答五个问题：它面对的真实约束是什么；核心设计原理是什么；它牺牲了什么换取什么；它适合哪些输入、硬件和故障模型；读者怎样在本册实验或 Compute Systems Engine 中复现一个简化版本。

工业案例的写法遵循“案例、原理、取舍、迁移”四步。案例部分说明一个真实工程问题，例如列式执行为什么按 batch 处理、shuffle 为什么需要 manifest、线程池为什么需要 work stealing、日志系统为什么需要幂等提交。原理部分抽出可迁移机制，例如数据局部性、pipeline breaker、背压、状态快照、确定性输出、内存回收、分区路由和故障恢复。取舍部分说明它不是银弹，例如 batch 提高吞吐却增加延迟，packing 提高 GEMM 内核效率却引入额外拷贝，lock-free 降低阻塞却提高内存回收复杂度。迁移部分要求读者把设计缩小到本书项目中实现、测量和复盘。

每章至少应选择一个工业参照系。硬件和测量章节可以参照 Linux perf、PMU Top-Down、BLIS/OpenBLAS 的 kernel 组织；数据布局和内核章节可以参照 DuckDB、ClickHouse、Velox 或类似执行器的列式 batch、selection vector、hash aggregate 和 pipeline breaker；同步和运行时章节可以参照 oneTBB、OpenMP runtime、Folly executor、glibc/musl pthread 和 Linux futex；I/O 和分布式章节可以参照 Redis 事件循环、Kafka 分区与幂等、Spark shuffle、RocksDB compaction 和 checkpoint。正文不要求读者背这些系统的全部实现，但必须讲清它们优秀设计背后的核心原理。

常见误区

工业案例不能替代原理推导。若一段文字只是说“某系统这样做，所以很好”，它没有教学价值。必须说明为什么这样设计、解决了什么约束、引入了什么代价，以及读者如何用小实验验证。

贯穿大项目

第二册贯穿项目命名为 Compute Systems Engine。它不是一个孤立实验，而是整本书的工程主线。项目主场景采用本机可恢复的日志统计计算引擎：输入由 **InputSplit** 标识，串行 reference 固定语义，单核热循环用 **HotKernelReport** 保存输入族、反汇编、计数器和结论边界，优化路径把记录解析成 **HotBatch**，map 端做本地 combine，shuffle 阶段写出 **ShuffleBlock**，控制面通过 **ManifestEntry** 选择有效 attempt，**Checkpoint** 保存恢复边界，**RunReport** 输出正确性、性能、资源、故障和未验证边界。这个项目可以在一台机器上运行，却按分布式系统写语义：消息有身份，attempt 可以重复，响应可以迟到，输出必须幂等提交。

项目按阶段推进。第一阶段实现顺序 reference、输入身份、checksum、**HotKernelReport**、阶段报告和 benchmark harness。第二阶段加入数据布局、**HotBatch**、冷热字段分离、线程本地 partial、false sharing 对照和 NUMA 初始化策略。第三阶段加入 stable filter、scan、partition manifest、histogram、top-k、join、sort 和 graph frontier 等 kernel suite。第四阶段加入线程池执行资源报告、**ThreadPoolPlan**、**WorkerSnapshot**、**ThreadTimeline**、mutex 队列、条件变量、有界队列、**QueueContract**、原子计数器、**AtomicProtocolNote**、SPSC ring、ABA 和 **LockFreeLifecycleReport**。第五阶段加入任务图、work stealing 指标、ready count、continuation 和任务完成状态。第六阶段加入可靠写、短写处理、buffer pool、异步 completion、背压和 checkpoint。第七阶段加入 MessageBus、**MessageEnvelope**、request id、attempt、deadline、retry budget 和故障注入。第八阶段加入 route epoch、**OwnershipRecord**、热点 key 拆分、简化复制、恢复 audit 和可选两进程部署。

带 **event_time**、**ingest_time**、窗口和迟到数据的事件流实验仍然保留，但它是 I/O 和状态窗口的专项扩展，不再替代最终项目主线。矩阵、图遍历、stencil 和排序也作为 kernel suite 的专项实验保留。它们负责训练 cache blocking、邻域复用、随机访问、frontier 切换和排序/分区成本；最终日志统计引擎负责把这些内核连接到 I/O、并发、恢复和分布式提交。

每章扩写时都要回答两个问题：本章为 Compute Systems Engine 增加了什么能力；这个能力需要哪些测试和测量才能被认为可靠。若某章不能直接落到项目上，也必须说明它提供的是哪一种模型能力，例如分支预测帮助解释热循环，TLB 帮助解释大工作集，NUMA 帮助解释多 socket 扩展曲线，futex 帮助解释队列等待，page cache 帮助解释可靠写边界，MessageBus 帮助解释跨进程

调用栈消失后的状态机。

目录

第一部分	硬件执行模型：计算为什么会快也会慢	
第 1 章	计算系统全景	2
第 2 章	核心流水线、乱序执行与分支预测	46
第 3 章	缓存、TLB 与页级性能	56
第 4 章	NUMA、互连与缓存一致性	81
第二部分	单机高性能计算：从测量到数据流	
第 5 章	可信测量、Roofline 与性能计数器	105
第 6 章	数据布局、局部性与预取	127
第 7 章	SIMD、指令级并行与向量化	154
第 8 章	典型计算内核模式	178
第三部分	多线程、同步与内存模型	
第 9 章	线程、调度与亲和性	196
第 10 章	锁、条件变量与信号量	205
第 11 章	原子操作与内存序	217
第 12 章	无锁数据结构	230
第四部分	并行算法与运行时	
第 13 章	归约、扫描与分区	242
第 14 章	任务运行时与工作窃取	253
第 15 章	I/O、异步与背压	265
第五部分	分布式计算：把单机原则扩展到集群	
第 16 章	分布式计算模型	278
第 17 章	分片、复制与共识	295
第 18 章	分布式计算工程实践	307
附录 A	实验路线	317
附录 B	源码阅读索引	322

第零部分

硬件执行模型：计算为什么会快也会慢

第 1 章

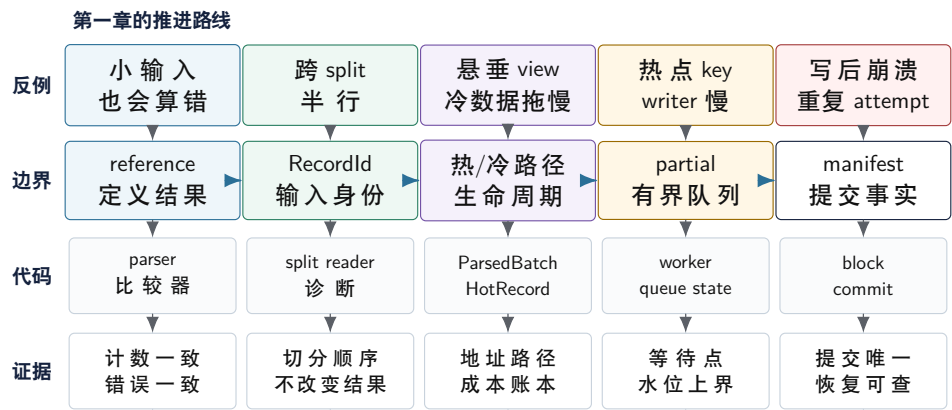
计算系统全景

1.1 问题与目标

现代计算系统的学习可以从一个很小的程序开始。假设手里有一批服务日志，每行记录一个时间戳、一个用户、一个事件名和一个数值。最初的任务很朴素：读取这些日志，统计每个事件出现了多少次，然后把结果写出去。这个任务看上去只需要循环、解析和哈希表；输入一小段文本时，几十行 C++ 就能完成。然而，计算系统中很多重要问题都藏在这个简单任务后面：输入变大以后，字节搬运会超过整数加法本身；线程增多以后，共享计数表会变成等待点；结果写出以后，临时文件存在并不等于结果已经生效；任务重试以后，同一个 split（输入切片）可能留下多个输出；扩展到多台机器以后，分片、shuffle（按 key 重新分发数据）、提交点和幂等性（重复执行不重复生效）会成为系统接口的一部分。

本章以这个日志统计作业建立全书的地图。它承担两个任务。第一，把读者已经能够理解的小程序拆成一组稳定边界：输入合同、reference（正确性基准）、记录身份、数据路径、执行实体、等待点、提交事实、恢复证据和实验报告。第二，把这些边界和后续章节连接起来：硬件章节解释一段循环为什么受流水线、缓存和 TLB（地址翻译缓存）影响；单机高性能章节解释数据布局、测量和向量化怎样改变成本；并发章节解释线程、锁、原子和队列怎样维护共享状态；运行时章节解释任务、依赖、取消和背压（下游慢时让上游减速）怎样组织执行；分布式章节解释同一套身份、提交和恢复原则怎样放大到网络和节点故障。

这条学习路径不是平铺概念，而是围绕同一个任务不断增加约束：



图示：每一列都从一个能破坏朴素程序的反例开始，向下落到边界、代码接口和验收证据；横向箭头表示这些边界会被后续章节继续放大。

第一册已经建立了单个程序的基础：源码怎样变成可执行文件，进程怎样获得虚拟地址空间，汇编指令怎样改变寄存器和内存，Linux 工具怎样观察运行结果。本册把同一个程序放入更完整的计算环境：多个核心同时执行，多级缓存保存数据副本，多个线程共享状态，多个任务在队列中等待，多个 I/O 请求在内核和设备之间排队，多个节点通过网络交换数据，多个失败路径随时可能打断正常流程。全书关注的核心问题可以概括为一句话：怎样让一个计算任务在更多资源、更复杂等待和更多失败边界下仍然正确、快速、可解释。

核心思想

计算系统不是若干概念条目的简单并列，而是一组约束共同作用后的结果。正确性规定结果是什么，硬件规定数据移动有多贵，运行时规定谁执行和谁等待，恢复协议规定结果何时有效，实验报告规定结论如何被复查。

学习目标

本章的阅读顺序是固定的。先用 reference 固定“什么叫算对”，再用 split 和 RecordId 固定“每条输入是谁”，接着用 batch 和冷热路径解释“字节怎样穿过机器”，再用 partial、有界队列和 trace 解释“谁执行、谁等待”，最后用 manifest、attempt 和 epoch 解释“结果何时真正生效”。后续章节只是在这条链上放大一个局部：硬件章节放大数据路径，并发章节放大等待和共享，I/O 与分布式章节放大提交和恢复。

本章读完后，读者应当能回答六类问题。第一，日志统计的正确结果怎样定义，reference 应当承担什么职责。第二，一条记录从输入字节到最终计数经历哪些形态，哪些字段属于热路径（反复执行的主流程），哪些信息属于冷诊断（少数错误或调试时才查看的信息）。第三，CPU 核心、缓存、TLB、内存带宽和 NUMA（多处理器机器中内存有远近差别）为什么会影响一个普通循环。第四，多个执行实体怎样协作，等待点怎样从隐藏状态变成可观察证据。第五，临时结果怎样通过 manifest（已提交结果清单）变成提交事实，失败后如何避免重复计算和重复合并。第六，后续章节如何沿同一条逻辑继续展开，形成一套可以用于阅读 Linux、数据库、计算框架和推理引擎源码的分析方法。

三条主线

全书可以沿三条主线阅读。第一条是语义主线，关心结果是什么、错误如何分类、身份如何保持、提交事实如何形成。它回答“算出来的东西能不能相信”。第二条是成本主线，关心字节怎样移动、缓存怎样命中、TLB 怎样覆盖、线程怎样占用核心、I/O 怎样排队、网络怎样放大延迟。它回答“时间和资源花在哪里”。第三条是控制主线，关心任务怎样进入队列、谁等待谁、失败怎样传播、取消怎样收束、恢复怎样重新开始。它回答“系统怎样持续推进”。

这三条主线不会在书中分离成三本小书。一个具体问题往往同时触碰它们。热点事件名首先是语义问题：所有 view 记录都必须计入同一个 key；随后是成本问题：同一个计数节点会被频繁访问；接着是控制问题：多个 worker 是否都要进入同一把锁。写出 partial 也是如此。语义上要说明 partial 是否属于最终结果；成本上要考虑写入 buffer、系统调用和 checksum；控制上要考虑 writer 阻塞时上游如何背压。读者阅读后续章节时，应当习惯把一个概念放回这三条主线，而不是把它当作孤立名词。

为什么用一个小项目贯穿

计算系统知识容易被写成并列清单：流水线、缓存、TLB、NUMA、锁、原子、无锁队列、线程池、异步 I/O、分布式协议。清单有助于索引，却很难帮助学习。读者真正需要的是因果顺序：一个小程序在哪个输入下暴露问题，哪个对象因此被引入，哪个实验证明它有效，哪个限制说明它还不完整。日志统计作业提供了这种顺序。它不依赖复杂业务，又能自然触发数据布局、并发、恢复和分布式问题。

贯穿项目还有一个好处：同一批输入可以反复使用。第二章用它观察循环依赖和分支；第三章用它观察缓存、TLB 和 page cache；第六章用它比较 AoS、SoA、字符串池和哈希表布局；第十章用它制造锁竞争；第十四章用它观察任务调度；第十五章用它验证 I/O 背压；第十六章以后用它模拟消息丢失和重复提交。输入不变，关注点变化，读者才能看清“同一个系统被不同章节放大”的感觉。

核心概念的操作语义

操作语义指一个对象在程序中的可检查含义：它对应哪个程序对象，允许哪些状态变化，哪些行为是合法的，违反边界后会产生什么错误。本章先给出后续章节反复使用的一组操作语义。这样定义之后，读者看到代码、实验报告或系统日志时，才能判断某个对象究竟承担了正确性职责、性能职责，还是恢复职责。

reference 是正确性基准。它通常是一份结构清晰、执行顺序确定、便于人工复查的实现，用来定义同一输入下应当得到的可观察结果。五行日志手算得到 view=2，reference 程序也必须得到 view=2；坏行被拒绝时，reference 还要给出 rejected records 的数量和对应的 RecordId。后续 SIMD 版本、多线程版本、异步 I/O 版本和分布式版本都要和 reference 比较。只要输入合同没有改变，优化版本就必须保持相同计数、相同错误分类和相同提交语义。Reference 因此承担三项职责：规定结果、生成期望值、为性能优化设置不可跨越的正确性边界。

split 是输入切片。大日志不能总是由一个线程从头读到尾，系统会把文件划成多个范围，例

如 split 0 处理前三行，split 1 处理后两行。Split 的定义包含两层约束：物理上它是一段字节范围，语义上它必须包含若干完整记录。若切分点落在一行中间，reader 需要移动到行边界；否则一条日志可能被漏掉、截断或重复处理。Split 让并行读取成为可能，也让失败恢复有了粒度：某个 split 失败时，系统可以只重做这一片输入。

partial 是局部结果。Split 0 得到 **view=1**，split 1 也得到 **view=1**，这两个结果都只是各自输入范围对全局答案的贡献；合并两个 partial 后才得到 **view=2**。Partial 的操作语义要求它能说明来源：来自哪个 split、哪个 attempt、使用哪个输入版本、覆盖哪些 key。缺少这些身份字段时，合并阶段只能看到一堆计数，却无法判断它们是否重复、过期或缺失。

attempt 是同一个逻辑任务的一次物理执行。Split 1 第一次执行写出了临时文件后崩溃，driver 可以再次调度 split 1；逻辑任务仍然是 split 1，物理执行却有 attempt 1 和 attempt 2。Attempt id 的作用是区分旧输出和新输出。没有 attempt id，系统会把“同一任务重跑产生的两个 block”误认为“两份不同输入的贡献”，最终把同一段日志算两次。

manifest 是已提交结果清单。目录中存在一个 block，只说明文件系统里出现了这个文件；manifest 中存在一条被接受的记录，才说明该 block 进入作业的提交事实，reduce 阶段可以读取它。对于写出后崩溃的 split，磁盘上可能残留多个临时 block，manifest 只能承认其中一个有效贡献。Manifest 把“文件存在”提升为“结果生效”，它是恢复协议和下游读取之间的边界。

backpressure 是背压，表示下游处理能力不足时，上游必须被限速。Reader 读 split 很快，writer 写磁盘很慢，如果中间队列没有容量上限，内存会持续上涨，延迟也会失去边界。背压的操作语义是：队列达到水位后，reader 暂停读取、worker 暂停产生更多 partial，或者调度器降低上游并发。背压牺牲一部分瞬时吞吐，换来有界内存、可解释延迟和更小的失败恢复范围。

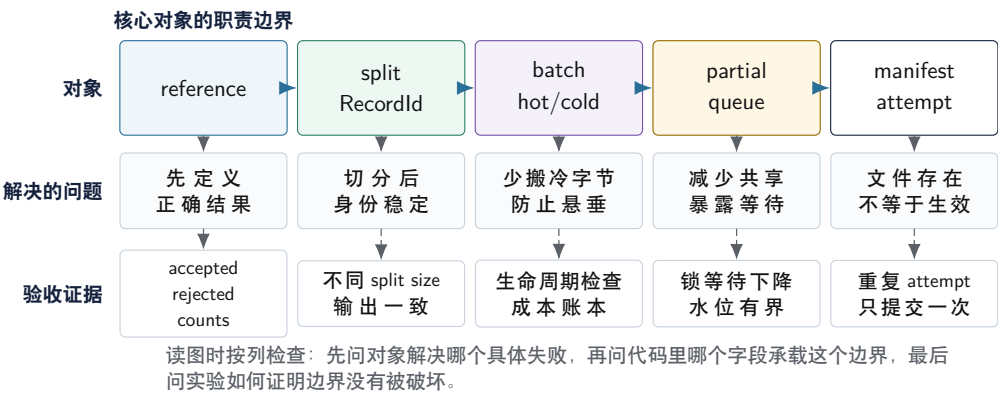
trace 是执行轨迹。它把 task 的关键状态按时间串起来，例如 queued、running、writing、committing 和 committed。普通日志只记录某一时刻发生了什么，trace 还要求按任务身份连接事件。系统变慢时，trace 能说明任务卡在等待队列、CPU 执行、写出阶段还是提交阶段；系统失败时，trace 能说明哪个 attempt 已经写出、哪个 attempt 已经提交、哪个响应已经过期。

热路径是每条正常记录反复经过的主流程，冷路径是少数错误、诊断或调试场景才会访问的流程。日志解析和计数属于热路径；错误原文、详细诊断和调试上下文属于冷路径。把冷信息放进热对象，会让 CPU 反复搬运暂时用不到的字节；把热字段集中起来，缓存行能容纳更多有效数据，也更容易进行批量处理。热路径和冷路径的划分，是数据布局章节的第一条成本边界。

TLB 是地址翻译缓存。程序访问虚拟地址时，CPU 需要把虚拟地址翻译成物理地址；TLB 保存最近使用过的翻译结果。若工作集过大、对象分散或访问缺乏局部性，TLB 命中率下降，循环会因为地址翻译而变慢。NUMA 是非一致内存访问模型，多插槽机器上的核心访问本地内存更快，访问远端内存更慢。Worker 在一个 CPU socket 上运行，却频繁读取另一个 socket 初始化的哈希表或 buffer，就会产生远端访问成本。TLB 和 NUMA 解释的是同一段 C++ 在不同数据布局和线程放置下为什么会出现性能差异。

epoch 是所有权轮次。假设某个分片原来归 worker A，后来迁移给 worker B。A 在旧轮次下发出的成功响应如果迟到，不能再写入 manifest；只有当前 epoch 的拥有者才有提交权。Epoch 把“谁现在有权宣布结果有效”编码进状态表，后续讲分片迁移、复制、leader 切换和模型热更新时，都要依靠它拒绝过期响应。

这些概念共同形成第一章的边界表：reference 定义结果，split 定义输入归属，partial 定义局部贡献，attempt 定义物理执行，manifest 定义提交事实，backpressure 定义等待传播，trace 定义可观测证据，热路径与冷路径定义数据布局，TLB 与 NUMA 定义硬件成本，epoch 定义所有权轮次。后续章节会不断扩展这张边界表，而不是重新发明一套分析语言。



图示：这张图把概念放回工程职责：每个对象都必须能对应一个失败模式和一条可执行验收。

1.2 贯穿全书的计算作业

本书使用一个贯穿项目来承接概念，项目名为 Compute Systems Engine。它的第一版只处理一种简单日志：

timestamp user_id event_name value

例如一批输入可以长成这样：

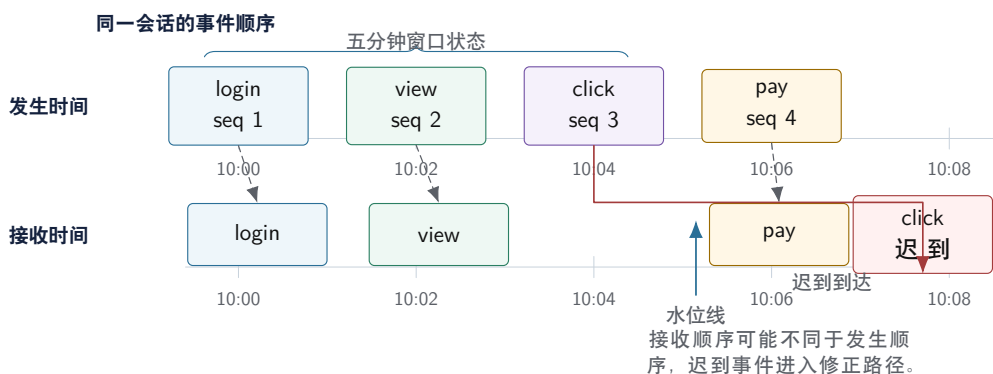
```
10:00 u001 login 1
10:01 u002 view 1
10:02 u001 click 1
10:03 u003 logout 1
10:04 u002 view 1
```

最小目标是统计 event_name 的出现次数。这个目标足够小，读者可以手工检查结果；同时它足够真实，规模、并发、I/O 和恢复都会自然出现。不过，事件计数只是第一层实验。若每一行都彼此独立，项目只能解释哈希聚合、锁竞争和 I/O 背压，很难解释顺序依赖、窗口状态、乱序到达、会话切分和状态机优化。因此，Compute Systems Engine 的正式方向不是“普通日志计数器”，而是“事件流分析引擎”：先用独立事件计数建立 reference，再逐步加入时间顺序、用户会话、事件序号、窗口聚合、转化链路和迟到数据。

第二层输入会扩展为事件流记录：

event_time ingest_time user_id session_id seq
event_name object_id value

event_time 是事件真实发生时间，ingest_time 是系统接收时间，二者不一定相同；session_id 和 seq 用来表达同一用户会话内的顺序；object_id 可以表示页面、商品、文件或请求对象。这样一批数据既能退化成简单计数，也能形成更复杂的实验：同一用户是否先 login 再 view，五分钟窗口内 click 次数如何变化，view->click->pay 转化链是否成立，迟到事件是否还能修正窗口结果。后续高性能章节会把这些顺序关系纳入复杂度分析：已按 user_id, event_time, seq 排好的输入可以线性扫描；完全乱序输入可能需要排序或按用户分区后局部排序；有界乱序输入需要重排缓冲、水位线和活动会话状态。



图示：事件流项目比普通日志计数更适合贯穿本册，因为它能自然引出顺序、窗口、乱序、状态和背压；水位线推进后，更早到达的事件会被归入迟到处理。

用日志系统作为主线是合适的前提，是把它定义成事件流系统，而不是停在独立行统计。矩阵、图遍历和图像 stencil 很适合做单独高性能内核实验，但它们不容易自然承载 parser、I/O、背压、manifest、重试、幂等和分布式 shuffle。事件流主线可以把这些系统问题串起来；矩阵、图和 stencil 则作为 kernel suite 的专项实验插入第二部分。后续章节会继续扩展同一个项目：先有串行 reference，再有 split、batch、冷热分离和局部聚合；接着加入会话状态、窗口聚合、稳定输出和维表 join；再加入有界队列、线程池、锁、原子、无锁结构和任务图；最后加入异步写出、manifest、故障注入、shuffle 和分布式 driver。项目每增加一个能力，都要回答语义、性能和恢复三个问题。

手工推导最小样例

先不写任何优化代码，直接手算上面五行输入。合法事件依次是 **login**、**view**、**click**、**logout**、**view**。因此 reference 的计数应为 **login=1**、**view=2**、**click=1**、**logout=1**，accepted records 为五，rejected records 为零。这个小结果看起来简单，却是所有后续版本的第一把尺子。只要后续 batch、partial、manifest 或分布式版本得到不同结果，就必须先解释语义是否改变；若语义没有改变，就是实现出错。

再把同一份输入切成两个 split。Split 0 包含前三行，局部结果是 **login=1**、**view=1**、**click=1**；Split 1 包含后两行，局部结果是 **logout=1**、**view=1**。合并两个 partial 后，**view** 的两个局部贡献相加得到二，其余 key 保持一。这个手算过程说明了 partial 的语义：partial 不是最终结果，而是某个输入范围对最终结果的贡献。只要每条记录只属于一个 split，且合并操作满足相同 key 相加，split 的执行顺序就不影响最终结果。

然后加入 manifest。假设 split 1 的第一次 attempt 已经写出临时 block，但在提交前崩溃；第二次 attempt 成功提交。手工推导时应写出两个事实：目录里可以有两个物理 block，manifest 中只能有一个被接受的逻辑贡献。Reducer 读取 manifest 后，只合并被接受 block；扫描目录则会把 split 1 统计两次，得到 **view=3** 和 **logout=2** 这样的错误结果。这个小样例让读者在纸上就能看出 manifest 的必要性。

最后加入错误行。若把第五行改成 **10:04u002**，reference 应变成 accepted records 为四，rejected records 为一，计数为 **login=1**、**view=1**、**click=1**、**logout=1**。错误诊断必须指向第五行的 **RecordId**。如果切成两个 split，诊断应仍然指向同一条逻辑记录。这个手工推导把输入合同、reference、split、partial、manifest 和诊断连在一起，也说明本章为什么从小样例开始：能手算，才能知道自动化测试在保护什么。

手算结果应尽快变成断言。第一组断言检查 reference：accepted、rejected、每个 key 的计数和错误列表都与手算一致。第二组断言检查 split：改变 split 大小和处理顺序后，最终计数不变，错误 record id 不变。第三组断言检查 partial：每个 partial 的局部贡献可以单独解释，合并后等于 reference。第四组断言检查 manifest：重复 attempt 产生多个物理 block 时，reducer 只读取被接受的逻辑贡献。第五组断言检查报告：输入摘要、输出 checksum、manifest 条目和故障注入结果都能被另一个人复查。

这些断言让小样例具备长期价值。后续修改 parser 时，它能发现字段规则被破坏；修改 batch 生命周期时，它能发现 **std::string_view** 悬垂；修改聚合容器时，它能发现 key 丢失或重复；修改 manifest 时，它能发现重复提交；修改报告脚本时，它能发现证据字段缺失。一个只有五行的输入，经过这样的组织，就成为整个系统的最小回归集。

回归集还应说明失败后的定位顺序。若 `reference` 断言失败，先看输入合同和 `parser`，而不是看线程池。若 `reference` 通过、`split` 版本失败，先看行边界、`offset` 和 `RecordId`。若 `split` 通过、`partial` 合并失败，先看每个 `partial` 的局部贡献和 `key` 合并。若 `partial` 通过、`manifest` 版本失败，先看 `attempt`、`block id` 和 `reducer` 是否扫描了临时目录。若所有小输入都通过，压力输入失败，再进入缓存、锁等待、I/O 和分布式重试分析。这个顺序能让调试路径保持稳定。

最小回归集也要和压力测试分工。小输入负责证明语义，压力输入负责暴露成本。五行输入可以证明重复 `attempt` 不应重复计数，却不能证明哈希表布局适合一千万个 `key`；热点输入可以暴露锁等待，却不适合人工检查每条诊断；冷盘读取可以暴露 I/O 成本，却不能替代 `parser` 边界测试。系统测试要分层，而不是用一个大输入承担所有责任。

因此，第一章的项目从一开始就应保存两类材料：一类是能手算的语义输入，一类是能放大成本的压力输入。语义输入进入每次提交的快速回归；压力输入进入章节实验和性能报告。后续读者修改任何模块，都能先用语义输入确认没有算错，再用压力输入观察瓶颈是否迁移。这样的测试组织比单纯追求代码长度更重要。

这些断言还要进入自动脚本。脚本先生成固定输入，再运行 `reference`，随后运行 `split`、`batch`、`partial`、`manifest` 四个版本，最后比较计数、诊断、`manifest` 和报告字段。每一步都保存机器可读摘要，例如 `accepted`、`rejected`、`checksum`、`manifest_entries`、`orphan_blocks` 和 `max_queue_depth`。这样一次失败能指向具体阶段，而不是只留下一个模糊的非零退出码。

自动脚本也要控制环境差异。语义回归应使用小输入、固定随机种子和稳定输出顺序；性能实验才记录机器型号、CPU 拓扑、编译参数、线程数和缓存状态。把语义回归和性能实验分开，可以让日常提交保持快速可靠，也让章节实验保留足够证据。后续项目如果接入 CI，先跑语义回归，再按需跑压力实验；这比每次都运行最大输入更适合长期维护。

比较程序的失败输出也要面向定位。计数不一致时，输出具体 `key`、`expected` 和 `actual`；诊断不一致时，输出 `record id`、`expected error` 和 `actual error`；`manifest` 不一致时，输出 `split id`、`accepted attempt` 和重复 `block`；队列指标异常时，输出队列名、最大水位和发生阶段。这样的失败信息能直接把读者带回本章的三张图：数据身份图定位是谁，执行等待图定位谁在等，成本账本定位时间或空间花在哪里。

读完本节后，读者可以用五个检查点自测。第一，能否手算五行输入的 `reference` 和两个 `split` 的 `partial`。第二，能否解释坏行为为什么需要 `RecordId`。第三，能否说明 `std::string_view` 为什么要求明确 `buffer` 生命周期。第四，能否用一句话区分临时 `block` 和 `manifest entry`。第五，能否根据比较程序的一条失败输出，判断应该先检查 `parser`、`split`、`partial`、`manifest` 还是 `runtime`。能回答这五个问题，说明最小样例已经真正转化为系统分析能力。

同时也要记住最小样例的边界。它擅长保护语义，却不能证明缓存局部性、锁竞争、I/O 吞吐或网络延迟。进入性能章节后，仍要使用更大的输入、更多 `key` 分布、更真实的等待和更完整的故障矩阵。小样例负责让系统不偏离正确性，压力实验负责让系统暴露成本；两者合在一起，才构成可靠项目。

若最小样例失败，性能数据应暂时搁置；系统还没有证明自己算对，任何吞吐数字都缺少解释价值。这也是本章反复强调 `reference` 的原因：`reference` 不是低性能版本的别名，而是团队讨论正确性的公共语言。先正确，再优化；先把结果、错误、身份和提交事实固定下来，再让机器更快地执行同一个语义。所有实验都应服从这条顺序。

为了让这条原则足够具体，最小样例的期望结果应写成机器可读的检查项，而不是只写在正文里。下面这份清单可以直接变成测试数据，也可以作为读者手工检查结果的账本：

Listing 1.1 五行输入的最小回归账本

```
case: normal_5_lines
input:
  0: "10:00 u001 login 1"
  1: "10:01 u002 view 1"
  2: "10:02 u001 cclick 1"
  3: "10:03 u003 logout 1"
  4: "10:04 u002 view 1"
```

```
reference:
  accepted_records: 5
  rejected_records: 0
  counts:
    click: 1
    login: 1
    logout: 1
    view: 2

split_plan:
  split 0: lines [0, 3)
  split 1: lines [3, 5)

partials:
  split 0: {click: 1, login: 1, view: 1}
  split 1: {logout: 1, view: 1}

manifest_failure_case:
  split: 1
  attempt 1: wrote block, not committed
  attempt 2: wrote block, committed
  reducer_entry: manifest only
  expected_final_count.view: 2
```

这份账本把一个小问题压成三类可检查事实：人工可以算出的结果，程序可以输出的断言，故障可以触发的反例。若某个优化版本输出 `view=3`，错误不是“差一点”，而是提交语义已经被破坏。若某个 `split` 版本把第五行坏输入报告成第二行，错误不是“日志格式问题”，而是记录身份已经不稳定。若某个性能报告缺少 `accepted_records` 和 `rejected_records`，即使吞吐很高，也没有证明自己处理的是同一个问题。

最小回归账本适合随代码一起保存。它不随机器型号、线程数和编译参数变化；它只随输入合同变化。以后如果决定支持引号、转义或多空格分隔，账本可以更新，但更新必须明确说明语义改变了什么。这样后续章节讨论缓存、锁、原子和分布式协议时，所有人仍然站在同一个正确性基线上。

规模放大与工作负载画像

十行输入和十亿行输入不是同一个问题的简单重复。十行输入中，文件打开、容器初始化和输出打印可能比真正计数还显眼；十亿行输入中，程序要处理数十 GB 甚至更多字节，解析、哈希、字符串分配、内存带宽、page cache 和写出策略都会进入主成本。十行输入可以人工检查坏行；十亿行输入必须把错误变成结构化诊断。十行输入崩溃后可以重新运行；长时间作业崩溃后需要知道哪些 `split` 已经提交，哪些 `attempt` 只是留下了临时状态。

规模放大还会改变设计取舍。逐行处理接口简单，但每条记录都要承受函数调用、边界检查和容器操作。按 batch 处理可以摊薄固定成本，却增加了 batch 生命周期、内存峰值和失败重做范围。Split 很小，调度和 manifest 条目会变多；split 很大，负载不均和失败重做成本会上升。线程很多，局部计算可能更快，但共享合并、内存带宽和 NUMA 放置可能成为新瓶颈。系统设计的难点不在于记住“批量化更快”这样的结论，而在于解释批量大小、任务粒度和提交粒度为什么适合当前工作负载。

工作负载画像

一句“处理一亿行日志”不足以描述工作负载。系统报告至少要写清总字节数、平均行长、最大行长、事件种类数、最高频事件占比、错误行比例、split 数量、输入是否命中 page cache、输出是否需要持久化。事件种类只有十种和事件种类有一千万种，聚合结构完全不同；最高频 key 占九成和 key 均匀分布，锁等待和局部聚合收益完全不同；错误均匀分布和错误集中在一个 split，取消和重试策略也不同。

事件流还必须记录顺序画像。输入是否按 `event_time` 全局有序，是否只保证同一 `user_id` 内有序，是否可能乱序到达；`event_time` 和 `ingest_time` 的最大差值是多少；同一会话的平均长度和最大长度是多少；迟到事件比例是多少；窗口宽度和滑动步长是多少；同一用户是否会跨 split；同一对象是否形成热点。没有这些信息，窗口聚合和会话分析的复杂度无法判断。已排序输入可以一遍扫描，复杂度接近 $O(n)$ ；乱序输入要么先排序，成本接近 $O(n \log n)$ ，要么按 key 分区后局部排序，成本转移到 partition、buffer 和合并；有界乱序则需要按水位线维护重排缓冲，空间复杂度取决于活跃用户数、迟到范围和窗口数量。

工作负载画像决定后续实验是否可信。若只用均匀 key 测试，就很难说明系统能处理热点；若只用内存中的输入，就不能说明冷盘读取性能；若只用小输入，就看不到哈希表扩容、TLB 覆盖和内存带宽拐点；若只用独立事件，就看不到顺序状态、窗口缓冲和乱序修正的成本。第一章要求从一开始就记录输入摘要和顺序摘要，是为了让后续每个性能结论都有适用范围。

从一行解析到串行 reference

最小问题是解析一行日志。给定一行文本，程序需要稳定取出第三个字段，也就是事件名。这一步看似简单，却已经包含输入合同。字段怎样分隔，空字段是否合法，最大字段长度是多少，最后一行没有换行符是否有效，错误行如何分类，这些规则决定后续所有版本的语义。

Listing 1.2 解析一行日志的事件字段

```

1 #include <cstdint>
2 #include <cstdint>
3 #include <string_view>
4
5 enum class ParseError : std::uint32_t {
6     none = 0,
7     empty_line = 1,
8     missing_field = 2,
9     event_too_long = 3,
10    malformed_value = 4
11 };
12
13 struct EventField {
14     bool ok = false;
15     std::string_view event_name;
16     ParseError error = ParseError::none;
17 };
18
19 EventField parse_event_field(std::string_view line)
20 {
21     constexpr char FIELD_DELIMITER = ' ';
22     constexpr std::size_t MAX_EVENT_NAME_BYTES = 64;
23
24     const std::size_t first_space = line.find(FIELD_DELIMITER);
25     if (first_space == std::string_view::npos) {
26         return {false, {}, ParseError::missing_field};
27     }
28
29     const std::size_t second_space =
30         line.find(FIELD_DELIMITER, first_space + 1);
31     if (second_space == std::string_view::npos) {
32         return {false, {}, ParseError::missing_field};
33     }
34

```



```

35     const std::size_t third_space =
36         line.find(FIELD_DELIMITER, second_space + 1);
37     if (third_space == std::string_view::npos) {
38         return {false, {}, ParseError::missing_field};
39     }
40
41     const std::size_t event_begin = second_space + 1;
42     const std::size_t event_size = third_space - event_begin;
43     if (event_size == 0) {
44         return {false, {}, ParseError::missing_field};
45     }
46     if (event_size > MAX_EVENT_NAME_BYTES) {
47         return {false, {}, ParseError::event_too_long};
48     }
49
50     return {
51         true,
52         line.substr(event_begin, event_size),
53         ParseError::none
54     };
55 }

```

这段代码有意保持简单。它处理单空格分隔，不处理引号、转义和跨行字段。这个起点的价值在于边界清楚：读者能一眼看出它支持什么、拒绝什么、返回什么。后续扩展复杂格式时，可以明确说明复杂性从哪里进入，而不是把所有规则一次性塞入 parser。

不过，简单 parser 也不能写成含糊 parser。上面的实现隐含了几个很具体的合同。第一，字段之间只能用单个空格分隔；如果输入是 `10:00u001view1`，第二个空格会让第三字段变成空片段，当前实现会把它判为缺字段。第二，它要求事件名后面还有第四个字段；因此 `10:00u001view` 会被拒绝。第三，它返回的是 `std::string_view`，这个 view 指向传入的 `line`，调用者不能在保存 view 后销毁或移动原始缓冲。第四，它虽然定义了 `malformed_value`，但当前没有解析第四字段，所以不会产生这个错误。若这些限制没有写进接口说明，枚举值会给人一种“所有错误类型都已经实现”的错觉。

边界测试可以把这些合同写成表，而不是只跑一条正常输入：

Listing 1.3 单行 parser 的边界输入

input	expected
-----	-----
"10:00 u001 view 1"	ok, event_name = "view"
" "	missing_field
"10:00"	missing_field
"10:00 u001 view"	missing_field
"10:00 u001 view 1"	missing_field
"10:00 u001 1"	missing_field
"10:00 u001 very_long... 1"	event_too_long

这张表让输入合同变得可执行。若项目后来决定支持多个空格，测试中的 `10:00u001view1` 就从失败样例移动到成功样例；同时代码要改成跳过连续分隔符，错误列号也要重新定义。若项目决定解析 `value` 字段，`malformed_value` 才真正进入可达状态。每一次语义扩展都会同时影响输入合同、reference、边界测试和后续实验。

还要注意 `std::string_view` 的生命周期。下面的伪代码是错误的：

Listing 1.4 悬垂视图的反例

```

1 std::vector<std::string_view> collect_events(
2     std::vector<std::string> lines)
3 {
4     std::vector<std::string_view> events;
5     for (const std::string& line : lines) {
6         const EventField field = parse_event_field(line);
7         if (field.ok) {
8             events.push_back(field.event_name);
9         }
10    }
11    return events;
12 }

```

函数返回时，`lines` 被销毁，`events` 中的 view 全部悬垂。这个反例比“零拷贝要小心”更具体：parser 返回 view 可以减少复制，但调用者必须让输入 buffer 活到 view 的最后一次使用。后续 batch 设计会把原始字节保存在 `ParsedBatch::input_bytes` 中，正是为了给热路径里的 view 一个明确所有者。优化不是删除复制这么简单，而是重新设计对象生命周期。

从多行输入得到 reference

单行解析确定以后，就可以写出串行 reference。Reference 的职责是固定正确性，而不是追求速度。它在固定输入上给出稳定输出，后续的批处理版本、多线程版本、SIMD 版本、异步 I/O 版本和分布式版本都要与它对齐。

Listing 1.5 串行 reference 的最小形态

```

1 #include <cstdint>
2 #include <string>
3 #include <unordered_map>
4 #include <vector>
5
6 using EventCountMap = std::unordered_map<std::string, std::uint64_t>;
7
8 struct ReferenceResult {
9     EventCountMap counts;
10    std::uint64_t accepted_records = 0;
11    std::uint64_t rejected_records = 0;
12 };
13
14 ReferenceResult run_reference_checked(
15     const std::vector<std::string>& lines)
16 {
17     ReferenceResult result;
18
19     for (const std::string& line : lines) {
20         const EventField field = parse_event_field(line);
21         if (!field.ok) {
22             ++result.rejected_records;
23             continue;
24         }
25
26         ++result.counts[std::string(field.event_name)];

```

```

27         ++result.accepted_records;
28     }
29
30     return result;
31 }

```

这个 reference 使用 `std::unordered_map`，并且把 `std::string_view` 复制成 `std::string`。这些选择会产生分配和哈希表局部性问题，但 reference 阶段先接受它们。此时最重要的目标是语义稳定：正常行被计数，错误行被拒绝，accepted 和 rejected 的数量可复查。性能优化要建立在这条语义基线上。

Reference 故意复制事件名，是因为它要切断输入 buffer 的生命周期风险。Parser 返回的 `std::string_view` 只在当前 line 存活期间安全；reference 把事件名复制进 map 后，计数表就不再依赖输入数组。这个设计牺牲性能，换来一个简单而可靠的正确性基准。后续优化版本可以使用字符串池、字典编码或批次所有权来避免逐条复制，但它们必须证明输出仍与 reference 一致。

Reference 还需要稳定输出顺序。直接遍历 `std::unordered_map` 会得到与实现、哈希种子和插入历史相关的顺序，不适合作为文本比较的黄金输出。输出前可以把 key 排序：

Listing 1.6 把 reference 结果转成稳定输出

```

1 #include <algorithm>
2 #include <utility>
3
4 using CountRow = std::pair<std::string, std::uint64_t>;
5
6 std::vector<CountRow> sorted_count_rows(
7     const EventCountMap& counts)
8 {
9     std::vector<CountRow> rows;
10    rows.reserve(counts.size());
11
12    for (const auto& [event_name, count] : counts) {
13        rows.push_back({event_name, count});
14    }
15
16    std::sort(rows.begin(), rows.end(),
17        [](const CountRow& lhs, const CountRow& rhs) {
18            return lhs.first < rhs.first;
19        });
20    return rows;
21 }

```

稳定输出让测试失败更容易读。若 view 的计数从 2 变成 3，比较程序应该直接报告 `key=viewexpected=2actual=3`，而不是输出两份顺序随机的 map 让读者肉眼比对。确定性是工程能力的一部分：它减少调试噪声，也让性能实验可以把注意力放在真正的差异上。

从小程序到系统流程

从代码表面看，日志统计只有读取、解析、计数和输出四步。把这四步放大后，每一步都会展开成系统边界。读取阶段要处理文件、offset、buffer、page cache 和 split。解析阶段要处理字段规则、错误分类和诊断定位。聚合阶段要处理 key 的身份、字符串生命周期、哈希表布局、热点 key 和计数溢出。输出阶段要处理临时文件、checksum、manifest、提交点和恢复。

边界越早清楚，后续章节越容易连接。缓存章节会放大热字段和冷字段的布局；线程章节会放大共享状态和等待点；锁章节会放大临界区保护的不变量；无锁章节会放大单写者和发布关系；I/O 章

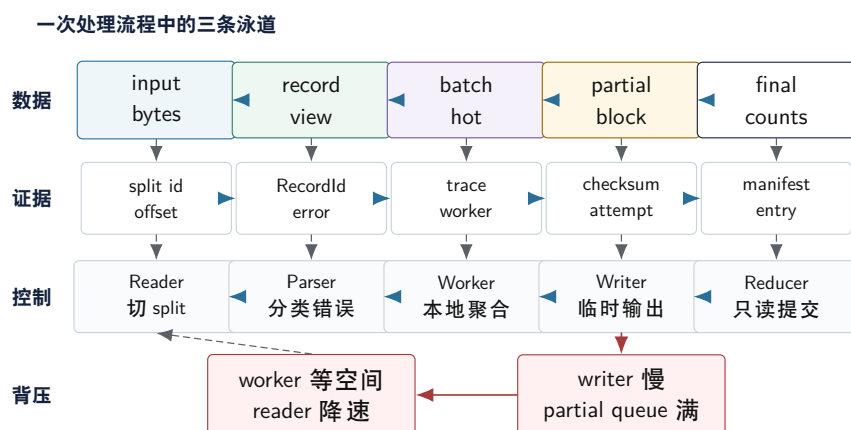
节会放大写出完成和结果生效的差别；分布式章节会放大 split、attempt、shuffle block 和 manifest 的身份关系。

心智模型

小程序隐藏了许多默认假设：输入很小、内存足够、顺序固定、不会失败、输出立即有效。系统设计从这些假设被逐步打破的地方开始。

一个具体处理流程

把日志统计写成流程，可以看到每一步都对应后续章节的主题。Reader 先读取一段字节，确定 split 边界；parser 扫描字段，产生 **ParsedRecord** 或诊断；batch builder 把热字段集中起来，并保存冷诊断；worker 在本地 partial 中聚合 key；writer 把 partial 写成临时 block；driver 把 block 提交进 manifest；reducer 读取 manifest 中的 block 并输出最终结果。



图示：上排是数据形态，中排是可复查证据，第三排是执行控制；背压单独占一行，避免把等待关系画进正常数据流。

这个流程的每个箭头都可能失败或变慢。Reader 可能遇到短读、编码错误或 page cache miss；parser 可能遇到坏行、分支误预测或 SIMD 尾部处理；worker 可能遇到哈希冲突、字符串分配、锁等待或 NUMA 远端内存；writer 可能遇到队列积压、短写或 fsync 延迟；driver 可能遇到迟到响应、重复 attempt 或 manifest 冲突；reducer 可能遇到热点 key、checksum 错误或输入 block 缺失。全书就是围绕这些箭头逐步放大。

1.3 正确性先于性能

系统工程的第一个问题是怎样判断结果正确。对于日志统计，正确性至少包括输入合同、解析规则、错误策略、计数语义、输出格式和重复执行语义。输入合同说明每行有哪些字段、字段如何分隔、最大行长是多少、编码是什么。解析规则说明事件名从哪里取、空字段如何处理、数值字段是否参与判断。错误策略说明坏行进入错误计数、诊断输出还是终止作业。计数语义说明重复行、重复文件和重试 attempt 如何影响最终结果。输出格式说明 key 是否排序、数值宽度如何选择、checksum 如何生成。

这些规则看起来像业务细节，实际是所有优化的地基。没有输入合同，就无法判断 split 能否从任意字节切开。没有错误策略，就无法判断 worker 发现坏行后是否应该取消全局作业。没有计数语义，就无法判断 partial 能否无序合并。没有输出格式，就无法自动比较优化版本和 reference。性能工程并不绕开这些细节，而是把它们变成可检查接口。

正确性决策应写成账本，而不是散落在 parser、worker 和 writer 的注释里。账本的每一行都要回答三个问题：系统选择了哪条语义，为什么这条语义会影响后续实现，用哪个测试固定它。下面这份最小账本足以覆盖第一章项目的主线：

Listing 1.7 第一章项目的正确性决策账本

```

decision: field delimiter
rule:      exactly one space
  
```

```

test:      boundary parser cases

decision:  missing event_name
rule:      reject record
test:      rejected count plus RecordId

decision:  last line without newline
rule:      accept if all fields are complete
test:      one-line input case

decision:  split starts inside a line
rule:      skip to the next full record
test:      split-size matrix

decision:  event counter width
rule:      uint64_t; overflow is an error
test:      overflow injection

decision:  output order
rule:      sort by event_name
test:      byte-for-byte compare

decision:  duplicate attempt output
rule:      manifest accepts one attempt only
test:      crash/retry case

decision:  orphan block
rule:      reducer ignores uncommitted blocks
test:      directory scan must not publish

decision:  late success response
rule:      reject stale epoch or attempt
test:      stale response case

```

这张表的价值在于把“正确”变成可维护对象。若以后决定支持多个空格，第一行要修改，parser 测试要迁移，reference 输出也要重新生成；若以后决定最后一行必须有换行符，第三行要修改，reader 和 split 测试也要跟着改。语义变化可以发生，但必须集中呈现，不能由某个优化版本悄悄改变。

记录身份让错误可追踪

单线程小输入可以靠调用栈理解当前处理哪一行。多线程之后，记录会被切成 split，放进队列，由不同 worker 处理，再合并成 partial。分布式之后，记录还可能跨进程、跨节点、跨 attempt 迁移。系统需要一种稳定身份来回答“这一条记录是谁”。

Listing 1.8 记录身份和结构化解析结果

```

1 #include <stdint>
2 #include <string>
3 #include <string_view>
4
5 struct RecordId {
6     std::uint32_t input_id = 0;
7     std::uint32_t split_id = 0;
8     std::uint64_t byte_offset = 0;

```

```

9     std::uint32_t line_index = 0;
10 };
11
12 struct ParsedRecord {
13     RecordId id;
14     std::string_view event_name;
15     ParseError error = ParseError::none;
16 };
17
18 ParsedRecord parse_record(RecordId id, std::string_view line)
19 {
20     const EventField field = parse_event_field(line);
21     if (!field.ok) {
22         return {id, {}, field.error};
23     }
24
25     return {id, field.event_name, ParseError::none};
26 }

```

RecordId 把记录和输入位置绑定起来。解析成功时，热路径可以只携带事件名和必要标识；解析失败时，错误报告可以通过 **RecordId** 回到原始输入。后续缓存优化、线程调度、异步写出和分布式调度都可能改变执行顺序，但只要身份保持稳定，系统仍然能解释每条数据的来源和命运。

错误策略必须进入接口

假设输入中某一行缺少事件字段。一个程序跳过它，一个程序把它计入 **bad_record**，另一个程序遇到错误立即终止。三种做法都有可能适合某个场景，但它们代表不同语义。并行化之后，错误策略会影响更多路径：一个 worker 发现错误时，其他 worker 已经写出的 partial 是否保留；队列中的任务是否取消；manifest 中已经提交的 block 是否仍然有效；错误报告是否要保存输入位置。

因此，错误需要成为接口的一部分。一个解析结果要能表达成功和失败，一个批次要能分别保存热事件和冷诊断，一个作业状态要能表达失败原因和影响范围。把错误写成结构化数据以后，后续章节讲锁、原子、I/O 和分布式时，错误路径就能和正常路径一起被分析。

Listing 1.9 结构化诊断的教学形态

```

1 struct ParseDiagnostic {
2     RecordId id;
3     ParseError error = ParseError::none;
4     std::uint32_t column = 0;
5 };
6
7 struct CheckedRecord {
8     RecordId id;
9     std::string_view event_name;
10    ParseDiagnostic diagnostic;
11 };
12
13 CheckedRecord parse_checked(RecordId id, std::string_view line)
14 {
15     const EventField field = parse_event_field(line);
16     if (!field.ok) {
17         return {
18             id,
19             {},

```



```

20         {id, field.error, 0}
21     };
22 }
23
24 return {
25     id,
26     field.event_name,
27     {id, ParseError::none, 0}
28 };
29 }

```

这里的 `column` 先保留为 0，后续 `parser` 更复杂时再由扫描状态填充。重要的是接口已经承认错误是系统语义的一部分。错误不再只是标准错误输出里的文字，而是可以被测试、比较和恢复逻辑查询的数据。

边界输入推动语义变清楚

Reference 要覆盖正常输入，也要覆盖边界输入。第一章的最小集合应包括空文件、只有一行、最后一行没有换行符、超长行、字段缺失、事件名重复、事件名极多、某个事件名占据大部分记录、输入被切成多个 `split` 后跨边界等情况。这些输入的目的不是增加测试数量，而是迫使语义变清楚。

例如，跨 `split` 的半行问题会迫使 `reader` 和调度器建立合同。若 `split` 可以从任意字节开始，`reader` 必须找到第一条完整记录，并把 `offset` 修正到行边界；若 `split` 必须由上游按行边界生成，调度器要保存这个事实。热点事件名会迫使聚合器说明共享计数表是否存在热点。错误集中在一个 `split` 会迫使取消和重试策略给出答案。边界输入越早进入实验，后续优化越不容易偏离正确性。

reference 的验收方式

Reference 的输出不能只靠人工阅读。最小验收应包含计数表、`accepted` 记录数、`rejected` 记录数、错误诊断列表和输出 `checksum`。优化版本运行后，比较程序检查这些结果是否一致。只比较最终计数会漏掉一些严重错误，例如某个 `split` 被重复计算而另一个 `split` 丢失，最终总数可能偶然相同；诊断和 `manifest` 能暴露这类问题。

Listing 1.10 比较计数结果的最小骨架

```

1 struct CompareMismatch {
2     std::string key;
3     std::uint64_t expected = 0;
4     std::uint64_t actual = 0;
5 };
6
7 std::vector<CompareMismatch> compare_counts(
8     const EventCountMap& expected,
9     const EventCountMap& actual)
10 {
11     std::vector<CompareMismatch> mismatches;
12
13     for (const auto& [key, expected_count] : expected) {
14         const auto found = actual.find(key);
15         const std::uint64_t actual_count =
16             found == actual.end() ? 0 : found->second;
17         if (actual_count != expected_count) {
18             mismatches.push_back({
19                 key,
20                 expected_count,
21                 actual_count

```

```

22         });
23     }
24 }
25
26 for (const auto& [key, actual_count] : actual) {
27     if (!expected.contains(key)) {
28         mismatches.push_back({key, 0, actual_count});
29     }
30 }
31
32 return mismatches;
33 }

```

真实比较还要检查错误诊断、accepted records、rejected records、manifest 条目和 checksum。本章先展示计数比较，是为了说明 reference 的使用方式：它进入自动验证链，而不是作为一份人工查看的输出。

比较器的输出也应结构化。一次失败如果只打印 **mismatch**，读者仍要回头翻日志。更好的做法是输出一份稳定摘要，让失败直接指向本章的某个边界：

Listing 1.11 验收器应输出的稳定摘要

```

summary:
  input_id: 0
  input_bytes: 128
  split_count: 2
  accepted_records: 5
  rejected_records: 0
  count_checksum: 0x0c32a910
  diagnostic_checksum: 0x00000000
  manifest_entries: 2
  orphan_blocks: 0

on mismatch:
  kind: count_mismatch | diagnostic_mismatch | manifest_mismatch
  key_or_record: "view" | "input=0 split=1 line=4"
  expected: ...
  actual: ...
  first_suspect: parser | split | partial | manifest | runtime

```

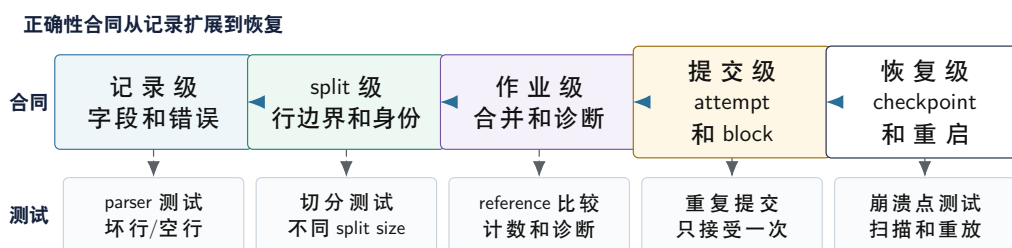
first_suspect 不是猜测实现细节，而是按照合同层次给出的定位入口。计数不一致先看 parser 和 partial；诊断不一致先看 **RecordId**；manifest 不一致先看 attempt 和提交点；等待指标异常才进入 runtime。这样的失败输出能把测试报告直接连接到源码边界。

正确性合同的层次

正确性合同也有层次。第一层是记录级合同：每行是否有效，事件名怎样提取，错误怎样分类。第二层是 split 级合同：一个 split 覆盖哪些字节或哪些行，跨 split 的半行怎样处理，split 内 record id 是否稳定。第三层是作业级合同：所有 split 的 accepted 记录合并后得到最终计数，错误诊断汇总后仍能定位到原输入。第四层是提交级合同：只有 manifest 接受的 block 进入 reduce，重复 attempt 不能重复计数。第五层是恢复级合同：崩溃和重启后，系统能够从 manifest、checkpoint 和临时目录恢复到一致状态。

这些层次会影响测试组织。记录级测试适合几十行输入，能手工检查字段和错误。Split 级测试要故意切分输入，验证行边界和 offset。作业级测试要比较串行 reference 和并行输出。提交级测试要故意制造重复 attempt。恢复级测试要模拟崩溃点和重启。若所有测试都只跑正常输入，系统看似

稳定，实际只是没有碰到边界。



图示：正确性不是一个总开关，而是一组逐层扩大的合同；每一层都要有对应的反例和测试。

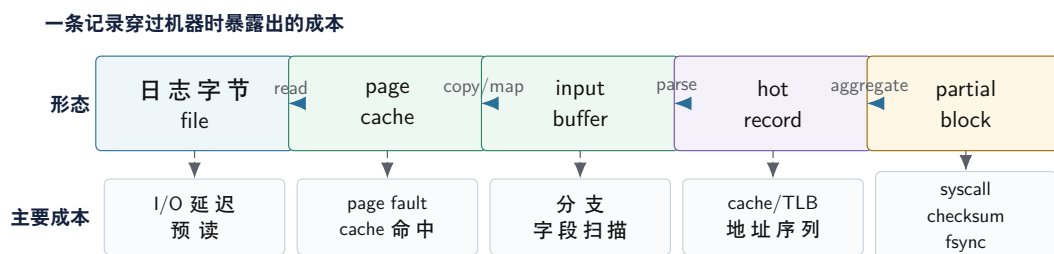
计数溢出和确定性

日志统计还会遇到两个容易被忽略的问题。第一个是计数溢出。示例使用 `std::uint64_t` 保存计数，但真实系统仍要说明溢出策略：拒绝输入、饱和、报错，还是使用更宽表示。第二个是输出确定性。哈希表遍历顺序不稳定，如果输出文件用于比较、缓存或下游处理，就需要排序或稳定编码。确定性不是为了美观，而是为了让同一输入在多次运行、多线程调度变化和不同机器上产生可比较结果。

确定性还会影响浮点和近似算法。日志计数是整数加法，合并顺序不会改变数学结果；后续若统计平均值、方差、分位数或机器学习指标，合并顺序可能影响舍入误差。第一章先用整数计数建立稳定语义，后续章节再逐步引入更复杂的数值边界。

1.4 数据路径与硬件成本

当输入从几百行变成几亿行，程序的主要成本通常不再是几次整数加法，而是数据怎样穿过机器。日志字节从存储设备进入内核页缓存，进入用户态 buffer，进入解析器，进入聚合结构，最后进入输出文件。每一次复制、解析、哈希、分配和写回，都可能比 `++count` 本身贵得多。硬件执行模型从这条数据路径中出现：CPU 核心怎样执行循环，缓存按什么粒度搬数据，TLB 如何缓存地址转换，内存带宽何时成为上限，NUMA 怎样让同一台机器内部也出现远近差别。



图示：硬件成本来自具体数据路径：同一条记录在不同阶段会变成顺序字节、视图、哈希 key、局部计数和提交 block。

热路径和冷路径

朴素 reference 把解析、诊断和聚合混在一起。小输入时影响很小；大输入时，热循环会反复触碰许多暂时无关的字节。日志统计的热数据包括事件名起止位置、事件名哈希值、事件 id 和计数。冷数据包括原始行文本、错误详情、输入文件名、调试标签和 trace。热数据被频繁访问，应该尽量紧凑；冷数据仍然重要，但适合放在诊断路径上，通过身份回查。

Listing 1.12 把热字段和冷字段分开

```
1 #include <cstdint>
2 #include <string>
3 #include <string_view>
4 #include <vector>
5
6 struct HotRecord {
```

```

7     std::uint64_t event_hash = 0;
8     std::uint32_t event_begin = 0;
9     std::uint32_t event_size = 0;
10    std::uint32_t local_line_index = 0;
11 };
12
13 struct ColdRecordInfo {
14     RecordId id;
15     std::string original_line;
16     ParseError error = ParseError::none;
17 };
18
19 struct ParsedBatch {
20     std::string input_bytes;
21     std::vector<HotRecord> hot_records;
22     std::vector<ColdRecordInfo> cold_infos;
23 };

```

这段代码表达一个原则：热路径携带反复计算所需的最小信息，冷路径保存诊断和恢复上下文。CPU 按 cache line 搬运数据，而不是按 C++ 字段的语义重要性搬运数据。把热字段集中起来，可以让一次缓存行加载服务更多有效工作；把冷字段混入热对象，会让聚合阶段无意搬入大量暂时用不到的字节。

从字节流到结构化记录

读取文件时，程序看到的是字节流；解析之后，程序才知道哪些字节构成时间戳、用户、事件名和值。这个转换决定并行性。若每条记录独立，多个 worker 可以处理不同 split；若格式允许引号、转义和多行字段，解析状态可能跨字符传播，切分策略就要更谨慎。输入格式越复杂，reader 和 parser 的合同越重要。

日志统计的教学版本先采用每行独立的格式。这个起点能清楚展示 split、batch、partial 和 manifest 的主线。后续支持更复杂日志时，新增规则可以逐个落到 reader、parser 和诊断对象上。这样读者能看到复杂性来自输入合同，而不是来自某个抽象名词。

连续访问和随机访问的差异

连续数组通常让硬件更容易工作。CPU 读取某个地址时，缓存会以 cache line 为单位搬入一段连续字节；硬件预取器也更容易识别顺序访问模式。链表节点、哈希表节点和散落小对象会让访问地址难以预测，带来更多 cache miss 和 TLB 压力。两个版本都可能是 $O(n)$ ，但连续扫描和随机追指针的机器成本完全不同。

`std::unordered_map` 在 reference 中很方便，但它通常包含桶、节点、字符串和分配器状态。每个 key 可能是独立分配的字符串，每个节点可能分散在堆上。对于事件种类较少的日志，线程本地 partial 可以把共享更新推迟到合并阶段；对于事件种类很多的日志，字符串池、开放寻址哈希、排序归约或分片哈希可能更合适。选择方案时应观察工作负载：事件种类数、最高频 key 占比、平均字符串长度、错误率、split 大小和内存预算。

复杂度和机器成本共同工作

大 O 标记描述规模增长趋势，机器成本描述硬件如何执行。朴素扫描、批量解析、本地聚合和排序归约都可能处理每条记录一次，但地址序列、分配次数、分支行为和写入模式不同。同样是常数时间操作，本地整数加法、原子加法、互斥锁竞争、系统调用、磁盘写、跨机 RPC 也有完全不同的成本尺度。

日志统计中的成本判断可以按阶段展开：读取阶段关注 I/O、page cache 和拷贝；解析阶段关注分支、SIMD 机会和输入带宽；聚合阶段关注哈希、字符串生命周期、分配和 cache 局部性；合并阶段关注 key 基数和内存带宽；写出阶段关注 buffer、系统调用和文件系统；提交阶段关注 manifest 序列化、checksum 和 fsync 策略。复杂度提供第一层判断，成本账本提供工程判断。

把成本账本写细一点，就能从源码行直接走向机器行为。下面的表不是性能结论，而是测量前的假设清单：

Listing 1.13 日志统计的阶段成本账本

stage	dominant operations	likely pressure
read	read/copy/map input bytes	I/O latency, page cache
parse	scan for delimiters	branches, byte bandwidth
diagnose	save bad line and RecordId	cold allocation, logging
aggregate	hash event, compare key, update	cache miss, allocation
merge	combine partial maps	memory bandwidth, key count
write	serialize block, checksum	copy bandwidth, syscall
commit	append manifest entry	fsync, lock, retry path

同一个 $O(n)$ 程序可能在这些行上呈现完全不同的瓶颈。若输入很小且在 page cache 中，**read** 几乎看不见；若冷盘读取，I/O 会支配运行时间。若事件名只有十种，聚合可能反复命中少数 key；若事件名有一千万种，哈希表容量、字符串分配和 TLB 压力会快速上升。若错误行很少，诊断是冷路径；若输入格式大面积错误，诊断分配和日志写出会变成主路径。写性能报告时，不能只写“总耗时降低 20%”，还要说明瓶颈从哪一行账本移动到了哪一行。

阶段成本要被程序记录。第一章无需读者马上使用 PMU，但至少要把每个阶段的记录数、字节数和耗时写出来。这样后续章节加上 perf、cache miss 或 TLB miss 时，已经有了可以对齐的阶段边界：

Listing 1.14 阶段测量记录的最小结构

```

1 #include <cstdint>
2 #include <string>
3 #include <vector>
4
5 struct StageMetric {
6     std::string stage;
7     std::uint64_t records = 0;
8     std::uint64_t bytes = 0;
9     std::uint64_t elapsed_ns = 0;
10    std::uint64_t allocations = 0;
11    std::uint64_t queue_wait_ns = 0;
12 };
13
14 struct RunMetrics {
15     std::uint64_t input_bytes = 0;
16     std::uint64_t accepted_records = 0;
17     std::uint64_t rejected_records = 0;
18     std::uint64_t distinct_keys = 0;
19     std::uint32_t worker_count = 0;
20     std::uint32_t split_count = 0;
21     std::vector<StageMetric> stages;
22 };

```

这组字段有明确用途。若 `parse.elapsed_ns/input_bytes` 随输入变大保持稳定，parser 可能接近顺序扫描；若 `aggregate.allocations` 随记录数线性增长，说明每条记录都在制造堆对象；若 `queue_wait_ns` 在 writer 阶段之后显著上升，背压已经成为控制面现象；若 `distinct_keys` 上升后 merge 时间急剧增长，瓶颈可能从解析迁移到哈希表和内存带宽。测量记录不是为了

装饰报告，而是为了让每个优化假设能被推翻。

一个常见误区是把“减少复制”当作自动优化。把 `std::string` 换成 `std::string_view` 确实可能减少事件名复制，但它也把输入 buffer 的生命周期推到了下游；若下游需要在 writer 或 reducer 阶段才使用事件名，buffer 就必须更久地留在内存中。复制会花 CPU 和带宽，保留 view 会花生命周期管理和内存峰值。系统设计要比较这两种成本，而不是只看某一行代码是否少了拷贝。

另一个常见误区是只看平均行长。平均行长 80 字节不代表每行都接近 80 字节。少量超长行会改变 parser 的分支行为、buffer 扩容和错误路径；某些输入还会让一条记录跨越多个 page，增加 TLB 和 cache 行访问。工作负载画像中记录最大行长和分位数，是为了发现这类长尾，而不是为了让报告看起来完整。

硬件章节的主线

后面讲流水线时，会解释一个循环为什么受依赖链、分支预测和乱序窗口影响。讲缓存时，会解释热字段为何应当紧凑，随机访问为何会慢。讲 TLB 时，会解释大工作集和大量小对象为何影响地址转换。讲 NUMA 时，会解释同一个虚拟地址背后的物理页可能靠近某个 socket，远端访问为何拖慢多线程程序。所有硬件概念都回到一个问题：这批日志字节怎样穿过机器。

心智模型

先画数据路径，再谈硬件优化。若不知道字节从哪里来、变成什么结构、被谁反复访问、在哪里写出，就无法判断缓存、TLB、预取、SIMD 或 NUMA 优化作用在哪一段。

以一条记录的地址旅行为例

一条日志记录最初只是文件中的一段字节。通过 `read` 进入用户态 buffer 后，它具有连续地址；parser 扫描空格时，访问模式接近顺序流；事件名被复制进 `std::string` 后，它可能变成堆上的小对象；插入哈希表后，程序先访问桶，再访问节点，再访问字符串内容；写出 partial 时，数据又被序列化连续字节。源码层面这都是“一条记录”，机器层面却经历了完全不同的地址形态。

理解地址旅程能解释许多优化。字符串视图减少复制，但要求输入 buffer 生命周期足够长。字符串池减少小对象分配，但要求池的释放时机和错误诊断需求协调。开放寻址哈希表让桶和值更紧凑，但删除、扩容和高负载因子需要谨慎。排序归约把随机哈希更新变成排序和线性扫描，但需要额外内存和比较成本。没有一种布局天然适合所有工作负载，真正的问题是当前阶段最频繁访问哪些字节。

工作集阶梯

缓存和 TLB 的影响通常会随着工作集扩大而出现阶梯。小到能放入 L1 的数据，循环可能主要受指令和依赖限制；超过 L2 后，访问延迟和带宽开始变化；超过 LLC 后，内存带宽和预取行为更重要；跨越大量页面后，TLB 覆盖和 page fault 也会进入观察范围。实验如果只选择一个规模，很容易误判。一个版本在小输入上快，可能只是因为所有数据都在缓存中；另一个版本在大输入上更稳，可能来自更好的地址序列。

因此后续硬件实验会采用阶梯式输入。固定记录格式，逐步扩大记录数；固定总记录数，改变 key 基数；固定字节数，改变访问步长和页面分布；固定算法，改变容器布局。每次只改变一个主因素，才能把性能曲线和硬件层级联系起来。

硬件成本会反向塑造接口

硬件不是被动背景。若 hot record 连续存放，parser 和聚合器之间的接口就应传递 span 或批次对象，而不是一堆分散指针。若 TLB 压力来自大量小对象，诊断对象就应进入冷路径，而不是每条热记录都携带完整错误文本。若 false sharing 来自相邻 worker 的计数器，任务状态和统计计数就应按 cache line 或 worker 分片放置。若 NUMA 远端访问明显，split 分配和 buffer 初始化就要考虑 first-touch 和线程亲和性。

这就是“从硬件原理到软件设计”的含义。硬件概念不只是考试名词，而会改变类型、函数边界、内存所有权和任务调度策略。后续章节每次讲一个硬件机制，都要回到项目接口：它要求哪类对象更紧凑，哪类状态更分散，哪类访问更顺序，哪类共享更靠后。

第一章不需要立即展开 PMU 事件，但要建立测量纪律。没有硬件计数器时，也能先记录输入字节数、记录数、key 基数、分配次数、最大队列水位、每阶段耗时和输出字节数；有权限时，再补充

cycles、instructions、branch misses、cache misses、LLC misses、TLB misses 和 context switches。指标不是越多越好，每个指标都要对应成本账本中的一行。若怀疑 parser，优先看输入带宽、分支和每字节耗时；若怀疑哈希表，优先看 key 基数、分配和 cache/TLB；若怀疑 I/O，优先看读写字节、in-flight 请求和 fsync 延迟。

实验：成本账本。取三组输入：一组只有十个事件名，一组有一百万个事件名，一组包含 1% 的坏行。对同一个 reference 程序记录每阶段耗时、accepted/rejected 数量、事件种类数和输出 checksum。先解释三组输入的成本差异来自哪一行账本，再讨论优化方向。

1.5 执行实体、队列与等待

串行 reference 清楚以后，最自然的扩展是把输入切成多个 split，让多个 worker 并行处理。表面上这只是把循环拆开，实际会引入执行实体、任务队列、共享状态、等待点和背压。多个执行实体一旦共享资源，就要回答谁拥有状态、谁能修改状态、谁等待谁、等待多久、失败如何传播。

全局共享计数表的反例

一种直观设计是所有 worker 共享一个全局计数 map，每解析到一条记录就加锁更新。

Listing 1.15 共享全局计数表的直观版本

```
1 #include <mutex>
2
3 struct SharedCounts {
4     std::mutex mutex;
5     EventCountMap counts;
6 };
7
8 void update_shared_counts(
9     SharedCounts& shared,
10    std::string_view event_name)
11 {
12     const std::lock_guard<std::mutex> lock(shared.mutex);
13     ++shared.counts[std::string(event_name)];
14 }
```

这段代码语义清楚，在小输入上也能工作。热点输入会放大它的结构问题：每条记录都要进入同一把锁，每次更新高频 key 都会修改同一个计数节点，多个核心在同一小块状态上排队。线程增加以后，程序可能从解析和聚合转向锁等待、上下文切换和 cache line 所有权迁移。问题的根源在于共享发生得太频繁、太早、太集中。

线程本地 partial

更稳妥的第一步是线程本地聚合。每个 worker 处理自己的 split，把结果写入本地 partial；最后再合并 partial。这个设计牺牲一部分内存，换来热路径上更少共享写。它也把计算拆成两个阶段：map 阶段并行处理输入，reduce 阶段合并局部结果。

Listing 1.16 线程本地 partial 的接口形状

```
1 #include <cstdint>
2 #include <string>
3 #include <unordered_map>
4 #include <vector>
5
6 struct Split {
7     std::uint32_t input_id = 0;
```

```

8     std::uint32_t split_id = 0;
9     std::uint64_t begin_offset = 0;
10    std::uint64_t end_offset = 0;
11 };
12
13 struct PartialCount {
14     std::uint32_t split_id = 0;
15     std::unordered_map<std::string, std::uint64_t> counts;
16 };
17
18 PartialCount process_split(const Split& split);
19
20 EventCountMap merge_partials(const std::vector<PartialCount>& partials)
21 {
22     EventCountMap merged;
23
24     for (const PartialCount& partial : partials) {
25         for (const auto& [event_name, count] : partial.counts) {
26             merged[event_name] += count;
27         }
28     }
29
30     return merged;
31 }

```

这段代码把共享写从每条记录一次降低到每个 partial 合并时发生。它仍然使用节点式哈希表，也仍然复制字符串；这些问题属于下一层数据结构优化。当前阶段先把共享边界理清：大部分记录在本地更新，跨线程共享集中在合并阶段。

有界队列让速度差可见

Reader 产生 split，worker 消费 split，writer 写出 partial。只要上下游速度不同，队列就会增长或变空。无界队列会把速度不匹配藏进内存，直到延迟和内存峰值失控。有界队列把容量变成系统合同：队列满时，上游等待或降速；队列关闭时，下游醒来退出；任务失败时，等待者收到错误。

有界队列至少需要表达四种状态。Open 表示可以 push，也可以 pop。Closing 表示停止接受新任务，但已有任务仍可消费。Closed 表示队列为空，消费者退出。Failed 表示队列因错误关闭，等待者收到失败原因。仅用一个 **done** 布尔值很难区分正常结束和错误取消，这两种情况对上游、下游和恢复路径都不同。

队列接口可以先写成一个很小的合同。这里不实现完整并发容器，只列出调用者需要依赖的语义：

Listing 1.17 有界队列的状态和返回值

```

1 #include <cstdint>
2 #include <optional>
3 #include <string>
4
5 enum class QueueState : std::uint32_t {
6     open = 0,
7     closing = 1,
8     closed = 2,
9     failed = 3
10 };
11

```

```

12 enum class PushResult : std::uint32_t {
13     accepted = 0,
14     closed = 1,
15     failed = 2
16 };
17
18 enum class PopResultKind : std::uint32_t {
19     value = 0,
20     closed = 1,
21     failed = 2
22 };
23
24 template <typename T>
25 struct PopResult {
26     PopResultKind kind = PopResultKind::closed;
27     std::optional<T> value;
28     std::string error;
29 };

```

`push` 返回 `accepted`，说明元素已经进入队列；返回 `closed`，说明上游不应再生产；返回 `failed`，说明等待者需要走错误路径。Consumer 的 `pop` 返回 `value`，说明得到一个任务；返回 `closed`，说明队列已耗尽，可以正常退出；返回 `failed`，说明队列被错误打断。这个接口把“暂时没有元素”和“永远不会再有元素”分开，把“正常结束”和“错误取消”分开。

等待谓词也可以直接写成逻辑条件。Producer 等待：

```
queue.size() < capacity || state != QueueState::open
```

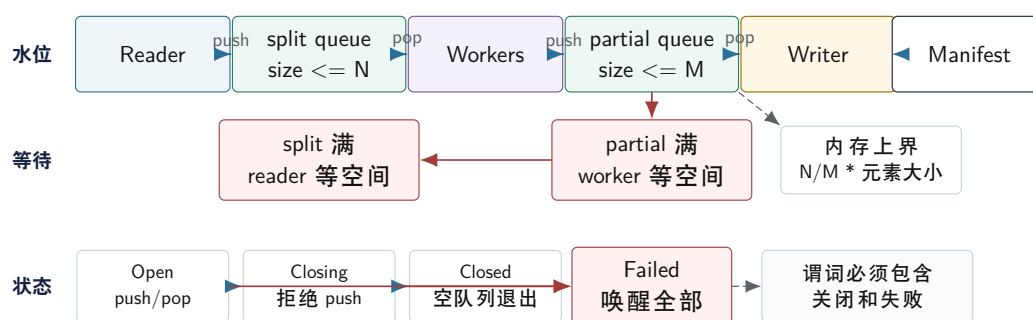
Consumer 等待：

```
!queue.empty() || state == QueueState::closed || state == QueueState::failed
```

这两个谓词解释了为什么关闭和失败都必须唤醒等待者。若 writer 失败时只设置错误标志却不唤醒 producer，producer 可能永远睡在“队列未满”的条件变量上。若 reader 完成时只停止 push 却不改变状态，consumer 可能永远睡在“队列非空”的条件变量上。并发错误往往不是来自互斥锁本身，而是来自状态机和谓词不完整。

队列容量也不是随手填写的常数。容量上限乘以单个元素的平均大小，给出这一段管线的内存上界；容量上限除以 writer 吞吐，给出下游阻塞时上游还能继续运行多久；容量上限乘以失败重做成本，给出取消时在途工作的浪费范围。若 `partialqueue` 容量为 1024，每个 `partial` 平均 2 MiB，队列本身就可能持有 2 GiB 数据。容量参数因此属于系统设计，而不是单纯调参。

有界队列的两件事：水位和状态



图示：水位回答资源上界，状态回答等待者何时醒来；二者分开以后，背压和关闭路径都能写成可测试谓词。

等待谓词就是并发不变量

条件变量的关键不是睡眠，而是等待某个谓词变真。对有界队列来说，producer 等待的谓词通常是队列未满、队列已关闭或队列已失败；consumer 等待的谓词通常是队列非空、队列已关闭或队列已失败。谓词写不清楚，虚假唤醒、关闭竞态和错误传播都会出问题。

后面讲 mutex、condition variable、semaphore 和 futex 时，会围绕这些不变量展开。元素数量不超过容量，open 状态允许 push，closing 状态拒绝新 push，closed 状态没有剩余元素，failed 状态唤醒所有等待者，任何等待都能被关闭或失败打断。API 是实现这些不变量的工具。

任务状态让等待可解释

并发程序最难解释的时间往往是等待时间。等待队列空间、等待任务、等待锁、等待 I/O、等待另一个线程完成、等待重试预算，都会改变吞吐和延迟。若代码没有记录等待点，性能报告只能写“变慢了”，无法判断是 reader 太快、worker 太少、writer 太慢、队列太小、锁太粗，还是任务粒度太碎。

Listing 1.18 任务状态记录的教学形态

```

1 #include <stdint>
2
3 enum class TaskState : std::uint32_t {
4     created = 0,
5     queued = 1,
6     running = 2,
7     finished = 3,
8     committed = 4,
9     failed = 5,
10    cancelled = 6
11 };
12
13 struct TaskTrace {
14     std::uint32_t task_id = 0;
15     std::uint32_t split_id = 0;
16     std::uint32_t attempt_id = 0;
17     TaskState state = TaskState::created;
18     std::uint64_t timestamp_ns = 0;
19     std::uint32_t worker_id = 0;
20 };

```

如果大量任务停在 queued，问题可能在 worker 数量、任务粒度或上游提交方式。若 running 时间很长，问题可能在计算、I/O 或数据倾斜。若 finished 到 committed 间隔很长，writer 或 manifest 可能成为瓶颈。状态机让等待变成证据。

线程池在本书中的位置

线程池的价值是控制执行资源。没有线程池时，每个任务创建线程，成本高且数量失控；有线程池后，系统可以限制 worker 数量，复用线程，集中处理任务生命周期。线程池也会带来接口责任：任务提交后是否一定执行，shutdown 是否等待正在运行任务，任务抛出异常如何传播，worker 内部等待另一个任务是否可能造成饥饿。

日志统计项目会把这些问题具体化。Reader 提交 split，worker 解析并聚合，writer 写出 partial。如果 writer 阻塞，worker 是否继续产生结果；如果某个 split 失败，其他 split 是否取消；如果主线程等待所有 future，而 worker 又等待同一线程池中的子任务，执行图是否形成等待环。后续运行时章节会从这些等待点继续推导任务图、continuation、work stealing 和结构化并发。

核心思想

队列的本质是管理速度差、等待关系和关闭语义。线程池的本质是管理执行资源、任务生命周期和错误传播。二者结合起来，才构成可以分析的并发系统。

线程数和并行度

线程数不等于有效并行度。一个 16 线程程序可能只有 4 个 worker 真正在计算，其他线程在等待锁、I/O、队列空间或 future。一个 8 核机器上开 64 个 CPU 计算线程，可能只会增加调度开销和 cache 污染。有效并行度取决于可独立执行的任务数量、每个任务的计算量、共享状态的频率、内存带宽和操作系统调度。线程池接口必须让这些事实可观察，而不是只提供一个“提交函数”。

日志统计中，可以用几组曲线观察有效并行度。固定输入和 split size，改变 worker 数，记录吞吐、锁等待、队列水位和 CPU 利用；固定 worker 数，改变 split size，观察任务粒度和长尾；固定 split size，改变 key 分布，观察热点对合并阶段的影响。若线程数增加但吞吐不变，报告要说明瓶颈位置，而不是简单宣称多线程无效。

关闭路径是并发质量的试金石

并发程序在正常路径上容易看起来正确，关闭路径更能暴露设计质量。Reader 完成后，worker 如何知道没有新 split；worker 失败后，reader 是否继续读取；writer 失败后，partial queue 中已有数据如何处理；主线程收到取消后，正在运行的 worker 是否能安全停止；等待 push 的 producer 是否能被唤醒。每一个问题都需要明确状态和唤醒规则。

并发测试应覆盖 reader 正常结束、reader 失败、worker 失败、writer 失败、队列满时取消、队列空时关闭、manifest 提交失败和主线程主动取消。这样后续讲 condition variable、semaphore、atomic flag 和 work stealing 时，API 才能连接到实际关闭路径。

观测本身也有成本

记录 trace 能解释等待，但 trace 也会消耗内存、锁和 I/O。小实验可以记录每个任务的详细时间点；百万任务场景中，逐任务文本日志会成为新瓶颈。工程上常用分层观测：错误路径完整记录，正常路径记录结构化摘要；慢任务完整记录，短任务采样；每个 worker 使用本地 buffer，阶段结束再合并；控制面状态精确记录，性能计数可以近似采样。

这条原则后续会反复出现。观测要服务假设，避免记录无法支撑结论的无关数据。若怀疑队列阻塞，就记录队列水位和等待时长；若怀疑锁竞争，就记录锁等待和临界区时间；若怀疑 I/O，就记录 in-flight bytes 和 completion 延迟。每个指标都应有问题来源和使用方式。

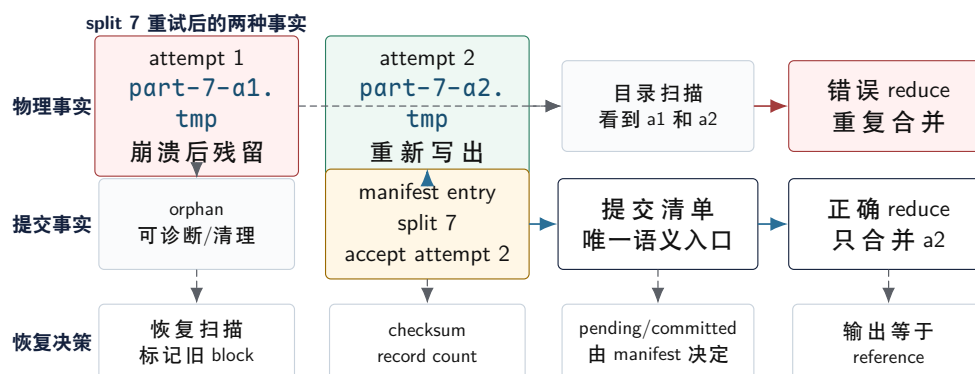
1.6 提交事实与恢复边界

串行程序返回一个内存 map 时，结果有效性比较直观。系统变大后，结果往往先进入临时状态，再通过提交动作对外有效。一个 worker 写出临时 partial 文件，只说明它完成了一次尝试；reduce 阶段是否能读取它，取决于提交事实。一个异步写请求返回，也只说明某个 I/O 阶段完成；结果是否进入全局输出，取决于 manifest。一个 attempt 告诉 driver 成功，也要判断它是不是被接受的那一次成功。

一次写后崩溃

考虑一个具体失败场景。Worker 处理 split 7，attempt id 为 1。它解析输入，生成 partial，写出文件 `part-7-a1.tmp`，随后进程崩溃。Driver 没收到成功响应，于是重试 split 7，attempt id 为 2，新 attempt 写出 `part-7-a2.tmp` 并成功提交。此时目录里可能存在两个文件。如果 reduce 阶段只是扫描目录，把两个文件都合并，split 7 就被统计了两次。

这个错误来自提交语义，而不是来自哈希表或线程调度。目录记录的是文件系统事实，manifest 记录的是作业语义事实。临时文件可以来自失败 attempt、旧 attempt、部分写入、调试输出或清理失败；下游只应读取 manifest 接受的 block。



图示：上排的目录事实可以包含多个物理文件；中排的 manifest 事实只接受一个逻辑贡献；reduce 必须沿中排读取，不能沿上排扫描目录。

Listing 1.19 用 manifest 表达提交事实

```

1 #include <stdint>
2 #include <string>
3 #include <vector>
4
5 enum class BlockState : std::uint32_t {
6     temporary = 0,
7     committed = 1,
8     discarded = 2
9 };
10
11 struct ShuffleBlock {
12     std::uint32_t split_id = 0;
13     std::uint32_t attempt_id = 0;
14     std::uint32_t block_id = 0;
15     std::uint64_t record_count = 0;
16     std::uint64_t checksum = 0;
17     std::string path;
18     BlockState state = BlockState::temporary;
19 };
20
21 struct ManifestEntry {
22     std::uint32_t manifest_id = 0;
23     std::uint32_t split_id = 0;
24     std::uint32_t accepted_attempt_id = 0;
25     std::uint32_t block_id = 0;
26     std::uint64_t checksum = 0;
27 };
28
29 using Manifest = std::vector<ManifestEntry>;

```

Manifest 把“产生了文件”和“结果已经被系统接受”分开。Reduce 只读取 manifest 中出现的 block，因此重复 attempt 留下的临时文件不会污染最终结果。后续 I/O、检查点、分布式提交和恢复协议都会复用这个思想。

Manifest 的最小持久化格式可以是追加式文本或二进制记录。第一版不需要复杂事务系统，但每条记录至少要包含逻辑身份、物理输出、完整性证据和状态：

Listing 1.20 manifest 条目的最小持久化字段

```

job_id=42
split_id=7
accepted_attempt_id=2
block_id=19
path=blocks/part-7-a2.block
record_count=1048576
checksum=0x4a91cf02
state=committed

```

Reducer 不根据目录遍历结果决定输入集合，而是逐条读取 manifest entry，再按 entry 中的路径加载 block。这样目录可以包含临时文件、orphan 文件、调试文件和未清理文件；它们不会进入 reduce，除非 manifest 明确承认。这个方向看起来多了一层间接访问，实际把正确性边界从“不可靠的目录形态”移到了“可校验的提交清单”。

第一版持久化协议可以保持简单，但顺序必须清楚。下面的步骤故意写得像操作手册，因为每一步都对应一个崩溃点：

Listing 1.21 单机 manifest 提交顺序

```

1. write temp block: blocks/part-7-a2.tmp
2. finish write and close temp block
3. compute checksum and record_count
4. rename temp block to blocks/part-7-a2.block
5. append manifest entry with split_id, attempt_id, path, checksum
6. flush manifest according to durability policy
7. publish completion to driver

```

若系统在第 1 步或第 2 步崩溃，恢复时只能看到未完成的临时文件；它不能进入 reduce。若第 4 步完成但第 5 步未完成，目录中会有完整 block，但 manifest 没有接受它；它仍然是 orphan。若第 5 步完成但第 7 步前响应丢失，driver 重试时应发现 manifest 已经接受某个 attempt，并拒绝重复提交。真正困难的不是把文件写出来，而是让每个中间状态都有唯一解释。

提交点连接正确性和性能

提交点越早，下游越快看到结果，但失败恢复的状态更多。提交点越晚，系统更容易一次性校验和提交，但延迟更高，临时状态更多。批量提交减少 I/O 和锁开销，也扩大崩溃后的重做范围；逐条提交恢复简单，却可能吞吐很低。提交点既是正确性边界，也是性能取舍点。

日志统计项目的第一版可以按 split 提交 partial。每个 split 完成后，worker 写出临时 block，计算 checksum，向 manifest 提交一条记录。这个粒度足够小，失败后只重做一个 split；也足够大，避免每条记录都进入提交路径。后续章节讲 batch size、异步 I/O、fsync 策略和分布式 checkpoint 时，会继续讨论粒度如何调整。

幂等性需要身份支撑

幂等性依赖稳定身份和提交事实。每个 attempt 有 `AttemptId`，每个输出 block 有 `BlockId`，manifest 记录某个 split 接受了哪个 attempt。重复消息、迟到成功和重复扫描都可以被判定为已接受、已过期或待提交。缺少这些身份时，系统只能依赖时间顺序和文件名猜测，失败路径会变得脆弱。

Listing 1.22 manifest 只接受一个 attempt

```

1 struct CommitDecision {
2     bool accepted = false;
3     ManifestEntry entry;
4 };

```

```

5
6 CommitDecision commit_block(
7     Manifest& manifest,
8     const ShuffleBlock& block)
9 {
10     for (const ManifestEntry& entry : manifest) {
11         if (entry.split_id == block.split_id) {
12             return {false, entry};
13         }
14     }
15
16     ManifestEntry entry;
17     entry.manifest_id =
18         static_cast<std::uint32_t>(manifest.size());
19     entry.split_id = block.split_id;
20     entry.accepted_attempt_id = block.attempt_id;
21     entry.block_id = block.block_id;
22     entry.checksum = block.checksum;
23
24     manifest.push_back(entry);
25     return {true, entry};
26 }

```

这段代码仍然是单进程内存 manifest，不具备崩溃持久化能力，也没有并发提交保护。它展示的是提交语义：同一个 split 只接受一个 attempt。后续需要补充的是持久化、互斥保护、并发安全和分布式复制。

内存版 `commit_block` 还故意省略了两个工程问题。第一，并发提交需要互斥或事务；否则两个 worker 同时提交同一个 split，可能都在扫描 manifest 时看不到已有 entry，然后都插入成功。第二，崩溃持久化需要写入顺序；否则 manifest 写了一半，重启后 reducer 可能读到残缺 entry。第一章先把语义写清楚，后续 I/O 和分布式章节再把互斥、追加、rename、fsync 和复制补上。

恢复扫描顺序

恢复程序启动后，第一步应读取 manifest。Manifest 告诉系统哪些 split 已经有有效输出。随后扫描临时目录，把不在 manifest 中的旧临时文件标记为 orphan，按策略清理或保留诊断。接着 driver 重新提交未完成 split。这个顺序把语义事实放在文件系统残留之前，恢复逻辑从猜测变成查询。

Checksum 给恢复提供最低限度的完整性证据。Writer 写出 block 后计算 checksum，并把它写入 manifest。Reducer 读取 block 时重新计算，若不一致，就拒绝这个 block 并触发恢复。没有 checksum，短写、截断、旧文件覆盖和路径错误都可能变成静默错误。第一版 checksum 可以先很简单，但接口要保留它的位置。

恢复流程可以写成下面的伪代码：

Listing 1.23 manifest 优先的恢复扫描

```

1 struct RecoveryPlan {
2     std::vector<std::uint32_t> committed_splits;
3     std::vector<std::uint32_t> pending_splits;
4     std::vector<ShuffleBlock> orphan_blocks;
5 };
6
7 RecoveryPlan recover_job(
8     const Manifest& manifest,
9     const std::vector<ShuffleBlock>& directory_blocks,

```

```

10  const std::vector<std::uint32_t>& all_splits)
11  {
12      RecoveryPlan plan;
13      std::unordered_map<std::uint32_t, ManifestEntry> accepted;
14
15      for (const ManifestEntry& entry : manifest) {
16          accepted[entry.split_id] = entry;
17          plan.committed_splits.push_back(entry.split_id);
18      }
19
20      for (const ShuffleBlock& block : directory_blocks) {
21          const auto found = accepted.find(block.split_id);
22          const bool accepted_block =
23              found != accepted.end() &&
24              found->second.block_id == block.block_id &&
25              found->second.accepted_attempt_id == block.attempt_id;
26
27          if (!accepted_block) {
28              plan.orphan_blocks.push_back(block);
29          }
30      }
31
32      for (const std::uint32_t split_id : all_splits) {
33          if (!accepted.contains(split_id)) {
34              plan.pending_splits.push_back(split_id);
35          }
36      }
37
38      return plan;
39  }

```

这段伪代码表达恢复顺序，而不是最终性能实现。它先相信 manifest，再解释目录；先找已提交 split，再找 orphan block；最后找 pending split。若顺序反过来，恢复程序会被目录残留牵着走，旧 attempt、半写文件和调试文件都可能干扰语义判断。

重试预算和错误分类

重试能够恢复临时失败，也可能放大故障。磁盘已满时，writer 无限重试只会消耗 CPU 和日志空间；输入合同错误时，重试同一个 split 仍然失败；网络拥塞时，所有 worker 同时重试会加重拥塞。Attempt 需要重试预算和错误分类。可恢复错误进入重试，不可恢复错误进入 failed，超出预算则作业失败。

单机和分布式遵循同一条原则。单机 writer 短暂失败可以重试，输入格式错误适合进入诊断；分布式 RPC 超时可以重试，业务语义冲突需要上层决策。恢复协议并非简单再跑一遍，而是根据错误类型、attempt 身份和提交事实决定下一步。

深入理解

一次崩溃就能推出四个系统概念：临时状态、提交事实、attempt 身份和恢复扫描顺序。它们都来自同一个问题：下游凭什么相信某份结果已经有效。

状态表比日志更可靠

恢复系统不能只依赖日志文本。日志适合人阅读，状态表适合程序决策。Driver 至少需要 task table、attempt table、block table 和 manifest。Task table 描述逻辑任务是否 pending、running、committed、failed 或 cancelled。Attempt table 描述某次物理执行的 worker、开始时间、deadline、

错误分类和输出 block。Block table 描述临时文件路径、字节数、checksum 和状态。Manifest 描述对外可见的提交事实。

有了这些表，恢复流程可以机械执行：读取 manifest，重建已提交 split；读取 checkpoint 或状态表，找出 running 但未提交的 attempt；扫描临时目录，把未被 manifest 引用的 block 标记为 orphan；根据错误分类和重试预算重新提交 pending split。缺少状态表时，恢复程序只能在目录、日志和时间戳之间猜测，边界条件会迅速失控。

崩溃点矩阵

一个写出流程可以在很多位置崩溃：写临时文件前、写到一半、写完但未计算 checksum、checksum 完成但未提交 manifest、manifest 写入一半、manifest 提交后响应丢失、响应返回后清理临时文件前。每个崩溃点都要有预期恢复行为。写到一半的 block 应被拒绝；写完未提交的 block 可以清理或保留诊断；manifest 已提交但响应丢失时，重试 attempt 应被识别为重复；manifest 写入一半时，需要依赖持久化格式或事务保证读取者不会看到半条记录。

崩溃点矩阵把恢复要求变成可执行测试。后续 I/O 章节会讨论 rename、fsync、目录 fsync、短写和 page cache；分布式章节会讨论响应丢失、旧 epoch、重复请求和 leader 切换。它们的共同基础就是第一章的提交事实。

Listing 1.24 写出路径的崩溃点矩阵

crash point	recovery expectation
before temp block write	split remains pending
during temp block write	partial block rejected by checksum
after block, before checksum	block treated as orphan
after checksum, before manifest	block treated as orphan
during manifest append	incomplete entry ignored or repaired
after manifest, before response	retry is detected as duplicate
after response, before cleanup	orphan cleanup may run later

这个矩阵区分了两件事：哪些状态已经对 reduce 可见，哪些状态只是恢复时需要处理的残留。只有 manifest 中完整、可校验的 entry 才对 reduce 可见。其他状态即使占用磁盘空间，也不能改变最终结果。

清理不是提交路径

系统经常需要清理 orphan block、过期 attempt 和旧 checkpoint。清理可以延迟、重试或批量执行，但提交路径必须先保证下游不会读取错误数据。换句话说，系统正确性不能依赖“残留文件一定及时清理”。目录里存在旧文件并不危险，危险的是 reducer 把旧文件当成有效输入。Manifest 把有效性和清理分离，使清理变成维护任务，而不是正确性的唯一防线。

这个原则在分布式系统中更重要。远端节点可能暂时不可达，旧 attempt 可能迟到，清理消息可能丢失。只要提交事实唯一且下游只读提交事实，清理延迟就不会改变结果。后续章节会把这条原则扩展到 shuffle block、checkpoint、复制日志和模型 artifact。

1.7 从单机扩展到分布式

单机 pipeline 和分布式计算之间没有截然断裂。单机里，多个 worker 线程处理多个 split；分布式里，多个 worker 进程或节点处理多个 split。单机里，线程本地 partial 最后合并；分布式里，map worker 产生 shuffle block，reduce worker 按 key 合并。单机里，manifest 防止重复 attempt；分布式里，manifest、事务日志或提交协议防止重复输出。尺度变大以后，成本和失败模式更复杂，核心边界仍然延续。

分片同时服务并行和恢复

把输入切成 split，一开始是为了并行：多个 worker 同时处理不同范围。它也服务恢复：一个 split 失败，只重做这个 split。分布式系统的 partition 也有类似作用。按输入范围分片可以并行读取，按 key 分区可以并行 reduce，按时间窗口分片可以控制恢复范围。分片决定数据本地性、负载

均衡、失败粒度和提交粒度。

分片过小，调度和 manifest 条目过多，固定成本上升。分片过大，负载不均和失败重做成本上升。若某个事件名占据大部分记录，按输入范围平均切分仍可能在 reduce 阶段形成热点 key。后续分布式章节会从 split 粒度、key 分布和 tail latency 继续推导负载均衡。

shuffle 是语义要求带来的数据重排

单机线程本地 partial 合并时，数据还在同一进程或同一台机器里。分布式 map-reduce 中，map worker 需要把相同 key 的 partial 发送到同一个 reduce 分区，这就是 shuffle。Shuffle 的本质是让“同一个 key 的所有贡献”在某个地方相遇。这个需求来自聚合语义。

Shuffle 昂贵，因为它把数据从本地内存和本地磁盘推向网络、远端内存和远端磁盘。能在 map 端先合并，就能减少网络字节；能压缩，就能用 CPU 换带宽；能按 key 稳定分区，就能让 reduce 少做随机查找；能把热点 key 单独处理，就能避免一个 reduce 被拖住。后续章节会用同一成本账本分析这些策略。

慢任务让平均值失效

单机多线程里，一个 worker 慢会让最终合并等待它。分布式里，一个节点慢会让整个 stage 等它，这类任务常被称为 straggler。它可能来自硬件差异、远端读取、数据倾斜、重试、后台负载或网络拥塞。只看平均任务时间，会看不到最后几个任务拖住全局进度。执行等待图要从平均时间扩展到尾部时间。

日志统计的例子很直观。某个 split 包含异常长的行，parser 慢；某个 split 包含热点 key，reduce 慢；某个 worker 所在节点磁盘慢，写出慢；某个 block 需要跨机拉取，shuffle 慢。分布式优化的重点不是简单增加节点，而是让慢任务可见、可拆、可重试，并保证重试不破坏提交语义。

提交事实继续长成一致性问题

单机 manifest 可以是本地文件或本地数据结构。到了分布式环境，manifest 本身可能需要复制，driver 可能失败，多个节点可能同时尝试提交同一个 split。此时会引出租约、epoch、leader、复制日志、共识或事务。它们的共同源头是同一个问题：谁有权宣布某份结果有效。

如果只有一个可靠 driver，manifest 可以简单很多。若 driver 可能失败，就要把提交事实放到可靠存储或复制服务里。若多个 driver 可能同时存在，就要有 leader 或 epoch 防止旧 driver 继续提交。若多个副本要对提交顺序达成一致，就进入共识协议。第一章不展开协议细节，但要让读者看到分布式一致性从提交事实自然生长出来。

本机也能训练分布式语义

没有真实集群时，也可以在单机模拟分布式关键问题。用多个进程或线程模拟 worker，用消息队列模拟 RPC，用随机延迟模拟网络抖动，用重复消息模拟重试，用临时目录模拟远端 block，用 manifest 模拟提交事实。这个模拟不能证明真实性能，但可以验证语义：重复请求不会重复计数，迟到响应不会覆盖新提交，失败 attempt 不会被 reduce 读取。

这种实验适合普通 Linux 机器和 Termux 环境。真实集群中的网络成本、调度系统和存储系统更复杂，但身份、提交、幂等、重试和恢复原则会延续。读者先把这些语义边界练清楚，再进入真实集群，理解负担会小很多。

核心思想

分布式计算把单机的切分、等待、数据移动和提交事实放大到网络和节点故障环境。单机边界越清楚，分布式概念越容易落到具体问题上。

远程调用改变了默认保证

本地函数调用通常给程序员几个默认保证：参数还在同一地址空间，返回值和调用栈有顺序关系，崩溃通常带着整个进程一起结束，内存对象可以通过指针共享。远程调用打破这些保证。消息一旦发出，就变成独立字节序列；接收方可能迟到、重复、乱序或根本收不到；发送方可能在等待期间超时并启动新 attempt；旧响应可能在新 attempt 提交后到达。

因此分布式接口要把本地调用栈曾经隐含的东西显式写出来。消息要携带 request id、task id、attempt id、epoch、deadline 和 checksum；driver 要记录当前接受哪个 attempt；worker 要知道自己的输出在提交前只是临时结果；reducer 要通过 manifest 而非目录猜测读取输入。分布式概念不

是凭空增加复杂度，而是把跨进程和跨网络后失去的默认保证重新写成状态和协议。

单机模拟的边界

本机模拟很有价值，也有边界。线程队列模拟消息，可以验证乱序和重复语义，但不能代表网络带宽和 RTT。临时目录模拟远端 block，可以验证 manifest 和清理，但不能代表真实分布式存储的一致性。随机延迟模拟 straggler，可以训练 deadline 和重试，但不能覆盖真实集群中的负载、机架网络和节点失效。报告要清楚说明这些限制。

边界写清楚后，本机模拟仍然足够训练关键思想。重复请求不重复提交，迟到响应不覆盖新提交，旧 epoch 写入被拒绝，checksum 错误被发现，driver 重启能恢复状态。这些语义和真实集群一致。性能数字不能外推，协议边界可以提前验证。

1.8 贯穿项目与代码演进

Compute Systems Engine 的学习价值来自渐进式演化。项目不会从一个庞大框架开始，而是先解决一个小问题，观察它暴露出的缺口，再扩展一点代码。每次扩展都要说明它解决了上一版的哪个问题，又留下了哪些限制。这样读者看到的不是凭空出现的对象名，而是一条可复述的工程推导链。

从 reference 到 batch

项目的前四个阶段只处理单机输入路径。它们的目标不是追求并发，而是把输入合同、正确性基准、记录身份和数据生命周期固定下来。若这一段没有站稳，后面的 partial、队列和 manifest 都会建立在含糊语义上。

第一阶段：单行 parser

第一阶段只解析一行日志的 `event_name`。输入合同窄，代码短，测试清楚。验收包括正常行、缺字段、空事件名、事件名过长和最后一行没有换行符。这个阶段训练的是字段规则和错误分类。

第二阶段：串行 reference

第二阶段把单行 parser 放入循环，生成 `ReferenceResult`。它保存计数表、accepted 记录数和 rejected 记录数。验收要求固定输入得到固定输出，所有优化版本都能与它比较。这个阶段训练的是正确性基线。

第三阶段：RecordId 和诊断

第三阶段给每条记录加入 `RecordId`。错误不再只是一个计数，而能回到 input、split、byte offset 和 line index。验收要求坏输入能够定位到具体记录，且切分顺序变化后身份仍然稳定。这个阶段训练的是数据可追踪性。

第四阶段：batch 和冷热分离

第四阶段把若干行组成 batch，热路径保存事件名位置、hash 和局部行号，冷路径保存原始行和错误诊断。此时要明确生命周期：`std::string_view` 指向的输入 buffer 在下游使用期间必须存活。验收要求 hot event 数量、cold error 数量和 reference 对齐。这个阶段训练的是数据路径。

Listing 1.25 批处理解析的教学骨架

```
1 #include <span>
2
3 std::uint64_t stable_hash(std::string_view text);
4
5 struct EventView {
6     RecordId id;
7     std::string_view event_name;
8     std::uint64_t event_hash = 0;
9 };
10
```

```

11 struct BatchParseResult {
12     std::vector<EventView> events;
13     std::vector<ColdRecordInfo> errors;
14 };
15
16 BatchParseResult parse_split_lines(
17     std::uint32_t input_id,
18     std::uint32_t split_id,
19     std::span<const std::string> lines)
20 {
21     BatchParseResult batch;
22
23     for (std::size_t index = 0; index < lines.size(); ++index) {
24         const RecordId id{
25             input_id,
26             split_id,
27             0,
28             static_cast<std::uint32_t>(index)
29         };
30         const ParsedRecord record = parse_record(id, lines[index]);
31
32         if (record.error != ParseError::none) {
33             batch.errors.push_back({
34                 id,
35                 lines[index],
36                 record.error
37             });
38             continue;
39         }
40
41         batch.events.push_back({
42             id,
43             record.event_name,
44             stable_hash(record.event_name)
45         });
46     }
47
48     return batch;
49 }

```

这段代码保留两个显式限制。第一，`byte_offset` 暂时填 0，说明完整 reader 还要提供准确 offset。第二，`static_cast<std::uint32_t>(index)` 要求一个 split 的行数受控；真实工程要么限制 split 大小，要么把行号字段改成更宽类型。限制写清楚以后，下一步扩展才有明确入口。

从 partial 到 manifest

后四个阶段把单机输入路径扩展成小型计算引擎。局部聚合减少共享，有界队列暴露等待，manifest 把输出从物理文件提升为提交事实，本机分布式模拟则用重复消息和迟到响应检验 attempt、epoch 和恢复边界。

第五阶段：局部聚合

第五阶段把 batch 聚合成 `PartialCount`。每个 split 生成自己的 partial，最终合并。共享写从每条记录一次变成每个 partial 的每个 key 一次。验收要求串行 reference、共享全局 map 版本和

线程本地 partial 版本输出一致；同时记录锁等待、合并成本和队列等待。

Listing 1.26 从 batch 生成局部 partial

```

1 PartialCount aggregate_batch(
2     std::uint32_t split_id,
3     const BatchParseResult& batch)
4 {
5     PartialCount partial;
6     partial.split_id = split_id;
7
8     for (const EventView& event : batch.events) {
9         ++partial.counts[std::string(event.event_name)];
10    }
11
12    return partial;
13 }
```

这段代码仍然复制字符串，仍然使用节点式哈希表。它解决的是共享边界，不是最终容器布局。后面数据结构和高性能章节会继续替换局部容器，例如字符串池、开放寻址哈希、排序归约或字典编码。

第六阶段：manifest 提交

第六阶段把 partial 写成 **ShuffleBlock**，再提交到 manifest。Reduce 从 manifest 读取已提交 block。验收包含故障注入：attempt 1 写出临时 block 后崩溃，attempt 2 成功提交，目录中可以存在两个 block，但 manifest 只接受一个，最终 reduce 与 reference 一致。

第七阶段：任务和 trace

第七阶段加入 reader、worker、writer 和 reducer 的任务状态。每个任务记录 created、queued、running、finished、committed、failed 或 cancelled。验收要求报告能解释等待点：队列水位、worker 空闲、writer 阻塞、manifest 提交延迟和重试次数。这个阶段训练的是运行时可观察性。

第八阶段：本机分布式模拟

第八阶段用多个进程或线程模拟 worker，用消息队列模拟 RPC，用随机延迟和重复消息模拟网络。Map 端产生 shuffle block，reduce 端按 key 合并，manifest 处理重复 attempt 和迟到响应。验收要求重复消息、乱序完成和 worker 崩溃都不破坏最终结果。这个阶段为分布式章节提供最小实验环境。

保留基线和责任目录

项目不是每做完一版就抛弃上一版。旧版本提供对照，目录提供责任边界；两者共同保证读者能解释“为什么现在需要这个对象”，而不是只看到最终代码。

每个阶段都保留可退回基线

项目演进时，旧版本不应从学习材料中消失。单行 parser 是字段合同的基线；串行 reference 是结果语义的基线；共享全局 map 是锁竞争的反例；线程本地 partial 是减少共享的基线；扫描目录 reduce 是提交错误的反例；manifest reduce 是恢复语义的基线。保留这些版本，读者才能看到最终结构如何从问题中长出来。

工程仓库可以把旧版本放进实验、测试或文档，而不是让生产路径充满分支。重要的是保留可运行对照。后续每当改 parser、batch、partial、manifest 或 runtime，都能回到对应基线检查语义是否改变。这种可退回路线也是大型系统维护的基础。

项目目录应表达责任边界

目录结构不只是整理文件。它让责任边界在仓库中可见。一个清晰的项目可以把 `reference` 放在 `engine/reference`, `parser` 和输入放在 `engine/input`, 热内核放在 `engine/kernel`, 运行时放在 `engine/runtime`, I/O 和 `manifest` 放在 `engine/io`, 分布式模拟放在 `engine/distributed`, 实验和报告放在 `labs` 与 `tests`。若所有代码都塞进一个大文件, 身份、状态和提交边界就很难落地。

目录边界还帮助源码阅读。读者打开 `runtime`, 就应看到任务状态、队列、`worker` 和取消; 打开 `io`, 就应看到临时文件、`completion`、`checksum` 和 `manifest`; 打开 `distributed`, 就应看到 `message`、`attempt`、`epoch` 和 `driver`。这样的项目不追求一开始就大, 但要让每个模块从第一天起有明确责任。

1.9 一次端到端复盘

前面的内容分别讨论了正确性、数据路径、执行等待、提交事实和分布式扩展。为了让这些边界形成连续图景, 本节把它们放进一次完整运行。假设输入目录中有四个日志文件, 每个文件被切成若干 `split`。输入包含三种特征: 少量坏行、一个占比很高的热点事件 `view`、一次被故意注入的 `writer` 崩溃。目标仍然是按事件名统计次数。这个例子足够小, 可以画出状态变化; 又足够真实, 可以看到系统概念怎样依次出现。

从正确性到数据路径

复盘的前半段只关心数据本身: 先有串行 `reference`, 随后切分输入, 再把记录组织成 `batch`。三轮运行共同回答一个问题: 同一批日志在改变执行形态后, 是否仍然保持同一语义和同一身份。

第一轮运行: 串行 `reference`

第一轮只运行串行 `reference`。它按文件顺序读取所有行, 调用 `parse_event_field`, 把合法事件计入 `EventCountMap`, 把错误行计入诊断。此时系统不追求速度, 只追求可审查。运行结束后, 报告给出输入摘要、计数摘要和错误摘要: 总行数、`accepted` 行数、`rejected` 行数、事件种类数、最高频事件、输出 `checksum`。这个结果成为后续所有版本的黄金对照。

`Reference` 报告中若只出现最终计数, 信息仍然不够。坏行要带 `RecordId`, 否则后续切成 `split` 后无法判断同一条坏行是否仍被同样处理。输出要有稳定排序或 `checksum`, 否则哈希表遍历顺序变化会影响比较。这里的关键不是 `reference` 代码多复杂, 而是它把“正确”变成了可复查对象。没有这一步, 后续任何加速都可能只是更快地算错。

第二轮运行: 切 `split` 后保持语义

第二轮把输入切成 `split`。`Reader` 为每个 `split` 记录 `input id`、`split id`、起止 `offset` 和行边界策略。若 `split` 从任意字节开始, `reader` 需要跳到下一条完整记录; 若 `split` 由上游保证在行边界, 调度器要把这个合同写进输入摘要。随后每条记录得到 `RecordId`。这一步看起来没有改变结果, 却改变了系统能否解释结果。

验收方式很直接。把 `split` 顺序打乱, 输出仍应与 `reference` 一致; 把同一输入按不同 `split size` 切分, 错误诊断仍应指向同一条逻辑记录; 最后一行没有换行符时, `reader` 的合同要说明它是否构成完整记录。这个阶段训练的是数据身份。身份稳定以后, 后面的线程调度、重试、迟到响应和恢复才有判定基础。

第三轮运行: `batch` 和冷热路径

第三轮引入 `batch`。`Reader` 不再把每一行单独交给 `worker`, 而是把一个 `split` 内的若干行组成 `ParsedBatch`。`Batch` 内部把事件名位置、`hash` 和局部行号放进热路径, 把原始行和错误诊断放进冷路径。这个变化的动机来自数据移动: 聚合阶段反复访问事件 `id`、`hash` 和计数, 错误文本只在诊断时需要。把两者混在同一个胖对象里, 会让热循环搬入许多无关字节。

这个阶段要验收三件事。第一, `batch` 版本的计数和错误诊断与 `reference` 一致。第二, `std::string_view` 的生命周期清楚: 它指向的输入 `buffer` 在聚合完成前不能释放。第三, 报告写出 `batch size`、每个 `batch` 的记录数、错误数和热字段字节数。`Batch size` 过小, 队列和函数调用成本高; `batch size` 过大, 内存峰值和失败重做范围上升。这个取舍会在后续缓存、SIMD、异步 I/O 和分布式 `shuffle` 中反复出现。

从共享等待到提交事实

复盘的后半段开始放大执行和失败。共享 map 暴露锁等待，慢 writer 暴露背压，写后崩溃暴露 manifest，迟到响应暴露 attempt 和 epoch。这里的每一步都不是新增概念名，而是一个朴素版本被具体反例击穿后的修复。

第四轮运行：共享 map 暴露等待

第四轮尝试多线程，并故意先使用共享全局 map。每个 worker 从 split queue 取任务，解析并更新同一个 `SharedCounts`。在均匀 key 输入上，这个版本可能有一定加速；在热点 view 输入上，吞吐很快停止增长，甚至下降。Trace 显示大量时间花在 `SharedCounts::mutex`，队列中任务等待变长，CPU 利用看似很高，实际很多核心在排队。

这个现象推动线程本地 partial。每个 worker 先在本地 map 中聚合，处理完 split 后生成 `PartialCount`，最后由 merge 阶段合并。共享写从每条记录一次变成每个 partial 的每个 key 一次。验收必须同时看正确性和瓶颈迁移：输出要等于 reference；锁等待应下降；合并阶段耗时可能上升；若合并成为主成本，下一步要分析 key 基数、partial 数量、合并顺序和容器布局。这样读者看到的不是“partial 永远更快”，而是共享边界改变了成本位置。

第五轮运行：writer 变慢引出背压

第五轮加入 writer，把每个 partial 写成临时 block。若 writer 被磁盘、文件系统或人为延迟拖慢，partial queue 会持续增长。无界队列能让 worker 暂时继续运行，但内存峰值会不断上升，延迟被藏在队列里。此时系统需要有界队列和背压：partial queue 满时，worker 在提交 partial 时等待；worker 等待后，split queue 消费变慢；reader 也会被上游队列容量限制。

背压的价值是把速度不匹配变成可观察状态，而不是让内存替系统承担无限缓冲。验收指标包括 partial queue 最大水位、worker 等待时长、writer completion 延迟、内存峰值和最终输出一致性。若队列容量调得太小，worker 频繁等待；容量太大，故障时在途 partial 太多。容量选择要能用资源预算解释：每个 partial 平均大小、writer 吞吐、允许内存峰值、worker 数量和失败恢复成本。

第六轮运行：写后崩溃引出 manifest

第六轮注入一次 writer 崩溃。Worker 已经完成 split 3 的 partial，writer 写出 `tmp-s3-a1.block`，随后在 manifest commit 前崩溃。Driver 重启后看不到 split 3 的提交事实，于是提交 attempt 2，写出 `tmp-s3-a2.block` 并成功写入 manifest。目录里有两个 block，但 manifest 只有 attempt 2 的条目。

安全 reducer 从 manifest 读取，因此只合并 attempt 2。危险 reducer 扫描目录，会把 attempt 1 和 attempt 2 都合并，split 3 被重复计数。这个例子把提交事实讲得非常具体：文件存在只是文件系统事实，manifest 接受才是作业语义事实。验收方式也很具体：故障注入后目录允许存在 orphan block；manifest 只接受一个 attempt；reduce 输出等于 reference；恢复报告列出 orphan block 的路径、checksum 和清理策略。

第七轮运行：迟到响应引出 attempt 和 epoch

分布式模拟中，再加入响应丢失和迟到响应。Driver 把 split 5 发送给 worker，attempt id 为 1。Worker 执行成功并写出 block，但成功响应丢失。Driver 到达 deadline 后提交 attempt id 为 2。Attempt 2 成功提交 manifest。过了一段时间，attempt 1 的成功响应迟到。若 driver 只看“成功”两个字，就可能把旧结果再次提交或覆盖当前状态。

正确响应必须携带 task id、attempt id、epoch、block id 和 checksum。Driver 根据当前 manifest 和 epoch 判断它属于已过期 attempt，于是记录 late attempt，保留诊断或清理临时 block，但不写 manifest。这里的核心变化是：远程调用失去了本地调用栈的顺序保证，协议必须用身份字段恢复判定能力。这个机制和单机 manifest 是同一条线，只是故障更多、消息顺序更弱。

把复盘写成报告

一次端到端复盘最后要落成报告，而不是停在口头解释。报告可以按阶段写出一张表：reference checksum、split 数量、batch size、worker 数、partial 数量、queue 最大水位、writer completion 延迟、manifest 条目数、orphan block 数、重试次数、最终 checksum。每个数字都对应一个问题：结果是否正确，切分是否稳定，批次是否合适，是否存在等待，提交是否唯一，恢复是否可靠。

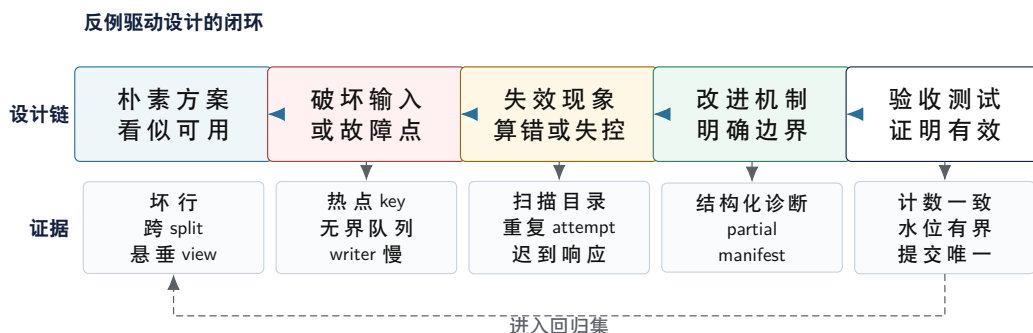
报告还要写限制。若输入在 page cache 中，I/O 结论不能外推到冷盘；若 manifest 只是内存 vector，不能证明崩溃持久化；若网络用本地队列模拟，不能代表真实 RTT；若没有 PMU 权限，cache 和 TLB 只能用间接指标。清楚写出限制可以防止结论越界，避免把一次实验结果误读成普遍规律。

核心思想

端到端复盘把第一章所有概念串成同一条运行轨迹：reference 定义正确性，RecordId 保持身份，batch 管理数据路径，partial 减少共享，背压控制等待，manifest 定义提交事实，attempt 和 epoch 处理迟到事件，报告把结论固定为证据。

1.10 反例矩阵与验收标准

如果只看正确路径，最终设计会显得像是自然长出来的。真正暴露结构的是反例：哪一个小输入、哪一次等待、哪一个故障把朴素实现击穿；朴素实现为什么错；改进后怎样证明它真的解决了问题。本节给出第一章的反例矩阵。每个重要机制都应能回到对应反例和验收方式。



图示：每个机制都应能说清楚它修复哪个反例，验收测试则把这个反例固定成以后不能退化的合同。

输入与生命周期反例

第一组反例都发生在数据进入系统的早期。坏行要求结构化错误，跨 split 要求稳定身份，悬垂 view 要求明确生命周期。它们看起来不是性能问题，却决定后续任何性能优化能否被信任。

坏行反例：布尔返回值不够

输入中出现一行 `10:04u004`，缺少事件名和数值。最早的 parser 如果只返回 `bool`，调用者只能知道“失败了”，不知道失败在哪个输入、哪个 split、哪一行，也不知道错误类型。小输入可以人工查看，大输入中这种错误会变成无法定位的统计差异。更严重的是，并行版本里错误可能只发生在 worker 的日志中，driver 和比较程序看不到结构化事实。

这个反例推动 `ParseError`、`RecordId` 和 `ParseDiagnostic`。验收标准很明确：同一条坏行在串行 reference、batch parser、多线程 worker 和重试 attempt 中都应生成同一个逻辑身份；错误分类应稳定；比较程序能报告具体 record id，而不是只报告总数不一致。通过这个验收，读者才能相信后续优化没有吞掉错误。

跨 split 反例：切分不是简单取字节范围

假设一个 split 从文件中间的字节开始，而这个字节正好落在一行日志内部。如果 reader 直接从 begin offset 开始解析，它会把半行当成完整记录，产生伪错误或伪事件。另一个 split 又可能处理同一行的后半部分，导致记录丢失或重复。这个错误在小文件整段读取时不存在，只有切分后才出现。

这个反例推动 split 合同。系统必须选择一种策略：由上游保证 split 落在行边界，或者 reader 自行修正到下一条完整记录，并保留原始 offset。验收标准是改变 split size 和 split 边界后，最终计数与 reference 一致；错误诊断中的 record id 能回到同一条原始记录；报告写明行边界策略。没有这个合同，分布式 partition 和 checkpoint 都没有可靠输入身份。

悬垂 view 反例：零拷贝也有生命周期成本

为了减少字符串复制，parser 返回 `std::string_view`。如果 batch 持有的 view 指向临时 `std::string`，而临时字符串在聚合前被释放，后续哈希和比较就会读取悬垂内存。这个错误可能表现为偶发计数错误、崩溃，或者只在优化编译和多线程调度下出现。它说明“减少复制”不是单纯性能技巧，而是所有权和生命周期设计。

这个反例推动 `ParsedBatch` 拥有输入字节 buffer，或者要求上游 buffer 生命周期覆盖下游所有使用。验收标准包括：在 `AddressSanitizer` 或等效检查下运行小输入；故意让 reader 复用 buffer，确认测试能抓到生命周期错误；在正确实现中，batch 释放前所有 event view 都有效。后续讲 `std::span`、arena、字符串池和 tensor buffer 时，都会复用这条思路。

并发、提交与观测反例

第二组反例来自系统运行之后：热点 key 暴露共享写，无界队列暴露下游慢，目录扫描暴露提交语义，迟到响应暴露 epoch，指标缺失则暴露报告无法支撑结论。这组反例把第一章从“小程序正确”推进到“系统可解释”。

热点 key 反例：全局锁让并行退化

一千万行输入中九百万行都是 `view`。全局共享 map 版本每条记录都加锁更新 `view`。线程数从 1 增加到 2 可能有提升，从 4 增加到 8 后吞吐停滞，甚至下降。Trace 显示 worker 大量时间在等待 mutex，perf 或替代指标显示上下文切换和 cache line 迁移增加。此时问题不是“线程不够”，而是共享边界设计错误。

这个反例推动线程本地 partial、分片聚合和延迟合并。验收标准包括三项：输出与 reference 一致；锁等待或共享更新次数显著下降；报告说明新瓶颈是否迁移到 merge 阶段。若 partial 版本在高基数 key 输入下内存占用过高，也要写出限制。这样读者能理解局部聚合的适用条件，而不是把它当成固定答案。

无界队列反例：吞吐假象掩盖内存风险

Reader 很快，worker 中等，writer 被人为延迟。无界 partial queue 会让 worker 继续产生结果，短时间吞吐看起来不错，但队列长度和内存占用持续上升。等 writer 恢复时，系统已经积压大量旧 partial；若此时发生取消或失败，恢复要处理大量在途状态。无界队列把速度不匹配延后成更大的状态问题。

这个反例推动有界队列和背压。验收标准不是简单“程序还能跑”，而是队列水位有上界、内存峰值有上界、writer 慢时 worker 等待可见、取消时等待者能被唤醒。报告要写出队列容量选择依据：每个 partial 平均大小、worker 数量、writer 吞吐、允许延迟和内存预算。后续 I/O 和分布式章节会把同一反例扩展到 in-flight bytes 和网络背压。

扫描目录反例：文件存在不等于结果有效

Worker 写出 `tmp-s3-a1.block` 后崩溃，重试 attempt 写出 `tmp-s3-a2.block` 并提交。若 reducer 扫描目录，两个文件都会被合并，split 3 重复计数。这个反例特别重要，因为它在正常路径上完全看不出来：没有崩溃时，扫描目录似乎也能工作；一旦出现重试，语义就错了。

这个反例推动 manifest。验收标准包括：故障注入后目录中允许存在多个 block；manifest 对同一个 split 只接受一个 attempt；reducer 只读取 manifest；危险的扫描目录版本在测试中失败；恢复报告列出 orphan block。这样读者能分清“临时状态”“文件系统事实”和“提交事实”三个层次。

迟到响应反例：成功消息也可能过期

Driver 给 split 5 提交 attempt 1，响应丢失后触发 attempt 2。Attempt 2 已经提交 manifest，attempt 1 的成功响应才到达。若 driver 只根据响应中的 `Ok` 更新状态，旧响应会污染当前作业。这个反例说明成功并不自动有效，响应必须携带身份并经过当前状态表验证。

这个反例推动 attempt id、epoch、deadline 和 commit table。验收标准是迟到响应被分类为 late attempt 或 stale epoch，不写 manifest，不覆盖新状态；trace 中能看到 attempt 1 和 attempt 2 的时间线；最终 reduce 结果与 reference 一致。后续共识、租约和复制日志都可以从这个反例继续推导。

指标缺失反例：优化无法解释

把共享 map 换成 partial 后，程序快了一些。若报告只有一个总耗时，读者无法知道为什么变快，也无法知道下一步该优化哪里。可能是锁等待下降，可能是输入命中 cache，可能是编译参数变化，可能是数据分布不同。没有输入摘要、版本说明和阶段指标，性能数字没有解释力。

这个反例推动系统报告。验收标准是每次实验至少保存输入摘要、构建参数、运行次数、reference checksum、阶段耗时、队列水位、等待指标和未验证边界。若没有 PMU 权限，就写替代证据；若只测热缓存，就写明不能代表冷盘。报告不是把数字列满，而是让数字回答明确问题。

反例矩阵的使用方法

反例矩阵要进入日常开发。修改 parser 时，跑坏行和跨 split；修改 batch 时，跑悬垂 view；修改聚合时，跑热点 key；修改队列时，跑 writer 慢和取消；修改 manifest 时，跑写后崩溃；修改 driver 时，跑迟到响应；修改测量脚本时，检查报告字段。这样项目不会依赖记忆和口头约定，而是用测试把系统边界固定下来。

反例矩阵也能帮助阅读成熟源码。看到 Linux 的 RCU，就问它解决了哪类并发读写和回收反例；看到数据库 WAL，就问它解决了哪类崩溃点反例；看到分布式系统的 term 和 index，就问它解决了哪类旧 leader 和日志回退反例；看到推理引擎的 tensor arena，就问它解决了哪类生命周期和局部性反例。会提出反例，才说明真正理解了机制。

深入理解

反例不是为了挑刺，而是让设计有来处。每个机制都应能回答四个问题：哪个朴素方案失败了，失败输入或故障是什么，改进机制改变了哪个边界，验收怎样证明它有效。

1.11 案例与实验材料

前面的章节已经给出对象、接口和失败模式，本节把它们落到可复用的实验材料。第一章不追求制造大而全的框架，而是准备三类小案例：坏日志用来验证输入合同和诊断，热点事件用来验证共享边界和背压，写后崩溃用来验证 manifest 和恢复。三类案例都应能手工解释，又能进入脚本自动比较；这样后续章节讨论流水线、缓存、锁、I/O 或分布式时，不需要重新发明输入和验收方式。

第一章作为全书概览，需要把学习材料组织成可运行、可复盘、可扩展的闭环。只有文字说明，读者容易觉得概念正确却空泛；只有最终代码，读者又看不到对象为何存在。合适的方式是把案例、代码、实验和源码阅读串在一起：先用小案例暴露问题，再用代码承接边界，然后用实验验证结果，最后把同一类边界映射到成熟系统源码中。

坏日志案例

输入中有一行缺少 `event_name`：

```
10:00 u001 login 1
10:01 u002 view 1
10:02 u001 click 1
10:03 u003 logout 1
10:04 u004
10:05 u001 login 1
```

最小 reference 可以给出 `login=2、view=1、click=1、logout=1`，accepted 为五，rejected 为一。系统化的版本还应给出一条诊断，指向 input id、split id 和第五行的 line index。若输入被切成两个 split，诊断仍应指向同一条逻辑记录。这个案例说明 `RecordId` 解决的是错误定位和并发后的可追踪性。

热点事件案例

假设一千万行日志中，九百万行都是 `view`。共享全局 map 的版本每条记录都进入同一把锁，热点 key 所在状态不断被多个核心争用。线程本地 partial 版本让每个 worker 先在本地图聚，最后把 `view` 的局部总数合并到全局。这个案例的验收包括两部分：输出与 reference 一致，等待时间从逐条共享更新转移到合并阶段。若合并阶段成为新瓶颈，说明瓶颈已经迁移，下一步应分析 key 基

数、合并顺序、容器布局和内存带宽。

写后崩溃案例

Split 3 的 attempt 1 写出 `tmp-s3-a1.block` 后崩溃。Driver 重试 split 3, attempt 2 写出 `tmp-s3-a2.block` 并提交。目录里有两个文件，manifest 只有一个条目。安全的 reducer 从 manifest 读取，危险的 reducer 扫描目录。实验应证明扫描目录版本会重复计数，manifest 版本与 reference 一致。这个案例说明提交事实的必要性。

Listing 1.27 reduce 从 manifest 读取已提交 block

```

1 std::vector<ShuffleBlock> load_committed_blocks(
2     const Manifest& manifest);
3
4 void merge_block_into_result(
5     EventCountMap& result,
6     const ShuffleBlock& block);
7
8 EventCountMap reduce_from_manifest(const Manifest& manifest)
9 {
10     EventCountMap result;
11     const std::vector<ShuffleBlock> blocks =
12         load_committed_blocks(manifest);
13
14     for (const ShuffleBlock& block : blocks) {
15         merge_block_into_result(result, block);
16     }
17
18     return result;
19 }
```

实验目录

第一章的实验可以保持朴素，但目录要表达学习路线：

```

experiments/ch01/
  inputs/normal.log
  inputs/dirty.log
  inputs/hot-key.log
  inputs/retry-split-3.log
  expected/reference-counts.txt
  expected/reference-errors.txt
  run-reference
  run-partial
  run-manifest-failure
```

正常输入对应 reference, dirty 输入对应 RecordId 和诊断, hot-key 输入对应共享 map 与 partial 对比, retry 输入对应 manifest。以后第二章讲流水线，可以继续使用普通输入观察循环执行；第三章讲缓存，可以继续使用 parsed batch 观察布局；锁章节可以继续使用共享 map 反例；I/O 章节可以继续使用 manifest 和临时 block。实验材料复用，学习主线就不会断。

报告格式

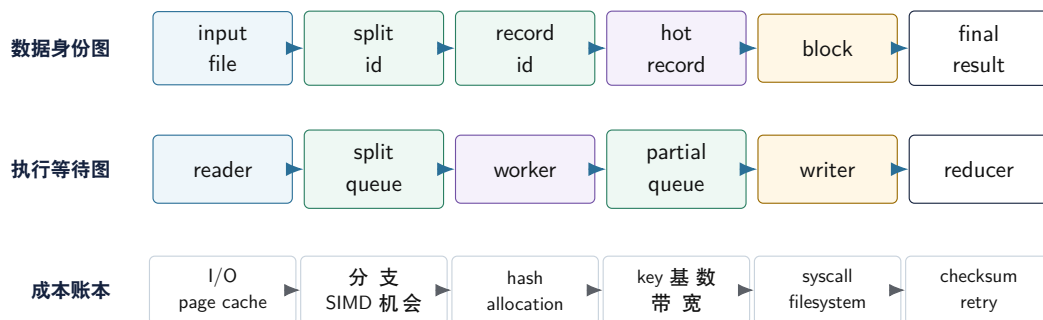
系统报告要包含输入合同、输入摘要、reference 输出摘要、数据身份图、执行等待图、成本账本、故障注入结果和未验证边界。输入摘要至少包括总行数、字节数、平均行长、最大行长、事件

种类数、最高频事件占比、错误行数和 split 数量。未验证边界可以写得很具体：输入在 page cache 中，不能代表冷盘读取；机器只有单 socket，不能验证 NUMA；没有 PMU 权限，不能给出 cache miss 事件；manifest 只是内存 vector，不能证明崩溃后的持久化。

这种报告格式训练的是证据意识。看到多线程不加速，先提出假设，再找指标；看到一次运行更快，先确认 reference 是否一致，再确认输入、编译参数、机器状态和运行次数；看到一个优化有效，说明它移动了哪个瓶颈，同时承认哪些结论还不能外推。

三张图

面对一个计算系统，先画三张图。第一张是数据身份图，追踪对象从输入到结果的身份；第二张是执行等待图，追踪任务从 reader 到 reducer 的等待关系；第三张是成本账本，追踪每个阶段消耗的主要资源。



图示：三张图不是装饰，而是分析入口：先确认对象身份，再确认等待关系，最后确认成本来源。

数据身份图保证系统知道“这是谁”；执行等待图保证系统知道“谁在等谁”；成本账本保证系统知道“时间花在哪里”。后续章节每次扩展项目，都可以回到这三张图，标出新增身份、新增等待和新增成本。

1.12 源码阅读与后续路线

本章最后把第一章的方法迁移到后续阅读。一个成熟系统的源码规模可能远大于本章项目，但读法仍然相同：先找数据身份，再找执行等待，再找提交事实，最后找证据和实验边界。下面的路线不是课程广告，而是后续章节的依赖图；读者知道每一部分放大第一章哪条边界，后续学习才不会散成术语清单。

源码阅读路线

第一章还要为后续源码阅读建立方法。阅读 Linux、数据库或计算框架源码时，先找与本章三张图对应的对象，而不是按文件名漫游。读 Linux workqueue 时，找工作项、worker pool、队列状态、唤醒和取消。读页缓存时，找页身份、状态、回写、失效和错误。读数据库执行引擎时，找 batch、operator、buffer、scheduler 和 checkpoint。读分布式计算框架时，找 split、task attempt、shuffle block、driver、manifest 或提交日志。

这种读法把复杂源码还原成问题链：输入如何获得身份，任务如何进入执行，等待如何被记录，结果如何提交，失败如何恢复，性能结论如何被证据支持。成熟系统的对象很多，但它们通常都能落回这些边界。

后续章节的连接

第二章把视角缩进 CPU 核心内部，回答同样处理一批记录时，为什么某些循环能被核心高吞吐执行，某些循环却被依赖、分支或取数等待卡住。第三章放大缓存、TLB 和虚拟内存，回答热字段、冷字段、页缓存和地址序列怎样影响吞吐。第四章进入 NUMA 和一致性，回答共享状态在多个核心之间迁移的成本。

第五到第八章把测量、数据布局、SIMD 和计算 kernel 连接到同一个项目。第九到第十二章讨论线程、调度、锁、原子和无锁结构，重点是共享状态和内存可见性。第十三到第十五章讨论并行归约、任务运行时和异步 I/O，重点是任务图、背压、取消和提交。第十六到第十八章把 split、shuffle、manifest 和恢复扩展到分布式环境。每一部分都围绕日志统计作业扩展，但每一部分的原理都能迁移到数据库、搜索、编译、图计算和推理引擎。

阅读成熟系统时的提问清单

后续阅读成熟源码时，可以带着一组固定问题。数据问题：对象的身份是什么，生命周期由谁拥有，热字段和冷字段是否分离，序列化后还能否回到原对象。执行问题：任务如何进入 ready 状态，等待在哪里发生，取消如何传播，线程是否可能阻塞等待同一个资源池。同步问题：锁保护什么不变量，原子发布什么状态，读者如何知道某个对象已经初始化完成。恢复问题：结果何时对外可见，临时状态如何清理，重复消息如何去重，checkpoint 如何验证。证据问题：如何复现输入，如何比较 reference，如何定位瓶颈，哪些限制还没有验证。

这份清单适用于很多系统。Linux 页缓存可以问页的身份、dirty 状态和回写；Linux workqueue 可以问工作项、worker pool 和救援线程；数据库执行引擎可以问 batch、operator、buffer 和 checkpoint；分布式计算框架可以问 task attempt、shuffle block、driver epoch 和 commit log；未来推理引擎可以问 tensor buffer、kernel task、model artifact 和 KV cache。第一章先建立这些问题的入口，后续章节再把每一条路径展开到代码、实验和源码阅读。

硬件部分怎样展开

第一部分从 CPU 核心内部开始。多线程性能最终仍要落到每个核心如何执行指令、如何等待数据、如何与其他核心共享缓存状态。第二章会从一个整数累加循环和一个分支循环出发，解释流水线、乱序执行、分支预测、前端供给、后端端口和依赖链。它关注的是单个核心内部如何把指令变成吞吐，也解释为什么相同源码在不同输入分布下会出现不同速度。

第三章把视角放到地址序列。日志统计里，顺序扫描输入、解析字段、访问哈希桶、比较字符串、写出 block，都会产生不同地址模式。缓存、TLB、page cache 和虚拟内存不是孤立层级图，而是这些地址模式的执行环境。第四章继续把单核放入多核系统：多个核心共享缓存层级，一致性协议维护 cache line 状态，NUMA 让物理内存具有位置差异。此时，线程本地 partial 和 false sharing 不再只是编程技巧，而是硬件成本推动的软件设计。

单机高性能部分怎样展开

第二部分回答“已经知道硬件成本后，怎样做有证据的优化”。测量章节先建立实验纪律：输入摘要、编译参数、运行次数、误差、PMU 或替代指标、未验证边界。没有测量纪律，后续任何优化都只能停留在感觉。数据布局章节把日志记录从胖对象拆成热字段、冷诊断和稳定 id，解释 AoS、SoA、字符串池、arena、排序归约和开放寻址哈希怎样改变地址序列。

SIMD 和 ILP 章节继续放大 parser 和 kernel。解析空格、过滤记录、计算简单 hash、批量比较字段，这些都可以成为向量化材料。内核模式章节则把前面的原则整理成常见计算形态：map、filter、reduce、histogram、partition、scan、sort 和 join 的简化版本。它们不是算法书里的抽象题，而是贯穿项目里真实会出现的处理阶段。

并发与同步部分怎样展开

第三部分从线程和调度开始。线程不是一个语法结构，而是操作系统调度的执行实体。线程亲和性、上下文切换、CPU 拓扑、优先级和阻塞 I/O 都会影响 worker 是否真正并行。锁和条件变量章节会围绕有界队列、共享 map 和 manifest commit 展开，重点不是列举 API，而是写清临界区保护的不变量、等待谓词和关闭路径。

原子和内存模型章节会处理更细的发布关系：一个 worker 写完 task 结构后，另一个 worker 何时能安全读取；一个 flag 置位是否意味着数据已经初始化；release/acquire 怎样把对象状态从生产者发布给消费者。无锁数据结构章节再讨论无锁队列、栈、ring buffer、ABA、hazard pointer 和 epoch 回收。这些机制来自减少锁等待和控制 tail latency 的需求。

运行时和 I/O 部分怎样展开

第四部分把并发基础组织成更高级的执行系统。并行归约和 partition 章节从线程本地 partial 继续讲：多个 partial 如何合并，key 怎样分区，热点怎样处理，scan 和 partition 怎样成为并行算法的基本组件。任务运行时章节把固定线程池扩展成任务图、work stealing、continuation、取消和结构化并发，回答 future 等待、长尾任务和依赖调度问题。

异步 I/O 和背压章节把 writer、manifest 和 reducer 放进同一条管线。异步写出可以隐藏等待，也会制造 in-flight 数据、buffer 生命周期、completion 顺序和错误传播问题。背压不是简单让程序变慢，而是让系统在下游阻塞时保持内存有界、状态可解释。到这里，单机系统已经具备一个小型

计算引擎的形状：reader、parser、worker、writer、manifest、reducer 和 trace。

分布式部分怎样展开

第五部分把同一套边界推到网络环境。分布式模型章节从消息身份、超时、重试和迟到响应开始，解释远程调用失去本地调用栈保证后，需要哪些字段和状态表。复制和一致性章节讨论 leader、epoch、日志、共识、租约和事务边界，源头仍是“谁有权宣布结果有效”。最终工程章节把输入 split、map worker、shuffle block、reduce worker、manifest、checkpoint 和故障注入组合成端到端项目。

分布式部分的目标不是让读者背协议名，而是让读者能分析一个具体事故：为什么响应丢失会导致重复提交，为什么旧 driver 不能继续写 manifest，为什么 shuffle block 要有 checksum，为什么慢 reduce 会把背压传回 map，为什么 checkpoint 要区分 committed、running 和 pending。所有这些问题都能回到第一章的日志统计主线。

与后续推理引擎的关系

第三册会进入 AI 模型、张量、算子、量化和 CPU 推理引擎。本册先把现代计算系统讲清楚，是为了让第三册的算子开发有工程地基。权重加载需要 I/O 和内存布局；tensor buffer 需要生命周期和对齐；矩阵乘、归一化和采样需要测量、SIMD 和 cache 思维；token 调度需要任务运行时；KV cache 需要数据结构和内存管理；模型文件和缓存需要 checksum、manifest 和恢复。第三册会在本册的计算系统能力上继续加深，把同一套身份、数据路径、任务调度和提交事实迁移到模型推理场景。

因此，第一章的日志统计作业也可以被看作未来推理引擎的前置训练。日志记录会换成 tensor block，event key 会换成算子或 token，partial 会换成 kernel 输出，manifest 会换成模型 artifact 或缓存提交，trace 会换成 layer 和 token 的执行报告。对象不同，边界相通。读者先学会分析日志统计，就更容易理解后续推理引擎为什么需要同样严谨的身份、数据布局、任务调度和提交事实。

学习顺序和复盘方法

读这一册时，建议每章都做三件事。第一，把章节开头的小问题手工推导一遍，例如单行 parser、共享计数器、无界队列、重复 attempt。第二，把章节里的代码映射到贯穿项目，确认它解决的是哪条边界，而不是只记住函数名。第三，用一页报告写出输入、reference、现象、指标、结论和限制。这样读完一章，就能把概念、代码和证据连起来。

复盘时也按同一方法进行。分析一个概念，先确定它解决的问题；分析一个对象，先确定它保存的身份或状态；分析一个优化，先确定它移动的瓶颈；分析一个协议，先确定它保护的提交事实；分析一个实验，先确定它能否被别人复现。这个复盘方法比记住定义更重要，因为真实系统总会给出新形态，只有问题链和证据链能迁移。

1.13 本章实验

第一章的实验不追求性能极限，而是把系统边界跑通。建议准备一个三十到一百行的小输入，再准备几个破坏朴素假设的输入：最后一行没有换行符，某行缺少事件字段，某个事件名非常长，同一个 split 被重复执行，worker 在写出临时 block 后模拟崩溃。实验规模故意小，是为了让读者能手工检查每条记录的命运。

实验一：写出串行 reference

实现 `run_reference_checked`，输入是内存中的行数组，输出是事件计数 map、accepted 记录数、rejected 记录数和错误诊断。验收方式是固定输入得到固定输出，并能解释每个错误行为为何被归类。这个实验训练语义边界。Reference 稳定以后，后续性能实验才有锚点。

实验二：引入 split 和 record id

把输入切成两个或多个 split，每个 split 记录起止 offset 或行范围。每条记录获得稳定 `RecordId`，无论 split 处理顺序如何，最终 reference 输出一致。若某条记录跨 split，要么在合同中禁止这种切分，要么由 reader 修正到行边界。实验报告说明所选合同。

实验三：线程本地 partial

实现多个 split 的局部聚合，每个 split 生成 `PartialCount`，最后合并。最终结果与串行 reference 一致。报告记录 split 数量、每个 partial 的记录数、每个 partial 的 key 数量、合并耗时和输

出 checksum。这个实验训练减少共享和观察瓶颈迁移。

实验四：临时 block 和 manifest

把每个 partial 写成临时 block，再通过 manifest 提交。Reduce 只读取 manifest。模拟一次 worker 写出临时文件后崩溃，再重试同一个 split。验收条件是目录里可以有多个临时文件，manifest 只接受一个 attempt，最终结果不重复计数。这个实验训练提交点和恢复扫描顺序。

实验五：写一页系统报告

报告包含输入合同、reference 输出摘要、数据身份图、执行等待图、成本账本、故障注入结果和未验证边界。未验证边界写得越具体，结论越可信。这个实验训练把代码、原理和证据合在一起表达。

习题与作业

习题一：为日志统计定义三组输入：正常输入、边界输入和破坏朴素假设的输入。说明每组输入验证哪条语义。

习题二：把共享全局 map 版本和线程本地 partial 版本画成执行等待图。指出两者主要等待点有什么不同。

习题三：构造一个重复 attempt 导致重复计数的反例，再用 manifest 修复它。要求写出每个 attempt 的 `AttemptId`、临时路径和最终 manifest 条目。

习题四：为项目增加最小 trace。每个任务至少记录 created、queued、running、finished、committed 五个状态，并说明这些状态怎样帮助定位等待。

习题五：写出一份一页系统报告，包含输入摘要、版本说明、reference 校验、主要指标和未验证边界。

1.14 本章小结

本章从一个日志统计作业建立了全书地图。朴素程序先给出 reference；reference 迫使正确性、错误策略和记录身份变清楚；输入放大后，数据路径、缓存、TLB、分配和字符串生命周期进入分析；并行化后，线程本地 partial、有界队列、任务状态和等待证据进入分析；写出结果后，manifest、checksum、attempt 和恢复扫描顺序进入分析；扩展到多机后，split、shuffle、straggler、一致性和分布式提交继续沿同一条线生长。

本章建立的学习姿势是：概念从问题中出现，代码承接边界，实验验证结论，报告承认限制。后续每一章都会选择这条链上的一个局部放大。读者如果能复述一条日志怎样被解析、怎样获得身份、怎样进入 batch、怎样形成 partial、怎样通过 manifest 生效、失败后怎样恢复、实验怎样证明这些边界没有坏，就已经掌握了进入第二册的地图。

下一章把视角缩进 CPU 核心内部，讨论流水线、乱序执行、分支预测和依赖链。它回答本章留下的第一个硬件问题：同样处理一批记录，为什么某些循环能持续供给执行单元，某些循环却被依赖、分支或取数等待卡住。只要带着日志统计的数据路径继续阅读，硬件知识就会成为解释系统性能的第一块地基。

第 2 章

核心流水线、乱序执行与分支预测

线上日志解析器曾经遇到过一个很典型的退化：固定格式日志吞吐稳定，换成用户自定义字段后，代码没有改，单核吞吐却下降，尾延迟也抖动。源码层面仍然只是一个循环：读取字节，判断分隔符、数字、引号、转义或非法字符，推进状态，偶尔写出字段边界。若只看 C++ 行数，很容易把问题说成“字符串处理慢”；把输入分布、反汇编和性能计数器放到一起，才会看到真正的因果链：随机分支让预测失败变多，错误路径混进热循环让前端供给变差，状态变量形成循环携带依赖，某些字段统计又把加法串成一条长链。

本章不从“五段流水线”这类静态图开始，而从热循环为什么突然变慢开始。流水线解释为什么一次猜错分支会丢掉已经取入和译码的工作；乱序执行解释为什么多累加器能拆开一条依赖链；前端、执行端口和 load/store queue 解释为什么源码行数相同，机器看到的工作形状却不同。硬件概念只能解释一段具体代码的性能异常，才真正有用。

学习目标

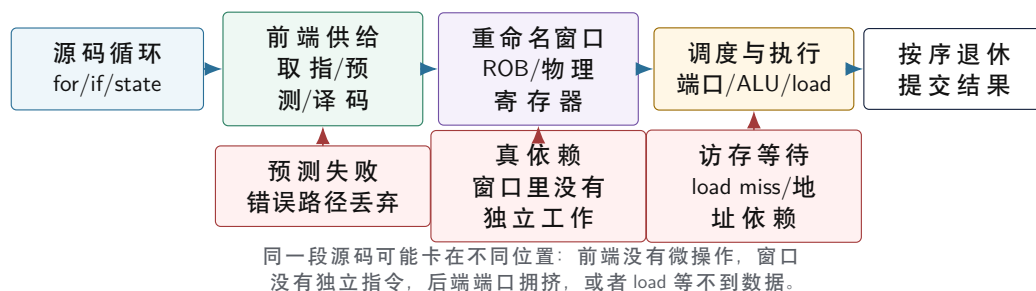
学完本章后，读者应能完成六件事：

1. 把一段 C++ 热循环拆成前端供给、依赖链、执行端口、load/store、分支预测和退休提交几个可观察问题。
2. 区分延迟、吞吐、IPC、前端受限、后端受限、错误投机和内存等待，而不是只说“CPU 没跑满”。
3. 用单累加器、多累加器、branchless 计数、指针追踪和冷路径分离解释乱序执行能隐藏什么、不能隐藏什么。
4. 根据输入分布判断分支预测是否可能是主因，并说明 branchless、查表、分桶和批处理各自付出的代价。
5. 用反汇编、编译器优化报告和 `perfstat/perfannotate` 或替代证据构造一份可复查的热循环证据包。
6. 在 Compute Systems Engine 中实现 `HotKernelReport`，让后续 SIMD、数据布局、线程和任务运行时章节复用同一套单核证据。

工程案例

贯穿案例是日志解析热循环。输入有三类：规则日志、随机用户字段和错误密集字段。规则日志让分支预测器容易学习；随机字段让字符类别分支接近不可预测；错误密集字段把本该很少进入的错误路径拉进热循环。每个优化版本都必须输出相同的字段数、错误摘要和 checksum，否则性能比较无效。

热循环在核心内部的路径



图示: 本图要证明的命题是: 热循环优化不是“少写几行代码”, 而是改变核心内部的供给、依赖和等待形状。

2.1 从现象到假设

性能分析的第一步不是猜硬件部件, 而是把现象写成可证伪假设。日志解析器变慢时, 可以先提出四个候选解释: 输入更大导致内存带宽不足; 字符类别更随机导致分支预测失败; 错误行增加导致冷路径进入热循环; 字段处理函数无法内联导致前端供给不足。四个假设都能让程序变慢, 但它们要求完全不同的修复。

若瓶颈是内存带宽, 减少几个整数分支可能没有端到端收益; 若瓶颈是分支预测, 把线程数翻倍只是把低效循环复制到更多核心; 若瓶颈是错误路径, branchless 改写不如冷热路径分离; 若瓶颈是调用边界, 数据布局 and API 可能比局部表达式更重要。大师级性能判断的核心, 不是知道很多技巧, 而是知道哪个技巧对应哪个证据。

四类证据 第一类证据是输入分布。记录每类字符、字段长度、错误比例、短行比例、长字段比例和随机种子。没有输入分布, 分支预测分析会变成猜测。第二类证据是二进制形状。反汇编回答热循环是否有调用、间接跳转、额外边界检查、向量指令、栈 spill 和冷路径代码。第三类证据是运行指标。理想情况下记录 cycles、instructions、branches、branch-misses、cache-misses、IPC 和耗时分布; 没有 PMU 权限时, 也要保留耗时阶梯、反汇编和输入对照。第四类证据是正确性。优化版本必须和 reference 的 checksum、错误摘要和输出规模一致。

不要先开线程 单核热循环没有看清之前, 开线程常常会制造新的噪声。多个线程会引入线程创建、调度、cache line 迁移、内存带宽竞争和 NUMA 放置问题。若单核版本每个字符都被随机分支击穿, 八个线程只是让八个核心都浪费在错误投机上; 若单核版本每条记录都写共享原子, 更多线程还会放大 cache line 争用。第二册后面会系统讲线程池和任务运行时, 本章先把单核执行模型打牢。

核心理想

热循环报告的基本句式是: “在某类输入上, 某个二进制形状导致某类硬件等待; 某个改写改变了这个形状; 指标支持或不支持这个因果链; 新的瓶颈迁移到哪里。”

2.2 流水线不是源码顺序

处理器不会按源码行一行一行执行。前端预测下一条取指地址, 把指令从 instruction cache 取出, 译码成内部微操作, 可能命中 micro-op cache; 中间阶段做寄存器重命名, 分配重排序缓冲区、load buffer、store buffer 和调度队列; 后端等待操作数准备好, 把微操作发射到执行端口、加载单元、存储单元或分支单元; 最后退休阶段按程序顺序提交结果, 保证单线程可观察语义没有被破坏。

这个结构带来一个重要事实: 执行顺序和提交顺序不是一回事。乱序核心可以先执行后面已经准备好的独立指令, 隐藏一部分延迟; 但它必须按程序顺序提交, 让异常、分支和内存语义看起来仍然正确。重排序缓冲区可以理解成一个有限的“未来窗口”: 窗口里如果有独立工作, 核心能继续推进; 窗口里如果全是依赖同一个 load、同一个分支或同一个累加器, 核心只能等。

延迟和吞吐 延迟是一个操作从开始到结果可用需要多久; 吞吐是稳定状态下单位时间能完成多少操作。一个加法延迟可能不为零, 但核心可以每周期发射多个独立加法; 一个 load 命中 L1 也

要等待若干周期，但多个独立 load 可以同时在路上。优化热循环时，要问的是关键路径延迟限制了你，还是资源吞吐限制了你。单累加器常受关键路径限制；连续数组扫描在大输入上更容易受内存带宽限制；复杂 switch 可能受前端和分支恢复限制。

前端供给 前端的任务是持续给后端供应正确路径上的微操作。它要处理取指、分支目标预测、译码、micro-op cache、instruction cache 和代码布局。前端瓶颈的典型症状是：数据不大，算术也不重，后端端口看起来没有打满，但 IPC 仍然低。常见原因包括热函数过大、过度内联导致 I-cache 压力、虚调用和函数指针导致间接分支、大 switch 目标随机、错误路径和日志构造混在热路径。

后端执行 后端不是“算术单元”一个盒子。整数 ALU、向量执行、加载端口、存储地址端口、存储数据端口、分支单元和地址生成单元都有不同吞吐。一个循环可能总指令数不多，却因为每轮需要多个 load 卡在加载端口；也可能因为地址表达式复杂卡在地址生成；还可能因为只有一条累加依赖链，让执行端口空着也发不出新工作。后端分析要看微操作混合、端口压力、依赖链和访存等待，而不是只数源码里的运算符。

退休和精确异常 乱序执行必须让程序看起来像按顺序执行。退休阶段按程序顺序提交结果，保证异常和分支错误能恢复到清楚边界。这就是为什么分支预测失败会浪费工作：错误路径上的微操作可能已经取指、译码甚至执行，但它们不能退休，最终要被丢弃。错误投机消耗前端、执行端口和缓存资源，却不贡献有用结果。

深入理解

“流水线越深越快”是入门阶段的过度简化。深流水线可以支持更高频率，但分支预测失败、依赖等待和前端供给不足都会放大浪费。现代核心的性能来自预测、缓存、乱序窗口、执行端口和内存层级的组合，不来自单一流水线长度。

2.3 乱序执行和依赖链

乱序执行只能重排独立工作，不能消除真实数据依赖。寄存器重命名可以消除某些假依赖，例如两次写同一个架构寄存器但并不互相需要结果；它不能让下一次加法在上一次累加结果还没出来时凭空完成。很多高性能改写，本质上就是把一个长关键路径拆成多个短路径，让乱序窗口里出现更多独立工作。

求和循环是最小例子。朴素版本清楚、正确、适合作 reference，但它把所有加法串成一条依赖链。

Listing 2.1 baseline: 单累加器顺序归约

```
1 std::int64_t sum_baseline(std::span<const std::int32_t> values) {
2     std::int64_t sum = 0;
3     for (const std::int32_t value : values) {
4         sum += value;
5     }
6     return sum;
7 }
```

多累加器版本没有改变算法复杂度，也没有减少读取字节；它改变的是依赖图。多个 partial 之间互不依赖，核心可以在一个 partial 等结果时推进另一个 partial。

Listing 2.2 多累加器：把一条长依赖链拆成多条短链

```
1 constexpr std::size_t SUM_LANE_COUNT = 4;
2
3 std::int64_t sum_multi_accumulator(std::span<const std::int32_t> values) {
4     std::array<std::int64_t, SUM_LANE_COUNT> partials{};
5     std::size_t index = 0;
6     const std::size_t limit =
7         values.size() / SUM_LANE_COUNT * SUM_LANE_COUNT;
```

```

8
9  for (; index < limit; index += SUM_LANE_COUNT) {
10     for (std::size_t lane = 0; lane < SUM_LANE_COUNT; ++lane) {
11         partials[lane] += values[index + lane];
12     }
13 }
14 for (; index < values.size(); ++index) {
15     partials.front() += values[index];
16 }
17
18 std::int64_t total = 0;
19 for (const std::int64_t partial : partials) {
20     total += partial;
21 }
22 return total;
23 }

```

这个例子的重点不是要求读者手写所有归约。现代编译器可能自动展开和向量化，某些平台上手写版本没有收益。教材要训练的是判断：如果 baseline 慢在累加依赖，多累加器可能有效；如果输入已经远大于缓存并接近内存带宽，多累加器收益会很小；如果是浮点求和，改变合并树会改变舍入结果，必须先写误差合同；如果展开过度，寄存器压力和代码体积会反过来伤害前端。

指针追踪是相反例子 连续数组扫描虽然会 miss，但地址规律明显，硬件预取器和乱序窗口可以让多个 load 同时在路上。链表遍历不同：下一次地址藏在当前节点里，必须等当前 load 返回。

Listing 2.3 指针追踪：下一次地址依赖当前加载结果

```

1 struct Node {
2     std::int32_t value = 0;
3     const Node* next = nullptr;
4 };
5
6 std::int64_t sum_list(const Node* head) {
7     std::int64_t sum = 0;
8     const Node* current = head;
9     while (current != nullptr) {
10         sum += current->value;
11         current = current->next;
12     }
13     return sum;
14 }

```

两个循环都是线性复杂度，但等待形状完全不同。链表可能每个节点都在不同 cache line 或不同页，TLB 和 cache 都不友好；更关键的是，核心很难提前发起下一次请求。第 3 章会从缓存和 TLB 角度展开，本章先建立流水线直觉：局部性不只是命中率，也是硬件能否提前知道下一步地址。

内存级并行度 内存级并行度（memory-level parallelism）指核心同时挂起多个独立内存请求的能力。数组、多个独立游标、分块处理和批量 probe 都能提高它；单链表、递归对象图和每步依赖共享原子结果都会降低它。乱序窗口再大，也不能让一个地址依赖链变成多个独立地址。软件要做的事，是在语义允许时把独立工作显式放进窗口。

2.4 Load/Store Queue、别名和发布可见性

内存操作比寄存器操作复杂。核心要判断一个 load 是否可能读取前面尚未完成的 store；store 可以先进入 store buffer，稍后再对其他核心可见；load/store queue、store buffer、TLB、cache 和一致性协议都会参与成本。硬件会做内存消歧预测，但预测有边界。编译器也会被语言别名语义约束，不能随意重排可能互相影响的读写。

下面的循环语义清楚，却没有说明 **input** 和 **output** 是否重叠。

Listing 2.4 别名不清会让编译器和硬件更保守

```
1 void add_scaled(std::span<const std::int32_t> input,
2               std::span<std::int32_t> output,
3               std::int32_t scale) {
4     const std::size_t count = std::min(input.size(), output.size());
5     for (std::size_t index = 0; index < count; ++index) {
6         output[index] += input[index] * scale;
7     }
8 }
```

若调用方传入同一数组的重叠视图，某一轮写入可能影响后续读取。编译器为了保持 C++ 语义，必须保守。工程上解决别名问题，不应只靠“我知道不会重叠”的口头承诺，而要靠接口、所有权和测试：输入使用只读视图，输出由调用者保证独占，调试版本检查范围不重叠，或者把算法改成先读后写的两阶段结构。

store buffer 和并发语义 普通 store 进入 store buffer 后，本线程可以继续执行，但其他核心不一定按你想象的顺序看到写入。后面讲 C++ release/acquire 时，会把这条硬件直觉变成 happens-before 关系。此处先记住一件事：单线程流水线为了吞吐会暂存、预测和重排内部工作；多线程程序若需要可见性顺序，必须通过锁、原子和内存序把边界说清。

热原子不是好的归约 许多初学者把并行求和写成多个线程同时更新一个 `std::atomic`。结果可能正确，却经常很慢。即使用 `memory_order_relaxed`，所有线程仍在争夺同一条 cache line 的独占修改权。Relaxed 只放松顺序约束，不能消除一致性流量。正确方向通常是线程本地 partial：热路径在寄存器和本地槽位中累加，最后再合并。这个原则会在第 4 章、第 9 章和第 13 章反复出现。

2.5 分支预测来自输入分布

分支预测器不是读懂业务逻辑，它根据分支地址、历史方向、目标和上下文猜下一步。循环回边通常容易预测；数据相关、接近随机、目标多变的分支难预测。一次错误预测的成本不是“分支指令本身慢”，而是错误路径上已经取入、译码甚至执行的工作被丢弃，前端重新从正确路径供给，乱序窗口里的有用工作突然减少。

阈值计数是最小例子。

Listing 2.5 带条件分支的过滤计数

```
1 std::int64_t count_greater_branch(std::span<const std::int32_t> values,
2                                  std::int32_t threshold) {
3     std::int64_t count = 0;
4     for (const std::int32_t value : values) {
5         if (value > threshold) {
6             ++count;
7         }
8     }
9     return count;
10 }
```


如果输入已经排序，或者几乎全都大于阈值，分支很容易被猜中。若输入随机且一半大于阈值、一半不大于阈值，预测会困难得多。可以写一个无显式分支版本：

Listing 2.6 把条件转成整数贡献

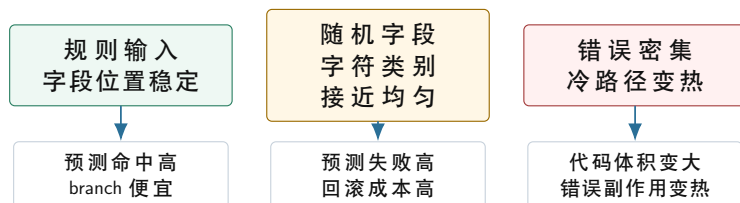
```
1 std::int64_t count_greater_branchless(std::span<const std::int32_t> values,
2                                     std::int32_t threshold) {
3     std::int64_t count = 0;
4     for (const std::int32_t value : values) {
5         count += value > threshold ? 1 : 0;
6     }
7     return count;
8 }
```

这段源码仍然包含条件表达式，编译器可能生成条件移动、集合指令、向量比较，也可能仍生成分支。教材的重点不是保证某种机器码，而是训练实验设计：规则输入和随机输入是否表现不同；branch-misses 是否同向变化；branchless 版本 instructions 是否增加；总时间是否下降；当输入变得可预测后，收益是否消失。若这些现象吻合，分支预测才是有证据的解释。

branchless 不是银弹 Branchless 改写通常用更多算术、掩码或查表换更少控制依赖。如果原分支高度可预测，改写可能更慢；如果两边工作量差异很大，branchless 会执行本来可以跳过的工作；如果查表引入额外 load，瓶颈可能迁移到缓存或端口；如果掩码表达式增加寄存器压力，编译器可能 spill 到栈。判断 branchless 的唯一可靠方法，是在明确输入分布上测量，并阅读热循环机器码。

冷路径分离 解析器里最常见的前端污染，是把罕见错误处理写在每个字符扫描的内层循环中。错误路径可能构造字符串、记录上下文、格式化日志、分配对象或抛出异常。高质量工程代码不是不处理错误，而是让热循环只记录紧凑错误摘要，例如 record id、byte offset、错误码和少量上下文边界；完整错误对象在块结束后或慢路径中构造。这样正常路径短、直、可预测，错误语义仍然完整。

输入分布怎样改变分支成本



同一段 if 在不同输入上有不同硬件成本：实验必须覆盖输入族，不能只跑一种随机数据。

图示：本图要证明的命题是：分支成本来自输入历史是否可预测，不来自源码里是否出现 **if** 这个词。

2.6 前端设计：热路径要短、直、可批量

前端问题最容易被低估。数据库执行器、日志解析器、解释器、正则引擎、协议解析器和多态对象处理器，常常不是因为算术重而慢，而是因为每条记录都做复杂分派、每个元素都走虚调用、每个 case 都带着冷错误逻辑，导致 instruction cache、分支目标预测和译码供给受压。

虚函数和间接分支 面向对象接口在控制面很有价值，但每个元素一次虚调用可能让热路径变成间接分支和不可内联调用。若对象类型分布稳定，预测器还能学习；若类型随机，目标预测困难，编译器也看不到具体循环体，无法展开或向量化。优化方向通常不是全局废除抽象，而是分离控制面和数据面：控制面保留清晰接口，数据面把一批同类记录交给具体内核批量处理。

大 switch 和解释器循环 解释器或字节码执行器常见一个大 **switch**。若 opcode 分布随机，分派成本高；若每个 case 很小，分派占比更高；若每个 case 很大，I-cache 压力上升。工业界常用的

方向包括 direct threading、superinstruction、热点 opcode 专门化、JIT、按 opcode 分桶和批处理。每个方向都在做同一件事：减少每个元素上的不可预测控制流，让一批同类工作走同一条紧凑路径。

解析器的批量化 日志解析器也可以借鉴这个原则。朴素状态机按字符推进，语义清楚，适合作 reference。优化版本可以先批量扫描结构字符，例如分隔符、换行、引号和转义候选，再让标量慢路径确认复杂语法。这样普通字节走短路径，复杂输入回到完整状态机。这个设计不是免费：短行上启动成本可能不划算，错误密集输入会频繁退回慢路径，代码也更复杂。因此报告必须写普通路径命中率、回退次数和错误摘要一致性。

工程案例

工业设计：simdjson 的两阶段思想。约束是 JSON 解析既要高吞吐，又要保持完整语义。核心原理是先快速找结构字符和索引，再按语法阶段解析，避免每个普通字节都进入复杂状态机。换来的能力是前端路径更规则、批量扫描更容易向量化；代价是中间结构索引、阶段边界和错误回退更复杂。迁移到 Compute Systems Engine 时，不要求照搬 JSON 算法，而是采用“先找结构位置，再做语义确认”的缩小版解析器实验。

2.7 工业界的诊断和内核组织

工业案例不能只列系统名。要看它面对什么约束，用什么机制购买什么能力，又付出了什么代价。本章选择四类参照：PMU Top-Down、编译器优化报告、数据库向量化执行和 HPC microkernel。它们的共同点是把“慢”拆成可观察边界，而不是凭直觉改代码。

PMU Top-Down：先分类，不直接给答案 Intel 的 Top-Down 思路把核心周期粗分为 retiring、bad speculation、frontend bound、backend bound 等方向。它的价值不是自动告诉你怎么改，而是阻止你在错误方向上花时间。若 bad speculation 明显，先看输入分布和分支形状；若 frontend bound 明显，先看代码体积、I-cache、间接跳转和译码；若 backend bound 明显，再分依赖、端口、load 和内存层级；若 retiring 高，说明核心大部分时间在提交有用工作，端到端瓶颈可能在别处。

这个方法也有代价。PMU 事件与微架构相关，不同 CPU、虚拟化环境、权限设置和采样方式都会影响可用性。报告不能把某台机器上的事件名写成永久事实。Compute Systems Engine 只采用它的诊断结构：先分类，再深入；有计数器时记录事件，没有计数器时写替代证据和边界。

编译器优化报告：让编译器解释拒绝理由 LLVM、GCC 和其他编译器能输出向量化、内联、循环优化和优化失败诊断。高性能 C++ 不是和编译器对抗，而是让真实语义进入源码：连续范围、只读输入、独占输出、明确合并合同、可见的循环边界、可内联的小函数。若报告显示因为别名、无法证明依赖或循环控制复杂而不能向量化，修复点往往是 API 和数据结构，而不是手写几条指令。

数据库向量化执行：每次处理一批，而不是每次处理一行 DuckDB、ClickHouse、Velox 等系统的向量化执行器都体现同一个原则：把一批列式数据送入算子，使用 selection vector、bitmap、column vector 和 batch 级状态减少每行分派。约束是分析型查询吞吐优先、数据列式友好、算子可批量执行；能力是更好的缓存局部性、更少虚调用、更容易 SIMD；代价是批次边界、空值语义、选择向量和 pipeline breaker 更复杂。迁移到本书项目，就是让 HotBatch 承载热字段列，让解析、过滤和统计尽量按 batch 运行，而不是每条记录都跨层调用。

HPC microkernel：把依赖、寄存器和内存层级一起设计 BLIS、OpenBLAS 等高性能库的 GEMM 内核不是简单写三重循环。它们用 blocking 控制缓存工作集，用 packing 改变访存形状，用 microkernel 固定寄存器 tile，用 ISA dispatch 选择平台实现。约束是矩阵乘的语义稳定、算术密度高、形状可分块；能力是接近硬件峰值；代价是代码复杂、平台特化和打包开销。本章不要求读者写 GEMM，但要学习它的工程态度：先有 reference，再设计数据移动，再设计寄存器级独立工作，最后用报告说明适用形状。

2.8 测量闭环：源码、二进制、计数器

本章的测量闭环有三个环节。源码告诉你想表达的语义；二进制告诉你编译器实际生成了什么；计数器和耗时告诉你运行时发生了什么。只看源码，会把“看起来简单”误判成“机器工作少”；只看反汇编，会忘记输入分布和语义边界；只看耗时，会得到无法解释的数字。

反汇编读什么 不要求读者背所有指令。先找循环体范围、回边、条件跳转、函数调用、间接跳

转、load/store 数量、是否有向量指令、是否有栈访问、尾部如何处理。再把这些事实对应回源码：哪个 `if` 变成了分支，哪个条件表达式变成了条件移动，哪个小函数没有内联，哪个 `span` 访问生成了边界或别名保护，哪个错误路径还留在热函数里。

计数器怎样解释 `perfstat` 可以记录 `cycles`、`instructions`、`branches`、`branch-misses`、`cache-misses` 等指标；`perfrecord` 和 `perfanotate` 可以把热点关联到指令地址。它们都不是自动答案。IPC 低可能是前端饿、后端等内存、分支频繁回滚，也可能是程序本来等待同步。Branch misses 高要结合输入分布；cache misses 高要结合字节流量和地址序列；instructions 下降而时间不降，说明瓶颈可能已经迁移。报告要写“当前证据支持什么”，也要写“当前证据不能证明什么”。

受限环境的降级路径 手机、容器、CI 和普通用户权限下，硬件计数器可能不可用。教材不能因此停止训练。降级路径包括：固定输入族和随机种子，重复多轮记录耗时分位数；保存编译器版本、优化选项和反汇编；改变输入规模做 L1/L2/LLC/内存阶梯；改变输入分布观察趋势；保存 checksum 防止被优化掉；明确写出无法读取 PMU 的原因。没有 PMU 权限，不等于可以写无证据结论。

源码阅读任务

本章源码阅读建议从三个入口任选一个。第一，阅读 Linux `kernel/events/` 和 `tools/perf/` 附近路径，解释用户态 `perfstat` 如何请求事件、内核如何管理计数器、权限如何限制结果。第二，阅读 LLVM `LoopVectorize` 或 `SLP vectorizer` 的文档和一条小循环诊断，解释为什么某个循环没有向量化。第三，阅读一个工业解析器或数据库执行器的热路径，找出 `reference`、`fast path`、`slow path`、`batch` 边界和错误回退。报告必须把源码路径映射回本章某个实验现象。

2.9 项目交付：HotKernelReport

本章给 Compute Systems Engine 留下的对象是 `HotKernelReport`。它不是热路径对象，而是实验和回归对象。每一个热内核优化提交，都必须把语义、输入、二进制和运行证据放进同一份报告。后续第 7 章 SIMD、第 8 章内核套件、第 13 章归约和第 18 章最终项目都复用它。

Listing 2.7 HotKernelReport 的教学形态

```
1 enum class CounterState {
2     Available,
3     Unavailable
4 };
5
6 struct CounterValue {
7     CounterState state = CounterState::Unavailable;
8     std::uint64_t value = 0;
9     std::string note;
10 };
11
12 struct HotKernelReport {
13     std::string kernel_name;
14     std::string variant_name;
15     std::string input_family;
16     std::string compiler_flags;
17     std::string disassembly_note;
18     std::uint64_t input_bytes = 0;
19     std::uint64_t record_count = 0;
20     std::uint64_t output_checksum = 0;
21     std::chrono::nanoseconds elapsed{};
22     CounterValue cycles;
23     CounterValue instructions;
```

```

24 CounterValue branches;
25 CounterValue branch_misses;
26 CounterValue cache_misses;
27 };

```

这个结构故意把 unavailable 当成显式状态。没有硬件计数器时，字段不能默默填零，因为零会被误读成事件不存在。报告还要包含输入族和反汇编摘要，避免把单次耗时当成结论。若一个优化版本只在 **RandomFields** 上快，在 **RegularFields** 上慢，它不是失败，而是结论边界更清楚；若 checksum 变了，则无论耗时多漂亮都必须回滚或修复。

解析器四版本实验 本章项目实验至少实现四个版本。第一版是朴素 switch 状态机，作为 reference。第二版把字符类别判断改成小表，只改变分支形态。第三版把错误构造移出热循环，主循环只记录错误摘要。第四版做块级扫描，先找分隔符和特殊字符候选，再让标量状态机确认复杂语法。四个版本使用同一输入族，输出同一 checksum、字段数和错误摘要。

版本	改变的主要形状	预期收益	主要风险
朴素状态机 查表分类	无，语义最直观 控制依赖换成 table load	reference 可信 随机字符更稳	随机输入分支多 增加 load 和寄存器压力
冷路径分离 块级扫描	正常路径更短更直 每字节状态机变成批处理	前端和预测更干净 为 SIMD 留边界	错误摘要可能不足 短行和错误密集输入可能退化

输入族 至少准备四类输入。第一，规则日志：字段位置稳定，错误率接近零。第二，随机字段：字段长度和字符类别随机。第三，错误密集：非法字节、未闭合引号、连续分隔符和超长字段增多。第四，短行输入：每行很短，批处理启动成本更明显。每类输入都要记录随机种子、字节数、记录数、错误数和字符类别分布。

实验：为解析热循环生成 **HotKernelReport**。先运行朴素状态机，保存 checksum、错误摘要、反汇编和耗时；再分别运行查表分类、冷路径分离和块级扫描版本。报告必须解释每个版本改变的是分支形态、前端体积、依赖链、load 形状还是错误路径。若无法使用 **perf**，写清不可用原因，并用反汇编、输入分布和多轮耗时替代。

2.10 复查清单和习题

验收与评分标准

本章验收按五项检查。语义 25 分：reference、checksum、错误摘要和边界输入完整。二进制证据 20 分：保存编译器、选项、热循环反汇编和优化报告。运行证据 20 分：至少覆盖规则、随机、错误密集和短行输入，记录可用计数器或替代证据。因果解释 25 分：能说明旧版本卡在哪里，新版本改变了什么，瓶颈迁移到哪里。工程边界 10 分：能说明哪些结论依赖机器、编译器、权限和输入分布。

习题与作业

习题一：为 **sum_baseline** 和 **sum_multi_accumulator** 写同一组正确性测试。输入必须包含空数组、单元素数组、长度不是 **SUM_LANE_COUNT** 倍数的数组、负数数组和大规模随机数组。报告要说明每个测试覆盖哪一种语义边界。

习题二：构造四组 **count_greater** 输入：全真、全假、周期性和随机。比较 branch 版本和 branchless 版本。若可以使用 **perfstat**，记录 branches 和 branch-misses；若不能使用，保存命令、错误原因、反汇编和耗时趋势。

习题三：把一个链表求和改写成连续数组求和。不要只写“数组更快”，要从地址依赖、预取、TLB、cache line、分配器和对象生命周期解释两者差异，并说明哪些业务语义会

阻止替换。

习题四：选择项目中的一个解析或统计热循环，提交一份 **HotKernelReport**。报告必须包含输入族、reference、checksum、反汇编摘要、计数器或替代证据、结论边界和下一步实验。只提交优化版代码不合格。

下一章把视角从核心内部推进到数据怎样到达核心。乱序窗口可以隐藏一部分延迟，但它需要独立请求；分支预测可以保持前端供给，但它不能修复随机访存；多累加器可以拆依赖链，但它不能让远端内存变近。第 3 章将用 cache line、TLB、页表和地址序列解释这些等待来自哪里。

第 3 章

缓存、TLB 与页级性能

先看一个反常结果：两个程序都处理一千万条记录，理论复杂度都是线性扫描，甚至第二个程序少做了几次比较，但它更慢。继续看代码才发现，第一版把记录放在连续数组中，热路径顺序读取少数字段；第二版为了“对象更清晰”，把每条记录拆成多个堆对象，字段通过指针互相跳转。算法复杂度没有变，地址序列变了。核心不再连续拿 cache line，TLB 要覆盖更多页，预取器也很难提前猜到下一次访问。

本章从这种地址序列的变化出发讲缓存和 TLB。缓存不是抽象的“快内存”，它按 cache line 搬运数据；TLB 不是背景细节，它决定虚拟地址到物理地址翻译是否能被快速命中；页面首次触碰、映射关系和写共享会把同一个源码循环变成完全不同的硬件行为。读者要学会先画出地址访问顺序，再谈缓存层级、page walk、首次触页和局部性优化。

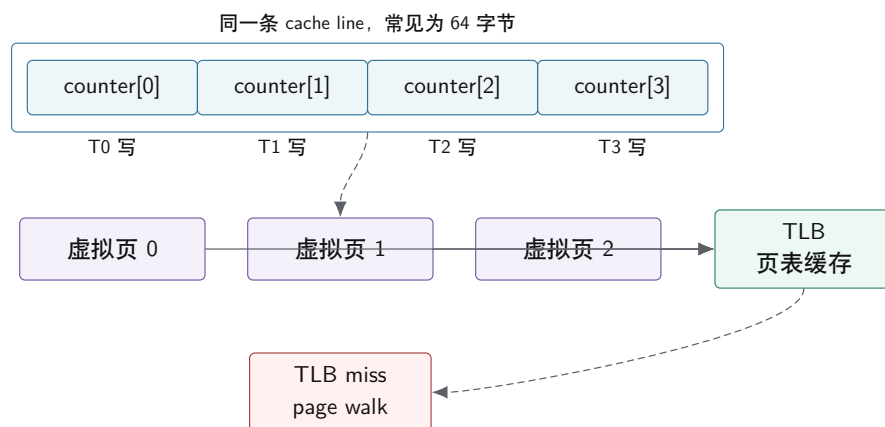
3.1 从地址访问到缓存层级

缓存不是按单个字节移动数据，而是按 cache line 移动。常见 cache line 大小是 64 字节。顺序访问数组时，一次 miss 可以带来后续多个元素；随机访问链表时，每个节点可能引入一次新的 miss。这个差异解释了为什么连续数组常常比指针结构更快。

缓存有容量、相联度和替换策略。容量不足会产生容量 miss，相联度不足会产生冲突 miss，多核心共享写会产生一致性流量。一个算法理论复杂度更低，不代表真实更快；如果它把连续访问变成随机访问，可能输给复杂度略高但局部性更好的算法。

从第二册的角度看，cache line 还是多核协作的最小一致性单元。两个线程哪怕写的是两个不同字段，只要字段落在同一条 cache line 上，硬件就必须在核心之间移动同一个一致性单元。很多并发程序的扩展性问题，不是算法复杂度问题，而是写入布局问题。后面讲锁、原子和无锁队列时，都要回到这个事实。

对象边界、cache line 边界和页边界不是同一回事



源码里的四个元素互不相关，硬件可能把它们作为一个一致性单元迁移；工作集跨页后，还要看 TLB 覆盖。

这类图要让读者形成一个直觉：源码里的变量边界，不等于硬件里的传输边界。硬件搬的是 cache line，TLB 记的是页，操作系统调度的是线程，分布式系统发送的是消息。边界错位，是系统性能问题最常见的来源之一。

缓存映射和冲突 缓存通常按 set associative 组织。一个地址会映射到某个 set，同一个 set 里只有有限个 way。如果多个热点地址恰好映射到同一个 set，而数量超过相联度，就会频繁互相驱逐，即使总工作集小于缓存容量也会慢。这类冲突 miss 很隐蔽，因为从容量角度看“不应该放不下”。

矩阵步长、数组对齐和结构体大小都可能触发冲突。若多个数组按相同大步长访问，并且基地址关系不佳，它们可能持续争用同一组 cache set。改变 padding、改变块大小或调整访问顺序，有时能显著改善性能。

Inclusive、Exclusive 和共享缓存 不同处理器的缓存层级策略不同。有的层级倾向 inclusive，也就是上层 cache 的内容也在下层保留；有的倾向 exclusive，尽量让不同层级存放不同内容；还有很多混合策略。软件通常不需要依赖某个具体策略，但要知道共享 LLC 不是无限资源。多个线程在同一 socket 上跑不同任务，可能互相驱逐 LLC。

这对并行任务调度有直接影响。把共享数据的任务放在共享缓存附近，可能复用 LLC；把互相干扰的大流式任务放在同一 socket，可能争抢带宽和 LLC。操作系统调度器会做一般性决策，但高性能程序常需要自己记录拓扑并做亲和性实验。

3.2 共享、写人和一致性成本

读缓存可以共享，写缓存必须取得 cache line 的独占所有权。两个线程写不同变量，如果这些变量落在同一条 cache line 上，硬件仍然会把它们当成同一块一致性单元反复迁移，这就是 false sharing。它不是锁竞争，却会表现出类似的扩展性下降。

解决 false sharing 的方法包括按线程分配独立槽位、使用对齐和填充、批量合并结果、减少共享写频率。并行归约通常先让每个线程写自己的 partial sum，最后再合并，正是为了避免每次加法都写同一个共享变量。

Listing 3.1 用对齐槽位减少 false sharing

```
1 struct alignas(64) LocalCounter {
2     std::int64_t value = 0;
3 };
4
5 std::vector<LocalCounter> counters(worker_count);
6 for (std::int32_t worker = 0; worker < worker_count; ++worker) {
7     counters[static_cast<std::size_t>(worker)].value += 1;
8 }
```

这段代码不是说所有结构体都要强行 64 字节对齐，而是表达一个设计原则：如果多个线程会高频写不同槽位，这些槽位不要共享同一条 cache line。否则程序表面没有共享变量，硬件层面仍然在共享一致性单元。

false sharing 的危险在于它不像数据竞争那样直接破坏结果。程序可能完全正确，测试也全部通过，只是在核心数增加时吞吐突然停止增长。排查时要看写入频率、字段布局、线程到槽位的映射和 cache line 迁移事件。若没有 `perfctr` 或类似工具，也可以用对齐填充做对照实验：只改变布局，不改变算法。如果对齐版本显著更快，就说明共享写路径值得重点检查。

读共享和写共享 共享数据不总是坏事。只读共享表、常量配置、不可变索引和代码段可以被多个核心同时缓存，通常扩展性很好。真正昂贵的是写共享，尤其是多个核心高频写同一条 cache line。读多写少的数据可以使用读写锁、RCU、版本化快照或 copy-on-write，目的都是让读路径尽量不进入写共享热路径。

写共享也分层次。最差情况是每次操作都写同一个全局变量，例如所有线程对一个原子计数器做 `fetch_add`。较好的情况是每个线程写本地槽位，周期性合并。更好的情况是让写入天然按数据分片分散，例如按 key 分片的哈希表或按 NUMA 节点分组的内存池。布局、同步和算法在这里是同一个问题的三个面。

3.3 从虚拟地址到页表和 TLB

虚拟地址访问物理内存前需要地址翻译。TLB 缓存虚拟页到物理页的映射。工作集页数太多或访问模式太随机时，TLB miss 会增加，核心需要走页表。大页可以减少页数，提高 TLB 覆盖范围，但也会影响内存分配、NUMA 放置和碎片。

理解 TLB 对数据结构设计很重要。一个跨很多小对象的图遍历，不只会造成 cache miss，还可

能造成 TLB miss。紧凑存储、池化分配和按块遍历，可以同时改善 cache 和 TLB 行为。

TLB 的特殊之处在于，它限制的是“能高效覆盖多少虚拟页”。一个程序的工作集如果只看字节数不大，但分散在大量页面上，仍然可能受到 TLB miss 影响。链表、树、哈希表桶、对象池和图邻接结构都可能出现这个问题。把对象压缩到连续数组里，不只是减少 cache miss，也是在减少活跃页数。

Page walk 的成本 TLB miss 后，硬件需要按页表层级查找映射。x86-64 常见四级或五级页表，每一级页表项本身也在内存里，虽然页表项可能被缓存，但最坏情况下一个地址翻译会触发多次内存访问。若程序随机访问大量页面，真正拖慢它的不只是数据 load，还有地址翻译。

大页把页大小从 4 KiB 扩大到 2 MiB 或更大，显著增加 TLB 覆盖范围。它适合大数组、数据库缓存和数值计算工作集，但不适合所有场景。大页可能增加内存碎片，NUMA 放置也要更谨慎。使用大页前应先使用计数器确认 TLB 确实是瓶颈。

Page walk 还会和缓存层级交织。页表项本身也要从内存层级读取，也可能命中或错过缓存。一个随机访问大数组的程序，表面上每次只读一个元素，实际上可能同时触发数据 cache miss 和页表访问。若工作集跨越大量页面，TLB miss 的成本会叠加在数据访问成本上。优化时不能只看“每个元素读几个字节”，还要看“每个时间窗口内活跃多少页”。

缺页和内存分配 分配内存不等于物理页已经准备好。操作系统通常按需分配页面，第一次触碰页面时可能发生 page fault。benchmark 如果把第一次触碰计入核心循环，会把页错误成本混入算法时间。严谨实验通常要预热、固定输入规模，并记录 page-faults。

缺页也分 minor fault 和 major fault。minor fault 不需要从磁盘读数据，可能只是建立页表映射；major fault 需要从存储设备读取，成本高得多。高性能计算通常要避免在热路径出现 major fault，也要避免把大量 minor fault 误判为算法本身慢。

第一册已经要求 benchmark 避免把初始化混进热路径。第二册要进一步要求初始化方式和计算方式一致。NUMA 机器上，如果主线程单独触碰全部页面，再让多个 socket 上的 worker 计算，页面可能集中在主线程所在节点，后续计算变成远端内存访问。正确实验通常让每个 worker 初始化自己之后要处理的分块，这样 page fault、物理页分配和计算拓扑保持一致。

3.4 文件、页缓存和持久化

mmap 可以把文件或匿名内存映射到进程地址空间。它让文件 I/O 看起来像内存访问，但不意味着没有 I/O 成本。第一次访问未驻留页面时仍然会缺页，顺序扫描大文件时内核预读可能有效，随机访问时可能产生大量 page fault。

数据库、搜索引擎和日志系统常利用 **mmap** 或 page cache，但必须理解内核何时回收页面、何时写回脏页、何时产生 I/O 抖动。系统工程里，虚拟内存既是便利抽象，也是性能变量。

Linux 源码阅读入口 第一册的 Linux 源码阅读只要求读者能从用户态现象找到入口。第二册要读出数据结构和性能路径。围绕本章，可以固定一个 Linux 版本，沿下面几条路径做窄范围阅读：页错误入口可从体系结构相关的 fault handler 进入，再走到通用内存管理逻辑；VMA 管理可关注 **mm/mmap.c** 和相关 maple tree 结构；页表和 fault 处理可关注 **mm/memory.c**；NUMA 策略可关注 **mm/mempolicy.c**；page cache 和文件映射可从 **mm/filemap.c** 开始。

源码阅读报告不要写成“Linux 使用某某机制”这么宽泛。合格报告要写清内核版本、阅读路径、关键结构、用户态实验和结论边界。例如研究首次触页，可以写一个大数组实验，记录 page-faults，再阅读 fault 路径中如何建立匿名页映射。若当前设备权限不足，仍然可以保留源码阅读，但必须承认它不是当前运行内核的直接证据。

3.5 内存实验与排查方法

缓存和 TLB 优化可以压缩成四条原则。第一，让热路径连续访问，减少随机指针跳转。第二，让共享写分散，避免多个线程争用同一条 cache line。第三，让工作集按块复用，避免超出缓存后仍反复访问。第四，让页面放置和线程执行位置一致，避免首次触页造成远端访问。这四条原则会贯穿后面的数据布局、并行归约、线程池、工作窃取和分布式 shuffle。

常见误区

不要把 `std::vector` 的连续性理解成所有成本都消失了。连续布局改善 cache line 和预取，但如果容量超过缓存，仍然会受内存带宽限制；如果首次访问大量新页，仍然会看到缺页成本。

从变量边界到硬件边界 本章最重要的模型，是把软件边界和硬件边界分开。C++ 源码里，`counter[0]` 和 `counter[1]` 是两个不同元素；硬件里，如果它们落在同一条 cache line 上，写入时就是同一个一致性单元。C++ 里，两个对象可能由两个 `new` 表达式产生；分配器里，它们可能挨在同一个页上，也可能散落到不同页。进程里，虚拟地址看起来连续；物理页可能分布在不同 NUMA 节点。性能问题经常不是因为某一层“错了”，而是因为层与层的边界没有对齐。

因此分析内存性能时，不要只画类图和对对象关系，还要画四种边界。第一是对象边界，说明字段和容器怎样组织。第二是 cache line 边界，说明哪些字段会一起被搬运和一致性维护。第三是页边界，说明工作集跨越多少页，TLB 是否能覆盖。第四是 NUMA 边界，说明页面在哪个节点，线程在哪个节点执行。只有对象图没有硬件边界，分析会停留在源码层；只有硬件术语没有对象图，优化又会脱离业务语义。

Compute Systems Engine 的 partial 数组就是一个很好的例子。源码里每个 worker 写一个 partial，逻辑上互不共享；如果每个 partial 只是一个 `std::int64_t`，八个 partial 可能正好落在同一条 64 字节 cache line 上。八个 worker 写不同元素，却让同一条 cache line 在核心之间迁移。若给每个 partial 做 `alignas(64)`，源码上的数组变胖了，内存占用增加了，但写共享路径被拆开。这里没有绝对答案，只有工作负载下的权衡：若 worker 很少或写入很少，padding 可能不值得；若每个 worker 高频写 partial，对齐常常值得。

局部性不是一个词 局部性至少有三种。时间局部性表示刚访问过的数据很可能很快再次访问；空间局部性表示访问一个地址后，附近地址很可能被访问；结构局部性表示程序的数据结构让相关数据天然挨在一起。很多教材只讲时间和空间局部性，但工程中结构局部性更容易被设计改变。

数组顺序扫描空间局部性很好，时间局部性可能不强，因为每个元素只用一次。矩阵分块同时制造时间局部性和空间局部性，因为一个块中的数据会被多次复用。哈希表查询如果 key 随机，空间局部性差；若桶数组连续但节点链表分散，第一跳和后续跳的局部性还不同。图算法更复杂，邻接表可能连续，但顶点状态数组访问可能随机。说“这个算法局部性不好”太粗，应说明是哪一类局部性不好，坏在数据结构的哪个位置。

局部性还要和读写性质一起看。只读随机访问通常只是慢，写随机共享访问可能同时慢且影响其他核心。只读大表可以被多个核心缓存；写热点计数器会触发一致性迁移；写不同字段如果落在同一 cache line 上，会形成 false sharing。后续讲无锁队列时，head 和 tail 的布局就是典型局部性问题：把它们放近可以减少对象体积，但若生产者写 tail、消费者写 head，放太近反而让两端互相干扰。

案例：三种计数器的内存路径 为了把缓存一致性讲清楚，可以比较三个计数器版本。第一个版本用 mutex 保护全局计数器，第二个版本用 atomic 全局计数器，第三个版本用分片计数器。三者都能给出正确总数，但内存路径完全不同。

Listing 3.2 baseline: mutex counter 表达最清楚的不变量

```
1 class MutexCounter {
2 public:
3     void add(std::int64_t delta) {
4         std::lock_guard<std::mutex> lock(this->mutex_);
5         this->value_ += delta;
6     }
7
8     std::int64_t value() const {
9         std::lock_guard<std::mutex> lock(this->mutex_);
10        return this->value_;
11    }
```

```

12
13 private:
14     mutable std::mutex mutex_;
15     std::int64_t value_ = 0;
16 };

```

mutex 版本的语义最直接：锁保护 `value_`。低竞争时，它可能只走用户态 fast path；竞争上升后，线程可能进入 futex 等待，调度器和唤醒成本会进入系统。它的瓶颈通常是临界区串行化和锁等待，不只是那一次整数加法。

Listing 3.3 改进但不一定扩展：atomic counter

```

1 class AtomicCounter {
2 public:
3     void add(std::int64_t delta) {
4         this->value_.fetch_add(delta, std::memory_order_relaxed);
5     }
6
7     std::int64_t value() const {
8         return this->value_.load(std::memory_order_relaxed);
9     }
10
11 private:
12     std::atomic<std::int64_t> value_{0};
13 };

```

atomic 版本消除了锁等待和内核睡眠路径，但所有线程仍写同一 cache line。低竞争时它可能比 mutex 快；高竞争时它可能因为 cache line 所有权迁移而扩展性很差。这里的 relaxed 只说明不需要用计数器同步其他数据，不说明这次写没有硬件成本。

Listing 3.4 扩展方向：按 worker 分片计数

```

1 struct alignas(64) CounterShard {
2     std::int64_t value = 0;
3 };
4
5 class ShardedCounter {
6 public:
7     explicit ShardedCounter(std::int32_t shard_count)
8         : shards_(static_cast<std::size_t>(shard_count)) {}
9
10    void add(std::int32_t shard, std::int64_t delta) {
11        this->shards_[static_cast<std::size_t>(shard)].value += delta;
12    }
13
14    std::int64_t value() const {
15        std::int64_t total = 0;
16        for (const CounterShard& shard : this->shards_) {
17            total += shard.value;
18        }
19        return total;
20    }
21

```



```

22 private:
23     std::vector<CounterShard> shards_;
24 };

```

分片版本把高频写入分散到多个 cache line。它牺牲了读取总数的成本和实时性：每次读取总数要扫描所有分片；如果读取和写入并发，还需要额外同步或定义弱一致语义。它适合写多读少的统计路径，不适合每次操作都需要立即得到全局精确值的协议。这个案例说明高性能设计常常不是“换一个更快原语”，而是改变读写比例和共享形状。

false sharing 的对照实验 false sharing 很适合用对照实验学习，因为它不改变算法结果，只改变布局。实验可以构造两个版本。版本一使用紧凑数组，每个线程反复写自己的槽位；版本二使用 `alignas(64)` 的槽位。两个版本的循环次数、线程数和写入次数完全相同，只改变槽位是否共享 cache line。如果对齐版本在多线程下显著更快，而单线程差异不大，就说明写共享边界是关键变量。

实验报告要避免两个误区。第一，不要只跑一个线程数。false sharing 的危害随线程数和核心拓扑变化，至少应从 1 线程跑到物理核心数。第二，不要只看耗时。若工具允许，应观察 cache line 迁移或相关事件；若没有权限，也应写出对照设计为什么能隔离变量。第三，不要把 padding 当成默认答案。对齐会增加内存占用，可能降低缓存容量利用率。只有高频并发写的槽位才优先考虑 padding。

TLB 覆盖范围和数据结构 很多人把 TLB miss 当成“虚拟内存细节”，但它会直接改变数据结构选择。假设页面大小是 4 KiB，一个数据结构每个节点只有几十字节，但节点随机分布在几百万个页面上。每次访问节点不仅可能 cache miss，还可能 TLB miss。TLB 覆盖范围是有限的，活跃页数超过覆盖范围后，地址翻译本身就会变成瓶颈。

紧凑数组同时改善两件事：一条 cache line 带来多个相邻元素，一个 TLB 项覆盖同一页内多个元素。对象池也有类似作用：即使保留指针结构，至少让节点来自较少的连续页面。图算法中的 CSR 格式把邻接边压成连续数组，不只是为了少分配，也是为了提高 cache 和 TLB 友好性。哈希表开放寻址常常比链地址法更 cache 友好，也部分因为它减少指针跳转和页面分散。

大页是另一种提高 TLB 覆盖范围的方法。把 4 KiB 页换成 2 MiB 页，同样数量的 TLB 项能覆盖更大地址范围。它适合大数组和长时间驻留的大工作集，但要考虑碎片、权限、部署和 NUMA 放置。不要一看到 TLB miss 就无脑开大页。正确顺序是先证明活跃页数和 TLB miss 是瓶颈，再评估大页对分配、首次触页和系统配置的影响。

首次触页和 NUMA 的隐藏变量 NUMA 机器上，页面通常在首次写入时分配到触碰它的线程所在节点。若主线程分配并初始化整个数组，然后把计算分给其他 socket 上的线程，其他线程可能大量访问远端内存。程序源码里看不到这个问题，因为数组地址连续、线程切分也正确；硬件拓扑里却出现远端访问。

这就是为什么 benchmark 的初始化也属于实验设计。若计算阶段按 worker 分块，那么初始化阶段也应按同样分块让 worker 触碰自己的页面。这样 page fault、物理页分配和计算线程位置更一致。若为了方便只让主线程初始化，测到的可能不是算法性能，而是错误页面放置后的远端带宽。

Compute Systems Engine 后续要把输入生成拆成两种模式。第一种是 single-touch，由主线程初始化，用于展示错误实验设计可能带来的远端访问。第二种是 parallel first-touch，由每个 worker 初始化自己负责的块，用于展示正确放置。两者结果如果在单 socket 机器上接近，在多 socket 机器上可能差异明显。教材不能假设每个读者都有 NUMA 设备，但要让读者知道该怎样设计实验。

矩阵访问：从行列顺序到分块 矩阵是讲缓存最经典的案例。行优先存储的二维数组按行扫描，连续访问；按列扫描，步长等于一行长度，若矩阵很大，每次访问都可能落到新的 cache line 甚至新的页。这个例子不能只停在“行优先快”上，还要继续推进到分块。

矩阵乘法中，朴素三重循环会反复访问矩阵块。如果循环顺序不合适，某些数据刚被加载就被驱逐，下次又要从更慢层级取回。分块的思想是让一小块数据在缓存中被多次复用，提高算术强度。块太小，循环开销和复用不足；块太大，超过缓存后又失效。块大小还与 cache 容量、相联度、TLB、SIMD 宽度和数据类型有关。

这个案例的意义不是要求第二册变成矩阵计算专著，而是让读者学会一类推理：先估算工作集大小，再估算每个块的数据复用，再根据缓存层级选择实验矩阵。后面讲分布式 shuffle 时，思想相同：先本地聚合减少网络数据，再按分区批量发送，避免每条记录都独立跨网络移动。cache block

和 network batch 在不同尺度上解决同一个问题：让昂贵移动服务更多有用计算。

分配器和对象布局 C++ 程序常把性能问题归咎于容器，却忽略分配器。频繁小对象分配会让对象分散、增加元数据开销、破坏 cache 和 TLB 局部性，还可能引入锁竞争。多线程分配器通常有线程本地缓存、arena 或 tcache 机制，用于减少全局锁，但也会影响对象在哪个线程、哪个 NUMA 节点上实际分配。

对象池不是万能答案。它可以改善局部性和分配成本，但也会引入生命周期管理、碎片复用、调试困难和峰值内存占用问题。如果对象跨线程释放，池的所有权和回收路径又会变复杂。无锁数据结构尤其要小心：节点从结构中摘除，不等于可以立即释放；分配器复用同一地址还可能放大 ABA 问题。缓存、TLB、分配器和内存回收最终会在无锁章节汇合。

写回、脏页和 I/O 误判 并不是所有内存成本都来自 DRAM。文件映射、page cache 和脏页写回会把内存访问和 I/O 混在一起。一个程序用 `mmap` 扫描文件，第一次访问可能触发缺页和磁盘读取；写入映射文件后，脏页可能稍后由内核写回，导致延迟在看似无关的时刻出现。若 benchmark 只测用户态循环，不记录 page faults 和 I/O，就可能把系统行为误判为算法波动。

日志系统和数据库特别容易遇到这个问题。用户写入内存缓冲并不等于数据已经安全落盘；调用 `fsync` 又会把持久化成本集中暴露出来。第二册虽然重点是计算系统，但分布式计算章节会讨论检查点和输出提交，那里必须区分“写入内存”“写入 page cache”“持久化到存储”和“对外宣布提交”。单机虚拟内存知识会直接影响分布式正确性。

Cache line 是共享和移动单位 CPU cache 不是按单个字节搬运，而是按 cache line 搬运。常见 cache line 大小是 64 字节，具体平台要查询。程序读取一个 `std::int32_t`，硬件通常会把它所在的整条 cache line 带入缓存。顺序扫描数组时，这很好，因为一条 cache line 包含多个相邻元素；随机访问时，可能每条 cache line 只用一个元素，大部分带宽被浪费。

Cache line 也是一致性协议的基本单位。两个线程写同一条 cache line 上的不同变量，也会互相争夺所有权，这就是 false sharing。软件层面看它们没有共享变量，硬件层面它们共享移动单位。理解这一点后，`alignas(64)`、结构体 padding、线程本地 partial 和 per-worker stats 的意义就很清楚：它们不是为了美观，而是为了让高频写不落在同一条 line 上。

但 padding 也有成本。一个 8 字节计数器对齐到 64 字节，内存占用变成 8 倍。若有百万个计数器，padding 会浪费大量 cache 容量和内存。只有高频跨线程写的槽位值得优先隔离。只读共享或低频写共享不一定需要 padding。布局优化始终是成本交换。

缓存相联度和冲突 miss Cache 容量不是唯一限制。多数 cache 是组相联结构，一个地址会映射到某个 set，set 中只有有限 way。如果多个热地址映射到同一个 set，超过相联度后会互相驱逐，即使总工作集小于 cache 容量，也可能 miss 很多。这叫 conflict miss。矩阵步长、哈希表大小和数组对齐都可能触发冲突。

这也是为什么实验要改变输入大小和步长。若某个规模突然变慢，可能不是算法复杂度变化，而是 cache set 冲突、TLB 覆盖范围或页边界效应。通过改变数组 padding、起始偏移、步长和容量，可以验证是否存在冲突。不要把所有拐点都简单归因于 L1、L2、L3 容量。

写分配和写回 写入内存也会触发 cache 行为。普通写 miss 常采用 write-allocate：先把目标 cache line 读入缓存，再修改，之后再写回内存。对将来不会再读取的大块输出，这可能浪费带宽，因为读入旧内容没有价值。某些架构提供 non-temporal store，用于流式写出，减少缓存污染和写分配。但它也有边界：如果后续马上读取这些数据，绕过缓存可能变慢。

并行算法写输出时要考虑写分配。Stream compaction 的输出可能是连续写，后续 reduce 也许马上读；此时普通写可能有益。文件 I/O 或网络发送缓冲如果只是写完交给设备，缓存污染可能更明显。教材不要求读者手写 non-temporal 指令，但要知道写也会产生内存流量。

硬件预取器能看懂什么 硬件预取器擅长顺序访问和某些固定步长访问。它不理解复杂指针结构，也不理解哈希表随机访问。若程序顺序扫描 `std::vector`，预取器能提前请求后续 cache line；若程序遍历链表，下一地址必须等当前节点加载出来，预取器很难帮忙。若程序按随机索引访问数组，即使数组本身连续，访问序列仍然不可预测。

这说明“连续存储”和“连续访问”是两件事。一个 vector 按随机索引访问，仍可能 miss 很多；一个链表如果由对象池按顺序分配，也许比完全随机链表好，但仍受下一指针依赖限制。优化访问模式比手写 prefetch 更可靠。软件预取只有在地址能提前算出、预取距离合适、不会污染缓存时才可能有效。

TLB 的多级结构 地址翻译也有缓存。TLB 保存虚拟页到物理页的映射，常有 L1 dTLB、iTLB 和更大的二级 TLB。TLB miss 时，硬件或内核需要遍历页表，成本可能很高。一个程序如果访问很多页面且每页只用很少数据，TLB miss 会成为瓶颈。链表、巨大哈希表和随机图遍历都容易遇到这个问题。

TLB 覆盖范围等于 TLB 项数乘以页面大小。使用大页可以扩大覆盖范围，但会带来分配、碎片、权限和 NUMA 放置问题。透明大页可能自动启用，也可能因为内存碎片或访问模式没有生效。性能报告中若讨论 TLB，应记录页面大小、是否启用透明大页、工作集跨越多少页，以及 dTLB 事件是否可用。

Page fault 的儿类含义 Page fault 不一定表示错误。Minor fault 可能表示虚拟页第一次映射到物理页、按需分配零页、建立页表项；Major fault 通常涉及磁盘 I/O，例如文件页不在 page cache 中。还有写时复制 fault：fork 后父子共享页面，某一方写入时内核复制页面。不同 fault 的成本差别很大。

Benchmark 如果没有预热和初始化，第一次访问大数组会包含大量 minor fault；第一次 mmap 文件会包含 page cache miss 和 major fault；fork 后写内存可能包含 COW 成本。若目标是热循环性能，就应把这些成本分离；若目标是冷启动或批处理端到端，就应保留并单独报告。不要让 page fault 悄悄混进“算法时间”。

虚拟内存和安全边界 虚拟内存不仅是性能机制，也是隔离机制。每个进程看到自己的虚拟地址空间，页表控制哪些页可读、可写、可执行。缺页异常让内核有机会按需分配、加载文件、处理 COW 或拒绝非法访问。理解这些机制有助于解释为什么越界访问可能崩溃，为什么 mmap 可以把文件映射为内存，为什么进程间共享内存需要明确映射。

高性能系统有时会使用共享内存绕过网络或减少拷贝，但共享内存会重新引入并发同步和生命周期问题。两个进程共享一块内存，不代表它们自动有一致的协议。仍然需要 ring buffer、sequence、futex、eventfd 或其他同步机制。虚拟内存提供地址映射，不提供业务语义。

Page cache 和持久化 Linux 的 page cache 会缓存文件数据。读取文件时，数据可能来自内存中的 page cache，而不是磁盘；写文件时，写入通常先进入 page cache，稍后由内核写回。调用 write 成功，不等于数据已经安全落盘。调用 fsync 或 fdatasync 才把持久化成本显式拉到当前路径，但具体语义还受文件系统和设备影响。

这对检查点非常重要。一个分布式作业写 checkpoint 文件后，如果没有正确 fsync，进程崩溃或机器掉电后文件可能不存在或内容不完整。写 manifest 时还要考虑目录项持久化，rename 后是否 fsync 目录。教学项目可以简化持久化，但文字上必须说明“写文件”和“可恢复提交”不是同一件事。

mmap 的优点和陷阱 mmap 可以把文件映射到地址空间，让程序像访问内存一样访问文件。它适合随机读取、由内核按需分页、避免显式 read 缓冲管理。缺点是 page fault 出现在访问指令处，延迟可能分散；错误处理也不同，访问损坏或截断文件可能触发信号；预取和释放需要 madvise 等机制辅助；写映射文件还涉及脏页和持久化。

顺序扫描大文件时，普通 read 加显式缓冲有时更容易控制 I/O 节奏和错误处理；mmap 更简洁，但性能不一定总更好。系统教材应避免把 mmap 神化。选择 read 还是 mmap，要看访问模式、错误处理、文件大小、内存压力和持久化要求。

内存分配和页粒度 用户态分配器向内核申请大块内存，再切给小对象。小对象释放后，内存可能回到分配器缓存，不一定立即还给内核。进程 RSS、虚拟内存大小和业务对象大小之间可能差很多。并发分配器还可能为每个线程保留缓存，线程退出或跨线程释放会影响内存回收和局部性。

这会影响性能实验。一个测试反复分配释放小对象，第二轮可能比第一轮快，因为分配器缓存已经热；也可能 RSS 不下降，让人误以为泄漏。对象池和 arena 可以改善可预测性，但要报告峰值内存和释放策略。内存优化不是只看 malloc 次数，还要看页、TLB、cache 和生命周期。

缓存一致性和虚拟内存的连接 Cache coherence 处理多个核心对物理内存中同一 cache line 的一致视图，虚拟内存处理虚拟地址到物理页的映射。两个不同虚拟地址可能映射到同一物理页，例如共享内存或文件映射；两个进程通过不同地址写同一物理页，仍会触发 cache line 一致性。理解这个连接，有助于分析 mmap 共享文件、进程间队列和 page cache。

分布式系统中，跨机器没有硬件 cache coherence，必须用协议维护一致性。单机共享内存的 cache line 迁移，在集群中变成 RPC、复制和共识。第二册先讲虚拟内存和 cache coherence，再讲

分布式复制，就是为了让读者看到“一致视图”的成本如何随尺度上升。

内存实验的设计 本章实验应覆盖三类访问。第一，顺序扫描，观察带宽和预取。第二，固定步长访问，观察 cache line 利用率、set 冲突和 TLB 覆盖。第三，随机访问或指针追踪，观察预取失效和内存级并行度下降。每类实验都要保持逻辑工作量一致，例如访问相同数量的元素，返回相同类型的校验结果。

实验还要控制数据初始化。先分配再初始化，记录初始化是否计时；如果比较 NUMA，初始化线程和计算线程要明确；如果比较 page fault，冷启动和热启动要分开；如果比较 mmap，page cache 状态要说明。内存实验最容易被隐藏状态影响，因此报告必须详细。

案例：对象数组和指针数组 一个常见布局选择是保存对象数组，还是保存指针数组。对象数组 `std::vector<Record>` 中对象连续，遍历时 cache 友好；指针数组 `std::vector<Record*>` 本身连续，但每个指针指向的对象可能分散。若热路径需要访问每个对象字段，指针数组会多一次间接访问，并可能导致随机 cache miss 和 TLB miss。

指针数组也有价值。对象很大且需要稳定地址，或者多个索引共享同一对象，或者对象由不同生命周期管理时，指针更灵活。高性能系统常用“稳定 ID 加对象池”折中：外部持有 ID，内部对象池尽量连续。这个案例提醒读者：局部性和稳定引用经常冲突，需要设计层面的取舍。

案例：内存映射日志文件 使用 mmap 扫描日志文件很方便。程序可以把文件映射成字节 span，然后查找换行和分隔符。但第一次访问页面可能触发 page fault，文件被截断时访问可能出错，映射区域的生命周期必须覆盖解析过程。若解析阶段把 `raw_offsets` 指向 mmap 区域，后续错误报告必须保证映射仍然存在。

普通 read 版本则显式管理缓冲。它可能多一次复制，但错误处理和背压更直接：读取一个 batch，解析完释放或复用缓冲。mmap 和 read 的选择取决于访问模式、错误处理和生命周期。项目可以同时保留两种输入后端作为实验，而不是假设 mmap 永远更快。

案例：Page fault 进入计时区 假设 benchmark 分配一个 1 GiB vector，然后直接开始计时求和，但初始化没有触碰所有页面。求和第一次读到页面时可能触发缺页，计时结果包含页表建立和物理页分配。另一个版本提前初始化数组，求和时没有这些 fault。两者时间差不一定来自求和算法。这个案例非常常见。

正确实验应明确分配、初始化、预热和计时。若要测端到端，就报告 page faults；若要测热循环，就在计时前触碰页面。没有这个区分，内存实验经常得出错误结论。

内存安全和性能 越界访问、use-after-free 和数据竞争不仅是正确性问题，也会破坏性能分析。未定义行为可能让编译器做出激进优化，导致 benchmark 结果没有意义。释放后访问可能偶尔命中旧内存，看似正常，压力下崩溃。性能代码更需要 sanitizer 和边界测试，因为它常使用手写布局、对象池和指针。

项目应在 debug/asan/tsan 构建中跑 correctness tests，在 release 构建中跑性能。不要使用 sanitizer 构建得出性能结论，也不要使用 release-only 测试证明内存安全。正确性工具和性能工具各有位置。

案例：Page cache 和 checkpoint Checkpoint 写入常被误测。程序把 checkpoint 写到文件后立即返回，测试显示很快；但真正崩溃恢复时，文件可能还在 page cache，没有持久化。若下一章分布式项目把这个时间当作提交点，就会得到错误可靠性。正确实验要区分 write 时间、fsync 时间和 rename 提交时间。

可以设计一个小实验：写固定大小 checkpoint，分别只 write、write 后 fsync、临时文件 fsync 后 rename 并 fsync 目录。比较时间，并说明每种语义。这个实验会让读者理解为什么可靠输出比普通文件写入复杂。性能和持久性是交换关系，不能只看最快路径。

案例：共享内存队列 两个进程通过 mmap 共享内存实现队列时，cache、TLB、虚拟内存和同步会同时出现。共享内存提供同一物理页映射，但 head/tail 的发布仍需要原子和内存序；进程退出时，队列状态可能停在中间；不同进程地址不同，不能在共享结构中保存普通指针，应保存 offset 或索引。

这个案例非常适合作为本章到无锁章节的桥梁。虚拟内存解决映射，cache coherence 解决同一机器上的可见性，C++ 原子解决语言级同步，状态机解决关闭和恢复。任何一层缺失，队列都不完整。

缓存章节总结 缓存、TLB 和虚拟内存共同决定“访问一个地址”的真实成本。连续访问利用 cache line 和预取，随机访问浪费带宽并增加 TLB 压力；page fault 和 page cache 会把内存访问与 I/O 混在一起；mmap 提供方便但不消除状态机；写入不等于持久化。理解内存层级后，数据布局、并行算法和分布式 checkpoint 才有坚实基础。

内存问题排查路径 遇到疑似内存瓶颈，可以按路径排查。第一，看工作集大小是否超过缓存层级，输入规模曲线是否有拐点。第二，看访问模式，是顺序、步长、随机还是指针追踪。第三，看每个元素实际使用多少字节，cache line 里有多少无用数据。第四，看页面数量和 TLB 覆盖范围，是否存在大量随机跨页访问。第五，看是否有 page fault、mmap 冷启动或 page cache 干扰。第六，看写入是否产生写分配、脏页和 fsync 成本。

这个路径可以应用到很多问题。哈希聚合慢，可能是随机访问和分配；图遍历慢，可能是邻接数据分散和 TLB；日志扫描慢，可能是 page cache 冷启动；checkpoint 慢，可能是 fsync 和写回。把“内存慢”拆成具体层次，优化方向才清楚。

内存章节项目要求 项目中每个大数据结构都应写明三件事：内存布局、生命周期、访问模式。ParsedLogHotBatch 是列式热字段，生命周期为一个 batch，访问模式为顺序扫描；ShuffleBlock 是序列化后的中间数据，生命周期到 manifest 清理，访问模式为写一次读一次；TaskGraphStorage 是连续节点和边，生命周期为一个 stage，访问模式为随机 ready 更新和顺序 successor 遍历。这样写文档能让后续优化有依据。

缓存章的回归阅读要从地址序列开始，而不是从层级参数开始。选同一批记录，分别用连续结构数组、指针数组、列式热字段和 mmap 冷读四种方式处理。算法复杂度都可以是线性的，但硬件看到的地址完全不同：连续数组利用 cache line，指针数组暴露随机 load，列式热字段减少无用搬运，冷 mmap 可能把 page fault 混进热路径。

实验设计要把 cache 和 TLB 拆开。小工作集主要看 L1/L2 行为，中等工作集看 LLC 和预取，大工作集看内存带宽，跨越大量页面时再观察 TLB 和 page fault。步长扫描要覆盖 1、cache line 大小、页大小和非整齐步长；矩阵访问要比较行主序和列主序；链式访问要说明预取器为什么很难帮忙。只有这样，局部性才不是一句空话。

项目中的 **MemoryShape** 应记录字节数、元素大小、热字段、页数、对齐和访问顺序。它不需要预测所有性能，但要让报告有对象可指。若一个优化只写“缓存友好”，却不说明减少了哪些字段搬运、少跨了多少页、地址序列如何改变，就应退回重写。缓存章的核心能力，是把数据结构翻译成硬件会经历的地址流。

Linux 观察入口也要服务地址流。读者可以用 `/proc/self/maps` 确认映射范围，用 `/proc/self/smaps` 观察 resident、dirty 和 huge page 线索，用 `perfstat` 记录 cache 与 dTLB 相关事件，在权限不足时退回页错误计数、工作集阶梯和冷热运行对照。源码阅读可以从 `mm/memory.c` 的缺页路径、`mm/filemap.c` 的页缓存路径和 `mm/mmap.c` 的映射管理开始，但阅读目标不是背内核函数，而是理解一次地址访问怎样从用户态 load 走到页表、页缓存和回收策略。

C++ 容器也要放进这张图里。连续 `std::vector` 通常给预取器更清楚的地址序列，`std::deque` 分段连续，`std::list` 节点追踪会制造大量指针 load，`std::unordered_map` 可能在 bucket、node 和 key 上来回跳。容器选择不是只看复杂度表，复杂度相同的 $O(n)$ 遍历可能有完全不同的 cache 和 TLB 行为。后面数据结构与算法书也会复用这个原则。

页级行为还要和文件 I/O 联系。顺序扫描一个大文件时，页缓存和 readahead 可能让访问看起来像内存读取；随机访问同一文件时，页缓存命中、缺页和磁盘调度会交织在一起；使用 mmap 时，load 指令可能触发 page fault，错误路径不再显式出现在源码里。教材必须让读者知道，虚拟内存不是抽象背景，它会直接改变热路径的可见成本。

本章也适合讲“冷启动”和“热启动”的区别。第一次运行可能包含磁盘读取、页表建立和 cache 冷 miss，第二次运行可能大量命中页缓存。若报告没有区分冷读和热读，就很容易把操作系统缓存当成算法优化。项目中每个文件扫描实验都应记录是否清理缓存、是否预热、输入是否重新生成，以及 page fault 数量是否和结论一致。

3.6 从地址序列理解缓存、TLB 和虚拟内存

缓存和 TLB 不应从层级图开始背。先写一串地址：连续数组访问、固定步长访问、随机索引访问、链表追踪、二维矩阵按行访问、二维矩阵按列访问。硬件看到的不是变量名，而是地址序列。地

址相邻，cache line 搬进来的一整块数据都可能被用上；地址跨页太多，TLB 覆盖不住，页表遍历进入成本；地址依赖上一次 load，预取器难以提前工作。把地址序列画出来，缓存章才不会变成“L1、L2、L3 多大”的参数表。

第一册讲过虚拟地址到物理地址的翻译，第二册要追问翻译成本怎样进入热路径。每次 load/store 都要经过地址翻译，TLB 命中时成本低，TLB miss 时可能触发多级页表遍历。若工作集跨越大量小页，TLB miss 会把看似顺序的扫描拖慢；若使用大页，TLB 覆盖范围变大，但分配、权限、碎片和系统配置也变复杂。教材要让读者看到大页不是开关，而是“用更大的页粒度换更少翻译项”的取舍。

Cache line 是搬运单位，也是争用单位

源码里你读取的是一个字段，硬件搬运的是一个 cache line。若结构体里热字段和冷字段混放，扫描热字段时会顺带搬动冷字段，浪费带宽。若多个线程更新不同变量，但这些变量落在同一条 cache line，硬件一致性协议仍会让整条 line 的所有权来回迁移，形成 false sharing。读者必须同时记住两句话：cache line 是空间局部性的机会，也是共享写的冲突边界。

教学实验可以从计数器数组开始。版本一让每个 worker 更新 `std::vector<std::uint64_t>` 中相邻槽位；版本二给每个槽位按 cache line 对齐和填充；版本三让每个 worker 写线程本地对象，最后合并。若版本一随线程数增加反而变慢，版本二或三改善，就能把 false sharing 变成可观察事实。实验报告还要说明代价：填充会增加内存占用，线程本地合并会增加最终汇总阶段，二者都不是免费。

Cache line 还影响写策略。普通写 miss 常常会先把旧 line 读入缓存再修改，这叫 write allocate。对一次性大块输出，读取旧内容可能没有价值；non-temporal store 可以减少缓存污染，但若后续马上读取输出，就可能变慢。高质量章节要写出这种边界，避免把特殊指令讲成单向优化。

局部性不是容器属性，而是访问属性

`std::vector` 连续存储，但使用随机索引访问时仍然可能局部性很差；链表节点逻辑相邻，但物理地址可能散落各处；结构体数组适合每次读取完整对象，列式数组适合只扫描少数字段。局部性由布局和访问顺序共同决定，不由容器名字决定。

日志项目可以用热冷字段说明。原始记录包含时间戳、用户 id、事件类型、原始文本、错误原因和调试标记。热路径只需要事件类型和少量数值字段，如果所有字段放在一个大对象数组里，扫描会搬动许多无用字节。列式 batch 把热字段拆出来，可以让解析后的计算阶段只读必要数据。冷字段仍然保留，用于错误报告和审计，但不进入每条记录的热循环。

局部性还要考虑生命周期。一个对象若在队列中跨线程传递，频繁分配释放会打散地址，也会给分配器和 TLB 带来压力。Arena 或对象池能把同生命周期对象放在一起，减少分配开销和提升局部性；但它也会推迟释放，可能增加峰值内存。章节要把“分配器优化”从“更快 malloc”提升到“生命周期和地址布局管理”。

TLB、页错误和 Linux 观察入口

TLB 是虚拟地址翻译的缓存。一个 4 KiB 页面能覆盖的连续数据很有限，大数组随机访问会让 TLB 工作集膨胀。若每次访问跨到新页，即使 cache line 只用少量字节，地址翻译也可能成为主成本。大页、批量访问、排序索引和分块都可以减少 TLB 压力，但它们影响不同层次：大页改变页粒度，分块改变访问顺序，排序索引改变算法输出顺序或需要额外恢复顺序。

页错误要分清 major 和 minor。Minor fault 可能只是建立页表映射或分配匿名页，不一定读磁盘；major fault 通常涉及 I/O。第一次触碰大数组时，初始化成本、页面分配和 NUMA first-touch 都会混入测量。若 benchmark 把分配和初始化放进热循环，总耗时就不能代表内核计算成本。测量章会系统讲报告格式，但缓存章必须提前提醒：内存相关实验要区分分配、触页、预热和核心循环。

Linux 观察可以从 `perfstat` 的 cache、TLB 和 page-fault 事件开始，再结合 `/proc/self/smaps`、`numastat` 和 `perfmem`。在手机或受限环境没有权限时，仍可通过输入规模阶梯、访问步长曲线和预热对照获取间接证据。重要的是把“没权限看计数器”写进报告，而不是假装已经证明了硬件事件。

把地址序列优化落回项目

贯穿项目在本章应完成三个版本。第一个版本保留对象数组，作为最直观 reference；第二个版本拆出热字段 batch，比较解析后计算阶段的字节移动和 cache 行为；第三个版本为 worker partial、队列槽位和统计指标加入 cache line 对齐，验证 false sharing 是否下降。每个版本都要保持输出一

致，不能为了局部性改变语义。

还要加入一个反例：若输入很小，小到完全落在 L1/L2 内，列式布局和对齐可能收益不明显，甚至因为代码复杂和额外索引而变慢。若热字段几乎总是和冷字段一起使用，拆分也可能增加访存次数。教材要训练读者判断适用范围，而不是把 SoA、padding 和大页当成固定答案。

本章最终判断力是：看到一个数据结构，能说出它会产生什么地址序列；看到一个地址序列，能推断可能的 cache、TLB、预取和一致性问题；看到一个优化建议，能设计对照实验证明它确实改变了地址序列或搬运成本。只会背缓存层级参数，还没有进入系统工程。

案例追查：同样是 vector，为什么速度差很多

构造两个版本的用户统计表。版本一是 `std::vector<UserRecord>`，每个 record 包含 id、name、debug string、last timestamp 和 counter。热循环只更新 counter。版本二把 counter 单独放成 `std::vector<std::uint32_t>`，冷字段留在另一组结构里。两者都使用 vector，但地址序列和有效字节比例完全不同。

实验要分三步。第一步只读 counter，比较带宽和 cache miss；第二步随机访问用户 id，观察 TLB 和缓存行为变化；第三步多线程更新相邻 counter，加入对齐 partial 版本比较 false sharing。这样读者能看到 vector 不是局部性的保证，字段冷热、访问顺序和共享写才是关键。

报告还要写内存占用。列式拆分减少热路径字节，但增加多个数组和索引；对齐 partial 降低 false sharing，但增加空间。高性能数据结构不是只追求最快一组数据，而是在速度、空间、接口和可维护性之间做可解释取舍。

缓存实验的输入规模阶梯

缓存实验必须跨多个输入规模。只测一个大数组，会把 L1、L2、LLC、DRAM 和 TLB 混在一起。建议准备按容量阶梯增长的数据：小到 L1 能容纳，中等到 L2，接近 LLC，超过 LLC，最后到内存。每个规模下比较连续扫描、固定步长、随机索引、链表追踪。曲线比单点数字更能说明层级边界。

步长实验要注意解释。步长为 1 时空间局部性最好；步长等于 cache line 内元素数时，每条 line 只用一个元素；步长跨页时，TLB 压力上升；随机索引会破坏预取。报告应写出每次访问有效字节比例，而不是只写速度下降。这样读者能把“cache line 是搬运单位”落到数字上。

链表实验要避免分配器偶然性。可以先随机打乱数组索引，用索引模拟链表，保证节点在一个连续数组中但访问顺序随机；再比较真正分配节点的链表。前者隔离预取和地址依赖，后者加入分配碎片和 TLB。两者差异能帮助读者区分不规则访问的不同来源。

Linux 内存观察的降级路径

理想情况下，读者可以用 `perfstat` 看 cache 和 TLB 事件，用 `numactl` 控制绑定，用 `/proc/self/smaps` 看映射，用 `pmap` 看地址空间。但手机或容器环境可能权限不足。教材要给降级路径：没有 PMU 事件时，用输入规模曲线和步长曲线；没有 `numactl` 时，只做单节点 first-touch 说明；没有大页权限时，只解释设计和风险。

降级不是放弃严谨。报告要明确写“本实验未能直接读取 dTLB 事件，因此只能通过页数和耗时曲线间接支持假设”。这种表述比假装有硬件证据更专业。系统工程常在权限不足、生产环境不可扰动、硬件不可控的条件下工作，诚实写证据边界本身就是能力。

本章还建议读 [Documentation/admin-guide/mm](#) 下与 huge pages、transparent huge pages、NUMA balancing 相关的说明，再结合 `mm/` 目录中的 page fault 和 reclaim 入口做浅阅读。目标是知道内核中确实存在这些机制，而不是把它们当成用户态工具输出的黑盒。

综合练习：设计地址序列基准

本章综合题要求读者实现一个地址序列基准，而不是一个泛泛的内存 benchmark。基准提供四种访问器：连续访问、固定步长访问、随机索引访问、依赖式指针追踪。每种访问器都必须产生同样的 checksum，避免编译器删除循环。输入规模按 L1、L2、LLC、超过 LLC、远超内存缓存几个阶梯设置。报告不能只写“随机慢”，而要解释随机慢在哪里：空间局部性、预取、TLB、地址依赖分别扮演什么角色。

第二步把数据结构换成三个版本：对象数组、热字段列式数组、索引模拟链表。对象数组用于展示无效字段搬运，列式数组用于展示热路径变窄，索引链表用于分离地址依赖和分配碎片。若读者直接用真实链表，分配器碎片、节点大小和地址依赖混在一起，结论会不清楚。先用索引模拟，再加真实链表，才是逐步加因素。

第三步加入跨线程写。让多个线程更新相邻槽位，再比较紧凑槽位、cache line 填充槽位和线程本地槽位。这个实验把本章的 cache line 搬运和下一章的一致性流量连接起来。报告要同时写时间和内存占用，因为 padding 的收益来自空间换冲突减少。

这个综合题的评分标准不是谁跑得最快，而是谁能画出地址序列并解释曲线形状。若曲线出现异常，例如某个规模突然变快或变慢，要要求读者写候选原因：预取、页大小、CPU 频率、系统噪声、编译器优化、TLB 或分配器。不能解释异常的 benchmark 只是一组数字。

容易出错的边界：局部性优化的副作用

局部性优化最常见的副作用是空间换时间。Cache line 对齐可以减少 false sharing，但会扩大对象；列式布局可以减少热路径搬运，但会增加数组数量和索引；大页可以减少 TLB 压力，但可能增加内存碎片和配置复杂度；对象池可以减少分配开销，但可能推迟释放，抬高峰值内存。教材要把这些副作用和收益放在同一段里讲，否则读者会把所有布局技巧当成单向加速。

第二个副作用是接口复杂。AoS 对业务代码友好，SoA 对热循环友好。若把列式数组直接暴露给所有模块，业务代码会到处操作多个平行 vector，容易破坏不变量。更好的做法是提供批量视图和有限修改接口。这样热路径看见连续 span，控制面仍然通过清晰对象管理生命周期。布局优化不是让所有代码都变底层，而是让底层成本在合适边界内可见。

第三个副作用是调试难度。材料化中间对象多，内存大但容易检查；完全融合和列式化后，中间状态少，性能好但调试困难。贯穿项目可以保留 debug 模式：输出 batch 摘要、字段范围和错误索引。性能模式关闭这些检查，保留 checksum。这样读者会看到工程系统常常同时保留可诊断路径和高性能路径。

本章的源码索引也应包含 C++ 容器规则。`std::vector` 的 reallocation 失效，`std::deque` 的分段存储，`std::list` 的节点稳定和局部性差，`std::unordered_map` 的 rehash 和分配，这些都是数据布局的一部分。学习缓存不能只看硬件手册，也要看语言容器如何制造地址序列。

本章核心接口：MemoryShape

缓存章给项目留下 **MemoryShape** 描述。它记录对象大小、热字段字节数、冷字段字节数、连续数组数量、是否 cache line 对齐、是否可能 false sharing、估算工作集页数。每个核心数据结构都应能生成这个描述。描述不需要完美，但要迫使设计者写出数据如何占用内存。

后续布局优化和 NUMA 策略都依赖它。线程本地 partial 要声明是否按 cache line 隔离；batch 要声明热字段跨度；shuffle block 要声明字节数和页数。没有 MemoryShape，局部性讨论会退回感觉。

从缓存层级进入 NUMA 和一致性

本章解释了单个线程或单个核心附近的数据移动。下一章把多个核心放进同一张图：同一条 cache line 被谁读，谁写，页面归属哪个节点，远端访问经过哪条互连。很多 false sharing 和热点 atomic 问题，在本章看来只是 cache line 搬运，在下一章会变成一致性所有权迁移。很多大数组扫描，在本章看来是内存带宽，在下一章会变成内存控制器和 NUMA 放置。

项目推进时，先让数据结构具备可解释的 MemoryShape，再谈拓扑策略。没有对象大小、热字段字节、partial 布局和页数估算，NUMA 优化就无从下手。缓存章给的是局部地址地图，NUMA 章给的是多核心、多节点之间的交通规则。

3.7 项目落地

Compute Systems Engine 在本章至少要增加四类实验。第一，partial 对齐实验：比较紧凑 partial 和 `alignas(64)` partial，观察多线程写入扩展曲线。第二，访问模式实验：比较顺序数组、固定步长、随机索引和链表遍历。第三，页级实验：改变工作集跨越页数，记录 page faults 和可能的 dTLB 事件。第四，first-touch 实验：在支持 NUMA 的机器上比较主线程初始化和 worker 分块初始化。

这些实验都必须有 reference。访问模式版本必须返回同样的逻辑结果；partial 对齐版本必须只改变布局，不改变算法；first-touch 版本必须分离初始化时间和计算时间，同时也可以单独报告初始化成本。若实验一次改变多个变量，结论就不可靠。

cache line 处理器搬运的基本粒度通常不是一个字段，而是一条 cache line。读取一个 int 可能顺带搬入相邻字段；写一个小字段也可能让整条 line 进入独占状态。冷热字段混在一起，会让热循环被冷数据拖住。

空间局部性 连续数组之所以常快，不是因为名字叫数组，而是因为地址递增稳定，预取器能

工作，一条 line 能服务多个元素。链表或随机指针会让每个元素都可能变成一次独立等待。

时间局部性 时间局部性要求数据在被驱逐前再次使用。若工作集超过缓存容量，重复扫描也可能没有收益。实验要逐步扩大输入规模，找到 L1、L2、L3 和内存之间的拐点。

TLB 覆盖 TLB 缓存虚拟页到物理页的转换。访问很多页、每页只用少量数据，会把周期花在地址翻译上。大页、紧凑布局和分块访问都可以改善 TLB 覆盖，但它们各自有分配、碎片和权限成本。

false sharing 两个线程写不同变量，如果变量落在同一条 cache line，也会互相使对方缓存失效。这个问题不靠锁数量判断，而靠地址和写入频率判断。按 cache line 分隔计数器，是为了隔离一致性流量。

页缓存 文件 I/O 经常先进入 page cache。第一次读取可能包含缺页和磁盘成本，第二次读取可能只是内存访问。benchmark 若不区分冷读和热读，就会把 I/O、缺页和解析混成一个数字。

mmap 边界 mmap 让文件像内存一样访问，但不是把 I/O 成本变没了。缺页发生在访问时，错误也可能在访问时暴露。教材应强调 mmap 的生命周期、页粒度和故障处理，而不是把它当作魔法优化。

C++ 表达 地址序列应进入接口。只读连续输入用 `std::span`，热字段拆成列式数组，跨线程计数器显式对齐，索引替代指针链。接口让访问模式可见，后续 SIMD 和并行章节才能复用。

案例：数组、指针数组、链表 三种结构处理同样数量元素，比较每元素耗时和 cache/TLB 现象。报告必须解释地址递增、随机跳转和节点分配对预取的影响。

案例：冷热字段拆分 结构体里放 `hot_value`、`status`、`debug_text`、`timestamp`。扫描 `hot_value` 时先用 AoS，再改成 SoA，记录搬运字节和收益边界。

案例：两线程计数器 让两个线程写相邻计数器，再加 padding 分隔。这个实验把 false sharing 从概念变成可观察现象。

源码阅读路线 Linux 源码阅读从 `mm/filemap.c`、`mm/memory.c`、`mm/mmap.c` 附近进入即可。阅读目标不是把整套子系统抄进书里，而是追一条和本章实验相关的路径：用户态动作怎样变成内核对象，内核对象怎样排队、等待、唤醒、计数、提交或回收。

阅读报告要写证据等级。能直接测到的数字放在第一层；从源码控制流推导出的解释放在第二层；当前设备无法验证、只能作为合理猜测的内容放在第三层。这样读者会知道地址序列报告中哪些结论可复现，哪些结论需要换机器继续确认。

源码阅读还要克制。看到一个复杂函数，不要把每个分支都展开；先回到本章的问题，问它能否解释一个等待点、一次状态转移或一条性能曲线。解释不了的内容可以留到索引，不要打断正文主线。

数组、指针数组、链表 三种结构处理同样数量元素，比较每元素耗时和 cache/TLB 现象。报告必须解释地址递增、随机跳转和节点分配对预取的影响。

冷热字段拆分 结构体里放 `hot_value`、`status`、`debug_text`、`timestamp`。扫描 `hot_value` 时先用 AoS，再改成 SoA，记录搬运字节和收益边界。

两线程计数器 让两个线程写相邻计数器，再加 padding 分隔。这个实验把 false sharing 从概念变成可观察现象。

cache line

硬件按 cache line 搬运数据，不按 C++ 字段搬运。读取一个热字段时，旁边冷字段也可能被带进缓存，浪费带宽。

实验设计：构造包含热计数、状态、调试文本和时间戳的结构体，对比 AoS 与 SoA 扫描。

工作集

小工作集停留在 cache 中，大工作集进入内存带宽和 TLB 压力。只说输入大小不够，要换算成访问字节和活跃页面数。

实验设计：从几 KiB 扫到数百 MiB，记录曲线拐点，并写出可能对应的层级。

步长访问

固定步长会改变 cache line 利用率和预取效果。步长越大，每次搬运的有效字节越少，TLB 覆盖也可能更早耗尽。

实验设计：对同一数组用步长一、八、六十四和页大小扫描，比较每元素时间。

TLB 覆盖

TLB 缓存虚拟页到物理页的翻译。页面数超过覆盖范围后，即使命中 cache 的数据不多，地址翻译也会成为等待来源。

实验设计：在相同字节量下改变页数量，观察小页、大页和随机页访问的差异。

页错误

minor fault、major fault 和写时复制 fault 语义不同。第一次触碰匿名页、读取文件页和 fork 后写共享页，成本不能混成一类。

实验设计：读取新映射文件、重复读取热文件和写入 fork 后页面，分别记录 fault 类型。

page cache

文件读取常先进入 page cache。第二次读取快，不一定是程序优化，可能是内核缓存。写入也可能先返回，稍后才真正落盘。

实验设计：比较冷读、热读、普通 read 和 mmap，报告明确是否测到存储设备。

mmap 生命周期

mmap 让文件内容像内存一样访问，但页面错误、SIGBUS、文件截断和映射生命周期都要处理。它不是免费数组。

实验设计：映射文件后模拟文件变化或输入缓冲释放，写出使用 mmap 的限制。

false sharing

两个线程写不同变量，如果变量落在同一 cache line，仍会争夺所有权。结构体字段不共享不等于硬件不共享。

实验设计：把每线程计数放在相邻元素和对齐槽位中，比较扩展曲线。

冷热分离

热字段进入连续数组，冷字段留在旁路结构，可以降低带宽和 cache 污染。代价是索引、生命周期和错误报告要重新设计。

实验设计：让解析阶段只输出热字段视图，错误报告通过稳定记录号回查冷字段。

源码观察

Linux 的页表和 page cache 路径很深，本章只追一条可解释实验的线：缺页怎样进入处理，文件页怎样查找，读写怎样触发缓存。

实验设计：阅读时只摘和实验现象相关的入口，不把整套内存管理搬进正文。

3.8 从地址序列而不是变量名理解内存系统

缓存和 TLB 章节不能只讲 L1、L2、L3 的层级表。读者真正需要学会的是把程序变成地址序列。源码说“遍历记录数组”，硬件看到的是一串 cache line；源码说“访问字段”，硬件搬运的是包含这个字段的一整段字节；源码说“读取文件”，内核可能先从 page cache 给出页，再由缺页路径建立映射。变量名和对象边界是给程序员看的，地址序列和页面边界才是硬件与内核看到的。

先看结构体扫描。日志记录里有事件类型、时间戳、用户 id、错误文本和调试字段。如果统计只需要事件类型，AoS 会把冷字段一起带进缓存；SoA 把热字段连成数组，扫描时每个 cache line 包含更多有效信息。这个优化不是简单地把结构体拆开，因为拆开后错误报告、生命周期和索引关系也要改变。若热字段数组里的第 1024 个元素出错，系统仍要能回到原始记录和冷字段。布局优化必须和语义账本一起写。

再看 TLB。很多教材会说 TLB 缓存页表项，但工程里要问的是：一次扫描触碰多少页面，访问顺序是否规律，页面大小和工作集如何影响覆盖。一个随机访问 64 MiB 数组的程序，可能不是数据本身太大，而是活跃页面数太多，地址翻译成为等待。实验可以固定总字节数，改变元素分布在页面上的方式：连续页面、每页一个元素、随机页。这样读者会发现“同样读 N 个元素”在机器上不是同一件事。

文件 I/O 也要放在同一套地址思维里。mmap 让文件像内存，但第一次访问可能触发缺页；文件被截断可能出现错误；映射区域的生命周期和文件描述符生命周期也要区分。普通 read 多一次缓冲管理，却让错误处理和背压更直接。不要告诉读者 mmap 永远更快，也不要告诉读者 read 永远安全。正确写法是按访问模式判断：顺序大读、随机小读、错误恢复、生命周期和 page cache 状态分别怎样影响选择。

page cache 是本章和 I/O 章的桥。第一次读文件可能慢在存储，第二次读同一文件可能快在内存；写入成功可能只是数据进入 page cache，真正持久化要看 fsync 和文件系统语义。性能报告如果不写冷缓存还是热缓存，读者无法判断优化是否有效。手机环境下可能没有权限清理 page cache，那就诚实写明限制，用不同文件或大于内存的数据集做替代，不要把热缓存结果当成磁盘性能。

本章实验可以做成四个小而清楚的系列。第一，AoS 与 SoA 扫描热字段，记录搬运字节估计和运行时间。第二，步长访问，从步长一到页大小，观察每元素成本。第三，TLB 覆盖，固定元素数但分散到更多页面。第四，mmap 与 read，对比首次读取、重复读取和错误处理代码复杂度。每个实验都应附一段地址序列解释，而不是只给图表。图表在 EPUB 里也许不方便，文字表和结论链更可靠。

Linux 源码阅读可从三个入口进入：缺页处理、filemap 查找页缓存、read/write 系统调用路径。读源码时不要求记住所有分支，而要回答实验里的问题：为什么第一次访问映射页会慢，为什么第二次读可能不碰磁盘，为什么写入返回和持久化不是一回事。这样缓存、TLB、虚拟内存和文件系统才会在读者脑中连成一条线。

案例推导：冷热字段拆分为什么必须同时处理错误报告

旧日志记录结构体把事件类型、时间戳、用户 id、原始行偏移、错误文本和调试标签放在一起。统计事件类型时，每个 worker 顺序扫描结构体数组，只读取事件类型和用户 id。profile 显示内存带宽压力很高。最直接的优化是把热字段拆成列式数组，让扫描只读取事件类型和用户 id。这个优化看似纯性能，但很快会碰到语义问题：当某个聚合结果异常时，如何回到原始行和错误文本？

如果拆分后丢掉原始记录身份，错误报告就会退化。旧结构体里字段在一起，调试方便；新布局里热字段和冷字段分离，必须用稳定 record id 或 offset 连接。LayoutPlan 要记录热字段列、冷字段表、原始输入位置和生命周期。热循环不搬冷字段，但报告路径仍能按 id 找到冷信息。这样才能同时满足性能和可诊断性。

实验可以设计两组。第一组只测扫描吞吐：AoS、SoA、冷热分离三种版本，输入包含长错误文本和短正常文本。第二组测错误路径：故意让某些记录解析失败，要求最终报告能输出原始偏移、错误原因和对应热字段。若 SoA 版本吞吐提高但错误报告丢失，它不是合格优化。教材要明确这一点，因为工程系统不能用“快但不可诊断”作为最终答案。

TLB 也会参与这个案例。冷热分离后，热字段连续，活跃页面减少；冷字段表虽然仍占内存，但不在热循环中触碰。若冷字段被随机回查，回查路径可能慢，但频率低。报告应写出热路径和冷路径的访问频率，而不是把所有路径平均。这样读者能理解为什么某些布局让主路径快，同时保留慢但少见的诊断路径。

源码阅读可从 page cache 和缺页路径连接到输入读取。若热字段来自 mmap 文件，第一次扫描触发缺页；若来自 read 后的解析批次，成本转移到读取和复制阶段。两种方案都可以成立，但生命周期不同。mmap 版本需要处理文件截断和映射有效期；read 版本需要管理缓冲复用和背压。缓存章节的深度就在于：每个地址序列背后都有所有权和恢复边界。

实验：比较行优先访问矩阵和列优先访问矩阵。记录 cache-misses、dTLB 相关事件和时间。再把矩阵规模从 L1、L2、L3 可容纳范围逐步扩大，观察拐点。

地址序列比变量名更接近真相 缓存和 TLB 的学习要从地址序列开始，而不是从 cache line、set、way 的名词开始。贯穿项目里，读取日志文件、解析记录、更新哈希表、写出 partial block，都会产生不同地址序列。顺序扫描输入接近线性流，硬件预取容易工作；更新哈希表会根据 key 跳到不同桶，随机性更强；写输出 buffer 若按 partition 分散写，可能在多个区域之间来回切换；读取 mmap 文件会把缺页、页缓存和用户态指针连在一起。变量名叫 input 或 map 不重要，真正影响性能的是 CPU 看到的地址怎样变化。

第一类地址序列是流式访问。它的特点是步长稳定、局部性好、可预取。流式读取最容易把瓶颈推到内存带宽或 I/O，而不是指令执行。第二类是小工作集反复访问，例如固定大小的计数数组或字符分类表。它适合待在 L1 或 L2。第三类是大工作集随机访问，例如高基数哈希表。它会暴露 cache miss、TLB miss 和分配器碎片。第四类是冷热字段混合访问，例如每条记录的热字段只需要事件类型和数值，冷字段却包含长字符串。若结构体把冷热字段绑在一起，缓存会被不需要的数据占满。

页缓存和 mmap 要从失败语义讲清 很多教材把 mmap 写成“少一次拷贝”，这太粗。mmap

把文件页映射到进程地址空间，访问时可能触发缺页，由内核把页从页缓存或磁盘带入。它让代码像访问内存一样访问文件，但错误语义、生命周期和页级行为都变复杂。文件被截断后访问映射区域可能出错；随机访问大文件可能造成大量缺页；映射生命周期和文件描述符生命周期要区分；预读策略也会影响吞吐。

普通 read 路径更显式：系统调用把数据从内核缓冲拷到用户缓冲，用户代码处理这块 buffer。它有拷贝成本，但边界清楚，错误点清楚，buffer 生命周期由程序掌控。mmap 适合某些随机读取、共享映射或把文件当地址空间处理的场景；read 适合顺序流、明确 buffer 管理和错误控制。项目不应直接宣布哪种更快，而应按输入大小、访问模式、页缓存状态和错误处理实验。

TLB miss 为什么会突然出现 TLB 缓存虚拟页到物理页的翻译。若工作集跨越太多页面，或者访问模式在大范围随机跳页，TLB 容量会成为瓶颈。哈希表、图邻接表、对象池和多个大数组交错访问都可能增加 TLB 压力。把记录按块处理、压缩热字段、使用连续数组、减少指针对象数量，都可以降低页级随机性。

大页不是万能答案。它能减少页表项数量和 TLB 压力，但会影响内存分配、碎片、NUMA 放置和权限粒度。若工作集本来很小，大页没有意义；若访问只碰每页少量数据，大页可能浪费内存；若需要频繁映射和释放，大页管理成本也要考虑。本章应要求读者先用页级访问统计描述问题，再讨论大页。

实验：同一算法的三种内存形状 可以设计一个 key 计数实验。版本一使用 `std::unordered_map` 直接更新，观察随机桶访问和分配成本。版本二先把记录分块，每块使用局部 map，再合并，观察工作集是否变小。版本三若 key 范围可压缩，把 key 映射成紧凑整数，使用连续计数数组，观察 cache 和 TLB 行为。三个版本输出必须相同，但地址序列完全不同。

报告要写清 key 分布：低基数、高基数、Zipf 热点、随机字符串。低基数下连续数组可能极快；高基数字符串下哈希和分配可能主导；热点分布下局部 map 能减少共享写；随机字符串下字符串比较和内存分配可能超过哈希本身。这样读者会明白数据结构选择不是按名称决定，而是按地址序列、工作集和语义决定。

Linux 源码阅读连接点 读 Linux 页缓存时，重点看文件页如何被标识、查找、标记脏、回写和失效。页缓存不是一个魔法加速层，而是一套以页为单位的共享状态机。读虚拟内存相关代码时，关注 VMA、页表、缺页处理和权限。项目中的 manifest 和 block cache 可以借鉴这种思路：每个缓存对象必须有身份、状态、引用、失效规则和回写规则。

源码阅读还要注意快慢路径分离。缺页是慢路径，命中页缓存是快路径；写回是异步过程，fsync 是同步边界；内存回收可能在看似普通分配时触发。把这些现象带回项目，读者就会理解为什么一次 vector push、一次文件读取或一次哈希插入的延迟会有长尾。系统性能不是平均成本的线性相加，慢路径会决定尾延迟。

事故复盘：同样复杂度为什么跑出不同阶梯 两个版本都说自己是线性复杂度：一个把记录解析成节点对象后放进 unordered map，另一个把热字段压成连续数组并用索引关联冷字段。小输入时差异不明显，输入扩大以后，节点版时间突然出现阶梯，perf 显示 cache miss、TLB miss 和缺页都参与进来。复杂度没有说谎，只是它没有描述地址序列。CPU 看到的不是“大 O”，而是一串加载地址、页号和 cache line。

分析要把访问路径画出来。节点对象让 key、value、错误信息和 next 指针散在堆上，哈希桶跳转又引入随机访问；连续数组让扫描阶段按 cache line 前进，但可能需要额外索引回到原始文本；mmap 首次访问可能触发缺页，read 可能多一次拷贝却让预取和页缓存行为更稳定。大页能减少 TLB 压力，也可能浪费内存并扩大无用搬运。每个方案都不是口号，而是一组地址行为。

实验不应只给一组总时间。要逐步扩大工作集，记录冷运行和热运行，区分页缓存、缺页、TLB 覆盖和数据缓存容量的拐点。哈希表实验还要报告 key 分布、装载因子、冲突长度和节点分配策略。若一次优化让扫描更快却让错误定位丢失，就不是合格优化。缓存章节真正要建立的能力，是从程序对象推导地址序列，再从地址序列解释硬件和操作系统成本。

地址实验实验判读 缓存实验要看拐点，不只看最终时间。输入逐步扩大时，若时间在某个规模突然上升，要回到工作集、页数和访问步长；冷热运行差异大，说明页缓存或缺页参与；顺序和随机差异大，说明预取和 TLB 覆盖不同。报告中应写清每个版本的地址形状，而不是只写容器名。

这一段也服务于复习。读者合上书后，应该能凭本章的判读口径重新审查自己的代码：有没有最小正确版本，有没有能击穿旧方案的输入，有没有状态记录，有没有资源指标，有没有失败后的

恢复动作。若五个问题缺两个以上，就说明实现还停在演示层，而不是工程层。

页缓存实验必须区分冷、热和污染 文件读取实验至少要跑三种状态。冷状态尽量减少页缓存影响，用来观察缺页、磁盘和预读；热状态重复读取同一文件，用来观察页缓存命中后的解析与内存路径；污染状态在两次运行之间读一个更大的无关文件，用来观察缓存被挤出后的行为。三种状态都不是完美真相，但合在一起能防止把页缓存命中误写成算法优化。

mmap 和 read 的比较也要写错误语义。mmap 让访问看起来像普通内存，但缺页发生在 load 指令附近，文件截断和映射生命周期会带来额外边界；read 把系统调用和用户缓冲区分开，拷贝成本更显式，也更容易控制 buffer 复用。若只比较一次运行时间，读者会错过最重要的生命周期差异。

地址实验的报告建议附一张文本表：访问形状、工作集大小、页面数量、是否顺序、是否随机、是否触碰冷字段、是否写回。这个表比容器名称更重要。unordered map、vector、mmap 都只是接口，真正决定硬件行为的是地址流。

实验课：同一个计数任务的四种地址形状 输入仍然是日志事件计数。第一版每条记录分配一个节点对象，节点里保存字符串 key，然后更新 `std::unordered_map`。第二版先把事件名 intern 成整数 id，再更新连续计数数组。第三版按 split 建局部 map，最后合并。第四版把热字段列式化，聚合阶段只扫描 event id 列。四个版本输出必须和串行 reference 一致。

实验不要一上来跑最大输入。先按工作集阶梯扩大规模：能放入 L1 的小数据，能放入 L2 的中等数据，超过 LLC 的大数据，再到跨越大量页面的超大数据。每个规模记录时间、页错误、缓存相关事件或替代指标、内存峰值和输出 checksum。权限不足时，也要记录冷运行与热运行差异、输入规模拐点和访问步长曲线。

判读时重点看地址形状。节点对象版本可能在小输入上够快，但规模增大后随机指针和字符串比较会暴露；整数 id 数组版本可能极快，但要求有字典编码；局部 map 降低共享写，却增加合并成本；列式化减少热路径字节，却要保留冷字段回查。没有一个版本天然最好，选择取决于 key 分布、错误报告需求和内存预算。

再把文件读取加入实验。用 read 版本和 mmap 版本分别喂同样的解析器，跑冷、热、污染三种状态。报告写清哪部分时间来自读取、哪部分来自解析、哪部分来自页缓存状态。这样读者会看到：虚拟内存、页缓存和容器布局共同决定实际成本，复杂度记号只能提供很粗的上界。

地址序列练习验收重点 合格答案要把容器名翻译成地址序列。vector 是连续还是分段，unordered map 是否节点分配，mmap 是否会触发缺页，read 是否有用户缓冲复用。答案要区分冷运行和热运行，并写出如果没有硬件计数器时用什么替代证据。

答案还要写“为什么不采用更复杂方案”。例如没有证据证明队列是瓶颈，就不应把锁队列换成无锁；没有证明输出顺序可变，就不应随意重排；没有证明错误路径能恢复，就不应只提交正常路径。能解释不做什么，说明读者开始具备工程判断。

最后，答案必须包含可复查材料。至少有输入摘要、关键状态、期望输出、实际输出、运行命令和未验证边界。没有这些材料，老师或同伴无法判断结论是否成立。第二册的作业不是让读者写漂亮段落，而是训练读者把系统问题变成可验证证据。

项目实现：缓存、TLB 与页级性能 本章对应的贯穿项目实现不要求一次写成完整框架，但必须有最小可运行切片。这个切片包含三个部分：一个 reference，一个带本章机制的实现，一个能击穿 reference 之外隐含假设的测试。Reference 用来固定语义，机制实现用来展示本章知识，测试用来证明新增复杂度确实解决了问题。

代码组织上，不要把所有内容放进一个大文件。基础类型、输入生成、核心机制、报告输出和测试应分开。即使当前只是教学项目，也要让目录能表达责任边界。这样后续章节接入时，不会因为找不到状态归属而重新发明一套对象。

每个实现都要输出一段机器可读摘要。摘要至少包含输入规模、关键状态、错误数、输出校验、耗时和未验证边界。这里的机器可读不要求复杂格式，简单的 key value 文本也可以。关键是让脚本和读者都能复查，而不是只能看屏幕截图。

测试应包含正常路径、边界路径和失败路径。正常路径证明功能可用；边界路径证明长度、容量、尾部、空输入、关闭或超时没有遗漏；失败路径证明本章机制不是装饰。若失败路径写不出来，往往说明本章概念还没有落到工程对象上。

报告最后写一个取舍段。新增机制带来了什么好处，增加了什么成本，哪些场景不应该使用它，下一章会复用哪个字段或对象。这个取舍段非常重要，因为系统工程不是把所有高级机制都加进去，

而是在证据和约束下选择合适的边界。

调试笔记：缓存问题先画访问步长 怀疑 cache 或 TLB 时，先画访问步长和工作集，不要先换容器。顺序步长、固定大步长、随机桶访问、指针追踪和冷热字段混读，对硬件完全不同。若工作集从小到大出现台阶，就把台阶和缓存层级或页数量联系起来；若冷运行慢、热运行快，就把页缓存和缺页写入报告；若随机访问慢，就检查是否能排序、分桶或压缩索引。

复查清单：缓存、TLB 与页级性能 复查本章时，先检查 reference 是否仍然存在。没有 reference，后面的并发、优化、异步或分布式改造都无法证明语义没有变。再检查压力输入是否尖锐：它应当攻击本章最核心的边界，而不是只把数据量放大。第三，检查状态是否可观察：关键对象的身份、状态、错误和提交结果应能在报告中找到。

第四，检查代码示例是否说明禁止行为。调用者不能做什么，往往比能做什么更能体现工程质量。第五，检查实验是否写出未验证边界；当前设备、权限或输入无法证明的结论，必须标注。第六，检查本章对象是否能被下一章复用，不能复用就说明边界还不稳。

完成这六项后，再看文字是否连贯。每个概念都应从问题推出，每个案例都应从朴素方案走到改进方案，每个实验都应能解释一个判断。若某段话放到其他章节也成立，就把它改成本章的具体对象、具体输入和具体失败。

3.9 工程案例：用地址序列解释缓存、TLB 和页缓存

缓存、TLB 和虚拟内存不是三块互不相干的知识。它们都在回答同一个问题：程序给机器的地址序列是什么形状，硬件和操作系统能否用有限的近处存储承接这个序列。若地址连续、重复利用高，缓存和预取器会帮忙；若地址随机、工作集跨越太多页，核心会频繁等待；若文件访问没有和页缓存语义对齐，I/O 和内存会互相污染。

一、从地址序列开始，而不是从缓存名词开始 同样是 $O(n)$ ，顺序扫描数组和追踪链表完全不同。数组扫描每次访问下一个元素，cache line 被充分利用，硬件预取器容易预测；链表追踪每次 load 下一个指针，下一步地址依赖当前 load 结果，预取困难，乱序窗口也难以产生足够独立 miss。算法复杂度没有错，但它没有描述地址序列。

因此本章实验要记录访问模式。连续、固定步长、随机索引、热点加长尾、二维行优先、二维列优先、分块访问，这些序列要用同一套输入规模比较。报告不要只写耗时，还要写元素大小、总字节数、访问页数、步长、是否重复使用、是否修改。这样缓存讨论才不会停留在“局部性好”这种口号。

二、cache line 是数据搬运单位 处理器通常按 cache line 搬运数据，而不是按单个字段。若热循环只用结构体中的两个小字段，却每次把包含字符串、调试信息和冷状态的大结构体一起搬进 cache，就会浪费带宽。数据布局章会继续讲 AoS 和 SoA，但本章先解释为什么浪费发生：硬件无法只搬你逻辑上关心的字段，它按连续字节块搬。

false sharing 也是同一件事的并发版本。两个线程写不同变量，如果变量落在同一 cache line 上，硬件仍要在核心之间迁移这条 line。源码看起来没有共享同一个字段，cache line 层面却共享了搬运和所有权。实验可以让多个线程分别写相邻计数器，再加入 padding 或按线程分块，观察扩展曲线变化。

三、TLB 把虚拟页转换变成热点资源 虚拟地址访问需要转换到物理地址。TLB 缓存最近使用的页表翻译。若工作集跨越的页太多，TLB miss 会引发 page walk，核心即使 cache line 在内存中也要先解决地址翻译。随机访问大数组、稀疏矩阵、哈希表和大图遍历都容易遇到这个问题。它们慢的不只是 cache miss，还有页级覆盖不足。

实验可以固定访问字节数，改变元素步长和页数。比如每页访问一个 64 位整数，与连续扫描同样数量元素比较。前者浪费 cache line，也打爆 TLB 覆盖；后者充分利用每页和每条 line。若系统支持 huge page，可以比较普通页和大页，但报告要写明大页分配是否成功。不要把“开启大页”写成普遍建议，它会影响内存碎片、NUMA 放置和启动成本。

四、虚拟内存让首次触页成为实验变量 分配大数组不等于物理内存已经就位。很多系统采用按需分配，第一次写某页时才真正建立物理页映射。若 benchmark 把首次触页混在热路径里，测到的可能是 page fault 和清零成本，而不是算法本身。多线程程序还会受到 first-touch 策略影响：哪个线程首次写页，页就可能放到哪个 NUMA 节点。

因此实验要分清初始化、预热和测量。先分配，再按计划触页，再进入热循环。若研究冷启动，就明确把首次触页作为被测对象；若研究内核性能，就把首次触页排除。很多看似玄学的性能波动，

来自没有控制页状态、缓存状态和 CPU 频率状态。

五、mmap 和 page cache 不是普通内存的简单别名 `mmap` 让文件内容映射到虚拟地址空间，读写看起来像内存访问，但背后仍有 page fault、page cache、脏页回写和文件系统错误。顺序读取大文件时，`read` 和 `mmap` 各有适用边界；随机小访问时，`mmap` 可能简化代码；写入持久数据时，何时 `msync` 或 `fsync` 又会进入语义。

日志统计项目可以用 `mmap` 做输入读取实验，但要写清：映射成功不代表所有页面已经在内存中；访问坏页或文件截断可能产生错误；page cache 命中和冷读差距很大。I/O 章节会继续讲可靠写入，本章只强调 page cache 是内存层级和文件系统之间的桥，不能当成普通数组完全忽略。

六、C++ 数据结构如何暴露地址形状 `std::vector` 暴露连续地址，适合扫描和 SIMD；`std::deque` 分段连续，适合两端操作但整体扫描局部性不如 `vector`；链表节点分散，插入删除稳定但遍历慢；哈希表平均查找快，却常有随机访问和分支。选择容器时，不能只背复杂度，还要看访问频率和地址序列。

项目中的 `parsed batch` 应优先使用连续列式数组保存热字段。原始字符串和错误文本可以放在冷存储，通过偏移关联。这样解析、统计和分区阶段看到的是连续数值列；诊断阶段需要时再追冷数据。这个设计会在第 6 章展开，本章先从 cache line 和 TLB 解释它的必要性。

七、实验交付：AccessTrace 本章可以给项目留下 `AccessTrace` 报告。它记录数据结构类型、元素大小、元素数量、访问顺序、总字节数、页数、是否写入、是否预热、是否使用 huge page、是否 `mmap`。每个实验输出 checksum，防止编译器优化掉访问。报告还要记录无法观测的指标，例如没有权限读取 dTLB miss 时，明确标记 unavailable。

验收时选择三组对照：`vector` 顺序扫描对链表遍历，AoS 热字段扫描对 SoA 热字段扫描，普通页随机访问对大页随机访问。每组都要求读者先预测地址序列，再运行实验。若结果和预测不同，回到编译器优化、缓存状态、页大小和硬件预取器解释差异。这样缓存章节就会变成可操作的分析方法。

3.10 源码阅读：从 Linux 页表和页缓存看内存成本

缓存和 TLB 的实验解释了现象，但还需要读者知道操作系统在背后维护哪些状态。源码阅读不要求把 Linux 内存子系统读完，而是选几条和项目直接相关的路径：缺页如何建立映射，页缓存如何承接文件读取，脏页如何回写，内存映射如何影响错误处理。这样读者会知道“内存访问慢”有时不是硬件 cache alone，而是硬件和内核状态共同作用。

一、缺页路径回答首次触页为什么贵 用户程序访问尚未映射的虚拟页时，会触发 page fault。内核要检查 VMA，判断权限，分配或找到物理页，建立页表项，可能清零页面，最后返回用户态重试指令。这个过程远比普通 load 昂贵。大数组第一次写入时，很多周期花在这里，而不是花在算法本身。

阅读时关注概念，不背函数名。VMA 描述一段虚拟地址的权限和来源；页表项描述虚拟页到物理页的映射；匿名页和文件页处理不同；写时复制会让读共享和写私有分开。项目报告中如果把首次触页计入热循环，就应该明确说明；如果排除，就要预触页。读者要能把 page fault 这个系统事件和 benchmark 曲线对应起来。

二、页缓存解释文件读为什么像内存又不像内存 文件读取常先进入 page cache。第二次读取同一文件可能很快，因为数据已经在内存；第一次读取可能受磁盘和 readahead 影响；内存压力下，页缓存会被回收。`mmap` 访问文件页时，第一次访问也可能触发缺页，把文件页带入 page cache。于是同一段代码在冷 cache 和热 cache 下差距巨大。

实验报告必须区分冷读和热读。若要测解析器，应尽量把输入预热或使用内存 buffer；若要测 I/O pipeline，就明确测冷读、热读和写回。不要把 page cache 命中后的速度当成磁盘吞吐，也不要把冷读瓶颈误判成解析器慢。页缓存让文件和内存连接起来，也让测量更需要边界。

三、mmap 的错误模型要进设计 `mmap` 简化了读取接口，但错误不一定在映射时出现。文件被截断、页读取失败、访问越界，都可能在后续内存访问中以信号或错误表现出来。普通 C++ 异常无法自然捕获这些情况。若项目使用 `mmap` 读取输入，就要在设计中说明文件生命周期、映射范围、并发修改和错误处理策略。

对教学项目来说，使用普通 `read` 加 buffer 可能更容易处理错误；使用 `mmap` 可以作为高级实验，重点观察 page fault、readahead 和地址序列。不要为了显得高级强行 `mmap` 所有输入。接口选

择应服务语义和证据，而不是服务风格。

四、脏页和持久化连接 I/O 章节 写入文件后，数据可能只在 page cache 中变脏，尚未到设备。内核会异步回写，也可以通过 `fsync` 等接口要求持久化。脏页太多会触发回写压力，写入线程可能被迫等待。这个机制解释了为什么写出阶段有时一开始很快，随后突然变慢：系统先吸收脏页，后来必须回写。

日志项目的 shuffle block 如果只写临时文件不 `fsync`，在进程崩溃和机器掉电下语义不同。教学中可以先处理进程崩溃，不承诺掉电持久化；如果要承诺掉电恢复，就要把 `fsync` 策略写清楚。内存章先埋下这个边界，I/O 章再详细处理。

五、内存压力会改变所有结论 当工作集接近或超过物理内存，系统开始回收页缓存、换出匿名页或触发更频繁的缺页。此时一个本来只受 cache miss 限制的程序，会变成 I/O 和内存回收问题。Benchmark 如果在不同内存压力下运行，结果可能完全不同。报告中至少要记录可用内存、输入大小、是否使用 swap、是否有其他进程占用内存。

Compute Systems Engine 的 block size、batch size 和队列容量都影响内存压力。队列太大，page cache 被挤出，后续读文件变慢；batch 太大，TLB 和 cache 局部性下降；保留太多旧配置或 retired node，内存峰值上升。内存不是无限背景资源，而是所有章节共享的预算。

六、源码阅读交付 本章的源码阅读交付不是函数调用图，而是一张“内存事件到系统影响”的表。page fault 对应首次触页和 mmap 冷读；TLB miss 对应页覆盖不足；dirty page 对应写回压力；page cache hit 对应热读；reclaim 对应内存压力；COW 对应 fork 或共享映射写入。每一项写一个项目中可能出现的位置和一个观测方法。

这张表会在后续章节复用。线程池队列积压可能导致内存压力，I/O 写出可能制造脏页，分布式模拟的 shuffle block 可能挤掉输入 page cache。读者若能把这些联系起来，就不会把内存问题局限在“cache 大小”层面。

3.11 补强案例：哈希表查询为什么经常慢在内存

哈希表平均复杂度是常数，但系统性能常被哈希表拖慢。原因是一次查询可能包含哈希计算、bucket 访问、控制字节读取、key 比较、节点指针追踪和 value 读取。每一步都可能触发 cache miss 或分支失败。复杂度告诉你元素数量增长趋势，却不告诉你一次查询要等多少个随机地址。

一、节点式哈希表的地址形状 节点式哈希表通常 bucket 数组连续，但节点分散分配。查找先访问 bucket，再追节点指针，再比较 key。若 key 是字符串，还要访问字符串内容。这个地址序列很难被预取器预测，也不容易被乱序执行隐藏，因为下一步地址依赖当前节点内容。工作集大时，LLC miss 和 TLB miss 会同时出现。

开放寻址哈希表把控制字节和元素放得更紧凑，减少指针追踪，但会受到负载因子、探测长度和删除标记影响。它不是永远更好，但在读多写少、key 可移动、需要高局部性的场景常有优势。教材不必实现完整高性能哈希表，但要让读者理解节点分配和局部性的关系。

二、字典编码改变问题形状 日志统计中，很多字符串 key 可以先字典编码成整数。编码阶段可能使用哈希表，但后续聚合变成数组或紧凑整数哈希，地址形状大幅改善。若同一批数据要多次查询或聚合，编码成本可以被摊销；若只处理一次且 key 很多，编码未必划算。选择取决于复用次数和 key 分布。

字典编码还带来语义问题。编码表的生命周期多长，是否跨 batch 复用，是否需要稳定 id，错误 key 如何处理，输出时如何反查字符串。性能优化一旦改变数据表示，就必须同时定义这些语义。否则后面分布式阶段会出现不同 worker 编码不一致的问题。

三、热点和长尾 若 key 分布高度倾斜，热点 key 会频繁命中同一 bucket 或同一计数器。它可能提高 cache 命中，也可能造成共享写热点。读多场景下热点有利，写多并发场景下热点危险。报告不能只写 key 数量，还要写分布，例如最大 key 占比、前十 key 占比、长尾长度。

项目中可以先对热点 key 做本地特殊计数，再把长尾交给哈希表。这样减少哈希探测和共享写。这个策略连接 histogram、partition 和分布式热点 shard。一个局部哈希表问题，最终会变成系统负载均衡问题。

四、实验 比较四种统计方式：节点式 unordered map、排序后线性合并、字典编码加数组桶、热点 key 特化加长尾哈希。输入覆盖均匀字符串、少量热点、大量长尾和批次复用。报告记录每条

记录平均查询次数、分配次数、内存峰值、cache miss 替代指标和最终输出一致性。这样读者会看到复杂度、地址序列和业务分布怎样共同决定数据结构。

3.12 补充案例：工作集大小如何决定算法阶段

工作集是当前阶段频繁访问的数据集合。它可能是一个数组、一个哈希表、一个 block、一个 batch，也可能是任务队列和状态表。工作集能否放进 L1、L2、LLC 或内存，会直接改变算法选择。

一、分块的本质是控制工作集 矩阵乘分块的目的，是让一小块 A、B、C 在 cache 中重复使用。日志处理也能分块：每次处理一个 batch，把热字段、局部 histogram 和输出 buffer 控制在 cache 友好的大小。分块不是矩阵专用技巧，而是让工作集适合层级存储。

Batch 太小，调度和函数调用开销高；batch 太大，cache 和 TLB 压力上升，错误恢复范围变大。选择 batch size 要同时看缓存、I/O 和恢复。一个纯算法角度的最佳块大小，不一定是系统最佳块大小。

二、哈希表工作集随 key 基数变化 事件统计中，key 数少时局部表可以留在 cache；key 数大时，每次更新可能访问随机 bucket。若 key 基数随输入变化，性能曲线会出现拐点。报告要测不同 key 基数，而不是只测一个固定分布。

当局部表超过 cache，可以考虑分区处理：先按 key hash 把记录分成多个 partition，再分别聚合。这样增加一次数据重排，但每个 partition 的工作集变小。这个策略在单机和分布式 shuffle 中同源。

三、工作集和并发度互相影响 每个线程都有自己的局部工作集。线程数增加，总工作集可能超过 LLC；每线程 partial 增加，内存峰值也增加。一个单线程 cache 友好的算法，多线程后可能因为总工作集过大而变慢。报告要记录每线程工作集和总工作集。

四、实验 固定总输入，改变 batch size、key 基数和线程数。记录吞吐、cache miss 替代指标、TLB miss 可用性和内存峰值。让读者观察拐点，并解释它对应哪个缓存层级或内存预算。工作集分析能把“快慢变化”变成可解释现象。

3.13 从对象地址推导工作集模型

缓存、TLB 和虚拟内存最容易被讲成层次结构图：L1、L2、L3、内存、页表、磁盘。图有用，但还不够。工程中真正需要的是工作集模型：程序在一段时间内会反复访问哪些地址，这些地址落在多少 cache line、多少页面、多少 TLB 项里，访问顺序是否让硬件预取器看得懂，写入是否让多个核心争抢同一条 cache line。把对象名字换成地址序列以后，很多看似软件层面的选择都会变成硬件问题。

一、数组快不是因为数组高级，而是地址可预测 连续数组的优势来自两个事实：相邻元素通常位于相邻 cache line，下一次访问地址可以由当前地址加固定步长得到。硬件预取器擅长这种模式，TLB 也能用较少项覆盖较大的连续区域。链表慢也不是因为链表这个抽象低级，而是因为下一跳地址保存在当前节点里，处理器拿到当前节点前不知道下一个地址。乱序执行可以并行处理独立 miss，却很难提前追逐单条指针链。读者应把“数组和链表复杂度都是线性”继续细分为：数组的线性扫描是带宽问题，链表的线性扫描常常是延迟问题。

二、工作集大小决定局部性是否真实 一个算法声称有局部性，必须说清局部性窗口。若热数据只有几十 KiB，它可能长期留在 L1 或 L2；若热数据是几 MiB，就要看 L3 和内存带宽；若随机访问跨越数 GiB，TLB miss 和页表遍历会进入主路径。哈希表是很好的例子：平均复杂度看起来是 $O(1)$ ，但一次查询可能触发哈希计算、bucket 访问、节点访问和字符串比较，每一步都可能落在不同 cache line。若 key 很短、表负载低、bucket 连续，表现很好；若 key 很长、节点分散、rehash 频繁，常数会大到压过算法复杂度。

三、页不是透明的背景 虚拟内存让每个进程拥有连续地址空间，但硬件执行时仍要把虚拟地址翻译成物理地址。TLB 缓存翻译结果，页表保存完整映射。大数组顺序扫描时，TLB miss 相对可控；随机访问大表时，工作集可能超过 TLB 覆盖范围，导致翻译开销和数据 miss 叠加。大页能扩大 TLB 覆盖范围，但会影响内存分配、碎片、NUMA 放置和权限粒度。教学中不能把大页讲成“开了就快”，应把它放进地址序列和部署约束里分析。

四、布局优化要回答谁在同一条线 结构体布局的核心问题是哪些字段会一起读，哪些字段会

一起写，哪些字段属于冷路径。若热循环只读 `id` 和 `score`，却把很长的错误消息、调试字段和状态字符串放在同一结构体里，每次扫描都会搬运不需要的数据。把热字段拆成列式数组可以减少带宽，但会改变接口、构造成本和缓存行为。反过来，过度列式化也会让本来一起使用的字段分散。布局优化没有固定答案，只有访问模式、对象生命周期和接口稳定性的共同约束。

五、用实验建立地址直觉 本章实验建议把同一份数据做成三种形态：连续数组、索引数组和指针节点。每种形态都跑顺序访问、固定步长访问和随机访问，记录吞吐、cache miss、TLB miss 和内存峰值。然后再加入冷热字段拆分、arena 分配和大页配置。若结果和预期不一致，不要急着改代码，先检查编译器是否优化掉循环、数据规模是否超过缓存、线程是否被调度到不同核心、输入是否真的随机。地址直觉不是靠背层级图得到的，而是靠一次次把对象还原成可测的地址序列得到的。

3.14 贯穿案例：同样是线性扫描，为什么一个像顺流，一个像追尾

考虑一个日志聚合作业。版本一把每行解析成 `LogRecord` 对象，对象里有用户字符串、事件字符串、时间戳、错误信息和若干指针，随后把对象指针放进哈希表。版本二先把热字段抽出来：时间戳放进连续数组，事件名经过字典编码变成整数，错误信息只在需要时通过 `RecordId` 回查。两版都处理每条记录一次，复杂度都可以被写成线性，但机器执行完全不同。第一版像在堆上追一串路标，第二版像沿连续道路顺序前进。

这个案例的目的不是说对象模型不好，而是让读者学会问：热循环真正碰了哪些字节，冷字段是否被无意搬进 cache，指针跳转跨了多少页，TLB 能否覆盖当前工作集，硬件预取器能否猜到下一地址。缓存和 TLB 不应该是章节标题里的名词，而应该是代码设计时能被追问的具体成本。

一、从对象生命周期看热字段和冷字段 解析阶段需要原始字符串和错误上下文，聚合阶段通常只需要事件 `id`、时间窗口和数值。若两阶段共用同一个胖对象，聚合时会反复把冷字段所在的 cache line 带进来。冷热拆分的动机就从这里出现：不是为了追求列式布局这个名字，而是因为不同阶段需要不同字段。拆分以后，热路径更紧凑，冷数据仍可通过 `record id` 找回。若错误报告要求保留完整原文，冷数据不能丢，只是不能让它占据热循环带宽。

二、从一次哈希查询拆出地址序列 一次 `unordered_map` 查询表面是 $O(1)$ ，地址序列却可能包含多步：读取 `key` 字节计算哈希，访问 `bucket` 数组，跳到节点，比较字符串长度和内容，必要时继续链表或探测。每一步都可能落在不同 cache line。若 `key` 很多、节点分配分散、字符串不短，常数成本会非常高。相反，字典编码后用整数 `key` 查询紧凑数组，可能把多次随机访问压成一次连续或近似连续访问。复杂度符号没有错，但它隐藏了地址形状。

三、TLB 从“页数”而不是“元素数”进入模型 一百万个小对象如果分散在堆页上，热循环可能触碰大量页面；一百万个整数连续存放，页面数量仍然大，但访问顺序稳定，TLB miss 和预取行为更可控。TLB 的问题不是数据总量大这么简单，而是单位时间内活跃页面数量超过覆盖能力。Arena 分配、冷热拆分和紧凑数组都能减少活跃页面。大页也能扩大覆盖，但它不能修复乱跳的指针图，最多推迟拐点。

四、把布局优化写成可逆设计 高质量布局改造要保留语义回路。热数组里只有 `event id`，不代表系统失去事件名；字典表必须能从 `id` 反查字符串；错误路径必须能从 `record id` 找到原始行和 `offset`；checkpoint 必须保存字典版本。若布局优化让调试、恢复或输出变得不可解释，它就是把成本转移给后续章节。经典系统设计舒服的地方在于每个抽象都能回到证据，本书也要保持这一点。

五、实验要覆盖工作集拐点 这个案例的实验不应只测一个数据规模。应固定记录形状，逐步增加记录数和 `key` 基数，观察吞吐在哪些点下降。第一个拐点可能对应 L2 工作集，第二个可能对应 LLC，第三个可能对应 TLB 或内存带宽。然后改变布局：胖对象、冷热拆分、字典编码、排序合并。若拐点移动，说明布局确实改变了工作集；若不动，说明瓶颈可能在 I/O、分支或写出。实验目标是找成本迁移，而不是证明某容器永远快。

六、把地址报告服务后续并发 单线程地址序列报告不是孤立材料。后面加线程时，热数组是否按线程分片、`partial` 是否 `false sharing`、页面是否 `first touch`、NUMA 节点是否远端，都依赖这里的布局。若第三章把地址地图写清，后续并发优化就不是重新猜。比如每线程写自己的连续 `partial`，合并时按 `key` 顺序扫；或者每线程哈希表分散在堆上，合并时随机访问很多页。两种并行策略在代码层面都能工作，但地址地图会提前告诉我们风险。

3.15 案例深化：地址序列报告如何写

缓存和 TLB 的学习要落到地址序列报告。假设同样是统计 key 出现次数，版本一为每条记录分配节点并插入 `unordered_map`，版本二先把 key id 压成连续数组，再做局部排序和线性合并。两者在复杂度上都能被说成接近线性，但硬件看到的路径完全不同。一个追指针、跳桶、分配节点；另一个顺序扫描、局部排序、批量合并。报告若只写大 O，就看不到差异。

一、把对象图翻译成地址图 对象图描述程序员看到的关系：记录、字段、节点、桶、错误信息。地址图描述 CPU 看到的访问：连续数组、随机桶、堆节点、页边界、cache line。优化前后先画这两张图，读者能立刻看到热字段和冷字段是否混在一起，指针跳转是否跨页，扫描是否能被预取。地址图不是插图任务，而是性能推理工具。

二、分清冷运行和热运行 第一次读取文件可能触发缺页和页缓存填充，第二次运行更多是在内存里扫描。若不区分冷运行和热运行，`mmap`、`read`、预读和页缓存的影响会被混合。实验应至少记录第一次运行、重复运行、改变输入文件后的运行。若冷运行慢而热运行快，瓶颈可能在 I/O 和缺页；若热运行仍慢，才更可能是 cache、TLB 或算法访问模式。

三、TLB 问题经常由布局间接制造 TLB miss 不只来自数据大，也来自有效工作集跨越太多页面。节点对象分散在堆上，可能让一个热循环同时触碰很多页；冷热字段拆分后，热路径页数下降，TLB 覆盖就改善。大页可以扩大覆盖，但若程序仍然随机追指针，只是把问题推迟。先改善布局，再考虑大页，这个顺序通常更稳。

四、哈希表报告要写分布 哈希表性能不能只看平均查找。要报告 key 分布、装载因子、桶长度、rehash 次数和分配次数。热点 key 多时，局部聚合可能比直接全局 hash 更稳；key 空间小且可压缩时，数组或排序合并可能比 hash 更可预测；key 很稀疏时，hash 仍然合适。不同结构的选择来自数据形状，而不是来自“某容器快”。

五、把内存证据连接到后续章节 地址序列报告会被后面复用。SIMD 章节需要知道数据是否连续；线程章节需要知道多个 worker 是否写同一 cache line；NUMA 章节需要知道页面放在哪个节点；I/O 章节需要知道读取是顺序还是随机。第三章如果写清地址行为，后面的优化就有共同地图。

3.16 案例复盘：复杂度相同，地址序列不同

两个统计程序都可以声称是线性复杂度：一个把记录做成堆节点再进哈希表，一个把热字段压成连续数组后排序合并。小数据下差异不明显，数据一大，节点版出现 cache 和 TLB 拐点。原因不在大 O，而在 CPU 看到的地址序列：连续 cache line、随机桶、堆节点、页边界和缺页。

把对象图翻译成地址图 对象图说明记录、字段、桶和错误信息的关系；地址图说明它们落在哪些页和 cache line。节点对象把热字段和冷字段散在堆上，哈希桶引入随机跳转；连续数组让热路径容易预取，但需要 record id 回溯冷数据。优化不是“换容器”，而是改变硬件访问轨迹。

实验判据 报告要区分冷运行和热运行，记录 page fault、cache miss、TLB miss、分配次数和 key 分布。哈希表还要写装载因子和桶长度。若扫描变快但错误定位丢失，优化不合格，因为系统只是在性能和可恢复性之间做了未声明交换。

3.17 本章习题

习题与作业

习题一：在当前项目中为 partial 数组设计两个布局，一个紧凑，一个按 cache line 对齐。说明每个字段的大小、对齐、每个 cache line 可能容纳几个 partial，以及为什么这个设计能隔离 false sharing。

习题二：写三个遍历版本：连续数组遍历、随机索引数组遍历、链表遍历。要求三者处理相同数量的逻辑元素。报告中分别分析 cache line、TLB、预取器和内存级并行度，不允许只写“随机慢”。

习题三：设计一个首次触页实验。说明如何分离分配、初始化、预热和计算；如何记录 page-faults；在没有 NUMA 机器时，哪些结论不能验证，哪些结论仍然可以通过 page fault 观察。

习题四：阅读 `mm/memory.c`、`mm/mmap.c` 或对应内核版本中的 fault 路径入口。写一

份不超过两页的源码阅读报告，要求把用户态实验现象和内核路径联系起来，不要罗列无关函数。

NUMA、互连与缓存一致性

先看一个并行归约事故：单线程求和很稳定，四线程以后吞吐几乎不涨，八线程还变慢。代码里没有复杂锁，只是每个线程对同一个全局 atomic 计数器做 `fetch_add`。从 C++ 看，它已经没有数据竞争；从硬件看，所有核心在反复争夺同一条 cache line 的修改权。再换一个版本，每个线程写自己的 `partial`，最后合并，吞吐突然恢复。这里逼出的不是某个协议名，而是一个事实：共享读容易复制，共享写必须协调所有权。

本章从这个共享写热点进入 NUMA、互连和缓存一致性。多核心 CPU 不是把多个核心简单放在一起。核心之间通过片上互连通信，共享某些缓存层级，连接内存控制器，并用一致性协议维护共享内存语义。多 socket 机器还会引入 NUMA：不同核心访问不同内存节点的成本不同。只有把“哪个 cache line 被谁写、哪个页面属于哪个节点、合并阶段穿过哪条互连”说清楚，并程序的扩展曲线才有解释。

4.1 从共享内存到一致性流量

在 C++ 程序里，多个线程似乎可以直接读写同一地址。但硬件上，写一个共享 cache line 需要让其他核心的副本失效或更新。一个原子自增不是普通加法，它需要获得 cache line 独占权，并按内存序约束发布结果。争用越激烈，cache line 在核心之间来回迁移越频繁。

这解释了为什么全局计数器扩展性差。四个线程分别做本地计数，最后合并，通常比四个线程同时 `fetch_add` 同一个原子变量快。前者减少共享写，后者让每次更新都进入一致性协议热路径。

MESI 的直觉 很多一致性协议都可以用 MESI 直觉理解：Modified 表示本核心有被修改且尚未写回的独占副本，Exclusive 表示本核心有未修改的独占副本，Shared 表示多个核心可以读，Invalid 表示副本无效。当一个核心要写 Shared 行时，需要让其他核心副本失效；当其他核心随后要读，又要重新获取数据。

软件看不到这些状态名，但能看到它们的后果。一个只读共享表可以被多个核心同时缓存，扩展性很好；一个频繁更新的共享 head 指针会让 cache line 在核心之间跳来跳去；一个包含多个线程本地字段的结构体如果没有 padding，也会因为 false sharing 触发同样的迁移。

真实处理器可能使用 MESIF、MOESI 或其他变体，但教学时先抓住一个核心事实：读共享容易扩展，写共享需要取得所有权。Modified 或 Owned 这类状态的具体名称不重要，重要的是写入会触发状态迁移，状态迁移会占用互连和缓存层级资源。高性能并发设计的第一反应，不应该是“换一条更快的原子指令”，而应该是“能不能减少共享写”。

```
read mostly table:
core0 Shared
core1 Shared
core2 Shared

hot atomic counter:
core0 Modified -> core1 Modified -> core2 Modified -> core0 Modified
```

上面的文本图表达两种完全不同的共享。前者多个核心一起读，硬件可以复制；后者多个核心轮流写，所有权不断迁移。很多扩展性问题就藏在这条差异里。

一致性和内存模型的分工 缓存一致性解决的是“同一内存位置的多个副本如何保持一致”，内存模型解决的是“不同内存操作之间能被哪些线程以什么顺序观察”。硬件一致性不等于程序自动无数据竞争。没有同步的普通读写仍然可能在 C++ 层面形成未定义行为。

一致性协议通常围绕状态机工作，例如 Modified、Exclusive、Shared、Invalid 这类状态。软件不需要手写这些状态，但必须理解状态迁移的成本：共享读便宜，共享写贵，频繁写同一行最贵。

C++ 内存模型和硬件一致性的边界也要分清。硬件一致性通常保证同一 cache line 或同一地址的副本最终按协议协调，但 C++ 程序如果对普通变量并发读写而没有同步，语言层面就是数据竞争。语言规则先决定程序是否有定义，硬件规则再决定有定义的同步怎样落地。不要用“我的 CPU 有缓存一致性”来为未同步普通变量辩护。

Store buffer 和可见性 核心执行 store 后，写入可能先进入 store buffer。对本线程来说，后续 load 可能通过 store forwarding 看到自己的写；对其他核心来说，这个写何时可见还受缓存一致性和内存序约束影响。锁释放、release store 和内存屏障会把这种可见性变成可依赖的同步关系。

这就是为什么“我在写 flag 前已经写了 data”在并发中不够。编译器和硬件都可能在允许范围内重排或延迟可见性。正确发布对象需要用 release/acquire、mutex 或其他同步机制建立 happens-before。

Store buffer 还会影响锁的性能。释放锁时，持锁线程在临界区内的写入必须以满足同步语义的方式对后续拿锁线程可见。获取锁时，线程可能需要等待锁所在 cache line 的最新状态。低竞争时这条路径很短；高竞争时，锁变量所在 cache line 会成为所有核心争夺的热点。锁的成本不是一个固定常数，而是竞争、拓扑和写入路径共同决定的结果。

4.2 从页面位置到拓扑感知

NUMA 机器上，内存离某些核心更近。线程在节点 A 上运行，却频繁访问节点 B 分配的内存，会产生远端访问。Linux 的 first-touch 策略让首次触碰页面的线程影响页面放置，因此并行初始化和线程绑定会影响后续计算性能。

NUMA 优化的基本原则是让计算靠近数据。大数据按块分给线程时，初始化也应按相同块并行完成。跨节点共享队列和全局分配器要谨慎，因为它们可能把远端访问和同步争用叠加在一起。

NUMA 不只出现在多 socket 服务器上。有些桌面和移动平台也有非均匀的缓存、内存或芯片 let 拓扑，只是操作系统暴露方式不同。教材里使用 NUMA 这个词，是为了让读者形成拓扑意识：核心不是一个无差别集合，内存也不是一个无差别池。线程、数据和 I/O 设备之间的位置关系会影响成本。

First-touch 和并行初始化 Linux 常见 first-touch 策略意味着页面通常分配到第一次写入它的线程所在 NUMA 节点。如果主线程单线程初始化一个大数组，然后工作线程分布在多个节点上处理这个数组，很多工作线程可能访问远端内存。更好的方式是让每个工作线程初始化自己后续要处理的分块。

这条规则在 benchmark 中尤其重要。若初始化方式和计算方式不一致，测到的可能是内存放置错误，而不是算法本身。实验报告必须记录初始化策略、线程绑定和 NUMA 策略。

互连不是无限带宽 核心、缓存、内存控制器和 I/O 设备之间的互连带宽有限。一个程序即使单线程局部性很好，多线程扩展后也可能撞上内存带宽或互连带宽上限。此时继续增加线程，只会让每个线程分到更少带宽，吞吐不再提高。

互连瓶颈常常表现为“每个线程单独跑都快，一起跑就都慢”。原因可能是多个核心争用同一内存控制器，多个 socket 之间远端访问过多，或者共享写导致 cache line 在互连上来回移动。排查时要把线程数、绑定位置、数据放置和访问模式作为实验矩阵，而不是只改算法代码。

4.3 从拓扑记录到分层资源管理

拓扑感知不是过早优化，而是避免把硬件当作平面资源池。线程池可以按 NUMA 节点分组，内存池可以按节点分配，队列可以按生产者或消费者分片，归约可以先在 socket 内合并再跨 socket 合并。这样的层次化设计和分布式系统中的本地聚合很像：先减少近处流量，再把少量结果发到远处。

Linux 观察入口 在 Linux 上，拓扑可以从 `lscpu`、`/sys/devices/system/cpu/`、`/sys/devices/system/node/` 和 `numactl--hardware` 开始观察。线程绑定可以用 `taskset`、`numactl--physcpubind` 或程序内亲和性 API。内存策略可以用 `numactl--membind`、`numactl--interleave` 和 first-touch 实验观察。若权限允许，`perf` 可以帮助识别 cache-to-cache 流量，`perfstat` 的 cache、LLC、context-switch 和迁核指标也能提供间接证据。

源码阅读可以从调度拓扑、NUMA 策略和内存分配路径切入。不要试图一次看完整调度器或完整内存管理。更好的问题是：线程迁移时哪些状态会变化，first-touch 影响页面放置的路径在哪里，NUMA 策略怎样被记录和查询，某个 benchmark 的现象能不能从这些路径得到合理解释。

层次化归约案例 并行归约是理解拓扑的好例子。最差设计是所有线程对同一个原子变量累加。改进设计是每个线程写自己的 partial，最后一个线程合并。进一步的拓扑感知设计是每个核心或每个 worker 先本地累加，每个 NUMA 节点内再合并，最后跨节点合并。这样做减少共享写，也减少远端通信。

这个结构和分布式计算中的树形聚合完全一致。不同只是成本单位变化：单机里远处是另一个 NUMA 节点，分布式里远处是另一台机器。理解这条相似性，可以帮助读者把第二册前后内容串起来。

一致性流量的三个等级 共享内存程序里的数据共享可以分成三个等级。第一个等级是只读共享，例如配置表、只读索引、常量权重、不可变输入数组。多个核心可以同时缓存这些数据，扩展性通常很好。第二个等级是低频写共享，例如阶段结束时写一次完成标志、每秒汇总一次指标、偶尔替换一份配置快照。它需要同步，但不一定成为瓶颈。第三个等级是高频写共享，例如所有请求更新同一个原子计数器、所有 worker 争用同一个队列 tail、多个线程反复写同一 cache line 上的不同字段。这个等级最危险，往往表现为核心数越多越慢。

优化时要先判断共享等级，而不是先决定使用 mutex、atomic 还是无锁结构。mutex 保护低频复杂状态可能完全合理；atomic 保护高频热点计数器仍然可能很慢；无锁队列如果所有生产者都争用同一个 tail，也逃不开一致性流量。把共享写变成分片写、批量写、线程本地写或按 NUMA 节点写，通常比替换同步原语更根本。

这也是第二册区分“正确性同步”和“扩展性设计”的原因。正确性同步回答的是数据竞争、可见性和不变量；扩展性设计回答的是共享写频率、cache line 迁移和拓扑距离。一个程序可以同步正确但扩展性很差，也可以局部很快但同步语义错误。高质量系统必须同时满足二者。

目录协议和窥探协议的直觉 一致性协议在小规模系统中可以用广播窥探思想理解：一个核心想写某条 cache line，就向其他核心广播或通过互连通知，让其他副本失效。核心数量增加后，纯广播会变贵，因此很多系统使用目录式信息，记录某条 cache line 可能在哪些核心或节点有副本，再有针对性地发送消息。具体实现非常复杂，但软件层面的直觉很简单：共享范围越大，维护一致性的通信越难便宜。

这条直觉能解释为什么“全局共享结构”在小机器上看不出问题，在大机器上突然变差。四核心机器上，一个全局队列的 cache line 迁移还勉强能承受；多 socket 服务器上，迁移可能跨互连和 NUMA 节点，延迟和带宽成本都更高。工程设计不能只在开发机上看正确性测试通过，还要在目标拓扑上看扩展曲线。

目录和窥探的差别不需要读者背协议细节，但要形成层次化思维。让数据只在一个核心私有，成本最低；让数据在一个 socket 内共享，成本较低；让数据跨 socket 高频写共享，成本高；让数据跨机器共享，成本更高。线程本地 partial、socket 内合并、跨 socket 合并、跨机器 reduce，就是把这个成本阶梯显式写进算法。

内存控制器和带宽饱和 NUMA 节点通常连接自己的内存控制器。一个节点上的线程大量访问本地内存，会消耗本地控制器带宽；远端访问则需要经过互连到另一个节点的内存控制器。若所有线程都访问一个节点上的内存，即使 CPU 核心分布在多个节点上，带宽也可能集中压在一个控制器上。此时线程数增加，吞吐未必提高，因为瓶颈不是核心数量，而是某个内存节点的带宽。

first-touch 实验的价值就在这里。主线程初始化全部页面，可能让所有页面落在一个 NUMA 节点；worker 分布到多个节点计算时，远端访问集中出现。按 worker 分块初始化，可以让页面分散到各自节点，使多个内存控制器共同工作。若实验显示 parallel first-touch 版本带宽更高，就说明优化来自页面放置和控制器并行，而不是加法本身改变。

带宽饱和也有反直觉现象。使用更多线程可能让总吞吐略增，但每线程吞吐下降；使用 SMT 可能让内存延迟隐藏更好，也可能只增加争用；把线程平均分到两个节点可能提高总带宽，也可能因为跨节点合并阶段变慢。结论必须来自实验矩阵，不应写成固定经验。

拓扑感知队列 队列是互连成本的集中点。一个全局 MPMC 队列很容易成为共享写热点：所有生产者争用 tail，所有消费者争用 head 或槽位状态。低线程数下它简单好用，高线程数下它可能把所有 worker 绑在一组 cache line 上。拓扑感知设计通常会拆成多级队列：每个 worker 或每个 NUMA 节点有本地队列，只有负载不均时才访问其他队列。

线程池的工作窃取就是这个思想。worker 优先使用本地 deque，空闲时才偷别人的任务。NUMA 感知线程池还可以先在同节点 worker 中窃取，再跨节点窃取。这样做不是为了复杂而复杂，而是为

了让高频路径局部化、低频路径承担远端成本。

有界队列的背压也要考虑拓扑。若所有生产者把任务推到单个队列，队列满时所有生产者都等待同一个条件变量，唤醒和竞争集中；若每个分片有自己的队列，热点分片仍会背压，但无关分片可以继续执行。分片设计会增加任务路由和负载均衡复杂度，因此要先测出全局队列确实是瓶颈，再拆分。

Linux NUMA 策略和迁移 Linux 提供多种 NUMA 相关策略：默认 first-touch、按节点绑定、交错分配、自动 NUMA balancing 等。自动机制可以在一般场景改善局部性，但它不是性能实验的替代品。自动迁移页面本身有成本，统计窗口和迁移决策也可能与 benchmark 生命周期不匹配。短时间 benchmark 尤其容易受到初始化策略影响。

用户态实验应记录三类信息。第一，线程位置：线程是否绑定，绑定到哪些 CPU，是否发生迁移。第二，页面位置：是否通过 first-touch、mbind 或 interleaved 控制，是否能观测各节点内存使用。第三，访问形状：线程是否主要访问本地分块，是否有跨节点共享写，合并阶段是否集中到一个节点。只记录 `numactl--hardware` 不够，必须把拓扑和程序行为联系起来。

源码阅读可以围绕 `mm/mempolicy.c`、NUMA balancing 相关路径和调度拓扑展开。不要试图一次读懂所有内存管理。合格阅读报告应选择一个问题，例如“为什么主线程初始化会影响页面节点”，然后追踪 VMA 策略、缺页处理和页面分配之间的关系。

C++ 接口如何表达拓扑 C++ 标准库不直接暴露 NUMA 拓扑，但工程代码可以在接口层表达位置意识。例如线程池可以接收 worker 拓扑描述；内存池可以按节点创建 arena；任务可以带有 preferred node；归约可以返回分层 partial；队列可以按 shard id 提交。接口不需要一开始绑定某个 Linux API，但要给后续拓扑策略留下位置。

下面是一个概念性的任务描述。

Listing 4.1 任务元数据中保留位置偏好

```
1 struct TaskPlacement {
2     std::int32_t preferred_worker = -1;
3     std::int32_t preferred_numa_node = -1;
4 };
5
6 struct ComputeTask {
7     std::size_t begin = 0;
8     std::size_t end = 0;
9     TaskPlacement placement;
10 };
```

这段代码不是要读者立刻实现完整 NUMA runtime，而是提醒接口设计不要把任务看成无位置的函数。对数组块任务，`begin/end` 暗含数据位置；对分布式 shuffle，partition id 暗含目标节点；对线程池，本地队列暗含 worker 位置。位置语义一旦在接口中完全丢失，后面很难再补回来。

拓扑不是核心数列表 很多程序只读取“有多少 CPU”，然后创建同样数量线程。这不足以描述现代机器。CPU 拓扑包含 socket、NUMA node、core、SMT sibling、cache 共享层级、内存控制器和 I/O 设备位置。两个逻辑 CPU 可能是同一个物理核心的 SMT sibling，也可能在同一个 socket 的不同核心，也可能跨 socket。它们之间共享资源和通信成本完全不同。

性能报告应至少记录 `lscpu` 中的核心数、线程数、socket 数、NUMA 节点和 cache 信息。在多 socket 机器上，还应记录每个 NUMA node 的 CPU 列表。线程池设计也要区分 worker id 和 CPU id。worker id 是运行时内部编号，CPU id 是操作系统调度对象，NUMA node 是内存位置。把它们混为一谈，会让绑定和 first-touch 实验失去意义。

互连成本如何出现 多核心之间通过片上网络、环、mesh 或 socket 间互连通信。软件不会直接调用“互连”，但互连成本会出现在 cache line 迁移、远端内存访问、跨 socket 锁竞争和共享 LLC 争用中。一个全局原子计数器在单 socket 小机器上看起来还能接受，在多 socket 上可能因为 cache line 在 socket 间来回迁移而急剧变慢。

互连成本也会出现在读多写少结构中。只读共享通常可扩展，因为多个核心可以缓存副本；写

人会让其他副本失效。若一个配置对象每秒更新一次，问题不大；若一个统计字段每次请求都写，问题很大。软件层面要区分只读共享、低频写共享和高频写共享。这比盲目选择 atomic 或 mutex 更重要。

目录协议的工程影响 目录协议维护某条 cache line 的副本位置或所有权信息。软件工程师不需要实现目录协议，但要理解它的结果：共享范围越大，一致性维护越贵；写者越分散，所有权迁移越频繁；数据越能保持本地私有，扩展性越好。分片、线程本地、per-NUMA partial 和分层合并，都是把共享范围缩小。

这也解释为什么“每个线程写自己的数组下标”不一定安全高效。如果这些下标落在同一 cache line，硬件仍然认为它们共享同一个一致性单位。软件的逻辑独立必须映射成硬件的物理独立，才真正减少一致性流量。

First-touch 的细节 First-touch 通常发生在页面第一次被写入或实际分配物理页时。只调用 `reserve` 或 `malloc` 可能只获得虚拟地址，不一定分配物理页。第一次写入触发缺页，内核选择物理页所在 NUMA 节点。若主线程初始化全部数组，页面可能集中在主线程所在节点；后续其他节点线程访问就变成远端访问。

因此 NUMA 实验要把分配、初始化和计算分开。分配阶段只申请地址；初始化阶段按目标线程分块写入，建立页面位置；计算阶段使用同样分块读取。若初始化也计入总时间，要单独报告，因为 parallel first-touch 可能把初始化变快或变慢。若只关心计算阶段，就要在计时前完成初始化并预热。

```
NUMA experiment phases:
allocate virtual memory
initialize pages by owner worker
optionally warm up
compute with the same worker-to-range mapping
merge partials by node then globally
```

线程绑定的收益和风险 线程绑定可以减少调度迁移，让 worker 更稳定地访问本地 cache 和本地 NUMA 内存。它也可能降低操作系统调度灵活性。如果绑定策略错误，把多个 heavy worker 绑到同一个核心或同一个 NUMA 节点，就会变慢。绑定还要考虑系统上其他进程、容器配额和可用 CPU 集合。

实验中，绑定的价值是减少变量。未绑定实验可能因为操作系统迁移而样本波动；绑定后更容易解释线程和数据位置。但生产系统是否绑定，要看服务类型和部署环境。批处理计算通常更适合显式绑定；通用服务可能需要给调度器更多空间。教材应要求读者记录绑定策略，而不是强制所有程序绑定。

分层归约 NUMA 友好的归约通常分层。每个 worker 计算本地 partial；同一 NUMA 节点内先合并成 node partial；最后跨节点合并。这样高频读写发生在本地，跨节点通信只在较小 partial 上发生。这个结构也对应分布式系统中的 local combine、node-level reduce 和 global reduce。

```
hierarchical reduce:
worker partials: no sharing during hot loop
node partials: merge within NUMA node
global result: merge node partials
```

分层合并的代价是代码复杂度和额外阶段。若数据很小或机器只有单 NUMA 节点，收益不明显。实验要比较平坦合并和分层合并，并记录每阶段耗时。不要把 NUMA 策略写成不可关闭的复杂路径。

跨节点队列的代价 全局队列在 NUMA 机器上常成为跨节点热点。生产者 and 消费者分布在不同节点，队列锁、head/tail 和槽位状态会在节点间迁移。分片队列或每节点队列可以减少这种迁移。任务优先进入本节点队列；本节点空闲时先偷同节点任务，再跨节点偷。这个策略和分布式调度中的数据本地性一致。

有界队列的容量也可以按节点分配。若每个 NUMA 节点有自己的输入队列，上游可以根据数

据位置提交任务；某个节点过载时，其他节点不必被同一个全局队列阻塞。缺点是负载均衡更复杂，任务路由需要知道位置。设计要看工作负载是否有明显数据本地性。

远端释放和内存池 NUMA 不只影响分配，也影响释放。一个对象在节点 A 分配，由节点 B 释放，可能导致分配器元数据和 cache line 在节点间移动。高性能分配器常使用线程本地或节点本地缓存，但跨线程释放仍然需要特殊路径。对象池设计应考虑 owner：对象最好由分配它的线程或节点回收，跨节点释放可以先放入 remote free 队列，再由 owner 批量回收。

无锁数据结构的回收协议也会遇到这个问题。Hazard pointer 或 epoch 延迟释放后，最终由哪个线程 delete 节点？如果所有删除都集中到一个线程，可能造成远端释放和内存热点。内存管理和拓扑不能完全分开。

I/O 设备和 NUMA 网卡、NVMe 和 GPU 等设备通常连接到某个 NUMA 节点附近。线程处理设备 I/O 时，如果运行在远端节点，可能增加 PCIe 和内存访问成本。高性能网络和存储系统常把中断、队列、缓冲区和处理线程放在同一 NUMA 节点。虽然本册第二册不深入设备驱动，但要让读者知道 I/O 也有拓扑。

分布式 shuffle 如果在真实服务器上运行，网络接收线程、解压线程和 reduce worker 的位置会影响吞吐。若网卡在节点 0，接收缓冲在节点 0，但 reduce worker 在节点 1 读取，远端内存流量会增加。拓扑意识不只属于计算内核，也属于 I/O 管线。

自动 NUMA balancing Linux 自动 NUMA balancing 会采样页面访问，并可能迁移页面到更常访问它的节点。它能改善一些长期运行程序，但也会引入额外 page fault、迁移成本和实验不确定性。短 benchmark 可能还没等自动机制收敛就结束；长服务可能在负载变化时发生页面迁移。

实验报告应记录是否启用自动 NUMA balancing，至少要知道它可能影响结果。若要做可控实验，可以使用 numactl、线程绑定和明确 first-touch。若要评估生产行为，则应在接近真实配置下测量，并记录自动迁移指标。不要让内核自动策略成为看不见的变量。

容器和 cgroup 的影响 现代部署常在容器里运行。容器可能限制 CPU 集合、内存节点、CPU 配额和内存上限。程序看到的 `std::thread::hardware_concurrency` 未必等于实际可用核心数；cgroup 配额可能让线程周期性被 throttled；cpuset 可能只允许访问部分 CPU。性能报告若忽略容器限制，就可能误判扩展曲线。

在 Termux、手机或云环境中，也要承认设备限制。手机通常没有多 socket NUMA，但有大小核、温度和频率限制；云主机可能共享物理资源；容器可能禁止读取某些性能事件。教材项目应记录环境，而不是假设每个读者都有理想服务器。

拓扑感知不等于硬编码 接口应表达拓扑信息，但不应把某台机器的拓扑硬编码进算法。正确做法是运行时发现或配置拓扑，然后把任务、内存池和队列映射到抽象 node。算法只知道 preferred node 或 shard id，不直接写死 CPU 编号。这样同一代码可以在单 socket、双 socket、容器和手机上运行，只是策略不同。

Compute Systems Engine 可以从简单配置开始：`worker_count`、是否绑定、每个 worker 的 node id。没有 NUMA 信息时，所有 worker 属于 node 0。后续在支持的 Linux 机器上再读取真实拓扑。这样项目不会因为缺少硬件而无法运行，也不会放弃拓扑扩展。

4.4 实验边界、降级和项目落地

没有 NUMA 机器时，不能声称验证了远端内存优化；但仍可以验证 false sharing、线程绑定、任务粒度和内存带宽曲线。单 socket 上 parallel first-touch 和主线程初始化可能差不多，这是正常结果，不是实验失败。报告要写清：本环境只能验证某些机制，不能验证跨 socket 页面放置。

有 NUMA 机器时，也不能只跑一次。需要比较绑定与未绑定、主线程初始化与 worker 初始化、平坦合并与分层合并、单节点内存与 interleave 内存。只有多组对照一致，才能说 NUMA 放置参与了性能结果。

案例：双 socket 归约 在双 socket 机器上，大数组归约可以展示 NUMA 的完整链条。版本一由主线程初始化数组，然后所有 worker 分块求和。版本二由每个 worker 初始化自己的块，再求和。版本三按 socket 分组：每个 socket 的 worker 求本地 partial，socket 内合并，再跨 socket 合并。若版本二和三明显更好，说明页面放置和分层合并参与了性能。

实验要记录线程绑定。若 worker 在计算阶段被迁移到其他 socket，first-touch 的页面位置和线程位置又不匹配。还要记录数组大小，太小可能全部被 cache 影响，太大才体现内存控制器。这个

案例把第 3 章的 page fault、第 4 章的 NUMA、第 13 章的分层 reduce 连在一起。

案例：跨 socket 队列 另一个案例是全局任务队列。所有 worker 从同一个队列取任务，队列锁和 head/tail 位于某个内存节点。跨 socket worker 频繁访问，会让队列状态在 socket 间迁移。本地队列加工作窃取可以减少高频跨 socket 访问：worker 优先本地 pop，空闲时才偷取。

实验可以构造大量短任务，让队列成本显著；再构造少量长任务，让队列成本被计算掩盖。这样读者能看到：同一个队列设计在不同任务粒度下表现不同。NUMA 优化不能脱离任务粒度。

分层资源管理 NUMA 机器适合分层资源管理。每个节点有本地 worker、本地内存池、本地队列、本地 partial。跨节点只交换压缩后的 partial 或被窃取的任务。这个结构和分布式系统的节点本地聚合非常像。若在单机上已经学会分层，扩展到集群时会更自然。

分层也会增加复杂度。每个层级都需要指标：本地队列长度、跨节点 steal 次数、节点 partial 大小、远端访问比例。没有指标，分层策略可能只是让代码复杂。项目可以先用单层实现，再在实验分支中加入分层，对比收益。

大小核和 NUMA 的区别 手机常见大小核，服务器常见 NUMA。二者都意味着执行资源不均匀，但原因不同。大小核是核心性能和能耗不同，同一内存访问拓扑未必像多 socket 那样分节点；NUMA 是内存位置和访问延迟不同，核心本身可能同构。调度策略也不同：大小核更关注把重计算放大核、后台任务放小核；NUMA 更关注线程和内存同节点。

在 Termux 环境中，读者更可能遇到大小核而不是多 socket NUMA。实验报告应避免把大小核现象误称为 NUMA。若线程数增加后性能下降，可能是小核加入、降频、温度或调度迁移，不一定是远端内存。记录环境是第一步。

拓扑抽象接口 项目可以定义一个抽象拓扑接口，而不是直接依赖某个 Linux API。接口返回 `worker_count`、`node_count`、`worker_to_node`、是否支持绑定。单 socket 或手机环境返回一个 node；双 socket 服务器返回多个 node。算法根据 `node_count` 决定是否启用分层 partial。这样代码能在不同设备上运行。

Listing 4.2 拓扑描述的接口形态

```
1 struct WorkerTopology {
2     std::int32_t worker_count = 1;
3     std::int32_t node_count = 1;
4     std::vector<std::int32_t> worker_to_node;
5 };
```

这个结构不直接包含 CPU id，是为了让教学项目先表达策略，再逐步接入真实 affinity。若绑定不可用，仍然可以使用 `worker_to_node` 做逻辑分层实验。接口应允许能力缺失，而不是让项目只能在服务器上运行。

拓扑指标 拓扑感知优化需要指标验证。每个 node 的任务数、处理字节、partial 大小、队列长度、跨节点 steal 次数、远端调度次数都应记录。若启用分层后跨节点 steal 很多，说明负载不均或任务划分不好；若某个 node 长期空闲，说明调度没有利用资源；若 node partial 合并很慢，说明最终合并成为瓶颈。

没有这些指标，拓扑策略容易变成不可解释的复杂代码。项目应先输出简单文本摘要，后续再画图。

拓扑和故障域 拓扑不只影响性能，也影响故障域。单机里，一个 NUMA 节点过载可能拖慢本地 worker；集群里，一个 rack、zone 或机房故障会影响一组节点。分布式副本放置通常要跨故障域，避免一处故障丢失多数副本。单机拓扑意识可以自然扩展到集群故障域意识：不要把所有关键副本、任务或队列集中在同一个风险位置。

Compute Systems Engine 的本机模拟可以先用 node id 表示逻辑故障域。后续多进程或多机版本中，node id 可以映射到主机、rack 或 zone。调度器在追求本地性的同时，也要避免把所有副本放同一域。性能和可靠性有时会冲突，系统设计要显式权衡。

拓扑章节总结 本章的核心不是让读者背 NUMA API，而是建立位置意识。线程有位置，内存有位置，队列有位置，I/O 设备有位置，副本也有位置。位置决定通信成本和故障相关性。没有位置意识的程序在小机器上可能正常，在大机器上会遇到扩展性和可靠性问题。

拓扑决策表 做拓扑相关设计时，先问机器是否真的有多多个内存节点，是否能设置亲和性，数据是否大到超过 cache，任务是否访问固定数据块，是否有跨节点共享写，是否需要跨故障域放置副本。若没有 NUMA 或数据很小，复杂拓扑策略可能无收益；若数据大且任务固定访问某块，first-touch 和绑定值得实验；若共享写频繁，先分片再谈绑定；若是分布式副本，优先考虑故障域而不是只考虑延迟。

拓扑策略的降级设计 拓扑感知代码必须能降级。很多读者会在手机、容器、云虚拟机或普通单 socket 机器上运行实验，程序不一定能读取完整 NUMA 信息，也不一定有权限设置 affinity。如果代码把真实 NUMA 当成必需条件，就会让教材项目难以运行。更好的策略是把拓扑能力分层：能读取真实 node 时使用真实 node；不能读取时退化成一个逻辑 node；能绑定线程时执行绑定；绑定失败时记录失败并继续运行；能控制内存策略时做 first-touch 实验；不能控制时只做普通分块。

降级不等于忽略拓扑。即使只有一个逻辑 node，代码仍然可以保留 worker 到 node 的映射、分层 partial 的接口和队列分片的抽象。这样程序在小机器上正确运行，在大机器上能启用更多策略。教材项目尤其适合这种设计：先让抽象存在，再逐步接入真实硬件能力。读者不会因为缺少双 socket 服务器就学不了拓扑意识。

拓扑策略还要写入输出。若报告只写“启用 NUMA 优化”，但实际上 affinity 设置失败，读者会得到错误结论。程序应输出能力检测结果、降级原因和最终策略。例如 `node_count` 是 1，`binding_enabled` 是 false，`binding_error` 是 permission denied，`memory_policy` 是 default。这样实验结果才诚实。高性能系统的第一步不是让所有优化都打开，而是知道哪些优化真的生效。

降级设计还有一个好处：它迫使接口表达意图，而不是表达某个平台的偶然细节。算法真正需要的是“这个任务最好靠近哪一组数据”和“这个 worker 属于哪个执行域”，不一定需要知道具体 CPU 编号。平台能力强时，执行域可以映射到真实 NUMA 节点；平台能力弱时，执行域只是逻辑分组。把意图和机制分开，代码才不会因为换设备而到处改。

这种抽象也能保护学习路径。读者先在普通设备上理解任务、数据和执行域的关系，再到服务器上验证真实远端访问成本。先学清楚问题，再打开平台能力，实验结论会稳得多。

NUMA 和一致性章的回归阅读应围绕一个并行归约推进。第一版所有线程更新一个全局计数器，暴露 cache line 所有权反复迁移；第二版每线程 partial，消除热写但仍可能出现 false sharing；第三版按 NUMA 节点分块初始化和合并，观察 first-touch 和远端内存；第四版把全局队列改成分层队列，减少跨节点争用。每一步都只改变一个因素，才能说明瓶颈从哪里迁移。

一致性协议在本章只提供硬件背景，不替代 C++ 内存模型。它解释为什么多个核心写同一条 cache line 会贵，为什么共享只读和共享写不是一回事，为什么 padding 有时有效。但程序正确性仍要靠锁、原子和明确的同步关系。把 MESI 当作语言级保证，是本章必须避免的误区。

本章验收要允许设备差异。手机或单 socket 机器可能没有可观察 NUMA，但仍能完成 false sharing、全局队列争用和线程本地 partial 实验；有多 socket 机器时，再加入 first-touch 和绑定策略。报告不能把某台机器上的绝对数字写成普遍规律，应写成“这种数据放置在这种拓扑下产生了这种成本”。

一致性成本要落到 cache line 所有权。两个线程更新不同变量，如果变量落在同一条 cache line 上，硬件仍可能把这条 line 在核心之间来回转移；这就是 false sharing 的根。把字段 padding 到不同 line 可能降低转移，但也会增加内存占用和 cache footprint。教材不能只说“加 padding”，还要让读者计算对象数量、额外字节和是否真的位于热写路径。

NUMA 策略要区分初始化、访问和合并。first-touch 只影响页面最初归属，后续线程迁移、重新分配和文件映射都可能改变实际成本；线程绑定只约束执行位置，不自动移动内存；本地 partial 能减少远端写，但最终合并仍然要跨节点读。项目报告应把这三步分开写：页面由谁触碰，热循环由谁访问，合并由谁执行。这样读者才能看出拓扑优化到底解决哪一段成本。

互连成本还会改变数据结构选择。一个全局无锁队列在单 socket 上可能还能接受，跨 socket 时每次 head 或 tail 更新都可能拉动 cache line 所有权；一个全局 hash table 的分片锁如果没有按 NUMA 节点分层，热点 bucket 会把远端访问集中到少数 line；一个线程本地 partial 如果最后由单线程合并，合并阶段又可能变成串行远端读。拓扑意识不是只给线程绑核，而是让共享状态也分层。

本章可以让读者写一个降级策略。若机器没有 NUMA，就运行 false sharing 和分层 partial；若机器有多个 NUMA 节点，就增加 first-touch、线程绑定和远端访问对照；若没有 PMU 权限，就用吞吐曲线和内存带宽估算替代计数器。高质量教材应告诉读者在不同设备上如何诚实验证，而不是假设每个人都有同一台服务器。

4.5 把共享内存放回拓扑和一致性协议

多核机器给人的错觉是“所有核心访问同一块内存”。程序模型可以这么说，性能模型不能这么说。不同核心有自己的私有缓存，多个核心通过一致性协议维护共享数据的可见性；多 socket 或多 NUMA 节点机器上，内存控制器和物理页面也有位置。一次读写看似普通，实际可能涉及 cache line 所有权、远端内存、互连带宽和页面迁移。NUMA 章要从共享写和页面归属推出来，而不是从硬件拓扑图开始背。

最小例子是全局计数器。单线程里它只是一个变量；多线程里每个 worker 都更新它，cache line 所有权会在核心之间迁移，吞吐不会线性增长。把计数器改成 atomic 只修复数据竞争，不会消除共享写热点。把计数器改成线程本地 partial，再做分层合并，才真正改变共享模式。这里的推导非常关键：正确性原语和扩展性结构不是同一件事。

一致性协议提供可见性基础，不替代内存模型

MESI 之类状态模型适合建立直觉：一条 cache line 可以被多个核心共享读取，但写入需要获得独占或修改权限。若两个核心反复写同一条 line，即使写的是不同字段，也会发生所有权迁移。这个直觉足以解释 false sharing、热点 atomic 和全局队列锁为什么扩展性差。教材不需要让读者背每个协议变种的全部状态，但要让读者理解“写共享比读共享贵得多”。

硬件一致性和 C++ 内存模型分工不同。硬件负责在给定体系结构规则下维护缓存一致性，C++ 内存模型定义程序中哪些跨线程读写有数据竞争、哪些原子操作建立 happens-before。把普通变量交给硬件一致性并不能让数据竞争变成正确程序；把所有操作都改成 seq_cst 也不能消除热点 line 的性能成本。后续原子章会继续展开，本章只需要把边界立住：正确性要按语言模型证明，性能要按一致性流量和拓扑分析。

一致性流量也会影响锁。一个 mutex 不只是“进入临界区”这么简单，它内部会有原子状态、等待队列和可能的 futex 系统调用。低争用时锁成本很低，高争用时锁变量所在 line 会成为热点，等待线程可能睡眠和唤醒。锁章会讲不变量和条件变量，本章先让读者看到锁的硬件侧成本：临界区越短不一定越好，若进入次数极高，锁本身的 cache line 迁移就可能主导。

NUMA first-touch 和页面归属

NUMA 的第一条工程规则是 first-touch：页面通常归属第一次触碰它的节点。若主线程在 node 0 初始化一个大数组，随后 node 1 上的 worker 大量访问这批页面，worker 就会读远端内存。这个问题在小机器上可能看不出来，在双 socket 服务器上会让扩展曲线突然变差。多线程初始化不是形式主义，而是页面放置策略。

实验要设计成能触发差异。版本一由主线程初始化大数组，再让所有 worker 分块求和；版本二由每个 worker 初始化自己将要处理的块，再在同一绑定策略下计算；版本三故意交换 worker 和数据块，制造远端访问。报告要记录线程绑定、页面大小、输入规模、内存带宽和 NUMA 设备情况。若当前手机或笔记本没有 NUMA，就把实验写成设计和单节点替代，不把结论伪装成已验证。

NUMA 策略还要考虑迁移。Linux 的自动 NUMA balancing 可能把页面迁移到访问频繁的节点，但迁移本身有成本，也可能在 benchmark 中造成波动。生产程序不能只依赖“系统会帮我调好”。对于稳定大任务，显式 first-touch、线程绑定和数据分片更可控；对于动态任务，运行时要记录数据位置，让调度尽量先本地再远端。

互连带宽、设备拓扑和分层设计

互连不是无限总线。多个核心同时访问远端内存、争用共享 line 或从同一个内存控制器读数据时，互连和内存控制器会成为瓶颈。线程数增加后变慢，不一定是算法错了，也可能是远端访问、LLC 争用或内存带宽已经到顶。分层归约和本地队列的共同思想，是先在近处完成大部分工作，再把少量结果跨节点合并。

设备也有拓扑。网卡、NVMe、GPU 或其他设备连接在特定 PCIe root complex 下，离某些 CPU/NUMA 节点更近。虽然第二册第二册不进入 GPU AI 算子，但现代计算系统必须知道：I/O 线程、内存 buffer 和设备队列最好考虑拓扑。否则一个线程在远端节点准备 buffer，再交给近端设备 DMA，路径会绕远，cache 和 IOMMU 成本也可能增加。

分层设计不等于硬编码拓扑。项目可以定义一个 Topology 抽象：列出 worker 所在逻辑 CPU、NUMA node、缓存组和可用设备；调度器根据抽象做本地优先策略；没有 NUMA 的设备上退化成单节点策略。这样教学代码不会把某台机器的编号写死，也能让读者理解“拓扑感知”和“不可移植硬编码”的区别。

项目中的拓扑验收

贯穿项目在本章应完成 sharded counter、NUMA-aware block placement 和分层 reduce 三个检查点。Sharded counter 用来证明共享写热点；block placement 用来证明页面归属；分层 reduce 用来证明先本地合并再跨节点合并的结构。每个检查点都要保留不使用拓扑的 baseline，并写出何时 baseline 反而更好，例如小输入、单 socket、线程数少、合并成本大于远端成本。

源码阅读可以从 Linux 调度域、NUMA balancing 和内存策略入口开始，不要求一次读懂内核全部路径。读者只需要建立映射：用户态的线程绑定对应内核 task 的 CPU mask，页面触碰对应 VMA 和 page fault，NUMA 统计对应页面位置和访问事件。能把用户态实验现象映射到这些对象，本章就达到了目的。

本章最后要形成一句判断：凡是多个核心反复写同一份状态，先怀疑一致性流量；凡是多个节点访问大工作集，先确认页面归属；凡是线程数增加后不再加速，先画拓扑和共享写路径，再决定是改算法、改布局、改调度还是接受带宽上限。

案例走查：线程本地 partial 为什么还要分层合并

把全局 atomic counter 改成每线程 partial 后，共享写热点消失，但最终合并仍可能成为单点。若线程数少、partial 小，串行合并很好；若 partial 是大数组，且线程分布在多个 NUMA 节点，单线程跨节点读取所有 partial 会访问远端内存。分层合并先在每个节点本地合并，再跨节点合并少量结果，减少远端流量。

实验在单 NUMA 设备上也能做简化：把 partial 按 worker 分组，比较串行合并和树形合并的阶段耗时；若有 NUMA 设备，再加 first-touch 和绑定。指标包括每阶段合并字节、合并时间、线程空闲时间和最终 checksum。若没有 NUMA，报告写清只验证了算法结构，未验证远端内存收益。

这个案例说明拓扑优化不应写死。项目代码可以提供 `Topology::groups()`，单节点返回一个组，多节点返回多个组。算法按组做分层，而不是硬编码 socket 编号。这样同一份代码能在手机、笔记本和服务器的降级运行。

NUMA 章节的项目代码形状

拓扑感知不应散落在每个算法里。项目可以定义 `CpuTopology` 和 `PlacementHint`：前者描述 worker 属于哪个组，后者描述任务最好在哪个组运行。算法只产生 hint，例如 input block 属于 group 2；运行时负责尽力调度；无法满足时记录一次 remote execution。这样拓扑是可观策略，而不是硬编码条件。

`CpuTopology` 在没有 NUMA 的设备上返回一个组，所有任务本地执行。双 socket 机器上返回多个组。大小核设备可以按性能等级分组。这个抽象让同一章节能服务手机、笔记本和服务器的用户。现在在 Termux 上，也可以先实现单组版本，保留接口，后续换机器再验证多组策略。

代码还要避免让拓扑策略破坏正确性。任务无论在哪个 worker 执行，输出 checksum 必须相同；拓扑只影响性能和 locality，不影响语义。测试先随机打乱 placement，确认结果一致；再开启本地优先策略，测性能。这样读者知道调度策略和业务正确性是分开的。

一致性流量的观测替代

不是所有设备都能直接测 cache coherency traffic。可以用反例间接观察：全局 atomic 计数器、紧凑 partial 数组、填充 partial 数组、线程本地 vector 四个版本。若全局 atomic 随线程数增加很快饱和，而线程本地版本扩展更好，说明共享写路径参与成本。若填充 partial 改善紧凑 partial，说明 false sharing 参与成本。

报告要避免过度断言。没有专用 uncore 计数器时，不应写“互连流量为多少”，而应写“结果符合共享写热点假设”。若能在服务器上使用 uncore 或 numastat，再补更强证据。教材要教读者根据证据强度说话。

源码阅读可以从调度拓扑和内存策略文档开始，再看 `kernel/sched/topology.c`、`mm/mempolicy.c`、`mm/huge_memory.c` 的入口。只读入口，不追求深挖每个分支。能把 first-touch、线程绑定和 page migration 映射到内核概念，本章目的就达到了。

综合练习：拓扑感知归约的降级设计

本章综合题要求读者为归约写一个拓扑感知版本，并且必须支持没有 NUMA 的设备。接口先定义拓扑组，不直接暴露 socket 编号。单组设备上，所有 worker 属于组 0；多节点设备上，worker 按 NUMA node 分组；大小核设备上，可以按性能等级分组。算法只依赖组抽象：组内 partial 先合并，组间再合并。这样设计能在手机上运行，也能在服务器上验证远端成本。

实现时先写不感知拓扑的 baseline，再写分层归约。分层归约不允许改变最终语义，输出 checksum 必须一致。若浮点归约顺序改变，要写误差合同或固定合并树。数据初始化也要分版本：主线程初始化、worker 本地初始化、故意交叉访问。没有 NUMA 硬件时，只验证接口和合并树；有 NUMA 时，再验证 first-touch 和绑定收益。

报告中必须有降级说明。若系统无法读取 NUMA 拓扑，运行时如何退化；若线程绑定失败，是否继续运行；若拓扑组信息和实际性能不符，指标如何暴露。工程设计不能把“我的机器支持”当成前提。用户在 Termux 上也应能跑通功能，只是某些性能结论标记为未验证。

这个题还能训练边界意识。拓扑感知若写入算法核心，后续每个内核都要处理平台差异；若放在运行时和 placement hint 中，算法只描述数据位置偏好。好的设计不是到处写 if，而是把硬件差异收束到少数接口。

容易出错的边界：拓扑感知什么时候不值得

拓扑感知不是越复杂越好。若输入很小，所有数据都在 LLC 内，NUMA 策略可能没有收益；若任务极不均匀，强本地优先可能让某些 worker 空闲；若系统是手机或单 socket 机器，复杂 NUMA 分组只会增加代码负担；若数据很快被压缩成本地 partial，远端访问的时间占比可能很低。教材必须告诉读者：先测是否存在远端成本，再引入拓扑策略。

第二个反例是“绑定线程一定稳定”。绑定可能稳定实验，也可能破坏调度器对大小核、温控和后台任务的管理。移动设备上，固定大核长时间运行会触发降频，后半段反而慢。服务器上，绑定不当可能把线程集中到同一 LLC 组或同一 NUMA 节点，造成资源争用。绑定策略必须和拓扑、输入、持续时间一起报告。

第三个反例是“atomic 修复了共享写问题”。Atomic 修复数据竞争，不修复 cache line 所有权迁移。热点 atomic 仍然可能把互连打满。正确修复往往是减少共享写频率，例如线程本地 partial、分片、批量合并或改变算法。这个反例会在锁章和原子章再次出现，本章先从硬件成本角度立住。

源码阅读可以把 Linux 的自动 NUMA balancing 当作反例材料。系统可能迁移页面，但迁移不是免费，也不是每个 workload 都能判断正确。用户态设计若完全依赖自动迁移，就会把性能决定权交给难以观察的后台机制。更稳的方式是让项目记录 placement 和实际执行位置，至少知道策略是否被执行。

本章核心接口：PlacementHint

NUMA 章给项目留下 PlacementHint。任务可以声明 preferred group、data block id、expected bytes、can migrate 四个信息。运行时尽量按 hint 调度，但不保证一定满足；如果未满足，就在 trace 中记录 remote 或 fallback。这样语义和性能分开：任务在哪里跑都应正确，但 hint 可以帮助解释性能。

后续任务运行时和分布式调度会复用这个思想。本机是 NUMA group，集群里就是数据本地节点；本机 remote execution 只是慢，集群 remote execution 可能多一次网络读取。接口稳定后，概念能自然迁移。

从硬件模型进入测量模型

前四章给出硬件侧解释，但解释必须通过测量落地。知道 cache line、TLB、NUMA、互连和一致性，只能产生候选假设；要判断哪一个假设支配当前程序，还需要实验矩阵、计数器、profile 和 trace。下一部分从测量开始，是为了防止读者把硬件知识当成凭经验猜瓶颈的工具。

项目在这里应暂停一次，整理前四章产生的候选指标：热循环 cycles 和 instructions，地址序列和工作集，false sharing 对照，first-touch 和线程绑定。下一章把这些指标组织成报告格式。这样硬件知识不会停留在文字，而会变成后续每次优化都能复用的证据模板。

4.6 项目落地

Compute Systems Engine 在本章应加入三个设计点。第一，拓扑记录：benchmark 输出可见 CPU 数、用户指定 worker 数、是否启用绑定、是否能读取 NUMA 信息。第二，分层 partial：即使当前只实现每 worker partial，也在设计文档中说明未来可以扩展成每 NUMA 节点 partial。第三，队列分片路线：线程池初版可以用全局队列，但要记录全局队列可能成为共享热点，后续工作窃取章节再拆成本地队列。

本章的验收不是一定在手机或单 socket 机器上测出 NUMA 差异，而是实验设计正确。没有 NUMA 设备时，报告应写明限制：可以验证 false sharing、线程绑定和内存带宽曲线，但不能验证

远端内存放置。承认证据边界，比编造一个漂亮结论更专业。

一致性流量 多个核心读同一份数据通常容易扩展，多个核心写同一条 cache line 会制造所有权来回迁移。原子计数、共享队列尾指针、全局统计表都是典型热点。分析时先问“谁写，写在哪里，频率多少”。

NUMA 归属 页面通常在首次触碰时分配到某个 NUMA node。若主线程初始化全部数据，worker 在线程绑定后访问远端页面，性能会被隐藏地拉低。first-touch 不是小技巧，而是数据归属策略。

拓扑感知调度 线程绑定不是越死越好。绑定能稳定实验和保护本地性，也可能降低调度器处理负载不均的能力。工程报告要写清绑定策略、CPU 列表、容器 cpuset 和不可用时的降级。

分层合并 跨 node 共享写昂贵时，应先在 core 或 node 本地生成 partial，再分层合并。这个原则后面会出现在 reduce、histogram、shuffle 和分布式 combine 里。

远端访问预算 远端访问不一定绝对禁止，但要有预算。若本地 node 忙而远端空闲，远端执行也许更快；若数据巨大而计算很轻，移动线程比移动数据更合理。NUMA 策略要服务 workload。

自动迁移 Linux 可能尝试自动 NUMA balancing，但迁移页面有成本，也可能判断不准。用户态项目不能把性能正确性完全交给后台机制，至少要记录页面放置和线程位置。

容器环境 手机、虚拟机、容器和云实例可能隐藏部分拓扑信息。教材要教读者写“不可验证”的报告，而不是编造 NUMA 结论。可观察 CPU mask、调度迁移次数和缓存行为仍然有价值。

项目接口 PlacementPlan 应描述数据分片、建议 node、执行 worker、实际 CPU、partial 合并层级和远端访问估计。它不是调度器的全部实现，却让后续运行时有可讨论的对象。

案例：主线程初始化反例 先由主线程初始化大数组，再由绑定到另一个 node 的 worker 扫描；再改成 worker 本地 first-touch。比较两者，说明页面归属不是抽象名词。

案例：共享计数器热点 所有线程 `fetch_add` 同一计数器，再改成每 node partial。观察扩展曲线，从一致性流量解释为什么线程越多越慢。

案例：分层归约 core 本地 partial、node 本地 partial、全局合并三层实现同一 reduce。这个案例连接硬件拓扑和并行算法。

源码阅读路线 Linux 源码阅读从 `kernel/sched/topology.c`、`mm/mempolicy.c`、`mm/huge_memory.c` 附近进入即可。阅读目标不是把整套子系统抄进书里，而是追一条和本章实验相关的路径：用户态动作怎样变成内核对象，内核对象怎样排队、等待、唤醒、计数、提交或回收。

阅读报告要写证据等级。能直接测到的数字放在第一层；从源码控制流推导出的解释放在第二层；当前设备无法验证、只能作为合理猜测的内容放在第三层。这样读者会知道拓扑放置报告中哪些结论可复现，哪些结论需要换机器继续确认。

源码阅读还要克制。看到一个复杂函数，不要把每个分支都展开；先回到本章的问题，问它能否解释一个等待点、一次状态转移或一条性能曲线。解释不了的内容可以留到索引，不要打断正文主线。

主线程初始化反例 先由主线程初始化大数组，再由绑定到另一个 node 的 worker 扫描；再改成 worker 本地 first-touch。比较两者，说明页面归属不是抽象名词。

共享计数器热点 所有线程 `fetch_add` 同一计数器，再改成每 node partial。观察扩展曲线，从一致性流量解释为什么线程越多越慢。

分层归约 core 本地 partial、node 本地 partial、全局合并三层实现同一 reduce。这个案例连接硬件拓扑和并行算法。

一致性所有权

多个核心读同一 cache line 容易扩展，多个核心反复写同一 line 会让所有权在核心之间迁移。原子热点和 false sharing 都来自这个边界。

实验设计：让所有线程写同一计数器，再改成每线程槽位，比较扩展曲线。

NUMA 归属

页面通常由第一次触碰的线程决定物理节点。主线程初始化全部数组后让远端 worker 访问，可能把远端内存成本藏进计算阶段。

实验设计：分别用主线程初始化和 worker 本地初始化，记录带宽和线程绑定策略。

first touch

first touch 不是仪式，而是把数据归属和执行位置对齐。它要求初始化循环和计算切分采用同一拓扑视角。

实验设计：把分片初始化函数加入项目，要求每个分片记录建议 node 和实际 worker。

拓扑感知

CPU 列表、socket、core、SMT 和容器 cpuset 都会影响实验。绑定策略不能只写线程数，要写可见 CPU 和不可见限制。

实验设计：在 Termux、容器或普通 Linux 中记录可见 CPU mask，并说明哪些 NUMA 结论无法验证。

分层 partial

共享写昂贵时，先在线程本地或 node 本地生成 partial，再树形合并。这个原则后来会出现在 histogram、reduce 和 shuffle combine 中。

实验设计：为每个 worker 保存本地桶，再按 node 合并，最后全局合并。

自动迁移

操作系统可能迁移线程或页面。迁移有时修正局部性，有时破坏可重复实验。报告要区分默认调度和人为绑定。

实验设计：先跑默认策略，再跑固定亲和性，并记录迁移次数或替代指标。

互连瓶颈

当核心数增加后吞吐不再增长，原因可能不是算术，而是内存控制器、互连或一致性目录压力。证据要从共享写和远端访问入手。

实验设计：把任务从共享队列改成分片队列，观察是否减少所有权迁移。

队列槽位

多生产者队列的 head、tail 和状态字段容易形成共享热点。队列设计必须把控制字段与数据槽位分开看。

实验设计：给队列状态加对齐和分片指标，说明哪些字段被频繁写。

降级策略

没有 NUMA 设备时仍能学习原则：用线程绑定、false sharing、共享计数器和队列热点模拟拓扑成本。不能验证远端内存时要诚实说明。

实验设计：报告列出可验证项、替代实验和不可验证项。

项目接口

PlacementPlan 不只是记录 node 编号，还要连接数据分片、worker、partial、合并层级和证据来源。没有这个对象，拓扑优化会散落在命令行里。

实验设计：每个分片输出建议位置、实际位置和合并路径。

4.7 共享内存真正共享的是 cache line、页面和互连带宽

NUMA 和一致性章节要从一个反直觉现象开始：线程越多，程序越慢。初学者会怀疑线程池写错了，或者认为 CPU 核心不够；但如果所有线程都 `fetch_add` 同一个计数器，真正发生的是同一条 cache line 的所有权在核心之间来回迁移。共享内存模型让地址看起来统一，硬件却必须维护“谁能写这条线”的协议。读者理解这一点，才会知道为什么线程本地 partial 是原则，不是微优化。

一致性流量和 false sharing 可以用同一张图解释。多个核心读同一条线，硬件可以复制；多个核心写同一条线，写者需要独占；多个线程写相邻数组元素，如果这些元素落在同一条 cache line，也会互相干扰。源码里变量不同，不代表硬件里位置不同。实验不要一上来讲复杂 NUMA，先让每线程计数槽位相邻，再加 `alignas(64)` 或填充，对比扩展曲线。这个小实验能解释大量并发性能问题。

NUMA 把问题再放大一层。页面由哪个 node 提供，线程在哪个 CPU 上运行，决定了访问是本地还是远端。主线程初始化一个大数组，然后把任务分给其他 node 上的 worker，可能让所有 worker 都访问主线程所在 node 的内存。first touch 的意义就在这里：初始化和计算使用同样的切分，让页面归属靠近使用者。它不是一条命令，而是一种数据放置策略。

但是绑定也不是万能。线程绑得太死，负载不均时调度器无法帮你迁移；容器或手机环境可能隐藏拓扑；云实例的 NUMA 信息也可能受虚拟化影响。教材要教读者写限制：我能看到哪些 CPU，

能否看到 node, 能否读 `/proc/self/numa_maps`, 能否使用 `numactl`。不能验证远端访问时, 就退回 false sharing、共享计数器和线程迁移实验, 不要编造 NUMA 结论。

本章的工程设计落到分层 partial。日志统计可以先按 worker 聚合, 再按 node 合并, 最后全局合并。这样做的好处不只是减少锁竞争, 也是在拓扑上减少跨 node 写入。代价是合并阶段更复杂, 内存占用增加, 结果提交要等更多阶段完成。教材要把收益和代价一起讲: 若输入很小, 分层合并可能不值得; 若 key 高度倾斜, 分层只能缓解部分热点; 若任务调度频繁跨 node, 局部性仍会被破坏。

源码阅读可以从调度拓扑、内存策略和 numa maps 开始。用户态不需要照搬内核调度器, 但要学会它如何描述 CPU 层级、负载和亲和性。读者做报告时, 应把实际 CPU、建议 node、实际 worker、partial 所在位置和合并路径写在一起。这样后续任务运行时和分布式 combine 才能复用本章知识: 跨 node 的远端内存, 在集群里会变成跨机器网络; 线程本地 partial, 在分布式里会变成 map-side combine。

案例推导: 全局计数器如何一步步变成分层聚合

旧版本统计错误数量时, 所有 worker 对同一个 `std::atomic` 计数器执行 `fetch_add`。单线程很快, 四线程也还能接受, 线程再多后吞吐下降。表面看, atomic 已经避免数据竞争; 实际瓶颈是所有核心争夺同一条 cache line 的修改权。这个案例适合让读者区分“正确”和“可扩展”: atomic 让结果正确, 却没有让共享写变便宜。

第一步改成每线程槽位。每个 worker 写自己的计数, 最后合并。若槽位相邻, 仍可能 false sharing, 因此要让槽位按 cache line 分隔, 或把多个热计数组织成线程本地结构。第二步按 NUMA node 合并。每个 node 内部先合并成本地 partial, 再跨 node 合并, 减少远端写。第三步把这个原则推广到 histogram: 每线程本地桶、每 node 合并、最后全局合并。

这个优化也有代价。读取“当前全局计数”不再是一次 load, 而要合并多个槽位; 内存占用增加; 线程数量变化时槽位管理更复杂; 若 key 数量很大, 每线程桶可能爆内存。教材不能只说“分片更快”, 还要告诉读者什么时候不值得。低频计数、少线程、小输入, mutex 或 atomic 可能更简单; 高频共享写和多核扩展, 分层 partial 才值得。

实验要有扩展曲线。横轴是线程数, 纵轴是吞吐和每条记录成本; 版本包括全局 atomic、相邻槽位、对齐槽位、node partial。若设备没有 NUMA, 就至少完成前三个版本, 并在报告中写明 NUMA 不可验证。若能读取拓扑, 就记录 worker 绑定、页面 first touch 和合并层级。不要只写最终数字, 要写瓶颈从共享写迁移到内存带宽或合并成本的过程。

Linux 源码阅读可以看调度拓扑和内存策略, 但读者真正要带走的是一个工程问题: 数据在哪里被首次触碰, 之后由谁频繁写。贯穿项目中, PlacementPlan 应让每个分片知道建议位置 and 实际执行 worker。否则调度器、数据布局 and 合并算法会各自为政, NUMA 知识又会退回口号。

第一部分综合案例: 从一条记录的地址到一台机器的拓扑

第一部分四章合在一起, 要让读者形成一个完整判断: 一段程序慢, 不一定慢在源码看起来最复杂的地方, 而是慢在执行路径上最受限制的资源。我们用同一个日志解析任务贯穿。输入是一批文本行, 解析器找分隔符, 提取事件类型和用户 id, 本地聚合后写出 partial。先单线程运行, 再多线程运行, 再观察缓存和 NUMA 行为。这个任务不复杂, 正适合把硬件执行模型讲清。

第一步看热循环。解析器逐字节扫描, 如果每个字符都进入复杂状态机, 前端供给、分支预测和依赖链都会影响吞吐。优化前先看输入分布: 分隔符位置是否规律, 错误行比例是多少, 字段长度是否随机。若随机性高, 分支预测失败会增加; 若函数边界多, 前端和调用开销会增加; 若状态依赖强, 自动向量化可能失败。热循环报告要记录这些观察, 而不是只写“parser 慢”。

第二步看地址序列。解析后如果保存大结构体数组, 聚合只需要事件类型, 却会搬运原始文本指针、调试字段和错误信息。冷热分离把热字段放进连续数组, 减少 cache line 浪费, 也让 SIMD 和预取更容易。可是冷字段不能丢, 因为错误报告仍需要原始行和 offset。于是 AddressTrace 和 LayoutPlan 必须连接: 热路径看连续字段, 冷路径通过稳定 id 回查原始信息。硬件优化不能破坏可诊断性。

第三步看 TLB 和 page cache。输入来自文件时, 第一次读取和第二次读取成本可能不同; mmap 版本第一次访问页面时可能触发 minor fault; 普通 read 版本把读取和解析的生命周期显式化。报告要写清楚测的是冷文件、热 page cache, 还是解析后内存批次。若没有权限清理 page cache, 就用不同文件或大于内存的数据集替代, 并标注限制。这样的诚实比伪精确更有价值。

第四步看多线程后的共享写。单线程 parser 优化完成后, 多线程聚合可能不再慢在解析, 而慢在共享计数器。所有 worker 更新同一个 atomic 计数器, 会让同一条 cache line 的所有权来回迁移;

改成本地 partial 后，瓶颈可能迁移到合并阶段或内存带宽。这里要把第二章的热循环证据和第四章的一致性证据放在一张表里：优化前慢在哪里，优化后慢到哪里去了。

第五步看 NUMA 和拓扑。若主线程读取并初始化所有 batch，再把 batch 分给其他 node 上的 worker，远端访问可能隐藏在计算阶段。first touch 策略要求初始化和计算采用同样切分；线程绑定要求报告写清 CPU mask 和实际 worker。没有 NUMA 设备时，也可以用 false sharing 和线程迁移实验训练同一思维。不要因为环境限制就跳过拓扑，也不要无证据时写“NUMA 导致”。

这个综合案例的最终输出是一份硬件证据包。它至少包含：热循环反汇编摘要、输入分布、字段冷热表、地址序列说明、page cache 状态、共享写位置、partial 合并层级、线程绑定策略和不可验证边界。读者若能从这份报告解释“为什么某次优化有效、为什么下一次优化无效、瓶颈迁移到了哪里”，第一部分就达到了目标。硬件原理不是芯片图背诵，而是能把源码、地址、线程和拓扑放进同一条因果链。

实验：在支持 NUMA 的机器上，用 `numactl--hardware` 查看节点。写一个大数组初始化和求和程序，比较单线程初始化、多线程初始化、线程绑定和未绑定情况下的带宽。如果没有 NUMA 机器，也要用 `lscpu` 记录拓扑并解释限制。

从一个全局计数器推导一致性流量 多核程序最容易写出的错误性能模型，是认为多个核心同时加一个计数器就会线性变快。全局计数器在语义上只有一个变量，但在硬件上对应一个 cache line 的所有权反复迁移。核心 A 写完后，核心 B 要写就必须获得该 cache line 的独占状态，随后核心 C 又来抢。这个过程让互连上传输的不只是数据，还有所有权和失效消息。计数器越热，核心越多，迁移越频繁，程序越不像并行计算，越像所有核心排队修改同一小块内存。

因此本章要从全局计数器事故讲起。第一版每条记录直接更新全局 `std::atomic<std::uint64_t>`，正确但可能慢。第二版每个线程一个局部计数器，最后归并，减少共享写。第三版为局部计数器加 cache line padding，避免多个线程的计数器落在同一 cache line 上造成 false sharing。第四版按 NUMA node 做分层 partial，先节点内合并，再跨节点合并。每一步都不是技巧堆叠，而是把共享写从高频路径移到低频路径。

False sharing 要用内存布局解释 False sharing 的本质是两个逻辑无关的变量落在同一 cache line 上。线程 A 写变量 x，线程 B 写变量 y，源码上看没有共享；硬件上它们共享同一 cache line，写入仍会导致所有权争用。很多初学者只在“共享变量”上找问题，忽略了 cache line 是一致性的基本粒度。结构体数组、相邻计数器、队列头尾字段、worker 统计字段，都可能出现这种问题。

修复 false sharing 也不是到处 padding。Padding 会增加内存占用，可能降低缓存容量利用率。只有高频跨线程写字段才值得隔离。低频配置字段、只读字段或同一线程访问的字段不需要每个都占一行。项目里的 `WorkerStats` 应把每个 worker 高频写的 counters 放在独立 cache line，汇总线程低频读取；而共享配置和只读表可以紧凑存放。这样做体现的是访问模式驱动布局。

NUMA 的第一原则是内存有归属 NUMA 不是“内存慢一点”的抽象说法，而是页面有物理归属，核心访问本地节点和远端节点的成本不同。Linux 通常采用 first touch 策略，哪个 CPU 首次写入页面，页面就可能分配到对应节点。若主线程初始化全部大数组，然后 worker 分布在多个 NUMA node 上处理，很多 worker 会访问远端页面。正确做法可能是并行初始化，让每个 worker 触摸自己负责的页；也可能是显式设置绑定和内存策略；也可能是按 node 切分任务和数据。

贯穿项目中，输入 split、局部聚合表、shuffle buffer 和任务队列都应该有放置策略。线程本地 partial 若被分配在错误节点，局部性会丢失；work stealing 若随意跨节点偷任务，负载均衡可能换来远端内存访问；全局 merge 若集中到一个节点，会把所有 partial 拉过互连。NUMA 优化的目标不是永远不跨节点，而是把跨节点访问变成低频、批量、可解释的操作。

实验：从共享计数到分层归约 本章实验可以统计事件类型。输入分布固定，算法语义固定，逐步改变共享形态。版本一全局 atomic 计数；版本二每线程数组；版本三 padding 每线程数组；版本四按 NUMA node 分组；版本五把最终合并改成树形。每个版本记录吞吐、cache miss、远端访问指标、线程间方差和合并时间。若缺少硬件计数器，也至少记录不同线程数下的扩展曲线。

这个实验要特别关注反常情况。线程少时，全局 atomic 可能足够快，分层结构反而增加合并成本；事件类型很多时，每线程数组内存变大，缓存命中下降；热点极强时，局部聚合收益明显；NUMA node 很少或移动设备上拓扑简单时，复杂放置策略没有收益。教材要教读者解释这些差异，而不是给出固定答案。

Linux 源码阅读连接点 读 Linux NUMA 和调度相关代码时，关注任务迁移、CPU mask、内存策略和 page fault 后的放置。内核不会把“线程”当作孤立执行单元，它一直在处理任务、CPU、内存节点和负载之间的关系。读 RCU、per-cpu 变量和 slab 分配器时，也能看到把共享状态拆到 per-cpu 或局部缓存的思想。它们不是为了炫技，而是为了减少高频共享写和锁争用。

把这些思想带回 C++ 项目，读者应该能设计三层统计结构：worker-local、node-local、global。worker-local 追求无共享和 cache 热；node-local 负责同节点合并；global 只在阶段边界出现。这个结构会在后续并行归约、任务运行时和分布式 combine 中反复出现，是第二册的重要主线。

事故复盘：全局 atomic 为什么把拓扑拉进来 一个全局 atomic 计数器在四个线程下看起来还能接受，线程数继续增加后吞吐突然停住。代码里只有一次 fetch add，表面上比加锁短很多；硬件里却发生了 cache line 所有权在核心之间来回迁移。若线程跨 NUMA 节点，迁移还要经过互连。所谓共享内存并不是没有位置，cache line 的 owner、页的 home node、线程运行在哪个 CPU，都会进入成本模型。

这个事故能推出分层 partial。先让每个 worker 写自己的局部计数，避免热 cache line 在核心间抖动；再按 NUMA node 合并，减少跨节点流量；最后由一个线程汇总全局结果。这个结构改变了写路径，也改变了报告方式：不能只看最终计数，要记录每层 partial 的数量、合并时机、线程绑定和页面放置。Atomic 不是低级错误，它适合低频状态或线性化点，不适合高频共享写热点。

实验要把拓扑写进结论。线程数、CPU 列表、NUMA node、内存首次触碰位置、绑定策略都要可复现。若关闭绑定后结果波动很大，说明调度迁移已经影响 cache 热度；若本地 partial 快但最终合并长尾，说明瓶颈迁移到了归约阶段；若远端内存访问占比高，说明 first-touch 或分配策略有问题。NUMA 章不是让读者背拓扑名，而是让共享状态重新获得物理位置。

共享写实验实验判读 全局 atomic、线程本地 partial 和 NUMA 分层版本的结果要放在同一张扩展曲线里看。线程数少时全局 atomic 不一定差，线程数多后若吞吐停住，就检查 cache line 迁移和合并阶段。padding 后变快说明 false sharing 可能存在；padding 后不变，说明瓶颈可能不在同一行写。NUMA 结论必须写机器拓扑，不能从普通单节点设备外推。

这一段也服务于复习。读者合上书后，应该能凭本章的判读口径重新审查自己的代码：有没有最小正确版本，有没有能击穿旧方案的输入，有没有状态记录，有没有资源指标，有没有失败后的恢复动作。若五个问题缺两个以上，就说明实现还停在演示层，而不是工程层。

从 false sharing 反推结构体布局 false sharing 的教学案例可以从 worker 统计开始。每个 worker 只写自己的 completed、failed、stolen 计数，看起来没有共享；如果这些字段在数组里连续排列，多个 worker 可能写同一条 cache line。修复不是给所有结构体都 padding，而是先确认哪些字段被不同线程高频写，再只隔离这些字段。

报告里应画出逻辑字段和 cache line 的关系。字段 A 和字段 B 在源码上相邻，不代表它们应该共享缓存行；字段 C 和字段 D 虽然属于同一对象，但若一个只初始化时写、另一个每个任务都写，热度不同也应分开。布局不是按面向对象归属决定，而是按写者、频率和共享粒度决定。

NUMA 版本继续把这个思想放大。每个 node 内先合并局部 partial，跨 node 只在阶段边界交换汇总。若工作窃取跨 node 发生，要记录被偷任务访问的数据是否仍在远端。这样读者能看到单条 cache line 的所有权迁移和跨节点内存访问是一条连续问题，而不是两个独立概念。

实验课：从一个全局计数器走到分层 partial 第一版让所有 worker 对一个全局 atomic 计数器做加法。第二版改成每个 worker 一个计数器，最后串行合并。第三版给每个 worker 计数器加 cache line 隔离。第四版按 NUMA node 建 node local partial，再做全局合并。第五版把合并改成树形，避免单个线程长时间处理所有 partial。

每个版本都要在不同线程数下画扩展曲线，并记录每个 worker 的任务数、写入次数、合并时间和最终 checksum。若有工具支持，再记录 cache-to-cache 或 LLC 相关事件；没有工具时，也可以用 false sharing 对照实验：相邻计数器和隔离计数器在相同输入上的差异。实验的目标不是证明某个技巧永远快，而是观察共享写频率怎样改变系统形状。

判读时要合并阶段单独列出。线程本地 partial 常常让热路径变快，但增加最后合并；padding 可能减少 false sharing，但增加内存占用；NUMA 分层可能减少远端访问，但如果任务负载不均，跨节点窃取仍会出现。报告要写成成本迁移：从共享写迁移到合并，从全局争用迁移到局部内存，从计算路径迁移到调度路径。

这个实验还要连接分布式思想。Worker local partial 对应线程本地聚合，node local partial 对

应节点内合并，最终 global merge 对应集群 reduce。读者如果理解这个层次，后面看到 map side combine、shuffle partition 和 reduce tree 时，就不会把它们当成新概念。

共享写练习验收重点 合格答案要画出共享 cache line 或共享页面。只说“用线程本地变量”不够，必须说明全局写如何变成局部写和阶段合并，padding 保护了什么字段，first touch 怎样影响页面位置。没有 NUMA 机器时，答案要诚实写只能验证 false sharing 和扩展曲线。

答案还要写“为什么不采用更复杂方案”。例如没有证据证明队列是瓶颈，就不应把锁队列换成无锁；没有证明输出顺序可变，就不应随意重排；没有证明错误路径能恢复，就不应只提交正常路径。能解释不做什么，说明读者开始具备工程判断。

最后，答案必须包含可复查材料。至少有输入摘要、关键状态、期望输出、实际输出、运行命令和未验证边界。没有这些材料，老师或同伴无法判断结论是否成立。第二册的作业不是让读者写漂亮段落，而是训练读者把系统问题变成可验证证据。

项目实施：NUMA、互连与缓存一致性 本章对应的贯穿项目实施不要求一次写成完整框架，但必须有最小可运行切片。这个切片包含三个部分：一个 reference，一个带本章机制的实现，一个能击穿 reference 之外隐含假设的测试。Reference 用来固定语义，机制实现用来展示本章知识，测试用来证明新增复杂度确实解决了问题。

代码组织上，不要把所有内容放进一个大文件。基础类型、输入生成、核心机制、报告输出和测试应分开。即使当前只是教学项目，也要让目录能表达责任边界。这样后续章节接入时，不会因为找不到状态归属而重新发明一套对象。

每个实现都要输出一段机器可读摘要。摘要至少包含输入规模、关键状态、错误数、输出校验、耗时和未验证边界。这里的机器可读不要求复杂格式，简单的 key value 文本也可以。关键是让脚本和读者都能复查，而不是只能看屏幕截图。

测试应包含正常路径、边界路径和失败路径。正常路径证明功能可用；边界路径证明长度、容量、尾部、空输入、关闭或超时没有遗漏；失败路径证明本章机制不是装饰。若失败路径写不出来，往往说明本章概念还没有落到工程对象上。

报告最后写一个取舍段。新增机制带来了什么好处，增加了什么成本，哪些场景不应该使用它，下一章会复用哪个字段或对象。这个取舍段非常重要，因为系统工程不是把所有高级机制都加进去，而是在证据和约束下选择合适的边界。

调试笔记：扩展曲线停住时分离热路径和合并路径 多线程扩展曲线停住后，先看热路径是否还在共享写，再看合并路径是否变成长尾。全局 atomic 停住，可能是一致性流量；线程本地 partial 停住，可能是内存带宽或合并成本；NUMA 版本停住，可能是跨节点偷任务或远端页面。每个解释都需要不同证据。不要把所有扩展失败都归因于线程太多。

复查清单：NUMA、互连与缓存一致性 复查本章时，先检查 reference 是否仍然存在。没有 reference，后面的并发、优化、异步或分布式改造都无法证明语义没有变。再检查压力输入是否尖锐：它应当攻击本章最核心的边界，而不是只把数据量放大。第三，检查状态是否可观察：关键对象的身份、状态、错误和提交结果应能在报告中找到。

第四，检查代码示例是否说明禁止行为。调用者不能做什么，往往比能做什么更能体现工程质量。第五，检查实验是否写出未验证边界；当前设备、权限或输入无法证明的结论，必须标注。第六，检查本章对象是否能被下一章复用，不能复用就说明边界还不稳。

完成这六项后，再看文字是否连贯。每个概念都应从问题推出，每个案例都应从朴素方案走到改进方案，每个实验都应能解释一个判断。若某段话放到其他章节也成立，就把它改成本章的具体对象、具体输入和具体失败。

4.8 工程案例：把多线程归约放回 NUMA 和一致性协议

多线程归约看起来很简单：把数组切成几段，每个线程算一段，最后合并。可是在线程数增加到跨 socket 或跨 NUMA 节点后，性能可能突然停住甚至下降。原因往往不在加法本身，而在页面放置、cache line 所有权、一致性流量、互连带宽和线程迁移。NUMA 章要把“共享内存”从抽象地址空间放回真实拓扑。

一、共享地址空间不等于同等距离 在 NUMA 机器上，所有线程看见同一虚拟地址空间，但物理页有位置。线程访问本地节点内存和远端节点内存，延迟和带宽不同。若主线程单线程初始化整个数组，页面可能集中在一个 NUMA 节点；随后多个节点上的 worker 并行读取时，远端访问会淹

没计算。这个问题在小机器上不明显，在多 socket 服务器上非常常见。

first-touch 策略说明页面位置常由首次写入决定。因此并程序不仅要并行计算，还要按最终访问计划并行初始化。若每个 worker 将来处理某个块，就让它先触碰这个块的页面。这样内存放置和执行位置一致。这个原则比背 NUMA API 更重要：数据在哪里，线程就尽量在哪里执行。

二、线程本地 partial 同时解决共享写和远端写 全局原子计数器在多核下有两个问题。第一，所有线程写同一 cache line，造成一致性迁移。第二，若该 line 常驻某个节点，其他节点写入还要跨互连。线程本地 partial 把写入分散到每个线程或每个 NUMA 节点的本地内存，热路径只写本地 line，最后再合并。它不是单纯算法技巧，而是拓扑感知的数据放置。

partial 的层次也要匹配机器。每线程 partial 可以避免线程之间写冲突，但合并时可能跨节点读很多小对象；每 NUMA 节点 partial 先在节点内合并，再跨节点做一次最终合并，往往更符合拓扑。项目中的事件计数可以按 worker、NUMA node、全局三层组织。报告要写清每一层的写入频率和合并频率。

三、一致性协议保护的是 cache line 所有权 当多个核心读同一份只读数据时，一致性成本很低；当多个核心写同一条 cache line 时，所有权必须不断迁移。锁、原子计数、队列 head/tail、引用计数都可能成为这种热点。性能曲线的典型形状是：少量线程很快，线程继续增加后吞吐不升，甚至因为互连流量下降。这个现象不能靠增加核心解决，因为瓶颈已经在协调。

减少一致性流量有几种方式。第一，减少共享写，例如 partial 和批量合并。第二，隔离热点 line，例如 padding 和 alignas。第三，改变协议，例如从全局队列改成本地队列加窃取。第四，降低实时一致要求，例如监控计数周期性汇总。每种方式都在用空间、延迟或复杂度换少量共享写。

四、拓扑感知队列不是越本地越好 任务运行时常把队列做成本地 deque，以保留 cache 和 NUMA 本地性。可是完全不窃取会让某些 worker 空闲，完全随机窃取会破坏本地性。较好的策略通常是先本地，再同 NUMA 节点，再跨节点。任务粒度也影响选择：大任务跨节点迁移成本可能可以接受，小任务迁移成本可能超过计算。

Compute Systems Engine 可以在任务 trace 里记录 worker id、NUMA node、输入块所在节点和是否窃取。这样当某个阶段不扩展时，读者可以看是本地队列不均、跨节点窃取过多，还是页面放置错误。没有拓扑 trace，NUMA 问题很容易被误判为算法不够快。

五、Linux 观察入口 Linux 提供 numactl、lscpu、/proc/self/numa_maps、perf c2c 等观察入口，具体可用性取决于环境和权限。学习时不必一次掌握所有工具，但要知道每个工具回答什么问题。lscpu 看拓扑，numactl --hardware 看节点和距离，numa_maps 看映射页分布，性能计数器看远端访问和 cache line 争用。

如果设备没有 NUMA，例如很多手机或单 socket 机器，也能做降级实验。用线程绑定和分片 partial 观察 false sharing，用首次触页和冷启动观察 page fault，用本地队列和全局队列观察锁竞争。报告必须写明“当前设备无法验证远端 NUMA 访问”，但仍然可以学习拓扑意识。

六、C++ 结构表达拓扑 拓扑信息不要散落在命令行字符串里。项目可以定义 TopologyPlan：worker 数量、每个 worker 所在逻辑 CPU、NUMA node、负责的输入块、partial 存放位置。算法不一定直接操作 NUMA API，但报告和运行时可以使用这个计划解释任务放置。这样从一开始就把硬件拓扑纳入工程对象。

Listing 4.3 拓扑计划记录 worker 和数据块的放置

```
1 struct WorkerPlacement {
2     std::uint32_t worker_id = 0;
3     std::uint32_t logical_cpu = 0;
4     std::uint32_t numa_node = 0;
5     std::uint64_t input_begin = 0;
6     std::uint64_t input_end = 0;
7 };
```

这个结构不承诺一定能绑定成功。绑定是操作系统能力，可能被权限、容器或移动设备限制。计划和实际观测要分开记录：计划希望 worker 在哪里，实际运行时是否迁移，输入页是否真的落在对应节点。高质量报告不会把愿望当事实。

七、实验交付：分层归约 本章实验从数组归约开始。版本一，单线程 reference。版本二，多个线程直接写全局原子。版本三，每线程 partial。版本四，每 NUMA 节点 partial 后全局合并。版本五，加入并行 first-touch。每个版本记录总耗时、合并耗时、线程数、绑定策略、页面放置证据和 checksum。

预期结果不是固定数字。小数组可能单线程最快；单 socket 机器上 NUMA 分层收益不明显；线程太多会被内存带宽限制；绑定失败时结果可能波动。教材要让读者学会解释这些情况，而不是追求一条漂亮的线性加速曲线。NUMA 学习的目标是知道什么时候拓扑是主因，什么时候不是。

4.9 源码阅读：从调度域和内存策略理解拓扑

NUMA 和一致性协议不只存在于硬件手册。操作系统也在用拓扑信息做调度和内存策略。源码阅读时不需要读完整调度器，而要抓住几个概念：CPU 拓扑如何组织，任务迁移为什么影响 cache，内存策略如何决定页面放置，自动 NUMA balancing 为什么可能改变实验结果。这样读者能把“线程跑在哪里”和“数据放在哪里”连起来。

一、调度域说明迁移不是免费动作 Linux 调度器会按拓扑组织 CPU，例如 SMT sibling、同核心、同 socket、不同 NUMA 节点。负载均衡会在这些层次之间迁移任务。迁移可以让空闲 CPU 得到工作，但也可能让任务失去 cache 热度，访问远端内存。调度器不是不知道局部性，而是在负载均衡和局部性之间取舍。

项目里的任务运行时也面对同样取舍。本地队列保留局部性，但可能负载不均；跨节点窃取改善利用率，但增加远端访问。读调度域的价值，是理解成熟系统也在做分层决策，而不是简单“永远绑定”或“永远迁移”。

二、内存策略影响页面归属 内存策略可以是默认 first-touch，也可以是绑定到某节点、交错分配或优先本地。不同策略适合不同工作负载。只读大数组按访问线程分块，first-touch 很自然；共享读多写少数据可能适合 interleave；强本地队列和 partial 适合绑定。错误策略会让线程和数据距离变远。

实验时要记录策略。若用了 `numactl--interleave`，就不要再把页面分散归因于 first-touch；若容器限制了 CPU 和内存节点，结果也会不同。系统书要让读者养成记录环境的习惯。

三、自动 NUMA balancing 是隐藏变量 某些 Linux 配置会周期性采样页面访问并迁移页面，让内存更接近访问线程。它可能改善长期运行程序，也可能让短 benchmark 波动。若实验中页面位置随时间变化，第一次运行和稳定运行结果不同，就要怀疑自动 NUMA balancing 或回收策略。报告可记录相关系统设置，至少说明没有验证。

这提醒读者：硬件拓扑不是静态背景，操作系统会主动调整。性能实验不能只看代码，也要看系统策略。

四、拓扑章节的复核问题 读完本章，面对多线程程序应能回答五个问题。线程是否固定或可能迁移；数据页由谁首次触碰；共享写是否集中到少数 cache line；任务队列是否跨节点窃取；合并阶段是否把远端数据集中到一个线程。每个问题都有证据入口，而不是凭感觉判断。

4.10 补强案例：一致性流量如何在锁和队列中放大

一致性协议不只影响原子计数器，也影响锁和队列。一个互斥锁变量本身是一条 cache line，等待线程反复读写它；队列的 head、tail、size 若放在同一 line，不同线程会争同一所有权；条件变量唤醒后，多个线程又会竞争同一锁。理解这些细节，才能解释为什么低竞争锁很好，高竞争锁突然很差。

一、锁变量也是共享写热点 获取锁通常要修改锁状态。多个核心竞争同一锁时，锁所在 cache line 在核心之间迁移。临界区越短，锁变量本身的迁移占比越高；临界区越长，等待和上下文切换占比越高。两种慢法不同，优化方向也不同。短临界区高竞争可以分片或减少共享；长临界区要缩短锁内工作。

这解释了为什么“锁次数”不是唯一指标。一个锁每秒被获取很多次但几乎无竞争，可能没问题；一个锁每秒次数不多但每次持有很久，尾延迟很差。报告要记录等待时间和持有时间。

二、队列元数据要分离读写者 生产者主要写 tail 和 buffer 槽位，消费者主要写 head。若 head、tail、size 放在同一 cache line，生产者和消费者会互相干扰。SPSC ring 常把 head 和 tail 分开 padding；MPMC 队列还会让每个槽位有序号，分散状态。布局不是装饰，而是减少一致性所有权迁移。

带锁队列也能受益于布局。锁、条件变量、容量、统计指标、热 head tail 和冷诊断字段不应随意堆在一起。监控计数器频繁更新时，也可能和锁变量 false sharing。队列结构体要按访问者和频率组织。

三、分片锁降低协调粒度 哈希表、任务表和计数器都可以用分片锁。每个 shard 有自己的锁和局部状态，线程只竞争相关 shard。这样减少一致性流量，但引入跨 shard 操作复杂性。例如需要全局快照、迁移 key 或关闭全部 shard 时，要按顺序获取多个锁或使用阶段协议。

分片锁不是免费午餐。Shard 太少，竞争仍高；shard 太多，内存和遍历成本上升；key 倾斜会让某个 shard 仍然热点。报告应记录每个 shard 的请求分布，而不是只看总吞吐。

四、拓扑实验 实现全局锁队列、分片队列和每 worker 本地队列三版。输入包含均匀任务和热点生产者。记录锁等待、cache line 争用替代指标、队列长度和任务完成时间。若没有 cache-to-cache 工具，也可以通过扩展曲线和等待时间推断。目标是让读者看到一致性流量如何从硬件概念变成锁和队列的工程问题。

4.11 补充案例：分层合并如何减少跨节点流量

多 NUMA 节点归约时，如果所有线程把 partial 发送到一个全局合并线程，跨节点流量集中，最后阶段成为瓶颈。分层合并先在每个节点内部合并，再跨节点合并少量结果。这个结构和树形 reduce、map-side combine、分布式聚合是一条线。

一、节点内合并 同一 NUMA 节点内的 worker 把 partial 合并到 node local partial。这个阶段尽量使用本地内存和本地队列。若 worker 被迁移或 partial 分配在远端，收益会下降。因此分层合并依赖线程放置和 first-touch。

二、跨节点合并 跨节点合并只处理每个节点的摘要，而不是每个线程的全部细节。这样减少互连字节和 cache line 迁移。代价是增加一层合并逻辑和内存。若节点内数据极不均衡，某个节点仍可能长尾。

三、和分布式聚合对应 节点内合并对应 worker local combine，跨节点合并对应 reduce。分布式系统中，map-side combine 减少网络字节；NUMA 系统中，node local combine 减少互连字节。尺度不同，原则相同：先在便宜通信范围内合并，再跨昂贵边界传摘要。

四、实验 模拟两层合并，即使在单 NUMA 机器上也可用线程组模拟。比较全局合并、每线程直接全局 atomic、分组 partial 后合并。记录合并时间和扩展曲线。若有 NUMA 机器，再加入节点绑定。实验目标是理解分层思想，不是依赖某台机器的具体数字。

4.12 从共享变量推导拓扑感知设计

多核程序常把共享内存看成一个均匀的大空间，仿佛任意核心读写任意地址的成本都一样。真实机器不是这样。每个核心有私有缓存，多个核心通过一致性协议维护同一地址的可见性；多个 socket 或多个 NUMA 节点之间还有不同距离的内存控制器和互连。于是“共享一个变量”这件小事，会把 cache line 所有权、页面归属、远端访问和合并策略全部牵出来。NUMA 章不应从名词开始，而应从一个扩展曲线突然停住的共享计数器开始。

一、共享写的瓶颈是所有权来回移动 多个线程同时递增同一个计数器，看起来每次只做一次加法。硬件上，这条 cache line 必须在核心之间转移独占所有权。线程越多，算术工作没有增加多少，一致性流量却快速增加。把计数器改成每线程本地 partial 后，热路径变成各自写本地 cache line，最后再合并。这个改法的本质不是“减少锁”，而是减少同一条 cache line 的所有权争夺。读者理解了这一点，后面看到 per-CPU 统计、分层归约、map-side combine 时，就能认出同一个结构。

二、false sharing 是布局层面的共享写 两个线程写不同变量，也可能争同一条 cache line。最容易出错的例子是数组中的相邻计数器：逻辑上每个线程有自己的槽位，物理上多个槽位可能在同一条 cache line 里。padding 可以解决，但 padding 不是免费午餐，它增加内存占用，可能降低 TLB 覆盖，也可能让扫描阶段搬运更多数据。工程上应先确认写热点确实落在同一条线，再决定是否对齐、填充或改成分层布局。不能看到并发就无脑 padding。

三、页面归属来自第一次触碰和后续迁移 NUMA 成本常被隐藏在初始化代码里。主线程分配并清零大数组，页面可能归属主线程所在节点；后续 worker 分布在其他节点上处理这些页面，就会产生远端访问。first touch 策略要求谁将来主要使用页面，谁就先触碰页面。若任务会迁移，页面和

线程的关系又会变化，自动 NUMA 平衡可能迁移页面，也可能在短作业中来不及发挥作用。报告要记录线程绑定、页面初始化方式和节点内存分布，否则 NUMA 结论没有可复现性。

四、分层合并把拓扑写进算法 拓扑感知设计的常见形态是本地处理、节点内合并、跨节点合并。比如统计日志时，每个线程先写本地计数；同一 NUMA 节点上的线程先合并成本节点结果；最后再跨节点合并。这样做的目的不是让代码看起来复杂，而是让高频写停留在近处，让低频合并承担远端成本。若输入极小，分层结构会得不偿失；若数据巨大且线程很多，分层结构可能决定扩展上限。算法是否需要拓扑感知，要由数据规模、共享写比例、远端访问成本和调度稳定性共同决定。

五、把扩展曲线分解成几段看 评估多核程序时，不要只画线程数到总耗时的曲线。还要把耗时拆成读取、计算、本地合并、跨节点合并和写出；把指标拆成本地内存访问、远端内存访问、cache line 争用和调度迁移。若一到八线程扩展很好，十六线程以后停住，可能是内存带宽饱和，也可能是共享写热点，也可能是跨 NUMA 访问，也可能是合并阶段串行化。只有把曲线拆段，才能知道下一步该改布局、改绑定、改合并树，还是承认机器资源已经到顶。

4.13 NUMA 实验如何写成可复现材料

NUMA 相关问题很容易因为机器差异而变得难以复现，所以实验设计比结论本身更重要。一个合格实验不能只说“绑定以后变快”，而要写明机器拓扑、线程绑定、内存初始化、输入大小、访问模式、合并策略和测量工具。即使读者的手机或开发机没有多 socket，也可以用同一套报告格式训练拓扑意识：哪些数据靠近哪个执行实体，哪些共享写会跨核心移动，哪些合并阶段会形成串行瓶颈。

一、拓扑先记录，不要事后猜 实验开始前先记录核心数、逻辑线程数、缓存层级、NUMA 节点和内存分布。Linux 上可以用 `lscpu`、`numactl--hardware`、`/sys/devices/system/node` 等信息；受限环境中至少记录可见 CPU 数和调度限制。没有拓扑材料，后面看到性能曲线变化时只能猜。经典系统教材读起来舒服，原因之一就是它会先把实验背景交代清楚，让读者知道结论站在哪块地上。

二、初始化方式必须和执行方式配对 大数组如果由主线程统一初始化，页面很可能集中在主线程所在节点；若后续多个 worker 跨节点访问，就会把远端访问带入主路径。实验应比较三种初始化：主线程初始化、worker 按分片 first touch、交错分配。每种初始化都要配同一份计算逻辑。这样读者能看到“同一算法、同一线程数”也会因页面归属不同而产生差异，NUMA 不是额外玄学，而是内存所有权的一部分。

三、共享写和远端读要分开测 一个扩展曲线停住，可能是共享计数器在争 cache line，也可能是远端读占用互连，也可能是最后合并阶段串行化。实验要设计三个版本：只读本地数据、写线程本地 partial、写共享全局计数器。只读版本观察带宽和放置，partial 版本观察本地写和合并，全局计数器版本观察一致性流量。三个版本的差异会比单个“优化前后”更能说明问题。

四、合并阶段要单独计时 分层归约常让主计算阶段变快，却把成本移动到合并阶段。报告应把 worker 本地处理、节点内合并、跨节点合并和最终写出拆开计时。若节点内合并很快，跨节点合并很慢，说明拓扑设计有效但最后聚合仍需优化；若本地处理时间差异巨大，说明负载切分比 NUMA 更紧急。没有分段计时，优化很容易把成本藏到结尾。

五、没有 NUMA 机器也能学习同一思想 单 socket 机器上也有缓存层级、核心迁移和共享 cache line。可以用线程绑定、padding、线程本地 partial 和合并树训练同一套思路。等迁移到多节点机器时，只是把“近”和“远”的距离拉大。教学不应把 NUMA 作为少数服务器才需要的知识，而应把它视为局部性、所有权和拓扑成本的自然延伸。

4.14 贯穿案例：全局计数器怎样一步步变成分层归约

本章最适合从一个很小的错误开始：日志统计程序用一个全局 atomic 记录已处理行数，每处理一行就执行一次 `fetch_add`。单线程正确，两线程也正确，线程数继续增加后吞吐几乎不涨，甚至下降。这个事故不需要先背 NUMA、互连、一致性协议这些词。先看事实：所有核心都在高频写同一条 cache line。每次写都要获得该 cache line 的独占权，其他核心手里的副本失效，下一次写又把所有权迁走。算术操作很小，所有权迁移很大。

从这个事故出发，分层设计自然长出来。第一层，每个线程写自己的局部计数器，让高频写停

在本核心附近。第二层，同一个 NUMA 节点内先合并，减少跨节点流量。第三层，再做全局合并，把远端成本降到低频阶段。这个过程不是套用“map reduce”模式，而是根据 cache line 所有权和拓扑距离推导出来的算法结构。

一、atomic 解决数据竞争，不保证扩展性 全局 atomic 的语义是正确的：没有数据竞争，最终值准确。但正确性和扩展性不是同一件事。Atomic 给出了线性化点，代价是所有线程争一个写位置。若这个线性化点发生在每条记录上，扩展性会被一致性协议吞掉；若它发生在每个任务完成时，频率低得多，代价可能可以接受。判断 atomic 是否合适，要先问它在热路径出现多少次，而不是看到 atomic 就删除。

二、本地 partial 需要处理读取时刻 把计数器拆成每线程 partial 后，热写变快了，但“什么时候读取全局值”变复杂了。阶段结束后合并很简单；运行中实时展示进度，则需要读多个 partial 并求和，结果可能只是一个近似快照。若业务要求严格单调进度，就要定义读取协议。这个细节说明优化不会只改变性能，也会改变可见状态的语义。教材必须把这种交换讲出来。

三、padding 要跟访问模式一起设计 每线程 partial 如果存在数组中，相邻元素可能落在同一 cache line，导致 false sharing。给每个 partial 对齐到 cache line 可以解决写写争用，但内存占用变大，合并扫描时也会跨更多 cache line。若 partial 很少且写频率高，padding 合理；若 partial 很多且写频率低，padding 可能浪费。设计不应是“所有结构都 padding”，而是先标出高频写字段，再对这些字段隔离。

四、first touch 把初始化也纳入算法 局部 partial 不只是谁写的问题，还包括页面归属。若主线程创建并清零所有 partial，页面可能集中在主线程所在节点；后续 worker 即使写自己的 partial，也可能写远端页面。更好的做法是让使用该 partial 的 worker 完成初始化，或者按 NUMA 节点批量初始化。初始化不再是无关准备工作，它决定页面第一次落在哪里。报告若不写初始化方式，NUMA 结论就不可复现。

五、合并树要和拓扑对应 最终合并可以由一个线程扫所有 partial，也可以做树形合并，也可以按 NUMA 节点先合并。本地合并减少远端读，树形合并减少单点串行时间，但会增加同步阶段。输入小的时候，复杂合并树不划算；输入大且线程多时，它能避免单线程尾部成为瓶颈。这个选择要通过分段计时判断：本地处理、节点内合并、跨节点合并分别占多少。

六、把拓扑信息放进 trace 当一次运行突然变慢，trace 里如果只有线程 id，很难判断原因。更好的记录包括 worker id、CPU id、NUMA node、partial 地址范围、初始化线程、合并阶段和迁移次数。这样报告能回答：慢线程是否跑到远端节点，页面是否集中在一个节点，某个 partial 是否和其他线程 false sharing。拓扑信息不是低级细节，而是解释多核性能的证据。

4.15 案例深化：共享计数器的分层改造

NUMA 和一致性协议最适合通过共享计数器讲清楚。朴素版本让所有线程执行 `fetch_add` 更新全局计数。它语义清楚，也能避免数据竞争；但线程数上来以后，cache line 所有权在核心之间反复迁移，互连流量上升，吞吐停住。这个事故能把 atomic、cache line、NUMA、线程绑定和局部归约串起来。

一、不要先否定 atomic Atomic 的问题不是“慢”，而是不适合高频共享写热点。低频状态、线性化点、任务完成计数都可能适合 atomic。教材要让读者先判断频率和共享范围：每条记录一次的计数是高频路径，适合局部 partial；每个 task 完成一次的状态更新频率低，可以用 atomic 或锁。把所有 atomic 都视为错误，会让工程判断变粗糙。

二、第一层拆到线程本地 线程本地 partial 让热写入落在各自 cache line 上。每个 worker 扫描自己的输入，更新自己的计数表，阶段结束后再合并。这个结构减少一致性流量，但增加合并阶段。若 key 很多，线程本地表会占用更多内存；若 task 很小，合并成本可能超过收益。报告要同时写扫描时间、合并时间和内存峰值。

三、第二层按 NUMA 节点合并 跨 socket 合并比同节点合并更贵。可以先在每个 NUMA node 内合并，再由一个节点汇总全局。页面首次触碰也要配合：哪个 worker 初始化 partial，页面就更可能放在哪个节点。若初始化线程和使用线程分离，可能出现远端访问。实验要记录绑定策略和 first touch 策略，否则数字很难复现。

四、避免 false sharing 即使每个线程有自己的计数，如果多个计数结构落在同一 cache line，

也会发生 false sharing。结构体数组尤其容易出现这个问题。解决方法可以是按 cache line 对齐、填充或改成每线程独立分配。报告里不要只写“用了 thread local”，还要说明 hot counter 是否可能共享 cache line。这个细节能把一致性协议从抽象概念落到 C++ 对象布局。

五、把拓扑写进接口 最终项目可以把 worker id、CPU id、NUMA node、local partial id 写进 trace。这样一次异常慢的合并能追到具体拓扑。接口不一定强制绑定，但要允许记录和控制。高性能计算不是假设机器均匀，而是在接口中保留拓扑信息，让调度器和报告有能力使用它。

4.16 案例复盘：一个 atomic 怎样拖出整台机器

所有线程对同一个 atomic 计数器执行 fetch add，代码很短，语义也正确。线程数增加后吞吐停住，是因为同一条 cache line 的所有权在核心之间迁移；跨 NUMA 节点时，还要经过互连。这个案例让读者看到，共享内存并不意味着没有位置，cache line、页和线程都在拓扑里。

分层 partial 的推导 第一步把高频写拆到线程本地，第二步按 NUMA 节点合并，第三步再全局汇总。这样热写入留在本地 cache line 上，跨节点流量只发生在合并阶段。Atomic 仍可用于低频状态和线性化点，但不适合每条记录一次的共享热点。

实验判据 报告写线程绑定、first touch、NUMA node、线程数扩展、合并时间和远端访问比例。若本地 partial 快而合并慢，瓶颈只是迁移到下一阶段；若绑定关闭后波动大，调度迁移已经进入成本模型。拓扑信息必须成为 trace 的一部分。

4.17 本章习题

习题与作业

习题一：把一个全局原子计数器、每线程计数器、每 NUMA 节点计数器画成三张数据共享图。分别标出哪些 cache line 被谁写，哪些阶段需要合并，读取全局值的成本在哪里。

习题二：设计一个 NUMA first-touch 实验。要求写清初始化线程、计算线程、绑定策略、输入规模、测量指标和预期现象。若当前设备没有 NUMA，写出哪些部分只能作为计划保留。

习题三：为线程池设计一个两级队列方案：worker 本地队列加全局溢出队列。说明高频路径、低频路径、锁保护对象、可能的窃取顺序和关闭语义。

习题四：阅读 Linux NUMA 策略或调度拓扑的一个窄入口。报告必须从一个用户态实验问题出发，不允许只复制函数名。

第零部分

单机高性能计算：从测量到数据流

可信测量、Roofline 与性能计数器

先看一个常见争论：同一个归约优化，有人测出三倍加速，有人测出几乎没提升。两边都没有撒谎，只是测的不是同一个对象。一个人把初始化、首次缺页和输出校验都放进计时区，另一个人只测热循环；一个人在小数组上测依赖链，另一个人在大数组上已经打满内存带宽；一个人固定线程，另一个人的线程在 NUMA 节点之间迁移。没有测量边界，所有性能结论都会变成口水仗。

本章从这种“数字互相矛盾”的场景出发。可信测量不是为了仪式感，而是为了知道一个改动到底作用在哪个瓶颈上。Roofline 帮我们判断当前内核是否已经被字节移动限制；性能计数器帮我们区分前端、后端、缓存、TLB、分支、同步和调度；对照实验帮我们排除错误解释。读者要学会先定义测量对象，再解释数字。

5.1 从问题到测量边界

测量前先定义对象。是测单次操作延迟，还是测批量吞吐；是包含内存分配，还是只测热循环；是冷缓存，还是热缓存；是单线程，还是多线程；是平均值，还是尾延迟；是关心总吞吐，还是关心每个 worker 是否均衡。不同对象需要不同方法，不能混用结论。

常见错误包括：把初始化时间算进内核，把编译器优化掉的循环当作结果，把首次缺页当作算法成本，把输出打印放进计时区，把不同 CPU 频率状态下的结果直接比较。

第二册还要特别避免一种错误：只测总时间，不看扩展曲线。多线程程序的核心问题不是“某一次运行快不快”，而是从 1 个线程到多个线程时，吞吐如何变化。一个程序在 2 个线程时加速，在 8 个线程时停滞，在 16 个线程时变慢，这条曲线本身就是证据。它提示瓶颈可能从单核执行转移到了内存带宽、锁竞争、调度、NUMA 或 I/O。

计时器和统计方法 计时器应使用单调时钟，避免系统时间调整影响。短任务要重复执行足够多次，并防止编译器消除结果。报告结果时不要只给一次运行时间，应至少给出多次运行的最小值、中位数和波动范围。最小值常代表干扰较少的情况，中位数更接近日常情况，尾部异常则提示系统噪声或资源竞争。

预热同样重要。第一次运行可能包含页错误、动态链接、分支预测冷启动、缓存冷启动和 CPU 频率爬升。若目标是冷启动延迟，就应该保留这些成本；若目标是热循环吞吐，就应该把它们排除。测量对象不同，预热策略不同。

多核 benchmark 还要记录线程绑定和运行顺序。若每轮测试先跑单线程再跑多线程，CPU 温度、频率和缓存状态可能系统性变化。更稳妥的做法是随机化版本顺序，保留原始样本，并记录每轮线程数、绑定策略和输入位置。若差异小于系统波动，就不要写强结论。

5.2 从字节数到 Roofline 和扩展曲线

Roofline 用算术强度连接计算峰值和内存带宽。算术强度是操作数与数据移动字节数的比值。低算术强度程序通常受内存带宽限制，高算术强度程序才可能接近计算峰值。这个模型不是精确预测器，而是帮助判断优化方向。

数组求和每读取一个整数只做一次加法，算术强度低；矩阵乘如果分块得当，每个元素可以被多次复用，算术强度高。若一个内核已经接近内存带宽上限，继续手写更复杂的算术指令可能没有意义；若内核远低于计算上限且数据复用充分，就要检查 SIMD、指令级并行和执行端口。

Roofline 的价值在于先给出“最多可能多快”的粗上限。假设一个内核每处理一个元素读取 8 字节、写入 8 字节，只做少量整数运算，那么它的算术强度很低。若平台可持续内存带宽是 50 GiB/s，那么不管循环写得多漂亮，只要每个元素必须从内存读写 16 字节，吞吐上限就被数据移动限制。反过来，若内核通过分块让同一批数据被重复使用，算术强度提高，才有资格讨论 FMA、SIMD 宽度和执行端口。

心智模型

Roofline 不是为了画漂亮图，而是为了阻止错误优化：内存带宽瓶颈不要只改算术指令，计算端口瓶颈不要只改数据结构，同步瓶颈不要只改循环展开。

从字节数推导瓶颈 Roofline 的关键是先估算字节数。以 `std::int32_t` 数组求和为例，每个元素读取 4 字节，做一次整数加法。如果数组远大于缓存，主要流量来自内存读取。若机器可持续内存带宽是 40 GiB/s，那么理想情况下每秒最多读取约 100 亿个整数。实际还要扣除循环开销、预取效率、TLB 和其他系统干扰。

这个估算能提前判断优化方向。如果实测已经接近带宽上限，就不该期待换一种加法写法获得数量级提升。若实测只有带宽上限的一小部分，就要检查访问是否连续、是否触发 TLB miss、是否有分支、是否被编译器向量化、是否被线程调度干扰。

字节数估算要区分“算法逻辑字节”和“实际流量”。逻辑上，一个原子计数器每次只更新 8 字节；硬件上，它可能让一整条 cache line 在核心之间来回迁移。逻辑上，一个对象只读取一个字段；硬件上，cache line 可能带入多个冷字段。逻辑上，`mmap` 文件像数组；系统上，第一次访问可能触发缺页和 page cache 读取。第二册的实验报告必须说明这些差异，否则字节数估算会过于乐观。

5.3 计数器、profile 和 trace

Linux 上常用 `perfstat` 查看 cycles、instructions、branches、branch-misses、cache-misses、page-faults、context-switches 等指标。更深入时可以用 `perfrecord` 和 `perfreport` 定位热点。不同 CPU 的事件名称不同，实验报告要记录命令和平台。

计数器要组合解释。IPC 低可能因为 cache miss，也可能因为分支失败或同步等待。cache-misses 高不一定说明程序差，流式扫描大数组本来就会不断 miss，但可能充分利用带宽。context-switches 多可能说明线程过多、阻塞频繁或系统有干扰。

进阶测量更重要的是“假设驱动”。如果怀疑前端瓶颈，就比较指令缓存、分支、uops 供给和代码布局；如果怀疑内存带宽，就比较工作集大小、线程数、带宽工具和 LLC miss；如果怀疑 TLB，就改变页大小或访问页数；如果怀疑锁竞争，就看吞吐曲线、等待时间、futex 路径和临界区长度；如果怀疑 NUMA，就改变 first-touch 和绑定策略。不要一次性收集一大堆事件再事后找故事。

Top-Down 思维 现代性能分析常把 CPU 周期粗分为前端受限、错误预测受限、后端受限和有效退休。不同平台工具名称不同，但思路相同：先判断核心为什么没有持续退休有用指令，再进入更细分类。前端受限时检查取指、译码、代码布局和间接跳转；错误预测受限时检查分支输入分布；后端受限时继续区分执行端口、内存延迟、内存带宽和同步等待；有效退休比例高时，说明核心大部分时间确实在做有用工作，优化要回到算法或工作量本身。

这套思维可以避免把 IPC 低简单等同于“CPU 不行”。同样是 IPC 低，链表随机访问、锁等待、page fault、系统调用阻塞和分支乱跳的解决方向完全不同。报告里应该写“当前证据更支持哪一类受限”，而不是只列一个 IPC 数字。

同步和调度的测量 多线程程序经常慢在不退休用户态指令的地方。线程可能睡在 futex 上，可能被调度器切走，可能自旋等待 cache line，可能阻塞在 I/O，也可能因为 cgroup 配额用完而被节流。只看 cycles 和 instructions，容易忽略这些等待。

测同步要记录至少三类事实。第一，吞吐随线程数怎样变化；第二，等待在哪里发生，是用户态自旋、内核 futex、条件变量、I/O 还是队列为空；第三，竞争对象是什么，是一把锁、一个原子变量、一条 cache line、一个队列还是一个分片。若能使用 `perfsched`、`perflock`、`perfc2c` 或 eBPF 工具，应记录工具版本和权限；若不能使用，也要用对照实验替代，例如拆分计数器、增加 padding、改变线程绑定和改变队列容量。

多核扩展曲线 一个高质量性能报告不只给“优化前后快了多少”，还要给扩展曲线。横轴是线程数或 worker 数，纵轴可以是吞吐、耗时、加速比、效率或尾延迟。理想线性加速很少出现，重要的是解释拐点。若 1 到 4 线程接近线性，8 线程开始变慢，可能是共享缓存、内存带宽、NUMA 或同步热点；若从 2 线程开始就没有提升，可能是任务粒度太小、串行部分太大或全局锁太重。

扩展曲线还要配合正确性校验。并行程序在高线程数下更快但偶尔错误，不是优化成功，而是暴露了竞争。每轮性能样本都应关联校验结果，校验失败的样本不能混入性能统计后继续分析速度。

5.4 实验设计、统计和报告

一份合格性能记录至少包含：机器型号、核心和 NUMA 拓扑、内存容量、编译器和编译选项、输入规模、数据类型、线程数、绑定策略、运行次数、计时范围、正确性校验、perf 命令和原始输出摘要。缺少这些信息，别人无法复现实验，你自己几周后也很难判断数字是否可靠。

性能结论要写成果因果假设，而不是口号。例如“并行版本在线程数超过 8 后不再加速，perf 显示 LLC miss 和内存带宽接近平台上限，因此瓶颈更像内存带宽，而不是线程创建成本”。这样的结论可以继续被验证或推翻。

本册建议把报告分为五段。第一段写被测问题和正确性 oracle。第二段写机器、系统、编译和绑定环境。第三段写实验矩阵，包括输入规模、线程数、版本、运行轮次和顺序。第四段写原始结果摘要，不只给平均值，也给中位数、最小值和异常。第五段写归因假设和下一步验证。若证据不足，要明确写“当前只能排除某些解释，不能确认最终瓶颈”。

从问题到实验矩阵 真正的性能实验不是随手跑一个命令，而是把问题拆成可证伪的假设。以 Compute Systems Engine 的归约为例，用户观察到“并行版本没有线性加速”。这句话本身不能直接指导修改，因为可能原因太多：线程创建成本太高，任务粒度太小，单线程 baseline 已经被编译器向量化，输入超过内存带宽上限，partial 发生 false sharing，worker 被调度到同一个物理核心的 SMT sibling，主线程首次触页导致远端内存，或者 benchmark 把输入生成混入了计时区。

实验矩阵要把这些解释逐步分开。第一组只测单线程：baseline、四累加器、可能的 SIMD 版本，输入规模从 L1 可容纳、L2 可容纳、LLC 可容纳一直扩到内存。若小规模下四累加器有效、大规模下无效，说明瓶颈可能从依赖链迁移到内存。第二组固定算法，改变线程数，从 1 到物理核心数，再到逻辑 CPU 数。若到物理核心数后增长变慢，再使用 SMT 变差，说明资源共享或带宽上限值得检查。第三组固定线程数，改变 partial 布局，比较紧凑和对齐。若对齐版本明显改善，说明 false sharing 是关键证据。第四组改变初始化方式，比较主线程初始化和 worker first-touch。若 NUMA 机器上差异显著，说明页面放置参与了结果。

这样的矩阵看起来比“优化前后跑一下”繁琐，但它能防止错误归因。性能工程最怕的是改了十个变量后得到一个数字，然后把提升归功于自己最喜欢的解释。严肃实验宁愿一次只改变一个变量，哪怕多跑几组，也不要吧线程数、布局、输入、编译选项和绑定策略同时改变。

正确性 oracle 和防优化 性能实验必须有正确性 oracle。归约的 oracle 可以是顺序 reference；过滤计数的 oracle 可以是简单版本；并发队列的 oracle 不能只看最终数量，还要验证 FIFO 或线性化语义；分布式 shuffle 的 oracle 需要验证每个 key 的最终聚合值和重复请求处理。没有 oracle 的 benchmark 很危险，因为最快的程序可能只是算错了，或者被编译器优化掉了。

防止优化器删除计算也要讲清。常见错误是循环计算了结果但从未使用，优化器发现结果对程序可观察行为没有影响，于是删除整个循环。为了保留计算，可以把结果参与校验、打印摘要、写入 **volatile** sink，或者通过 benchmark 框架的接口告知编译器。更好的方式是把性能样本和 correctness check 绑定：每轮运行都返回结果，结果不对则丢弃样本并报告错误，而不是继续统计耗时。

但防优化不能破坏测量对象。如果把每轮结果都打印到终端，测到的就不是计算时间，而是 I/O 时间。如果把结果写入同一个全局原子，可能引入新的共享写瓶颈。如果把校验放进热循环，校验成本会污染内核。正确做法通常是：热循环只做被测工作，循环外做轻量校验，输出只写汇总，不在计时区打印每个元素。

预热、冷启动和稳态 预热不是形式主义。第一次运行可能包含动态链接、页面分配、minor fault、缓存冷启动、分支预测器冷启动和 CPU 频率变化。若目标是服务冷启动延迟，这些成本应保留；若目标是稳定吞吐，就应先预热再测。实验报告必须说明测的是冷启动、热循环还是混合流程。

稳态也不是永远稳定。CPU 温度升高可能降频，后台任务可能抢占，容器 CPU 配额可能周期性限流，内核写回线程可能突然工作，透明大页可能在后台整理内存。一个严肃报告应保留多次样本，而不是只给最漂亮的一次。若最小值和中位数差距很大，说明系统噪声或尾部事件值得解释。若样本呈周期性波动，可能要检查 cgroup、频率、热限制或 I/O 写回。

本册建议在报告里至少列出三类值：最小值、中位数和最大值或尾部百分位。最小值常代表干扰较少的理想近似；中位数代表常规情况；尾部代表稳定性风险。高性能系统不只追求平均吞吐，还要理解波动来源。一个吞吐高但尾延迟不可控的队列，可能不适合作为服务请求路径。

计数器不是事实本身 性能计数器非常有用，但它们不是无条件真理。硬件事件可能受平台支

持、内核权限、计数器复用、采样误差和事件定义影响。同一个事件名在不同微架构上含义可能不同；某些高层事件由工具组合推导，不是硬件直接计数；同时请求太多事件时，内核可能 multiplex，导致估计值精度下降。

因此报告中应写出命令、事件列表、是否出现 multiplex、运行时间是否足够长、是否固定 CPU、是否只测当前进程。若工具输出里有权限限制或不支持事件，不能忽略。特别是在手机、容器或受限 Linux 环境中，很多硬件事件无法读取，这时可以退而使用时间、上下文切换、page faults、输入规模对照和源码结构分析，但要明确证据等级降低。

计数器解释也要组合。IPC 高不一定代表程序好，可能只是做了很多无用指令但都能退休；IPC 低不一定代表前端差，可能是内存等待、锁等待或分支错误。cache-misses 高也不一定坏，流式扫描大数组天然会 miss，但如果带宽接近上限，miss 可能是有效数据移动。branch-misses 高才需要结合输入分布、分支数量和耗时变化解释。单个数字不能代替因果链。

Roofline 的实际计算步骤 Roofline 很容易被画成漂亮图，却没有指导实验。实际使用时先做三步。第一步，估算每个元素的逻辑数据移动和算术操作。例如 `std::int32_t` 求和每个元素读取 4 字节，做一次整数加法，最终写一个总和。第二步，估算实际数据移动的额外成本：cache line 带入未使用数据，写分配带来额外读，原子写导致 cache line 迁移，TLB miss 导致页表访问。第三步，用平台可持续带宽和计算吞吐给出上限，再和实测比较。

例如一个简单拷贝内核读取 4 字节并写入 4 字节，逻辑上每元素 8 字节。但如果写入使用普通 write-allocate 路径，写 miss 可能先把目标 cache line 读入，再修改，实际内存流量高于 8 字节每元素。若使用非临时写入，可能减少写分配，但也改变缓存污染和后续复用。教材不能把字节数估算写成固定公式，要让读者知道实际流量取决于写策略、缓存命中和硬件行为。

Roofline 还要区分单线程和多线程。单线程可能受单核加载端口、预取能力或乱序窗口限制；多线程可能逐步接近内存控制器带宽。若单线程远低于平台总带宽，增加线程可能提升；若多线程已经接近可持续带宽，继续加线程只会增加争用。性能报告应把单线程曲线和多线程曲线放在同一个解释里。

Amdahl、Gustafson 和扩展曲线 多线程加速经常用 Amdahl 定律解释：如果程序中有不可并行的串行部分，那么线程数再多也会受串行部分限制。这个模型有用，但容易被误用。真实程序的串行部分不是固定不变的：任务切分、合并、内存分配、队列调度、锁竞争和 NUMA 远端访问都可能随线程数变化。换句话说，线程数增加后，程序结构本身也在变。

Gustafson 的视角强调问题规模随资源增加而增长。在很多计算任务中，固定输入规模时加速有限，但输入规模扩大后，并行资源可以处理更多工作。教材要同时讲这两种视角。面试刷题常把复杂度看成固定输入下的函数；系统工程还要看“给更多核心后，能否处理更大批量、更高吞吐或更高并发”。固定规模加速和弱扩展是两个不同实验。

扩展曲线至少有三种画法。强扩展固定总工作量，观察线程数增加后时间下降；弱扩展让每个线程工作量固定，观察线程数增加后总吞吐是否保持；尾延迟扩展关注并发请求增加后延迟分布怎样变化。Compute Systems Engine 的归约适合强扩展和弱扩展；有界队列和线程池更适合吞吐和尾延迟；分布式 shuffle 则要同时看吞吐、长尾任务和重试成本。

同步路径的测量陷阱 锁和原子很容易被微基准误导。一个锁在空循环里竞争，测到的是最坏的锁争用形态；在真实业务里，临界区可能保护较大的工作，锁成本占比反而不高。反过来，一个队列在单生产者单消费者测试里表现很好，不代表多生产者多消费者下仍能扩展。同步结构必须按目标竞争模式测。

测 mutex 时应区分无竞争、短竞争和阻塞路径。无竞争 fast path 通常很快；短竞争可能自旋；阻塞路径会进入 futex，涉及内核调度和唤醒。测 atomic 时应区分单线程原子成本和多线程 cache line 争用。测无锁队列时应记录 CAS 失败率或至少记录重试行为，否则只看吞吐无法解释尾延迟。测条件变量时应记录等待谓词、队列长度和唤醒策略，否则不知道线程是在等数据、等空间还是被错误唤醒。

案例：归约实验报告示范 下面给出一段报告式叙述，展示本册希望读者达到的分析密度。

深入理解

问题：比较顺序归约、四累加器归约、热原子并行归约和线程本地 partial 归约。输入为 256 MiB 的 `std::int32_t` 数组，结果用顺序 reference 校验。机器为某型号 8 物理

核心 16 逻辑 CPU，编译器使用固定版本和 **-O3**。每个版本预热一次，正式运行 10 次，报告中位数和最小值。线程数取 1、2、4、8、16。

预期：顺序 baseline 可能受依赖链和内存流量共同影响；四累加器在小输入下可能改善 IPC，但在大输入下可能接近带宽上限；热原子版本在多线程下会因为共享写扩展性差；线程本地 partial 应显著减少共享写，但 16 逻辑 CPU 可能因 SMT 和带宽争用收益下降。若观察结果与预期不同，下一步分别检查编译器是否已自动向量化、partial 是否 false sharing、初始化是否 first-touch、线程是否迁移。

这段报告没有给固定数字，因为数字属于具体机器；它给的是实验结构和解释路径。一本教材如果只给“某机器快了 3 倍”，读者换设备后很难迁移。更重要的是教读者怎样提出假设、怎样排除错误、怎样承认结论边界。

观测性进入项目设计 很多学生把观测性当成上线后再加的日志，这在系统工程里太晚。Compute Systems Engine 从第一天就应该记录版本、输入规模、线程数、运行时间、校验结果、队列长度、任务数和错误计数。后续加入线程池时，要记录每个 worker 执行任务数、空闲时间和偷取次数；加入有界队列时，要记录 push 等待次数、pop 空队列次数和最大队列长度；加入分布式模拟时，要记录消息数、重试数、重复请求数、提交延迟和检查点耗时。

观测性也要控制成本。热路径中每个元素都写日志会摧毁性能；每个任务都拿全局锁记录指标会引入新的瓶颈。常见做法是线程本地计数、批量汇总、采样、分级日志和只在实验模式下打开详细指标。观测系统本身也是计算系统，仍然受本册所有原则约束。

统计方法：不要只报平均值 性能数据有波动。平均值容易被少数异常样本拉偏，最小值容易过于乐观，最大值容易受偶然干扰影响。报告应至少给出中位数、最小值和尾部，必要时给出四分位或百分位。对于服务延迟，P95、P99 往往比平均值更重要；对于批处理吞吐，中位数和最小值能帮助区分稳定能力和偶发干扰。

样本数量也要足够。一次运行不能代表稳定性能。短任务应重复更多轮，长任务可以少一些但要记录环境。若样本方差很大，不应简单取平均，而要调查噪声来源：CPU 频率、后台任务、page fault、I/O 写回、GC、热限制、cgroup throttling、网络抖动。统计不是装饰，而是发现不稳定性的工具。

计时边界 计时边界决定实验测的是什么。一个归约实验如果把输入生成、随机数填充、线程创建、任务提交、计算和校验都放进同一个计时区，结果就不能解释计算内核。端到端时间有价值，但要单独命名。通常应同时报告两类时间：端到端时间和核心阶段时间。端到端时间回答用户体验，核心阶段时间回答优化方向。

线程池实验也类似。若每轮都创建和销毁线程，测到的是线程生命周期成本；若线程池预先创建，测到的是任务调度和执行成本。两者都可能有用，但不能混淆。分布式模拟中，是否把 checkpoint 写入、故障重试、输出提交放进计时，也要明确。

输入生成和随机性 随机输入要可复现。使用固定 seed，记录分布参数。不要使用运行时随机设备生成不可复现输入，否则性能异常无法重放。对于分支预测实验，要明确输入分布：全小于阈值、全大于阈值、排序后半段大于阈值、随机一半大于阈值。对于热点 key 实验，要明确 Zipf 参数或热点比例。输入分布是 workload 的一部分。

输入生成本身可能很慢，也可能污染 cache。若目标是测计算内核，生成后应在计时外完成，并决定是否预热 cache。若目标是端到端系统，输入生成可以进入计时，但报告要说明。性能实验的每个阶段都要有名字。

编译选项和二进制一致性 编译选项会大幅影响性能。**-O0** 适合调试，不适合性能结论；**-O2**、**-O3**、**-march=native**、LTO、PGO、fast-math 都会改变生成代码和语义边界。报告必须记录编译器版本和选项。若使用 **-march=native**，二进制可能只适合当前机器，换机器结果不可比。

浮点实验尤其要谨慎。fast-math 可能允许重排、忽略 NaN 或改变舍入语义。并行归约如果要求确定性，不能随意打开破坏语义的选项。性能优化不能偷偷改变正确性合同。每个编译选项都应有理由。

基准污染 Benchmark 可能被日志、断言、内存分配、锁、原子 sink、系统调用和调试构建污染。比如在热循环里输出日志，测到的是终端 I/O；在每个元素上调用虚函数，测到的是调度而不是算法；在每轮都分配大 vector，测到的是分配器和 page fault；在并行循环里更新全局统计，测到

的是原子热点。

这并不意味着这些成本不重要，而是要命名清楚。如果目标是测完整业务路径，日志和分配可能应保留；如果目标是测内核算子，它们应移出计时区。一个好 benchmark 应能拆分阶段，让读者看到每个成本分别是多少。

对照实验的单变量原则 优化前后只改变一个变量，结论才清楚。若同时把容器从 list 改成 vector、把算法从单线程改成多线程、把编译选项从 O2 改成 O3、把输入规模改大，就算变快也不知道原因。实际工程有时需要多改，但实验应尽量拆成多个对照：先只换布局，再只换线程，再只换编译选项。

单变量原则也适用于失败实验。如果某个优化没有效果，不要马上丢弃。先检查变量是否真的变化：编译器是否已自动做了同类优化，输入是否太小，瓶颈是否迁移，测量噪声是否太大。失败结果也是知识，只要记录清楚。

Benchmark harness 的数据结构 项目可以定义一个简单结果结构，统一记录实验。它不需要复杂框架，但要避免每个实验手写不同输出格式。

Listing 5.1 Benchmark 结果的基本字段

```
1 struct BenchmarkResult {
2     std::string name;
3     std::uint64_t input_items = 0;
4     std::int32_t worker_count = 1;
5     double milliseconds = 0.0;
6     bool correct = false;
7     std::uint64_t bytes_processed = 0;
8 };
```

生产代码应避免在热路径频繁构造字符串；这里的结构用于实验汇总。后续可以输出 CSV 或 JSON，方便画图。关键是每个样本都带 correct 字段，错误结果不能进入性能结论。

Linux perf 的使用层次 perfstat 适合整体计数，例如 cycles、instructions、branches、branch-misses、cache-misses、context-switches、page-faults。perfrecord 和 perfreport 适合采样热点函数。perftop 适合实时观察。调度问题可以看 perfsched，锁问题在支持环境中可以看 lock 相关工具，cache line 争用可以研究 c2c。不同发行版和权限支持不同，报告要写清不可用项。

使用 perf 时要避免测错进程。若程序启动子进程或线程，命令行要确认是否跟踪到目标。若运行时间很短，计数器误差会大。若事件 multiplex，工具会给出缩放估计，精度下降。若 CPU 迁移频繁，某些 per-CPU 事件解释更难。perf 是强工具，但不是免思考按钮。

Off-CPU 时间 很多程序慢在 off-CPU 时间，也就是线程没有在 CPU 上运行。原因可能是等待锁、等待 I/O、等待条件变量、被调度器抢占、cgroup 节流或睡眠。CPU profile 只看 on-CPU 热点，可能完全看不到等待原因。服务端延迟分析经常需要 off-CPU 视角。

简单替代方法是应用内部记录等待时间。例如 bounded queue 的 push 等待时间、pop 等待时间；线程池 worker 空闲时间；RPC in-flight 时间；I/O completion 等待时间。即使没有 eBPF 工具，也可以通过状态机指标构造 off-CPU 证据。第二册强调把观测性放进项目，就是为了在工具受限时仍能分析。

内存带宽测量 内存带宽不是硬件规格表上的峰值，而是当前工作负载下的可持续带宽。可以用顺序读、写、copy、triad 等简单内核估算平台上限，再和实际算法比较。若算法吞吐接近可持续带宽，继续减少几条算术指令通常收益小；若远低于带宽，可能受依赖、随机访问、TLB 或前端限制。

带宽也受线程数、NUMA、频率和写策略影响。单线程带宽远低于多线程总带宽很正常；跨 NUMA 访问可能降低带宽；写分配会增加实际流量。报告中写“内存带宽瓶颈”时，应说明依据：字节估算、线程扩展曲线、cache miss、平台带宽对照，而不是只凭感觉。

Profiling 和 Tracing Profiling 聚合“时间花在哪里”，tracing 保留“事件按什么顺序发生”。计算内核常用 profiling；运行时、队列、异步 I/O 和分布式系统更需要 tracing。一个线程池平均函

数耗时不高，但 trace 可能显示 worker 大量空闲或等待队列；一个分布式作业总耗时高，trace 可能显示最后一个 reduce partition 长尾。

Compute Systems Engine 应支持轻量 trace：任务开始、任务结束、队列满、消息发送、消息完成、重试、提交。Trace 要有开关，默认可以只记录摘要。详细 trace 会产生大量数据，适合调试而不是每次性能运行。

性能回归 项目变大后，需要防止性能回归。不是每次提交都跑完整 benchmark，但至少要有小规模 smoke benchmark 和关键路径指标。比如顺序归约、并行归约、有界队列吞吐、线程池任务调度和分布式模拟正确性。若某次修改让归约慢 30%，应能及时发现。

性能回归判断要允许噪声。可以设置相对阈值、重复运行取中位数，并保留历史环境信息。手机和共享云环境噪声大，阈值要更宽；专用机器可以更严格。回归测试的目标是发现明显退化，不是追求实验室级精确。

报告中的诚实边界 高质量报告会明确说哪些结论已经验证，哪些只是推测。比如“当前环境无法读取 dTLB 事件，因此 TLB 解释来自页数和访问模式对照，证据弱于直接计数”；“本机没有 NUMA，因此 first-touch 只作为设计保留”；“EPYC 服务器上结果不能直接外推到手机大小核”。承认证据边界不是减分，而是工程严谨。

相反，糟糕报告会是一次运行数字写成普遍结论，把某台机器的结果推广到所有 CPU，把不可复现随机输入当作事实，把没有正确性校验的最快版本当作成功。第二册要求读者避免这种写法。

案例：一次性能回归排查 假设某次提交后，parallel histogram 慢了 25%。第一步不是猜，而是确认正确性是否仍通过，输入和编译选项是否相同。第二步看阶段指标：本地统计慢了，还是合并慢了，还是队列等待增加。第三步看 diff：是否改变了桶布局、增加了全局指标、把本地 vector 改成 `unordered_map`、或在热循环中加入日志。第四步跑对照：关闭新指标、恢复旧布局、固定线程数。

这样的排查流程比“感觉是 cache 问题”可靠。性能回归常来自小改动：一个统计计数器从线程本地变成全局原子，一个调试日志进入热路径，一个结构体字段改变导致 padding 变大，一个 vector reserve 被删除导致频繁分配。性能工程需要把代码 diff、指标和实验联系起来。

案例：手机环境测量 在手机 Termux 上测性能，要特别谨慎。CPU 可能有大小核，温度和电量会影响频率，后台应用可能抢资源，某些 perf 事件不可用，存储 I/O 受系统策略影响。报告应记录设备型号、系统版本、是否充电、是否长时间运行、可见 CPU、是否能设置 affinity、哪些 perf 事件可用。不能把手机一次短跑结果当成服务器结论。

手机环境也有价值。它能训练受限环境下的证据意识。无法读取硬件计数器时，可以用输入规模曲线、线程数曲线、正确性校验、时间分布和源码分析。无法验证 NUMA 时，可以写明限制。工程严谨不要求工具完美，而要求诚实说明证据等级。

实验数据归档 性能实验应归档原始数据，而不是只在报告里写结论。至少保存命令、配置、输入摘要、运行时间、样本、git commit、机器信息和输出校验。CSV 或 JSON 都可以。图表可以从原始数据生成，报告引用图表。这样几周后发现问题，可以重新分析，而不是重新跑一切。

项目可以在 `results/` 下按日期或 commit 保存实验摘要。不要把巨大临时文件提交，但小的 CSV、报告和脚本可以保留。性能工程是可重复研究的一种形式。

Benchmark 伦理 不要只挑最好结果，不要删除失败样本后不说明，不要在不同机器结果之间做不公平比较，不要把 debug 构建和 release 构建混在一起，不要把不正确输出的时间当成成功，不要把一次偶然提升宣传成普遍优化。工程团队依赖性能报告做决策，报告必须可信。

这听起来像规范，但它直接影响技术质量。错误的 benchmark 会把团队带向错误优化，甚至牺牲正确性。第二册强调测量，就是为了建立这种专业习惯。

性能报告示例结构 本册建议每份性能报告采用固定结构。标题说明被测模块和 commit。环境说明机器、系统、编译器、编译选项、CPU 拓扑和限制。工作负载说明输入规模、分布、seed 和正确性 oracle。实验矩阵说明版本、线程数、batch 大小、队列容量和运行次数。结果部分给出样本统计、关键指标和图表。分析部分写瓶颈假设、支持证据、反证和下一步。限制部分说明哪些结论不能外推。

固定结构能提高比较效率。读者以后回看报告，可以直接找到环境和输入；别人审查报告，可以快速发现缺失信息。性能报告不是文学作品，稳定格式比花哨表达更有价值。

测量章节总结 测量的目标是建立证据链。计时告诉现象，计数器提供线索，trace 显示顺序，correctness oracle 保证没有优化错对象，对照实验排除错误解释。没有测量，优化是猜；没有语义，测量可能奖励错误程序。本章的方法会贯穿后面所有实验。

测量决策表 选择测量方法时，先问问题类型。单核循环慢，用计时、汇编、IPC、branch 和 cache 事件；多线程不扩展，用线程数曲线、context switches、锁等待、false sharing 对照；I/O 慢，用队列长度、等待时间、page faults、fsync 时间；分布式慢，用 trace、partition 指标、重试和队列。工具随问题变化，不要用一个 perf 命令解释所有系统。

测量章的回归阅读应审一份不合格性能报告。报告若只写“优化后快了 2 倍”，没有 reference、输入分布、预热、重复次数、方差、编译选项和原始命令，就不能进入后续章节。性能数字必须和可复查条件绑定，否则它只是一次机器状态的截图。

同一个内核至少要分三类测量。第一类是正确性测量，确认不同版本输出一致或在误差合同内；第二类是资源测量，估算字节移动、操作数、内存峰值和队列水位；第三类才是时间测量，包括 p50、p95、p99 或多次运行的置信区间。Roofline 不是为了画漂亮图，而是强迫读者问：这个内核可能受计算吞吐限制，还是受字节搬运限制。

计数器也要有边界意识。移动设备上可能没有 PMU 权限，云机器上可能有噪声，温度和频率会影响结果，容器可能限制事件。没有某个计数器时，可以退回 wall time、输入规模阶梯、工作集扫描和源码证据。高质量报告不是工具越多越好，而是每个结论都说明证据来源和证据限制。

Roofline 估算要从字节账本开始。对一个 map 内核，读入多少字节、写出多少字节、是否额外读写中间数组；对一个 histogram，读 key、写本地桶、合并桶分别是多少；对一个 join，构建表、探测表和输出放大各自占多少。很多报告直接给操作数，却没有写字节数，最后无法解释为什么算术优化没有收益。字节账本不一定精确到每个 cache miss，但必须足够说明算术强度的数量级。

测量脚本还应保存原始环境。至少记录编译器版本、优化选项、CPU 型号、核心数、频率策略、输入生成 seed、运行次数和失败的权限检查。若在 Termux 上跑，报告要说明哪些计数器不可用，哪些实验只做 wall time 和 checksum。可复查性不是学术洁癖，而是让下一次改代码时知道性能变化来自实现、输入、工具还是机器状态。

统计口径也要讲清。一个 benchmark 不能只报告最小值，因为最小值常常代表偶然没有干扰；也不能只报告平均值，因为长尾可能被平均掩盖。对于短内核，可以运行多轮并报告中位数、分位数和离群点处理；对于端到端作业，要报告吞吐、阶段耗时、p95 或 p99、内存峰值和失败重试。不同指标回答不同问题，不能混成一个“快了多少”。

测量章还应训练反证意识。若假设内核受内存带宽限制，可以通过减少计算、改变数据类型、改变工作集大小和比较带宽上限来反证；若假设受分支限制，可以改变输入分布；若假设受锁竞争限制，可以固定线程数和临界区工作量。一个好的性能结论往往来自排除几个错误解释，而不是找到一个看起来合理的解释就停。

5.5 把测量写成可复查的论证

性能测量不是给程序排名，而是建立论证。一个论证至少包含问题、假设、实验设计、原始数据、解释和边界。若只写“优化后快 30

同一个内核在不同阶段可能呈现不同瓶颈。小输入完全在 L1，依赖链和分支可能明显；中等输入落在 LLC，缓存容量和冲突进入成本；大输入超过 LLC，内存带宽主导；多线程后，NUMA、false sharing、调度和频率变化进入成本。若实验只测一个输入规模，就很容易把局部现象当成规律。Roofline 的价值不在图好看，而在逼迫你把算术量、字节移动和硬件上限放在同一张表里。

正确性 oracle 先于计时

没有正确性 oracle 的 benchmark 没有意义。优化版本可能少处理记录、溢出、丢弃错误路径、改变浮点顺序、重复提交或漏掉边界输入。测量前先写 reference，给每个输入生成 checksum 或字节级输出比较。对于浮点，写清允许误差和合并顺序；对于并行输出，写清是否要求稳定顺序；对于分布式提交，写清重复 attempt 如何去重。

Oracle 还要覆盖边界输入。数组长度为 0、1、非对齐长度、不是 SIMD 宽度倍数、记录跨 split 边界、key 高度倾斜、输入含错误行，这些都比随机大输入更能暴露语义问题。许多性能优化在 happy path 上正确，在边界上错。教材要让读者把边界测试视为测量准备，而不是功能测试的附属品。

计时范围也要先定义。分配、初始化、文件读取、预热、核心循环、提交和清理是否计入，取决

于问题本身。如果研究热内核，就应把分配初始化移出计时；如果研究端到端服务，就不能只测内核。报告必须明确计时边界，否则同一结果可以被解释成完全不同的瓶颈。

从字节和操作数建立 Roofline 估算

Roofline 先问两个数：做了多少计算，搬了多少字节。数组求和每个元素一次加载和一次加法，算术强度低，通常受内存带宽限制。矩阵乘如果分块得当，一个加载进缓存的数据块会参与许多乘加，算术强度高，可能受计算吞吐限制。Histogram 虽然每条记录计算不多，但随机写、冲突和分支会让简单 Roofline 不够，需要补充 cache miss 和同步成本。

估算不是为了精确预测每个周期，而是为了排除荒谬解释。若一个内核理论上只做少量算术，却声称通过优化加法指令获得 10 倍大输入加速，就要检查是否同时改变了字节移动或输入规模。若一个内核搬动的数据量已经接近机器内存带宽上限，再追求整数指令减少可能收益有限。Roofline 是把“我觉得”换成“至少要符合物理上限”。

项目里的报告可以固定三类内核：sum、saxpy、histogram。Sum 用来展示内存带宽，saxpy 展示流式读写，histogram 展示随机写和冲突。三者放在一起，读者会看到同样是单机内核，瓶颈模型并不相同。后续典型计算内核章再展开 map、reduce、scan、partition 时，就有测量基础。

计数器、profile 和 trace 分工

计数器回答“发生了多少事件”，profile 回答“时间花在哪里”，trace 回答“事件以什么顺序发生”。三者分工不同。一个锁竞争问题，perf 可能显示 futex 或自旋热点，profile 显示等待路径，trace 显示队列长度和唤醒延迟。一个缓存问题，计数器显示 cache miss，profile 显示热循环，trace 未必有帮助。一个分布式重试问题，硬件计数器几乎无关，trace 和协议指标更重要。

使用计数器要注意权限、复用和噪声。移动设备或容器可能无法访问某些 PMU 事件；多个事件同时测量可能复用，结果不稳定；CPU 频率、温度和后台任务会影响样本。报告中要写清事件名、工具版本、运行轮次、是否绑定 CPU、是否预热、是否丢弃异常值。没有这些信息，数字看起来精确，实际上无法复现。

Trace 的设计要低开销。每条记录都写详细日志会改变程序行为。更好的方式是对阶段边界、任务状态、队列水位、错误事件做结构化记录，必要时采样。trace id 要贯穿 driver、worker、队列和输出，否则三端日志无法拼起来。后续分布式章节会把这个原则用于 request id 和 attempt id。

统计和报告不是装饰

单次运行结果不可靠。至少要多轮运行，给出中位数、最小值、最大值或分位数，并记录异常。平均值容易被少数抖动拉偏；最小值可能代表理想环境；p95/p99 反映尾部。对于交互式或服务型系统，尾延迟比平均吞吐更关键；对于离线批处理，总吞吐和资源利用更关键。报告指标必须匹配目标。

实验矩阵要控制变量。一次只改变线程数、布局、batch size、队列容量或同步方式之一。若同时改变三项，即使变快也无法归因。矩阵不必无限大，但要覆盖能证伪假设的点。例如怀疑 false sharing，就比较紧凑 partial 和对齐 partial；怀疑 NUMA，就比较主线程初始化和 worker first-touch；怀疑任务太细，就扫描 chunk size；怀疑背压，就扫描队列容量和生产速度。

本章最终交付物是一份性能报告模板。模板包括：问题和 oracle，环境和限制，版本和变量，原始结果摘要，关键证据，瓶颈假设，反证或未验证边界，下一步实验。这个模板会贯穿全书。没有报告模板，后续每章的实验会变成零散截图和口头结论；有了模板，读者才能积累可比较的证据。

案例走查：一份不合格报告怎样改成合格报告

不合格报告常写成：“优化后从 120 ms 到 80 ms，提升 33

同样的 80 ms 可能有不同含义。若 instructions 大幅下降、cache miss 不变，优化可能减少了控制流；若 bytes/sec 接近内存带宽，后续应优化数据移动；若 p99 远高于中位数，可能有调度或 I/O 抖动；若 checksum 变了，所有性能数字作废。报告要把这些分支写出来，而不是只给百分比。

本章建议在仓库放一个 [reports/](#) 样例，哪怕只是文本。每次项目优化都按同一结构记录。这样几周后回看，读者能知道为什么当时做了某个选择，而不是只看到一堆快慢数字。

报告里的反证意识

好的性能报告不只支持一个假设，还要主动排除其他解释。若你认为瓶颈是内存带宽，就要说明为什么不是分支、锁、I/O 或调度。可以通过单线程热循环、预热输入、去掉输出、固定线程绑定、替换数据分布来排除。反证意识能防止报告变成“我喜欢哪个解释，就挑哪个数字”。

例如 parallel histogram 变慢，候选原因包括锁竞争、false sharing、key 倾斜、hash 分配、内存

带宽、线程过多。一次只改一个因素：把全局锁换成本地 partial，排除锁；给 partial padding，检查 false sharing；换均匀 key，检查倾斜；预分配 map，检查分配；固定线程数，检查过度订阅。报告把这些对照列出来，读者才能相信结论。

报告也要写负结果。某个优化没有收益，不代表失败，可能说明瓶颈不在此处。负结果如果有清楚实验，同样有教学价值。经典教材让人舒服的地方之一，就是不会把每个技术都写成神奇加速，而是展示什么时候不该用。

工具输出的保存和可复现

性能实验要保存命令、commit、编译选项和原始输出。不要只把手工整理后的表贴进书里。项目可以让每次 benchmark 生成一个目录，包含 `env.txt`、`commands.txt`、`results.csv`、`notes.md`。即使书中不展示全部文件，读者也知道工程上应如何保存证据。

可复现还要求 seed 固定。随机输入必须记录 seed，key 分布参数要写清，坏行比例要写清。若输入来自真实日志，至少记录摘要和脱敏方式。没有输入描述的性能数据无法比较。对于移动设备，还要记录温度、充电状态、CPU governor 或至少说明这些不可控因素可能影响结果。

本章作业可以要求学生互换报告复现。一个人只看另一个人的报告和命令，尝试跑出相近趋势。若复现失败，不一定是代码错，可能是报告缺环境信息。这种作业能让测量规范变成实际能力。

综合练习：审稿一份性能报告

本章综合题让读者当审稿人。给出一份故意不完整的报告：只写“多线程版本提升三倍”，不写输入、机器、正确性、运行轮次和指标。读者需要指出缺失项，并提出补实验计划。补实验至少包括：串行 reference checksum，输入规模阶梯，线程数扫描，预热和冷启动分离，partial 布局对照，至少三轮重复运行，异常值记录和结论边界。

审稿时要追问可证伪性。报告说“瓶颈是内存带宽”，那就要问是否估算了字节数，是否接近已知带宽上限，是否比较了小输入和大输入，是否排除了锁等待。报告说“锁太慢”，那就要问 futex 或等待指标在哪里，是否有无锁或线程本地版本对照，是否锁内做了 I/O。每一个瓶颈声明都必须有证据路径。

第二部分要求读者把不完整报告改写成合格报告。不能只补漂亮图表，必须补实验矩阵和原始命令。若某个工具不可用，要写替代证据和未验证风险。审稿训练能让读者以后写自己的报告时自动补齐这些项。

这个题也能防止书本变成“答案集合”。真实工程中，很多争论不是缺技术，而是缺证据。会审报告，才会做优化。测量章的最终能力不是会用 `perf`，而是能判断一个性能结论是否站得住。

容易出错的边界：测量数字为什么会骗人

第一个骗人数字是平均值。一次后台写回、温控降频或调度抖动可以把平均值拉偏。报告至少要给中位数、最小值、最大值或分位数，服务型路径还要看尾延迟。第二个骗人数字是总耗时。总耗时不拆阶段时，热循环、I/O、初始化、清理和提交都混在一起，任何解释都可能成立。第三个骗人数字是单次 speedup。没有正确性 oracle，优化可能只是少做了工作。

第四个骗人数字是硬件计数器本身。事件可能不可用、被复用、受内核版本影响或含义因架构不同而变化。计数器要和源码、反汇编、输入规模曲线互相印证。第五个骗人数字是“CPU 使用率”。高 CPU 可能是有用计算，也可能是自旋和重试；低 CPU 可能是 I/O 等待，也可能是背压保护系统。没有等待和队列指标，CPU 使用率无法单独解释性能。

第六个骗人数字是小输入结果。小输入适合暴露固定成本、分支和前端，大输入适合暴露内存、TLB 和 I/O。只测小输入得出的优化结论，可能在真实工作负载上消失。反过来，只测大输入也会掩盖小请求延迟问题。实验矩阵必须服务目标 workload。

本章可以要求每份报告都有“可能错误解释”一栏。比如当前结论认为瓶颈是内存带宽，但可能错误解释包括线程迁移、输入未预热、cache 事件不可用、编译器优化差异。把这些写出来不是削弱结论，而是让结论有边界。高质量测量的标志，是知道自己还不知道什么。

本章核心接口：BenchmarkCase

测量章给项目留下 `BenchmarkCase`。它定义输入生成、reference 校验、被测版本列表、运行轮次、预热策略、指标采集和报告路径。每个章节新增实验时，不再临时写一套 benchmark，而是注册一个 case。这样实验格式统一，结果可比较。

`BenchmarkCase` 还要支持跳过原因。某些 case 需要 NUMA、PMU、特定 SIMD 或文件系统能力，当前设备不满足时标记 `skipped with reason`，而不是静默不跑。这个机制对手机环境尤其重要：

能跑的认真跑，不能跑的明确说明边界。

从测量进入数据布局

测量章不直接告诉你改哪里，它告诉你慢在哪里、证据有多强、哪些解释被排除。若报告显示字节移动和 cache miss 是主成本，下一步自然进入数据布局；若报告显示分支和指令是主成本，回到热循环；若报告显示等待和队列是主成本，进入并发和 I/O。数据布局章之所以紧跟测量章，是因为很多高性能系统的第一大成本不是算得慢，而是搬得多、搬得散、搬得太远。

项目里，BenchmarkCase 的输出应直接喂给布局决策。比如报告显示统计阶段每条记录实际只需要 8 字节热字段，却从对象数组搬了 96 字节，就有充分理由做冷热分离。若报告显示热字段已经很窄，布局优化就不该成为下一步。测量给布局优化设置了进入条件。

5.6 项目落地

本章要求项目拥有一个最小但严肃的 benchmark harness。它至少要支持：选择输入规模，选择版本，选择线程数，重复运行多轮，校验输出，输出机器和编译信息，记录最小值和中位数。后续可以逐步加入 CPU 绑定、NUMA 初始化、perf 命令模板、JSON 输出和图表脚本。不要一开始就做复杂框架，但也不能继续依赖手工改代码和肉眼读终端。

项目里的每个优化提交都应附带实验说明：优化目标是什么，影响哪条路径，正确性如何验证，基线版本是什么，在哪些输入和线程数下有效，在哪些情况下无效。这样做看似增加工作量，但能避免“改完之后不知道为什么快，也不知道什么时候会慢”的维护风险。

测量问题 先写问题，再选指标。若问题是“热循环是否受内存带宽限制”，指标应包含字节数、时间、带宽估算和 cache/TLB 事件；若问题是“线程池是否过载”，指标应包含队列水位和等待时间。指标跟问题脱节，就会变成装饰。

基线 基线不是随便一个旧版本，而是能说明语义正确的最小实现。优化版必须和基线对同一输入、同一输出、同一错误条件比较。没有 reference，性能越好越危险。

Roofline Roofline 把计算强度和硬件上限放在同一张图里。它不能替代真实分析，却能帮助判断当前优化方向：若明显带宽受限，继续减少算术指令意义有限；若计算受限，改善数据布局可能收益小。

统计稳定性 单次运行容易被调度、温度、频率、后台任务影响。报告至少要有重复次数、中位数、分位数或误差范围。移动设备上还要记录电源状态和热 throttling 的可能性。

计数器限制 perf 权限、虚拟化、内核版本和硬件支持都会影响计数器可用性。不可用时写明原因，换用时间阶梯、输入缩放或应用 trace 作为替代证据，不能把缺失计数器的猜测写成事实。

输入设计 性能实验要有能击穿朴素直觉的输入：小工作集、大工作集、热点 key、随机访问、冷读、热读、单线程和多线程。只测一个默认输入，结论无法外推。

报告结构 一个好报告应从假设开始，以证据结束：假设、环境、命令、输入、正确性、指标、结果、解释、限制、下一步。这个结构后面每章都复用。

反证意识 测量要主动找反例。优化让一个输入变快，却让另一个输入变慢，并不一定失败；关键是知道适用范围。教材要鼓励读者写“此优化只适合顺序扫描大数组，不适合随机小对象”。

案例：同一算法三种输入 对同一个内核测小数组、L3 级数组、远超内存缓存数组。用曲线说明工作集变化，而不是只给一个平均时间。

案例：计数器不可用降级 在 perf 无权限时保留命令和错误输出，改用输入阶梯和反汇编解释。这个案例训练诚实报告。

案例：Roofline 初版 估算 bytes、operations 和运行时间，判断内核处于带宽区还是计算区。估算要写公式和限制，不要求一开始完美。

源码阅读路线 Linux 源码阅读从 `tools/perf` 的事件选择和输出路径进入，再对照 `kernel/events/core.c` 中 perf event 的创建、权限检查和计数更新。阅读目标是解释 BenchmarkReport 中的计数从哪里来，为什么某些事件在当前设备不可用，以及不可用时如何降级。

阅读报告要写证据等级。能直接测到的数字放在第一层；从源码控制流推导出的解释放在第二层；当前设备无法验证、只能作为合理猜测的内容放在第三层。这样读者会知道性能论证报告中哪些结论可复现，哪些结论需要换机器继续确认。

源码阅读还要克制。看到一个复杂函数，不要把每个分支都展开；先回到本章的问题，问它能否解释一个等待点、一次状态转移或一条性能曲线。解释不了的内容可以留到索引，不要打断正文主线。

同一算法三种输入 对同一个内核测小数组、L3 级数组、远超内存缓存数组。用曲线说明工作集变化，而不是只给一个平均时间。

计数器不可用降级 在 perf 无权限时保留命令和错误输出，改用输入阶梯和反汇编解释。这个案例训练诚实报告。

Roofline 初版 估算 bytes、operations 和运行时间，判断内核处于带宽区还是计算区。估算要写公式和限制，不要求一开始完美。

测量目标

先定义要比较什么，再决定测什么。吞吐、延迟、扩展曲线、内存峰值和恢复时间不是同一个目标。

实验设计：同一程序分别以总吞吐和尾延迟为目标，说明最佳参数可能不同。

reference

性能实验必须先证明结果正确。没有 reference 的加速可能只是少做了工作，尤其在并行和失败恢复中更危险。

实验设计：每个 benchmark 运行前后都校验 checksum、记录数和错误分类。

预热和冷态

JIT 不一定存在，但 page cache、CPU 频率、分配器缓存和分支预测都会造成前几轮异常。预热不是为了作弊，而是为了知道自己测的状态。

实验设计：保留每轮结果，不只保留最快值，观察前几轮是否稳定。

统计方法

一次运行没有说服力。最小值常接近理想路径，中位数反映常态，尾部暴露抖动。报告应说明选择哪个统计量。

实验设计：同一输入至少运行多轮，记录 min、median、p95 和异常轮原因。

Roofline

Roofline 不是彩色图装饰，而是把计算量、字节量、峰值算力和带宽放进同一张账本。它帮助判断程序受算力还是带宽限制。

实验设计：为数组求和、点积和矩阵分块分别估算字节与操作量。

计数器限制

perf 计数器可能受权限、复用、多路复用和硬件支持影响。数字出现小数或 warning 时，不能当作精确事实。

实验设计：把 perf 输出中的权限、事件支持和 multiplex 信息写进报告。

profile 与 trace

profile 告诉时间在哪里聚集，trace 告诉事件按什么顺序发生。锁等待、队列水位和 I/O 完成常常需要 trace 才能解释。

实验设计：为一个队列实验同时记录采样热点和每批次等待时间。

对照实验

好的对照一次只改一个因素。若同时改编译选项、输入规模、线程数和数据布局，结论无法归因。

实验设计：建立 baseline、单因素变化和回退版本三列结果。

环境记录

编译器、优化级别、CPU governor、线程数、输入文件和后台负载都会影响结果。报告缺环境就无法复现。

实验设计：自动打印编译命令、CPU 信息、可见核心和输入摘要。

负结果

没有提升也是结果。它可能说明瓶颈判断错误、优化被编译器已有策略覆盖，或成本迁移到别处。

实验设计：把失败优化写进报告，说明下一步为什么不继续。

5.7 性能报告不是跑分截图，而是工程论证

测量章节如果写得像工具说明，读者会学会几个命令，却不会学会判断。本章应该从一句模糊需求开始：“这个程序太慢了。”这句话没有工程意义，因为它没有说慢的是吞吐、平均延迟、尾延迟、启动时间、恢复时间，还是单位能耗。把目标拆清楚，才知道测什么。一个批处理作业追求总吞吐，一个交互服务关心尾延迟，一个 checkpoint 系统关心恢复时间；同一个优化可能改善其中一个目标，伤害另一个目标。

测量的第一原则是正确性先行。没有 reference 的性能数字不值得相信。并行版本少处理一部分记录，SIMD 版本忽略尾部，I/O 版本没有等待持久化，都可能得到漂亮速度。每个 benchmark 都应先写校验：记录数、错误数、checksum、输出顺序、浮点误差或 manifest 状态。性能报告里把校验放在环境之前也不过分，因为“算错但更快”不是优化。

第二原则是把实验状态写清。CPU 频率、温度、后台负载、page cache、输入文件、编译器、优化级别、线程数、亲和性和权限都会影响结果。手机 Termux 尤其要注意温度和调度限制：连续跑十轮，后几轮可能因为降频变慢。报告不要掩盖这种现象，应该把它当作系统行为记录下来。真实工程里，性能稳定性本来就是指标之一。

第三原则是统计。最快值能说明理想路径，中位数说明常态，尾部说明抖动。只给一次运行结果，没有复查价值。实验至少要多轮运行，保留每轮原始结果，再给出选择某个统计量的理由。若有异常轮，不要直接删掉；先判断是系统噪声、错误输入、page cache 状态、调度抖动还是程序 bug。异常轮经常比平均值更能暴露系统问题。

Roofline 的作用是把算术和数据移动放到同一张账本。数组求和每个元素只做很少计算，却要搬大量字节，通常受带宽限制；矩阵乘如果分块得当，同一数据被多次复用，可能转向计算瓶颈。读者不用一开始画很漂亮的图，但要会估算每条记录读多少字节、写多少字节、做多少操作。估算和测量差距大时，说明隐藏成本存在，例如字符串分配、cache miss、写放大或重试。

计数器要谨慎解读。perf 事件可能受权限限制、硬件支持、多路复用和采样误差影响。IPC 低不一定是前端问题，branch miss 高不一定就是总瓶颈，cache miss 数字也要结合工作集和输入分布。报告应把直接测量、工具限制和推断分开写。高质量的测量章不追求神秘命令，而追求可复查结论：为什么这个指标能支持当前假设，为什么它不能支持更强的结论。

本章可以要求读者实现一个最小 benchmark harness。它接收输入规模、版本名、线程数和重复次数；运行前构造输入，运行后校验输出；记录环境、原始结果、统计摘要和失败边界。后续每章都复用它。这样整本书的实验就不会变成散落的终端截图，而会形成同一种证据语言。

案例推导：一次“优化成功”的性能报告为什么可能不合格

某个并行归约版本比 baseline 快百分之三十，开发者准备提交。审稿时先问三个问题：输出是否和 reference 一致，输入和环境是否固定，瓶颈是否真的按预期移动。检查后发现，优化版本跳过了错误记录，baseline 把错误记录计入错误分类；两者做的工作不同。这个案例说明性能报告第一行不应是速度，而应是语义校验。

合格报告先列输入合同：记录数、字段范围、错误行比例、随机种子和预期输出摘要。再列环境：编译器、优化级别、CPU 可见核心、线程数、频率状态、输入是否热 cache。然后列版本差异：baseline、改进一、改进二分别改变了什么。最后才给结果表。这样的报告看起来繁琐，但它允许别人复查；没有这些信息，所谓提升只能算一次运行截图。

统计也要认真。若十轮结果分别是 10 ms、10.2 ms、10.1 ms、18 ms、10.0 ms，平均值会被异常轮拉高，最小值又掩盖抖动。报告可以同时给 min、median、p95，并解释异常轮。异常可能来自系统调度、page cache、温度，也可能来自程序内部偶发锁等待。性能工程不是把异常删掉，而是判断异常是否属于目标 workload。

Roofline 在这个案例中用于排除错误方向。若归约每个元素只做一次加法，却读取大量内存，优化算术指令可能没有明显收益；若矩阵分块后数据复用增加，算术端口可能成为瓶颈。报告要估算字节和操作量，再用测量校正。估算错误很正常，错误本身有价值：它提示隐藏拷贝、额外分配、cache miss 或写放大。

计数器使用要写限制。手机 Termux 或普通用户环境可能无法读取某些硬件事件，perf 也可能 multiplex。此时可以用阶段时间、输入规模、系统调用计数或 trace 替代，但要标注证据等级。不要因为拿不到理想计数器就放弃测量，也不要把替代指标说成硬件事实。测量章的目标是让读者诚实地建立证据，而不是崇拜工具。

最终，benchmark harness 应成为项目基础设施。每章新增优化都通过同一个 harness 运行：构

造输入、运行 reference、运行候选版本、校验输出、记录环境、保存原始数据。这样后续读者能看到性能结论如何积累，而不是每章重新发明一套测试脚本。

核心思想

测量不是为了得到一个数字，而是为了排除错误解释，缩小瓶颈范围，并验证优化是否真的改变了瓶颈。

实验：对 Compute Systems Labs 的顺序归约和并行归约运行 `perfstat`。记录时间、cycles、instructions、IPC、cache-misses 和 context-switches。解释并行版本是否真的更快，如果没有，瓶颈可能在哪里。

性能报告先回答一个具体问题 测量不是把程序跑十次取平均，而是先提出一个可回答的问题。比如“SIMD 版本是否更快”太粗；更好的问题是“在输入已经驻留内存、输出预分配、长度大于一百万、允许浮点重排的条件下，向量化归约是否提高热循环吞吐”。问题越清楚，测量边界越清楚，结论越不会被误用。端到端时间、热循环时间、系统调用时间和排队时间回答的是不同问题，不能混在一个数字里。

贯穿项目每个阶段都要有两套数字。第一套是端到端数字，包含真实用户会承受的全部成本：读取、解析、计算、写出、提交和恢复检查。第二套是隔离数字，只测某个内核或某个状态机，用来解释瓶颈。端到端数字告诉我们系统是否变快，隔离数字告诉我们为什么变快或为什么没变快。只有隔离数字没有端到端，容易优化错地方；只有端到端没有隔离数字，结论无法复查。

Roofline 要和实际字节数连接 Roofline 模型把内核分成算力受限和带宽受限，但它依赖两个输入：实际执行的操作量和实际搬运的字节数。很多报告只数源码里看得见的数组读写，漏掉临时对象、写分配、cache line 取整、哈希表桶访问、指针对象和输出材料化。若字节数估错，算术强度就错，Roofline 结论也错。项目报告应写出字节账本：每条记录读取多少字节，热字段加载多少，冷字段是否访问，partial map 写多少，最终输出写多少。

Roofline 也不是唯一答案。它适合解释规则数值内核，对分支密集、锁等待、I/O、任务调度和分布式重试不够直接。遇到这些场景，要配合 profile、trace、队列指标和系统调用统计。性能分析的工具链应按问题选择：热循环看 counters 和汇编，排队看 trace，I/O 看提交和完成，分布式看 request timeline。把所有问题都塞进 Roofline，只会让模型失真。

统计显著性来自实验纪律 性能数据有噪声：CPU 频率、温度、后台任务、页缓存状态、输入分布、内存分配地址、调度迁移都会影响结果。报告不能只给一次运行的时间。至少要给多次样本、median、p90 或 p99、最小值和最大值；还要说明是否预热、是否固定 CPU、是否清理或保留页缓存、是否禁用或记录 turbo 状态。若环境无法完全控制，就如实写出限制。

对系统项目来说，尾延迟往往比平均值重要。一个异步写入队列平均很快，但偶尔 fsync 卡住，会让上游堆积；一个任务调度器平均开销小，但少数任务长尾，会拖慢整个 stage；一个分布式请求 median 很低，但 retry 后 p99 很高，会影响用户感知。本章应让读者习惯同时看吞吐和分位数，而不是只看平均吞吐。

实验：审稿式复查一份坏报告 本章可以给一份故意不完整的性能报告：某优化声称速度提升三倍，但没有输入规模、没有编译选项、没有正确性校验、没有预热、没有原始样本、没有环境信息。读者要像审稿人一样提出问题：是否包含文件读取，是否复用了页缓存，是否改变数值语义，是否只测了一次，是否对比同一编译级别，是否有端到端收益。这个练习能训练工程判断。

随后让读者重写报告。固定输入生成器，保存 checksum，记录命令和版本，跑足够多样本，分离初始化和热循环，输出 CSV，画或用文字描述扩展曲线。报告最后必须有结论边界：本结果只说明某机器、某输入、某编译器下的行为；迁移到其他架构或输入分布需要重测。高质量性能教材必须反复强调这种边界。

Linux 工具链的定位 Linux 上的 `perfstat` 适合看整体事件计数，`perfrecord` 和 `perfreport` 适合定位热点，`ftrace` 或 `tracepoint` 适合看内核事件时间线，`strace` 适合确认系统调用形态，`iostat` 和 `pidstat` 适合观察 I/O 和进程行为。工具不是越多越好，而是每个工具回答一个问题。先写问题，再选工具。

项目中可以提供统一 benchmark 输出：配置、输入摘要、阶段时间、样本列表、checksum、错误数、队列水位和资源指标。这样后续章节每次优化都在同一格式下比较。报告格式一旦稳定，读

者会把注意力放在因果解释上，而不是每章重新猜测数字含义。

事故复盘：一份无法复现的三倍加速报告 性能报告里最危险的句子是“优化后三倍加速”，后面却没有输入、命令、预热、样本、正确性校验和环境说明。一次重跑发现加速消失，原因可能是页缓存热了、CPU 频率不同、编译选项改变、输入规模不一样，甚至优化版本少处理了错误行。测量不是按秒表，而是给一个性能结论建立可复查的证据链。

Roofline 的价值也在这里。先用字节账本估算必要数据搬运，再用算术操作数估算计算量，把点放到带宽和峰值算力之间。如果报告声称超过机器带宽，就要检查字节是否漏算；如果声称受计算限制，但 IPC 很低、cache miss 很高，就要回到 profile。计数器不是为了堆数字，而是和模型互相质疑：模型解释不了的地方，才值得继续挖。

合格报告要保留原始样本而不是只给均值。至少给出最小值、中位数、p95、标准差或多次运行列表，并说明是否固定频率、是否清理缓存、是否隔离后台负载。正确性校验放在性能数字之前：checksum、记录数、错误数、manifest 条目先一致，时间比较才有意义。后续每一章的实验都沿用这个规矩，否则“优化”可能只是测量噪声或语义损坏。

测量报告实验判读 判读性能报告时先找缺失项：有没有 reference，有没有输入摘要，有没有编译命令，有没有原始样本，有没有环境说明。再看数字之间是否互相支持：字节账本和带宽是否匹配，Roofline 判断和 profile 热点是否一致，trace 队列等待和端到端延迟是否一致。若数字相互矛盾，不要急着选喜欢的结论，应先修测量边界。

这一段也服务于复习。读者合上书后，应该能凭本章的判读口径重新审查自己的代码：有没有最小正确版本，有没有能击穿旧方案的输入，有没有状态记录，有没有资源指标，有没有失败后的恢复动作。若五个问题缺两个以上，就说明实现还停在演示层，而不是工程层。

给性能报告写审稿意见 一份性能报告可以像论文一样审稿。第一问是问题是否明确：它测的是端到端，还是热循环，还是某个系统调用路径。第二问是 reference 是否存在：优化后结果是否和基线一致。第三问是输入是否可复现：规模、分布、随机种子、冷热状态和错误比例是否写清。第四问是样本是否足够：一次运行不能说明稳定性。

审稿意见不要只写“补充数据”。应指出缺失数据会影响哪个结论。例如没有字节账本，就不能判断 Roofline；没有原始样本，就不能判断分位数；没有编译命令，就不能判断优化级别和 fast math；没有环境信息，就不能迁移到另一台机器；没有错误注入，就不能说明系统在失败下仍可靠。

这种审稿式训练能防止性能章节变成工具教程。工具只是证据来源，报告才是推理载体。读者学会审稿，就能反过来写出可复查的实验，也能识别网上大量“快了多少倍”但无法复现的结论。

实验课：重写一份不可复查的三倍加速报告 先给读者一份坏报告：某版本数组归约快了三倍，只写了两行时间，没有输入大小、编译命令、CPU 信息、线程数、预热策略、样本数量和 checksum。要求读者写审稿意见，指出每个缺失项会影响什么结论。比如没有 checksum，就不知道是否优化错程序；没有命令，就不知道是否改变优化级别；没有样本，就不知道是否只是偶然。

然后重写实验。固定输入生成器，保存随机种子和输入摘要；编译时记录编译器版本、优化级别和开关；运行时先预热，再收集多次样本；输出 raw CSV，不只输出平均值；同时记录端到端时间和热循环时间；最后验证 checksum。若使用 perf，再保存事件列表和权限限制。若没有 perf 权限，就写替代证据：输入阶梯、阶段时间和源码结构。

判读时要练习拒绝过度结论。若热循环快了三倍但端到端只快百分之十，说明瓶颈在其他阶段；若 median 快但 p99 变差，说明尾部路径出问题；若小输入变慢、大输入变快，说明固定成本和吞吐收益存在交叉点；若换输入分布后收益消失，说明优化依赖工作负载。报告要把这些边界写进结论，而不是只展示最好的一行。

最后把报告格式固化到项目。每次 benchmark 生成 env、commands、results、notes 和 checksum。书中可以只展示摘要，但工程上保留原始材料。性能优化是可复查过程，不是一次演示。

性能报告练习验收重点 合格答案要能被别人复跑。输入、命令、编译器、样本、checksum、原始输出、统计方法和结论边界都要有。Roofline 题必须写实际字节数，不允许只引用理论公式。若结论是“优化有效”，必须说明在哪个输入范围有效，在哪个范围不确定。

答案还要写“为什么不采用更复杂方案”。例如没有证据证明队列是瓶颈，就不应把锁队列换成无锁；没有证明输出顺序可变，就不应随意重排；没有证明错误路径能恢复，就不应只提交正常路径。能解释不做什么，说明读者开始具备工程判断。

最后，答案必须包含可复查材料。至少有输入摘要、关键状态、期望输出、实际输出、运行命令

和未验证边界。没有这些材料，老师或同伴无法判断结论是否成立。第二册的作业不是让读者写漂亮段落，而是训练读者把系统问题变成可验证证据。

项目实现：可信测量、Roofline 与性能计数器 本章对应的贯穿项目实现不要求一次写成完整框架，但必须有最小可运行切片。这个切片包含三个部分：一个 reference，一个带本章机制的实现，一个能击穿 reference 之外隐含假设的测试。Reference 用来固定语义，机制实现用来展示本章知识，测试用来证明新增复杂度确实解决了问题。

代码组织上，不要把所有内容放进一个大文件。基础类型、输入生成、核心机制、报告输出和测试应分开。即使当前只是教学项目，也要让目录能表达责任边界。这样后续章节接入时，不会因为找不到状态归属而重新发明一套对象。

每个实现都要输出一段机器可读摘要。摘要至少包含输入规模、关键状态、错误数、输出校验、耗时和未验证边界。这里的机器可读不要求复杂格式，简单的 key value 文本也可以。关键是让脚本和读者都能复查，而不是只能看屏幕截图。

测试应包含正常路径、边界路径和失败路径。正常路径证明功能可用；边界路径证明长度、容量、尾部、空输入、关闭或超时没有遗漏；失败路径证明本章机制不是装饰。若失败路径写不出来，往往说明本章概念还没有落到工程对象上。

报告最后写一个取舍段。新增机制带来了什么好处，增加了什么成本，哪些场景不应该使用它，下一章会复用哪个字段或对象。这个取舍段非常重要，因为系统工程不是把所有高级机制都加进去，而是在证据和约束下选择合适的边界。

调试笔记：数字矛盾时先怀疑边界 如果 perf 显示指令减少，但端到端时间没变，先检查测量是否包含 I/O、初始化或排队。如果 Roofline 判断是带宽受限，但 profile 热点在锁等待，说明模型用错了对象。如果 median 变快但 p99 变慢，说明慢路径被隐藏。性能报告的价值不是让所有数字一致，而是发现数字不一致时能回到边界重新拆分。

复查清单：可信测量、Roofline 与性能计数器 复查本章时，先检查 reference 是否仍然存在。没有 reference，后面的并发、优化、异步或分布式改造都无法证明语义没有变。再检查压力输入是否尖锐：它应当攻击本章最核心的边界，而不是只把数据量放大。第三，检查状态是否可观察：关键对象的身份、状态、错误和提交结果应能在报告中找到。

第四，检查代码示例是否说明禁止行为。调用者不能做什么，往往比能做什么更能体现工程质量。第五，检查实验是否写出未验证边界；当前设备、权限或输入无法证明的结论，必须标注。第六，检查本章对象是否能被下一章复用，不能复用就说明边界还不稳。

完成这六项后，再看文字是否连贯。每个概念都应从问题推出，每个案例都应从朴素方案走到改进方案，每个实验都应能解释一个判断。若某段话放到其他章节也成立，就把它改成本章的具体对象、具体输入和具体失败。

5.8 工程案例：把测量写成可复核的技术论证

性能测量最容易变成截图堆叠。一个耗时数字、一个 CPU 使用率、一个 perf 输出，看起来都很客观，但如果没有输入、边界、统计方法和反例，它们并不能支持工程结论。本章的核心不是教几个命令，而是训练读者把测量写成可复核的论证：假设是什么，变量是什么，证据是什么，哪些结论还不能说。

一、先定义被测问题和不变量 测量前先写问题。是比较两个解析器版本，还是评估线程数扩展，还是判断是否内存带宽到顶？每个问题对应不同边界。比较解析器时，输入、输出、错误处理和 checksum 必须一致；评估线程扩展时，线程创建、初始化和合并是否计入要写清；判断带宽时，要记录读写字节数和工作集大小。

不变量是防止测错程序的底线。优化版本必须输出同一 checksum 或满足同一误差合同；输入生成必须固定 seed；编译选项和目标架构必须记录；CPU 频率、温度、后台任务和权限限制要说明。没有这些信息，结果很难复现，也很难和别人的结果比较。

二、计时边界决定结论含义 同一个程序可以测端到端时间，也可以测热循环时间。端到端时间包含读取、初始化、调度、合并和写出，适合用户体验；热循环时间排除外围成本，适合分析内核。两者都有效，但不能混用。若你用热循环时间证明系统快，却端到端被 I/O 队列拖慢，结论就是错位的。

Compute Systems Engine 的报告可以把阶段拆成 input、parse、local aggregate、shuffle write、

reduce、manifest commit。每阶段记录开始结束时间、输入字节、输出字节和错误数量。这样当总时间变化时，能看到是热内核改善，还是成本迁移到写出或合并。测量的第一目的不是证明自己快，而是定位变化发生在哪里。

三、Roofline 从字节和算术强度推导 Roofline 模型把内核放到两个约束下：计算峰值和内存带宽。算术强度是每搬动一个字节做多少计算。数组求和算术强度低，容易受内存带宽限制；矩阵乘分块后重复使用数据，算术强度高，可能接近计算峰值。这个模型不要求读者背图形，而是先估算字节移动和操作数。

估算要诚实。读一个 `std::uint64_t` 数组求和，逻辑上每元素 8 字节，但如果写输出、读索引或访问对象指针，实际字节会更多；哈希表查询不仅读 key 和 value，还读 bucket、节点指针和控制字节；日志解析除了输入字节，还可能分配字符串和写错误对象。Roofline 的价值是逼迫你写出数据移动账本。

四、计数器是线索，不是判决书 `perfstat` 可以给 cycles、instructions、branches、branch misses、cache misses 等线索。但事件名、采样精度、权限、内核版本和 CPU 架构都会影响解释。IPC 低可能是内存等待，也可能是分支失败，也可能是前端供给不足；cache miss 高也要看 miss 的级别和访问频率。计数器必须和源码、输入、反汇编和阶段时间一起解释。

报告中最好写“该证据支持什么，不支持什么”。例如 branch misses 在随机输入上明显上升，支持“分支预测是变化因素”；但如果总时间没有变化，说明分支成本可能被其他瓶颈掩盖。LLC miss 下降但耗时不变，可能表示瓶颈已经迁移到写出或锁等待。成熟测量不是只找支持自己假设的数字，也要解释反例。

五、统计方法处理波动 单次运行时间不可靠。缓存状态、调度、温度、频率、后台任务、页错误都会造成波动。至少要多轮运行，报告中位数、最小值、最大值或分位数。若波动很大，应先找波动来源：是否有冷启动，是否有 I/O，是否把输入生成混入计时区间，是否有 CPU 迁移。

小差异尤其要谨慎。一个 2

六、防止 benchmark 被编译器优化掉 编译器会删除无用计算。若求和结果没有被观察，循环可能被优化掉；若输入是编译期常量，计算可能被常量折叠；若函数太小，benchmark 框架开销可能主导。正确做法是使用运行时输入、输出 checksum、避免未定义行为，并把防优化方法写进报告。

C++ 示例中可以把结果写入 `volatile` sink 或返回给调用者统一校验，但不要滥用 `volatile` 去表达并发同步。`volatile` 不是原子，也不是内存序工具。性能测量里的防优化和并发语义要分开讲，避免读者把 benchmark 技巧带到生产并发代码里。

七、回归阈值和性能预算 一次测量解决不了长期维护。项目应为核心内核设置性能预算：输入规模、机器类型、编译选项、允许波动、失败阈值。CI 环境可能噪声大，不适合卡很小差异；本地基准机可以设更严格阈值。性能回归报告要保存历史曲线，而不是只看当前一次。

预算也要区分阶段。解析内核退化 5

八、实验交付：BenchmarkCase 本章给项目留下 `BenchmarkCase`。它记录输入描述、生成 seed、版本名、编译选项、线程数、绑定策略、预热轮数、测量轮数、checksum、阶段时间、可用计数器和不可用原因。每个后续章节的实验都复用这个结构。这样硬件、布局、同步、I/O 和分布式结果可以放在同一套报告里比较。

验收时让另一个人只看报告复现实验。如果他不知道输入怎么生成，说明报告不合格；如果他不知道计时边界，说明结论不合格；如果他无法判断输出是否正确，说明 benchmark 不可信。测量的质量标准是可复核，而不是命令看起来专业。

5.9 案例报告：一次优化为什么必须写反证

性能报告只写“优化后快了百分之多少”是不够的。高质量报告还要写反证：哪些解释被排除，哪些输入上优化无效，哪些指标和假设矛盾。反证不是否定工作，而是让结论更可信。没有反证的报告通常只是在讲故事。

一、先列候选瓶颈 假设日志解析版本 B 比版本 A 慢。候选瓶颈可能是分支失败、冷字段搬运、字符串分配、队列等待、page fault、写出背压、CPU 降频。报告第一步不是猜一个最熟悉的原因，而是列候选并安排证据。分支失败看 branch misses 和输入分布；冷字段搬运看对象大小和

cache miss; 分配看 allocation count; 队列等待看 trace; page fault 看缺页计数; 写出背压看队列水位。

候选列表不需要无限长,但要覆盖最可能的层次。若只看 perf,不看队列,可能把等待误判成 CPU 内核慢;若只看应用日志,不看反汇编,可能把编译器生成差异漏掉。系统测量的难点是跨层归因,候选列表能防止过早收敛。

二、反证输入比理想输入更重要 一个优化在理想输入上有效,不说明它普遍有效。查表分类在随机字符上可能减少分支失败,在规则日志上可能增加 load;线程本地 partial 在高竞争计数上有效,在小输入上合并成本可能超过收益;大 batch 提高吞吐,却恶化尾延迟。报告要主动给出这些反例输入。

反证输入应针对机制弱点设计。优化分支,就提供可预测分支输入;优化 cache,就提供工作集小到全在 L1 的输入;优化多线程,就提供小输入和单热点输入;优化 I/O 批处理,就提供低延迟敏感输入。若优化在这些输入上无效,报告写清适用范围。知道边界的优化,比宣称全能的优化更可靠。

三、测量要区分效果大小和解释强度 效果大小是数字变化,解释强度是证据能否支持原因。一个版本快了 40

反过来,一个指标明显变化但总时间不变,也有价值。它说明优化确实改变了局部成本,但主瓶颈不在那里,或瓶颈迁移到其他阶段。这样的结论能避免继续在无效方向投入。没有提升也是结果,只要能解释。

四、统计显著性和工程显著性不同 多轮测量可能显示 1

这不是反优化,而是成本意识。教材要让读者敢于删除无效或收益不足的优化。性能工程不是只添加技巧,也是控制复杂度。

五、报告模板要服务推理而不是填空 项目报告可以固定字段:问题、输入、环境、版本、假设、指标、结果、反证、边界、下一步。但正文不能变成模板话。每个字段都要写本章具体内容。例如在 SIMD 章,反证是尾部和跨块状态;在锁章,反证是低竞争短临界区;在分布式章,反证是迟到响应和重复请求。格式统一,分析必须具体。

最终读者应能读报告复述因果链:为什么怀疑这个瓶颈,做了哪个最小改动,哪些指标支持,哪些输入限制结论,下一步瓶颈在哪里。能做到这一点,测量章才真正完成。

5.10 补强案例:阶段化报告如何防止局部优化误导

一个局部内核变快,不代表系统端到端变快。阶段化报告的意义就是防止局部优化误导。它把作业拆成读取、解析、聚合、分区、写出、提交和恢复几个阶段,每个阶段都有时间、字节、对象数量和错误统计。优化前后对比阶段表,才能看见成本迁移。

一、局部快可能暴露下游慢 解析器优化后,map 端更快地产生 shuffle block,writer 队列可能开始积压。端到端时间不变,不表示解析优化无效,而是主瓶颈迁移到写出。若没有阶段报告,团队可能继续优化解析,浪费时间。若有阶段报告,就会转向 I/O 背压或 block size。

这类迁移是性能工作的正常结果。优化像推瓶颈,不像消灭所有成本。报告要写“旧瓶颈是什么,新瓶颈是什么”,而不是只写“提升不明显”。

二、阶段边界要和语义边界一致 阶段不能随便按函数切。一个函数可能既解析又写队列,另一个函数可能只是调度任务。阶段应按数据状态划分:原始输入、解析后 batch、本地 partial、shuffle block、manifest。这样阶段时间能和对象生命周期对应。若阶段边界和语义边界不一致,恢复和性能报告会互相对不上。

三、报告表的最小字段 每阶段至少记录 input bytes、output bytes、records、tasks、elapsed time、queue wait、error count、retry count。硬件阶段可加 cycles 和 cache miss, I/O 阶段可加 fsync time,分布式阶段可加 timeout 和 late response。字段不必一次全满,但缺失要写明。

四、实验 先优化解析,再优化布局,再优化写出。每次只改一个阶段,阶段报告都保留。最终比较三次优化的端到端效果和瓶颈迁移。这个实验训练读者从系统角度看性能,而不是被单个 benchmark 牵着走。

5.11 补充案例：如何写一次性能回归说明

性能回归不可避免。重要的是写出能指导修复的说明，而不是只说“变慢了”。一份好的回归说明包含触发版本、输入、环境、变化幅度、阶段归因、最可能原因和下一步验证。

一、**先定位阶段** 回归报告先比较阶段表。如果总时间变慢 20

二、**再比较代码和数据** 性能回归可能来自代码，也可能来自输入分布。key 基数增加、错误率提高、行长变长、输出 block 变多，都可能让同一代码变慢。报告要记录输入摘要。如果输入变了，回归可能是容量问题，而不是代码退化。

三、**最小复现** 构造最小输入和命令，让别人能复现回归。不要只贴 CI 链接。最小复现应包含 seed、规模、线程数、环境和预期差异。若无法稳定复现，先写波动范围，不要过早下结论。

四、**修复后验证** 修复性能回归后，不只看总时间恢复，还要看原先被怀疑的阶段指标是否恢复，正确性 checksum 是否不变，反证输入是否没有恶化。性能修复也是代码变更，需要测试矩阵。

5.12 从一次跑分走向可信性能结论

性能测量最危险的地方在于它很容易给人确定感。程序跑了三次，取一个平均值，看起来就像有了结论。但系统性能不是考试算术题，任何一个数字都依赖输入、机器状态、编译器、温度、频率、后台任务、页缓存、调度和测量工具本身。可信测量的目标不是追求一个漂亮数字，而是建立一条证据链：我比较的是什么，我排除了哪些干扰，我的结论适用于哪些边界，我还不能证明什么。

一、**先定义问题，不要先开工具** 测量前要先写问题。是要判断算法是否被内存带宽限制，还是要判断一次优化是否减少了分支错误，还是要判断多线程扩展为何停住？不同问题需要不同输入和指标。若问题是内存带宽，输入要大到超过缓存，指标要包含字节数、时间和带宽上限；若问题是分支，输入要包含可预测和不可预测分布；若问题是尾延迟，平均值没有代表性，必须看分位数。没有问题定义，工具输出越多，报告越乱。

二、**基线不是慢版本，而是语义锚点** reference 或 baseline 的价值不只是对比速度。它定义正确输出、边界行为、异常处理和可接受误差。优化版本必须先和 baseline 对齐，再谈速度。若优化改变了浮点合并顺序，报告要写误差规则；若优化改变了输出顺序，报告要写顺序是否属于语义；若优化跳过了错误检查，速度比较就不公平。高质量性能报告会把“快了多少”放在“是否仍是同一个问题”之后。

三、**Roofline 是定位语言，不是装饰图** Roofline 的核心是算术强度和机器上限。若一个 kernel 每读写大量字节只做少量计算，它大概率受带宽限制；若每个字节对应很多计算，它可能受计算吞吐限制。图上的点不需要做得华丽，但要能解释下一步：若离带宽屋顶很远，先看访问模式、预取、对齐和写分配；若接近带宽屋顶，继续微调指令可能收益有限；若离计算屋顶很远，先看依赖链、向量化、端口和分支。Roofline 不是替代分析，而是把分析约束在机器资源上。

四、**计数器要互相校验** 性能计数器会受权限、虚拟化、采样、复用和微架构定义影响。一个指标不能单独定罪。比如 cache miss 高，可能是随机访问，也可能是预取策略不同；分支 miss 高，可能是输入随机，也可能是间接调用；IPC 低，可能是内存等待，也可能是前端供给不足。报告应把计数器、反汇编、输入构造和时间分段放在一起看。若计数器不可用，就要写明降级方案，比如用 wall time、字节数、可控输入和二进制证据构造较弱但诚实的结论。

五、**结论必须带边界** 一个优化在桌面 CPU 上有效，不代表在手机、云主机或另一个编译器上有效。报告结尾要写清机器型号、核心数、频率策略、编译器版本、编译选项、数据规模、输入分布和运行次数。还要写未验证边界：是否测试了冷缓存，是否测试了多进程干扰，是否测试了 NUMA，是否测试了错误输入。好的测量报告不是把结论说死，而是让别人知道怎样复现、怎样质疑、怎样继续验证。

5.13 案例：一次看似成功的优化怎样被重新判读

假设日志解析器优化后从每秒一百万行提升到一百三十万行。这个数字看起来很好，但还不能直接写进结论。首先要问输入是否相同：行长分布、错误行比例、字段数量、字符集、是否命中页缓存。其次要问输出是否相同：错误行是否仍被记录，字段顺序是否一致，统计值是否可复现。最后要问环境是否相同：CPU 频率、后台任务、编译选项、内存分配器和文件系统缓存。测量章要训练的就是这种“不急着相信好消息”的能力。

一、把提升拆成阶段 若总耗时下降，可能是解析变快，也可能是读取变快、写出减少、错误路径被跳过、内存分配减少或输出缓存变大。报告应分段计时：read、parse、filter、partition、write、commit。若 parse 没变而 write 变快，优化标题就不能写“解析器优化”；若错误行处理被跳过，优化可能是错误语义退化。分段计时能阻止结论偷换。

二、用输入族验证稳定性 同一个优化要跑短行、长行、固定字段、缺失字段、错误行密集和随机字符。若只在短行上变快，在长行上变慢，说明优化依赖输入分布。高质量报告会画成表：每类输入的行数、平均长度、错误率、输出大小和主要瓶颈。这样读者能知道优化适合什么场景，而不是记住一个单点速度。

三、计数器解释变化方向 如果优化减少了分支错误，分支相关计数器应有对应变化；如果优化改善了局部性，cache miss 或带宽利用应有证据；如果优化提高了 SIMD 使用，反汇编或向量化报告应能看到批量指令。计数器不一定精确，但方向应和叙述一致。若时间变快而所有证据都解释不了，先怀疑测量方法、输入差异或系统噪声，而不是编造原因。

四、记录失败的实验 报告里应保留失败尝试。比如手写预取没有收益、手写展开导致指令缓存压力、改用无锁队列让尾延迟变差。这些失败不是丢脸材料，而是边界知识。经典教材好看的原因之一，是它让读者知道方法为什么成立，也知道何时不成立。本书也要保留反证过程，让读者学会判断，而不是只背最终答案。

五、把结论写成可执行建议 最终结论不要写成“优化有效”，而应写成带条件的建议：在输入行长较短、错误率低、输出写出不是瓶颈时，冷热路径分离能降低分支回滚；在长行和错误密集输入上，应优先优化错误记录和内存分配。这样的结论能指导后续工程。性能报告的最高质量，不是数字最大，而是别人读完后知道下一步该做什么。

5.14 贯穿案例：三倍加速报告怎样被拆成证据链

假设团队提交了一份报告：新解析器比旧解析器快三倍。用户看到这个数字当然高兴，但教材不能在这里鼓掌结束。我们要像审稿人一样把它拆开。第一问：新旧版本是否处理同一份输入，错误行、空字段、超长字段、编码异常是否一致。第二问：输出是否一致，checksum、错误记录、字段顺序、manifest 条目是否匹配。第三问：时间范围是否一致，是否把读取、解析、聚合、写出和提交都算进去。第四问：环境是否一致，页缓存、CPU 频率、后台任务、编译器参数是否变化。三倍加速只有通过这些问题后，才可能成为工程结论。

这个案例要训练的不是怀疑一切，而是给数字建立出处。性能报告的每个数字都应该能追到输入、代码版本、测量范围和模型解释。经典教材读起来舒服，是因为它们不会把图表当作结论，而会让读者知道图表为什么能支持某个判断。本章也要保持这种纪律。

一、先把语义对齐写在第一页 优化版若少处理错误行、跳过校验、改变字段裁剪规则，它当然会快。报告第一页应放正确性表：输入行数、成功解析行数、错误行数、输出 key 数、输出 checksum、错误日志 checksum、manifest generation。若这些项不一致，性能比较暂停。性能优化不是另一份程序，它必须仍然解决同一个问题。

二、再把时间切成阶段 总时间下降可能来自解析本身，也可能来自写出减少、错误日志少写、内存分配变化、文件已经在页缓存中。报告要拆成 read、parse、aggregate、write、commit。若 parse 只快百分之十，总时间快三倍，就说明另一个阶段被改变了；若 write 时间消失，可能是输出语义变了；若 read 时间大幅下降，可能是热缓存而不是算法。分段计时让结论不能偷换。

三、用字节账本约束吞吐 每处理一行，程序读多少输入字节，写多少输出字节，访问多少临时数组，是否写错误日志。若报告声称吞吐远超机器内存带宽，可能是字节数漏算、缓存状态不同或只统计了热循环不统计 I/O。Roofline 和字节账本不是为了画漂亮图，而是用于质疑不合理数字。一个可信报告会主动说明自己离带宽上限多远，离计算上限多远，下一步瓶颈更可能在哪里。

四、计数器只能和反例一起使用 性能计数器不是法官。branch miss 降低支持“分支预测改善”这个解释，但还要看输入是否改变；cache miss 降低支持“局部性改善”，但还要看工作集是否相同；IPC 提高可能来自前端更顺，也可能来自少做了工作。报告应准备反例输入：规则行、随机行、错误密集行、长行。若新解析器只在规则行快三倍，在错误密集行变慢，结论就必须带条件。

五、保留失败尝试让结论更可信 高质量报告会写哪些优化无效。比如手写预取没有收益，说明硬件预取已经足够或访问模式不适合；展开循环让短行变慢，说明代码体积或启动成本增加；无

锁队列提高吞吐却恶化 p99，说明平均值掩盖尾延迟。失败尝试不是废话，它们告诉读者边界在哪里。没有失败记录的报告很容易像广告，而不是工程材料。

六、把结论写成下一步决策 审查结束后，报告不写“新版本更好”，而写成可执行决策：在规则日志和低错误率场景中默认启用；错误密集输入保留旧状态机或进入慢路径；长行输入继续优化内存分配和写出；移动设备上需要重新测频率和温度影响；没有 PMU 权限时用分段时间和输入族替代计数器。这样的结论能指导项目，而不是只留下一个漂亮倍数。

5.15 案例深化：性能报告审稿清单

可信测量章节要训练读者像审稿人一样看自己的报告。假设报告说“新解析器快了三倍”。第一反应不应是接受或否定，而是追问：输入是什么，版本是什么，编译命令是什么，机器状态是什么，结果是否一致，样本有多少，瓶颈解释是否和计数器一致。没有这些材料，三倍只是一个孤立数字。

一、先审正确性 性能报告第一项是 reference 对比。字段数、错误数、输出 checksum、manifest 条目、边界输入都要一致。若优化版跳过了错误行、改变了浮点合并顺序、少处理了尾部，它当然可能更快。正确性没有过关时，任何性能数字都不进入讨论。这个顺序要在全书保持一致。

二、再审样本和环境 一次运行不能代表性能。报告应说明运行次数、预热策略、是否固定 CPU 频率、是否隔离后台任务、输入是否在页缓存中、编译器和参数是什么。移动设备或 Termux 环境还可能受温度、调度和省电影响，更要保留多次样本。若无法控制环境，就诚实写出限制，并用趋势而不是绝对数做判断。

三、用模型质疑数字 Roofline 和字节账本是用来质疑结果的。若程序声称达到远超内存带宽的吞吐，可能是字节数漏算或缓存命中被误当成内存读；若程序声称受算力限制，但 IPC 很低且 cache miss 高，模型就不支持结论。计数器和模型相互校验，任何一方都不能单独成为真相。

四、Profile 要能定位到决策 火焰图显示某函数耗时高，只说明采样集中在那里，不自动说明怎么改。要继续问这个函数里是分支、内存、锁、I/O 还是系统调用。若 profile 显示 parser 热，可能是分支预测，也可能是错误路径，也可能是分配。报告要把 profile 结果连接到下一步实验，而不是把图当作结论。

五、最后写未验证边界 高质量报告会说明哪些东西没有测。没有 PMU 权限时，branch miss 只能推测；没有清理页缓存时，冷读结论不可靠；输入只覆盖规则日志时，随机字段结论未知；单机测试通过，不代表分布式重试正确。未验证边界不是扣分项，它告诉读者结论能用到哪里，不能用到哪里。

5.16 案例复盘：三倍加速为什么不能直接相信

一份报告说解析器三倍加速，却没有输入摘要、编译命令、运行次数、页缓存状态和 checksum。这样的数字不能指导工程。测量章节要让读者像审稿人一样追问：结果是否正确，样本是否稳定，模型是否支持，计数器是否解释得通。

先审正确性，再审速度 字段数、错误数、输出 checksum、manifest 条目先一致，时间比较才有意义。优化版如果少处理错误行或改变尾部规则，它当然可能更快。随后检查样本分布、CPU 频率、绑定策略、编译器版本和输入是否热缓存。

模型用来质疑数字 Roofline 和字节账本不是装饰。若吞吐超过可能带宽，说明字节漏算或缓存状态不同；若声称受算力限制但 IPC 很低、cache miss 很高，就要回到 profile。可信测量的价值在于让性能结论可复查，而不是给最终答案盖章。

5.17 本章习题

习题与作业

习题一：为顺序归约、多累加器归约和线程本地 partial 归约设计一份实验矩阵。要求列出输入规模、线程数、运行次数、预热策略、计时范围、正确性校验和需要记录的计数器。每个变量都要说明用于排除哪一种错误解释。

习题二：写一段报告，解释为什么热原子归约即使使用 relaxed 内存序也可能扩展性很差。报告必须同时使用 C++ 语义、cache line 所有权和测量指标三个层面的语言。

习题三：设计一个队列 benchmark。要求区分 SPSC、MPSC 和 MPMC 三种竞争模式，记录吞吐、队列长度和等待次数。说明为什么只测单生产者单消费者不能证明一个 MPMC 队列适合线程池全局队列。

习题四：阅读当前系统的 `perfstat` 可用事件。记录哪些事件可用，哪些事件因权限或平台不可用。对不可用事件，写出替代实验方案，而不是直接放弃分析。

第 6 章

数据布局、局部性与预取

先看一个结构体改造事故：团队给日志记录对象加了十几个诊断字段，热路径仍然只读取 timestamp、level 和 key，吞吐却下降。原因不是诊断字段参与了计算，而是每条记录变大后，同一条 cache line 能装下的热字段变少，扫描时带入了大量冷数据，TLB 覆盖也变差。后来把热字段拆成连续数组，冷诊断放到旁路表，性能恢复，诊断能力也没有丢。

本章从这种“字段组织方式改变硬件访问”的问题出发。数据布局决定程序怎样使用缓存、TLB、预取器和内存带宽。很多高性能优化不是改变算法复杂度，而是把热字段、冷字段、读字段、写字段和并发槽位摆成硬件更容易消费的形状。理解布局，是理解数组、结构体、队列、图和矩阵内核的共同基础。

6.1 从访问模式到数据布局

Array of Structures 把一个对象的字段放在一起，适合经常同时访问完整对象的场景。Structure of Arrays 把同类字段连续存放，适合批量处理某个字段的场景。例如粒子模拟中，如果每轮只更新所有粒子的 x 坐标，SoA 更容易形成连续访问和 SIMD；如果每次处理一个粒子的全部属性，AoS 更直观。

选择布局不能只看代码好不好写，要看热路径读写哪些字段、是否需要向量化、是否有 false sharing、是否跨页跳转。工程中常见折中是 AoSoA，把数据按小块组织，既保留局部对象关系，又适合 SIMD 和缓存块。

Listing 6.1 AoS 与 SoA 的访问差异

```
1 struct Particle {  
2     float x;  
3     float y;  
4     float z;  
5 };  
6  
7 struct ParticleSoA {  
8     std::vector<float> x;  
9     std::vector<float> y;  
10    std::vector<float> z;  
11 };
```

如果热路径只更新所有粒子的 x，SoA 版本会连续读取和写入 x 数组；AoS 版本每次还把 y 和 z 附近的数据带入 cache line。若热路径总是同时使用 x、y、z，AoS 的空间局部性反而更自然。

布局选择要从访问矩阵出发，而不是从结构体是否“面向对象”出发。可以把字段列成行，把热路径列成行，在每个格子里标出读、写、只读配置、冷路径调试。若一个热路径只读少数字段，SoA 或冷热分离更有优势；若多个字段总是一起使用，AoS 或 AoSoA 更清晰；若需要 SIMD，字段连续性和对齐会更重要；若需要并发写，线程到字段和槽位的映射必须避免 false sharing。

```
hot path: update position  
reads: x, y, z, vx, vy, vz  
writes: x, y, z
```

```
hot path: compute bounding box  
reads: x, y, z
```

```
writes: thread local min and max

cold path: debug print
reads: name, id, material, history
```

这张访问矩阵会自然推出布局：调试字段不应混在高频数值字段里；只在统计阶段使用的字段不应污染每帧更新；线程本地输出应分开存放，最后合并。

预取 硬件预取器擅长识别顺序访问和固定步长访问。连续数组扫描通常能被预取器提前加载，随机指针追踪则很难。软件预取可以在少数场景有效，但如果预取太早会挤出有用缓存，太晚则来不及隐藏延迟。

最好的预取优化通常不是手写预取指令，而是改变访问模式：把随机访问排序，按块处理，压缩节点，减少指针跳转，让硬件预取器能看懂程序。

软件预取只有在地址能提前算出、未来确实会访问、当前有足够独立工作、预取距离合适时才可能有效。指针追踪中，如果下一步地址必须等当前节点加载完成，预取很难发挥作用；批量处理多个游标时，才有机会提前为另一个游标发起预取。预取错误还可能污染缓存，把真正热的数据挤出去。

因此本书把预取看成最后一步，而不是第一步。先让数据连续，先分块，先减少对象分散，先测出内存延迟或带宽瓶颈，再考虑预取指令。很多工程代码的性能来自“让硬件预取器自然工作”，而不是手写大量 prefetch。

冷热数据分离 对象里不是所有字段都同样热。把热字段和冷字段混在一起，会让缓存带入很少使用的数据。冷热分离把热路径频繁访问的字段放在紧凑结构中，把调试信息、统计字段、名字、配置等冷数据放到旁边结构或间接存储。数据库执行器、游戏引擎和网络服务都常使用这种思路。

冷热分离也有代价：代码复杂度增加，对象关系不如原来直观。是否值得做，要看热路径字段访问比例和缓存压力。设计数据结构时，先写清哪个路径是热路径，哪个字段在热路径必读或必写。

6.2 生命周期、分配器和容器形状

小对象频繁分配会带来分配器开销、碎片、TLB 压力和缓存局部性问题。高性能系统常用 arena、对象池、批量分配或结构化生命周期来减少热路径分配。并发分配器还要避免全局锁和跨线程释放导致的远端缓存流量。

常见误区

不要把“使用堆”简单等同于慢。真正的问题是生命周期是否清晰、分配是否在热路径、对象是否分散、是否跨线程释放、是否破坏局部性。

对象池和 arena 的核心收益不是“分配函数少调用几次”这么简单。它们把生命周期相近的对象放在相近内存中，减少元数据开销和碎片，提高遍历局部性，也让释放可以批量发生。并发环境下，还可以按线程或 NUMA 节点维护本地池，减少全局分配器锁和远端释放。

代价同样要写清楚。Arena 适合批量释放，不适合单个对象精确析构；对象池可能保留大量空闲内存；跨线程归还对象可能重新引入同步；自定义分配器如果没有测试，可能比通用分配器更难维护。高质量教材不能只说“用内存池更快”，必须说明适用的生命周期形状。

6.3 并发写、索引和图布局

多线程程序的数据布局还要考虑写共享。线程本地数据应尽量分开，跨线程只读数据可以共享，跨线程写数据要减少同一 cache line 上的冲突。并行归约、线程池队列和日志缓冲区都常常需要为每个线程准备独立槽位。

结构体大小和 ABI 结构体字段顺序影响 padding 和总大小。较大的结构体会降低缓存密度，也会增加拷贝成本。公共 ABI 中随意调整字段顺序可能破坏二进制兼容，因此高性能库常把内部热结构和外部稳定接口分开。内部结构按性能布局，外部接口保持稳定语义。

6.4 迁移、测试和报告

数据布局优化应按流程进行。第一步，确认热路径和访问矩阵。第二步，估算每轮实际需要读写的字节数。第三步，检查对象是否连续、是否跨页、是否存在共享写。第四步，设计一个只改变布局、不改变算法语义的对照版本。第五步，用相同输入和相同绑定策略测量时间、cache、TLB 和扩展曲线。第六步，评估代码复杂度是否值得。

很多布局优化失败，是因为同时改了太多东西：换容器、改算法、改线程数、改内存分配、改输入规模。这样即使变快，也不知道原因。教材实验必须强调单变量对照，让读者学会建立因果证据。

AoSoA：折中不是妥协 AoS 和 SoA 常被讲成二选一，但真实系统经常使用 AoSoA。它把数据按小块分组，每个块内部采用 SoA，块与块之间保持对象批次关系。这样可以让一个块适配 SIMD 宽度或缓存块，同时又不会把整个系统改造成完全分离的字段数组。粒子系统、物理模拟和向量数据库里都能看到类似结构。

Listing 6.2 AoSoA 的基本形态

```
1 constexpr std::int32_t PARTICLE_BLOCK_WIDTH = 8;
2
3 struct ParticleBlock {
4     std::array<float, PARTICLE_BLOCK_WIDTH> x;
5     std::array<float, PARTICLE_BLOCK_WIDTH> y;
6     std::array<float, PARTICLE_BLOCK_WIDTH> z;
7     std::array<float, PARTICLE_BLOCK_WIDTH> vx;
8     std::array<float, PARTICLE_BLOCK_WIDTH> vy;
9     std::array<float, PARTICLE_BLOCK_WIDTH> vz;
10 };
11
12 struct ParticleBlocks {
13     std::vector<ParticleBlock> blocks;
14     std::int32_t size = 0;
15 };
```

这里的块宽度不是魔法数字。它可能来自 SIMD lane 数、cache line 大小、对齐要求、尾部处理成本和代码复杂度。块太小，循环层次变多但复用不足；块太大，尾部浪费和缓存压力上升。AoSoA 的价值是提供一个调节旋钮，让布局能同时服务向量化和对象组织。

AoSoA 也有成本。随机访问第 *i* 个对象需要计算块号和块内偏移；插入删除比 `std::vector<Particle>` 更复杂；调试打印不如 AoS 直观；接口要隐藏布局细节，否则业务代码会到处写块索引。工程中通常把 AoSoA 封装在专用容器或视图后面，让热路径拿到高效访问，普通路径仍保持清晰 API。

访问矩阵如何落到代码 访问矩阵不是文档装饰，它应该直接影响类型设计。假设日志统计系统的每条记录有时间戳、用户 ID、事件类型、原始文本、解析状态和调试信息。热路径只需要用户 ID 和事件类型做聚合，原始文本只在错误报告时使用。如果把所有字段放在一个大结构里，聚合时每次都把冷字段附近的数据带入缓存。更好的设计是把热字段压成连续数组，把冷字段放在旁边，通过索引关联。

Listing 6.3 冷热分离的记录批次

```
1 struct HotLogBatch {
2     std::vector<std::uint64_t> user_ids;
3     std::vector<std::int32_t> event_types;
4 };
5
6 struct ColdLogBatch {
7     std::vector<std::string> raw_lines;
```



```

8  std::vector<std::int32_t> parse_status;
9  };
10
11 struct LogBatch {
12     HotLogBatch hot;
13     ColdLogBatch cold;
14 };

```

这个设计牺牲了“一个对象包含所有字段”的直观性，换来热路径更少的数据移动。它还让并发更容易：聚合 worker 可以只读 **hot**，错误处理线程再查看 **cold**。若业务经常需要完整记录，冷热分离可能不值得；若绝大多数请求只走热路径，它就很自然。布局设计必须从访问频率和访问组合出发。

索引优于指针的场景 指针直接表达对象关系，但在大规模数据结构中，索引常常更适合性能和序列化。索引可以是 `std::uint32_t` 或 `std::uint64_t`，指向连续数组中的位置。它比指针更紧凑，容易持久化和跨进程传递，也让对象搬迁和压缩更容易。图的邻接表、编译器 IR、游戏 ECS 和列式存储都大量使用索引。

索引也有代价。每次访问需要多一步数组寻址；删除对象后要处理空洞、代际版本或稳定 ID；越界和悬空索引需要调试检查。若系统需要稳定引用，可以使用“索引加 generation”的句柄，避免旧索引误指向新对象。这个思想和无锁结构里的 ABA 类似：同一个位置再次被复用时，需要版本信息区分“看起来一样”和“语义上还是同一个对象”。

Listing 6.4 索引句柄表达稳定引用

```

1 struct EntityId {
2     std::uint32_t index = 0;
3     std::uint32_t generation = 0;
4 };
5
6 struct EntitySlot {
7     std::uint32_t generation = 0;
8     bool alive = false;
9 };

```

这里使用明确位宽类型，是为了让内存布局和序列化边界清楚。系统教材中不能随手使用平台相关宽度的整数类型。布局设计和数据类型选择是一件事：类型宽度会影响结构体大小、cache 密度、文件格式和网络协议。

压缩、编码和计算成本 有时减少内存流量比减少计算更重要。列式数据库和搜索系统常把整数压缩成变长编码、差分编码或位图。这样每个元素读取的字节数减少，但解码需要额外 CPU。是否划算取决于瓶颈：若系统受内存带宽限制，压缩可能提高吞吐；若系统受计算限制，压缩可能变慢。

这个权衡也出现在分布式 shuffle 中。压缩中间数据可以减少网络 and 磁盘 I/O，但会增加 CPU 时间和延迟。单机 cache 层面的“少搬字节，多做一点计算”和分布式层面的“少发网络，多做一点压缩”是同一个思想。教材应让读者从底层内存开始建立这种成本交换模型。

并发写布局 并发数据布局要先标出写者。字段是否共享，不看它是否在同一个结构体里，而看哪些线程写它、写入频率如何、是否落在同一 cache line 上。一个常见错误是把多个线程的统计字段放进同一个 **Stats** 数组，觉得每个线程写自己的下标就安全。结果每个字段很小，多个线程的字段落在同一条 cache line，形成 false sharing。

更好的设计是把写热字段按写者分开，并且让每个写者独占 cache line 或至少独占写热点。读者汇总时再扫描这些槽位。若读取频率很高，可以维护分层汇总；若读取只是指标展示，可以接受延迟。并发布局本质上是在写入吞吐、读取成本和内存占用之间取舍。

Listing 6.5 线程本地统计槽位

```

1 struct alignas(64) WorkerStats {
2     std::uint64_t tasks_executed = 0;
3     std::uint64_t steal_attempts = 0;
4     std::uint64_t empty_polls = 0;
5 };
6
7 std::vector<WorkerStats> stats(static_cast<std::size_t>(worker_count));

```

这类结构适合 worker 高频写自己的统计。它不适合每个请求都读取全局精确值。如果业务需要实时限流，统计槽位还要配合周期性汇总或原子发布。布局优化总是要回到语义。

迁移旧代码的策略 把成熟代码从 AoS 改成 SoA 或冷热分离，风险很高。直接大改会破坏接口、测试和调试习惯。稳妥策略是分阶段迁移。第一阶段写访问矩阵和性能证据，证明布局确实是瓶颈。第二阶段增加新布局的内部表示，但保留旧接口，通过 adapter 暴露兼容访问。第三阶段把热路径迁到新布局，保留 reference 对照。第四阶段逐步清理旧路径。

迁移时要防止“双写不一致”。如果旧结构和新结构同时存在，更新必须保证二者一致，或者明确一个是权威数据，另一个是缓存。缓存需要失效策略；双写需要事务边界；延迟同步需要版本号。布局重构不是单纯换容器，而是数据所有权迁移。

标准容器的内存形状 数据布局不只发生在自己定义的结构体里，也发生在每一次选择标准容器时。`std::vector` 的元素连续存放，适合顺序扫描、批量复制、SIMD 读取和按索引访问。它的扩容会搬迁元素，因此外部保存元素指针或引用时必须小心。`std::deque` 不是一个大连续数组，而是由多个固定大小块组成，头尾插入更稳定，但长距离顺序扫描的局部性通常不如 `std::vector`。`std::list` 每个节点独立分配，插入删除不移动其他节点，但遍历时会产生指针追踪、cache miss、TLB miss 和分配器开销。很多初学者只记住复杂度表，却没有把复杂度和硬件成本合起来看。

例如一个任务运行时维护 ready task。如果任务总数在构图后基本稳定，`std::vector` 加索引队列或环形缓冲通常更适合。如果任务需要频繁从中间删除，但遍历又很热，链表也未必正确，因为删除的理论优势可能被遍历成本吞掉。若真的需要稳定句柄，可以用索引加 generation，而不是直接把每个任务做成堆上节点。容器选择不是 API 风格问题，而是访问模式、生命周期和硬件局部性的共同结果。

Listing 6.6 索引队列比链表更容易保持局部性

```

1 struct ReadyTask {
2     std::uint32_t task_id = 0;
3     std::uint32_t generation = 0;
4 };
5
6 class ReadyRing {
7 public:
8     explicit ReadyRing(std::uint32_t capacity)
9         : buffer_(capacity), head_(0), tail_(0), size_(0) {}
10
11     bool push(ReadyTask task) {
12         if (this->size_ == this->buffer_.size()) {
13             return false;
14         }
15         this->buffer_[this->tail_] = task;
16         this->tail_ = (this->tail_ + 1) % this->buffer_.size();
17         ++this->size_;
18         return true;
19     }
20

```

```

21 std::optional<ReadyTask> pop() {
22     if (this->size_ == 0) {
23         return std::nullopt;
24     }
25     const ReadyTask task = this->buffer_[this->head_];
26     this->head_ = (this->head_ + 1) % this->buffer_.size();
27     --this->size_;
28     return task;
29 }
30
31 private:
32     std::vector<ReadyTask> buffer_;
33     std::size_t head_;
34     std::size_t tail_;
35     std::size_t size_;
36 };

```

这个例子还不是并发队列，因为它没有锁和内存序。它要说明的是另一个层次：在考虑同步之前，先让数据本身连续、容量有界、访问路径可预测。后续给它加互斥或 SPSC 语义时，至少不会再被链式节点和频繁分配拖住。很多高性能结构的第一步不是“无锁”，而是“固定容量、连续存储、清楚所有权”。

分配器不是只影响分配速度 分配器常被理解成“申请内存快不快”，但它对布局的影响更深。通用分配器需要支持不同大小、不同生命周期、不同线程的分配和释放，因此会维护元数据、空闲链表、线程本地缓存和大块映射。小对象如果被零散分配，遍历时访问的是分配器历史留下的空间分布，而不是业务逻辑希望的顺序。一个图算法使用链式邻接节点，即使每个节点只有十几个字节，也可能因为节点分散而消耗大量 cache 和 TLB 资源。

Arena 的优势来自生命周期整齐。解析一个输入文件时产生的临时节点，可以在解析结束后一次性释放；构建一轮任务图时产生的任务对象，可以在图执行结束后批量销毁。这样做减少了单个对象释放的成本，也让相近生命周期的数据更容易靠近。缺点是对象不能随意提前释放，析构顺序要清楚，长期存活对象不能混进短生命周期 arena。否则 arena 会保留大量已经无用但不能回收的空间。

对象池则适合固定大小、频繁复用的对象，例如网络请求上下文、日志批次缓冲、任务描述符。对象池可以把对象初始化和内存申请分开，减少热路径分配；也可以按线程或 NUMA 节点分池，减少跨线程竞争。对象池的风险是复用对象时忘记清理旧状态，或者跨线程归还导致远端 cache line 流量。高质量实现要把 reset、owner thread、debug generation 和峰值容量都写进设计。

Listing 6.7 固定大小对象池的基本接口形状

```

1 struct RequestContext {
2     std::uint64_t request_id = 0;
3     std::uint32_t attempt = 0;
4     std::uint32_t worker_id = 0;
5 };
6
7 class RequestPool {
8 public:
9     explicit RequestPool(std::size_t capacity) : storage_(capacity) {
10         for (std::uint32_t index = 0;
11             index < static_cast<std::uint32_t>(storage_.size());
12             ++index) {
13             this->free_.push_back(index);
14         }

```



```

15 }
16
17 std::optional<std::uint32_t> acquire() {
18     if (this->free_.empty()) {
19         return std::nullopt;
20     }
21     const std::uint32_t index = this->free_.back();
22     this->free_.pop_back();
23     this->storage_[index] = RequestContext{};
24     return index;
25 }
26
27 void release(std::uint32_t index) {
28     this->storage_[index] = RequestContext{};
29     this->free_.push_back(index);
30 }
31
32 private:
33     std::vector<RequestContext> storage_;
34     std::vector<std::uint32_t> free_;
35 };

```

这个池子仍然只是单线程教学版本。并发版本必须决定 free list 由谁保护，是否每个 worker 一个本地池，跨线程释放是否进入 remote free 队列，是否要用 generation 防止旧句柄复用。不要因为代码短就忽略这些语义。内存池的工程难点不在“写一个 vector”，而在生命周期、并发所有权和调试可见性。

面向数据的接口设计 布局优化失败的常见原因，是内部换了 SoA，外部接口仍然要求每次返回一个完整对象。这样热路径又不得不临时拼对象，或者频繁跨数组取字段，收益被接口吃掉。面向数据的接口应该让调用者表达批量意图。例如不是提供 `get_particle(i)` 返回完整粒子，而是提供 `x_span()`、`velocity_span()` 或批处理函数，让热路径直接操作连续字段。

接口也要避免泄漏实现细节。完全暴露 `std::vector<float>&` 会让外部随意 `resize`，破坏布局不变量；完全隐藏成单元素 `getter` 又丢掉批量优化机会。折中做法是暴露只读或可写 `std::span`，并用类型说明生命周期和访问权限。只读输入、可写输出、临时 `scratch`、线程本地 `partial` 应该在函数签名中区分开。

Listing 6.8 批量接口比单元素接口更能表达热路径

```

1 struct PositionView {
2     std::span<float> x;
3     std::span<float> y;
4     std::span<float> z;
5 };
6
7 void integrate(PositionView position,
8 PositionView velocity,
9 float delta_time) {
10     const std::size_t count = position.x.size();
11     for (std::size_t i = 0; i < count; ++i) {
12         position.x[i] += velocity.x[i] * delta_time;
13         position.y[i] += velocity.y[i] * delta_time;
14         position.z[i] += velocity.z[i] * delta_time;
15     }

```

16 }

这个函数把布局意图表达给编译器和读者：三个坐标数组连续，循环边界统一，写入位置清楚。实际生产代码还要检查 span 长度一致、别名关系和对齐条件。接口设计的目标不是把所有优化都写进函数，而是不要一开始就把优化可能性封死。

ECS 和列式布局 Entity Component System 常见于游戏引擎，但它背后的思想适用于任何大量对象、少数字段热访问的系统。实体只是 ID，组件按类型连续存放，系统按组件批量处理。渲染系统关心位置和网格，物理系统关心位置和速度，AI 系统关心状态和目标。每个系统只扫描自己需要的组件，避免把完整对象的冷字段一起带入缓存。

日志分析和数据库执行器也使用类似思想。列式存储把同一列连续存放，过滤某个字段时只读取相关列；投影阶段再把需要的列组合输出。若查询只需要用户 ID 和事件类型，就不应该读取完整 JSON 文本。若后续需要原始文本，再通过行号或 offset 访问冷数据。列式布局的核心不是“数据库专属格式”，而是按访问组合移动最少字节。

列式布局还有压缩优势。同一列的数据类型相同、分布相近，更容易做字典编码、差分编码、位图和运行长度编码。压缩后减少内存和 I/O 流量，但增加解码成本。若过滤条件能直接在压缩表示上执行，收益很大；若每次都要完整解码，收益可能下降。布局、编码和算法必须一起设计。

CSR：图结构的局部性案例 图算法最容易暴露指针结构的问题。一个朴素邻接表如果每条边都是独立节点，遍历邻居时会不断指针跳转。Compressed Sparse Row 把所有边目标压到一个连续数组里，再用 offsets 数组标出每个顶点的邻居范围。这样遍历顶点 v 的邻居时，只需要扫描 `edges[offsets[v]]` 到 `edges[offsets[v+1]]` 之间的连续区间。

Listing 6.9 CSR 图的核心布局

```
1 struct CsrGraph {
2     std::vector<std::uint32_t> offsets;
3     std::vector<std::uint32_t> edges;
4 };
5
6 std::uint64_t degree_sum(const CsrGraph& graph) {
7     std::uint64_t total = 0;
8     for (std::size_t vertex = 0; vertex + 1 < graph.offsets.size(); ++vertex) {
9         const std::uint32_t begin = graph.offsets[vertex];
10        const std::uint32_t end = graph.offsets[vertex + 1];
11        total += static_cast<std::uint64_t>(end - begin);
12    }
13    return total;
14 }
```

CSR 的代价也要讲清楚。它适合静态图或批量更新后的图，不适合高频单边插入删除。插入一条边可能需要移动后续大量边，或者维护增量结构再周期性重建。它还把顶点 ID 映射和边排序变成预处理问题。很多系统会使用双层结构：主图用 CSR 保持扫描性能，增量更新放在小的补丁区，定期合并。这个设计和 LSM-tree、列式数据库的 delta store 很像：热更新和冷扫描分层处理。

块大小的选择方法 分块是布局优化的核心技巧，但块大小不能拍脑袋。选择块大小时至少要考虑五个因素。第一，块内工作集是否能放进目标 cache 层级。第二，块内循环是否能形成足够 SIMD 和指令级并行。第三，块大小是否造成尾部处理过多。第四，块是否方便按线程或 NUMA 节点分配。第五，块元数据是否过多。

以矩阵乘法为例，一个块需要同时容纳 A 的子块、B 的子块和 C 的子块。若三者合起来超过 L1 或 L2，块内复用就会下降。以日志聚合为例，一个批次需要容纳 key、value、hash、输出缓冲和临时计数。批次太小，函数调用和调度成本高；批次太大，cache 复用差，背压粒度粗。块大小应通过候选集实验选择，例如从 1 KiB、4 KiB、16 KiB、64 KiB、256 KiB 开始，观察吞吐、cache miss、TLB miss 和尾延迟。

块大小还影响错误恢复。分布式计算中，任务块越大，调度开销越低，但失败重做成本越高；块越小，负载均衡更好，但调度和元数据成本更高。单机 cache block、线程池 task grain、分布式 partition chunk 其实是同一个问题在不同尺度上的版本。

可测量的布局报告 布局优化的报告至少应包含四类数据。第一，结构体大小和对齐：使用 `sizeof`、`alignof`，列出字段顺序和 padding。第二，访问字节数估算：每处理一个元素理论上读取和写入多少字节，哪些字节是冷字段。第三，硬件指标：时间、带宽、cache miss、TLB miss、分支和 IPC。第四，语义影响：接口是否改变，生命周期是否改变，是否影响稳定引用和并发写入。

只有耗时数字的报告不够。假设 SoA 比 AoS 快了 1.8 倍，你还要说明原因是否来自少读冷字段、SIMD 更好、预取更稳定、TLB 覆盖更高，还是因为新版本顺便少做了别的工作。若无法解释，就应该补实验。可以构造一个只访问全部字段的场景，如果 AoS 反而更快，说明 SoA 的优势确实来自字段子集访问，而不是神秘加速。

下面是一份布局报告的文字模板，但写报告时要填真实数据。

```
layout report:
workload: aggregate event type from N records
baseline: vector<Record>, hot fields mixed with raw line
candidate: HotLogBatch plus ColdLogBatch
semantic change: external record id remains stable
bytes per hot record: baseline estimated 80, candidate 12
metrics: time, cache misses, dTLB misses, bandwidth
result: candidate improves hot aggregation, cold error path unchanged
```

布局优化的反例 布局优化也会失败。第一类失败是工作集很小，本来就全部在 cache 里，换布局只增加代码复杂度。第二类失败是热路径需要完整对象，SoA 反而让字段分散在多个数组中，增加地址计算和 cache line 数。第三类失败是业务频繁插入删除，CSR 或紧凑数组重建成本太高。第四类失败是并发访问模式改变，新布局把原来分散的写入集中到同一 cache line。第五类失败是接口没有同步迁移，内部布局高效，外部仍然单元素调用。

因此本章要求读者把布局选择放回问题本身：热路径读哪些字段，写哪些字段，数据能否批量处理，生命周期是否成批，是否有稳定 ID 需求，是否多线程写，是否需要序列化，是否有恢复和调试要求。布局是系统设计，不是局部技巧。

日志系统的布局案例 贯穿项目的日志统计可以用一个具体布局案例串起来。原始日志行是冷数据，解析后的 event type、user id、timestamp 和 value 是热数据，解析错误信息是低频冷数据。若每条记录都保存成一个包含字符串和多个字段的大结构，聚合 event type 时会把大量原始文本和调试字段带入 cache。更好的设计是先把输入按 batch 解析，热字段放入列式数组，原始行或错误上下文按 offset 另存。

这种设计还方便错误处理。正常路径只扫描热字段；遇到解析错误时，把错误行号和原始 offset 写入冷错误列表；需要输出错误报告时再回到原始文本。热路径和冷路径分开后，性能和调试都更清楚。很多系统性能差，不是因为算法复杂，而是每次处理热字段都拖着一堆冷调试信息走。

Listing 6.10 日志批次的热冷布局

```
1 struct ParsedLogHotBatch {
2     std::vector<std::uint64_t> timestamps;
3     std::vector<std::uint64_t> user_ids;
4     std::vector<std::uint32_t> event_types;
5     std::vector<std::int64_t> values;
6 };
7
8 struct ParsedLogColdBatch {
9     std::vector<std::uint64_t> raw_offsets;
10    std::vector<std::uint32_t> error_rows;
11 };
```

这个布局不要求原始输入立即复制。`raw_offsets` 可以指向 `mmap` 文件或输入缓冲中的位置，但生命周期要清楚：如果输入缓冲释放，`offset` 就不能再用于错误报告。布局 and 生命周期永远绑定在一起。

ECS 变体：稀疏集合 ECS 常用 `sparse set` 维护实体到组件位置的映射。`Dense` 数组保存实际组件，`sparse` 数组从 `entity id` 映射到 `dense index`。遍历组件时扫描 `dense`，局部性好；查询某个 `entity` 是否有组件时，用 `sparse` 快速定位。删除时可以用 `swap-remove`，把最后一个元素移到删除位置，再更新 `sparse`。这个结构牺牲稳定顺序，换取紧凑遍历。

如果业务需要稳定顺序，`swap-remove` 就不合适；如果 `entity id` 非常稀疏，`sparse` 数组可能很大；如果组件跨线程频繁增删，需要同步。再次说明，数据结构没有脱离语义的绝对优劣。`Sparse set` 适合大量实体、组件遍历热、增删可接受重排的场景。

Columnar batch 和 vectorized execution 数据库的 `vectorized execution` 通常以 `batch` 为单位处理列式数据。一个 `batch` 可能包含几千行，每个算子处理一个 `batch` 后交给下一个算子。`Batch` 太小，函数调用和调度成本高；`batch` 太大，`cache` 压力和延迟增加。`Batch` 模型是 `row-at-a-time` 和全量材料化之间的折中。

日志统计项目可以借鉴这个模型。输入解析成 `batch`，`filter` 处理 `batch`，`histogram` 处理 `batch`，`shuffle` 按 `batch` 发送。每个 `batch` 有热字段列、选择向量、错误列表和统计摘要。这样项目既能展示数据布局，也能展示 I/O 背压：下游慢时，上游 `batch` 队列满，读取暂停。

布局和序列化的差异 内存布局和磁盘/网络格式不必相同。内存中为了计算使用 `SoA`，网络上为了兼容和压缩使用长度前缀或 `protobuf` 类格式，磁盘上为了恢复使用稳定 `manifest`。直接把内存结构 `dump` 到磁盘，短期方便，长期会被 `ABI`、`padding`、字节序和版本拖垮。高质量系统会显式转换格式。

转换本身有成本，因此需要批处理和缓冲。一次转换一条记录，函数调用和分配成本高；一次转换一个 `batch`，可以顺序读取列、批量编码、批量校验。布局优化和 I/O 章节在这里相遇：好的内存布局也要服务序列化。

缓存友好的哈希表 哈希表是系统里常见的性能热点。链地址法每个 `bucket` 指向链表节点，容易指针追踪；开放寻址把元素放在连续数组中，探测时更 `cache` 友好，但删除、扩容和高负载因子要小心。`Robin Hood hashing`、`SwissTable` 类结构通过控制探测长度和元数据字节改善局部性。教材不要求实现这些成熟结构，但要让读者知道 `std::unordered_map` 不是唯一选择。

日志聚合如果使用 `std::unordered_map<std::string, std::int64_t>`，每个 `key` 可能分配字符串和节点，局部性较差。若先把 `event type` `intern` 成 `std::uint32_t`，再用数组或开放寻址表，热路径会更清楚。数据布局优化常常从“把复杂 `key` 转成紧凑 `id`”开始。

内存布局的调试工具 布局优化需要工具。可以用 `sizeof` 和 `alignof` 检查结构体大小；用编译器警告或工具查看 `padding`；用地址打印检查数组连续性；用 `perfstat` 观察 `cache` 和 `TLB`；用 `heap profiler` 看分配热点；用 `sanitizer` 检查越界和 `use-after-free`。布局优化如果没有调试工具，很容易变成猜测。

项目可以增加一个 `layout report` 命令，打印关键结构体大小、对齐、字段数量、`batch` 容量和估算字节数。这个命令不直接提升性能，但能让读者看到数据结构的物理形状。系统教材应让抽象类型落到字节。

布局迁移的测试策略 从 `AoS` 迁移到 `SoA` 时，测试要覆盖读写一致性。可以保留旧 `AoS reference`，随机生成记录，分别填充 `AoS` 和 `SoA`，执行同一聚合，比较结果。还要测试边界：空 `batch`、单元素、非对齐长度、错误行、超长字符串、冷热字段缺失。迁移期间如果保留双表示，要测试更新一个字段后另一个表示是否同步或明确无效。

性能测试则要比对至少三条路径：完整记录访问、只访问热字段、错误报告访问。`SoA` 可能在热字段聚合上快，但完整记录访问变慢。报告要承认这种取舍，而不是只展示最好场景。

对齐分配器和实际收益 有些内核希望输入按 32 字节或 64 字节对齐。可以使用对齐分配器或平台 API 分配内存，也可以让容器内部保证对齐。对齐能减少跨 `cache line` 访问和满足某些 `SIMD` 要求，但它不是万能优化。若访问本来连续且 CPU 对非对齐访问支持良好，收益可能很小。若切片从对齐数组中间开始，起始地址仍可能不对齐。

项目中可以把对齐作为实验参数。生成同样数据，分别使用普通 vector、对齐缓冲、故意偏移一个元素的视图。比较自动向量化和手写 chunked 版本。报告中写清对齐如何保证、如何验证、对尾部 and 偏移如何处理。

Scratch buffer 管理 高性能算法常需要 scratch buffer，例如 scan 的 block sums、partition 的 counts、top-k 的候选、解析的临时位置列表。每次函数调用都分配 scratch，会引入分配器成本和内存波动。更好的做法是由运行时或调用者提供 scratch，或者每个 worker 复用本地 scratch。

Scratch 复用要清楚生命周期和大小。一个 worker 处理过大 batch 后，scratch 可能扩容到很大，后续小任务继续占用内存。可以设置最大保留容量，超过后释放或归还池。Scratch buffer 是数据布局和资源管理的交叉点。

Layout API 的封装 内部使用 SoA 或 AoSoA，不代表外部调用者要知道所有数组。可以提供 batch view，让热路径拿 span，冷路径通过 record id 获取完整记录。封装的目标是同时保护布局不变量和提供批量访问能力。单元素 getter 方便但可能慢；完全暴露 vector 容易破坏不变量。View 是折中。

Listing 6.11 批量视图封装布局

```
1 struct EventColumnsView {
2     std::span<const std::uint32_t> event_types;
3     std::span<const std::int64_t> values;
4 };
5
6 EventColumnsView event_columns(const ParsedLogHotBatch& batch) {
7     return EventColumnsView{batch.event_types, batch.values};
8 }
```

这个接口让聚合内核只看到自己需要的列，不依赖完整 batch 结构。后续如果 batch 从 vector 改成 mmap 或压缩列，接口仍有机会保持稳定。

布局章节总结 本章的主线是让数据形状服务访问形状。顺序热访问需要连续，字段子集热访问需要列式或冷热分离，高频并发写需要分片和对齐，稳定引用需要索引和 generation，静态图需要 CSR，动态更新需要增量结构。布局优化不是把所有东西改成一种格式，而是让每条热路径少搬无用字节、少追随机指针、少共享写。

设计布局时，先写访问矩阵，再写生命周期，再写并发写者，再写序列化边界，最后才写代码。这个顺序能避免“为了性能换结构，最后语义乱了”的常见错误。

布局决策表 选择布局时，可以先填表。字段是否总是一起访问？若是，AoS 可能清晰；若只访问少数字段，SoA 或冷热分离更好。对象是否需要稳定地址？若需要，考虑索引句柄或对象池；若不需要，连续数组更简单。是否高频插入删除？若是，CSR 这类紧凑静态结构可能不合适。是否要 SIMD？字段连续和对齐更重要。是否要序列化？内存布局和 wire format 要分开。是否多线程写？写者要分片或对齐。

这张表能避免布局争论变成风格争论。面向对象、面向数据、列式、池化都只是工具。真正决定工具的是访问矩阵、生命周期和并发语义。

案例：任务图的布局 任务图也需要布局。最直观的表示是每个节点一个对象，每个对象有 `std::vector` successors。小图这样写清楚，大图会有很多小 vector 分配，遍历后继时指针跳转多。CSR 式任务图把节点和边分别放入连续数组，ready count 在节点数组中，successor id 在边数组中。执行时，worker 完成节点后顺序扫描后继范围，局部性更好。

缺点是动态修改困难。如果任务图在执行中不断新增边，CSR 重建成本高。可以使用两层结构：静态主图用 CSR，动态新增任务用小的 pending edge 列表，stage 结束后合并。这个设计和静态图加增量边、列式主存加 delta store 是同一模式。布局思维可以迁移到运行时元数据。

案例：模型权重的预告 虽然 AI 在第三册展开，但模型权重布局可以作为预告。权重通常是大块只读数据，推理时被反复扫描；量化权重可能按 block 存放 scale、zero point 和 int8 数据；算子希望连续读取并适配 SIMD。这个场景和本章的列式、AoSoA、对齐、冷热分离完全相关。第二册学习日志 batch 和矩阵 block，不是只为日志系统，而是为后续权重和 tensor 布局打基础。

这也说明布局知识具有迁移性。字段从 event type 换成 tensor value，原则不变：热数据连续，元数据分离，block 大小匹配 cache 和 SIMD，序列化格式与内存计算格式分开。

布局章的回归阅读应围绕一次真实迁移，而不是只列 AoS、SoA 名词。起点是 **LogRecord** 对象数组：字段完整、接口直观，但热字段和冷字符串混在一起。第一步统计哪些字段进入热循环，哪些只用于错误报告；第二步把热字段迁移到列式 batch；第三步保留稳定 row id，让错误路径仍能回到原记录；第四步用测试保证迁移前后语义相同。

布局迁移最容易犯的错，是只看快路径而破坏生命周期。字符串视图是否指向仍然有效的 arena，列式数组扩容后 span 是否悬垂，冷热分离后错误报告还能否定位原始行，CSR 图结构的 offset 是否覆盖最后一个顶点，这些都属于布局语义。数据结构不是只为 cache 服务，也要为所有权、错误路径和测试服务。

本章验收应要求一份迁移说明。说明中写清旧结构、热字段、冷字段、新结构、兼容层、回滚方式和测量结果。若性能提升来自减少无用字节搬运，就估算减少了多少；若没有提升，也要说明瓶颈是否已转向分支、I/O 或同步。这样布局优化就不会变成凭感觉重排字段。

预取讨论也要保持克制。硬件预取器擅长连续和固定步长访问，不擅长依赖上一次 load 的链式结构；软件预取可能隐藏部分延迟，也可能污染 cache、增加指令压力或预取错误路径。讲预取时应先证明地址能提前知道，再证明被预取的数据会被使用，最后测量预取距离。没有这三步，预取只是把不确定性提前。

本章还应把索引和句柄讲清。列式 batch 常用整数索引连接热字段和原始记录，索引位宽要由最大 batch 大小决定，不能随意使用模糊整数。句柄比裸指针更适合跨阶段传递，因为它能在 arena 迁移、vector 扩容和错误回溯时保持稳定。布局优化的成熟标志，是热路径拿到连续数据，冷路径仍能可靠定位上下文。

布局迁移还要考虑写路径。读多写少的数据适合冷热分离和列式压缩；频繁写入的数据若被拆成太多列，可能增加写放大和一致性成本；多个线程写相邻字段时，还要考虑 false sharing。比如统计 partial 的桶数组适合按线程或按 NUMA 节点分开，而不是所有线程写同一个紧凑数组。数据布局不是只为读取服务，写入模式同样决定结构。

内存分配策略也要进入本章。大量小对象如果逐个 new，会带来分配器锁、元数据开销、碎片和随机地址；arena 可以把生命周期相同的对象放在一起，但会让单个对象释放变难；对象池可以复用内存，但要处理并发和悬垂引用。项目中解析阶段的临时字符串、错误记录和 batch 元数据都适合讨论这些取舍。

6.5 让数据结构服从访问模式

数据布局不是把 AoS、SoA、CSR、Arena 几个词背下来，而是先问访问模式。每条记录会读哪些字段，写哪些字段，字段是否一起出现，访问顺序是否连续，是否跨线程共享，生命周期是否一致，是否需要稳定引用。答案不同，布局就不同。若不先写访问矩阵，任何布局建议都像偏好。

访问矩阵可以很朴素：行是阶段，列是字段，格子写读、写、只在错误路径读或不访问。日志解析阶段需要原始文本、offset 和错误信息；统计阶段只需要 event type 和少量数值；输出阶段需要 key 和计数；调试阶段可能需要原始行。矩阵一画，冷热分离就不是术语，而是避免热阶段搬运冷字段的直接结果。

AoS、SoA 和 AoSoA 的选择

AoS 适合每次处理完整对象，代码直观，生命周期统一。SoA 适合批量处理少数字段，便于 SIMD、预取和列式扫描。AoSoA 在二者之间折中，把固定大小块内的字段列式化，既保留一部分对象局部性，又让热循环按列访问。选择不是凭风格，而是看一轮热循环到底读哪些字节。

例如日志 batch 若每次都要解析原始字符串并立刻生成输出，AoS 足够简单；若解析和统计分成两个阶段，统计阶段只看 event type、user bucket 和 timestamp bucket，SoA 更合适；若后续 SIMD 按 16 或 32 条记录处理，AoSoA 可以让一个块大小贴近向量宽度和 cache。章节要给出失败反例：若把所有内容都 SoA 化，但后续错误报告频繁需要回到原始记录，索引和跨列访问也会增加成本。

C++ 实现要把布局封装成批量视图，而不是让业务代码到处知道内部数组名字。接口可以提供 `std::span<const std::uint32_t>` 之类的热字段视图，避免拷贝，也让编译器看见连续范围。单元素 getter 在控制面可以接受，在热数据面会隐藏批量访问机会。布局优化最终会落到 API 设计，这一点比单纯换容器更重要。

索引、句柄和生命周期

指针表达“某个地址”，索引表达“某个容器中的位置或 id”。高性能系统常用索引句柄，因为对象可能存在 compact vector、arena 或 slab 中，索引更容易序列化、移动和检查代际。裸指针访问快，但生命周期一旦跨队列或异步边界，就容易悬垂。句柄可以带 generation，检测旧引用，代价是每次访问要多一次间接和检查。

对象池解决的是生命周期和分配形状，不是所有问题的答案。固定大小对象池适合大量同类型、同生命周期对象，例如 task node、queue slot、message header。若对象大小差异大、生命周期不规则，池可能造成内部碎片或延迟释放。Arena 适合整批释放，例如一个 stage 的临时节点；不适合需要单独回收的长期对象。教材要让读者把 allocator 看成生命周期设计的一部分。

索引还改善序列化和分布式边界。一个 task id、split id 或 batch id 可以写进 trace 和 manifest；一个进程内指针不能跨进程。第二册虽然主要讲单机计算，但后续分布式章节会把这些 id 继续使用。数据结构如果一开始就把稳定身份和内存地址分开，后面扩展到重试和持久化会顺很多。

预取、压缩和分块

预取器喜欢可预测地址。连续扫描和固定步长访问容易被硬件预取，随机哈希和指针链表难以预取。软件预取只在能提前知道未来地址、且当前有足够工作覆盖延迟时有意义；如果预取距离不合适，可能浪费带宽或污染缓存。讲预取时要从地址序列推导，不要把 `__builtin_prefetch` 当成固定优化。

压缩是用计算换带宽。小整数 key 可以用更窄类型保存，位图可以压缩布尔标记，字典编码可以把字符串变成整数 id。压缩后数据更小，cache 和 TLB 压力下降，但解码会增加指令和分支。若热路径频繁访问压缩字段，解码成本可能抵消收益；若压缩字段只用于 I/O 或跨网络传输，收益更明显。数据布局章要把压缩放进成本账本，而不是当作存储技巧。

分块连接缓存、SIMD 和任务粒度。块太小，调度和边界成本高；块太大，超过 cache 或造成负载不均。AoSoA 的块大小、任务 chunk size、I/O batch size 和 SIMD width 不是彼此无关的参数。贯穿项目可以先固定一个 batch size，再在测量章模板下扫描：记录 cache miss、吞吐、队列长度和尾延迟，说明为什么选择当前值。

布局迁移和测试策略

布局优化最容易破坏接口。把对象数组改成列式数组后，原先返回引用的 API 可能失效；原先指针稳定的容器可能因为 reallocation 失效；原先按插入顺序遍历的输出可能因为分区而改变顺序。迁移时先加 reference 和适配层，再逐步把热路径改到新布局。不要一次把语义、布局、并发和输出格式全改掉。

测试要覆盖内存安全和性能边界。功能测试比较输出；地址稳定性测试检查句柄 generation；压力测试覆盖大量插入删除；并发测试检查跨线程传递生命周期；性能测试比较字节移动和 cache 行为。若项目使用 sanitizers，要说明哪些路径能覆盖，哪些性能实验不能开 sanitizer 因为开销太大。

本章的项目交付物是 `ParsedLogBatch`、`TaskArena` 和 `StableHandle` 三类设计说明。每个说明都写访问模式、生命周期、所有权、失效规则和对照实验。这样后续 SIMD、任务图和分布式提交使用这些结构时，不需要重新猜它们为什么长这样。

案例走查：从对象数组迁移到列式 batch

迁移第一步不是改所有代码，而是增加一个适配层。保留原始 `LogRecord` reference，新增 `ParsedLogBatch`，由 parser 填充热字段数组和冷字段索引。统计内核只接收热字段 span，错误报告仍可通过索引回到原始文本。这样语义边界清楚，热路径也能变窄。

第二步写一致性测试。随机生成记录，分别走对象数组和列式 batch，比较最终计数、错误列表和稳定排序输出。再写边界测试：空 batch、单条记录、超长字段、错误行、重复 key。只有这些通过后，才做性能测量。否则布局迁移带来的错误会被误认为性能优化。

第三步测字节移动。报告不只写耗时，还写每条记录热字段字节数、冷字段字节数、batch 容量、内存峰值和 cache 行为。若列式版本在小输入上变慢，也要解释：可能是额外索引和初始化成本超过了缓存收益。这样的反例能帮助读者理解布局优化的边界。

布局 API 的设计作业

让读者为 `ParsedLogBatch` 设计 API：必须能追加记录、获得热字段 span、根据错误索引找到原始行、清空并复用内存、报告当前容量和字节数。禁止热路径返回可变内部 vector 引用，避免上层破坏不变量；允许控制面查询统计信息。这个作业能把布局和封装联系起来。

API 还要定义失效规则。追加记录是否会导致 span 失效？batch move 后句柄是否有效？clear 后旧索引是否可用？这些问题看似 C++ 容器细节，实际会影响异步任务和队列传递。若一个 span 被提交给 worker 后，上游继续追加导致 reallocation，worker 就可能读悬垂内存。所有权合同必须写清。

布局 API 的性能测试要检查是否发生意外拷贝。函数参数尽量用 `std::span` 和 `std::string_view`；大对象只读传 `const` 引用；返回热字段视图不复制。报告可以用分配计数或简单日志确认热循环没有每条记录分配。这样读者能把 C++ API 设计和缓存局部性连起来。

布局迁移的源码阅读

可以阅读成熟列式系统或数据库执行引擎的公开资料，但本书本地代码重点是自己实现小型 batch。Linux 源码侧则看内存分配和 page fault 的用户态表现，理解为什么大量小对象会造成分配器和页层面的压力。读 `/proc/self/status` 的 `VmRSS`、`/proc/self/smaps` 的匿名映射，也能帮助观察布局迁移后的内存峰值。

在 C++ 标准库层面，读 `std::vector` 的失效规则比读某个实现源码更重要。vector reallocation 会让指针、引用和迭代器失效；deque 分段存储，随机访问多一次间接；list 节点稳定但局部性差；unordered map rehash 会失效迭代器且分配多。数据布局章要把这些容器规则和硬件成本一起讲。

项目验收要求写“为什么不用链表”。不是因为链表低级，而是因为本项目热路径需要连续扫描、批量处理和稳定索引。能为不用某个结构给出理由，比只会说“vector 快”更接近工程判断。

综合练习：改造一份日志记录结构

给读者一个故意臃肿的 `LogRecord`：时间戳、用户字符串、事件字符串、原始行、解析错误、调试标签、计数字段全部放在一起。要求先画访问矩阵，而不是直接改结构。Parser 读原始行并写字段；统计阶段只读事件 id 和用户桶；错误报告读原始行和错误；输出阶段读聚合结果。根据矩阵提出热冷分离方案。

第二步实现迁移计划。先保留原结构作为 reference，再新增 `ParsedLogBatch`。Parser 同时填充两种结构，测试输出一致；然后统计内核改用 batch 热字段；最后删除热路径对旧结构的依赖。每一步都要能独立提交和回滚。若一次性删除旧结构，错误很难定位。

第三步写失效规则。Batch 追加是否使 span 失效，任务入队后 batch 是否可继续写，错误索引在 clear 后是否可用，跨线程传递是移动所有权还是共享只读。读者必须把这些写进接口注释或设计文档。布局优化如果不写生命周期，会变成悬垂引用来源。

验收包括功能、内存和性能三类。功能比较 reference，内存记录峰值和分配次数，性能记录热循环字节数和耗时。若性能提升不明显，也要解释是否输入太小、冷字段本来就少、编译器已经优化、或瓶颈不在内存。这个题训练的是完整迁移，而不是单点改结构。

容易出错的边界：数据布局不该破坏语义

第一个反例是为了列式布局丢失错误上下文。热路径只需要 event id，但错误报告需要原始行、列号和字段名。如果拆分后无法回到原始文本，系统可诊断性下降。正确设计是热字段和冷索引分离，冷字段不进热循环，但仍能按 id 找回。第二个反例是为了对象池复用 buffer，导致下游异步任务读到被覆盖数据。生命周期不清的布局优化，比慢代码更危险。

第三个反例是为了压缩字段引入溢出或错误解码。把 user id 从 `std::uint64_t` 压成 `std::uint32_t`，必须证明输入范围；把字符串字典化，必须处理未知 key 和版本；把布尔标记压成位图，必须处理并发写和位操作原子性。压缩是语义选择，不是单纯内存优化。

第四个反例是过度泛化布局 API。为了支持所有访问模式，接口提供任意字段组合和动态反射，结果热路径无法内联，数据访问重新变成间接。高性能系统常常保留窄而明确的热路径 API，同时在控制面提供通用查询。把这一区分写清楚，读者才能理解为什么经典系统代码经常有专用内核。

本章源码索引可以让读者阅读自己项目中的所有 `span`、`string_view` 和指针捕获位置，检查是否跨异步边界。每个跨边界的 view 都要回答上游对象是否仍然活着。这个检查比单纯看 cache miss 更能防止实际工程 bug。

本章核心接口：BatchView

布局章给项目留下 `BatchView`。它只暴露热字段的只读 span、记录数量、batch id 和错误索引查询，不暴露内部 vector 的可变引用。写入由 builder 完成，读取由 view 完成。这样上游可以构造 batch，下游只能按合同读取，避免热路径外部破坏布局。

BatchView 的生命周期写进类型文档：view 不拥有数据，不能跨过 owning batch 的生命周期；提交给异步任务时，要移动 owning batch 或使用共享只读所有权。这个接口会直接保护后续 SIMD、线程池和 I/O 章节的安全。

从数据布局进入 SIMD 和批量执行

数据布局把热字段整理成连续、可解释、生命周期清楚的批量视图。只有到了这一步，SIMD 和自动向量化才有稳定基础。若数据仍然散在对象、链表和字符串里，向量化只能面对 gather、分支和间接访问；若 BatchView 提供连续 span，编译器和手写内核才有机会把多个元素放进同一条批量语义。

项目应把 SIMD 章看作布局章的验收。一个布局如果不能支持批量扫描、尾部处理和错误索引，它就没有真正服务热路径。反过来，SIMD 失败也可能提示布局还不够清楚：字段不连续、别名不明确、输出位置不稳定或错误路径混入热循环。两章之间应形成反馈，而不是前后孤立。

6.6 项目落地

Compute Systems Engine 可以在本章加入两个实验模块。第一个是记录批处理模块：用 AoS、SoA 和冷热分离三种布局统计 key，比较热字段扫描、错误记录访问和内存占用。第二个是 worker stats 模块：紧凑统计槽位与 `alignas(64)` 槽位对照，观察多线程写入扩展性。这两个模块分别训练“读局部性”和“写共享”两种布局能力。

项目代码不必一口气实现复杂 AoSoA 容器，但要把布局实验和 benchmark harness 接起来。每个布局版本必须使用同一份输入、同一个 reference 结果和同一组线程策略。否则布局差异会和算法差异混在一起。

从业务记录推导列式批次 很多布局优化失败，是因为一开始就讨论“要不要 SoA”，却没有先讨论业务记录到底怎样被使用。正确推导应从一个具体记录开始。假设日志记录包含时间戳、用户编号、事件类型、数值、原始行、解析错误、调试标签和来源文件。解析阶段需要原始行、来源文件和错误字段；聚合阶段只需要用户编号、事件类型和数值；错误报告阶段需要原始行和错误字段；调试查询阶段才需要标签。把这些字段全部放在一个大对象里，聚合阶段每处理一条记录都会把大量冷字段一起带进缓存。聚合代码看起来只读三个字段，硬件实际搬动的字节却远多于三个字段。

推导列式批次时，可以先保留最直观的 AoS reference。它的价值是语义清晰，便于测试新布局。然后为热路径建立列式批次：用户编号是一列，事件类型是一列，数值是一列，解析状态可以是一列，原始行和调试信息放入冷批次。热聚合只接收热批次视图，错误报告才接收冷批次视图。这样做并不是为了追求某个抽象名称，而是为了让热路径少搬无用字节，让编译器更容易生成顺序扫描，让硬件预取器更容易工作。

迁移时要特别注意记录编号。列式批次里第 i 个用户编号、第 i 个事件类型、第 i 个数值和冷批次第 i 个原始行必须表示同一条逻辑记录。这个关系可以通过统一长度、统一追加、统一过滤掩码和稳定 record id 保持。如果某个阶段删除错误记录，不能只从一行删除而忘记其他列。更稳的做法是先生成 selection vector，表示哪些记录有效，后续聚合按 selection vector 读取；等到批次稳定后，再做 compact。这样能避免列之间长度和顺序不一致。

列式批次还要考虑异常和边界。解析一行失败时，热列是否填默认值？错误列如何记录原因？聚合阶段是否跳过失败记录？如果错误记录很多，selection vector 会不会成为新的热点？如果原始行很长，冷批次是否保存完整字符串，还是保存文件偏移和长度？这些问题都属于布局设计，而不是外围细节。真正的工程布局必须同时说明成功路径、失败路径、调试路径和序列化路径。

稀疏结构和图布局 图和稀疏矩阵是布局思想最集中的场景。最直观的图表示是每个节点一个对象，每个对象保存一个后继列表。这种写法适合教学和小图，但在大图上会产生大量小分配。遍历一个节点的邻居时，程序先读节点对象，再读 vector 元数据，再跳到邻居数组；如果每个邻居数组都来自不同分配位置，cache 和 TLB 压力会很高。CSR 表示把所有边放到一条连续边数组里，再用 offset 数组记录每个节点的边范围。遍历节点 v 的邻居时，只需要顺序扫描 edges 从 `offsets[v]` 到 `offsets[v + 1]` 的区间。

CSR 的核心收益是把指针追踪变成连续区间扫描。连续扫描让硬件预取器能提前加载，也让多线程分片更容易。若节点范围按编号切分，每个 worker 扫描一段节点和对应边，读路径相对稳定。代价是动态更新困难。插入一条边可能需要移动后续大量边，删除也可能留下空洞。因此 CSR 更适合静态图、阶段性重建图或批处理图。动态图可以使用主 CSR 加 delta 边列表：主图负责绝大多数

只读遍历，新增边暂存在小结构中，阶段边界再合并。

稀疏矩阵也类似。按行压缩适合行遍历和矩阵向量乘；按列压缩适合列遍历；块稀疏适合局部密集区域和 SIMD。选择格式时，不能只说“稀疏矩阵用 CSR”。要看内核按行访问还是按列访问，是否需要转置，是否需要并行写输出，非零分布是否倾斜，行长度差异是否很大。行长度极不均匀时，按行分配线程可能负载不均；可以按非零数量切分，也可以把长行拆成多个任务，但拆长行又会引入输出累加同步。布局、算法和并行调度在这里不能分开。

Compute Systems Engine 后续做 shuffle、join 和任务图时，都可以借用稀疏结构思维。任务图的 successors 数组就是边数组，任务的 successor range 就是 offset。分区后的 key 到记录位置，也可以看成倒排索引或稀疏结构。读者如果只把 CSR 当成图算法专用格式，就错过了更重要的思想：当大量对象之间的关系可以静态收集时，把关系压成连续区间，通常比每个对象挂一串指针更适合现代硬件。

布局实验报告怎样写 布局实验报告必须说明输入规模、记录大小、字段访问比例、线程策略、内存分配方式和测量指标。只报告“快了两倍”没有意义，因为读者不知道快在哪里。一个合格报告至少要写三组结果：完整对象访问、热字段访问、冷路径访问。SoA 可能让热字段聚合更快，但完整对象调试变慢；冷热分离可能减少缓存流量，但增加索引和封装复杂度。报告要把收益和代价都列出来。

指标上，时间只是起点。还应观察 cache miss、TLB miss、内存带宽、分配次数、峰值内存、输出是否一致。多线程布局实验还要看扩展曲线和 false sharing。若紧凑统计槽位在单线程下更快，对齐槽位在多线程下更快，这不是矛盾，而是写共享形状改变了瓶颈。单线程结论不能直接推到多线程，多线程结论也不能忽略内存占用。

报告还要防止把算法变化伪装成布局变化。AoS 版本如果使用普通循环，SoA 版本同时换成 SIMD 和分块，就无法判断收益来自哪里。更严谨的方式是逐步改变：先只改布局，算法保持一致；再加分块；再加 SIMD；再加线程。每一步都有 reference 结果和性能指标。这样得到的不是一组漂亮数字，而是一条可复查的因果链。

访问模式 布局设计从访问模式开始。若每次循环只读 key 和 value，就不应让 debug 字符串、错误码和时间戳陪着进入 cache。先列字段访问频率，再决定哪些字段同列、哪些字段拆出。

AoS 与 SoA AoS 适合整体对象一起处理，SoA 适合批量扫描同一字段。中间还有 AoSoA、分块列式、索引表等形式。教材要让读者根据访问和更新选择，而不是把 SoA 当成万能答案。

生命周期 布局也受生命周期影响。短生命周期对象适合 arena 或 batch buffer，长期对象需要稳定句柄，跨线程对象要明确所有权。只谈 cache，不谈释放和迁移，会把内存安全问题留到后面爆炸。

预取 硬件预取器喜欢稳定步长。手写 prefetch 只有在访问提前量、计算量和内存延迟匹配时才有意义。随机访问、分支复杂和小工作集下，prefetch 可能没有收益甚至污染 cache。

索引替代指针 指针链可读性强，但地址分散。索引能让数据保存在连续数组中，也便于序列化、移动和压缩。代价是需要稳定表和边界检查。教学项目优先用索引表达记录关系。

分配器 大量小对象会把时间花在分配和释放上，也会破坏局部性。arena、pool 和 monotonic buffer 可以减少分配成本，但会改变释放粒度。选择分配器必须和生命周期一起写。

迁移成本 把旧结构迁移到新布局本身要时间和内存。报告要记录转换成本，并说明它是否能被批处理摊销。如果每次查询都临时转 SoA，收益可能被转换抵消。

接口保护 LayoutPlan 应让调用者看到 span、列、索引和版本，而不是暴露内部 vector 让外部随意 push。局部性需要接口约束，否则一次方便调用就能破坏整个布局。

案例：记录表列式化 把 Recordid,key,value,debug 改成 key/value 列和 debug 辅助表。比较只扫描 key/value 的热路径，也记录构建列式 batch 的成本。

案例：arena 生命周期 为一次 batch 分配临时节点，用 arena 一次释放。报告说明为什么不适合长期持有的对象。

案例：索引图 用索引数组表示边，而不是每条边一个 new 节点。观察遍历、序列化和缓存行为的变化。

源码阅读路线 Linux 源码阅读从 mm/page_alloc.c、mm/slab.c、include/linux/mm_types.h 附近进入即可。阅读目标不是把整套子系统抄进书里，而是追一条和本章实验相关的路径：

用户态动作怎样变成内核对象，内核对象怎样排队、等待、唤醒、计数、提交或回收。

阅读报告要写证据等级。能直接测到的数字放在第一层；从源码控制流推导出的解释放在第二层；当前设备无法验证、只能作为合理猜测的内容放在第三层。这样读者会知道布局迁移报告中哪些结论可复现，哪些结论需要换机器继续确认。

源码阅读还要克制。看到一个复杂函数，不要把每个分支都展开；先回到本章的问题，问它能否解释一个等待点、一次状态转移或一条性能曲线。解释不了的内容可以留到索引，不要打断正文主线。

记录表列式化 把 Recordid,key,value,debug 改成 key/value 列和 debug 辅助表。比较只扫描 key/value 的热路径，也记录构建列式 batch 的成本。

arena 生命周期 为一次 batch 分配临时节点，用 arena 一次释放。报告说明为什么不适合长期持有的对象。

索引图 用索引数组表示边，而不是每条边一个 new 节点。观察遍历、序列化和缓存行为的变化。

访问模式

数据结构应从访问模式推出，而不是从习惯推出。顺序扫描、随机查询、按 key 聚合和图遍历需要不同布局。

实验设计：先写访问序列，再选择 vector、deque、哈希表、CSR 或列式批次。

AoS 与 SoA

AoS 适合整对象处理，SoA 适合批量处理单字段。冷热字段混在一起时，扫描热字段会搬运大量无用字节。

实验设计：把日志记录拆成热字段列和冷字段表，比较扫描字节。

容器语义

vector 连续但扩容会移动，deque 分块但长扫描局部性差些，list 插删稳定但遍历成本高。复杂度表必须和硬件成本合看。

实验设计：为同一任务分别用三种容器实现，说明为什么选择最终版本。

生命周期

视图不拥有数据，指针不说明寿命，索引句柄需要 generation 防止复用误判。布局迁移必须同时迁移所有权规则。

实验设计：构造上游释放缓冲、下游仍持有视图的错误案例，再用批次对象修正。

分配器

大量小对象分配会造成元数据成本、碎片和 TLB 压力。对象池能降低成本，但要处理归还、清理和线程归属。

实验设计：比较逐条 new、批量 arena 和固定对象池的分配次数。

预取

硬件预取喜欢规律访问，软件预取只在确实知道未来地址且有足够距离时才有意义。乱加预取可能增加带宽压力。

实验设计：对连续数组、固定步长和指针链分别尝试预取，记录收益边界。

图布局

图算法常受邻接访问和度数倾斜影响。CSR 把邻接表压成连续数组，可以减少节点分配和指针追踪。

实验设计：把邻接 vector 列表转换成 CSR，比较 BFS 的访问形状。

并发写布局

多个线程写相邻统计槽可能 false sharing。线程本地槽位、按 cache line 对齐和分层合并都是布局选择。

实验设计：让每线程写自己的 aligned slot，再与共享桶对照。

迁移计划

布局修改不能一次重写全部。先保留旧接口，新增批量视图和转换层，再逐步让热路径直接消费新布局。

实验设计：报告旧路径、新路径和兼容层成本，避免迁移中破坏语义。

项目对象

LayoutPlan 应说明字段冷热、存储容器、所有权、视图寿命、线程写入位置和迁移步骤。它是性能和接口设计的共同文档。

实验设计：每个布局优化都要更新 LayoutPlan，而不是只改结构体定义。

6.7 数据结构不是容器选择题，而是访问模式的结果

很多读者学数据结构时只记复杂度表：`std::vector` 随机访问 $O(1)$ ，`std::list` 插删 $O(1)$ ，哈希表平均 $O(1)$ 。第二册必须把这张表放回机器里。连续数组的 $O(1)$ 和链表节点的 $O(1)$ ，对 cache、TLB、分配器和预取器完全不同。布局章节的主线不是“哪个容器最好”，而是“访问模式要求数据怎样排列”。

以日志记录为例，原始输入是一行文本，解析后可能包含时间、用户、事件类型、错误文本和调试字段。统计事件类型时，热字段只有事件类型和计数；错误报告时，冷字段才重要。若直接保存一个大结构体数组，热循环会反复搬运冷字段。改成列式热字段后，扫描更快，但错误报告需要稳定 record id 回查原始位置。这个例子说明布局优化不能脱离语义：你减少了数据移动，也引入了索引和生命周期问题。

容器选择也要从访问序列推导。`std::vector` 适合顺序扫描、批量处理和 SIMD；扩容会移动元素，所以外部指针和引用要谨慎。`std::deque` 适合两端增长和较稳定引用，但分块布局让长扫描不如单块连续。`std::list` 插删不移动节点，但每个节点独立分配，遍历会变成指针追踪。教材要让读者看到：复杂度相同，常数和等待形状可能差几个数量级。

生命周期是布局迁移中最容易漏掉的部分。`std::span` 和 `std::string_view` 不拥有数据，只是视图。上游缓冲如果复用或释放，下游视图就悬垂。索引句柄也不等于对象身份：数组槽位复用后，旧 index 可能指向新对象，因此需要 generation 或稳定 id。布局章节若只讲 cache，不讲所有权，会把性能优化变成内存安全风险。

分配器成本同样具体。大量小对象分配会产生元数据、锁竞争、碎片和 TLB 压力。对象池或 arena 能降低成本，但要定义清理、复用和线程归属。一个线程分配、另一个线程释放，可能让分配器跨线程传递缓存；对象池如果没有 generation，旧句柄可能误用新对象。布局优化的代码实现必须包含这些边界，而不是只给一张结构体图。

预取要克制。硬件预取已经能处理许多规律访问；软件预取适合未来地址可知、计算距离足够、带宽仍有余量的场景。指针链上提前预取下一节点有时有用，但如果每次预取都错，或者工作集已经带宽受限，只会增加压力。实验要比较连续数组、固定步长和随机链表，不要把预取当作必胜技巧。

本章项目可以输出一个 LayoutPlan：字段冷热、容器选择、视图寿命、写入线程、对齐策略、分配策略和迁移步骤。迁移不能一次把全部代码改掉。先保留旧结构，新增批量视图和转换层；再让热路径直接消费新布局；最后删除旧路径。这样做更像工程教材，也更接近真实系统演化。

案例推导：把日志对象迁移成批量视图时，接口也要一起迁移

旧接口每次处理一个 `LogRecord` 对象。对象里有字符串、时间戳、事件类型和错误信息。热循环调用 `process(record)`，代码好读，但每次调用都带来对象访问、分支和潜在字符串处理。布局优化的第一反应是把 `std::vector<LogRecord>` 改成多个数组。若只改存储不改接口，热循环仍然每次构造临时对象，收益会被吃掉。真正要迁移的是接口：从单对象处理改成批量视图处理。

批量视图可以包含事件类型 span、用户 id span、时间戳 span 和 record id span。计算内核只读取热字段；错误报告和调试路径通过 record id 回到冷字段表。这个设计让编译器看见连续范围，也让所有权更清楚：视图不拥有数据，批次对象拥有底层存储。调用者不能把视图保存到批次生命周期之外。接口文档和类型要共同表达这个限制。

迁移时不要一次删除旧路径。第一阶段保留旧对象结构，新增从旧结构生成批量视图的转换层，用它验证新内核语义。第二阶段让解析器直接生成热字段列，减少中间对象。第三阶段清理旧对象路径。每一步都要测量转换成本和热循环收益，否则可能出现“内核快了，但总时间慢了”的情况。

对象池也要谨慎。批次反复分配大数组可能造成抖动，池化能降低分配成本；但池化带来复用风险。一个 batch 还在下游使用时，上游不能把同一块内存放回池里。可以用引用计数、队列所有权或显式状态保证。这里再次说明布局优化不是纯 cache 问题，它牵连生命周期和并发。

预取可以作为最后一层，而不是第一层。若热字段已经连续，硬件预取通常能处理；若访问用户表或字典编码表是随机的，可以尝试提前预取下一个 batch 的字典项。但预取距离要通过实验决定，

太近来不及，太远污染 cache。报告应写出输入分布，因为热点字典和随机字典的预取效果不同。

这个案例给读者一个重要判断：如果一个优化只改结构体定义，却没有改访问接口、生命周期和测试输入，它很可能是不完整的布局优化。LayoutPlan 应把这些内容放在同一页上。

实验：实现 AoS 和 SoA 两个版本的字段求和。比较顺序访问和随机访问。再构造两个线程分别写相邻字段和分离字段的场景，观察 false sharing。

数据布局从访问模式推导，不从类型设计推导 很多 C++ 程序先设计对象，再让计算迁就对象。高性能系统往往反过来：先写出热路径访问哪些字段、以什么顺序访问、是否批量访问、是否跨线程写，再决定对象形状。贯穿项目的日志记录可能有时间戳、事件类型、用户 ID、数值、原始文本和错误信息。解析阶段需要原始文本和字段边界，聚合阶段只需要事件类型和数值，错误报告才需要原始文本。若所有字段绑在一个大结构体里，聚合阶段会把大量冷字段带进缓存。

AoS 适合按记录完整处理，SoA 适合按字段批量处理。AoS 让一条记录的所有字段相邻，代码直观；SoA 让同一字段连续，适合 SIMD、预取和列式扫描。项目可以使用两阶段形态：解析阶段产生紧凑的中间列，保留必要的记录身份；错误路径单独保存原始切片；聚合阶段只扫描热列。这样做不是为了追求某种数据布局名词，而是让每个阶段只搬运自己需要的数据。

生命周期决定容器和分配器 数据布局还要看生命周期。Batch 内临时字段、stage 内 partial、全局 manifest、错误报告，它们的生命周期完全不同。短生命周期对象适合 arena 或 monotonic buffer，统一释放；长期对象需要稳定所有权和明确析构；跨线程传递对象需要不可变或独占转移；跨异步 I/O 的 buffer 需要保证提交后直到完成前不能释放。若只看容器 API，不看生命周期，程序很容易在异步和并发章节出问题。

C++ 容器选择要写清成本。`std::vector` 提供连续存储，适合批量扫描；`std::deque` 分段存储，引用稳定性和连续性不同；`std::unordered_map` 查找平均快，但桶、节点、哈希和分配会制造随机访问；`std::map` 有排序语义，但指针追踪多；`std::pmr` 可以把分配策略显式传入。教材不应只说“哪个容器快”，而要说明它保护什么语义，产生什么地址序列。

从 LogRecord 迁移到 BatchView 一个好的案例是把 `LogRecord` 对象数组迁移成 `BatchView`。第一版每条记录是一个对象，包含多个 `std::string` 和字段。它易懂，但解析后聚合时会反复追字符串和对象指针。第二版把热字段提取为连续数组：event type、timestamp、value、status；原始文本保留为 input buffer 的 offset 和 length；错误信息单独放到 error vector。第三版为不同阶段提供只读 view，让聚合函数无法意外访问冷字段。

迁移要保持语义。字段 offset 必须在 input buffer 生命周期内有效；错误报告需要能回到原始行；排序或过滤后，记录身份不能丢；并发处理时，BatchView 的只读部分可以共享，输出部分必须按 partition 独占。这个案例能把数据布局、所有权、内存安全和性能连接起来。

预取要建立在规律访问上 硬件预取器擅长稳定步长和相邻 cache line，不擅长哈希表随机桶和链表。软件预取可以提前请求未来地址，但如果地址本身要等当前 load 才知道，或者预取距离不合适，收益有限甚至有害。很多情况下，重排数据让访问规律，比手写 prefetch 更可靠。比如先收集要访问的索引并排序，或者按 partition 分组后批量处理，能把随机访问转成更接近顺序访问。

项目里可以比较三种 top key 统计方式：直接哈希更新、先按 key hash 分桶再局部更新、对小整数 key 使用连续数组。观察指标不仅是时间，还包括 cache miss、分配次数和输出材料化大小。若数据布局改变导致后续 stage 更容易处理，即使单个阶段收益不大，也可能端到端更好。布局优化要看流水线，而不只看局部函数。

Linux 源码阅读连接点 Linux 内核大量使用 intrusive list、rbtree、xarray、per-cpu 变量和 slab cache。阅读时不要只记结构名称，而要看为什么选择这种结构：是否需要稳定节点地址，是否需要按 key 查找，是否需要范围查询，是否要减少分配开销，是否要 per-cpu 缓存。内核结构往往把生命周期和访问模式写进数据结构设计。

C++ 项目也应学习这种思路。任务对象若频繁创建销毁，可以使用对象池；ready 队列若只在 worker 本地访问，可以用本地 deque；manifest 若需要按 block id 查找和按提交顺序遍历，可能需要两种索引。一个数据结构不能只按“大 O”选择，还要按内存布局、生命周期、并发访问和恢复语义选择。

事故复盘：LogRecord 为什么拖慢缓存 朴素设计喜欢把一行日志装进一个 LogRecord：原始文本、解析字段、错误信息、时间戳、临时标志全在一起。聚合阶段只需要 key 和 value，却把整

块对象搬进 cache。输入一大，热循环不断加载冷字段，TLB 覆盖也被无关对象消耗。这里的问题不是对象风格本身，而是对象边界没有服从访问模式。

从事故推导出的结构是 BatchView。热字段放进连续数组，冷字段保留在原始 arena 或诊断表里，用稳定 record id 关联。扫描和聚合阶段只读 key、value、record id；错误报告阶段再按 id 回到原始行。这样既减少热路径搬运，又不牺牲诊断能力。Span 是视图，不拥有内存；arena 拥有批次生命周期；record id 是热冷数据之间的桥。

迁移布局必须带着测试走。先用 AoS reference 保证语义，再切出 SoA 热字段，检查空字段、超长字段、错误位置和原文回溯。报告不只写扫描变快，还要写内存峰值、分配次数、cache miss、TLB miss、arena 回收点和 span 生命周期。预取可以作为最后的局部优化，但如果访问模式本身混乱，预取只是掩盖设计问题。数据布局章节的核心是先问“谁在什么阶段读哪些字段”。

布局迁移实验判读 布局迁移不是只看扫描速度。热字段变快后，要检查错误报告是否还能回到原始行，record id 是否稳定，arena 是否在批次结束释放，span 是否悬垂。若 SoA 版本快但调试信息丢失，它不是合格迁移；若 AoS 在小批次更快，也要记录原因，可能是转换成本超过收益。

这一段也服务于复习。读者合上书后，应该能凭本章的判读口径重新审查自己的代码：有没有最小正确版本，有没有能击穿旧方案的输入，有没有状态记录，有没有资源指标，有没有失败后的恢复动作。若五个问题缺两个以上，就说明实现还停在演示层，而不是工程层。

从错误报告倒推冷热分离的约束 冷热分离不是把冷字段删掉。日志聚合热路径只需要事件类型、时间戳和数值，但错误报告需要原始行、字段偏移和解析原因。正确设计是热列保持连续，冷表通过 record id 或 offset 回查。这样热路径不搬冷数据，诊断路径仍然完整。

迁移时可以保留双写阶段。解析器同时生成旧 LogRecord 和新 BatchView，用同一组输入比较输出、错误报告和性能。双写阶段成本高，但它是迁移安全网。等 reference、错误路径和性能报告都稳定后，再删除旧结构。工程教材要讲这种过渡，因为真实系统很少允许一次性重写全部数据结构。

还要注意生命周期。BatchView 里的 span 不能比底层 buffer 活得更久；arena 适合批次内对象，不适合跨 stage 长期保存；string view 指向原始输入时，输入缓冲不能提前复用。很多布局 bug 不是访问模式错，而是生命周期没写进类型。

实验课：从 LogRecord 迁移到 BatchView 旧版本定义 LogRecord，包含事件名字符串、用户字符串、时间戳、数值、原始行、错误信息和调试标签。聚合阶段只需要事件 id 和数值，却必须跨过整个对象数组。新版本定义 BatchView，把 event id、value、timestamp 放进连续热列，原始行和错误信息留在冷表，通过 record id 回查。

迁移分三步。第一步双写：解析器同时产出 LogRecord 和 BatchView，用同一批输入比较结果。第二步切换热路径：聚合只读 BatchView，但错误报告仍从冷表恢复原始行。第三步删除旧路径前，运行错误密集输入，验证错误位置、原始偏移和错误原因没有丢。每一步都要保存内存峰值、分配次数、扫描时间和输出 checksum。

判读时要看两类收益。热路径收益来自连续访问、少搬冷字段、少分配；工程收益来自生命周期清楚，batch 结束后 arena 可以统一释放。代价也要写：转换阶段更复杂，record id 必须稳定，冷路径回查可能更慢。若错误报告不再完整，布局迁移就失败；若小输入转换成本超过收益，也要在报告里写阈值。

这节实验还要讲 prefetch 的边界。若访问已经连续，硬件预取可能足够；若访问随机，软件预取也常常太晚。比起手写 prefetch，更稳的改进通常是改变布局，让未来地址更早、更规则地出现。

布局迁移练习验收重点 合格答案要同时覆盖热路径和诊断路径。BatchView 快不够，还要能通过 record id 找回原始行和错误原因。答案要写对象生命周期，说明 span 指向谁、arena 何时释放、冷字段是否可能被异步使用。若这些说不清，代码再快也不合格。

答案还要写“为什么不采用更复杂方案”。例如没有证据证明队列是瓶颈，就不应把锁队列换成无锁；没有证明输出顺序可变，就不应随意重排；没有证明错误路径能恢复，就不应只提交正常路径。能解释不做什么，说明读者开始具备工程判断。

最后，答案必须包含可复查材料。至少有输入摘要、关键状态、期望输出、实际输出、运行命令和未验证边界。没有这些材料，老师或同伴无法判断结论是否成立。第二册的作业不是让读者写漂亮段落，而是训练读者把系统问题变成可验证证据。

项目实现：数据布局、局部性与预取 本章对应的贯穿项目实现不要求一次写成完整框架，但必须有最小可运行切片。这个切片包含三个部分：一个 reference，一个带本章机制的实现，一个能

击穿 reference 之外隐含假设的测试。Reference 用来固定语义，机制实现用来展示本章知识，测试用来证明新增复杂度确实解决了问题。

代码组织上，不要把所有内容放进一个大文件。基础类型、输入生成、核心机制、报告输出和测试应分开。即使当前只是教学项目，也要让目录能表达责任边界。这样后续章节接入时，不会因为找不到状态归属而重新发明一套对象。

每个实现都要输出一段机器可读摘要。摘要至少包含输入规模、关键状态、错误数、输出校验、耗时和未验证边界。这里的机器可读不要求复杂格式，简单的 key value 文本也可以。关键是让脚本和读者都能复查，而不是只能看屏幕截图。

测试应包含正常路径、边界路径和失败路径。正常路径证明功能可用；边界路径证明长度、容量、尾部、空输入、关闭或超时没有遗漏；失败路径证明本章机制不是装饰。若失败路径写不出来，往往说明本章概念还没有落到工程对象上。

报告最后写一个取舍段。新增机制带来了什么好处，增加了什么成本，哪些场景不应该使用它，下一章会复用哪个字段或对象。这个取舍段非常重要，因为系统工程不是把所有高级机制都加进去，而是在证据和约束下选择合适的边界。

调试笔记：布局变快后检查诊断路径 冷热分离或列式化后，必须专门跑错误输入。若错误报告找不到原始行，说明 record id 或 offset 丢失；若 span 指向已释放 buffer，说明生命周期合同错误；若异步写出还持有 batch 指针，说明 owner 过早复用。布局优化最容易在正常路径显得漂亮，在诊断和异步路径暴露问题。

复查清单：数据布局、局部性与预取 复查本章时，先检查 reference 是否仍然存在。没有 reference，后面的并发、优化、异步或分布式改造都无法证明语义没有变。再检查压力输入是否尖锐：它应当攻击本章最核心的边界，而不是只把数据量放大。第三，检查状态是否可观察：关键对象的身份、状态、错误和提交结果应能在报告中找到。

第四，检查代码示例是否说明禁止行为。调用者不能做什么，往往比能做什么更能体现工程质量。第五，检查实验是否写出未验证边界；当前设备、权限或输入无法证明的结论，必须标注。第六，检查本章对象是否能被下一章复用，不能复用就说明边界还不稳。

完成这六项后，再看文字是否连贯。每个概念都应从问题推出，每个案例都应从朴素方案走到改进方案，每个实验都应能解释一个判断。若某段话放到其他章节也成立，就把它改成本章的具体对象、具体输入和具体失败。

6.8 工程案例：把日志记录从对象改造成数据流

数据布局不是把结构体换成数组这么简单。它要从访问模式、生命周期、并发写和错误诊断一起推导。日志记录在程序员眼里像一个对象：原始文本、时间戳、用户、事件类型、错误状态、调试字符串都放在一起很自然。可是在热路径里，解析和统计可能只需要时间戳、事件类型和少量数值字段。把所有东西塞进一个对象，会让 cache line 搬运大量冷数据。

一、先写访问矩阵 访问矩阵记录每个阶段读写哪些字段。读取阶段需要原始字节和偏移；解析阶段写热字段和错误标志；统计阶段读事件类型和数值；shuffle 阶段读 key、partition 和 partial；诊断阶段才读原始文本和错误原因。若一个字段只在错误路径使用，就不应出现在每条记录的热结构体中。

访问矩阵能防止盲目 SoA。并不是所有对象都要列式化。若一组字段总是一起访问，AoS 可能更简单；若某个阶段只扫描一个字段，SoA 会减少无效搬运；若字段数量多但访问组合固定，AoSoA 可能折中。布局选择来自访问矩阵，而不是风格偏好。

二、热字段和冷字段分离 日志项目可以把解析结果分成 HotBatch 和 ColdStore。HotBatch 保存 event_type、timestamp_delta、user_bucket、record_id 等连续数组；ColdStore 保存原始文本切片、错误消息和调试上下文。统计阶段只读 HotBatch；出错或需要审计时，再通过 record id 查 ColdStore。

Listing 6.12 热字段批次只保存高频扫描字段

```
1 struct ParsedHotBatch {
2     std::span<const std::uint32_t> event_types;
3     std::span<const std::uint64_t> timestamp_ns;
```

```

4     std::span<const std::uint32_t> user_buckets;
5     std::span<const std::uint64_t> record_ids;
6 };

```

这个接口用 `span` 表达只读视图，不拥有内存。拥有者可以是 `batch storage` 或 `arena`。热路径函数不需要知道错误字符串放在哪里，这让编译器和读者都能看清访问范围。冷字段不是不重要，而是访问频率低，应从热循环拿出去。

三、生命周期决定能不能用视图 `std::string_view` 和 `std::span` 不拥有数据。它们适合表达只读窗口，但前提是底层存储活得足够久。异步 I/O、线程池和队列会拉长生命周期：生产者解析完 `batch` 后，消费者可能稍后才读取；如果生产者复用 `buffer`，视图就悬垂。布局设计必须和所有权设计一起做。

安全路线是让 `batch` 拥有自己的 `backing storage`，或者使用引用计数块，或者通过对象池保证直到下游完成才复用。不要为了减少拷贝随手把局部字符串暴露成 `view`。高性能 C++ 里，避免拷贝和保证生命周期同等重要；只做到前者会制造更隐蔽的 `bug`。

四、分配器成本不只在分配时出现 大量小对象分配会带来锁竞争、元数据、碎片、TLB 压力和缓存污染。日志解析若每个字段都创建 `std::string`，即使字符串很短，也可能把时间花在分配器和拷贝上。批量 `arena` 或块级 `buffer` 能把许多小分配变成少量大分配，释放时按块回收。

但 `arena` 也有代价。块太大导致内存峰值高，错误路径保留太多冷数据，异步消费者拖住 `arena` 释放，都会造成问题。报告要记录 `batch` 大小、`arena` 容量、峰值内存和下游持有时间。布局优化不能只看热循环快不快，还要看内存生命周期是否可控。

五、预取只适合可预测但来不及的访问 软件预取不是万能钥匙。顺序扫描通常硬件预取已经足够；随机访问预取地址如果太晚，来不及；如果太早，可能污染 `cache`；如果预取大量不会用的数据，会浪费带宽。适合预取的场景是地址可以提前算出，但硬件预取器难以识别，例如分块哈希探测、压缩索引间接访问或图遍历中的下一层邻接块。

实验要用对照证明预取价值。只改预取距离，其他不变；输入覆盖缓存内、内存级、随机和热点混合；记录耗时和 `cache miss` 变化。若结果不稳定，就保留无预取版本。教材要让读者明白：预取是针对特定地址序列的手段，不是性能代码的装饰。

六、并发写布局要避免同线争用 多线程写输出时，布局要考虑每个线程写哪些 `cache line`。线程本地 `partial` 数组若相邻存放，可能 `false sharing`；每个线程写不同列时，也可能在结构体数组中相互挤在同一 `line`。解决方法包括按线程分块、对热写字段 `padding`、把只读字段和写字段分离、最后再压缩布局。

这会带来一个常见取舍：运行时布局和持久化布局可能不同。运行时为了并发写和 `cache` 局部性使用分块列式；写出文件时为了压缩和读取方便使用另一种格式。不要强迫一个布局同时服务所有阶段。系统中常见的 `materialize`、`transpose`、`encode`，本质上都是在阶段边界重新组织数据。

七、迁移旧代码的步骤 旧代码如果到处传 `LogRecord&`，不要一次性全改。第一步写访问矩阵，找出热路径真正使用的字段。第二步加一个只读视图接口，让热函数接受 `span` 或 `batch view`。第三步保留旧对象作为 `reference`，批量生成新布局并比对输出。第四步再拆冷字段和调整所有权。每一步都要有 `benchmark` 和正确性测试。

迁移中最危险的是半新半旧。某些函数读新列式字段，某些函数偷偷修改旧对象，状态就会分裂。项目应明确一个阶段的权威数据源：解析前原始字节权威，解析后 `HotBatch` 权威，错误诊断通过 `record id` 查冷存储。权威源不清，布局优化会破坏语义。

八、实验交付：MemoryShape 本章给项目留下 `MemoryShape` 报告。它记录对象大小、字段冷热、数组长度、`stride`、页数、分配次数、峰值内存、`batch` 生命周期和并发写模式。布局优化提交必须更新这份报告。这样后续缓存、SIMD、线程和 I/O 章节都能复用同一份数据描述。

验收实验比较三种版本：完整对象数组、热冷分离的 `SoA`、按块 `AoSoA`。输入包括短日志、长文本字段、错误密集和 `key` 倾斜。报告不仅写速度，还写内存峰值、错误诊断是否仍可追踪、下游是否需要额外转换。数据布局的质量标准，是让常见访问快，同时让生命周期和诊断仍然清楚。

6.9 案例深化：容器选择要同时看复杂度和地址形状

C++ 容器常被按接口和复杂度学习：vector 随机访问快，list 插入删除快，unordered map 平均查找快。系统性能还要问地址形状、分配次数、元素大小、迭代频率和并发模式。一个算法复杂度看似更优的容器，可能因为随机访问和分配器成本变慢；一个看似需要移动元素的 vector，可能因为连续扫描和 cache 友好更快。

一、vector 的优势来自连续和简单所有权 `std::vector` 的元素连续，适合批量扫描、SIMD、预取和一次性写出。缺点是扩容会移动元素，插入中间成本高，指针和引用可能失效。若数据按批次生成、批次内只追加、之后只读扫描，vector 非常适合。日志 HotBatch、partition offset、block metadata 都可以用 vector 承载。

使用 vector 时要控制容量。频繁扩容会分配和拷贝；过度 reserve 会浪费内存。项目可以根据输入 split 估算记录数，预留合理容量；若估算错误，报告记录扩容次数。这样容器选择就和测量连接起来，而不是只凭经验。

二、list 的插入优势常被遍历成本抵消 `std::list` 节点分散，每个节点有指针，遍历需要追踪随机地址。它的稳定迭代器和 $O(1)$ splice 在某些场景有价值，但在热路径扫描中通常很差。很多初学者因为“中间插入删除快”选择 list，却忽略程序真正高频的是遍历和查找。

如果任务队列需要频繁从中间删除，可能可以用 deque、intrusive list 或索引数组加 freelist；如果只是在末尾追加并顺序处理，vector 或 deque 更好。容器选择要看操作频率，不是看某个操作的复杂度。写访问矩阵时，也应写容器操作矩阵：push、pop、erase、iterate、lookup 各占多少。

三、unordered map 的平均 $O(1)$ 隐藏了很多常数 哈希表查找涉及 hash 计算、bucket 访问、冲突处理、key 比较和可能的节点指针追踪。节点式 unordered map 对 cache 不友好；开放寻址哈希表通常局部性更好，但删除和负载因子处理不同。key 是整数、短字符串还是长字符串，差别也很大。平均 $O(1)$ 只说明渐进，不说明内存行为。

事件统计如果 key 范围小，用数组桶比哈希表好；key 范围大但可以字典编码，先编码再数组统计可能更快；key 是任意字符串时，局部哈希表加最后合并更稳。教材要让读者看到数据分布如何改变容器选择。

四、deque 和环形缓冲适合队列语义 `std::deque` 分段存储，支持两端高效 push pop，适合普通任务队列；固定容量环形缓冲更适合有界队列和 SPSC。Deque 的分段会让整体扫描不如 vector，但队列通常不做全量扫描。环形缓冲容量固定，能表达背压，但要处理空满和元素生命周期。

选择队列容器时，先问是否需要有限，是否需要阻塞等待，是否多生产者多消费者，是否需要关闭，是否需要稳定引用。容器只是状态存储，完整队列还需要同步合同。第 10 章会从不变量继续展开。

五、pmr 和 arena 要写清释放边界 C++ 的 polymorphic allocator 和 arena 能把许多小分配集中管理。它适合 batch 生命周期一致的对象，例如解析错误列表、临时 key 字符串、block metadata。它不适合生命周期交错复杂、对象可能被异步长期持有的场景。使用 arena 前要先说明释放边界：一个 batch 完成后释放，一个 job 完成后释放，还是进程结束释放。

错误的 arena 使用会造成悬垂 view 或内存峰值过高。若下游还持有 `std::string_view` 指向 arena，arena 不能释放；若 job 长时间运行而 arena 只增不减，内存会持续增长。布局优化必须带生命周期图。

六、容器实验 本章补充一个容器实验：同一组事件 key，用 vector 数组桶、节点式 unordered map、排序 vector 后合并、局部 map 加全局合并四种方式统计。输入覆盖小 key 范围、大 key 范围、热点 key、随机字符串 key。报告记录耗时、分配次数、内存峰值、cache miss 替代指标和代码复杂度。

这个实验会让读者理解：没有“最好的容器”，只有适合当前访问模式和生命周期的容器。它也为算法书和面试刷题知识补上工程层：复杂度是第一步，地址形状和所有权是系统中的第二步。

6.10 补强案例：压缩、编码和计算成本

数据布局还包括压缩和编码。压缩能减少内存和 I/O 字节，却增加 CPU；编码能把复杂 key 变成整数，却增加字典维护；位图能减少空间，却增加位操作和可能的分支。选择这些技术时，要把数据移动和计算成本放在同一张账本里。

一、字节减少不一定更快 如果程序受内存带宽或 I/O 限制，压缩可能更快；如果程序受 CPU 限制，压缩可能更慢。轻量编码如 delta、varint、dictionary、bit packing 各有适用范围。时间戳单调递增适合 delta，低基数字段适合 dictionary，布尔标志适合 bitset。随机高熵字符串不适合简单压缩。

二、编码会改变错误处理 字典编码失败时，系统要决定新增字典项、报告未知 key，还是使用 fallback。若字典跨 batch 共享，还要处理版本和并发更新。若分布式 worker 各自编码，最终 reduce 不能直接合并整数 id，必须共享字典或在输出前反查。编码不是纯性能技巧，它改变数据合同。

三、向量化和编码的关系 列式整数编码常帮助 SIMD，因为字段变窄、连续、类型统一。变长编码则可能阻碍 SIMD，因为每个元素边界不同。工程中常在存储格式和执行格式之间转换：磁盘上压缩，执行前解码成适合 SIMD 的块，处理后再编码写出。这是材料化点的一种。

四、实验 对事件类型使用原始字符串、字典编码整数、排序字典编码、位宽压缩四种表示。比较解析成本、聚合成本、内存峰值、输出大小和错误处理复杂度。报告要写何时选择哪种表示，而不是单纯追求最小字节数。

6.11 补充案例：热冷分离后的调试能力

热冷分离常被担心会降低调试能力，因为错误信息不再和热字段放在同一个对象里。这个问题必须正面处理。好的热冷分离不是丢掉调试信息，而是用稳定身份把热字段和冷诊断连接起来。

一、record id 是连接点 HotBatch 中每行保存 record id。ColdStore 用 record id 查原始文本、文件偏移、错误上下文。统计阶段只搬热字段；出错时通过 id 回查冷数据。这样既保持热路径紧凑，又保留诊断能力。

二、冷数据可以分层保存 不是所有冷数据都要一直保留。原始输入文件可作为最终冷源；批次内可以保存 offset 和长度；错误密集时再保存摘要；调试模式下保存更多上下文。生产模式和调试模式可以不同，但合同要一致：给定 record id，系统能在承诺范围内解释错误。

三、隐私和安全 冷数据可能包含用户文本或敏感字段。热冷分离还能帮助控制敏感数据流：热路径只处理编码后的 key 或 bucket，冷路径受权限控制。系统教材虽不深入安全，但要提醒读者数据布局也影响隐私边界。

四、实验 比较完整对象、热冷分离但无冷索引、热冷分离加 record id 索引。触发错误后，看能否定位原始记录。性能和可调试性都要验收。这样读者不会把布局优化理解成牺牲工程可维护性。

6.12 从访问模式推导数据结构形态

数据布局章节最容易退化成容器选择建议：数组快、链表慢、哈希表平均常数、预取可以加速。这样的说法既粗糙，也容易误导。工程中先出现的是访问模式：读多还是写多，顺序扫描还是随机查找，热字段是否集中，生命周期是否一致，是否需要稳定引用，是否要跨线程移动。数据结构只是这些约束的结果。把访问模式问清楚，布局选择才不会变成经验口号。

一、AoS 和 SoA 的选择来自字段共现 结构数组适合对象整体被一起使用的场景，列式数组适合只扫描少数字段的场景。比如日志记录包含时间戳、级别、模块、消息、线程号和解析状态。如果热循环只按级别和时间过滤，把所有字段绑在一个大结构体里，会把冷字段一起搬进缓存。改成列式布局后，过滤阶段只读少数列，带宽下降。可是后续若要输出完整记录，又要按索引回到原始对象，接口会复杂。教材要让读者看到迁移的全过程：先发现热字段，再拆分存储，再保留稳定 ID，最后处理回填和调试成本。

二、生命周期一致时才适合 arena arena 分配器的优势是批量分配、批量释放、减少碎片和锁开销。它的前提是对象生命周期接近一致。若一个请求内产生大量临时节点，请求结束统一释放，arena 很合适；若对象会被缓存、跨任务传递或单独删除，arena 会制造悬空引用和内存滞留。读者应把“分配快”继续追问成“谁拥有对象，什么时候释放，是否需要单独析构，是否允许指针长期存在”。生命周期不清楚时，任何分配器都会放大错误。

三、索引不是低级替代品，而是边界工具 用整数索引代替裸指针有两个好处：可以让对象存储更紧凑，也可以把引用的有效性检查集中到容器边界。比如图结构可以把节点放在连续数组里，边保存 `std::uint32_t` 节点编号。这样遍历时地址更集中，序列化更容易，跨进程传输也更清楚。代价是每次访问需要通过容器解析索引，删除和压缩要维护映射。若项目需要稳定句柄，可以把索

引和 generation 合成 handle，避免旧索引误指新对象。这里的重点不是“索引比指针快”，而是索引让所有权和布局从对象内部移到容器层。

四、预取只帮助可预测的未来 软件预取不是万能加速按钮。它要求未来地址足够早知道，预取距离合适，且预取的数据确实会被使用。顺序数组通常由硬件预取处理得很好，软件预取收益有限；指针追逐中，下一跳地址太晚才知道，预取也很难提前；批量处理多个独立链表时，预取可能有用，因为可以在处理当前节点时预取另一条链的下一节点。错误预取会占带宽、污染缓存、增加指令。报告中必须比较无预取、不同距离预取和不同输入规模，而不是只保留最快一次。

五、布局迁移要保护接口 数据结构改布局时，最容易破坏调用者假设。原来调用者可能保存对象指针，列式化后对象不再有稳定地址；原来字段可直接修改，压缩编码后修改需要解码和回写；原来遍历顺序自然稳定，分区后顺序可能改变。成熟的迁移会先引入访问接口和 ID，再内部改变存储，最后用测试验证语义不变。布局优化是系统工程，不是把一个结构体改成几个数组那么简单。

6.13 C++ 容器选择背后的内存模型

很多性能问题表面上是“该用哪个 STL 容器”，本质上是对象如何排列、引用是否稳定、迭代是否连续、插入删除是否频繁、分配是否集中。第二册不把容器当语法知识讲，而是把它放进内存层级和数据布局里讲。读者已经知道数组、cache line、TLB 和预取，再回头看 `std::vector`、`std::deque`、`std::list`、`std::unordered_map`，就能理解它们为什么在不同访问模式下差异巨大。

一、vector 的优势来自连续和简单 `std::vector` 把元素连续存储，顺序遍历对 cache 和预取器友好，size 和 capacity 也让内存占用容易估算。它的代价是扩容可能搬迁元素，插入中间元素需要移动后续元素，保存元素地址的调用者可能在扩容后失效。若系统需要稳定 ID，可以保存索引而不是指针；若系统知道元素数量，可以提前 reserve；若元素很大，可以把热字段拆出，vector 只保存紧凑结构。vector 不是因为名字高级而快，而是因为它让地址序列简单。

二、deque 解决两端增长但牺牲连续性 `std::deque` 通常由多个固定块组成，两端 push 和 pop 成本稳定，元素引用在某些操作下更稳定，但整体不如 vector 连续。它适合任务队列、滑动窗口和 work stealing 的基础结构，却不适合需要大规模线性扫描的数值数组。读者要避免把 deque 当成“更通用的 vector”。通用通常意味着额外间接层，额外间接层会改变缓存行为。

三、list 的问题不是复杂度，而是每个节点太贵 `std::list` 插入删除迭代器位置是常数复杂度，但每个节点独立分配，包含前后指针，遍历时地址跳跃，TLB 和 cache 都不友好。若业务真的需要频繁在中间 splice 且元素很大，list 有价值；若只是为了避免 vector 移动，常常可以用索引、稳定池、gap buffer 或分块 vector 替代。算法复杂度表没有把 cache miss 和分配器成本写进去，所以系统教材必须补上这一层。

四、unordered map 的常数不是一个数 `std::unordered_map` 平均查询是常数，但常数包含哈希计算、bucket 访问、节点访问、key 比较和可能的分配。短整数 key 和长字符串 key 完全不同；开放寻址表和节点式哈希表也完全不同。若热路径需要大量查找，可以考虑预哈希、字符串驻留、平坦哈希表、排序数组二分或分区索引。选择前先问：key 多大，查询多频繁，插入是否在线，是否需要稳定引用，是否能批量构建。这样容器选择才和数据布局同一条线。

五、容器接口要隐藏可替换布局 如果业务代码到处保存容器内部指针，后续布局优化会非常困难。更稳的做法是暴露 ID、span、迭代范围或只读视图，让底层可以从 vector 换成列式数组、从节点哈希换成平坦哈希、从裸指针换成索引。接口不是性能的敌人，糟糕接口才是。好的接口把语义稳定下来，把布局留给实现调整，这正是本章和全书项目的连接点。

6.14 贯穿案例：LogRecord 拆开以后为什么不能丢掉证据

数据布局优化最容易让人只看到速度。假设原始系统有一个 LogRecord：原文、字段边界、用户、事件、数值、错误状态、调试标签都在里面。聚合阶段只需要事件 id 和数值，却每次把整条记录相关的 cache line 搬进来。我们把它改成 SoA：event_id、value、record_id 三列连续存储，原文和错误上下文放进冷表。聚合速度上去了，但第一次错误回溯就失败：报告只能告诉用户某个 event id 异常，却找不到原始行、文件 offset 和解析状态。

这个事故说明布局优化不能只问“热路径少读了多少字节”，还要问“系统还能不能解释自己的

结果”。冷热分离的正确形态不是丢弃冷数据，而是让冷数据不进入热路径，同时保留稳定身份。于是 `RecordId` 出现了：热路径用它关联 `partial`，错误路径用它回到原文，`manifest` 用它说明输出来自哪批输入，恢复路径用它判断重复 `attempt` 是否处理过同一记录。身份字段是布局优化和系统可复盘之间的桥。

一、先画阶段访问矩阵 这个案例的第一步不是改结构体，而是列阶段访问矩阵。解析阶段需要原始字节、字段边界和错误状态；过滤阶段只看时间和级别；聚合阶段只看 `key id` 和 `value`；错误报告阶段要回到原文；写出阶段只看 `partial`、`checksum` 和 `record id` 范围。矩阵一出来，热字段和冷字段自然分开。没有矩阵，拆分容易凭感觉进行，最后不是热路径仍然带着冷字段，就是冷路径失去证据。

二、再定义身份而不是保存指针 如果热数组里保存指向原始对象的指针，SoA 很快又被对象生命周期拖回去。更稳的做法是保存固定宽度 `record_id`，由批次索引表把它映射到文件 `id`、`offset`、`length` 和冷表位置。索引比指针多一步查询，却带来三个好处：可以序列化，可以跨进程传递，可以在 `arena` 移动后保持稳定。代价是索引表必须版本化，删除和压缩要维护映射。这个代价比悬垂指针更可控。

三、生命周期要跟异步边界对齐 布局拆分后，很多数据变成 `view`。聚合线程可能持有 `std::span<const std::uint32_t>`，写出线程可能异步持有 `partial` buffer，错误报告可能稍后读取冷表。谁拥有内存，谁只能观察，什么时候释放，必须写进接口。若 `BatchArena` 在聚合结束就释放，而异步写出还持有 `span`，性能优化会变成 `use after free`。布局章节必须提前训练这种边界，因为后面的 I/O 和分布式章节会把生命周期拉得更长。

四、容器替换要经过接口层 原来调用者也许直接拿 `LogRecord*` 修改字段。SoA 后不再有一个完整对象地址。如果接口没有隔离，改布局会牵动全项目。迁移步骤应是：先引入只读视图和写入 `builder`；调用者通过 `RecordView` 读字段，通过 `RecordBuilder` 追加记录；内部仍用 `AoS`；测试稳定后再把内部换成 SoA；最后删除直接访问旧对象的路径。接口层不是官僚抽象，而是让布局可演进。

五、性能报告要同时写调试能力 布局改造报告除了吞吐、`cache miss`、分配次数，还要写错误回溯成功率、`record id` 覆盖率、冷表大小、`arena` 生命周期和调试模式开销。若热路径快了百分之四十，但错误报告丢失原文，这不是高质量优化。系统教材要把可调试性当成一等指标，因为后续分布式故障中，缺少证据会让所有性能收益失去意义。

六、从本章连接到分布式 `manifest` `RecordId` 和 `manifest` 是同一思想在不同尺度上的表现。`RecordId` 说明某个 `partial` 来自哪条输入记录；`manifest` 说明哪个 `block` 被最终接受。两者都把数据和提交证据分开。热路径可以紧凑高效，证据路径保持可回溯。理解这一点后，读者看到第三册模型权重、`tensor block` 或 `KV cache` 分片时，也会知道不能只搬字节，还要保留版本、身份和可见性。

6.15 案例深化：从 AoS 到 SoA 的迁移纪律

数据布局优化最容易伤到可维护性。一个 `LogRecord` 对象把字段都放在一起，调试舒服，但热路径搬运太多；改成多列数组以后，扫描快了，错误报告却找不到原文，生命周期也更难管。高质量迁移不是把 `AoS` 全部推翻，而是把访问模式、身份和生命周期一起改。

一、先列阶段访问表 解析阶段需要原始字节、字段边界和错误状态；聚合阶段需要 `key id`、`value` 和 `record id`；报告阶段需要错误位置和原文；写出阶段需要 `partial` 和 `checksum`。把阶段访问表写出来，就能看出哪些字段是热字段，哪些字段是冷字段，哪些字段只是诊断。没有这张表，布局调整容易变成凭感觉拆结构体。

二、热字段连续，冷字段可回溯 SoA 版本把 `key id`、`value`、`record id` 放进连续数组，聚合阶段顺序扫描。原始文本和错误详情留在 `arena` 或冷表中，通过 `record id` 回溯。这样热路径不搬运原文，诊断又不丢。若某个优化把 `record id` 删除，短期更快，长期会破坏错误报告和故障恢复。身份字段是布局优化的安全绳。

三、生命周期比指针更重要 `std::span` 只是视图，不拥有内存。它指向的 `arena` 必须活到所有使用者结束。若批次结束就释放 `arena`，而异步写出或错误报告还持有 `span`，就会悬垂。接口应明确 `owner` 和 `view`：`BatchArena` 拥有字节，`BatchView` 只观察热字段，错误报告在 `arena` 释放前完成或复制必要信息。布局章节必须把生命周期和性能放在一起讲。

四、预取只能优化已知路径 预取适合规律访问，不适合修复混乱布局。若程序随机追节点，预取距离难选，提前加载也可能污染 cache；若访问已经连续，硬件预取可能足够。手写预取前，先确认瓶颈来自可预测的未来访问。报告要比较无预取、布局重排、预取三种版本，避免把布局收益误归因给预取。

五、迁移要保留旧版本作 oracle AoS reference 即使慢，也应保留到迁移稳定。每次布局变化都和 reference 比较字段数、错误数、checksum 和错误回溯。等 SoA 版本稳定后，reference 可以只在测试中运行。没有 oracle 的布局迁移，很容易把性能提升建立在丢字段、错生命周期或少报错误上。

6.16 案例复盘：LogRecord 拆分以后还能调试吗

一个 LogRecord 装下原始文本、字段、错误信息和临时状态，调试方便，却让聚合热路径搬运大量冷字段。改成列式布局后，扫描快了，但如果丢掉 record id，错误报告就找不到原文。数据布局优化的纪律，是热字段连续、冷字段可回溯、生命周期写清楚。

访问阶段决定布局 解析阶段需要原始字节和字段边界；聚合阶段只读 key、value 和 record id；错误报告阶段回到原文；写出阶段只处理 partial。阶段访问表写出来后，热字段和冷字段自然分开。Span 只是视图，arena 才是拥有者，批次结束前不能让异步使用者悬垂。

实验判据 AoS reference 保留到 SoA 稳定。每次迁移比较字段数、错误数、checksum、错误回溯成功率、分配次数、cache miss 和内存峰值。预取只在访问路径稳定后再试，不用它掩盖错误布局。

6.17 本章习题

习题与作业

习题一：为一个日志统计系统写访问矩阵。字段至少包括时间戳、用户 ID、事件类型、原始文本、解析错误、调试标签。热路径至少包括解析、按用户聚合、错误输出和调试查询。根据矩阵给出布局方案。

习题二：把一个指针链表设计改成索引数组设计。说明索引宽度、generation、删除策略、遍历局部性和序列化影响。不要只写“索引更快”，要写清生命周期如何保证。

习题三：设计一个冷热分离迁移计划。要求保留旧接口、建立 reference 测试、说明双写或缓存一致性策略，并给出何时删除旧布局的条件。

习题四：实现或设计 worker stats 的紧凑版和对齐版。报告每个版本的结构体大小、对齐、cache line 分布、写入线程和汇总成本。

第 7 章

SIMD、指令级并行与向量化

先看一个向量化失败的例子：把 ASCII 字段解析改成批量处理以后，规则输入变快了，错误输入却返回了错误位置。标量版本逐字节前进，遇到第一个非法字符就能给出精确 offset；向量版本一次处理多个 lane，只记录了“这一块有错误”，后续又继续提交部分结果。速度上去了，语义丢了。这个事故说明，SIMD 不是把循环换成更宽的指令，而是先证明多个元素可以在同一个语义合同下并行处理。

本章从标量 reference 出发讲 SIMD 和指令级并行。SIMD 让一条指令处理多个数据 lane，指令级并行让多个互不依赖的操作同时在不同执行资源上推进。二者都要求读者先回答：迭代之间是否独立，输出位置是否确定，尾部和错误如何处理，整数溢出和浮点误差是否允许重排。只有这些问题讲清，向量化才是优化，不是用更快的代码制造更隐蔽的错误。

7.1 从标量语义到向量执行

向量化不是把代码表面改成更复杂的形式，而是确认多个元素之间没有依赖，可以被同一条向量指令处理。数组逐元素加法容易向量化，因为每个元素独立；前缀和不容易直接向量化，因为后一个结果依赖前一个结果。

编译器自动向量化需要知道别名关系、对齐、循环边界和数据依赖。使用 `std::span` 表达连续区间有助于接口清晰，但编译器仍可能因为别名保守而放弃向量化。查看优化报告和汇编，比猜测更可靠。

判断循环能否向量化，要先写出每次迭代之间的关系。如果第 *i* 次迭代只读写第 *i* 个位置，通常容易向量化；如果第 *i* 次迭代读取第 *i-1* 次写出的结果，就存在循环携带依赖；如果每次迭代根据数据写入不同位置，就可能有冲突写；如果循环内部调用不可内联函数，编译器可能无法证明副作用边界。向量化首先是语义问题，其次才是指令问题。

别名和 restrict 思维 两个指针可能指向同一片内存时，编译器必须保守。例如 `a[i]=b[i]+c[i]` 中，如果 a、b、c 可能重叠，编译器要保证重叠情况下结果仍正确，这可能阻碍向量化。C++ 没有标准 `restrict` 关键字，但接口设计可以减少别名不确定性，例如使用不重叠容器、清晰文档和局部拷贝。

手写 SIMD 前，应该先让标量代码容易被编译器理解。清晰的循环边界、连续数据、少分支、少别名，是自动向量化的基本条件。

7.2 依赖、尾部和数值语义

即使不用 SIMD，多个独立累加器也能提高吞吐。单累加器形成长依赖链，每次加法依赖上一次结果；多个累加器把依赖链拆开，让乱序执行有更多可调度操作。SIMD 内核中也常见多个向量累加器，目的相同。

这就是并行归约在单线程内部也有优化空间的原因。先分成多个局部和，再合并，不仅适用于多线程，也适用于单核流水线。

Listing 7.1 多个累加器拆开依赖链

```
1 std::int64_t s0 = 0;
2 std::int64_t s1 = 0;
3 for (std::size_t i = 0; i + 1 < values.size(); i += 2) {
4     s0 += values[i];
5     s1 += values[i + 1];
6 }
7 const std::int64_t sum = s0 + s1;
```

这段代码的意义不是只用两个累加器，而是展示依赖链拆分。真实内核会根据执行端口、寄存器数量和向量宽度选择更多累加器。累加器太少隐藏不了延迟，太多会增加寄存器压力。

尾部处理和对齐 向量宽度通常不能整除数组长度，因此要处理尾部元素。常见方法是标量尾循环、掩码加载和补齐填充。对齐可以减少某些加载成本，但现代 CPU 对非对齐连续访问已经比较友好；真正要避免的是跨 cache line、跨页和随机访问。

尾部处理是性能代码里常见的正确性漏洞。主循环只处理完整向量宽度，剩余元素必须保证不丢、不重复、不越界。手写 intrinsics 时，尾部可以用标量循环、mask load/store 或提前 padding。选择哪种方式要看 ISA 支持、数据类型、越界读是否合法、输出是否允许覆盖 padding。教材示例宁愿保守清晰，也不能为了展示技巧隐藏边界条件。

常见误区

不要为了让 SIMD 主循环漂亮而忽略尾部。真实库里的很多错误，不发生在主循环，而发生在长度为 0、1、向量宽度减 1、刚好跨页或输入输出重叠的边界。

SIMD 与线程的关系 SIMD 和多线程不是替代关系。SIMD 提高单核吞吐，多线程使用多个核心。高性能程序通常先让单线程内核足够高效，再扩展到多线程。否则多线程只会复制低效内核，并更快撞上内存带宽。

数值语义 向量化可能改变浮点运算顺序，从而改变舍入误差。整数加法在不溢出的数学模型下更容易重排，浮点加法不严格满足结合律。编译器的 fast-math 选项会放宽数值语义，可能提高性能，也可能改变边界行为。工程中必须明确是否允许这种变化。

7.3 从自动向量化到手写 SIMD

优化顺序通常是先写清晰标量代码，再查看编译器是否自动向量化，再决定是否手写 intrinsics。自动向量化失败时，先看原因：别名不明确、循环边界复杂、函数调用副作用、数据依赖、类型转换、未对齐访问还是成本模型认为不值得。只有当标量语义已经清楚、数据布局稳定、热点足够重要、自动向量化不足时，手写 SIMD 才值得。

手写 SIMD 的维护成本很高。它引入 ISA 分发、不同宽度实现、尾部处理、测试矩阵和可移植性问题。生产库通常会把通用标量 reference、自动向量化版本和手写 ISA 版本分开，并用同一组 correctness tests 验证。第二册讲 SIMD 的目标不是让读者到处写 intrinsics，而是让读者知道什么时候应该追求向量吞吐，什么时候瓶颈根本不在这里。

向量化和内存带宽 SIMD 提高的是每条指令处理的数据量，但它不能凭空增加内存带宽。对于流式数组求和，如果单线程已经被内存带宽限制，更宽的 SIMD 可能只减少指令数，不显著提高总时间。对于算术强度高、数据能复用的内核，SIMD 才可能接近计算峰值。判断前必须回到 Roofline：这个内核到底缺算力，还是缺数据。

向量化前先定义语义 SIMD 教材最常见的错误，是上来就讲寄存器宽度和指令名，却没有先定义循环语义。向量化的前提不是“循环长得像数组”，而是多个迭代之间可以被重排、合并或并行执行。一个循环能否向量化，首先由数据依赖、异常语义、别名关系、写入冲突和数值模型决定。

以数组加法为例，每个输出位置只依赖同一位置的两个输入，且不同位置互不影响。只要输入输出不发生破坏性重叠，这个循环天然适合向量化。前缀和则不同，位置 *i* 的输出依赖位置 *i-1* 的结果，不能直接把所有元素当成独立 lane。直方图更复杂，每个元素会写入由数据决定的桶，不同 lane 可能写同一个桶，形成冲突写。图遍历则同时有随机读、分支和不均匀工作量。把这些循环都叫“处理数组”，对性能分析没有帮助。

因此本章的学习顺序是：先写出串行语义，再判断迭代独立性，再考虑编译器自动向量化，再考虑手写 SIMD。若语义不允许重排，强行使用 SIMD 只会写出错误程序。若语义允许重排但编译器没能向量化，应先找出失败原因，而不是直接责怪编译器。

baseline：标量数组变换 看一个最简单的变换：把两个数组相加写到输出。这个例子很朴素，但它包含自动向量化需要的几乎所有条件：连续内存、线性索引、无循环携带依赖、固定类型和简单表达式。

Listing 7.2 baseline：标量数组加法


```

1 void add_arrays_scalar(std::span<const float> left,
2   std::span<const float> right,
3   std::span<float> output) {
4   const std::size_t count =
5     std::min(left.size(), std::min(right.size(), output.size()));
6   for (std::size_t i = 0; i < count; ++i) {
7     output[i] = left[i] + right[i];
8   }
9 }

```

这段代码适合作为 reference，但它没有明确说明三个 span 是否重叠。若 `output` 和 `left` 指向同一数组，语义仍然可能正确，因为每个位置先读后写同一位置；若 `output` 是 `left` 向后偏移一个元素的重叠视图，写 `output[i]` 可能影响后续 `left[i+1]` 的读取。编译器如果不能排除这种情况，就可能生成运行时别名检查，或者放弃某些向量化策略。

工程接口要尽量表达不重叠语义。可以在文档中声明三个区间不得破坏性重叠，可以在 debug 版本检查地址范围，也可以把输入和输出放在不同所有者对象中。C++ 标准没有可移植的 `restrict` 关键字，因此高质量 API 设计比某个编译器扩展更重要。

自动向量化报告 自动向量化不应该靠猜。Clang 和 GCC 都能输出优化报告，告诉你某个循环是否向量化、为什么失败、使用了多宽向量、是否做了运行时别名检查。不同版本选项略有差异，报告中应记录编译器版本和命令。一个报告可以包含如下问题：循环是否被 vectorized；vector width 是多少；interleave count 是多少；失败原因是依赖、调用、副作用、成本模型还是不支持的数据类型。

如果报告说循环未向量化，下一步不是立刻手写 intrinsics，而是修改源码让语义更清楚。把复杂函数调用移出循环，把热字段变成连续数组，把输出位置改成唯一写入，把分支拆成内部区域和边界区域，把可能别名的输入输出拆开。很多时候，自动向量化失败不是编译器弱，而是源码没有给足证明条件。

优化报告还要和汇编对照。报告说向量化，不代表生成了你预期的高效代码；报告说未向量化，也可能编译器通过其他方式优化。查看汇编时要关注是否出现向量 load/store、向量 add/mul/fma、循环展开、尾部处理和运行时别名检查。读汇编的目标不是背指令名，而是验证执行形态。

循环携带依赖和重写 循环携带依赖是向量化的主要障碍。最典型的是单累加器求和：每次迭代都依赖上一次 sum。编译器可以使用向量归约，把多个 lane 局部累加后水平合并，但这要求它确认重排语义允许。整数在有符号溢出语义、浮点舍入语义和 fast-math 选项下会有不同结果。

很多循环可以通过改写暴露独立性。例如计算差分数组：

Listing 7.3 可向量化的相邻差分

```

1 void adjacent_difference_kernel(std::span<const std::int32_t> input,
2   std::span<std::int32_t> output) {
3   if (input.size() < 2 || output.empty()) {
4     return;
5   }
6   const std::size_t count = std::min(input.size() - 1, output.size());
7   for (std::size_t i = 0; i < count; ++i) {
8     output[i] = input[i + 1] - input[i];
9   }
10 }

```

这个循环读相邻元素，但每个输出位置独立，通常可以向量化。相反，前缀和每个输出依赖前一个输出，不能直接按同样方式处理。判断时不要只看“是否读相邻元素”，要看“是否读本轮之前写出的结果”。

尾部处理的三种策略 尾部处理有三种常见策略。第一是标量尾循环：主循环处理完整向量宽

度，剩余元素用普通循环。它清晰、可移植、适合教学。第二是 mask load/store：使用掩码只处理有效 lane，适合 AVX-512、SVE 等支持较好的 ISA。第三是 padding：让数组末尾多出安全空间，主循环可以越过逻辑长度但不越过分分配边界。这适合内部数据结构，不适合任意用户传入 span。

尾部策略还影响异常和内存安全。对输入做越界读即使结果被掩掉，在 C++ 语义上也可能是未定义行为；跨页越界读还可能触发 page fault。生产库不能随便假设尾部后面有安全字节，除非容器明确保证 padding。教材示例应优先使用标量尾循环，等语义清楚后再讨论 mask。

Listing 7.4 标量尾循环的结构

```
1 constexpr std::size_t SIMD_WIDTH = 8;
2
3 void add_arrays_chunked(std::span<const float> left,
4   std::span<const float> right,
5   std::span<float> output) {
6   const std::size_t count =
7     std::min(left.size(), std::min(right.size(), output.size()));
8   std::size_t i = 0;
9   const std::size_t vector_limit = count / SIMD_WIDTH * SIMD_WIDTH;
10  for (; i < vector_limit; i += SIMD_WIDTH) {
11    for (std::size_t lane = 0; lane < SIMD_WIDTH; ++lane) {
12      output[i + lane] = left[i + lane] + right[i + lane];
13    }
14  }
15  for (; i < count; ++i) {
16    output[i] = left[i] + right[i];
17  }
18 }
```

这段代码仍是标量 C++，不是 intrinsics，但它展示了向量主循环和尾循环的结构。编译器可能把内部 lane 循环向量化，也可能完全展开。教学中先写这种结构，有助于读者理解手写 SIMD 版本会怎样组织。

多版本派发和可移植性 手写 SIMD 不能只为一台机器写。x86 上有 SSE、AVX、AVX2、AVX-512；ARM 上有 NEON 和 SVE；不同机器支持的指令集不同。生产库通常需要多版本派发：保留标量 reference，再提供一个或多个 ISA 优化版本，运行时根据 CPU 能力选择。编译期只为当前机器生成一种版本，部署到其他机器可能无法运行。

多版本派发也有成本。每个版本都要测试，尾部处理要一致，数值结果要在允许误差内，构建系统要管理不同编译选项，调试时要知道实际跑的是哪个版本。对于不够热的循环，手写多版本不值得。自动向量化和标准库算法可能已经足够。

Compute Systems Engine 的第一阶段不需要手写所有 ISA 版本，但应保留接口层次：reference 标量版本、自动向量化友好版本、未来可能的 ISA 版本。这样项目不会因为一个手写内核绑死在某台 CPU 上。

Gather、scatter 和压缩写 SIMD 最喜欢连续 load/store。gather 从多个不连续地址加载，scatter 向多个不连续地址写入，通常比连续访问贵得多。它们能表达图遍历、哈希表查询和稀疏计算的一部分并行性，但不能把随机访问变成顺序访问。很多时候，重排数据、排序索引、分块处理比直接使用 gather 更有效。

压缩写常见于过滤：只把满足条件的元素写到输出。朴素分支版本会对每个元素判断并 push；并行版本需要先统计每个块保留多少，再扫描偏移，再写输出。某些 SIMD ISA 支持 compress store，但依然需要处理输出位置和尾部。过滤类问题把 SIMD、scan 和分区连接起来，因此第 13 章会专门展开。

分支、掩码和数据分布 向量化常把分支变成掩码。对于简单条件选择，例如 `output[i] = value > threshold ? a : b`，掩码很自然。对于复杂分支，如果两个分支都需要大量计算，把它们都执行再用掩码选择可能更慢。优化要看数据分布：如果分支高度可预测，标量分支可能很快；如

果分支随机且分支体短，掩码更可能有收益。

高性能内核常把数据预先分组，让同一批数据走同一条路径。日志解析可以按记录类型分桶，图算法可以按顶点度数分层，数据库执行器可以把 selection vector 传给后续算子。这样做的目的不是为了形式上“向量化”，而是让控制流和数据流更规则。

数值稳定性和重排 浮点 SIMD 需要更严肃的数值语义。浮点加法不满足数学结合律，改变合并顺序会改变舍入。向量化归约通常会改变加法顺序，因此结果可能与串行逐项求和不同。对于科学计算，这可能需要误差界、Kahan 求和、pairwise sum 或固定合并树；对于图形和机器学习某些场景，误差容忍度可能更高。教材不能只说“结果有一点不同”，要让读者决定业务是否允许。

整数也不是完全无忧。C++ 有符号整数溢出是未定义行为，编译器可以基于“不会溢出”做优化。若业务需要模 32 位或模 64 位行为，应使用无符号类型或明确的溢出处理。系统代码中的位宽类型和溢出语义要写清楚，否则优化和正确性会互相踩踏。

SIMD 章的回归阅读要先审语义合法性。数组变换、阈值过滤和 ASCII 字节分类看起来都能批量处理，但它们的输出语义不同：数组变换是一对一写回，过滤需要保存命中位置，字节分类需要报告第一个错误或所有错误。若输出位置和错误语义没有定义，直接写 intrinsics 只会把 bug 写得更隐蔽。

每个向量候选循环都要过六个检查：迭代之间是否独立，输入输出是否非法重叠，尾部如何处理，mask 如何转成语义结果，类型转换是否会溢出或改变舍入，平台不支持时是否有 scalar fallback。固定宽度整数类型必须写清楚，尤其是解析、量化和累加场景。窄 lane 中间结果一旦溢出，最后转成宽类型也无法修复。

本章的实验不要求读者马上手写 AVX 或 NEON。更稳的顺序是 scalar reference、自动向量化友好源码、优化报告、反汇编确认、强制 backend 对比、最后才是平台特化。这样第三册进入 AI 算子时，读者已经知道“先证明批量语义，再选择指令”，不会把 SIMD 当作神秘加速开关。

解析类 SIMD 更要强调结构拆分。完整 CSV 或日志解析包含引号、转义、变长字段、错误恢复和行号报告，很难一次性向量化；但查找换行、分隔符、ASCII 数字范围和非法字节可以拆成规则子内核。子内核输出 bit mask、位置列表或 selection vector，后续标量状态机消费这些结构信息。这样 SIMD 只承担适合它的字节分类，业务语义仍然保持清楚。

数值类 SIMD 则要写误差合同。整数 lane 要说明是否使用饱和、回绕、符号扩展和宽累加；浮点 lane 要说明是否允许重排、FMA 和不同合并顺序；混合精度要说明 narrow 和 widen 的位置。第三册量化推理会大量使用这些概念，第二册在这里先把语义边界讲清，避免以后把“向量指令更快”误写成“结果天然等价”。

SIMD 代码的接口也要避免污染业务层。公开函数应接收 `std::span`、配置和输出对象，不应暴露 AVX、NEON 或某个寄存器类型；平台特化藏在 dispatch 层，测试可以强制选择 scalar、auto-vectorized 和 native backend。这样读者会把 SIMD 看成可替换实现，而不是把业务函数绑死在一套指令集上。

尾部处理要单独测试。长度为 0、1、向量宽度减一、正好向量宽度、向量宽度加一、很大长度、非对齐起点和跨页边界，都可能触发不同路径。许多 SIMD bug 不在主循环，而在尾部、mask 到索引的转换、符号扩展和错误位置报告。教材中的每个批量内核都应列出这些边界，而不是只给理想长度。

7.4 先证明批量语义，再选择 SIMD 形式

SIMD 的核心不是“用了向量指令”，而是把多个独立元素放进同一条指令语义里执行。若元素之间有循环携带依赖，若输出位置依赖前面元素，若错误处理要求立即停止，若浮点顺序必须完全固定，那么批量执行就需要额外证明或重写。很多向量化失败不是编译器不够聪明，而是源码没有给出可批量执行的语义边界。

标量 reference 是 SIMD 的地基。先写清一个元素怎样处理，再写清一组元素能否独立处理，最后处理尾部和错误。若一开始就写 intrinsics，读者会把正确性和性能混在一起。高质量 SIMD 章节必须保留 scalar reference、自动向量化版本和手写版本，让读者看到每一步改变了什么。

自动向量化需要什么条件

编译器自动向量化通常需要连续内存、清晰循环边界、无未证明别名、无不可重排依赖和可接受的数值语义。`std::span`、restrict 思维、输入输出分离、批量接口都能帮助编译器看清这些条

件。相反，虚调用、函数指针、复杂迭代器、可能重叠的指针、循环中早退和异常路径都会让优化更保守。

优化报告是学习自动向量化的入口。GCC 和 Clang 都能输出为什么某个循环向量化或未向量化。报告不是性能证明，但能告诉你编译器卡在哪里。若报告说存在可能别名，就重写接口表达只读输入和独占输出；若报告说循环有依赖，就回到算法语义判断能否拆分；若报告说成本模型认为不值得，就比较不同输入规模和编译选项，不要盲目强制。

自动向量化成功后也要看反汇编和测量。编译器可能生成很宽的向量，也可能只做部分展开；可能为了尾部生成复杂 mask；可能因为对齐未知生成额外路径。报告要记录向量宽度、尾部策略、是否多版本派发，以及在小输入和大输入上的收益差异。

尾部、对齐和 mask 是正确性问题

数组长度通常不是向量宽度倍数，尾部处理不能靠“应该差不多”跳过。常见策略有标量尾循环、masked load/store、padding 输入。标量尾循环简单清楚，masked 版本减少分支但更复杂，padding 需要保证额外元素不改变语义。选择哪种策略，要看输入长度分布、平台指令集和错误处理要求。

对齐也不是单向答案。现代 CPU 能处理非对齐 load/store，但跨 cache line 或页面边界可能更贵。强制对齐需要分配器、容器和 API 一起保证；若只是写一个假设，代码就有未定义行为或隐蔽性能风险。教学项目可以先用不要求对齐的版本，测量后再决定是否加入 aligned allocator，并写清 fallback。

Mask 把控制流数据化。过滤、分隔符扫描、比较阈值都可以先生成 mask，再用 popcount、compress 或位置列表推进后续阶段。这里的难点是输出位置。若每个 lane 都可能输出，必须把 mask 转成紧凑位置，常见方式是局部计数、prefix 或查表。这个问题会连接到 scan/filter 章节，说明 SIMD 和并行算法不是割裂的。

数值、溢出和类型宽度

整数 SIMD 要写清溢出语义。std::uint8_t 加法是模 256，某些 saturating 指令是饱和到最大值，二者语义不同。解析 ASCII 数字时，临时乘加可能需要更宽类型；使用窄类型能提高并行度，但不能牺牲正确范围。代码示例应使用 std::uint32_t、std::uint64_t、std::int32_t 等位宽明确类型，不用含义随平台习惯变化的模糊长整型写法。

浮点 SIMD 更要说明重排。向量化归约会改变加法顺序，结果可能有微小差异；FMA 改变舍入路径；fast-math 允许更多变换但也放松 NaN、Inf、符号零等语义。教材不能只展示速度，要写出误差合同：允许绝对误差还是相对误差，是否要求确定性，是否保存固定归约树，是否禁用某些 fast-math 选项。

混合精度也要谨慎。用 float 替代 double 可以提高吞吐和减少带宽，但误差、溢出和稳定性要由 workload 决定。第三册 AI 算子会大量讨论量化和混合精度，本册先建立原则：类型变窄是语义改变，不只是性能优化。

手写 SIMD 的工程边界

手写 intrinsics 的成本很高：平台差异、代码可读性、测试矩阵、尾部路径、多版本派发、编译选项和维护成本。它适合热路径、稳定语义和明确收益，不适合随手替换所有循环。工程上常先写 scalar reference，再尝试自动向量化，最后只对关键内核写平台特化。

多版本派发要可测试。可以根据编译目标或运行时 CPU feature 选择 SSE、AVX2、AVX-512 或 NEON 版本，但每个版本都要跑同一套 oracle。若当前设备是 ARM 手机，x86 intrinsics 只能作为阅读材料，项目实现应保留 portable scalar 或标准库实验版本。教材必须尊重读者的 Termux/Linux 环境，不能假设每个人都有同一台服务器。

本章项目交付物可以是三个内核：ASCII 分隔符扫描、数字字段解析、向量化归一化或阈值过滤。每个内核都有 scalar reference、自动向量化尝试、手写或伪代码版本、尾部测试、错误输入测试和性能报告。这样读者学到的不是某条指令，而是 SIMD 化一个真实内核的完整流程。

案例走查：ASCII 数字解析的 SIMD 前置证明

ASCII 数字解析适合讲 SIMD，但必须先证明语义。输入字段长度可能不同，可能有非法字符，可能溢出目标类型，字段可能跨 batch 边界。标量 reference 要先定义这些行为：非法字符如何报告，溢出是否拒绝，空字段是否合法。没有这个合同，向量化版本即使快，也不知道对不对。

批量版本可以先不写 intrinsics，只把多个字符的分类和累加拆成数组操作，观察哪些步骤独立。字符是否在'0'到'9'之间可以批量判断；把位置权重乘上数字需要知道字段长度；错误 mask 需要能

回到具体行和列。这个分析会自然推出 mask、位置列表、尾部处理和旁路错误缓冲。

性能实验比较标量逐字节、批量标量、自动向量化、平台特化四个层次。每层都跑同一 oracle。若自动向量化已经足够，手写 SIMD 可以留作阅读；若手写版本收益明显，报告要写平台限制和 fallback。这样 SIMD 不再是炫技，而是从语义证明走向工程选择。

跨平台 SIMD 的教学策略

读者环境可能是 x86 服务器，也可能是 ARM 手机。教材不能把某一套 intrinsics 当作唯一主线。主线应是语义和数据形状：连续字节、固定宽度 lane、mask、尾部、水平归约、fallback。具体实现可以分成 portable scalar、编译器自动向量化、平台特化三层。平台特化作为深入案例，而不是唯一答案。

在 Termux/ARM 上，可以观察 NEON 自动向量化，也可以只做 scalar batch 版本。若书中展示 AVX2 代码，要同时写出它的语义等价伪代码和 fallback。这样手机读者不会因为硬件不同就无法学习。高质量教材要区分“原理必学”和“某平台指令可选”。

多版本派发的测试也要跨平台。每个版本都注册到同一个测试表，输入包括非对齐地址、短长度、长度不是 lane 倍数、非法字符和溢出。运行时 feature detection 选择某版本时，也要能强制指定 scalar 版本用于对照。否则平台特化出错很难定位。

向量化失败后的重构路线

当编译器报告无法向量化时，不要立刻写 intrinsics。先看失败原因。若是别名，改接口；若是循环中调用未知函数，把函数内联或改成批量回调；若是输出位置不确定，引入 scan；若是早停错误处理，把错误放到旁路 mask；若是数据不连续，回到布局章。这个路线能把 SIMD 和前面章节连接起来。

重构后仍可能不值得向量化。例如内核已经内存带宽受限，SIMD 减少算术指令也无法提升；或者 gather/scatter 太多，向量 lane 大量等待；或者尾部和错误路径占比高。报告要写出“为什么保留标量版本”。能选择不优化，是成熟工程能力。

本章作业可以给一个无法向量化的循环，让读者先读优化报告，再提出三种重构方案，并说明每种方案改变了哪些语义或接口。答案不要求都写代码，但必须说明正确性边界。

综合练习：SIMD 化前的语义审查

给出一个标量字段解析循环，要求读者先做语义审查。字段长度是否固定，是否允许空字段，非法字符如何报告，溢出如何处理，输出是否要求和标量完全相同，错误行是否继续解析后续字段。只有审查完成，才能进入批量化。若这些问题答不出来，写 intrinsics 只会把错误藏进更难读的代码。

第二步让读者写批量标量版本。它一次处理一个 batch，但仍然使用普通 C++ 循环。目标是暴露批量边界：哪些数组连续，哪些分支可以变成 mask，尾部如何处理，错误如何旁路。批量标量通常已经能帮助编译器自动向量化，也让后续手写 SIMD 有 reference。

第三步才比较平台实现。若是 ARM 设备，可以观察 NEON 或只保留自动向量化；若是 x86，可以写 AVX2 版本。每个平台版本必须跑同一套测试：长度小于向量宽度、等于向量宽度、不是倍数、非法字符、溢出、非对齐地址。报告写明当前平台，不能把某平台数字当成通用结论。

验收时要求读者解释三个“不优化”的情况：输入很小，错误路径占比高，内存带宽已经主导。能说出什么时候不写 SIMD，说明已经理解 SIMD 是工程选择，不是固定炫技动作。

容易出错的边界：向量化不是语义豁免

第一个反例是尾部丢元素。向量主循环处理宽度倍数，忘记处理剩余元素，小测试刚好长度对齐时不会暴露。第二个反例是错误 mask 丢失行号。批量检测非法字符后，只知道某个 lane 错，却无法报告输入位置。第三个反例是溢出。窄 lane 中间结果溢出，最后转回宽类型也救不回来。第四个反例是浮点顺序变化，导致测试在某些输入下不稳定。

第五个反例是平台假设。AVX2 版本在开发机快，手机 ARM 上不能运行；AVX-512 可能触发降频；非对齐 load 在某平台成本低，在跨页边界仍可能出问题。跨平台项目必须有 scalar fallback 和自动测试。书中展示平台代码时，也要写清它不是唯一实现。

第六个反例是 gather/scatter 滥用。看起来用了宽向量，实际每个 lane 都在随机访问，内存系统无法提供足够并行，性能不升反降。遇到 gather/scatter 频繁的内核，先回到数据布局和 partition，而不是继续堆指令。

本章源码索引建议读者给每个 SIMD 候选循环写“合法性清单”：无循环携带依赖，输入输出不非法重叠，尾部有定义，错误有位置，类型范围足够，浮点误差可接受，fallback 存在。清单过不

了，就不要进入 intrinsics。这个习惯能避免 SIMD 代码成为不可维护黑盒。

本章核心接口：KernelVariant

SIMD 章给项目留下 **KernelVariant**。每个热内核注册 scalar、auto vectorized、platform specific 等变体，统一签名、统一 oracle、统一 feature name。运行时可以选择最佳变体，测试也可以强制运行所有可用变体。这样平台差异不会散落在业务代码中。

KernelVariant 还记录语义标签：是否允许浮点重排，是否要求对齐，是否处理尾部，是否支持错误 mask。若某变体要求输入长度为特定倍数，它就只能作为内部 fast path，并必须有 fallback。接口把 SIMD 的前置条件写成代码对象。

从 SIMD 进入计算内核组合

SIMD 章关注一个内核内部怎样批量执行，下一章关注多个内核怎样组合成数据流。一个向量化扫描可能只是 pipeline 中的 filter 前置；一个批量解析内核的输出会进入 histogram；一个 mask 会变成 scan 的输入；一个局部 partial 会变成 reduce 的输入。单内核再快，也要放回 pipeline 才能判断端到端价值。

项目在这里要避免“为 SIMD 而 SIMD”。如果某个 SIMD 内核只占端到端 3

7.5 项目落地

Compute Systems Engine 在本章应加入三个版本的数组内核。第一，reference 标量版本，语义最清楚，用于校验。第二，自动向量化友好版本，数据连续、边界清晰、少别名、少分支，并输出编译器优化报告。第三，实验性 chunked 版本，显式展示主循环和尾部处理。项目不必马上手写 AVX 或 NEON，但要把 correctness tests 和 benchmark harness 准备好。

实验报告要记录：编译器版本、优化选项、目标 CPU、是否启用 fast-math、循环是否向量化、输入规模、对齐方式、尾部长度、结果校验和性能指标。若向量化版本没有更快，要分析是内存带宽、尾部、别名检查、前端压力还是数据太小。不要只写“SIMD 没用”。

Linux 和工具入口 本章更适合读编译器报告和反汇编，而不是直接读 Linux 内核源码。但 Linux 工具仍有用：**perfstat** 可以观察 instructions、cycles、IPC、cache miss；**perfrecord** 可以确认热点是否在目标循环；**lscpu** 可以记录 CPU flags；**objdump** 或编译器汇编输出可以确认指令形态。报告中应把工具输出和源码结构对应起来。

如果要读源码，可以从成熟库开始，例如 C++ 标准库实现中的算法派发、数学库中的多版本内核、数据库或向量搜索库的 SIMD 路径。源码阅读重点不是复制 intrinsics，而是看它如何组织 reference、ISA dispatch、尾部、对齐、测试和 fallback。

7.6 掩码、解析和批处理

数据库和批处理系统常使用 selection vector 表示哪些行通过过滤。它不是立刻把数据压缩到新数组，而是保存通过过滤的行号或位图，后续算子根据 selection 处理。这样可以避免每个 filter 都材料化输出，也能让多个过滤条件组合。Selection vector 连接了 SIMD 掩码、filter、compaction 和数据库执行。

若选择率很高，位图或掩码可能更紧凑；若选择率很低，索引数组只保存命中位置更高效。若后续算子需要随机访问原数组，索引数组会引入 gather；若后续先 compaction，再连续处理，可能更适合 SIMD。选择哪种表示，要看后续管线，而不只是当前 filter。

Listing 7.5 选择向量的接口形态

```
1 struct SelectionVector {
2     std::vector<std::uint32_t> selected_rows;
3 };
4
5 SelectionVector select_positive(std::span<const std::int32_t> values) {
6     SelectionVector selection;
7     selection.selected_rows.reserve(values.size());
8     for (std::uint32_t i = 0; i < static_cast<std::uint32_t>(values.size()); ++i) ←
    ↪ {
```



```

9  if (values[i] > 0) {
10 selection.selected_rows.push_back(i);
11 }
12 }
13 return selection;
14 }

```

这个版本是标量 reference。优化版可以用 SIMD mask 找出命中 lane，再把命中位置追加到 selection。无论实现如何，语义是稳定的：selection 中的行号按输入顺序排列。这个稳定性会影响后续 join、聚合和测试。

SIMD 和解析 文本解析看起来不适合 SIMD，因为字段变长、状态复杂、错误路径多。但解析中有一些子任务很适合向量化：查找换行符、逗号、冒号、引号；判断字节是否在 ASCII 数字范围；批量寻找非法字符；扫描固定格式字段。成熟 JSON、CSV 和日志解析器常先用 SIMD 找结构字符，再用标量状态机处理语义。

这种分层很重要。不要试图把完整解析器一次性 SIMD 化。先找出规则、重复、无依赖的子内核，例如扫描一块 64 字节里的分隔符位置。输出可以是 bit mask 或 position list，后续标量代码消费这些结构位置。这样 SIMD 处理的是字节分类，业务语义仍由清晰状态机负责。

Saturating、rounding 和溢出语义 某些 SIMD 指令提供饱和加法、舍入转换、最小最大、绝对值和打包。这些语义和普通 C++ 运算不一定相同。例如图像处理常需要 8 位饱和加法，超过 255 时保持 255；普通无符号 8 位加法则按模 256 回绕。若手写 SIMD 时不明确溢出和饱和语义，结果会和 reference 不一致。

量化和 AI 推理中，这个问题更重要。虽然 AI 放在第三册，本册仍要先建立概念：向量指令不仅影响速度，也可能携带特定数值语义。任何使用特殊指令的内核，都必须有 reference 和边界测试，覆盖最大值、最小值、负数、舍入边界和溢出边界。

混合精度和类型转换 SIMD 内核经常要在不同类型之间转换，例如 `std::int8_t` 到 `std::int32_t` 累加，`float` 到 `std::int32_t` 量化，`std::uint16_t` 到 `float` 计算。类型转换可能成为瓶颈，也可能改变精度和舍入。内核设计要把 load、extend、compute、narrow、store 的步骤写清楚。

混合精度优化必须有误差或溢出说明。若把 32 位累加改成 16 位累加，可能溢出；若把 double 改成 float，可能误差扩大；若把 float 转 int，舍入模式要定义。高性能不是牺牲语义的借口。

内核融合和寄存器压力 把多个循环融合成一个循环，可以减少内存读写。例如先 map 再 filter，若分开执行，需要写中间数组；融合后可以直接处理并写输出。但融合会增加循环体复杂度、分支、寄存器占用和代码大小。寄存器压力过高时，编译器会 spill 到栈，反而变慢。前端压力也可能增加。

因此内核融合要测量。适合融合的阶段通常是简单、连续、没有复杂依赖的 map/filter。Reduce、sort、join、shuffle 这类 pipeline breaker 不容易完全融合。数据库执行器中的 vectorized batch 模型就是折中：每个 batch 内尽量局部融合，但在需要重排或聚合的地方材料化。

SIMD 与线程并行的顺序 优化时常问：先做 SIMD 还是先多线程？稳妥顺序通常是先写标量 reference，再让单线程内核数据布局和自动向量化友好，然后再多线程。原因是多线程会引入调度、NUMA、同步和带宽变量，容易掩盖单核问题。单线程内核若已经受内存带宽限制，多线程能提高总带宽直到平台上限；单线程内核若受依赖链限制，多线程也许掩盖问题但不解决单核效率。

当然，某些场景先多线程更实用，例如业务任务天然独立且单个任务不热。工程选择取决于热点和成本。教材建议读者至少保留单线程优化报告，因为它是理解多线程扩展曲线的基础。

本章新增验收 本章在 Compute Systems Engine 中的验收应包括：每个内核有 scalar reference；自动向量化报告保存到实验记录；强制标量和自动向量化版本输出一致；尾部长度覆盖从 0 到向量宽度减一；对齐和非对齐输入都测试；浮点内核写明误差策略；小输入和大输入分别 benchmark；默认路径可回退到 scalar。

这些验收比手写一段漂亮 intrinsics 更重要。没有 reference 和回退，SIMD 内核很难维护；没有尾部测试，边界 bug 很常见；没有误差策略，浮点结果争议无法解决。

案例：ASCII 数字扫描 日志解析里常见子问题是判断一段字节是否全是 ASCII 数字。标量版本逐字节检查，遇到非数字退出。SIMD 版本可以一次加载多个字节，分别比较是否大于等于字

符 0 且小于等于字符 9，再把结果形成 mask。若 mask 表示所有 lane 都满足，就整块通过；否则找到第一个失败位置。

这个案例适合讲 SIMD，因为它规则、数据连续、每个字节独立，输出可以是一个位置或布尔值。它也展示了边界：输入长度不是向量宽度倍数时要处理尾部；跨页加载要小心；错误位置需要从 mask 转成索引；若字符串很短，标量可能更快；若输入大多第一字节就失败，标量早退出可能更好。SIMD 优化要看数据分布。

案例：分隔符位置列表 CSV 或日志解析还需要找逗号、空格、制表符或换行。SIMD 可以批量比较一块字节是否等于分隔符，生成 bit mask，再把 set bit 转成位置列表。位置列表就是一种 selection vector。后续标量解析器根据位置切分字段。这种结构把“找结构”与“解释语义”分开，常见于高性能解析器。

位置列表也有容量问题。若一块中分隔符很多，输出位置多；若分隔符很少，输出少。可以先用固定小数组保存一个 block 的位置，再追加到 batch-level vector。追加时要避免每个位置都抢全局锁，通常每个 worker 有本地位置缓冲，最后合并。

案例：向量化归一化 另一个适合 SIMD 的内核是数组归一化：`output[i]=(input[i]-mean)*inv_std`。它是纯 map，输入输出连续，迭代独立。若 mean 和 inv_std 已经计算好，循环很适合自动向量化。若 mean 也要从 input 求出，则整个算法分成 reduce 求和、计算 mean、map 归一化三阶段。这里可以看到 map 和 reduce 的组合。

归一化还展示浮点语义。求 mean 的归约顺序会影响结果，map 阶段的 fused multiply-add 也可能改变舍入。若业务要求严格复现，需要固定归约顺序和编译选项；若允许误差，需要写明 tolerance。一个看似简单的 SIMD map，也可能被前置 reduce 的数值语义影响。

SIMD 性能报告模板 SIMD 实验报告可以按固定模板写。第一，内核语义和 reference。第二，数据布局和对齐。第三，循环依赖和向量化理由。第四，编译器报告或汇编证据。第五，输入规模和分布。第六，正确性和误差。第七，性能结果和瓶颈归因。第八，反例和回退策略。

例如 ASCII 扫描报告应写：输入是随机 ASCII、全数字、首字节错误、末字节错误四种；输出是第一个非数字位置；reference 是标量逐字节；SIMD 版本对尾部使用标量；小于某个长度走标量；结果完全一致。这样报告才像工程文档，而不是只贴一段 intrinsics。

批大小和 SIMD SIMD 喜欢批量数据。若每次函数只处理一条记录，向量化很难摊薄固定成本；若把几千条记录放入 batch，很多字段可以列式处理，SIMD 才有空间。Batch 大小影响 SIMD、cache 和延迟。太小，向量利用率低；太大，cache 压力和尾延迟高。第 6 章的 columnar batch 是第 7 章 SIMD 的前提。

日志解析可以先按字节块扫描分隔符，再按记录 batch 解析字段，再按列做聚合。每一层 batch 大小可能不同。字节扫描适合较大连续块，字段解析适合记录 batch，聚合适合 event type 列。不要强求所有阶段同一个 batch 大小。

SIMD 和错误处理 错误处理会影响 SIMD 内核设计。若 SIMD 扫描发现某个 lane 非法，是立即返回错误，还是记录位置继续扫描？立即返回适合验证函数；记录位置适合批处理解析，因为一个 batch 里可能有多条错误记录，但仍希望处理其他记录。选择不同，输出结构也不同。

批处理系统通常把错误记录到 side buffer，不让少数错误打断整个向量内核。这样热路径保持规则，错误路径后处理。但如果错误表示输入损坏到无法确定边界，例如长度字段非法，必须停止当前消息。错误语义决定 SIMD 控制流。

跨平台 SIMD 的教学策略 本书不把某个 ISA 作为唯一答案，是因为读者设备可能是 x86、ARM 手机或云服务器。教学策略是先讲循环依赖和数据布局，再讲自动向量化，再讲抽象的 lane、mask、gather、horizontal reduce，最后才读具体 ISA。这样读者即使不写 AVX，也能理解为什么某个循环适合或不适合向量化。

第三册 AI 推理会更深入具体指令，但第二册先建立可迁移模型。现代系统工程师不应只会背某个指令名，而要能判断数据流是否值得向量化，如何设计 reference 和 fallback。

7.7 总结、决策表和反例

SIMD 的核心是把同一种操作作用到多个 lane 上。它要求数据规则、依赖清楚、输出位置明确、数值语义可接受。它经常和数据布局、batch、selection vector、scan、partition 一起出现，而不是

孤立出现。能自动向量化就先让编译器做；需要手写时，先设计 reference、dispatch、tail、对齐和测试。SIMD 是系统优化的一层，不是替代测量和算法设计的捷径。

SIMD 决策表 判断一个内核是否值得 SIMD 化，可以问：输入是否连续，迭代是否独立，输出位置是否固定，分支是否简单，数据量是否足够大，数值重排是否允许，目标平台是否稳定，是否已有自动向量化，是否有 scalar fallback。如果多个答案是否定，先改布局或算法。若答案大多肯定，再考虑编译器报告和手写优化。

这个决策表会在第三册 AI 算子中继续使用。矩阵乘、卷积、量化反量化适合 SIMD；复杂控制流和随机图遍历不一定适合。先判断形状，再选指令。

案例：向量化失败后的重构 假设编译器报告一个循环因为函数调用无法向量化。循环每轮调用 `classify(record)`，函数内部读取多个字段并返回类别。重构可以分两步：先把 `classify` 拆成只依赖热字段的纯函数，并确保可内联；再把记录字段从 AoS 改成列式，让 `classify` 处理连续数组。若仍有复杂分支，可以先按类型分桶。这个过程展示了 SIMD 优化不是最后一行指令，而是接口、布局和控制流共同重构。

重构后仍要保留原 reference。若 `classify` 涉及错误处理、字符串或时间解析，不应为了向量化改变语义。可以只向量化其中的简单子条件，把复杂记录留给标量慢路径。高质量 SIMD 优化常常是混合路径，而不是全量替换。

从依赖图看向量化 判断一个循环能否向量化，最直接的方法是画出一轮迭代依赖下一轮迭代的地方。若第 i 次迭代只读 `input[i]` 并写 `output[i]`，各次迭代相互独立，向量化自然。若第 i 次迭代要读上一轮写出的 `output[i-1]`，循环就有跨迭代依赖，普通 SIMD 很难直接处理。若依赖只是归约，例如所有元素累加到一个总和，可以把单一依赖链改成多累加器或树形归约。若依赖是状态机，例如字符串转义、括号嵌套或变长编码，通常要把能规则化的子任务拆出来。

依赖图还能解释为什么有些循环看起来简单却不能向量化。指针别名会让编译器不确定输入和输出是否重叠；函数调用会让编译器不知道它是否有副作用；异常和边界检查会让控制流变复杂；写入位置由数据决定时，多个 lane 可能写到同一位置。解决方法不是盲目加编译选项，而是改变接口：使用 `span` 表达连续范围，拆出纯函数，明确输入输出不重叠，把数据相关写入改成 `count`、`scan`、`scatter` 三阶段。

这也说明 SIMD 和并行算法是同一思想在不同粒度上的表现。SIMD lane 需要独立，线程任务也需要独立；SIMD 需要 mask 处理条件，线程并行需要 selection 和 partition 处理分支；SIMD 的 horizontal reduce 和线程归约都要面对结合性、顺序和误差。学 SIMD 时不要只盯着指令，应把它看成最细粒度的数据并行模型。

Gather 和 scatter 的真实代价 SIMD 并不要求所有数据都连续。有些 ISA 支持 gather 从多个地址收集 lane，也支持 scatter 把 lane 写到多个地址。但 gather 和 scatter 通常比连续 load 和 store 更贵，而且更容易受 cache miss、TLB miss 和地址生成限制影响。它们解决了表达能力，不等于解决了性能问题。一个随机索引数组上的 gather，可能只是把多个标量 miss 捆在一起，不能让内存系统变成连续访问。

因此，遇到随机访问时要先问能不能重排数据。若可以按 key 排序、按 bucket partition、把热点字段压成连续数组，通常比直接 gather 更可靠。若随机访问不可避免，可以批量处理多个请求，增加内存级并行，或者把索引按页和 cache line 分组，降低完全随机程度。只有在数据形状确实无法改变、且 gather 版本经过测量有收益时，才应把 gather 当成核心优化。

Scatter 更要小心写冲突。多个 lane 可能写到同一输出位置，语义是最后一个赢、累加、取最大还是报错？若是累加，就需要冲突检测、原子或分阶段归约；若是写位置唯一，最好用 scan 或 partition 先分配不重叠区间。数据相关写入是 SIMD 和线程并行共同的难点，不能因为用了向量指令就忽略输出语义。

选择标量回退的工程判断 高质量 SIMD 代码一定要有标量回退。回退不是失败，而是工程边界。短输入、未对齐尾部、罕见错误路径、平台不支持目标指令、调试模式和语义复杂路径，都可能走标量。标量 reference 让测试清楚，也让平台覆盖更稳。没有回退的 SIMD 内核容易把一个优化点变成全系统移植风险。

回退策略要写进接口。可以在运行时根据 CPU 能力选择 AVX、SSE、NEON 或 scalar；也可以在编译期生成不同目标；还可以让小输入直接走 scalar，避免向量准备成本。选择逻辑必须可测试，不能只在开发机器上跑最快路径。报告应包含 scalar、自动向量化、手写 SIMD 三组结果，而

不是只展示最好版本。

回退还帮助定位 bug。若 SIMD 输出和 scalar 不一致，先缩小输入，固定随机 seed，记录 lane mask、尾部长度和第一个不同位置。很多 SIMD bug 出在尾部、符号扩展、饱和和舍入。标量 reference 是这些 bug 的锚点。性能工程不能脱离正确性锚点，否则优化越多，系统越难信任。

实验：用编译器优化报告观察一个数组加法循环是否向量化。再写单累加器和四累加器求和，比较性能。记录编译选项和 CPU 指令集。

标量 reference 每个向量版本都要有清晰标量版本。标量版本定义边界条件、溢出处理、浮点误差、错误输入和输出顺序。向量版本只是在这个合同内改变执行方式。

lane 语义 向量寄存器里的 lane 不是独立线程。它们共享一条指令，分支需要 mask，异常和尾部也要统一处理。读者要学会把 if 转成 mask，而不是把控制流幻想成自动并行。

依赖与别名 循环携带依赖会阻止向量化，指针别名会让编译器保守。C++ 接口可以用 span、const、restrict 风格约束和不重叠断言帮助编译器，也帮助人理解。

尾部处理 长度不一定是向量宽度整数倍。尾部可以用标量收尾、mask load/store 或 padding。不同选择影响代码复杂度、越界安全和性能。

对齐 现代硬件对非对齐访问容忍更高，但跨 cache line 或页边界仍可能更贵。对齐策略要和分配器、数据布局和尾部策略一起设计。

数值误差 浮点 reduce 改变结合顺序会改变结果。性能报告不能只比较速度，还要比较误差界限。整数溢出也要明确，是按位宽回绕、饱和还是报错。

自动向量化 优先让编译器完成简单循环，再查看报告和反汇编。手写 intrinsic 应限于热点、语义已固定、编译器确实做不到的路径。

可移植边界 x86 AVX、ARM NEON、SVE 的宽度和能力不同。项目接口应把向量实现藏在策略层，外部仍看到同一标量语义。

案例：阈值过滤 标量扫描大数组生成 mask，再写向量版本。要求输出位置和稳定性与 reference 一致，尾部输入必须覆盖。

案例：浮点求和 比较串行、分块、向量 reduce 的误差和速度。结论必须说明误差来源，不允许只写 SIMD 更快。

案例：别名反例 让输入和输出可能重叠，观察错误或编译器无法向量化。再用接口禁止重叠，说明语义约束如何换来优化空间。

源码阅读路线 Linux 源码阅读从 [tools/perf](#)、[arch/x86/lib](#)、[arch/arm64/lib](#) 附近进入即可。阅读目标不是把整套子系统抄进书里，而是追一条和本章实验相关的路径：用户态动作怎样变成内核对象，内核对象怎样排队、等待、唤醒、计数、提交或回收。

阅读报告要写证据等级。能直接测到的数字放在第一层；从源码控制流推导出的解释放在第二层；当前设备无法验证、只能作为合理猜测的内容放在第三层。这样读者会知道向量化语义报告中哪些结论可复现，哪些结论需要换机器继续确认。

源码阅读还要克制。看到一个复杂函数，不要把每个分支都展开；先回到本章的问题，问它能否解释一个等待点、一次状态转移或一条性能曲线。解释不了的内容可以留到索引，不要打断正文主线。

阈值过滤 标量扫描大数组生成 mask，再写向量版本。要求输出位置和稳定性与 reference 一致，尾部输入必须覆盖。

浮点求和 比较串行、分块、向量 reduce 的误差和速度。结论必须说明误差来源，不允许只写 SIMD 更快。

别名反例 让输入和输出可能重叠，观察错误或编译器无法向量化。再用接口禁止重叠，说明语义约束如何换来优化空间。

标量语义

SIMD 之前必须有标量 reference。它定义溢出、错误输入、尾部元素、浮点误差和输出顺序。

实验设计：先写最普通循环，再用同一输入校验向量版本。

数据并行

SIMD 适合多个元素执行同一操作。若每个元素走不同控制流，掩码和压缩成本可能抵消收益。
实验设计：比较均匀输入和分支高度随机输入上的向量化效果。

尾部处理

输入长度不一定是 lane 数倍。尾部可以用标量循环、掩码加载或补齐缓冲，选择取决于语义和成本。

实验设计：对长度从零到多倍 lane 加一的输入做边界测试。

对齐

现代指令支持非对齐加载，但跨 cache line 或页边界仍可能有成本。对齐更多是稳定性和简化边界，不是绝对规则。

实验设计：比较对齐起点和偏移起点的吞吐，并记录是否出现异常。

掩码

掩码让向量代码表达条件，但掩码不是免费。它适合把控制流变成数据流，也可能增加寄存器和混洗压力。

实验设计：实现字节分类计数，比较分支、查表和掩码版本。

归约

向量归约需要横向合并，浮点结果还受合并顺序影响。整数求和、饱和加法和浮点累加不能混为一谈。

实验设计：对浮点数组固定合并树，记录误差范围。

自动向量化

编译器需要看到连续范围、无别名、无循环依赖和明确边界。写 intrinsics 前，先让自动向量化报告解释失败原因。

实验设计：保存编译器报告中未向量化的理由，并逐条消除。

手写 intrinsics

手写 SIMD 适合稳定热点和明确 ISA。它会增加维护、派发、fallback 和测试成本，不能作为第一反应。

实验设计：把内核分成 reference、portable fast path、ISA-specific path 和 dispatch。

数值语义

快速数学选项可能改变 NaN、舍入、结合律和异常语义。性能报告必须写清是否允许这种改变。

实验设计：同一输入比较严格模式和 fast math 结果差异。

项目接口

VectorizationPlan 记录输入布局、lane 宽度、尾部策略、误差规则、fallback 和测试矩阵。没有它，SIMD 代码很难维护。

实验设计：每个向量内核都保留标量 reference 和随机测试。

7.8 SIMD 之前先证明批量语义

SIMD 很容易被写成指令大全：加载、比较、掩码、混洗、归约。这样的章节看起来专业，却容易让读者误以为性能来自某条神秘指令。真正的顺序应相反：先证明标量语义，再证明多个元素能执行同一操作，再决定让编译器自动向量化还是手写 intrinsics。没有这条顺序，SIMD 代码只会变成难维护的特殊版本。

标量 reference 是第一步。它定义输入长度为零怎么办，尾部元素怎么办，整数溢出是否允许，浮点误差如何比较，遇到非法字符是跳过、计错还是终止。向量代码必须继承这些语义。比如字节扫描中，标量版本遇到非 ASCII 字节如何处理，SIMD 版本也要一致；浮点归约中，标量顺序和向量合并树可能产生不同舍入，报告必须说明误差范围。

数据并行是第二步。SIMD 适合多个元素做同一件事。若每个元素都走完全不同的控制流，向量化可能需要大量掩码和压缩，收益未必存在。掩码是把控制流变成数据流的工具，不是免费午餐。它会占寄存器，会增加逻辑操作，也可能让后续压缩或存储更复杂。因此本章不能只展示成功案例，也要展示随机分支、稀疏保留和复杂错误路径下 SIMD 的边界。

尾部处理是第三步。输入长度很少刚好等于 lane 数倍。标量尾循环最简单，掩码加载能减少分支但要小心越界，补齐缓冲会改变内存访问和错误处理。教材应要求读者对长度 0、1、lane-1、lane、lane+1 做测试。很多 SIMD bug 不在大输入上暴露，而在尾部和边界上暴露。

自动向量化是第四步。编译器需要看到无循环依赖、无别名、连续范围和明确边界。优化报告里常见失败原因包括可能别名、循环携带依赖、调用不可内联、控制流复杂。读者应该先读失败原因，而不是直接手写 intrinsics。手写 SIMD 适合稳定热点和明确 ISA，但会带来 dispatch、fallback、测试矩阵和跨平台维护成本。

本章可以围绕三个内核做实验。第一个是数组加法，用它讲连续范围、对齐和尾部。第二个是字节分类计数，用它讲掩码、查表和分支分布。第三个是浮点归约，用它讲合并树和误差。每个实验都要保留标量 reference、自动向量化版本和手写版本，报告中写清哪个版本在什么输入上成立。这样 SIMD 就不再是“会写指令”，而是“会证明批量执行没有改变语义”。

案例推导：字节分类内核怎样从标量版本走到 SIMD 版本

日志解析常要判断字符是数字、分隔符、空白还是其他。标量版本逐字节判断，很容易写对，也方便处理错误。向量化的第一步不是写 intrinsics，而是确认分类规则能批量表达：每个字节的分类只依赖它自己，还是依赖前后状态？如果要识别引号内分隔符，状态会跨字节传播；如果只是统计分隔符数量，批量比较就很适合。

先写 reference：输入为空、只有一个字符、包含非 ASCII、末尾无换行、连续分隔符、超长字段，都要定义输出。然后写一个自动向量化友好的版本：连续数组、简单循环、无函数调用、无别名。编译器如果仍不向量化，读优化报告，找原因是循环依赖、条件复杂还是数据类型不合适。很多时候，改好接口和循环结构，比手写 SIMD 更有价值。

手写 SIMD 版本可以先做最窄目标：一次加载多个字节，与分隔符向量比较，生成掩码，再统计掩码位数。尾部用标量处理，保证边界清楚。下一步再讨论多个分隔符、查表分类或压缩输出。不要一口气写完整解析器的 SIMD 版本，否则错误来源太多，读者无法归因。

掩码版本要和输入分布一起测。如果大多数行格式稳定，SIMD 字节扫描可能显著减少比较和分支；如果错误处理很多，向量路径频繁回退，收益会下降。若每次找到分隔符后还要立刻调用复杂字段解析，前端调用和状态机可能重新成为瓶颈。报告要写瓶颈迁移，而不是只写 SIMD 速度。

数值内核也有类似问题。浮点向量归约改变合并顺序，可能改变最后几位。若业务需要 bitwise 一致，就要固定树形合并或继续使用标量顺序；若业务允许误差，就写明误差界。编译选项如 fast math 会改变 NaN 和结合律语义，不能偷偷打开。SIMD 章节要把性能和语义绑在一起，才不会培养出只追指令的习惯。

最终 VectorizationPlan 应列出 scalar reference、批量语义、尾部策略、对齐假设、ISA dispatch、fallback、误差规则和测试矩阵。手写 SIMD 代码只有在这张计划表成立后才值得进入项目。

手写 SIMD 的接口边界 若最终确实需要手写 SIMD，第一步不是写 intrinsics，而是设计接口边界。一个好的 SIMD 内核通常有四层。第一层是公开 API，接收 span、配置和语义选项。第二层是 portable reference，用标准 C++ 实现，作为 correctness oracle。第三层是 dispatch，根据 CPU 特性选择实现。第四层是 ISA-specific kernel，只处理已经检查过的输入和尾部策略。

这种分层能把复杂性关在局部。公开 API 不暴露 AVX 或 NEON 类型；reference 不依赖平台；dispatch 只处理选择；具体内核只处理性能关键路径。若把 intrinsics 直接散落在业务代码里，后续测试、移植、调试和回退都会变得困难。

Listing 7.6 SIMD 内核的分层接口草图

```
1 enum class KernelBackend : std::int32_t {
2     Scalar,
3     AutoVectorized,
4     NativeSimd,
5 };
6
7 struct KernelOptions {
8     KernelBackend backend = KernelBackend::AutoVectorized;
9     bool allow_fast_math = false;
10 };
11
12 void transform_batch(std::span<const float> input,
13     std::span<float> output,
```



```
14 const KernelOptions& options);
```

这里没有把某个指令集写进公开接口，是为了让调用者表达语义需求，而不是绑定实现。运行时可以根据平台选择 AVX2、AVX-512、NEON 或标量 fallback。测试可以强制选择 Scalar 和 AutoVectorized，比较结果。若某个 ISA 版本出问题，可以临时关闭，而不影响业务 API。

运行时派发的测试矩阵 多版本派发的测试不只是“默认路径跑通”。必须强制每个 backend 跑同一组 correctness tests。输入要覆盖长度为 0、1、向量宽度减一、正好向量宽度、向量宽度加一、大数组、非对齐起点、输出和输入不同数组、允许的重叠或禁止重叠场景。浮点内核还要覆盖 NaN、Inf、负零、极大值和极小值，除非 API 明确不支持这些值。

性能测试则要区分小输入和大输入。小输入时 dispatch 成本、函数调用和尾部处理可能占主导；大输入时内存带宽或计算吞吐主导。若一个 SIMD 版本只在超大输入上快，API 可以设置阈值，小输入走标量路径。很多成熟库都有这种 size threshold，因为 SIMD 启动成本不是零。

对齐策略的工程现实 早期 SIMD 教材常强调对齐加载。现代 CPU 对连续非对齐访问已经更友好，但对齐仍然有价值：减少跨 cache line 或跨页访问，便于某些 ISA，简化内部块处理。工程上常见做法是让容器或分配器提供对齐保证，但公开 API 仍要能处理普通 span。内部可以先处理到对齐边界，再进入对齐主循环，最后处理尾部。

不过，对齐优化要防止过度复杂化。若内核主要受内存带宽限制，复杂的对齐前缀可能收益很小；若输入来自用户任意切片，对齐主循环比例可能不高；若需要跨平台，某些 ISA 的对齐要求不同。测量应覆盖对齐和非对齐输入，而不是只在理想对齐数组上报告性能。

水平归约和 lane 间通信 SIMD lane 之间并非完全免费通信。逐元素 map 每个 lane 独立，最适合 SIMD；归约需要把多个 lane 的 partial 合并成一个标量，涉及 horizontal add、shuffle 或树形 lane 合并。不同 ISA 对水平操作支持不同，成本也不同。因此向量归约通常会使用多个向量累加器，先在向量寄存器里积累很多元素，最后再做一次水平合并。

这个模式与线程本地 partial 类似。lane 内先局部累加，最后水平合并；线程内先本地 partial，最后线程间合并；节点内先 map-side combine，最后跨节点 reduce。层次化归约不是某一层技巧，而是贯穿硬件 lane、线程、NUMA 和集群的通用结构。

Vector-friendly 数据结构 不是所有数据结构都适合 SIMD。链表、树节点和虚函数对象通常难以向量化，因为地址不连续、分支多、类型不统一。若热路径需要 SIMD，应把数据组织成 vector-friendly 形状：连续数组、SoA、AoSoA、紧凑索引、固定宽度字段和分离冷数据。第 6 章的数据布局不是 SIMD 之前的附属内容，而是 SIMD 能否发挥作用的前提。

以日志解析为例，原始文本是变长字符串，很难直接 SIMD 化完整业务逻辑；但某些子问题可以向量化，例如扫描分隔符、统计字符类别、批量检查 ASCII 范围。工程中常把复杂解析拆成多个阶段：先用 SIMD 找结构位置，再用标量状态机处理复杂字段。不要要求 SIMD 解决整个业务函数，而要找出规则、重复、数据并行的子内核。

本章的反例清单 判断 SIMD 是否值得时，可以先看反例。第一，输入很小，dispatch 和尾部成本超过收益。第二，内核受内存带宽限制，SIMD 只减少指令数不减少字节。第三，数据结构是随机指针追踪，gather 成本高且预取困难。第四，分支复杂且各分支工作量大，掩码执行会做大量无用工作。第五，数值语义要求逐位一致，重排空间受限。第六，代码需要跨很多平台维护，但热点不够热。

这些反例不是让读者不用 SIMD，而是让优化更有目标。真正应该优先 SIMD 化的，是热、规则、数据连续、迭代独立、算术强度足够或函数调用成本可摊薄的内核。Compute Systems Engine 的 map、normalize、简单解析扫描、局部归约都可以成为候选；全局队列、锁等待和分布式重试不是 SIMD 能解决的问题。

自动向量化的前置条件 自动向量化不是编译器魔法，它需要源码表达出足够清楚的循环形状。典型前置条件包括：循环边界可分析，迭代之间没有无法证明的依赖，内存访问尽量连续或固定步长，分支能转成掩码或被分离，函数调用能内联或被证明无副作用，输入输出别名关系清楚。任何一个条件不满足，编译器可能保守放弃。

许多向量化失败不是因为编译器弱，而是接口太含糊。一个循环写 `output[i]`，同时读 `input[i+1]`，如果 input 和 output 可能重叠，编译器必须考虑写入影响后续读取。一个循环内部调用虚函数，编译器很难知道函数是否访问全局状态或抛异常。一个循环每轮可能 push 到 vector，

输出位置不是简单函数，也不容易向量化。软件工程师要做的是把数据依赖变成编译器能看懂的结构。

向量化报告要成为开发流程的一部分。GCC、Clang 和 MSVC 都有不同形式的优化报告。读报告时要关注“not vectorized”的原因，而不是只看成功。常见原因如 unsafe dependent memory operations、cannot identify array bounds、call in loop、control flow not understood。把这些原因翻译回源码，就能指导接口和布局改造。

ASCII 字节分类从 reference 开始 SIMD 章节最好的入口不是指令名，而是一个可完整定义的字节分类任务。输入是一段日志文本，输出是每个字节是否为数字、分隔符、换行或非法字节。标量 reference 逐字节判断，遇到非法字节记录位置，遇到换行更新行号。这个版本可能不快，但它定义了空输入、尾部、错误位置、非 ASCII 字节和连续分隔符的语义。没有 reference，SIMD 版本就没有校验对象。

接下来判断哪些部分能批量化。单字节类别判断只依赖当前字节，适合 SIMD；行号维护依赖前面换行数量，需要把 mask popcount 变成累计值；引号内分隔符识别依赖状态，不能简单把每个分隔符都当字段边界。这样拆开后，SIMD 只处理它擅长的子问题：批量比较、生成 mask、统计或输出候选位置；复杂语义仍由状态机消费候选结果。这个分层比“用 AVX 扫字符串”更准确。

尾部和错误位置要优先测试 字节分类的尾部很容易出错。长度为 0、1、lane 宽度减一、正好 lane 宽度、lane 宽度加一，都要覆盖。若用掩码加载，必须保证不会越界读；若用标量尾循环，必须保证主循环和尾循环没有重复或漏掉；若用 padding，必须保证 padding 字节不会被报告成真实错误或分隔符。错误位置也要精确：SIMD 一次发现多个非法字节时，是报告第一个、全部还是只计数，必须和 reference 一致。

很多性能 bug 来自“测试只用了长且规整的输入”。真实日志可能很短，可能没有换行，可能最后一行不完整，可能混入二进制字节。SIMD 内核要把这些情况当作常规输入，不是边缘装饰。项目的测试矩阵应该强制每个 backend 跑同一组输入，包含 scalar、auto-vectorized 和 native backend。

水平归约和多累加器 逐元素 map 很适合 SIMD，但归约需要 lane 间合并。比如统计数字字符数量，向量比较得到 mask 后，可以 popcount mask；统计多个类别时，可以分别累计多个 mask。若把每次 popcount 结果都加到同一个标量计数器，会形成依赖链；更好的方式是按块局部累计，或者使用多个独立累加器，最后合并。这个思路和多线程局部 partial 完全一致。

浮点归约还要处理数值语义。向量合并顺序和串行顺序不同，结果可能不逐位一致。若项目里的计算是整数计数，使用固定宽度无符号类型并检查溢出边界；若是浮点指标，报告要写清是否允许重排、FMA 和 fast math。SIMD 优化不能偷偷改变业务合同。

自动向量化失败时的排查顺序 如果编译器没有向量化，不要先写 intrinsics。先看优化报告。若失败原因是可能别名，检查输入输出是否可能重叠，是否能调整 API 表达只读输入和独占输出。若原因是调用不可内联，把分类函数改成 constexpr 表或内联逻辑。若原因是循环控制流复杂，把错误路径拆出热循环。若原因是成本模型不值得，比较不同输入长度，不要强迫小输入走重路径。

反汇编用于验证执行形态。报告说向量化，不代表生成了理想代码；可能有运行时别名检查，可能尾部代码很重，可能使用了窄向量。报告说没有向量化，也不代表函数慢，可能瓶颈在内存带宽或调用层。学习 SIMD 的关键是把源码语义、编译器证据和硬件行为连起来。

项目中的 VectorizationPlan 每个候选内核都要写 VectorizationPlan。它包含 reference 名称、批量语义、输入输出别名规则、尾部策略、对齐假设、数值规则、平台 backend、fallback 和测试矩阵。比如 ASCII 分类内核的计划会写：输入只读，输出 mask buffer 独占；尾部使用标量；不要求对齐；非法字节报告第一个位置和总数；默认 backend 为自动向量化，可强制 scalar；native SIMD 只在测试通过后启用。

这份计划让 SIMD 不再污染业务层。业务代码调用的是分类接口，不知道 AVX、NEON 或 SVE。dispatch 层根据机器能力选择实现，测试层可以强制每个实现。若某个平台出现 bug，可以关闭对应 backend，reference 仍可运行。这样的工程边界会在第三册 AI 算子中继续使用。

事故复盘：SIMD 主循环正确但尾部错了 手写 SIMD 最常见的事故不是编译不过，而是主循环很快，尾部、错误位置或边界长度悄悄错。输入长度刚好等于 lane 数时通过，长度为 lane 减一或 lane 加一时失败；标量版报错在第 17 字节，向量版只知道某个 32 字节块有错；字符串里出现转义和引号以后，批量分类和语法状态脱节。向量化如果没有标量语义做锚点，只会把 bug 批量放大。

推导顺序应先拆语义。哪些判断是逐字节独立的，例如分隔符候选、数字字符、空白字符；哪些

判断依赖前文状态，例如引号内外、转义、跨块字符串。独立部分适合 SIMD 批量产生 mask，状态相关部分可能需要标量确认或分块 carry。Mask、tail policy、alignment、fallback、dispatch 都来自这些语义边界，不是看到 for 循环就写 intrinsics。

实验要覆盖 0、1、lane 减一、lane、lane 加一、非对齐地址、错误密集输入和随机输入。小输入变慢不一定说明 SIMD 错，可能是 dispatch 成本超过收益；大输入变快也不代表语义完整，必须比较错误位置、字段数和 checksum。自动向量化报告、反汇编和 perf 只能回答机器是否生成了批量指令，不能替代 reference。SIMD 章节要训练的是“先证明可批量，再选择指令”。

向量化实验判读 SIMD 实验先看边界长度：0、1、lane 减一、lane、lane 加一是否全部通过。再看错误位置和标量 reference 是否一致。性能上，小输入慢不代表 SIMD 错，可能是 dispatch 和尾部成本；大输入不快也不代表指令没用，可能已经受带宽限制。浮点结果不同必须写误差合同，不能只写“略有误差”。

这一段也服务于复习。读者合上书后，应该能凭本章的判读口径重新审查自己的代码：有没有最小正确版本，有没有能击穿旧方案的输入，有没有状态记录，有没有资源指标，有没有失败后的恢复动作。若五个问题缺两个以上，就说明实现还停在演示层，而不是工程层。

字节分类的尾部测试矩阵 字节分类内核的测试矩阵应单独列出长度：0、1、2、lane 减一、lane、lane 加一、两倍 lane 减一、很大长度。每个长度再组合非对齐起点、非法字节、第一个字节非法、最后一个字节非法、连续分隔符和无换行结尾。主循环再漂亮，也必须通过这些输入。

mask 的语义也要写清。一个 mask 可以表示哪些字节是分隔符，也可以表示哪些字节非法，还可以表示候选换行位置。后续标量状态机消费 mask 时，必须知道 mask 对应原始偏移。若 SIMD 版本只统计数量，却业务需要第一个错误位置，它就不是等价实现。

这个案例会直接服务第三册算子开发。AI 算子里也有批量语义、尾部、对齐、误差和 backend dispatch。第二册在字节分类中练清这些边界，第三册进入量化矩阵乘时就不会把向量指令当作神秘开关。

实验课：ASCII 分类从标量 reference 到 backend dispatch 标量 reference 逐字节判断数字、分隔符、换行和非法字符，并返回第一个错误位置、错误总数和分隔符位置列表。这个版本定义所有边界：空输入、无换行结尾、非 ASCII、连续分隔符、尾部不足一个 lane。然后写一个自动向量化友好的版本，不使用平台 intrinsics，只让循环、数据和别名关系更清楚。

第三步才写 native backend。它一次处理多个字节，生成 mask，尾部用标量收尾。公开 API 只接受输入 span 和输出对象，backend 通过配置选择；业务层不知道 AVX、NEON 或其他 ISA。测试强制运行 scalar、auto 和 native 三个 backend，所有 backend 使用同一组输入。若 native backend 不支持当前平台，就明确走 fallback。

判读时先看正确性矩阵。长度 0、1、lane 减一、lane、lane 加一；非法字节在首尾；非对齐输入；输出缓冲正好大小；这些都通过后才看性能。性能报告区分小输入和大输入，小输入可能被 dispatch 成本淹没，大输入可能受内存带宽限制。若只在理想长度上测试，SIMD 代码不可靠。

最后补浮点归约对照。整数字节分类一般可以逐位一致，浮点归约不一定。报告要说明是否允许重排，是否使用 fast math，误差如何比较。这个习惯会直接迁移到第三册量化和推理算子。

SIMD 练习验收重点 合格答案先写 scalar reference 和批量语义，再讨论指令。必须列出尾部策略、错误位置语义、对齐假设、fallback 和测试矩阵。浮点题要写误差合同；整数题要写溢出合同。只给 intrinsics 代码，没有 reference 和尾部测试，不算完成。

答案还要写“为什么不采用更复杂方案”。例如没有证据证明队列是瓶颈，就不应把锁队列换成无锁；没有证明输出顺序可变，就不应随意重排；没有证明错误路径能恢复，就不应只提交正常路径。能解释不做什么，说明读者开始具备工程判断。

最后，答案必须包含可复查材料。至少有输入摘要、关键状态、期望输出、实际输出、运行命令和未验证边界。没有这些材料，老师或同伴无法判断结论是否成立。第二册的作业不是让读者写漂亮段落，而是训练读者把系统问题变成可验证证据。

项目实现：SIMD、指令级并行与向量化 本章对应的贯穿项目实现不要求一次写成完整框架，但必须有最小可运行切片。这个切片包含三个部分：一个 reference，一个带本章机制的实现，一个能击穿 reference 之外隐含假设的测试。Reference 用来固定语义，机制实现用来展示本章知识，测试用来证明新增复杂度确实解决了问题。

代码组织上，不要把所有内容放进一个大文件。基础类型、输入生成、核心机制、报告输出和测

试应分开。即使当前只是教学项目，也要让目录能表达责任边界。这样后续章节接入时，不会因为找不到状态归属而重新发明一套对象。

每个实现都要输出一段机器可读摘要。摘要至少包含输入规模、关键状态、错误数、输出校验、耗时和未验证边界。这里的机器可读不要求复杂格式，简单的 key value 文本也可以。关键是让脚本和读者都能复查，而不是只能看屏幕截图。

测试应包含正常路径、边界路径和失败路径。正常路径证明功能可用；边界路径证明长度、容量、尾部、空输入、关闭或超时没有遗漏；失败路径证明本章机制不是装饰。若失败路径写不出来，往往说明本章概念还没有落到工程对象上。

报告最后写一个取舍段。新增机制带来了什么好处，增加了什么成本，哪些场景不应该使用它，下一章会复用哪个字段或对象。这个取舍段非常重要，因为系统工程不是把所有高级机制都加进去，而是在证据和约束下选择合适的边界。

调试笔记：SIMD 失败多半在主循环之外 SIMD bug 常发生在尾部、对齐、错误位置和 fallback。排查时先强制 scalar backend，确认 reference；再强制 auto backend；最后强制 native backend。若只有 native 失败，缩小到长度矩阵和尾部策略。若所有 backend 都失败，说明批量语义或 reference 定义错了。不要一开始盯着向量指令，先把等价性边界查清。

复查清单：SIMD、指令级并行与向量化 复查本章时，先检查 reference 是否仍然存在。没有 reference，后面的并发、优化、异步或分布式改造都无法证明语义没有变。再检查压力输入是否尖锐：它应当攻击本章最核心的边界，而不是只把数据量放大。第三，检查状态是否可观察：关键对象的身份、状态、错误和提交结果应能在报告中找到。

第四，检查代码示例是否说明禁止行为。调用者不能做什么，往往比能做什么更能体现工程质量。第五，检查实验是否写出未验证边界；当前设备、权限或输入无法证明的结论，必须标注。第六，检查本章对象是否能被下一章复用，不能复用就说明边界还不稳。

完成这六项后，再看文字是否连贯。每个概念都应从问题推出，每个案例都应从朴素方案走到改进方案，每个实验都应能解释一个判断。若某段话放到其他章节也成立，就把它改成本章的具体对象、具体输入和具体失败。

7.9 工程案例：从标量解析语义推导 SIMD 版本

SIMD 最容易被误讲成“把 8 个或 16 个元素一起算”。真实工程中，难点通常不在指令本身，而在标量语义是否能批量化：是否有跨元素状态，尾部如何处理，错误如何报告，输出位置如何分配，数值结果是否允许改变。只有这些问题先清楚，向量化才是正确优化，而不是把 bug 做宽。

一、标量 reference 必须足够严格 以日志分隔符扫描为例，标量 reference 要定义逗号、换行、引号、转义和非法字符的语义。若引号内逗号不算分隔符，SIMD 版本不能简单比较所有字节是否等于逗号；若反斜杠转义影响下一个字符，状态会跨字节传播；若错误报告要求第一处错误位置，批量扫描必须保留顺序信息。reference 越清楚，SIMD 改写越安全。

很多 SIMD 教学例子选择无状态数组变换，这是必要入门，但不足以覆盖系统项目。日志解析、JSON 扫描、CSV 解析、UTF-8 校验都涉及状态传播。工程上常先用 SIMD 找候选位置，例如引号、反斜杠、逗号、换行，再用标量或位运算处理状态。这样不是完全替代标量，而是把批量能力用在最适合的子问题上。

二、掩码是批量控制流 向量比较产生掩码，掩码再用于选择、压缩、计数或查找第一个匹配位置。理解掩码，比背具体指令更重要。一个 32 字节块中哪些位置是逗号，哪些位置是换行，可以用位图表示；块内分隔符数量可以用 popcount；第一个分隔符可以用 countr zero；输出位置需要 scan 或 prefix popcount。

掩码让控制流从“每个字符一个分支”变成“每个块一个位集合”。这能减少分支失败，但也带来新问题：跨块状态怎么传，块尾不足如何处理，错误位置如何从掩码恢复到全局偏移。教材要把掩码和输出位置讲清楚，否则 SIMD 代码只会像魔法。

三、尾部处理是语义的一部分 数组长度不一定是向量宽度的倍数。尾部可以用标量处理，也可以用 masked load，也可以在输入末尾填充哨兵字节。每种方式都有边界。标量尾部简单可靠，但有额外分支；masked load 依赖指令集；填充哨兵要求输入 buffer 后面有安全空间，不能越过映射页或文件末尾。

尾部错误也要测试。空输入、长度 1、长度等于向量宽度减一、等于向量宽度、等于向量宽度加

一、最后一个字节是转义、最后一行无换行，这些都应进入测试集。很多 SIMD bug 不出现在大输入，而出现在尾部和页边界。

四、自动向量化需要源码配合 编译器自动向量化依赖别名、对齐、循环边界、数据依赖和异常路径。若输入输出可能重叠，编译器会保守；若循环内部有函数调用或复杂分支，向量化可能失败；若数据结构不是连续数组，编译器没有可向量化形状。源码要用清楚的 `span`、`restrict` 等价信息、简单循环和独立输出帮助编译器。

优化报告是必读材料。看到 “loop not vectorized” 不等于失败，而是线索。报告会告诉你是否存在依赖、无法证明别名、循环控制复杂，还是成本模型认为不值得。读者应先尝试让自动向量化成功，再决定是否手写 `intrinsics`。手写 SIMD 维护成本高，只有在自动向量化无法表达关键模式或收益足够明确时才值得。

五、手写 SIMD 要多版本派发 不同机器支持不同指令集。x86 有 SSE、AVX2、AVX-512，ARM 有 NEON 和 SVE。项目不能只在开发机器上写一个版本就结束。较稳妥的结构是：标量 `reference` 永远存在；`portable fast path` 作为默认；平台特化版本通过编译期或运行时派发选择；所有版本共享同一测试矩阵。

多版本派发还要考虑二进制体积和启动成本。若内核很小，派发开销可能不可忽略；若输入很短，标量可能更好；若平台特化很难维护，收益不稳定就不应进入主线。SIMD 不是信仰，而是一个带维护成本的实现选择。

六、数值语义不能被向量化偷偷改变 整数数组变换通常容易保持语义，浮点归约则不同。向量化和并行化会改变加法顺序，导致舍入差异。若业务要求 `bitwise identical`，就要固定顺序或使用补偿算法；若允许误差，就要在合同中写明误差范围和测试方法。不要让编译器的 `fast math` 选项悄悄改变 NaN、无穷、舍入和有符号零语义。

这个问题会在 AI 算子开发中继续出现。矩阵乘、归约、归一化都涉及数值误差和确定性。第二册先在普通高性能计算里讲清楚，第三册进入 AI 时才不会把“快”放在“数值合同”前面。

七、SIMD 和线程的组合顺序 同一个内核可以先向量化再多线程，也可以先多线程再向量化。工程上通常先固定单线程标量语义，再优化单线程批量执行，最后扩展到多线程。这样每一步的收益和 bug 都容易归因。如果同时改 SIMD、线程和布局，结果变快也不知道原因，结果变错更难定位。

线程会改变 SIMD 收益。单线程向量化后，内核可能从计算受限变成内存带宽受限；再加线程时，带宽更快到顶。报告要写瓶颈是否迁移。若 SIMD 版本已经打满内存带宽，多线程收益有限不是失败，而是系统达到另一层边界。

八、实验交付：VectorKernelMatrix 本章实验包括四类输入：普通数组变换、字节分类、稳定 `filter` 前的 `mask` 生成、浮点归约。每类至少有标量 `reference`、自动向量化版本、手写或平台特化版本。报告记录指令集、编译选项、向量化报告、`checksum`、尾部测试和不同长度下的性能。

验收要特别看失败输入。字节分类要测引号和转义跨块；`filter mask` 要测全保留、全丢弃和稀疏保留；浮点归约要测极大极小混合、NaN 和不同合并顺序。SIMD 章节的质量，不在于代码看起来底层，而在于批量语义和边界被证明清楚。

7.10 源码阅读：从编译器向量化报告反推代码边界

SIMD 学习不能只读 `intrinsics` 文档。很多时候，编译器已经能生成不错的向量代码；另一些时候，它拒绝向量化，并给出非常有价值的理由。读向量化报告，是理解代码边界的低成本入口。它会告诉你依赖、别名、控制流、成本模型和目标指令集怎样共同决定是否向量化。

一、别名信息决定编译器能否放心重排 若函数接受两个裸指针，编译器可能无法证明输入输出不重叠。它必须保守地按标量顺序生成代码，避免写输出影响后续读输入。使用 `std::span` 不能自动消除别名，但清晰接口、`const` 输入、独占输出和局部创建输出容器能帮助人和编译器理解边界。某些场景可以用编译器扩展表达 `restrict`，但要谨慎封装。

报告中出现 `alias` 相关信息时，不要立刻手写 SIMD。先看接口是否过宽：输入输出是否本来就不应重叠，是否可以拆成只读输入和独占输出，是否可以先复制小的控制表，是否可以把状态写入局部 `partial`。很多向量化失败其实是 API 语义没有表达清楚。

二、循环携带依赖要从算法上拆 编译器不能向量化真正的循环携带依赖。例如前缀和每个元素依赖前一个输出，普通方式不能直接变成独立 `lane`。解决不是强迫编译器，而是改算法：分块局

部 scan，再 scan block sums，再回填。这个思想和第 13 章一致。向量化报告指出依赖时，读者要判断它是真依赖还是可以通过算法改写消除的依赖。

日志解析中的引号状态是真依赖的一部分，但字符分类不是。可以先 SIMD 找出候选特殊字符，再用标量状态机处理候选区间。这样把可批量部分和顺序状态部分分开。高质量 SIMD 不是所有东西都向量化，而是把向量用在合适的子问题上。

三、成本模型拒绝向量化时要看输入规模 编译器有时认为向量化不值得，因为循环太短、额外准备成本高、可能需要复杂尾部处理。对于小输入，这可能是正确决定；对于项目中的大 batch，成本模型可能保守。报告要结合输入规模判断。若循环在真实 workload 中很长，可以尝试手动改写；若大部分输入很短，标量路径保留更重要。

多版本策略可以按长度选择。短输入走标量，长输入走向量。阈值由 benchmark 决定，而不是由向量宽度拍脑袋决定。报告记录阈值、输入长度分布和平台差异。这样 SIMD 优化才不会伤害短请求尾延迟。

四、反汇编验证 lane 宽和尾部 优化报告说 vectorized 后，还要看反汇编确认使用了什么指令、lane 宽多少、是否有 peel loop、是否有 scalar epilogue。某些情况下编译器会生成很宽的向量但降频明显，或者生成复杂 gather 导致收益不佳。反汇编和运行结果结合，才能判断向量化质量。

尾部路径也要看。若主体向量很快，但尾部标量包含复杂分支，小输入可能主要走尾部。若 masked tail 访问越界风险没有处理，代码可能依赖未定义行为。反汇编不是只看主体循环，也要看入口、对齐处理和尾部。

五、向量化报告交付 本章要求每个向量优化提交保存三份材料：编译器报告、关键反汇编、输入长度分布。报告解释为什么编译器接受或拒绝向量化；反汇编确认实际指令；输入分布说明该优化覆盖多少真实数据。缺少任一材料，结论都不完整。

读者做完这个练习后，会更少把 SIMD 当成神秘技巧。它本质上是把批量语义、编译器证明和硬件执行对齐。能对齐时收益很大；不能对齐时，保留清晰标量版本才是正确选择。

7.11 补强案例：SIMD filter 和 scan 的连接

SIMD 常先用于生成 mask，而真正写输出还需要 scan。以 filter 为例，向量比较能一次判断多个元素是否保留，得到一个 mask；但每个保留元素写到哪里，仍然取决于它之前有多少元素被保留。这就把 SIMD 章节和并行 scan 章节连接起来。

一、块内 mask 到块内 rank 一个 32 lane mask 可以用 popcount 得到保留数量，用 prefix popcount 得到每个 lane 的局部 rank。支持压缩存储的指令集可以直接 compress store，不支持时可以查表或标量写保留元素。无论实现方式如何，语义都是把 mask 转成输出位置。

二、块间 offset 来自 scan 块内 rank 只给出块内位置，块间起点需要对每个块的保留数量做 scan。这样 SIMD filter 的完整结构是：向量生成 mask，计算每块 count，scan 块 count，按块 offset 写输出。它不是单独 SIMD 技巧，而是批量执行和并行算法的组合。

三、稀疏和密集命中策略不同 命中率低时，输出很少，压缩写收益明显；命中率高时，几乎所有元素都保留，直接复制或简单 map 可能更快；命中率中等时，mask 和 scan 成本需要测量。输入分布决定策略。报告必须覆盖全丢弃、全保留和随机命中。

四、实验 实现标量 stable filter、SIMD mask 加标量写、SIMD mask 加块 count 和 scan。比较不同命中率、不同长度和不同尾部。输出必须和 reference 完全一致。这个实验会让读者看到 SIMD、scan、输出位置和稳定性怎样组合。

7.12 补充案例：平台差异和降级路径

SIMD 代码必须面对平台差异。一本面向计算系统的教材不能只在 x86 AVX2 上写通，也要说明 ARM NEON、不同向量宽度、不同编译器和无 SIMD fallback 的路径。

一、标量路径永远保留 标量 reference 不只是测试 oracle，也是运行时 fallback。未知平台、短输入、禁用特化、调试模式都可以走标量。手写 SIMD 版本必须和标量共享语义测试。删除标量路径会让移植和排错困难。

二、运行时派发要可测试 运行时检测 CPU feature 后选择实现。测试不能只测当前机器支持的最快路径，也要强制选择标量和较低级路径。可以通过环境变量或构建选项覆盖派发。否则某个

fallback 可能长期坏掉。

三、向量宽度不应进入业务语义 业务代码不应假设一次处理 16 或 32 个元素。向量宽度属于实现细节。接口只表达输入 span 和输出合同。内部可以按平台选择块宽，尾部处理保证所有长度正确。

四、实验 在同一机器上强制标量、自动向量化、手写特化三种路径。若有 ARM 设备，再比较 NEON。报告写出平台、指令集和 fallback 是否通过。这样 SIMD 章节为用户的 Termux 环境也留下合理路径。

7.13 从批量语义推导向量化

SIMD 常被误解成“用更宽的寄存器一次算多个数”。这句话只描述了表面。真正的前提是程序具有批量语义：多个元素之间没有必须串行等待的依赖，内存访问能按规则组织，尾部元素能被定义清楚，异常和数值误差有可接受边界。若这些语义没有先写清，向量代码就会变成难以审查的特殊版本。向量化章节要先讲为什么可以批量处理，再讲如何让编译器或手写 intrinsic 表达这种批量。

一、先证明 lane 之间独立 过滤、加法、乘法、阈值比较这些操作通常可以按 lane 并行，因为第 i 个元素的结果不依赖第 $i-1$ 个元素。前缀和、递归滤波、状态机解析则不同，后一个结果可能依赖前一个状态。读者看到循环时，第一步不是问“能不能用 AVX”，而是画出每次迭代之间的数据依赖。若依赖只在最后归约，可以用多个 partial；若依赖是前缀结构，可以用 scan；若依赖来自少数状态，可以考虑分块和边界状态。不同依赖形状对应不同算法，不是统一套一个向量模板。

二、内存形状决定向量质量 连续加载最适合 SIMD，步长访问次之，gather/scatter 成本更高，随机指针追逐最难。若数据布局仍是大结构体，而热循环只处理其中一个字段，向量加载会浪费带宽；若改成列式数组，lane 可以自然对应连续元素。对齐也要具体分析：现代处理器能处理未对齐加载，但跨 cache line 或页面仍可能增加成本。尾部处理同样是语义问题：用标量收尾、掩码收尾还是补齐输入，取决于输出是否允许额外元素、是否需要异常一致、是否追求固定路径。

三、自动向量化报告要逐条读 编译器没有向量化，通常会给出原因：可能是别名不明、循环边界不清、调用不可内联、存在循环携带依赖、内存访问非连续或浮点语义太严格。读者应把这些原因当成设计反馈。若是别名问题，可以调整接口为 `std::span<const float>` 和 `std::span<float>`，并保证输入输出不重叠；若是调用问题，可以把小函数内联或改成查表；若是浮点问题，要决定是否允许重排。不要一看到编译器没向量化就直接手写 intrinsic，先理解它为什么拒绝。

四、数值语义不能被速度掩盖 整数加法在不溢出的前提下容易重排，浮点加法不同。向量化和多累加器会改变合并顺序，结果最后几位可能变化。某些业务只要求误差范围，某些业务要求 bitwise reproducible。两者都合理，但必须写进合同。若要求完全可复现，可以固定归约树或使用更高精度中间值；若允许误差，要给出误差度量和测试输入。性能优化不能偷偷改变数值承诺。

五、可移植性是向量代码的组成部分 手写 AVX2、AVX-512 或 NEON 代码时，要准备标量 fallback 和特性检测。否则程序只能在少数特定机器上运行。接口层应把语义和实现分开：同一个 `filter_scores` 可以有标量版本、自动向量化版本和平台 intrinsic 版本；测试先比较语义，再比较性能。代码审查要检查尾部、对齐、别名、异常、数值误差和平台分派，而不是只看是否用了宽寄存器。向量化的最终目标是可维护的批量执行，不是展示指令名称。

7.14 尾部、掩码和异常路径的工程细节

向量化示例经常只展示能被向量宽度整除的输入，这会让读者误以为 SIMD 的主要难点是写出一条宽指令。真实工程里，尾部元素、条件掩码、异常路径和别名边界才是审查重点。一个处理 N 个元素的函数，必须在 N 小于向量宽度、 N 不是向量宽度倍数、输入输出重叠、出现非法值时仍然定义清楚。性能代码越底层，边界越不能模糊。

一、尾部处理有三种常见策略 第一种是主循环处理完整向量，剩余元素交给标量循环。这种方式最清楚，适合教学和多数业务。第二种是用掩码加载和掩码存储处理尾部，可以减少分支，但要求平台支持，且要确认 masked-off lane 不会触发非法访问。第三种是输入补齐，让数据长度向上对齐，尾部填充中性元素。补齐适合批处理和张量计算，但会改变内存占用和边界检查。选择哪一种，取决于平台、输入大小分布和错误处理要求。

二、掩码是数据化的分支 阈值过滤、条件赋值、取最大值都可以把分支变成 mask。mask 不

是魔法，它只是把“哪些 lane 有效”变成向量寄存器中的数据。若条件随机，mask 往往比分支更稳定；若条件高度可预测，分支可能更便宜；若两边工作量差异很大，mask 可能执行了大量本来不会执行的计算。报告要用输入分布解释 mask 的收益，而不是把“无分支”当成必然更快。

三、压缩输出需要 scan 或专用指令 向量过滤最难的不是比较，而是把保留元素紧凑写出。简单 mask store 会在原位置留下空洞，若要 compact，就需要根据 mask 计算每个有效 lane 的输出偏移。某些平台提供 compress store，某些平台需要查表或局部 prefix。这里和并行 scan 章节相连：输出位置来自前面保留了多少元素。把这个连接讲清楚，读者就能理解为什么 SIMD filter 不是几条比较指令就结束。

四、异常和非法值要保持语义一致 如果标量版本遇到非法输入会记录错误，向量版本也必须处理。常见做法是主向量路径先快速检测是否存在异常 lane，若没有异常就走快路径；若存在异常，再退回标量或慢路径逐个处理。这样正常输入保持高吞吐，异常输入仍有正确语义。不能为了向量化直接跳过错误检查，否则优化改变了业务合同。

五、向量代码审查表 审查一个向量函数时，至少检查七点：输入长度为零是否正确，长度小于向量宽度是否正确，尾部是否越界，输入输出是否允许重叠，浮点重排是否被允许，异常路径是否和标量一致，平台 fallback 是否存在。只有这些问题都有答案，向量代码才算工程代码。否则它只是某个输入上的演示。

7.15 贯穿案例：一个 SIMD 过滤器为什么把错误位置弄丢了

向量化最容易被讲成“把八个元素放进一个寄存器”。这句话没有错，但离工程还很远。考虑一个日志过滤器：输入是解析后的事件记录，规则是保留时间范围内、级别不低于阈值、事件名合法的记录。标量 reference 会在遇到非法事件名时记录精确 record id 和错误字节位置。手写 SIMD 版本先按字节批量比较，生成 mask，再把通过的 lane 写入输出。它在规则输入上很快，却在错误输入上只报告“某个向量块有错误”，甚至把块尾部的非法字节归到下一条记录。速度提升是真的，语义退化也是真的。

这个事故说明，向量化不是先选 AVX2、NEON 或 AVX-512 指令，而是先拆语义。哪些判断是逐元素独立的，哪些判断需要跨元素状态，哪些错误必须精确定位，哪些输出必须保持顺序，尾部元素如何处理，平台不支持时如何 fallback。只有这些问题答清，宽寄存器才是实现方式。

一、先把标量 reference 写成合同 Reference 不只是慢版本。它定义非法事件名如何报告、空输入如何返回、阈值边界是否包含、输出是否保持输入顺序、重复记录是否保留。向量版本必须先满足这份合同。若向量化后只返回块级错误，等于改变错误语义；若压缩输出改变稳定顺序，也要确认业务是否允许。性能优化没有资格偷偷修改合同。

二、局部判断批量化，跨记录状态保留标量确认 事件级别、时间戳范围、整数 key 是否在区间内，通常是逐记录独立判断，适合 SIMD 生成 mask。事件名合法性可能需要查字典、处理长度和字符编码，未必完全适合主向量路径。稳妥方案是：向量路径快速筛出候选，通过 mask 记录哪些 lane 需要慢路径确认；慢路径再用标量 reference 精确报告错误。这样普通输入快，异常输入仍保持语义。

三、压缩输出要借助 scan 思想 比较得到 mask 后，输出位置不是自然存在的。若直接按原位写，输出有空洞；若要紧凑写，需要知道当前 lane 之前有多少有效 lane。某些平台提供 compress store，其他平台要用查表或局部 prefix。无论实现如何，语义都和 scan 一样：根据前缀有效数量计算输出位置。第 7 章和第 13 章在这里连接起来，SIMD filter 不是几条比较指令，而是一套输出位置协议。

四、尾部处理优先选择清晰方案 输入长度小于向量宽度、等于向量宽度减一、刚好等于向量宽度、超过一位，这些都要测。教学实现优先使用标量尾部，哪怕慢一点，因为语义最清楚。Mask load 和 padding 都可以优化尾部，但各有前提：mask load 要平台支持，padding 要保证额外可读字节，且不能把 padding 字节当真实输入报告错误。尾部不是小角落，它经常是向量代码最容易越界的地方。

五、平台分派不能污染内核语义 同一个过滤接口可以有 scalar、auto-vectorized、AVX2、NEON 实现。分派应在外层完成，热循环只处理数据。测试先比较所有实现与 reference 的输出和错误报告，再比较性能。若某平台没有对应指令，fallback 仍必须正确。工程中的 SIMD 要维护一个语义相同、实现可替换的内核族，使程序能够跨不同 CPU 配置稳定运行。

六、向量化报告要写“没有向量化”的理由 有些循环不适合直接向量化：前缀和有跨元素依赖，哈希表更新有随机写和冲突，状态机解析有跨字节状态，浮点 reduce 改变合并顺序。报告应写出为什么没有直接 SIMD，以及是否通过算法重写把它变成可批量部分。能解释“不做”的理由，和能写出 intrinsics 一样重要。系统工程不是追求每个循环都向量化，而是把可批量语义和不可批量语义分清。

7.16 案例深化：向量化前的语义分层

SIMD 章节要避免给读者一种错觉：只要看到循环就应该写 intrinsics。更稳的入口是一个失败案例。标量解析器能准确报告错误字节位置，手写 SIMD 版本只返回“某个向量块有错误”；标量版本能处理跨块引号状态，SIMD 版本把块内字符独立分类后误判。这个案例说明向量化不是先选指令，而是先把语义分成可批量部分和需要状态传播的部分。

一、先找纯局部判断 字符是否为逗号、换行、数字、空白，通常可以逐字节独立判断。这部分适合 SIMD 生成 mask。Mask 的语义要写清：一位对应一个输入字节，1 表示候选分隔符或候选错误。只要 mask 定义清楚，后续可以用不同指令集实现同一语义。这样 portable reference、自动向量化和手写 SIMD 之间有共同合同。

二、再处理跨字节状态 CSV 引号、转义、UTF-8 多字节边界、行内状态都不是纯局部判断。可以先用 SIMD 找候选位置，再用标量状态机确认；也可以为每个块计算 carry in 和 carry out，让状态跨块传播。无论哪种方案，都要说明块边界如何处理。很多 SIMD bug 都出在尾部和跨块状态，而不是主循环。

三、尾部策略是语义的一部分 输入长度不是 lane 倍数时，尾部可以走标量 fallback，也可以用 mask load，也可以读到安全 padding。三种策略性能不同，安全条件也不同。Padding 需要分配器保证额外可读字节；mask load 需要目标平台支持；标量 fallback 最清楚但有分支。教材示例应先用标量尾部保证正确，再说明何时值得优化。

四、dispatch 不应污染热路径 多平台代码常根据 CPU 特性选择 AVX2、NEON 或标量版本。Dispatch 应在外层完成，不要每个小块都判断一次。函数指针、模板实例或策略对象都可以使用，但要让热循环只处理数据。报告要分开测 dispatch 成本和内核成本，小输入场景下 dispatch 可能主导时间。

五、验证要覆盖语义边界 测试集合要覆盖空输入、短输入、刚好一 lane、跨 lane 的引号、错误字节在尾部、非对齐地址和随机数据。性能集合则要分短行、长行、规则字段、随机字段。若只拿大块规则输入，SIMD 很容易显得完美。高质量向量化教材应让读者先害怕边界，再相信速度。

7.17 案例复盘：向量化先证明批量语义

手写 SIMD 主循环很快，但尾部错、错误位置错、跨块引号状态错，这类事故比编译错误更危险。向量化前要把语义分层：哪些判断逐字节独立，哪些判断需要前文状态，哪些边界必须回到标量确认。

mask 合同先于指令 逗号、换行、数字、空白可以批量生成 mask；引号内外、转义和 UTF-8 边界需要 carry 或标量确认。Mask 的每一位对应哪个输入字节，错误位置如何还原，尾部如何处理，都要成为合同。指令集只是这个合同的实现。

实验判据 测试覆盖空输入、短输入、lane 减一、lane、lane 加一、非对齐、错误在尾部和跨块状态。性能报告分 dispatch 成本和内核成本。小输入慢不说明 SIMD 错，大输入快也不证明语义完整。

7.18 本章习题

习题与作业

习题一：选择三个循环：数组加法、前缀和、直方图。分别写出它们的迭代依赖、写入位置和向量化难点。说明哪个适合直接向量化，哪个需要算法重写，哪个需要分阶段处理。
习题二：为 `add_arrays_scalar` 设计 debug 版别名检查。说明哪些重叠是安全的，哪些重叠会破坏语义。不要依赖非标准 `restrict` 作为唯一方案。

习题三：运行或设计一次自动向量化报告实验。记录编译器命令、报告内容、是否发生 vectorization、失败原因和汇编证据。若当前环境不支持某个报告选项，写出替代观察方式。

习题四：为浮点归约写误差说明。比较串行求和、四累加器求和、pairwise 求和和 fast-math 下的可能差异。说明你的业务场景是否允许改变合并顺序。

第 8 章

典型计算内核模式

先看一个很容易被忽略的事故。团队要统计日志中最活跃的一百个用户，于是把输入切成多个 chunk，每个 worker 在自己的 chunk 中维护局部 top-k，最后只合并这些局部候选。小数据测试通过，线上却漏掉了一个总量很高的用户。这个用户在每个 chunk 中都排在第一百名之外，所以局部 top-k 阶段把它丢掉了；全局合并阶段再也没有机会看见它。

这个错误不是某个堆操作写错了，而是内核组合的合同错了。局部 top-k 丢弃了后续精确全局 top-k 仍然需要的信息。单个内核在局部语义下正确，不代表它放进流水线后仍然正确。本章要训练的正是这种判断力：看到一个计算任务时，先问输入和输出的关系、输出位置如何确定、是否有共享写、数据访问是否规律、是否允许丢弃信息，然后再选择 map、reduce、scan、filter、partition、histogram、join、sort、top-k 或 graph frontier。

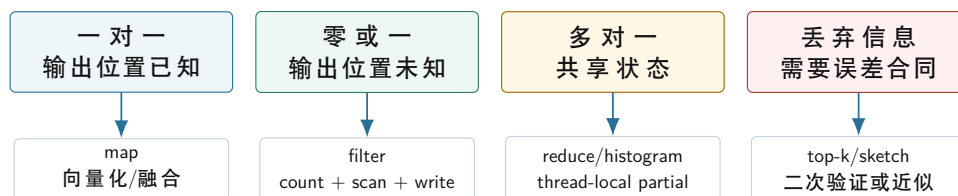
这里的 **内核** (kernel) 指一段高频、边界清楚、可以独立测量的计算片段。它不一定是操作系统内核，也不一定是 GPU kernel；在本册中，解析一批日志字段、过滤有效记录、统计事件类型、合并 partial、生成 top-k 候选，都可以是计算内核。每个内核必须有 **reference**（朴素但可信的对照实现），用来固定语义；优化版本先和 reference 对齐，再谈吞吐。

学习目标

学完本章后，读者应能完成六件事：

1. 把一个计算任务拆成 map、filter、scan、partition、histogram、join、sort、top-k 或 graph frontier 等内核，并说明拆分边界。
2. 为每个内核写出输入、输出、顺序、精确性、共享写、信息丢弃和可重试性的合同。
3. 为 reference、优化版和组合流水线分别设计 correctness oracle，而不是只看吞吐数字。
4. 根据输出位置是否确定、是否有共享写、key 是否倾斜和是否材料化，选择合适的数据结构与并行策略。
5. 构造能击穿错误实现的压力输入，例如低选择率、高选择率、热点 key、分散强 key、join 输出放大和 frontier 长尾。
6. 写出包含规模账本、阶段报告、checksum、结论边界和下一步实验的内核性能报告。

内核选择的第一张图：输出形状决定并行边界



先判断输出形状，再决定是否能直接并行；否则优化会把语义错误藏进更快的代码。

图示：本图要证明的命题是：内核优化的第一步不是选择线程数，而是确认输出位置、共享状态和信息保留合同。

工程案例

本章贯穿案例是事件流分析管线。输入是一批带有 `event_time`、`ingest_time`、`session_id`、`event_type` 和 `object_id` 的记录；输出包括有效记录、事件类型直方图、按 key 分区的 partial、维表 join 结果和热门 key。这个案例故意包含规则扫描、输出未知、共享写、key 倾斜和候选丢弃，方便把每一种内核模式都落到同一个工程里。

本章的读法 本章有两条线。浅线是识别常见内核：map、filter、reduce、scan、histogram、join、top-k 和 graph frontier。读者第一次阅读时，先抓住每种内核的输入输出关系，不必急着背所有实现策略。深线是训练工程判断：为什么同样叫 filter，有的可以原地写，有的必须 scan；为什么同样叫 top-k，有的可以局部候选，有的会漏掉全局强 key；为什么同样叫 histogram，小范围整数 key 和稀疏字符串 key 完全不是一个系统问题。

学习本章时，每看到一个内核，都要问四个问题。第一，输出位置在处理当前元素时是否已经知道。第二，多个输入是否会写同一份状态。第三，内核是否改变顺序、丢弃信息或放大输出。第四，后续 stage 需要哪些身份信息才能复查、重试或恢复。能稳定回答这四个问题，才算真正读懂“典型内核模式”。

深入理解

大师级判断力体现在负空间里：知道什么时候不该并行、什么时候不该融合、什么时候不该提前丢候选、什么时候不该把中间结果藏起来。本章后面的每个模式都不只讲“怎样做”，也讲“什么条件下这样做会错”。

8.1 先写内核合同，而不是先写并行代码

典型计算内核不能写成算法名清单。一个名字背后可能有完全不同的系统问题。Histogram 可以是 256 个小桶，也可以是亿级用户 ID 的稀疏聚合；filter 可以只输出计数，也可以要求稳定保留原始顺序；join 可以是一张表查大表，也可以是两张大表产生巨大输出。若不写清输入形状和输出语义，任何“最佳实现”都没有意义。

合同（contract）是内核对上下游作出的承诺。它至少回答九个问题：输入字段是什么，输出字段是什么，是否保持顺序，输出位置是否天然确定，是否丢弃信息，是否精确，是否可重试，是否有外部副作用，是否需要稳定的 tie-break 或随机种子。合同越早写清，后面的并发、I/O 和分布式章节越容易复用这一段设计。

Listing 8.1 KernelContract 的教学形态

```
1 enum class KernelOutputShape {
2     OneToOne,
3     ZeroOrOne,
4     ManyToOne,
5     OneToMany,
6     Reordered,
7     Approximate
8 };
9
10 struct KernelContract {
11     KernelOutputShape output_shape = KernelOutputShape::OneToOne;
12     bool preserves_input_order = true;
13     bool writes_shared_state = false;
14     bool may_drop_information = false;
15     bool requires_stable_tiebreak = false;
16     bool materializes_output = false;
17 };
```


这段结构不是要求读者立刻实现框架，而是把思考顺序固定下来。Map 多数是一对一输出；filter 是一对零或一；reduce 是多对一；join 可能一对多；sort 和 partition 会重排；sketch 和近似 heavy hitter 可能丢弃信息。合同让这些差别变成代码边界，而不是藏在函数名里。

核心思想

分析计算内核时，先问四件事：每个输入产生多少输出，输出位置是否已知，多个输入是否会写同一状态，内核是否会丢弃后续仍需要的信息。

内核形态	主要合同	常见错误	第一组证据
Map	一对一输出，位置确定，通常保持顺序	忽略输入输出重叠；融合后错误路径混进热循环	输出 checksum、反汇编、字节流量
Filter	一对零或一，输出位置由前缀计数决定	并行 <code>push_back</code> 争用；原子下标破坏稳定顺序	选择率矩阵、输出顺序校验、临时字节
Reduce	多对一，合并操作要有语义边界	全局原子热写；浮点合并树未声明	partial 数量、合并树、误差或整数溢出检查
Histogram	多对多桶，key 分布决定热点	小桶全局共享；稀疏 key 误用巨大数组	最大 bucket、key 分布、合并耗时
Partition	多路输出，manifest 描述输出身份	只写数据不写 bucket 边界；顺序假设丢失	bucket range、block checksum、路由函数
Join	输出可能放大，构建侧要受控	低估重复 key；把输出放大归因给实现慢	左右 key 频次、输出上界、热点 key 列表
Top-k	精确或近似必须声明	局部 top-k 提前丢掉全局强 key	二次验证、tie-break、候选覆盖率

Reference 和 oracle 不是同一件事 reference（参考实现）是一段尽量简单、可信、可读的程序；oracle（判定器）是比较参考结果和待测结果的规则。二者经常被混在一起，但在高性能内核中必须分开。Reference 可以生成完整输出，oracle 可以只检查 checksum、计数、不变量或误差界。对浮点归约，oracle 不应要求逐 bit 一致，而应写清误差合同；对 top-k，oracle 必须检查 tie-break 和候选覆盖；对 partition，oracle 不仅检查元素集合，还要检查每个 bucket 的边界和路由函数。

Listing 8.2 KernelOracle 的教学形态

```

1 struct KernelOracle {
2     bool requires_exact_order = true;
3     bool allows_floating_error = false;
4     double absolute_error = 0.0;
5     double relative_error = 0.0;
6     std::uint64_t expected_records = 0;
7     std::uint64_t expected_checksum = 0;
8 };

```

这个结构也不是要成为通用测试框架，而是提醒读者：不同内核的“正确”含义不同。Stable filter 要求输出顺序和 reference 一致；非稳定 filter 可以只要求输出集合一致；histogram 要求每个 key 的计数一致；top-k 要求顺序、分数和 tie-break 一致；近似 heavy hitter 则必须写明误差上界、漏报概率或二次验证策略。

错误输入要比随机输入更重要 随机输入能发现一部分错误，却很少正好击中合同边界。教材中的输入生成器要专门制造坏情况：空输入、单元素、最后块不满、全部通过、全部过滤、热点 key、全唯一 key、全相同 key、tie 分数、join 输出放大、图中超级节点、浮点 NaN 和整数溢出边界。高质量测试不是“规模很大”，而是“刚好攻击实现的假设”。

Listing 8.3 压力输入族的接口形态

```

1 enum class InputFamily {
2     Uniform,
3     AllPass,
4     AllReject,
5     SingleHotKey,
6     ZipfSkew,
7     AllUniqueKeys,
8     DistributedStrongKey,
9     JoinAmplification,
10    FrontierLongTail
11 };

```

这个枚举把实验从“跑一个随机数组”提升为“覆盖语义风险”。每次报告都要写明使用了哪些输入族。若某个优化只在 **Uniform** 上快，却在 **SingleHotKey** 上退化或错误，它不是通用结论，只是特定输入分布下的结果。

材料化、流水线和 pipeline breaker **材料化** (materialization) 是把中间结果写成数组、文件或其他可复查对象。它增加内存或 I/O 成本，却换来调试、复用、恢复和阶段测量。**流水线** (pipeline) 则让上游输出直接进入下游，减少中间写入，但背压、生命周期和错误传播更复杂。

pipeline breaker (会打断流水线的阶段) 是必须等待一批数据、重排数据、聚合状态或持久化边界后才能继续的内核。Sort、global reduce、join build、shuffle、checkpoint 常常是 breaker；简单 map 和普通 filter 更容易流水线。识别 breaker 不是运行时的附加工作，而是内核合同的一部分。

manifest 让中间结果可复查 当 partition 或 shuffle 生成多个输出块时，只返回一个 **std::vector** 不够。系统还需要知道每个块属于哪个 bucket、包含多少记录、写到哪里、校验和是什么、是否已经提交。**manifest** (清单) 就是描述这些输出对象身份和边界的记录。单机 partition 的 manifest 到第 18 章会自然变成分布式 shuffle block 的 manifest。

8.2 一对一和多对一：Map 与 Reduce

Map 是最容易理解的内核：第 **i** 个输入通常产生第 **i** 个输出。它没有跨元素依赖，适合顺序扫描、SIMD 和线程切分。可是 map 也最容易被低估。若每个元素只做一次乘法和一次加法，它很快会受内存带宽限制；若每个元素调用复杂函数、访问冷字段或分配字符串，瓶颈又会转移到前端、分支或分配器。

Listing 8.4 一对一 map 的 reference

```

1 void scale_and_shift(std::span<const float> input,
2                     std::span<float> output,
3                     float scale,
4                     float shift) {
5     const std::size_t count = std::min(input.size(), output.size());
6     for (std::size_t index = 0; index < count; ++index) {
7         output[index] = input[index] * scale + shift;
8     }
9 }

```

这个 reference 的正确性来自输出位置合同：每个输出位置只依赖同一位置输入。优化时可以切块、向量化或融合相邻 map，但这些优化不能改变位置关系。若输入输出区间可能破坏性重叠，还要把别名语义写进接口；否则编译器和维护者都无法确定循环能否安全重排。

为什么一对一内核适合优化 一对一内核最容易优化，是因为它同时满足三个条件：读写地址可以从索引直接算出，迭代之间没有语义依赖，输出规模等于输入规模。CPU 喜欢这种形状，因为

硬件预取器容易看到线性访问，编译器容易展开或向量化，多个线程也容易按连续区间切分。它的难点通常不在算法，而在字节流量、别名、尾部、错误路径和内存带宽。

这也是为什么一对一内核很容易给人错觉。代码看起来只有一行表达式，读者会以为它一定是 compute-bound；实际却可能每个元素只做两次浮点运算，却读写十几个字节，很快受内存带宽限制。优化这种内核时，先估算每个元素读多少字节、写多少字节、做多少运算，再判断融合、SIMD 或线程是否有意义。若瓶颈已经是内存带宽，把线程数翻倍可能只会让多个核心争同一条内存通道。

Map 融合减少字节，不是为了让代码更复杂 连续三个 map 若每一步都写出中间数组，会产生多轮内存读写。融合可以把这些逐元素变换放进一次循环，减少中间材料化。可是融合后循环体更大，寄存器压力更高，错误路径也可能混进热循环。融合是否值得，要比较未融合耗时、融合耗时、中间数据大小、错误报告是否变难，而不是凭直觉判断。

日志项目中，timestamp 归一化、event type 编码、用户 ID 映射都像 map。若解析器输出对象数组，map 会拖着冷字段一起走；若解析器输出列式 **HotBatch**，map 可以只扫热字段。第 6 章的数据布局因此不是孤立优化，而是 map 内核能否真正顺序扫描的前提。

心智模型

Map 的大师级问题不是“能不能写成一循环”，而是“这一行循环每处理一个元素让机器移动了多少字节，产生了多少临时状态，是否把冷数据拖进热路径”。

Reduce 的第一问题是合并语义 Reduce 把多个输入合成一个输出。它看起来像“加起来”，但真正的问题是合并操作的语义。整数最大值有清楚的单位元和结合律；浮点求和不严格结合，合并树会影响舍入；字符串拼接可能满足语义结合，却被分配成本支配；错误列表合并可能要求稳定顺序。没有合并语义，就没有正确的并行 reduce。

并行 reduce 的反例，是所有线程直接更新一个共享结果。下面的代码可能得到正确计数，但每个命中元素都要修改同一条 cache line，扩展性会很差。

Listing 8.5 反例：高频共享写的 reduce

```
1 void count_positive_bad(std::span<const std::int32_t> values,
2                         std::atomic<std::int64_t>& total) {
3     for (const std::int32_t value : values) {
4         if (value > 0) {
5             total.fetch_add(1, std::memory_order_relaxed);
6         }
7     }
8 }
```

memory_order_relaxed 只放松了顺序约束，并没有让共享写消失。更可靠的起点是每个 worker 写自己的 partial，最后合并。

Listing 8.6 线程本地 partial，把高频共享写移出热路径

```
1 constexpr std::size_t CACHE_LINE_BYTES = 64;
2
3 struct alignas(CACHE_LINE_BYTES) CountPartial {
4     std::int64_t positive = 0;
5 };
6
7 void count_positive_range(std::span<const std::int32_t> values,
8                           CountPartial& partial) {
9     std::int64_t local_count = 0;
10    for (const std::int32_t value : values) {
11        if (value > 0) {
12            ++local_count;
```



```

13     }
14 }
15 partial.positive = local_count;
16 }

```

这个版本的原则比代码更重要：热路径写寄存器和本地 `partial`，不写全局共享对象。它把问题从“每个元素一次同步”变成“每个 worker 一次合并”。当 `partial` 很大时，合并本身也要继续设计，可能需要树形合并、按 NUMA 节点合并或按 bucket 并行合并。

Reduce 的深水区：合并树就是语义 并行 reduce 必须选择合并树。整数计数通常允许任意合并树，只要不溢出；浮点求和不同，左折叠、pairwise sum、多线程树形合并和 SIMD 水平合并都可能得到不同低位结果。若业务要求 bitwise reproducibility，就要固定合并树，甚至牺牲一部分性能；若业务允许误差，就要写明绝对误差、相对误差或 ULP 边界。

合并树还影响性能。扁平合并让一个线程扫描所有 `partial`，简单但可能成为瓶颈；树形合并能并行推进，但需要更多同步和中间状态；按 NUMA 节点先本地合并可以减少跨节点流量；按 bucket 合并可以让热点桶和普通桶分开处理。Reduce 的“高性能”不是一句 `parallel_for`，而是把语义合并树和硬件拓扑一起设计。

8.3 输出位置未知：Scan、Filter 与 Partition

Filter 选择满足谓词的元素。串行版本可以直接 `push_back`，因为当前输出位置由前面已经保留了多少元素决定。并行后，这个隐含状态变成共享资源。多个线程同时 `push_back` 会争用；使用原子下标可以分配位置，却可能破坏稳定顺序；加锁保持顺序又回到瓶颈。

稳定 filter 的推导路线是三阶段。第一，每个块统计自己会保留多少元素。第二，对块计数做 scan，得到每个块的全局输出起点。第三，每个块再次扫描输入，写入自己的连续输出区间。Scan 在这里不是抽象算法题，而是输出位置分配器。

为什么串行 `push_back` 到并行会变难 串行 filter 中，`push_back` 隐含维护了一个变量：已经输出了多少个元素。每遇到一个保留元素，这个变量加一，并决定新元素写到哪里。单线程时这个状态简单；多线程时，所有 worker 都想修改同一个“下一个输出位置”。如果用锁保护它，热路径每个命中元素都要同步；如果用原子 `fetch_add`，位置唯一但顺序变成到达顺序，不再是输入顺序；如果每个线程写本地 vector，最后拼接，仍然需要知道每个本地 vector 放到全局输出的哪个区间。

Scan 的作用就是把“运行时抢位置”改成“写出前先算位置”。每个块的保留数量是局部事实，块计数的前缀和给出全局起点。写出阶段不需要锁，也不需要全局原子，因为每个 worker 已经拿到自己的独占区间。这个推导是并行算法里非常重要的套路：先把未知输出位置转成计数问题，再用前缀和分配位置。

Listing 8.7 过滤前先统计每个块的输出数量

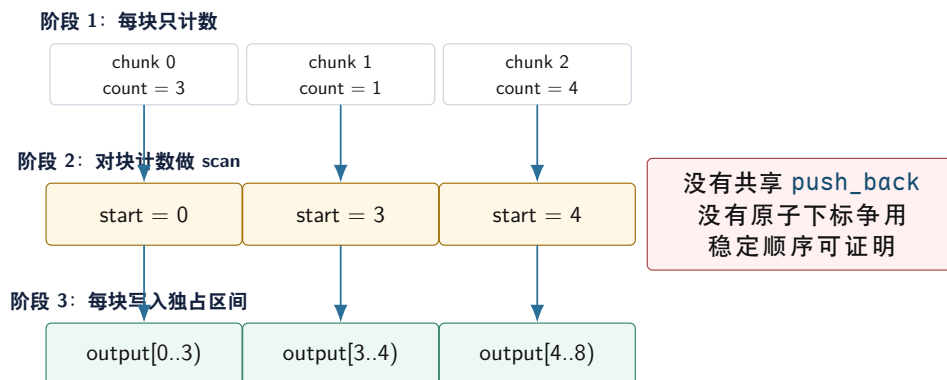
```

1 std::int64_t count_selected(std::span<const std::int32_t> values,
2                             std::int32_t threshold) {
3     std::int64_t selected_count = 0;
4     for (const std::int32_t value : values) {
5         if (value >= threshold) {
6             ++selected_count;
7         }
8     }
9     return selected_count;
10 }

```

这个函数只统计，不写输出，因此没有共享写。对所有块的 `selected_count` 做前缀和，就能得到每块的输出起点。后续写出时，每个 worker 都只写自己负责的区间。它多读了一遍输入，却消除了共享 push、动态扩容和顺序混乱。

Stable filter 的三阶段结构



图示：本图要证明的命题是：stable filter 的并行化核心不是并发容器，而是先用 scan 把输出位置变成每个 worker 的独占区间。

选择率决定 filter 的成本模型 Filter 的性能高度依赖选择率。选择率为 0 时，输出很小，但仍要读完整输入；选择率为 100% 时，filter 接近 copy；选择率在中间且随机时，分支和输出位置都复杂。低选择率可能适合 selection vector；高选择率可能适合 bitmap 或直接保留原数组加 mask；记录很大时，先保存 record id 而不是复制整条记录可能更好。

测试 filter 不能只测 50% 随机选择。至少要覆盖全不通过、全通过、低选择率、高选择率、长记录、错误记录和最后一块不满。否则实现很容易只对某个友好输入成立。

选择向量、bitmap 和材料化输出 Filter 不一定要立刻复制整条记录。若记录很大，直接复制有效记录会产生大量内存流量；另一种做法是输出 selection vector，只保存通过记录的下标；也可以输出 bitmap，每个 bit 表示一条记录是否通过。Selection vector 适合低选择率和后续随机访问可接受的场景；bitmap 适合需要快速 membership test 或保留原数组的场景；材料化输出适合后续要顺序扫描有效记录的场景。

这个选择会影响后续所有 stage。Histogram 如果只需要 event_type，可以用 selection vector 扫热字段列；Join 如果需要整条记录，可能迟早要材料化；错误报告如果需要原始 record id，就不能在 filter 时丢掉身份。Filter 的输出格式不是局部实现细节，而是管线合同。

Partition 是多路输出位置分配 Partition 把元素按 bucket 或 key 放到不同区域。二路 partition 是 filter 的近亲，多路 partition 则是 shuffle 的前置形态。可靠做法仍然是先统计每个 worker 到每个 bucket 的数量，再做二维前缀和，最后让每个 worker 写入每个 bucket 中自己的区间。

分区不只是为了并行写输出，也可以为了改变后续访问形态。把记录按 event type 分桶，后续每个桶里的分支更稳定；把记录按 user id 分区，后续 group-by 的工作集更小；把记录按 reduce target 写成 block，分布式阶段就可以直接传输。分区的成本是额外扫描、输出写入和可能打乱顺序；收益是减少共享写、控制工作集或形成 stage 边界。

二维前缀和给每个 worker 分配每个 bucket 的独占区间 多路 partition 的核心数据结构是 counts[worker][bucket]。第一遍扫描时，每个 worker 只统计自己的输入会落到哪些 bucket；第二步对这个二维计数做前缀和，得到每个 worker 在每个 bucket 中的写入起点；第三遍扫描时，worker 按起点写入输出。这样每个输出位置只由一个 worker 写，且每个 bucket 的范围可以被 manifest 描述。

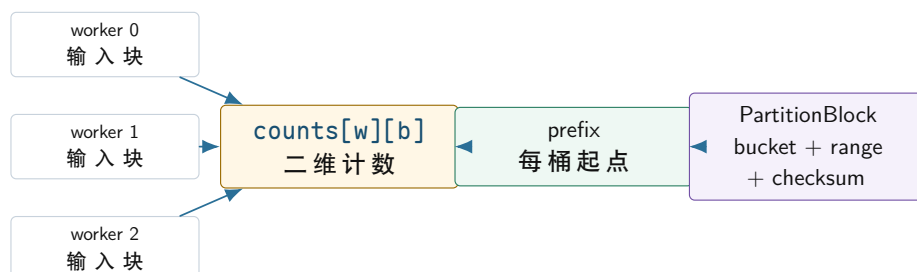
Listing 8.8 PartitionBlock 描述输出块身份

```
1 struct PartitionBlock {
2     std::int32_t bucket = 0;
3     std::uint64_t begin = 0;
4     std::uint64_t end = 0;
5     std::uint64_t records = 0;
6     std::uint64_t checksum = 0;
```

```
7 };
```

PartitionBlock 的价值不只是为了调试。后续分布式 shuffle 要把 block 发送给不同 reducer；checkpoint 要记录哪些 block 已经提交；重试要判断重复发送是否安全；背压要知道哪个 bucket 正在变成热点。若 partition 只返回一段数组而没有 block 身份，后续系统很难恢复和复查。

Partition 从本地分桶自然变成 shuffle manifest



本地 partition 只是在内存中重排记录；一旦 block 带有 bucket、范围和 checksum，它就具备了成为分布式 shuffle 单元的身份。

图示：本图要证明的命题是：partition manifest 是单机高性能内核和分布式 shuffle 之间的接口，不是附加日志。

稳定分区和非稳定分区 Partition 是否保持原始相对顺序，是合同的一部分。稳定分区让同一 bucket 内的记录按输入顺序保留，便于调试和复现，但通常需要更明确的计数和写出顺序。非稳定分区可以更自由地交换元素，适合某些原地算法，但后续 stage 不能假设记录顺序。事件流里若错误报告要保留原始 record id，稳定性和身份字段都很重要；若只是做无序 group-by，非稳定分区可能足够。

分区函数必须可复现 分区函数不能依赖进程随机盐、地址、非确定性时间或不稳定哈希，除非 manifest 记录了足够信息让重试时复现同样路由。分布式阶段尤其如此：同一条记录在重试时必须落到同一 reducer，否则幂等提交和 checkpoint 会失效。即使在单机阶段，也应把 hash seed、bucket 数和路由版本写进报告。

8.4 共享写和数据倾斜：Histogram 与 Top-k

Histogram 是共享写问题的典型训练场。它的计算很短：读 key，找到 bucket，计数加一。系统成本却取决于 bucket 数量、key 分布、线程数和内存布局。桶很少时，全局原子会形成热点；桶很多时，每线程完整桶数组可能占用大量内存；key 极度倾斜时，一个热点 bucket 会支配合并阶段。

Listing 8.9 稠密 key 的本地直方图

```
1 constexpr std::int32_t EVENT_BUCKET_COUNT = 1024;
2
3 struct LocalHistogram {
4     std::array<std::int64_t, EVENT_BUCKET_COUNT> buckets{};
5 };
6
7 void histogram_dense(std::span<const std::int32_t> event_types,
8                     LocalHistogram& histogram) {
9     for (const std::int32_t event_type : event_types) {
10         if (0 <= event_type && event_type < EVENT_BUCKET_COUNT) {
11             ++histogram.buckets[static_cast<std::size_t>(event_type)];
12         }
13     }
14 }
```


这个版本速度可能很高，因为 bucket 连续、无哈希、无共享写。但它要求 key 是小范围整数。若 key 是 64 位用户 ID，直接分配巨大数组不现实；若 key 是字符串，哈希、比较、分配和字典编码都会进入成本。工程中常见的路线是：小范围 key 用数组桶，稀疏 key 用本地哈希表或排序聚合，热点 key 用采样、隔离或二级聚合。

哈希聚合和排序聚合没有固定胜负 Group-by 聚合通常有两条路线。哈希聚合边读边更新表，平均复杂度好，适合无序输入和点更新；排序聚合先按 key 排序，再顺序扫描相同 key，适合需要有序输出、key 可比较、内存局部性重要或哈希冲突严重的场景。排序聚合多了排序成本，却可能减少随机访问和分配。

选择时要看 key 数量、输入大小、输出是否需要排序、内存预算、倾斜程度和是否已有顺序。若输入已经按 key 近似有序，排序聚合可能很有吸引力；若 key 几乎全唯一，局部 combine 的收益很小；若 key 先做字典编码能变成稠密整数，数组桶可能比通用哈希表更合适。

本地 partial 不是免费午餐 线程本地桶把热路径共享写移走，但它会增加内存和合并成本。假设有 worker_count 个 worker、bucket_count 个桶、每个桶 8 字节，partial 内存大约是 worker_count*bucket_count*8 字节。若桶数是 1024、worker 是 16，这很小；若桶数是一千万，本地完整桶数组会直接不可接受。此时要改用稀疏本地 map、排序聚合、两级分桶或先做字典编码。

合并也可能成为瓶颈。若每个 worker 都有完整桶数组，最后要扫描 worker_count*bucket_count 个计数，即使很多桶为 0。稀疏结构减少空桶扫描，却引入哈希、分配和随机访问。排序聚合让相同 key 连续，合并更顺序，但要付排序成本。读者需要把热路径成本、partial 内存、合并成本和输出需求一起比较。

策略	适用条件	主要风险
全局原子桶	桶多、冲突低、实现简单优先	热点 key 下 cache line 抖动严重
线程本地稠密桶	key 小范围、桶数可控、合并可接受	桶数大时内存和合并扫描爆炸
线程本地稀疏 map	key 稀疏、每 worker 命中 key 少	哈希和分配成本高，合并复杂
排序聚合	需要有序输出、随机访问昂贵、输入近似有序	排序是 pipeline breaker，临时空间大
两级聚合	key 倾斜明显、热点需要隔离	采样和分桶策略错误会制造新热点

热点 key 要单独观察 平均吞吐会掩盖热点。一个 key 占 40% 输入时，全局原子桶会集中争用；本地桶热路径不争用，但合并时这个 key 的所有 partial 都要汇总；分布式阶段这个 key 可能把单个 reducer 拖成长尾。报告至少要记录最大 bucket、前十个 bucket、最大 bucket 占比和合并阶段耗时。没有这些字段，读者无法区分“算法慢”和“数据倾斜”。

Top-k 不能随便提前丢信息 Top-k 的目标是减少数据移动：只保留最有可能进入最终结果的候选，而不是全量排序。每个 worker 维护局部小堆，最后合并候选，是常见结构。但它只有在合同允许时才正确。若局部阶段按 chunk 丢弃了全局精确结果仍然需要的 key，就会重演本章开头的事。

精确全局 top-k 通常要先完成按 key 的完整聚合，再从全局计数中选择 top-k。若 key 基数太大，可以使用近似 heavy hitter 或更大的局部候选集，但必须写清误差合同：可能漏报什么，是否做二次精确验证，输出是否标记为近似。监控趋势可以接受近似，计费 and 审计通常不能。

精确 top-k 的最小正确路线 精确 top-k 的安全路线是两步：先得到每个 key 的全局精确 score，再从全局 score 集合中选前 k。第一步可以用本地 hash aggregate、排序聚合或分区后聚合；第二步可以用大小为 k 的堆、选择算法或排序。无论第二步多快，只要第一步丢了 key，最终结果就无法保证精确。很多线上事故正是把“减少候选”误当成“减少中间数据但不改变语义”。

局部 top-k 并非永远错误。若每个 chunk 的划分和业务语义保证全局强 key 必定在某个 chunk 里也强，局部候选可以安全；若只是随机切块或按时间切块，就没有这个保证。另一种安全做法是局部阶段只做预筛选，后面必须对候选覆盖之外的风险做二次验证，例如使用更大的候选集、sketch 估计误差上界，或者重新扫描原始数据验证临界 key。

近似 heavy hitter 要诚实 近似 heavy hitter 算法可以用较小内存发现高频 key，但它输出的是带误差合同的结果，不是精确 top-k。报告必须说明算法可能产生 false positive、是否可能 false negative、误差上界如何计算、是否需要二次精确计数。监控报警、推荐系统候选召回、流量趋势分析常能接受近似；账单、审计、权限、安全策略通常不能接受无标记近似。

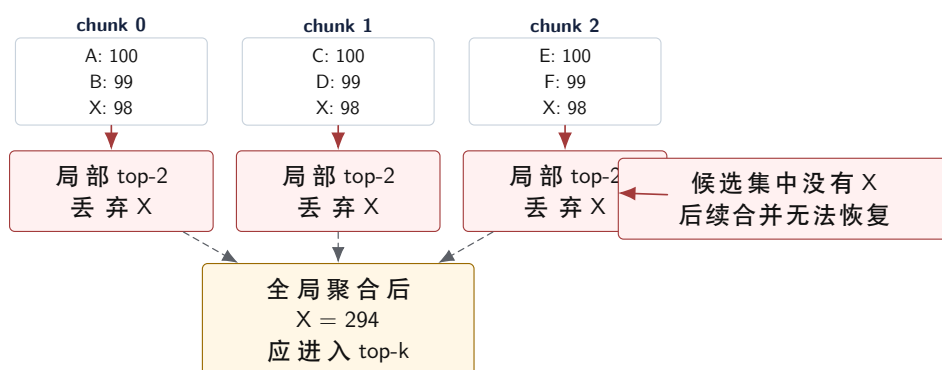
工程上常见的折中是“近似召回 + 精确验证”：第一阶段用 sketch 或局部大候选集召回可能的 heavy hitters；第二阶段只对这些候选做精确计数；第三阶段输出带 tie-break 的精确 top-k。这个方案仍有前提：召回阶段必须有足够高的覆盖保证，否则会漏掉真实 top-k。教材中的 top-k 实验要故意构造分散强 key，迫使读者证明候选覆盖。

Listing 8.10 确定性 top-k 候选的接口形态

```
1 struct TopCandidate {
2     std::int32_t key = 0;
3     std::int64_t score = 0;
4 };
5
6 bool comes_before(const TopCandidate& left,
7                   const TopCandidate& right) {
8     if (left.score != right.score) {
9         return left.score > right.score;
10    }
11    return left.key < right.key;
12 }
```

Tie-break 必须成为接口语义。若两个 key 分数相同，输出按 key 升序还是任意？不确定顺序会让测试、缓存和分布式重试难以复现。一个高质量 top-k 内核既要快，也要把稳定性写进合同。

局部 top-k 反例：全局强 key 可以在每个 chunk 中被丢弃



图示：本图要证明的命题是：局部候选阶段一旦丢掉精确全局 top-k 仍需要的信息，后续合并阶段没有任何算法能恢复这个 key。

8.5 数据形状改变策略：Sort、Join、Stencil 与 Graph Frontier

Sort 不只是算法题，也是系统工具。排序可以把相同 key 聚到连续区间，便于聚合、join、压缩和确定性输出。外部排序先生成有序 run，再多路归并；分布式排序先采样选择分区边界，再 shuffle 到 reducer。排序常常是 pipeline breaker，但它换来顺序访问、稳定输出和更容易复查的阶段边界。

Join 的成本在输出规模和构建侧 Join 最容易低估的是输出放大。两个输入各一百万行，若某个 key 在左表出现一千次，在右表也出现一千次，该 key 的 join 输出就是一百万行。即使输入不大，输出也可能直接把内存打爆。Join 内核必须估计 key 分布、匹配度、输出上界和背压策略。

Hash join 通常把较小表构建哈希表，再扫描较大表 probe；sort-merge join 先按 key 排序，再线性合并；nested loop 只适合极小输入或有强索引过滤的场景。选择不是固定的。若小表 key 极端倾斜，哈希 bucket 会很长；若大表过滤后很小，先 filter 再 join 更好；若最终输出需要按 key 有序，sort-merge join 可能减少后续成本。

先估算 join 输出，再选择实现 Join 输出规模由每个 key 在左右两侧的出现次数相乘决定。若 `left_count[k]` 是 key 在左表出现次数，`right_count[k]` 是右表出现次数，那么该 key 的输出数是 `left_count[k]*right_count[k]`，总输出是所有 key 的乘积求和。这个公式比“左表一百万、右表一百万”更有解释力，因为真正危险的是重复 key 和热点 key。

Listing 8.11 Join 输出上界估算的教学形态

```
1 std::uint64_t estimate_join_rows(
2     const std::unordered_map<std::int64_t, std::uint64_t>& left_counts,
3     const std::unordered_map<std::int64_t, std::uint64_t>& right_counts) {
4     std::uint64_t rows = 0;
5     for (const auto& [key, left_count] : left_counts) {
6         const auto found = right_counts.find(key);
7         if (found != right_counts.end()) {
8             rows += left_count * found->second;
9         }
10    }
11    return rows;
12 }
```

这段代码没有处理溢出，也不是生产实现；它的教学作用是让读者先面对输出规模。真实系统要处理 `rows` 溢出、内存预算、批量输出、背压和取消。若估算输出已经超过内存预算，继续写一个更快的 hash join 没有意义，应该先改变执行计划：预过滤、限制输出、分批 materialize、热点 key 单独处理或让下游具备流式消费能力。

构建侧和 probe 侧的选择 Hash join 通常选择较小的一侧作为 build side，因为哈希表状态需要常驻内存。但“小”不是只看行数，还要看 key 宽度、payload 宽度、重复 key、过滤后大小和是否已经编码。若维表很小，build/probe 清楚；若两侧都大，可能需要 partitioned hash join，先按 key 分区，再对每个分区做 build/probe；若两侧已经按 key 排序，sort-merge join 可以避免哈希表随机访问。

事件流里的维表 join 很适合训练这个判断。维表可能是 `object_id` 到类别的映射；事件表很大但只读几列；某些 object 可能极热。报告要记录 build 表大小、probe 行数、匹配率、输出行数、最大 key 放大倍数和内存峰值。若只写“hash join 更快”，说明没有真正分析 join。

Join 也是背压源 Join 输出可能比输入大得多，因此它天然可能制造背压。若下游是 top-k 或聚合，可以考虑在 join 后立即 combine，减少材料化；若下游需要完整明细，必须有有界缓冲、分批输出或落盘策略。这个问题会在第 15 章异步 I/O 和背压中继续展开。本章先让读者记住：join 不是一个孤立函数，它会改变后续 stage 的数据量和故障恢复成本。

Stencil 从邻域复用开始 Stencil 每个输出读取邻域输入，例如一维平滑、二维卷积和网格有限差分。它的关键是复用邻域数据，避免重复从内存加载。内部区域和边界区域最好分开：内部热循环没有复杂分支，边界单独处理；或者使用 padding 和 ghost cell 统一访问规则。

并行 stencil 还要处理块之间的 halo 数据。单机上，halo 是相邻块共享的边界输入；分布式上，它会变成节点之间交换的消息。这个例子说明，cache blocking 和分布式 halo exchange 是同一个思想在不同尺度上的表现。

Graph frontier 暴露不规则访问 图遍历是规则数组内核的反面。邻接表访问可能随机，顶点度数差异巨大，frontier 大小每层变化，写 visited 状态可能有竞争。CSR 把所有边放进连续数组，

能改善邻接扫描；但顶点属性访问、frontier 去重和负载均衡仍然复杂。

BFS 的 top-down 策略从当前 frontier 扩展邻居；bottom-up 策略从未访问顶点检查是否有邻居在 frontier。当前沿很小时 top-down 更好，当前沿很大时 bottom-up 可能减少边访问。方向优化的本质，是根据数据形状切换内核，而不是固定套一个并行模板。

心智模型

图遍历把本章的问题集中在一起：随机读、共享写、负载倾斜、输出去重、frontier 表示切换和任务调度。读懂图内核，后面看任务运行时的 ready set 会更自然。

Frontier 表示决定访问成本 Frontier 可以用 vector 表示，也可以用 bitmap 表示。Vector frontier 只存当前活跃顶点，当前沿很小时扫描成本低；但 membership test 需要额外结构，去重也要处理。Bitmap frontier 用一个 bit 表示顶点是否在当前沿，membership test 快，适合当前沿很大时做 bottom-up 检查；但扫描整个位图可能浪费。表示不是美学选择，而是由 frontier 密度、membership 查询次数、去重成本和缓存局部性决定。

表示	优点	风险
Vector frontier	稀疏当前沿扫描少，容易按顶点列表切分	超级节点造成负载长尾，去重和 membership 额外复杂
Bitmap frontier	membership 快，适合 dense frontier 和 bottom-up	稀疏时扫描浪费，更新 bit 可能有共享写
Queue frontier	适合动态发现和流式扩展	顺序和去重难控制，队列争用明显
分桶 frontier	可按度数、社区或分区组织负载	预处理复杂，错误分桶会制造倾斜

图实验的指标不能只看总时间 图遍历要记录每层 frontier 大小、边访问数、重复发现次数、最大顶点度数、worker 空闲时间和同步次数。总时间只能告诉你慢，不能告诉你为什么慢。若某层 frontier 很小但有一个超级节点，静态按顶点数切分会负载不均；若 frontier 很大，top-down 会访问大量边，bottom-up 可能更好；若重复发现很多，visited 更新和去重策略就是重点。

Graph frontier 还会暴露对象身份问题。一个顶点被多个 worker 同时发现时，谁负责提交？若只需要 visited 集合，原子 bit 或 CAS 可能足够；若需要记录父节点用于最短路径树，就必须定义 tie-break，例如最小父 id 或最早层次。没有 tie-break，结果可能正确地“到达”了顶点，却不能复现同一棵树。

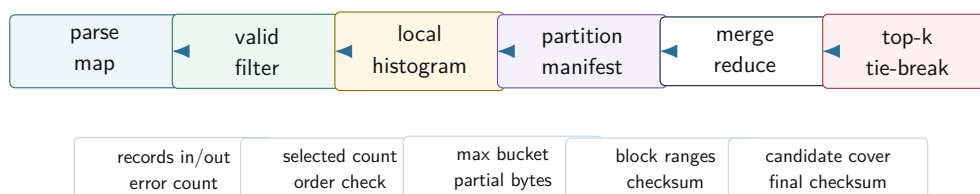
图内核实验：实现 CSR 图上的 BFS reference、vector frontier 版本和 bitmap frontier 版本。输入至少包括链式图、二维网格、随机图和幂律图。报告每层 frontier 大小、边访问数、重复发现次数、最大 worker 耗时和输出 checksum，并解释何时 vector 更好、何时 bitmap 更好。

8.6 把内核组合成可测流水线

真实系统很少只运行一个内核。日志统计可以拆成一条流水线：parse map 把原始文本变成字段列；valid filter 生成选择向量和错误列表；event histogram 对有效记录做本地计数；partition 把 partial 或记录按 reduce target 分桶；merge reduce 合并所有 worker 的 partial；top-k 输出热门事件。每一步都有独立 reference，也有组合后的不变量。

组合后的验收比单内核更严格。Filter 后记录数加被过滤数应等于输入数；histogram 所有桶计数总和应等于 filter 输出数；partition 每个 key 的目标必须和 reduce 路由一致；top-k 的候选策略不能丢失精确结果需要的 key；最终 checksum 要和串行 reference 对齐。若单步都正确但组合失败，通常是顺序、身份、材料化边界或信息丢弃出了问题。

事件流管线的阶段账本



图示：本图要证明的命题是：组合管线的每个 stage 都要留下规模、身份和校验证据，否则最终错误无法定位。

沿一条记录追踪 调试组合流水线时，选一条记录从输入追到输出。它是否通过 filter，进入哪个 partition，更新哪个局部桶，在 join 中匹配几条维表记录，最后是否进入 top-k 候选。沿单条记录追踪，比只看每个函数的单元测试更容易发现接口错误，例如 filter 后顺序改变导致 join 假设失效，或者局部 combine 丢失错误报告所需的 record id。

规模账本比单点耗时更重要 每个 stage 都要记录 records in、records out、bytes out、temporary bytes、state bytes、最大 bucket、join 放大倍数、错误数和 checksum。局部内核变快不代表系统变快。若 filter 选择率突然升高，join 输出会膨胀；若 partition 出现热点桶，下一阶段会长尾；若 top-k 为了保险保留太多候选，内存带宽可能被中间数据吃掉。

Listing 8.12 阶段报告的最小字段

```
1 struct StageReport {
2     std::string_view stage_name = {};
3     std::uint64_t records_in = 0;
4     std::uint64_t records_out = 0;
5     std::uint64_t temporary_bytes = 0;
6     std::uint64_t state_bytes = 0;
7     std::uint64_t checksum = 0;
8 };
```

这不是完整观测系统，却表达了第二册的实验纪律：每次优化都要能回答输入规模、输出规模、临时数据、状态大小和结果校验。没有这些字段，吞吐数字很容易误导读者。

实验对象	必测输入族	必交证据
Map 融合	小数组、大数组、冷字段对象、只扫热字段列	字节流量估计、反汇编、吞吐和 checksum
Stable filter	全通过、全不通过、1% 通过、50% 随机、最后块不满	records out、顺序校验、temporary bytes、分支行为解释
Histogram	均匀 key、单热点 key、Zipf 倾斜、稀疏 64 位 key	最大 bucket、partial 内存、合并耗时、全局原子对照
Top-k	无 tie、全 tie、分散强 key、候选数不足、二次验证	tie-break、候选覆盖率、漏报反例、精确/近似声明
Join	唯一 key、重复 key、热点 key、空匹配、输出爆炸	key 频次、输出上界、实际输出、背压策略
Graph frontier	规则图、随机图、幂律图、空 frontier、超大 frontier	每层 frontier、边访问数、重复发现数、worker 空闲时间

常见误区

本章所有性能实验都必须先通过串行 reference。若优化版只在随机输入上通过，却没有覆盖低选择率、热点 key、输出放大、分散强 key 和空输入，不能把它写成“正确实现”。

报告模板 本章报告建议固定为八段。第一段写任务合同：输入、输出、顺序、精确性、共享写、tie-break。第二段写 reference 和 oracle：如何生成正确结果，如何比较。第三段写实现路线：baseline、优化版、没有采用的方案。第四段写输入族：规模、分布、随机种子和压力输入。第五段写阶段账本：records in/out、temporary bytes、state bytes、最大 bucket、join 放大和 checksum。第六段写性能结果：重复次数、统计摘要、吞吐曲线和异常值。第七段写解释：瓶颈来自计算、内存、分支、同步、数据倾斜还是材料化。第八段写边界：哪些结论没有验证，下一步实验是什么。

Listing 8.13 阶段报告 CSV 的建议字段

```
1 stage,records_in,records_out,bytes_in,bytes_out,temporary_bytes,
2 state_bytes,max_bucket,checksum,elapsed_ns,notes
```

CSV 字段看起来朴素，却能强迫读者把“快了多少”放进上下文。若某个版本耗时下降，但 **temporary_bytes** 翻倍，后续大输入可能变慢；若 **records_out** 因 filter 选择率改变而下降，吞吐提高不一定来自内核优化；若 **max_bucket** 飙升，后续分区和 reduce 可能出现长尾。

8.7 工业界设计参照

工业系统里的优秀设计，很少是某个孤立技巧。它们通常把语义合同、数据布局、批处理、状态边界和故障恢复放在一起设计。学习这些系统时，不要只记住“某数据库是列式的”“某框架有 shuffle”“某库有 vectorized execution”。真正值得迁移的是背后的约束和取舍：为什么它要用列式 batch，为什么要保留 selection vector，为什么 sort 和 join 会成为 pipeline breaker，为什么 shuffle block 必须带 manifest，为什么 top-k、limit 和聚合必须声明确定性输出。

列式执行器：batch、selection vector 和热字段 DuckDB、ClickHouse、Velox 一类向量化执行器的共同思想，是让算子处理一批连续值，而不是一行一行解释执行。列式 batch 让同一字段连续存放，map、filter、hash 和聚合可以只扫热字段，CPU 更容易预取，SIMD 更容易介入，分支也更容易按列处理。它优秀的地方不只是“列式快”，而是把执行单位、内存布局和算子合同统一起来。

Selection vector 是其中一个关键设计。Filter 不一定立刻复制有效行，而是输出一组有效行下标；后续算子根据 selection vector 只访问通过过滤的行。这样可以减少材料化和拷贝，但也带来代价：后续访问可能变成间接访问，选择率高时 selection vector 本身也会占带宽，错误报告还要保留原始行号。优秀设计不是永远使用 selection vector，而是在选择率、字段宽度、后续算子和错误语义之间做判断。

列式 batch 的核心取舍：少搬冷字段，多保留身份



优秀设计的核心不是把所有数据都变成列式，而是让热路径只移动必要字段，同时保留后续复查、错误报告和 join 所需的身份。

图示：本图要证明的命题是：工业列式执行器的关键价值在于统一数据局部性和语义身份，而不是简单替换存储格式。

Pipeline breaker：为什么 sort、join build 和 global aggregate 特别重要 数据库执行器和流处理系统都会区分能流水线推进的算子和会打断流水线的算子。简单 map 和 filter 可以一批接一批向下游传递；sort 必须看到足够数据才能决定顺序；hash join 的 build side 必须先构建状态再 probe；global aggregate 必须合并局部状态后才能输出最终结果。这些阶段就是 pipeline breaker。

Pipeline breaker 的优秀设计在于承认边界，而不是假装一切都能流式。它通常会明确材料化对象、内存预算、spill 策略、状态大小和恢复边界。若内存不足，sort 可以写出 run 再归并；hash aggregate 可以分区或 spill；join 可以分批 build/probe；shuffle 可以把输出写成 block manifest。这里的共性是：当一个 stage 必须停下来积累状态时，系统要把状态的身份、大小、生命周期和失败恢

复讲清楚。

Shuffle 和 manifest: 从单机 partition 到分布式恢复 Spark、MapReduce 和很多分布式执行器都把 shuffle 当成核心边界。Shuffle 不只是“把相同 key 发到同一台机器”。它需要分区函数、block 文件、校验和、大小、位置、提交状态、重试和清理策略。没有 manifest, 系统不知道一个 block 是否完整、是否已经提交、失败后能否重读、重复提交是否安全。

本章的 **PartitionBlock** 是工业 shuffle manifest 的缩小版。单机里它帮助调试 bucket 范围和 checksum; 分布式里它变成 reducer 拉取数据、checkpoint 记录状态和失败重试的证据。优秀设计的核心原理是: 一旦数据跨越 stage 边界, 就必须拥有可命名、可校验、可恢复的身份。

Top-k、limit 和确定性输出 数据库和流处理系统经常支持 **ORDERBY...LIMITk**、top-k 聚合或近似 heavy hitter。工业实现必须非常小心确定性输出。若没有稳定排序键或 tie-break, 同一个查询在不同并行度、不同分区或不同重试下可能输出不同结果。对用户来说, 这不是“性能细节”, 而是结果语义不稳定。

因此优秀实现会把排序键、tie-break、null 排序、稳定性、近似误差和二次验证写进语义。精确 top-k 需要全局可比较 score; 近似 top-k 需要误差合同; 分布式 top-k 需要保证局部候选不会漏掉全局结果, 或者明确它只是近似召回。本章开头的事故, 本质上就是把工业系统必须声明的语义边界藏进了局部优化。

从工业设计迁移到本书项目 Compute Systems Engine 不需要复制 DuckDB、ClickHouse、Spark 或 Kafka 的完整实现, 但必须吸收它们的设计原则。列式 batch 迁移为 **HotEventBatch**; selection vector 迁移为 stable filter 的输出身份; pipeline breaker 迁移为 sort、join build、global reduce 和 checkpoint 的材料化边界; shuffle manifest 迁移为 **PartitionBlock**; 确定性 top-k 迁移为稳定 tie-break 和候选覆盖测试; 背压和重试会在后续章节迁移为有界队列、watermark、幂等提交和 checkpoint。

源码阅读任务

工业案例阅读任务: 选择 DuckDB、ClickHouse、Velox、Spark 或类似成熟系统中的一个算子实现, 阅读 hash aggregate、sort、top-k、selection vector、pipeline breaker 或 shuffle manifest 之一。报告必须写清: 这个设计面对什么约束; 核心数据结构是什么; 它怎样保留输出身份; 它怎样处理内存预算或失败; 它和本章哪个实验可以对照。不要写“某系统使用列式执行所以快”这种空泛结论。

8.8 项目落地: KernelPatternSuite

Compute Systems Engine 在本章应形成一个小而完整的 **KernelPatternSuite**。它不需要一次写成通用框架, 但必须覆盖几种不同形状: 一对一 map、输出未知的 stable filter、共享写明显的 histogram、局部候选的 top-k、会输出放大的 join 或替代案例、frontier 或不规则负载的小实验。每个内核都保留 reference、优化版本、输入生成器、校验器和 stage report。

输入族要攻击假设 压力输入不是简单放大规模, 而是攻击内核假设。Filter 要测试全通过、全不通过、低选择率和高选择率; histogram 要测试均匀 key、Zipf 分布和单热点 key; top-k 要测试“分散强 key”, 即每个 chunk 中不进局部 top-k、全局却很强的 key; join 要测试重复 key 导致的输出放大; graph 要测试规则网格图、随机图和幂律图。

源码阅读入口 Linux 源码阅读可以从 **lib/sort.c**、**include/linux/rhashtable.h** 和若干 bitmap/hashtable 辅助实现附近进入。阅读目标不是把整套内核代码抄进书里, 而是追一条和本章实验相关的路径: 排序怎样处理元素移动, 哈希表怎样处理扩容和桶, bitmap 怎样表达集合。读源码报告要区分证据等级: 直接测得的数字、从控制流推导的解释、当前设备无法验证的合理猜测。

实验: 扩展事件流高性能管线。第一步实现线程本地 histogram, 对比全局 mutex、全局 relaxed atomic 和本地 partial。第二步实现 stable filter, 用块计数和 scan 分配输出位置。第三步加入 top-k, 构造分散强 key, 证明局部 top-k 不能作为精确全局 top-k 的前置。报告必须写 records in、records out、temporary bytes、state bytes、最大 bucket、checksum 和未验证边界。

8.9 复查清单

一、**reference 是否仍然存在** 没有 reference，后面的 SIMD、多线程、异步或分布式改造都无法证明语义没有变。Reference 可以慢，但必须简单、边界清楚、错误路径可解释。

二、**内核合同是否写清** 每个内核都要写输入、输出、顺序、精确性、信息丢弃、外部副作用、可重试性和 tie-break。若合同写不出来，先不要优化。

三、**压力输入是否攻击核心边界** 只扩大数据量不够。要构造低选择率、高选择率、热点 key、稀疏 key、分散强 key、输出放大、frontier 长尾和边界长度输入。

四、**状态是否可观察** 报告中应能看到关键对象的身份、大小、状态、错误数和校验结果。对 partition 而言，要有 bucket 范围；对 shuffle 而言，要有 manifest；对 top-k 而言，要有候选数量和 tie-break。

五、**组合不变量是否成立** 单个内核正确不代表流水线正确。Filter 输出数、histogram 总和、partition 路由、join 输出规模、top-k 候选覆盖和最终 checksum 都要对齐 reference。

六、**是否说明不采用更复杂方案的理由** 没有证据证明队列是瓶颈，就不应把锁队列换成无锁；没有证明输出顺序可变，就不应随意重排；没有证明错误路径可恢复，就不应只提交正常路径。能解释不做什么，说明工程判断开始成形。

源码阅读任务

本章源码阅读只读窄入口，不做泛泛浏览。第一组入口是 Linux 的排序和集合辅助实现：阅读 `lib/sort.c`，关注元素移动、比较函数和临时空间边界；阅读 bitmap 相关辅助函数，关注集合如何用位表达和扫描。第二组入口是哈希结构：围绕 `include/linux/rhashtable.h` 和对应实现搜索扩容、bucket、冲突和遍历边界。第三组入口是生产库或数据库执行器中的 top-k、hash aggregate 或 sort aggregate，实现任选一个成熟项目，但报告必须把 reference、优化策略和错误边界分开。

源码报告必须回答五个问题：它解决的内核合同是什么；它怎样表示输入、输出和状态；它怎样处理错误或边界；它牺牲了什么换取性能；它和本章实验中的哪个实现可以对照。不要复制源码大段内容，重点画出数据流和状态边界。

验收与评分标准

本章大作业按 100 分评分。正确性 30 分：reference 清楚、边界覆盖完整、checksum 和顺序校验稳定。合同设计 20 分：每个内核写明输入输出、顺序、共享写、精确性、tie-break 和材料化边界。实验质量 20 分：输入族能攻击假设，报告包含 records in/out、temporary bytes、state bytes、最大 bucket、候选覆盖和结论边界。性能分析 15 分：能用数据解释瓶颈，不把所有问题归因于线程数。工程表达 15 分：代码接口清楚，命名和位宽类型明确，报告能复现命令和环境。

8.10 本章和后续章节的连接

本章不是内核模式清单，而是后续并发、运行时和分布式章节的接口层。Stable filter 里的 count + scan + write，会在第 13 章归约、扫描和分区中展开成并行算法；thread-local histogram 会在锁、原子和 false sharing 章节里变成共享状态设计问题；partition manifest 会在第 18 章变成 shuffle block 和 checkpoint manifest；graph frontier 会在任务运行时章节里变成 ready set、负载均衡和工作窃取问题；top-k 的信息丢弃合同会在分布式重试、近似计算和观测性里反复出现。

心智模型

第二卷后半部分的很多复杂系统问题，都可以回到本章的四个问题：输出位置是否确定，是否共享写，是否丢信息，是否留下可恢复的身份。

学到这里，读者应该能看见一个更深的结构：所谓高性能计算，不是把每个函数单独加速，而是让每个 stage 的语义、数据移动和恢复边界都清楚。一个内核快但破坏顺序，可能让后续 join 错；一个 filter 快但丢掉 record id，可能让错误报告不可用；一个 top-k 快但候选覆盖不足，可能让业

务结果错；一个 partition 快但没有 manifest，可能让分布式重试无法幂等。大师级工程师的判断力，就在这些边界上。

8.11 本章习题

习题与作业

习题一：把一个日志统计任务拆成 parse map、valid filter、partition、histogram、top-k 五个阶段。每个阶段写出输入、输出、是否保持顺序、是否有共享写、是否可能丢弃信息和主要瓶颈。

习题二：设计 stable filter。要求先写串行 `push_back` reference，再写共享锁反例，最后写块计数、scan、写出三阶段方案。测试覆盖选择率为 0、选择率为 100%、随机选择、最后一块不满和错误记录。

习题三：为 histogram 选择三种策略：全局原子、本地桶、稀疏本地 map。分别说明适用的 bucket 数量、key 分布、线程数、内存占用和合并成本。

习题四：构造本章开头的 top-k 反例。让一个 key 在每个 chunk 中都不进入局部 top-k，但全局总和进入前 k。说明怎样修改流水线才能得到精确结果，怎样声明近似结果才算诚实。

习题五：为一个 join 任务估算输出放大。给定每个 key 在左右输入中的出现次数，计算最坏输出规模，并说明何时应选择 hash join、sort-merge join、预过滤或热点 key 单独处理。

习题六：用一个小图实验比较 vector frontier 和 bitmap frontier。记录每层 frontier 大小、边访问数、重复发现次数和 worker 空闲时间，说明何时切换表示。

第零部分

多线程、同步与内存模型

第 9 章

线程、调度与亲和性

先看一个真实工程里反复出现的事故。Compute Systems Engine 的日志解析器使用 8 个 worker：大多数任务负责解析 `InputSplit`，少数任务负责把中间结果写成 `ShuffleBlock`。正常磁盘下，系统吞吐稳定；磁盘变慢后，几个写出任务阻塞在同步 `write/fsync`，仍然占着 compute worker。此时 ready 队列里有大量解析任务，CPU 使用率却下降，p99 排队时间上升。有人把线程数从 8 改到 32，短期看吞吐略有恢复，但 context switches、migrations、内存占用和尾延迟同时上升。真正的问题不是线程不够，而是阻塞任务吞掉了执行预算。

本章用这个事故建立线程调度的主线：任务是逻辑工作，线程是操作系统调度的执行实体，worker 是运行时长期持有的线程，CPU 是硬件执行资源。线程池能复用线程，却不会自动理解任务类型、等待点、亲和性、关闭语义和业务优先级。学会线程，不是背“线程数等于核心数”这种经验，而是能把任务生命周期、内核调度现象、硬件拓扑和项目指标连成可验证的解释。

学习目标

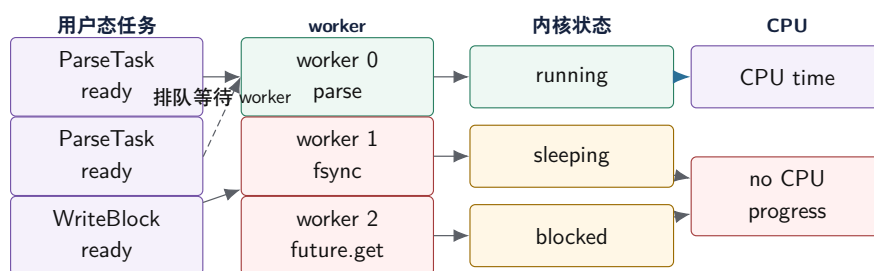
学完本章后，读者应能完成八件事：

1. 区分 task、thread、worker、CPU 和 run queue，并解释为什么 runnable 不等于 running。
2. 从阻塞任务占满线程池事故推导出 CPU 池、I/O 池、completion 和背压边界。
3. 用任务粒度曲线解释线程创建成本、队列成本、负载不均和取消延迟之间的取舍。
4. 说明上下文切换、抢占、睡眠、唤醒、迁移和 cgroup 限额如何影响吞吐与尾延迟。
5. 把亲和性理解为局部性约束，而不是默认性能按钮，并能设计可复查的拓扑实验。
6. 设计 `WorkerSnapshot` 和 `ThreadTimeline`，让线程池问题可以从证据中定位。
7. 从 Linux CFS、workqueue、oneTBB、OpenMP、Folly executor、Go scheduler 和移动大小核中提取可迁移原则。
8. 在 Compute Systems Engine 中落地线程配置、阻塞隔离、关闭语义和实验报告。

9.1 事故复盘：CPU 空闲，任务却不前进

事故现场最迷人的地方，是 CPU 使用率低。初学者容易把它解释成“线程开少了”。但如果 worker 都阻塞在同步写出、条件变量、future、锁或下游队列上，增加 compute worker 只是在同一个错误模型上加预算。正确排查顺序是：先问任务是否 ready，再问有没有 worker 取走，再问 worker 取走后是在 running 还是 blocked，最后才讨论线程数。

两层队列：ready 任务不等于正在运行



用户态队列、worker 状态、内核 run queue 和 CPU 时间是四个不同事实。CPU 空闲时，先找 worker 是否被隐藏等待占住。

图示：本图要证明的命题是：线程池慢不能只看 CPU 使用率，必须把用户态 ready 队列和内核运行状态分开。

任务、线程、worker 和 CPU Task 是可执行工作描述，例如解析一个 `InputSplit`、压缩一个 block、提交一个 manifest。Thread 是操作系统调度实体，拥有栈、寄存器上下文、调度状态和优先级。Worker 是运行时长期持有并反复执行 task 的 thread。CPU 是硬件执行资源。一个 task 可以排队很久，一个 worker 可以阻塞很久，一个 thread 可以 runnable 但不 running，一个 CPU 可以被其他进程、内核线程或 cgroup 策略占用。

把这些词混在一起，会导致错误修复。若说“任务有 32 个线程”，你不知道它是 32 个 ready task、32 个 worker、32 个 runnable kernel thread，还是 32 个逻辑 CPU。工程报告必须写清层次：提交了多少 task，线程池有多少 worker，内核看到多少 runnable thread，机器实际有多少可用 CPU。

Runnable 不等于 Running Runnable 表示线程具备运行条件，已经在某个运行队列中等待调度；running 表示它此刻真的占用 CPU。若 runnable 线程数长期大于可用 CPU，调度器只能轮流运行它们，context switch 增加，每个线程获得的连续 CPU 时间下降。若线程处于 sleeping 或 blocked，它不消耗 CPU，却可能占住 worker，导致用户态 ready 任务无人执行。

这一区分解释了两类相反现象。CPU 使用率很高但吞吐不涨，可能是线程过多、锁竞争、内存带宽或任务粒度太小。CPU 使用率很低但队列增长，可能是 worker 睡在 I/O、future、条件变量或下游背压上。前者要减少无效竞争或改善算法，后者要隔离阻塞或改变等待模型。

用等待图代替猜测 线程池排障应画等待图。节点可以是 task、worker、queue、mutex、condition variable、future、file descriptor、disk、network service；边表示“谁在等谁”。若 worker 等 I/O、I/O 等设备；若 worker 等 future，future 等队列中的 child task；若队列满，producer 等 consumer；若 consumer 又在等 producer 持有的锁，就可能形成等待环。等待图比线程列表更有解释力，因为它能说明执行预算为什么无法推进工作。

9.2 线程不是魔法倍增器

朴素程序常从“每个任务创建一个线程”开始。这个版本适合教学，因为 task 和 thread 几乎一一对应，生命周期直观。但真实系统中，线程创建、栈空间、内核登记、调度竞争、join、异常传播和关闭都会进入成本。线程池把线程生命周期拉长，让 task 通过队列复用 worker；这减少了创建成本，却引入了队列、任务粒度、背压、关闭和隐藏等待。

线程创建成本不是唯一问题 线程创建慢只是最表层成本。更深的成本来自执行图变复杂：短任务会被队列和唤醒成本吞掉；长任务会造成负载不均；阻塞任务会占住 worker；任务内部再提交子任务并等待，可能造成同池饥饿；异常若留在 worker 日志里，调用者无法知道结果失败；关闭时若不唤醒等待线程，析构可能悬挂。

因此，线程池不是把 `std::thread` 包进一个队列那么简单。一个最小工程线程池至少要说明：提交失败如何返回，队列容量是否有限，关闭后是否接收新任务，未开始任务是否取消，正在运行任务能否协作取消，任务异常如何传播，析构是否 drain，worker 内是否允许同步等待同池任务。

任务粒度的三段曲线 任务粒度决定线程池是否值得。粒度太小，每个 task 的队列操作、计数器、唤醒和 trace 成本超过有效计算；粒度太大，少数长 task 拖住尾部，取消延迟也变长；中间区

域才是合理范围。不要把 chunk size 写成魔法数字，应让实验曲线告诉你当前机器和当前 workload 的有效范围。

粒度区域	主要现象	根因	调整方向
过细	吞吐低、context switch 多、队列热点	调度成本大于计算	批量、合并 task、减少 trace
适中	worker 忙碌均衡、队列可控	计算量能摊薄固定成本	作为默认范围
过粗	尾部长、空闲 worker 多、取消慢	负载不均和关键路径长	拆分、动态调度、work stealing

粒度还要和 cache 及恢复边界一起看。连续数组归约可以用较大的 contiguous chunk，因为局部性好、成本均匀；日志解析遇到异常行、长行或热点 key 时，chunk 耗时会不均；shuffle 写出若以 block 为恢复单位，block 太大则重试昂贵，太小则 manifest 和 I/O 元数据膨胀。

嵌套并行会制造隐形超额订阅 上层服务可能已经为请求开了 8 个 worker，每个请求内部又调用默认 8 线程的压缩库、矩阵库或并行 STL，瞬间制造 64 个 runnable 线程。操作系统只看到线程竞争 CPU，不知道哪些线程属于同一个业务请求。工业系统常用共享 executor、限制库内线程数、按 stage 设置资源预算、禁止 worker 内部隐式并行等方式控制这类超额订阅。

9.3 阻塞隔离：线程池设计的第一条工程边界

阻塞并不等于错误。等待磁盘、网络、锁、future、条件变量、page fault 或下游背压，是系统运行的一部分。错误在于把长等待放到最宝贵的 compute worker 上，并让其他 ready 计算任务排队。线程池设计的第一条工程边界，就是 CPU 执行预算和阻塞等待预算要分开。

阻塞隔离：compute worker 不应该长期等待 I/O

错误设计

同一队列中，慢写出占住 compute worker，ready 解析任务等待。

改进设计

compute worker 生成 block 后释放；I/O pool 等待内核；completion 短小地更新状态。

图示：本图要证明的命题是：分池不是为了形式复杂，而是让长等待不占用计算执行预算。

任务类型要进入接口 线程池若只接受 `submit(function)`，调用者很容易把任意工作塞进去。更好的教学接口至少给 task 标记类型：CPU、BlockingIo、Control、Completion、Background。类型不是装饰，它决定进入哪个队列、允许不允许阻塞、是否有 deadline、能否被取消、是否计入 compute worker 饱和度。

Listing 9.1 线程池配置和任务类型要显式

```
1 enum class TaskClass : std::uint8_t {
2   Cpu,
3   BlockingIo,
4   Control,
5   Completion,
6   Background,
```

```

7 };
8
9 enum class ShutdownMode : std::uint8_t {
10     DrainAccepted,
11     CancelNotStarted,
12 };
13
14 struct ThreadPoolPlan {
15     std::uint32_t compute_workers = 0;
16     std::uint32_t blocking_workers = 0;
17     std::uint32_t queue_capacity = 0;
18     std::uint32_t chunk_records = 0;
19     ShutdownMode shutdown = ShutdownMode::DrainAccepted;
20     bool collect_timeline = false;
21 };

```

这段代码的重点不是实现线程池，而是把运行时契约写进类型。后续第 10 章会用 mutex 和 condition variable 实现队列，第 14 章会把 task 依赖和 work stealing 引入运行时，第 15 章会把 I/O completion 变成可靠写出路径。本章先要求接口不再掩盖任务类型。

Completion 必须短小 I/O 完成后通常需要更新状态、减少 unfinished 计数、通知 driver、提交 manifest 或唤醒下游任务。completion 线程不应执行长计算，也不应在持有全局锁时调用未知用户回调。工业系统会把 completion 当成控制面事件：快速记录事实，必要时把后续 CPU 工作重新投递到 compute pool。

增加线程只是补丁，不是解释 阻塞任务混入 compute pool 时，增加 worker 有时能短期改善，因为多出来的 worker 可以继续处理计算任务。但这不是根治：长等待仍然占用线程栈和调度对象，系统过载时队列会继续增长，尾延迟会更不稳定。工程报告必须比较三组：单池加线程、分离 compute/I/O、异步 completion。只有能解释每组的收益和代价，才算真正理解事故。

9.4 调度器看到的是线程，不是业务意图

用户态运行时决定 task 到 worker 的映射；内核调度器决定 worker thread 到 CPU 的映射。二者信息不同：运行时知道业务优先级、任务依赖、输入 split 和错误语义；内核知道 CPU、run queue、优先级、睡眠、唤醒、迁移、cgroup 和系统负载。线程调优往往失败，是因为用户态以为自己控制了全部调度，或者把内核现象误解释成业务现象。

上下文切换的成本来自直接和间接两部分 直接成本包括保存/恢复寄存器、切换内核调度结构、更新运行队列和进入/退出内核路径。间接成本更常见：cache 热度下降、TLB 和分支预测状态受影响、锁持有者被抢占导致其他线程等待、线程迁移后访问远端数据。上下文切换没有一个固定纳秒数，必须放到 workload 中解释。

报告要区分 voluntary 和 involuntary context switch。自愿切换常来自等待条件变量、I/O、futex 或 sleep；非自愿切换常来自时间片耗尽、抢占、更高优先级任务或 CPU 竞争。前者通常追等待点，后者通常追线程过多、系统负载、优先级和 cgroup 配额。

唤醒不是立即运行 `notify_one`、I/O 完成、futex wake 或信号到来，只是让线程有机会进入 runnable 状态。被唤醒的线程还要排队等待 CPU，拿到 CPU 后还可能重新竞争 mutex，竞争失败又睡回去。因此高竞争条件变量和锁会放大尾延迟。正确性依靠谓词循环，性能依靠减少竞争、缩短临界区、控制队列容量和合适粒度。

cgroup、容器和移动设备会改变可用 CPU 在容器中，`std::thread::hardware_concurrency()` 不一定等于实际可用 CPU；cpuset、CPU quota 和调度周期可能让线程呈周期性被 throttled。移动设备还有大小核、动态频率、温控和前后台策略。一个 benchmark 如果不记录这些环境事实，结论很难复现。

9.5 亲和性：局部性约束，不是默认开关

亲和性让线程倾向或固定在某些 CPU 上运行。它能减少迁移、保护 cache 热度、对齐 NUMA 节点或设备队列；也会降低调度器自由度，让负载不均更严重，或者在移动设备上被系统策略覆盖。亲和性不是“绑了就快”，而是一个要用证据检验的局部性假设。

拓扑先于绑定 绑定前必须记录拓扑：逻辑 CPU 数、物理核心数、SMT sibling、共享 L1/L2/L3 的范围、NUMA 节点、cpuset、CPU quota、大小核信息和当前频率策略。没有拓扑记录，亲和性结论不可审查。把两个重计算 worker 绑到同一个物理核心的两个 SMT sibling 上，可能看似“两个 CPU”，实际共享执行资源；把 worker 绑在某个 NUMA 节点而数据页在远端，会稳定地产生远端访问。

策略	可能收益	主要风险	默认
不绑定	保留调度器自由度，适合普通负载	迁移和 cache 热度不稳定	是
按物理核心绑定	减少迁移，避免 SMT 资源互抢	负载不均时核心空闲	实验
按 NUMA 绑定线程和内存	数据本地，远端访问少	迁移和分片策略复杂	实验
绑定大核或高性能核	短期峰值高，延迟稳定	温控、耗电、系统策略干扰	谨慎

亲和性要和数据分片一起设计 若 worker 0 固定在 CPU 0，却随机处理所有输入 split，绑定价值有限。更有意义的做法是把 worker、本地队列、输入 split、partial buffer 和内存节点对齐：本节点 worker 优先处理本节点数据，跨节点窃取作为兜底。这和第 17 章分片思想一致：计算尽量靠近数据，但必须保留负载均衡和失败恢复路径。

移动大小核的特殊边界 手机或 ARM big.LITTLE 设备上，核心性能不等价，频率会随温度和电源状态变化。线程数等于核心数并不等于拥有同等执行预算。短时间 benchmark 可能在大核上获得高峰值，长时间运行后因温控降频而变慢。Termux 环境还可能缺少设置 affinity 的权限。报告应记录设备型号、电源状态、温度趋势、可见 CPU 和绑定是否成功；无法验证的结论要标注。

9.6 可观测性：WorkerSnapshot 和 ThreadTimeline

没有状态记录，线程池问题只能靠猜。第 2 卷的贯穿项目应从本章开始加入 worker 级观测。观测不是为了做漂亮监控界面，而是为了把“CPU 低”“吞吐差”“尾延迟高”拆成排队、运行、阻塞、迁移、队列满、关闭等待和异常路径。

Listing 9.2 WorkerSnapshot：线程池排障的最小证据

```

1 enum class WorkerState : std::uint8_t {
2     Starting,
3     Idle,
4     RunningCpu,
5     RunningCompletion,
6     BlockedIo,
7     BlockedFuture,
8     BlockedQueue,
9     Stopping,
10    Joined,
11 };
12
13 struct WorkerSnapshot {
14     std::uint32_t worker_id = 0;
15     std::uint32_t last_cpu = 0;
16     WorkerState state = WorkerState::Starting;

```



```

17 TaskClass current_class = TaskClass::Cpu;
18 std::uint64_t current_task_id = 0;
19 std::uint64_t executed_tasks = 0;
20 std::uint64_t queue_pops = 0;
21 std::uint64_t idle_ns = 0;
22 std::uint64_t run_ns = 0;
23 std::uint64_t blocked_ns = 0;
24 std::uint64_t steal_count = 0;
25 };

```

这些字段都服务于问题判断。若 `idle_ns` 高且 ready 队列空，可能是上游供给不足；若 `idle_ns` 低但 `blocked_ns` 高，worker 被等待占住；若 `last_cpu` 频繁变化且 `migrations` 高，亲和性或系统干扰值得检查；若少数 worker 的 `executed_tasks` 和 `run_ns` 明显更高，可能有任务倾斜或窃取不足。

ThreadTimeline 记录事件，不记录故事 Timeline 至少记录 submit、enqueue、dequeue、start、block begin、block end、finish、cancel、fail、shutdown begin、worker joined。每个事件包含 task id、worker id、task class、时间戳、可选 CPU、错误码和队列长度。不要在热路径打印长文本日志，应该写结构化事件或采样，最后统一输出 CSV/JSON/key-value 摘要。

Listing 9.3 ThreadTimeline 事件的教学形态

```

1 enum class ThreadEventKind : std::uint8_t {
2     Submit,
3     Enqueue,
4     Dequeue,
5     Start,
6     BlockBegin,
7     BlockEnd,
8     Finish,
9     Fail,
10    Cancel,
11    ShutdownBegin,
12    WorkerJoined,
13 };
14
15 struct ThreadEvent {
16     std::uint64_t timestamp_ns = 0;
17     std::uint64_t task_id = 0;
18     std::uint32_t worker_id = 0;
19     std::uint32_t queue_size = 0;
20     std::uint32_t cpu_id = 0;
21     TaskClass task_class = TaskClass::Cpu;
22     ThreadEventKind kind = ThreadEventKind::Submit;
23 };

```

观测也有成本 每个任务都写全局锁日志，会改变调度行为。高频指标应使用每 worker 槽位、批量写入、采样或实验模式开关；低频控制状态可以用锁保护。ThreadSanitizer 能发现许多数据竞争，但会改变性能，不能用于性能报告。并发测试可以通过固定 seed、缩小队列容量、减少 worker 到 1 或 2、插入 yield、随机 sleep 来暴露时序 bug。

9.7 工业设计参照

工业案例不是项目名列表，而是约束、机制、能力和代价的组合。读源码或论文时，要问它解决的 workload 是什么，控制了哪些状态，放弃了哪些灵活性，哪些思想能迁移到教学项目，哪些复杂度不该照搬。

工程案例

Linux CFS 和调度器。Linux 调度器面向全系统公平、优先级、NUMA、cgroup、实时任务和电源管理，不理解你的业务 task。它通过 task state、runqueue、wakeup、preemption、migration 和 context switch 让内核线程共享 CPU。可迁移原则是：线程状态必须显式，唤醒只是进入可运行集合，绑定和迁移都要由证据驱动。教学项目不要仿写 CFS，而要把 worker 状态和队列等待记录清楚。

Linux workqueue。内核 workqueue 把“工作项”和“执行线程”分离，并处理并发度、CPU 绑定、救援线程和内存回收压力等问题。它提醒我们：即使在内核里，工作项也不等于线程；可能阻塞或参与回收路径的工作要有特殊策略。教学项目可借鉴工作项分类和执行域隔离，不需要复制内核复杂策略。

oneTBB、Cilk 和 OpenMP task。这些运行时把大量 task 映射到有限 worker，并通过本地队列、工作窃取、task group 或 fork-join 结构减少全局竞争。它们购买的能力是负载均衡和嵌套并行控制；代价是运行时状态、调试复杂度和任务粒度敏感。第 14 章会深入 work stealing，本章只采用原则：worker 不应长期等待同池 future，任务依赖应进入运行时。

Folly executor、Java ForkJoinPool 和 Go scheduler。这些系统都把执行资源抽象成 executor/scheduler，并围绕阻塞、队列、优先级、timer、I/O 和 continuation 设计边界。Go 通过 M:N 调度让 goroutine 数量远大于 OS thread，但 CPU 密集工作仍受实际线程和核心限制；Java ForkJoinPool 对 worker 内等待有 managed blocking 等机制；Folly 强调 executor 语义和异步链。可迁移原则是：阻塞必须被运行时知道，或者被隔离。

移动大小核调度。Android/Linux 能根据负载、功耗和温度在大小核之间迁移任务。它购买的是能效和交互响应，代价是 benchmark 波动和用户态控制受限。教学项目在手机上运行时，应把亲和性当作可选实验，并把持续吞吐、温度和频率写进报告。

9.8 项目落地：线程策略报告

本章不要求一次写出完整任务运行时，但必须把线程策略从隐式经验变成可测试对象。Compute Systems Engine 的本章交付物包括四部分：保留 reference，增加线程池配置，记录 worker/timeline，构造阻塞污染实验。

最小实现路线 第一，保留每次创建线程的 reference，用它固定输出语义。第二，实现固定 compute pool，只接受 `TaskClass::Cpu` 和短 completion。第三，把阻塞写出移到 blocking pool 或模拟异步 I/O，compute worker 只生成 `ShuffleBlock` 描述并返回。第四，输出 `WorkerSnapshot`、队列长度、任务排队时间、执行时间和阻塞时间。第五，扫描 worker 数和 chunk size，形成曲线。

线程池关闭语义 关闭语义必须写进报告。Drain 模式表示拒绝新任务，等待已接纳任务完成；CancelNotStarted 表示未开始任务被取消，正在运行任务只能协作结束；无论哪种模式，等待中的 worker 都要被唤醒，析构都不能静默丢失异常。这个主题会在第 10 章通过条件变量谓词落地，在第 14 章通过任务状态机扩展。

实验：阻塞任务污染计算线程池。构造 1000 个短 CPU task 和 8 个慢 I/O 模拟 task。对比三种设计：单池 8 worker、单池 32 worker、compute pool 8 worker 加 blocking pool 4 worker。记录吞吐、CPU task p95/p99 排队时间、context switches、worker blocked time、队列最大长度和输出校验。报告必须解释为什么“加线程”与“阻塞隔离”买到的能力不同，代价也不同。

实验矩阵 线程实验至少覆盖五组变量：worker 数从 1 到逻辑 CPU 数的两倍，chunk size 从很小到很大，是否混入阻塞任务，是否设置亲和性，是否在移动或容器环境运行。每组都输出相同

字段，避免只展示最好的图。若无法获得 `perf`、`pidstat` 或 `affinity` 权限，应在项目 `trace` 中给出替代指标，并写明未验证边界。

现象	优先检查	常见修复
CPU 低、队列涨	worker 是否 blocked, I/O 是否慢，下游是否背压	分池、异步 completion、容量队列
CPU 高、吞吐不涨	任务粒度、锁竞争、context switch、内存带宽	批量、减少共享写、调低 worker
p99 高、平均正常	迁移、长 task、锁持有者抢占、后台任务	拆任务、隔离资源、记录关键路径
绑定后更慢	负载不均、SMT sibling、远端内存、系统干扰	放松绑定、按拓扑分组、绑定内存
关闭卡住	条件变量谓词、未完成计数、worker 内等待	明确 drain/cancel，唤醒所有等待者

源码阅读任务

本章源码阅读按事故驱动，不按目录背诵。

- 1. Linux: 从 `kernel/sched/core.c`、`kernel/sched/fair.c`、`kernel/sched/topology.c` 追一条现象路径，例如 `wakeup` 后如何进入 `runqueue`，`migration` 为什么发生，`cgroup quota` 如何限制运行时间。
- 2. Linux `workqueue`: 阅读 `work item` 与 `worker pool` 的边界，重点看“工作项不是线程”和“可能阻塞的工作如何不拖垮全局进度”。
- 3. `oneTBB` 或 `OpenMP task`: 找 `task graph`、`arena`、`worker` 和 `task group` 的责任分界，说明它们如何控制嵌套并行。
- 4. `Folly executor` 或 `Java ForkJoinPool`: 观察 `executor` 如何表达执行域、队列、阻塞和 `continuation`；不要只列 API。
- 5. 移动设备资料: 记录大小核、频率和温控对一次线程实验的影响，标注哪些结论无法在服务器上直接迁移。

阅读报告必须从一个可复现实验或事故开始，最后回到 `Compute Systems Engine` 中应采用、简化或拒绝的设计。

验收与评分标准

本章满分 100 分：

- 1. 正确性与 `reference`，15 分：并发版本输出与 `reference` 一致，异常、取消和关闭路径有测试。
- 2. 任务/线程/worker/CPU 边界，15 分：报告能区分 `ready`、`runnable`、`running`、`blocked` 和 `sleeping`，不混用术语。
- 3. 阻塞隔离，20 分：能复现单池污染，给出分池或异步 `completion` 方案，并说明代价。
- 4. 任务粒度与线程数实验，15 分：扫描 `worker` 和 `chunk size`，解释曲线，而不是只给最佳值。
- 5. 可观测性，15 分：实现 `WorkerSnapshot` 或等价字段，输出 `timeline` 摘要，能定位等待点。
- 6. 亲和性与环境记录，10 分：记录拓扑、绑定策略、绑定是否成功、容器或移动设备限制。
- 7. 工业案例迁移，10 分：至少分析两个工业设计的约束、机制、能力和代价，并说明项目取舍。

9.9 本章小结

线程是执行预算，不是工作本身。用户态任务队列、worker 状态、内核 run queue 和 CPU 时间必须分开观察。线程数增加不加速，可能来自粒度、锁、I/O、内存带宽、抢占、迁移或下游背压；CPU 使用率低，也可能是 worker 被隐藏等待占住。亲和性可以保护局部性，但只有在拓扑、数据位置 and 实际迁移都被记录时才有意义。

本章真正留下的项目对象是 `ThreadPoolPlan`、`WorkerSnapshot` 和 `ThreadTimeline`。第 10 章会用锁、条件变量和信号量实现这些状态的等待谓词；第 14 章会把“等待”从 worker 栈移入任务图；第 15 章会把阻塞写出改成可恢复的异步 I/O；第 18 章会把这些局部证据汇总成 `RunReport`。

9.10 本章习题

习题与作业

1. 构造一个 4 worker 线程池，提交 4 个父任务，每个父任务再提交一个子任务并等待。解释为什么这不是 mutex 死锁，却会造成同池饥饿。
2. 对同一批解析任务扫描 chunk size。画出吞吐、p95 排队时间、worker idle time 和任务数曲线，指出过细、适中、过粗三个区域。
3. 在单池中混入慢写出任务，记录 CPU task 的 p99 排队时间。再实现 compute pool 与 blocking pool 分离，比较收益和代价。
4. 设计 `WorkerSnapshot` 的扩展字段，用来区分锁等待、I/O 等待、future 等待和队列满等待。说明哪些字段适合高频采集，哪些字段只能采样。
5. 在一台支持 affinity 的机器上比较不绑定、绑定物理核心、绑定 SMT sibling、按 NUMA 分组四种策略。若机器不支持 NUMA，写出替代实验和未验证边界。
6. 阅读一个工业 executor 或 scheduler，写一页事故驱动报告：它遇到的约束是什么，核心机制是什么，买到什么能力，付出什么代价，Compute Systems Engine 应采用哪一部分。

第 10 章

锁、条件变量与信号量

第 9 章让线程池问题变得可见：ready 任务排队、worker 阻塞、CPU 空闲、关闭卡住。现在看一个更小但更本质的事故。线程池 worker 在空队列上等待，主线程调用 `shutdown`，设置 `closed=true` 后只通知了 `not_full`。生产者被唤醒并退出，消费者仍睡在 `not_empty` 上，析构时 `join` 永远不返回。代码里有 `mutex`，有 `condition variable`，也有 `notify_one`，但同步语义仍然错，因为等待谓词没有覆盖关闭状态，通知对象也没有覆盖所有等待者。

本章的核心句子是：同步原语必须从共享状态的不变量和等待谓词推出来。Mutex 不是给代码块贴“线程安全”标签，而是保护一组字段之间的关系；condition variable 不是事件队列，而是让线程在谓词不满足时睡眠，并在状态可能变化时醒来重查；semaphore 不是更高级的锁，而是表达可等待的资源额度。贯穿案例是可关闭有界队列，因为它足够小，却同时包含互斥、等待、背压、关闭、异常、安全析构和测试。

学习目标

学完本章后，读者应能完成八件事：

1. 为一个并发对象写出锁保护的不变量，而不是只说“这里加锁”。
2. 解释条件变量为什么等待谓词，不等待通知次数，并能修复丢失唤醒和虚假唤醒问题。
3. 设计有界队列的 `push/pop/close/drain` 状态机，使关闭、背压和等待者退出都有定义。
4. 区分 `mutex`、`condition variable`、`semaphore`、`read-write lock`、`RCU` 和 `atomic` 的适用边界。
5. 说明锁顺序、锁内未知代码、异常路径和析构副作用如何制造死锁或尾延迟。
6. 从 `futex fast path/slow path` 理解用户态同步与内核睡眠唤醒的关系。
7. 为 Compute Systems Engine 写出 `QueueContract`，并让线程池、I/O 队列和 `MessageBus` 复用。
8. 设计容量为一、满队列关闭、空队列关闭、多生产者多消费者、异常任务和压力交错测试。

10.1 互斥锁保护的是不变量

很多并发错误来自一个误解：`mutex` 保护某个变量。更准确地说，`mutex` 保护一组状态之间的不变量。环形队列的 `head`、`tail`、`size`、`buffer` 和 `closed` 不是孤立字段，它们共同说明哪些槽位可读、哪些槽位可写、队列是否还接收新元素。只把 `size` 改成 `atomic`，不能让队列线程安全，因为 `size` 与 `head/tail/buffer` 的关系仍可能被并发修改破坏。

先写不变量表 有界队列的不变量至少包括：容量固定且大于零；`0<=size<=capacity`；`head` 和 `tail` 总在数组范围内；`size==0` 表示没有可读槽；`size==capacity` 表示没有可写槽；`head` 指向下一个可读槽；`tail` 指向下一个可写槽；`closed` 后不再接受新 `push`。所有会改变这些事实的代码，都必须在同一互斥边界内完成状态转换。

Listing 10.1 有界队列的不变量不是注释装饰，而是代码审查依据

```
bounded queue invariant:
  capacity > 0
  0 <= size <= capacity
  head and tail are valid slots
  size == 0 means no readable slot
```

```
size == capacity means no writable slot
closed means future push must fail
```

不变量表把代码审查变得具体。若 `push` 写入槽位后忘记更新 `size`，违反容量关系；若 `pop` 先减 `size` 再移动元素而移动抛异常，状态可能提交一半；若 `close` 在锁外设置 `closed`，等待者可能看见旧状态；若 `head` 在锁内改、`tail` 在锁外改，环形区间关系被拆碎。

临界区越短越好，但不能拆散不变量 临界区应该短，是因为持锁时间越长，其他线程等待越多，尾延迟越不稳定。但“短”不能优先于“完整”。把队列是否满的检查放在锁外，看似减少持锁时间，实际上让检查和写入之间出现竞态。正确原则是：锁内完成最小的、不可拆分的状态转换；锁外执行耗时且不改变受保护不变量的工作。

锁内尤其不能执行未知代码：用户回调、任务函数、磁盘 I/O、网络请求、复杂日志格式化、可能很慢的析构、会再进入运行时的函数。C++ 中最后一个 `std::shared_ptr` 在锁内析构，也可能触发复杂逻辑。高质量代码常在锁内把对象移动到局部变量，释放锁后再处理或析构。

Listing 10.2 互斥锁保护队列状态转换

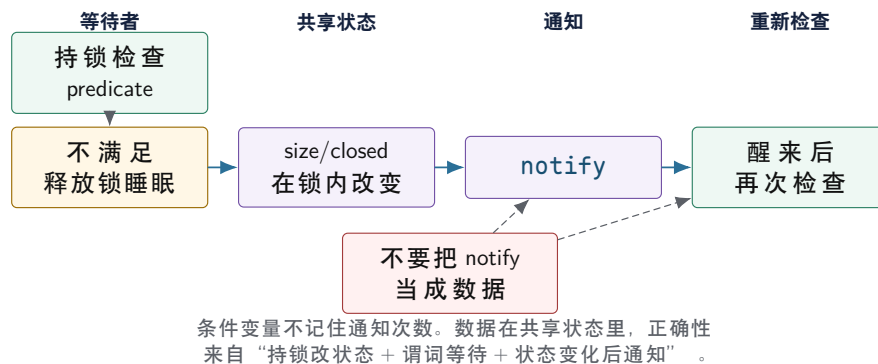
```
1 class QueueState {
2 public:
3     void push(std::int32_t value) {
4         std::lock_guard<std::mutex> lock(this->mutex_);
5         this->values_.push_back(value);
6     }
7
8 private:
9     std::mutex mutex_;
10    std::vector<std::int32_t> values_;
11 };
```

这个示例很小，但习惯要严格：成员访问使用 `this->`，锁对象用 RAII，锁的作用域就是临界区。后续加入 `head_`、`tail_`、`size_`、`closed_` 后，它们都应归入同一不变量边界，直到测量证明需要拆分。

10.2 条件变量等待的是谓词

条件变量常被误解为事件通知系统。正确模型是：线程持锁检查共享状态；若谓词不满足，`wait` 原子地释放锁并睡眠；被通知或虚假唤醒后，重新持锁检查谓词。通知本身不保存状态；如果通知发生时没有等待者，它不会留在条件变量里等后来者；被唤醒也不代表条件已经成立，因为别的线程可能先拿到锁并消费资源。

条件变量：通知只是让等待者重新检查谓词



图示：本图要证明的命题是：等待者醒来只说明状态可能变了，不说明谓词一定为真。

谓词必须覆盖退出条件 有界队列中，生产者等待的谓词不是“收到 `not_full` 通知”，而是“队列未满，或者队列已关闭”。消费者等待的谓词不是“收到 `not_empty` 通知”，而是“队列非空，或者队列已关闭”。关闭状态必须进入所有相关谓词，否则关闭时即使 `notify_all`，线程醒来后发现队列仍满或仍空，又会睡回去。

Listing 10.3 条件变量等待的是谓词，不是通知次数

```

1 std::optional<std::int32_t> pop_wait() {
2     std::unique_lock<std::mutex> lock(this->mutex_);
3     this->not_empty_.wait(lock, [this]() {
4         return this->closed_ || this->size_ > 0;
5     });
6
7     if (this->size_ == 0) {
8         return std::nullopt;
9     }
10
11     const std::int32_t value =
12         this->buffer_[static_cast<std::size_t>(this->head_)];
13     this->head_ = (this->head_ + 1) %
14         static_cast<std::int32_t>(this->buffer_.size());
15     --this->size_;
16     this->not_full_.notify_one();
17     return value;
18 }

```

这段代码的线性化点是 `head/size` 更新。消费者取出元素并释放一个槽位后，通知一个生产者重新检查 `not_full` 谓词。若此时有多个消费者被唤醒，只有持锁后看到 `size>0` 的消费者能继续；其他消费者必须继续等待或在关闭状态下退出。

notify 的时机 常见写法是先在锁内修改状态，再通知。通知可以发生在仍持锁时，也可以在释放锁后发生，两者有性能和竞争细节差异。教学阶段不要为了微优化打乱状态逻辑：先保证所有等待者的谓词正确，所有能改变谓词结果的状态修改都在锁保护下，关闭时通知所有相关等待者。

丢失唤醒不是“少 notify 一次” 丢失唤醒的根因通常不是通知次数少，而是“检查谓词”和“进入等待”没有作为一个受锁保护的動作。错误写法常先在锁外检查 `size==0`，再进入 `wait`；如果生产者在两个动作之间完成 `push` 和 `notify`，消费者随后睡下去，就可能永远等不到下一次通知。标准条件变量的 `wait(lock, predicate)` 把这个模式固定下来：先持锁检查，谓词不满足时

原子释放锁并睡眠，醒来后重新持锁检查。

Listing 10.4 错误模式：if 等待无法抵抗虚假唤醒和关闭竞态

```

1 std::optional<std::int32_t> broken_pop() {
2     std::unique_lock<std::mutex> lock(this->mutex_);
3     if (this->size_ == 0) {
4         this->not_empty_.wait(lock);
5     }
6     if (this->size_ == 0) {
7         return std::nullopt;
8     }
9     return this->pop_locked();
10 }

```

这段代码有两个问题。第一，**wait** 允许虚假唤醒，醒来后不能假设队列非空。第二，关闭语义缺失：若队列空且已关闭，消费者应退出，而不是继续等待。正确写法不是在 **wait** 后补更多 if，而是把完整谓词写进去：**closed_||size_>0**。同步代码的可读性来自谓词完整，而不是通知语句多。

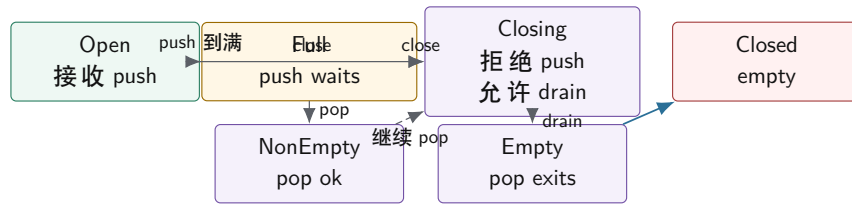
notify_one 和 **notify_all** 的边界 **notify_one** 适合一次状态变化最多只能让一个等待者继续，例如 **push** 一个元素后唤醒一个消费者，或 **pop** 一个元素后释放一个槽位唤醒一个生产者。**notify_all** 适合全局状态变化，例如 **close**、配置切换、取消、阶段结束。关闭时只 **notify_one** 很危险，因为可能有多个生产者和消费者分别睡在不同条件变量上；只醒一个等待者，其他等待者可能没有下一次状态变化来唤醒。

状态变化	推荐通知	原因
插入一个元素	not_empty.notify_one()	最多释放一个消费者的工作
弹出一个元素	not_full.notify_one()	最多释放一个生产者的槽位
队列关闭	两个条件变量都 notify_all()	所有等待者都要重新判断退出
取消全部任务	相关等待点 notify_all()	等待目标从“资源可用”变成“停止等待”
阶段屏障完成	notify_all() 或 barrier	多个等待者共享同一阶段结果

10.3 有界队列：同步语义的最小系统

Compute Systems Engine 的线程池、I/O 写出队列、MessageBus、分布式模拟请求队列，都需要一个共同概念：容量有限、能背压、能关闭、能 drain 的队列。这个队列不是普通容器，而是生产者和消费者之间的同步合同。

有界队列状态机：Open、Closing、Closed



close 不是析构细节，而是状态转换。Closing 拒绝新输入，但允许已接收元素被取完；Closed 表示队列空且等待者都应退出。

图示：本图要证明的命题是：关闭协议必须区分“拒绝新输入”和“排空旧输入”。

接口先表达合同 队列接口应分层：非阻塞尝试、阻塞等待、关闭和可选 drain/timeout。非阻塞 `try_push/try_pop` 适合事件循环或自定义调度；阻塞 `push/pop` 适合普通生产者消费者；`close` 负责唤醒等待者并改变生命周期状态。哨兵值不是好方案，因为泛型元素未必有安全哨兵，且哨兵无法表达多个等待者和关闭策略。

Listing 10.5 有关闭语义的有界队列接口草图

```

1 class BoundedQueue {
2 public:
3     explicit BoundedQueue(std::int32_t capacity);
4
5     bool try_push(std::int32_t value);
6     bool push(std::int32_t value);
7     std::optional<std::int32_t> pop();
8     void close();
9
10 private:
11     std::vector<std::int32_t> buffer_;
12     std::int32_t head_ = 0;
13     std::int32_t tail_ = 0;
14     std::int32_t size_ = 0;
15     bool closed_ = false;
16     std::mutex mutex_;
17     std::condition_variable not_empty_;
18     std::condition_variable not_full_;
19 };

```

`push` 返回 `false`，表示关闭后不再接受新元素；`pop` 返回 `std::nullopt`，表示队列已关闭且没有剩余元素。这比在内部偷偷抛异常或让线程永久阻塞更适合作为运行时基础。业务层可以选择把这种状态转换成错误码、异常或取消，但底层合同必须明确。

push 和 pop 的状态转换 生产者进入 `push` 后，先等待“有空间或关闭”。若关闭，返回失败；若有空间，写入槽位，更新 `tail/size`，通知消费者。消费者进入 `pop` 后，先等待“有元素或关闭”。若没有元素且关闭，返回空；若有元素，取出槽位，更新 `head/size`，通知生产者。

Listing 10.6 `push` 的谓词包含关闭状态

```

1 bool BoundedQueue::push(std::int32_t value) {
2     std::unique_lock<std::mutex> lock(this->mutex_);
3     this->not_full_.wait(lock, [this]() {

```



```

4     return this->closed_ ||
5         this->size_ < static_cast<std::int32_t>(this->buffer_.size());
6     });
7     if (this->closed_) {
8         return false;
9     }
10
11    this->buffer_[static_cast<std::size_t>(this->tail_)] = value;
12    this->tail_ = (this->tail_ + 1) %
13        static_cast<std::int32_t>(this->buffer_.size());
14    ++this->size_;
15    this->not_empty_.notify_one();
16    return true;
17 }

```

Listing 10.7 close 必须唤醒所有可能等待的谓词

```

1 void BoundedQueue::close() {
2     {
3         std::lock_guard<std::mutex> lock(this->mutex_);
4         this->closed_ = true;
5     }
6     this->not_empty_.notify_all();
7     this->not_full_.notify_all();
8 }

```

close 的线性化点和幂等性 close 的线性化点是 `closed_=true`。从这一刻开始, 新的 `push` 必须失败; 已经进入队列的元素仍可被 `pop` 排空; 空队列上的 `pop` 返回 `std::nullopt`。重复 `close` 应该是安全的, 因为析构、错误路径和取消路径可能从不同层次同时尝试关闭同一个队列。重复通知通常只是性能浪费, 不应破坏语义; 若需要减少通知风暴, 可以只在第一次关闭时通知。

Listing 10.8 只在第一次关闭时广播, 语义仍保持幂等

```

1 bool BoundedQueue::mark_closed() {
2     std::lock_guard<std::mutex> lock(this->mutex_);
3     if (this->closed_) {
4         return false;
5     }
6     this->closed_ = true;
7     return true;
8 }
9
10 void BoundedQueue::close_once() {
11     const bool changed = this->mark_closed();
12     if (!changed) {
13         return;
14     }
15     this->not_empty_.notify_all();
16     this->not_full_.notify_all();
17 }

```

drain、cancel、shutdown 和析构不是同义词 很多工业事故来自把这些词混成一个布尔量。

Drain 表示拒绝新输入,但处理已接收元素;cancel 表示未开始元素应尽快退出或标记取消;shutdown 表示系统进入停止流程,可能包含 close、cancel、等待 running 任务结束和释放资源;析构要求没有线程仍可能访问对象。一个队列可以支持 close + drain, 却不支持取消已经弹出的任务;线程池可以 cancel 未开始任务,但运行中任务只能协作检查停止标志。

术语	状态承诺	常见错误
close drain	拒绝新输入,唤醒等待者 已接收元素仍会被消费	只设置 flag, 不通知所有条件变量 close 后直接丢弃队列内容却不记录
cancel	未开始工作被标记取消或移除	把运行中线程强行停止,破坏不变量
shutdown	停止接收、排空或取消、等待线程退出	只关队列,不等待 running 任务
destructor	没有并发访问者,资源可释放	析构里调用未知回调或在持锁时 join

异常安全和提交点 教学代码用 `std::int32_t`, 元素赋值不会抛出复杂异常。泛型队列则必须考虑移动构造、拷贝、分配和析构。状态提交点要清楚: 元素成功进入槽位并更新 `tail/size` 后, `push` 才对消费者可见; 元素成功移出并更新 `head/size` 后, 槽位才重新可用。若元素操作可能抛异常, 应在不改变不变量的阶段完成可能失败的工作, 或者要求元素满足 `noexcept move`, 或者使用更复杂的槽位生命周期管理。

背压是正常控制流 队列满了, 生产者等待或失败, 这不是程序坏了, 而是背压在工作。背压防止下游慢时上游无限制制造任务和占用内存。容量不是越大越好: 容量过大能隐藏短期拥塞, 但会放大排队延迟和内存峰值; 容量过小会频繁阻塞, 降低吞吐。项目报告应记录队列满次数、生产者等待时间、最大队列长度、拒绝数和关闭耗时。

从队列到线程池: unfinished count 是另一个不变量 线程池不能只看队列是否为空。Worker 从队列弹出任务后, 任务已经不在队列里, 但系统仍不 idle。线程池至少需要一个未完成计数或等价状态: 提交任务时增加, 任务函数执行结束后减少, 异常路径也必须减少。等待空闲的谓词应是“队列为空且未完成计数为零”, 关闭等待的谓词还要包含“所有 worker 已退出”。这就是为什么同步设计要从对象不变量出发, 而不是从 API 名字出发。

Listing 10.9 任务异常也必须提交 unfinished 状态转换

```

1 void ThreadPoolState::finish_task() {
2     std::lock_guard<std::mutex> lock(this->mutex_);
3     --this->unfinished_count_;
4     if (this->unfinished_count_ == 0 && this->ready_queue_empty_) {
5         this->idle_.notify_all();
6     }
7 }
```

真实代码还要保证 `finish_task` 被 RAII guard 或 `try/catch` 包住, 用户任务抛异常时也执行。若异常跳过未完成计数递减, `wait_idle` 会永久阻塞。若在持锁时执行用户任务, 则用户任务可能递归提交任务、等待 future、记录日志或抛异常, 把线程池状态机拖进未知代码。

10.4 信号量: 资源额度, 不是对象不变量

信号量表达“还有多少许可”。它适合连接池、I/O in-flight 限额、同时写文件数量、令牌桶、空槽/元素数量等资源计数。Mutex 保护复杂状态关系, semaphore 控制有多少个并发使用权。拿到许可后, 线程仍可能需要 mutex 修改共享队列; 释放许可时, 还要保证失败和取消路径归还资源。

用信号量理解 bounded buffer 有界队列可以用两个信号量理解: `empty_slots` 初始为 capacity, `filled_slots` 初始为 0。生产者先获取空槽许可, 再写入; 消费者先获取元素许可, 再读

取。环形索引仍需要 mutex 或单生产者/单消费者特化协议保护。这个模型能帮助读者看到“容量”是一种资源。

但信号量版本同样需要关闭语义。消费者阻塞在 `filled_slots.acquire()` 时，`close` 如何唤醒？生产者阻塞在 `empty_slots.acquire()` 时，关闭后如何返回失败？已经拿到许可但任务取消，许可是否归还？标准信号量本身不携带 `closed` 状态，因此还需要额外状态和取消/唤醒机制。信号量不是条件变量的万能替代品，只是更直接地表达数量资源。

令牌和队列容量不是一回事 令牌可以限制并发 I/O 数量，但不等于队列容量。一个系统可能允许队列里积压 100 个待写 block，但同时只允许 4 个 fsync in-flight；也可能允许 32 个 RPC in-flight，但请求队列容量按内存字节控制。把所有限制都写成一个队列长度，会混淆内存预算、设备并发度和业务优先级。第 15 章的异步 I/O 会复用这个区分。

C++20 同步原语的适用边界 `std::latch` 适合一次性等待 N 个参与者完成，例如主线程等待所有 worker 初始化；`std::barrier` 适合多轮阶段同步，例如每轮模拟都要等所有分片完成再进入下一轮；`std::jthread` 和 `std::stop_token` 适合把协作停止变成显式参数；`std::atomic::wait` 适合围绕单个原子状态等待。它们都不能替代队列合同：队列仍要定义容量、所有权、关闭、drain 和异常路径。

原语	适合表达	不适合替代
<code>std::latch</code>	一次性倒计时完成	可重复队列等待和关闭
<code>std::barrier</code>	多参与者多阶段推进	生产者消费者背压
<code>std::counting_semaphore</code>	资源额度、in-flight 限制	多字段不变量和生命周期
<code>std::stop_token</code>	协作停止请求	强制杀死线程或回滚已提交状态
<code>std::atomic::wait</code>	单原子状态等待	复杂谓词和多条件唤醒

10.5 锁顺序、性能路径和工业现实

单个锁保护单个队列容易证明；多个锁组合后，死锁和尾延迟变得真实。死锁的必要条件包括互斥、持有并等待、不可抢占和循环等待。工程上最常见的破坏方式是规定锁顺序，或者把组合操作改成消息传递/两阶段提交，避免同时持有多把锁。

锁顺序表 项目应维护锁顺序表。例如：全局配置锁先于任务图锁，任务图锁先于队列锁，队列锁先于指标锁。任何反向获取都应被重构或严格证明不会并发。锁顺序表还要写明哪些锁持有时禁止调用用户回调、禁止 I/O、禁止等待 future。持锁进入未知代码，比持有两把锁更危险，因为未知代码可能再进入运行时并获取另一把锁。

优先级反转和 convoy 优先级反转发生在低优先级线程持有锁，高优先级线程等待它，而中等优先级线程持续占用 CPU，使低优先级线程迟迟不能释放锁。普通服务端也有类似现象：前台请求等待后台 checkpoint 持有的锁，尾延迟被放大。Lock convoy 则是大量线程排队等待同一锁，释放后唤醒、抢占和再次竞争降低吞吐。

公平锁可以减少饥饿，但可能降低吞吐；非公平锁吞吐可能更高，但某些线程等待时间更长。同步原语的选择要服务业务目标：批处理更关注总吞吐，交互服务更关注尾延迟和饥饿边界。

mutex 的 fast path 和 futex 慢路径 Linux 上常见 mutex 在无竞争时尽量只做用户态原子状态改变；竞争严重时，线程通过 futex 进入内核睡眠，释放方再唤醒等待者。条件变量等待也会释放 mutex 并进入等待，唤醒后重新竞争 mutex。理解这点后，“锁慢”就不再是单一结论：低竞争短临界区很便宜，高竞争睡眠唤醒会涉及内核和调度，过度订阅下自旋会浪费 CPU。

同步性能要按等待原因归因 只记录吞吐不够。同步性能问题要拆成：锁等待时间、临界区执行时间、条件变量睡眠时间、被唤醒后重新竞争锁的时间、队列满导致的背压时间、队列空导致的 idle 时间、关闭广播后的退出时间。若没有这些指标，团队很容易把“下游 I/O 慢”误判成“mutex 慢”，或者把“用户任务太长”误判成“condition variable 唤醒慢”。

指标	回答的问题	典型解释
<code>lock_wait_ns</code>	拿锁前等了多久	临界区过长或锁粒度过粗
<code>critical_ns</code>	持锁执行多久	锁内做了 I/O、日志或用户代码
<code>not_empty_wait_ns</code>	消费者睡眠多久	上游产出不足或关闭未通知
<code>not_full_wait_ns</code>	生产者睡眠多久	下游慢，背压正在生效
<code>notify_to_run_ns</code>	唤醒到真正运行多久	过度订阅、调度延迟或 <code>convoy</code>
<code>close_join_ns</code>	<code>close</code> 到所有线程退出多久	运行中任务太长或退出谓词错误

工程案例

Linux futex。futex 的核心工程思想是：同步状态放在用户态地址，内核只在需要睡眠和唤醒时介入。它购买的能力是低争用 fast path 和高争用 blocking path 的分离；代价是用户态必须正确维护等待条件，避免丢失唤醒。教学项目不实现 futex，但要学习“状态检查和睡眠必须成对”。

pthread mutex/condition variable。pthread 把互斥、等待、唤醒和调度交给成熟库和内核配合处理。它购买的是可移植、可靠和经过大量边界打磨的同步语义；代价是抽象层隐藏了一些性能细节。教学代码应使用标准库 RAII 包装，而不是手写锁原语。

Linux wait queue 和 workqueue。内核 wait queue 强调等待条件和唤醒来源；workqueue 强调工作项与执行线程分离。两者共同提醒我们：等待者要围绕状态睡眠，工作项不等于线程。Compute Systems Engine 的 `QueueContract` 和 worker 退出协议应吸收这个原则。

生产级 executor。oneTBB、Folly executor、Java executor 和 Go scheduler 都把“提交工作”“执行工作”“等待完成”“关闭运行时”拆成不同协议。成熟设计通常不会让队列析构隐式杀死 worker，也不会把任务函数放在队列锁内执行。它们购买的是清楚生命周期和可观测性；代价是状态更多，测试矩阵更大。教学项目应先实现清楚的锁版 executor，再把热点迁移到更细的队列或 work stealing。

数据库后台线程。RocksDB compaction、PostgreSQL background writer、Kafka log cleaner 这类后台任务通常有队列、条件变量、停止标志、in-flight 工作和错误传播。它们最重视的不是“锁最少”，而是关闭时不丢已提交事实、故障路径可恢复、等待状态可诊断。可迁移原则是：后台线程必须能解释自己在等什么，停止请求必须能唤醒所有等待点。

RCU。RCU 通过“读者读旧版本，写者发布新版本，延迟回收旧对象”优化读多写少路径。它购买的是极轻读路径；代价是对象生命周期和回收协议复杂。教学项目可在配置快照或路由表章节借鉴“不可变版本发布”，但不要 RCU 替代需要复杂关闭语义的队列。

10.6 读写锁、RCU 和原子的边界

读写锁允许多个读者并发、写者独占，适合读临界区相对较长、写入低频、读路径不修改共享状态的对象，例如配置表或路由快照。它不适合每次请求都要更新统计、LRU、引用计数或访问时间的“伪读操作”。读临界区极短时，读写锁维护读者计数和公平策略的成本可能超过普通 mutex。

RCU 或 copy-on-write 更适合不可变快照：读者读取稳定版本，写者复制并发布新版本，旧版本延迟回收。它让读路径很轻，却把复杂度转移到写路径和内存回收。第 12 章会深入生命周期和回收问题。

Atomic 适合更窄的协议：统计计数、一次性发布、小状态机、槽位 ready 标志、单生产者单消费者 ring 的索引。Atomic 不是锁的替代品。若一个状态需要维护多字段不变量、复杂关闭和等待队列，先用 mutex/reference 写清语义，再考虑是否有足够窄的原子协议。

10.7 QueueContract: 项目中的同步合同

QueueContract 不是一个必须存在的 C++ 类，而是每个队列必须填写的工程合同。线程池队列、I/O 队列、MessageBus 队列、completion 队列可以使用不同实现，但必须回答同一组问题：容量按任务数还是字节数计算；满时阻塞、失败还是丢弃；空时等待还是返回；关闭后是否允许 drain；是否支持 timeout；元素所有权何时转移；等待谓词是什么；关闭时通知哪些等待者。

合同字段	要回答的问题	Compute Systems Engine 默认
容量	按元素、字节还是 in-flight 许可	线程池按任务数，I/O 按字节和 in-flight
满队列	阻塞、返回失败、丢弃还是背压上游	计算任务阻塞或背压，不静默丢弃
空队列	阻塞等待、返回空还是轮询	worker 阻塞等待谓词
关闭	拒绝新输入，是否 drain 已接收元素	默认 drain accepted
取消	未开始元素如何处理，运行中如何协作退出	状态记录为 canceled
所有权	push 成功后元素归谁，pop 后谁负责处理	队列只转移所有权，不执行任务
通知	哪些状态变化会唤醒哪些等待者	push/pop/close/task finish 明确列出
线性化点	操作何时对其他线程可见	push 更新 size/tail , close 设置 closed
错误传播	任务异常或写出失败如何报告	不吞异常，转成 result/error 状态
观测指标	等待、拒绝、关闭、尾延迟如何记录	每个等待点都有计数和耗时

从队列迁移到线程池 线程池比队列多两个状态：正在运行的任务和未完成计数。队列为空不代表线程池 idle，因为 worker 可能已经取出任务正在执行；队列关闭不代表任务完成，因为已接收任务可能还在 running。线程池的 **wait_idle** 谓词应围绕 unfinished count，而不是队列长度。任务抛异常时，也必须减少 unfinished count 并通知等待者。

从队列迁移到 I/O 背压 I/O 队列还要考虑字节容量和 in-flight 许可。一个 **ShuffleBlock** 可能大小差异很大，只按任务数限制会导致内存峰值失控。可靠写出还会引入 temp file、checksum、fsync、rename、manifest commit，队列只负责交付 block 描述，不能把长 I/O 放进队列锁。第 15 章会把这里的背压合同扩展成完整 I/O 管线。

实验：可关闭有界队列。实现一个 **BoundedQueue<int32_t>** 或等价教学队列。测试容量为 1、容量为 2、多个 producer/consumer、空队列 close、满队列 close、重复 close、运行中 close、消费者慢、生产者慢。每个元素包含 producer id 和 sequence，最终检查不丢、不重、每个 producer 内部顺序符合接口声明。记录 push 等待次数、pop 等待次数、最大队列长度、close 到所有线程退出的时间。

测试要攻击等待路径 并发测试应故意制造极端条件：队列容量为一，worker 数为一或二，随机 **yield**，随机短 sleep，固定 seed 重放失败，任务抛异常，close 与 push/pop 并发。大容量和大线程数有时只会掩盖状态机错误。ThreadSanitizer 可以发现数据竞争，但会改变性能，不能用于性能结论。

反例驱动的验收 本章实验不应只提交一个“正确版本”。高质量报告要包含至少三个反例：等待谓词不含 **closed_** 导致空队列 close 卡住；任务异常跳过 **unfinished_count_** 递减导致 **wait_idle** 卡住；持队列锁执行任务导致任务递归提交时死锁。每个反例都要有可复现 seed、线程数量、容量和事件日志。读者能修复反例，才说明同步原理真的进入了工程判断。

源码阅读任务

本章源码阅读按“等待为何能正确醒来”这条线组织。

1. Linux `futex`: 阅读 `kernel/futex/core.c` 相关等待和唤醒路径, 重点看用户态地址、等待队列和 `wake` 的关系, 不需要追完整优先级继承实现。
2. Linux `mutex/wait queue`: 阅读 `kernel/locking/mutex.c` 和 `include/linux/wait.h`, 说明 `fast path`、阻塞路径和唤醒条件如何分层。
3. `pthread` 或 `libc++` 同步原语: 观察 `std::condition_variable` 如何映射到底层 `pthread`, 理解标准库为什么要求配合 `std::unique_lock`。
4. 工业队列或 `executor`: 找一个有 `shutdown/drain` 语义的队列, 分析它怎样定义关闭、取消和等待者唤醒。

报告必须从一个具体失败开始, 例如“close 后消费者不醒”或“任务异常后 `wait` 永久阻塞”, 然后把源码机制和项目 `QueueContract` 对上。

验收与评分标准

本章满分 100 分:

1. 不变量说明, 15 分: 明确列出队列字段、保护锁和状态关系。
2. 条件变量谓词, 15 分: `push/pop/wait/close` 的谓词完整覆盖关闭和虚假唤醒。
3. 关闭与背压, 20 分: 区分 `close`、`drain`、`cancel`、满队列等待和拒绝策略。
4. 正确性测试, 20 分: 覆盖容量为一、多生产者多消费者、满/空 `close`、重复 `close`、异常路径和压力交错。
5. 锁边界与异常安全, 10 分: 锁内不执行任务/I/O/未知回调, 异常路径不破坏计数和不变量。
6. 性能与观测, 10 分: 记录等待次数、等待时间、最大队列长度、关闭耗时, 能解释锁慢还是下游慢。
7. 工业案例迁移, 10 分: 能从 `futex`、`pthread`、`workqueue` 或 `RCU` 中分析约束、机制、能力、代价和项目取舍。

10.8 本章小结

锁、条件变量和信号量不是三个孤立 API, 而是三种表达同步语义的工具。`Mutex` 保护不变量; `condition variable` 等待谓词; `semaphore` 表达资源额度。同步设计的质量取决于状态机是否完整、等待者是否有退出路径、关闭和异常是否覆盖、锁内是否只做可控状态转换、测试是否故意攻击边界。

本章留下的项目对象是 `QueueContract`。第 9 章的 `ThreadPoolPlan` 和 `WorkerSnapshot` 需要队列合同才能可靠 `shutdown`; 第 14 章的任务运行时会在队列之上增加 `ready count` 和 `unfinished count`; 第 15 章的 I/O 管线会把容量从任务数扩展到字节和 `in-flight`; 第 16 章以后的 `MessageBus` 会复用同样的 `close/drain/backpressure` 语义。

10.9 本章习题

习题与作业

1. 写出 `BoundedQueue<T>` 的不变量表, 并标出每个字段由哪把锁保护。解释为什么只把 `size` 改成 `atomic` 不能让队列正确。
2. 给出一个错误版本: `pop` 的等待谓词只检查 `size>0`。构造空队列 `close` 的测试, 让消费者永久睡眠, 再修复它。
3. 实现容量为一的有界队列压力测试。多个 `producer` 产生唯一 `id`, 多个 `consumer` 记录消费日志, 最终检查不丢、不重、关闭后所有线程退出。
4. 设计 `QueueContract` 文档, 分别填写 `compute task queue`、I/O `byte queue`、`MessageBus queue` 三种队列的容量、满队列、空队列、`close`、`drain` 和取消语义。

5. 分析一个锁顺序死锁：线程 A 持有队列锁后等待指标锁，线程 B 持有指标锁后等待队列锁。给出锁顺序表和重构方案。
6. 阅读 `futex` 或 `pthread condition variable` 的资料，解释为什么“修改状态”和“进入等待”必须配合，不能只靠一次 `notify`。

原子操作与内存序

先看一个在强内存模型机器上很容易被掩盖的 bug。生产者构造 `RuntimeConfig`，写入 `worker_count`、`queue_capacity` 和 `version`，然后把 `ready` 标志设为 `true`。消费者轮询 `ready`，看到 `true` 后读取配置字段。开发机上测试长期通过，于是有人认为“`ready` 已经是 `atomic`，肯定没数据竞争”。问题在于：这个 `atomic` 标志只是保证自己不被撕裂，并不自动把旁边的普通字段发布给另一个线程。若 `ready` 使用 `memory_order_relaxed`，程序缺少可移植的跨线程发布关系。

本章不把内存序讲成一张枚举表，而是讲成协议。一个原子变量必须有用途：它只是统计计数，还是发布普通数据；它是状态机线性化点，还是指针生命周期的一部分；它是否会被等待，是否会参与关闭。只有先写清这些语义，才能选择 `relaxed`、`release/acquire`、`acq_rel` 或 `seq_cst`。原子代码贵在少而准，最危险的不是不会用 `std::atomic`，而是把 `atomic` 当成“无锁万能胶”贴到复杂状态上。

学习目标

学完本章后，读者应能完成八件事：

1. 区分原子性、可见性、顺序性和对象生命周期，解释为什么 `atomic flag` 不等于完整发布协议。
2. 判断一个原子变量是指标、发布点、状态机、索引、引用计数还是等待状态。
3. 为 `relaxed` 指标写出“不发布其他数据”的使用边界，并解释多指标快照为何可能不一致。
4. 用 `release/acquire` 发布不可变对象、槽位内容或任务结果，并能写出 `happens-before` 关系。
5. 设计 CAS 状态机，说明成功点、失败内存序、`expected` 更新和不可重复副作用边界。
6. 说明原子指针、引用计数、`shared pointer`、`hazard pointer`、`epoch` 和 `RCU` 解决的是生命周期问题，不只是可见性问题。
7. 从 Linux `atomics/refcount/RCU`、C++ 标准库、Folly/Abseil 等工业设计中抽取可迁移原则。
8. 在 Compute Systems Engine 中建立 `AtomicProtocolNote`，让每个 `atomic` 都可审查、可测试、可退回锁版 `reference`。

11.1 从锁版 Reference 到原子协议

第 10 章用 `mutex` 和 `condition variable` 保护有界队列的不变量。锁的优点是把多个字段的状态转换放进同一个临界区，容易证明正确。原子操作适合更窄的同步协议：单个计数器、一次发布、一个槽位状态、一个 CAS 线性化点、一个引用计数变化。若状态本身是多字段不变量，直接把字段改成 `std::atomic` 通常只会把错误藏得更深。

Atomic 解决什么，不解决什么 `std::atomic<T>` 首先保证这个 `atomic` 对象本身的并发访问没有数据竞争，读写或读改写不可撕裂。它可以参与同步，建立发布和获取关系，也可以作为 CAS 状态机的线性化点。但它不自动保护多个变量之间的不变量，不自动让普通字段可见，不自动管理指针指向对象的生命周期，也不自动让等待者睡眠和醒来。

因此，原子化之前先问四个问题。第一，这个 `atomic` 的线性化含义是什么：计数、状态、指针、索引、版本，还是引用计数。第二，它是否发布其他普通数据。第三，读取者拿到值后，关联资源如何保持存活。第四，它是否会被当成等待条件，如果会，长时间等待靠忙等、`atomic::wait`、条件变量还是运行时调度。

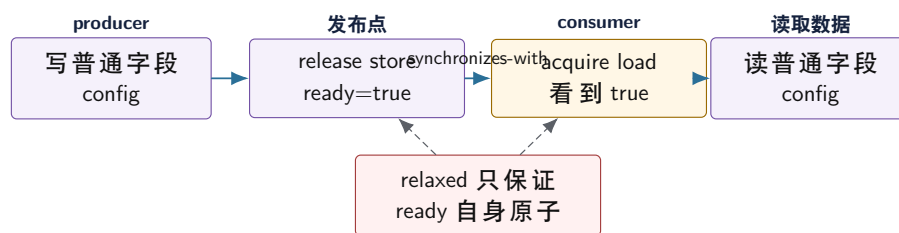
三种最常见的正确小协议 本章把原子先限制在三类小协议中。指标协议：多个线程递增计数，读取者只看数值，不借计数同步其他数据。发布协议：生产者先写普通数据，再 `release` 发布标志或指针；消费者 `acquire` 看到该发布后读取普通数据。CAS 状态机：多个线程竞争一个状态转换，CAS 成功的线程获得责任，失败线程根据新状态重新决策。

原子用途	典型内存序	必须写清	不适合做什么
指标计数	<code>relaxed</code>	只关心自身数值	发布普通数据
对象发布	<code>release/acquire</code>	发布了哪些字段，谁负责回收	热更新回收无协议
CAS 状态机	<code>acq_rel</code> / <code>acquire</code> / <code>relaxed</code>	成功点、失败路径、责任转移	循环内做外部副作用
索引/槽位	<code>release/acquire</code> 或 <code>relaxed</code> 组合	单写者是谁，槽位何时可读	多生产者随意共享
引用计数	分阶段选择	最后释放同步和对象存活条件	保护对象内部字段

11.2 内存模型的四个问题

学习原子操作要把四件事分开。原子性回答“这个变量自身的读写是否不可分割、是否有数据竞争”。可见性回答“一个线程写的普通字段，另一个线程什么时候能看到”。顺序性回答“编译器和硬件能否重排，其他线程能观察到什么顺序”。生命周期回答“读者拿到指针、引用或状态后，对象是否还活着”。很多错误来自用原子性去替代后三个问题。

发布协议：ready 标志必须同步普通字段



`release/acquire` 的作用不是“刷新所有缓存”，而是在读到对应发布值时建立 `happens-before`，让发布前的普通写对获取后的读取可见。

图示：本图要证明的命题是：atomic flag 本身没有数据竞争，不代表它同步了旁边的普通字段。

Happens-before 不是物理时间 生产者在物理时间上先写 `data`，消费者在物理时间上后读 `data`，不代表 C++ 程序有同步关系。并发正确性看的是 `happens-before`。Mutex 的 `unlock` 与后续 `lock` 可以建立同步；`release store` 与读到它的 `acquire load` 可以建立同步；线程 `join` 也有同步语义。没有这些关系，编译器和硬件有自由度，测试顺序不能替代语言保证。

Acquire 和 Release 有方向 `release` 用在发布者，把之前的普通写放在发布点之前；`acquire` 用在消费者，读到发布值后，让之后的读写不能越过获取点。把消费者的 `store` 写成 `release`，不能让它看到生产者的数据；把生产者的 `load` 写成 `acquire`，也不能发布数据。内存序必须放在建立同步关系的那次读写上。

11.3 用反例推导内存序

内存序最容易学错的方式，是先背枚举，再给每个枚举找例子。更可靠的路线是先构造反例：如果没有这条同步边，读者会看到什么坏结果。每个内存序选择都应回答一个反例。若回答不出来，要么这个 `atomic` 只是指标，可以 `relaxed`；要么协议还没有写清，不应急着用原子。

消息传递反例 最小发布协议是 message passing。生产者先写普通数据，再发布 ready；消费者看到 ready 后读取普通数据。错误版本把 ready 写成 relaxed，测试在 x86 上可能长期通过，但 C++ 语义没有保证消费者看到 data 的新值。

Listing 11.1 错误反例：relaxed ready 没有发布普通数据

```

1 struct Message {
2     std::int32_t value = 0;
3 };
4
5 Message message;
6 std::atomic<bool> ready{false};
7
8 void producer() {
9     message.value = 42;
10    ready.store(true, std::memory_order_relaxed);
11 }
12
13 void consumer() {
14     while (!ready.load(std::memory_order_relaxed)) {
15     }
16     // C++ 语义上不能保证这里看到 message.value == 42。
17     use(message.value);
18 }

```

修复不是把所有操作都改成 `seq_cst`，而是在真正传递事实的那条边上建立同步：生产者 release 存储 `ready=true`，消费者 acquire 读取到 true。这样发布前的 `message.value=42` happens-before 获取后的读取。这个例子也说明：同步关系绑定在“读到那次发布的值”上；消费者 acquire 读到 false，不代表它已经获取了消息。

Store buffering 反例 另一个经典反例是两个线程分别写自己的变量，再读对方变量。若都使用 relaxed，两个线程都可能读到旧值 0。这个结果不违反 relaxed，因为 relaxed 只保证每个 atomic 自身有修改顺序，不保证跨变量全局顺序。

Listing 11.2 Store buffering：两个 relaxed 读都看到旧值并不矛盾

```

1 std::atomic<std::int32_t> x{0};
2 std::atomic<std::int32_t> y{0};
3
4 // Thread A
5 x.store(1, std::memory_order_relaxed);
6 const std::int32_t observed_y = y.load(std::memory_order_relaxed);
7
8 // Thread B
9 y.store(1, std::memory_order_relaxed);
10 const std::int32_t observed_x = x.load(std::memory_order_relaxed);

```

若协议要求不允许 `observed_x==0&&observed_y==0`，需要更强的同步设计。可能是 `seq_cst`，也可能是引入一个锁、一个阶段 barrier、一个单独发布点或一个消息队列。内存序不是局部优化选项，而是协议的一部分；只把其中一个 load 改成 acquire 并不能凭空建立同步，因为没有对应 release 被它读到。

从反例到审查句 审查原子代码时，可以强制写一句反例句：

1. 若这个 load 读到旧值，会不会破坏业务语义？

2. 若这两个指标来自不同时间点，报告是否仍然可接受？
3. 若 CAS 失败，失败路径是否需要看到赢家发布的数据？
4. 若读者拿到指针后写者释放对象，生命周期由谁保护？
5. 若等待者睡眠，谁负责唤醒，状态变化是否可观察？

能用反例句解释的内存序，才是可维护的内存序。否则，项目应退回锁版 reference 或把状态机拆窄。

11.4 Relaxed: 只关心自身数值

memory_order_relaxed 保证原子对象自身不会数据竞争，不保证其他变量可见，不建立跨变量顺序。它适合统计计数、采样次数、CAS 失败次数、低风险指标和某些引用计数阶段。使用 relaxed 时，必须能说出一句话：读取这个数值的人不借它观察其他普通数据。

Listing 11.3 Relaxed 适合不发布其他数据的统计计数

```
1 class RequestStats {
2 public:
3     void record_success() {
4         this->success_count_.fetch_add(1, std::memory_order_relaxed);
5     }
6
7     std::uint64_t success_count() const {
8         return this->success_count_.load(std::memory_order_relaxed);
9     }
10
11 private:
12     std::atomic<std::uint64_t> success_count_{0};
13 };
```

这个类没有承诺读取 **success_count** 时能看到某个请求对象、错误对象或队列状态。它只承诺计数器自身不会丢更新。若后来有人把“计数增加”当作“对应结果对象已经发布”的信号，就发生了语义漂移。指标变量被升级成协议变量时，必须重新设计同步。

多指标快照可能不一致 监控线程分别 relaxed 读取 **accepted**、**completed** 和 **failed**，这些数值可能来自不同时间点。短时间内 **completed+failed** 小于或大于某个直觉关系，不一定是 bug，而是近似指标语义。如果业务不能接受这种不一致，需要锁保护快照、sequence lock、双缓冲、周期性汇总或单线程采样器。把每个计数都改成 **seq_cst**，也不会自动生成业务一致快照。

热点原子计数器的成本 **fetch_add** 不是免费。多个核心反复修改同一个 atomic，会争夺同一 cache line 的独占所有权，产生一致性流量。线程越多，热点越严重。高性能计数常用 per-worker 分片，写路径只写本地槽，读取时再汇总。它牺牲实时全局精确读取，换取写入热路径扩展性。第 9 章的 **WorkerSnapshot** 就应优先采用这种思路。

分片计数器的语义 分片计数器的本质是改变读取语义。写路径每个 worker 更新自己的分片，读路径汇总所有分片。这个设计适合吞吐指标、周期性报告和调试统计，不适合做“是否所有任务完成”的协议判断。若 **unfinished_count** 决定 **wait_idle** 是否返回，它必须有严格状态机；若 **processed_records** 只是监控曲线，它可以分片并允许近似。

计数用途	推荐设计	不能承诺什么
请求吞吐统计	per-worker relaxed 分片	读取瞬间的全局一致快照
CAS 失败次数	relaxed 累加或采样	与某次操作一一对应
未完成任务数	mutex 状态机或严格 atomic 协议	近似值不能驱动 wait_idle
内存预算字节	有界队列/信号量/锁保护账本	不能用滞后指标做容量判断
错误发布	result 状态 + release/acquire	单独错误计数不能发布错误对象

11.5 Release/Acquire: 发布普通数据

最适合学习 release/acquire 的场景，是一次性发布不可变对象。生产者独占构造对象，写完所有普通字段，用 release store 发布指针或状态。消费者 acquire load 读到发布值后，读取对象字段。若对象会热更新或释放，还要额外生命周期协议；release/acquire 只解决“字段写入可见”，不解决“对象何时活着”。

Listing 11.4 一次性发布不可变配置

```

1 struct RuntimeConfig {
2     std::uint32_t worker_count = 0;
3     std::uint32_t queue_capacity = 0;
4     std::uint64_t version = 0;
5 };
6
7 class OneShotConfig {
8 public:
9     void publish(const RuntimeConfig* config) {
10         this->current_.store(config, std::memory_order_release);
11     }
12
13     const RuntimeConfig* wait_current() const {
14         const RuntimeConfig* config = nullptr;
15         while (config == nullptr) {
16             config = this->current_.load(std::memory_order_acquire);
17         }
18         return config;
19     }
20
21 private:
22     std::atomic<const RuntimeConfig*> current_{nullptr};
23 };

```

这个示例刻意把生命周期简化为“配置活到进程结束”。若 `RuntimeConfig` 发布后还会被修改，代码不够；若旧配置会被释放，代码也不够。发布协议要写清边界，否则读者会误以为 acquire 读到指针后，所有后续生命周期问题都自动消失。

槽位状态发布数据 队列槽位、SPSC ring 和任务完成对象都常用“先写数据，再发布状态”的协议。生产者写入槽位内容，然后 release store 状态为 Ready；消费者 acquire load 看到 Ready 后读取槽位内容。反过来，消费者处理完槽位后 release 发布 Empty，生产者 acquire 看到 Empty 后才能复用槽位。

Listing 11.5 槽位状态发布数据

```

1 inline constexpr std::size_t BUFFER_SLOT_CAPACITY = 16;
2
3 enum class BufferState : std::int32_t {
4     Empty,
5     Ready,
6 };
7
8 struct BufferSlot {
9     std::array<std::int32_t, BUFFER_SLOT_CAPACITY> values{};
10    std::atomic<BufferState> state{BufferState::Empty};
11 };

```

```

12
13 void publish_slot(BufferSlot& slot, std::span<const std::int32_t> input) {
14     const std::size_t limit = std::min(input.size(), slot.values.size());
15     for (std::size_t index = 0; index < limit; ++index) {
16         slot.values[index] = input[index];
17     }
18     slot.state.store(BufferState::Ready, std::memory_order_release);
19 }

```

这个例子只展示生产者发布。消费者必须用 `acquire` 读取 `state`，看到 `Ready` 后再读 `values`。如果先发布 `Ready` 再写 `values`，消费者可能读到未完成数据。如果使用 `relaxed` 读取 `Ready`，语言模型上没有建立普通字段的可见性关系。

版本化发布 发布配置时，版本号通常应放进不可变配置对象中，随指针一起发布。消费者拿到 `snapshot` 后记录 `version`，日志和 `trace` 才能说明某次请求使用哪一版。不要把指针和版本分成两个独立 `atomic` 后随便读取，因为读者可能读到新指针旧版本，或旧指针新版本。若需要多字段一致快照，把字段放进同一个不可变对象，通过一个发布点发布。

原子等待：等待单一状态，不等待复杂谓词 C++20 提供 `atomic::wait` 和 `atomic::notify_one/all`。它适合等待单个 `atomic` 从某个值变成另一个值，例如任务完成状态从 `Pending` 进入终态。它不适合替代第 10 章的条件变量队列，因为队列等待谓词通常同时依赖 `size`、`closed`、容量、取消状态和未完成计数。

Listing 11.6 `atomic wait` 适合单原子状态等待

```

1 enum class CompletionState : std::int32_t {
2     Pending,
3     Succeeded,
4     Failed,
5 };
6
7 class CompletionFlag {
8 public:
9     void publish_success() {
10         this->state_.store(CompletionState::Succeeded,
11                           std::memory_order_release);
12         this->state_.notify_all();
13     }
14
15     CompletionState wait() const {
16         CompletionState state =
17             this->state_.load(std::memory_order_acquire);
18         while (state == CompletionState::Pending) {
19             this->state_.wait(state, std::memory_order_relaxed);
20             state = this->state_.load(std::memory_order_acquire);
21         }
22         return state;
23     }
24
25 private:
26     std::atomic<CompletionState> state_{CompletionState::Pending};
27 };

```

`wait` 的参数是“只要值仍等于这个旧值就睡眠”，醒来后仍要重新 `load`。发布者必须先写结果

对象，再 release 存储终态，最后通知等待者。若等待者 acquire 读到 Succeeded，才能读取结果。这个协议比忙等省 CPU，但它仍然只围绕一个 atomic 状态；复杂生命周期和取消传播仍要单独设计。

Seqlock：一致快照的另一种取舍 多字段指标若需要一致快照，但写者很少、读者很多，可以学习 seqlock 思想：写者在更新前把版本变成奇数，写完后变成偶数；读者先读版本，再读字段，再读版本，若版本变化或为奇数就重试。Seqlock 的代价是读者可能重试，且读者读取的数据类型和访问方式必须不会产生未定义行为；在 C++ 中不能随使用普通非原子字段跨线程读写。

Listing 11.7 教学版 seqlock 形状：真实 C++ 实现还需处理字段数据竞争

```

1 struct MetricsSnapshot {
2     std::uint64_t accepted = 0;
3     std::uint64_t completed = 0;
4     std::uint64_t failed = 0;
5 };
6
7 class VersionedMetrics {
8 public:
9     bool try_read(MetricsSnapshot& output) const {
10         const std::uint64_t begin =
11             this->version_.load(std::memory_order_acquire);
12         if ((begin & 1U) != 0U) {
13             return false;
14         }
15         output = this->snapshot_;
16         const std::uint64_t end =
17             this->version_.load(std::memory_order_acquire);
18         return begin == end;
19     }
20
21 private:
22     std::atomic<std::uint64_t> version_{0};
23     MetricsSnapshot snapshot_{};
24 };

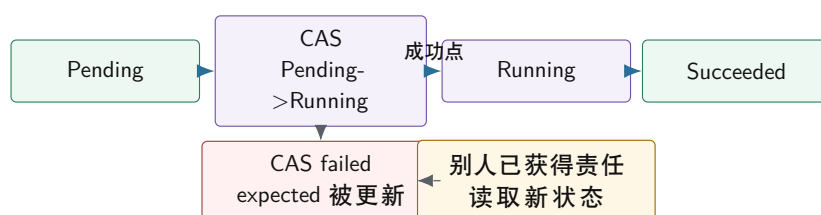
```

这段代码是语义形状，不是可直接复制的生产实现。它要读者理解一致快照要么用锁，要么用全原子字段和版本协议，要么用双缓冲不可变快照。重点不是“seqlock 很快”，而是“多字段一致性必须有协议”。

11.6 CAS：线性化点和责任转移

Compare-and-swap 读取当前值，若等于期望值就写入新值。CAS 成功的那个瞬间常常是并发对象的线性化点：从外部看，操作在这里生效。CAS 失败不是错误，而是其他线程已经改变状态；失败时 **expected** 会被写成实际观察值，调用者应根据新状态重新判断。

CAS 状态机：成功者获得责任，失败者读取事实



CAS 循环里只能准备候选状态。写文件、释放对象、提交消息、记录不可重复日志，都应放在成功之后或设计成幂等。

图示：本图要证明的命题是：CAS 成功不仅改值，还把后续责任交给成功线程；失败路径同样需要语义。

Listing 11.8 CAS 循环只在成功后提交外部副作用

```

1 bool try_set_max(std::atomic<std::int64_t>& target,
2                 std::int64_t candidate) {
3     std::int64_t observed = target.load(std::memory_order_relaxed);
4     while (observed < candidate) {
5         if (target.compare_exchange_weak(
6             observed,
7             candidate,
8             std::memory_order_release,
9             std::memory_order_relaxed)) {
10            return true;
11        }
12    }
13    return false;
14 }
  
```

失败时，`observed` 被更新为当前实际值，下一轮循环基于新值判断。函数没有在循环内部写外部日志、释放节点或提交文件，因此重试不会产生重复副作用。若调用者要记录“最大值被更新”，应在函数返回 `true` 后记录。

成功和失败内存序 `compare_exchange` 有成功内存序和失败内存序。成功时操作既读旧值又写新值，可以发布或获取状态；失败时只读取当前值，不能使用 `release`，因为没有发布新值。失败内存序也不能比成功内存序更强。常见写法是成功使用 `acq_rel` 或 `release`，失败使用 `acquire` 或 `relaxed`，取决于失败路径是否需要基于读取到的状态观察其他数据。

Weak 和 strong `compare_exchange_weak` 允许伪失败，通常放在循环里；`compare_exchange_strong` 不允许伪失败，更适合单次尝试或状态转换诊断。不要机械背“weak 快、strong 慢”。先看协议：是否本来就在循环里，失败后是否重算候选值，失败状态是否要报告给调用者。

ABA 和生命周期 CAS 比较的是位模式，不理解对象历史。一个指针从 A 变成 B 又变回 A，CAS 可能认为“没有变化”。这就是 ABA。解决方法包括标签指针、hazard pointer、epoch 回收、RCU 或避免地址复用。ABA 不是单纯内存序问题，而是状态历史和对象回收问题。第 12 章会把它放进无锁数据结构中详细展开。

CAS 状态机要有完整转移表 工业代码中，CAS 状态机不能只写成功路径。每个状态都要说明谁能进入、谁能离开、失败时返回什么、是否发布数据、是否通知等待者。没有转移表，CAS 循环很容易在取消、超时和重复完成时制造双提交。

当前状态	允许转换	成功者责任	失败者动作
Pending	Pending -> Running	获得执行权, 开始任务	读取新状态, 不能执行任务
Running	Running -> Succeeded	release 发布结果, 通知等待者	不应由其他线程改写结果
Running	Running -> Failed	release 发布错误, 通知等待者	记录重复完成或取消竞态
Pending	Pending -> Canceled	拒绝未来执行, 通知等待者	若已 Running, 只能请求协作取消
Succeeded/Failed	终态不可改	只允许读取	重复 completion 必须被识别

这个表能直接暴露设计缺口: 如果 Running 状态下收到 cancel, 是强制失败、协作取消、还是只设置 stop request? 如果 I/O completion 迟到, 终态已经存在, 迟到事件清理资源还是覆盖结果? CAS 成功点只是责任转移, 完整系统还要定义迟到、重复和失败路径。

11.7 生命周期: 可见不等于活着

发布一个指针, 有两个独立问题: 消费者能否看到对象字段, 消费者使用期间对象是否还活着。release/acquire 解决前者; 引用计数、共享所有权、hazard pointer、epoch 或 RCU 解决后者。把这两个问题混成 “atomic pointer 很安全”, 是无锁代码中最危险的误解之一。

共享所有权的教学方案 热更新配置的教学实现可以使用不可变对象加 `std::shared_ptr`。发布者构造新对象, 原子发布 shared pointer; 读者加载后持有一份 shared pointer, 本次请求期间对象保持存活。这个方案不一定是最快的, 但它把发布可见性和生命周期都交给成熟库, 适合教材和项目早期。

Listing 11.9 热更新配置使用不可变对象和共享所有权

```

1 class ConfigRegistry {
2 public:
3     void publish(std::shared_ptr<const RuntimeConfig> config) {
4         this->current_.store(std::move(config), std::memory_order_release);
5     }
6
7     std::shared_ptr<const RuntimeConfig> snapshot() const {
8         return this->current_.load(std::memory_order_acquire);
9     }
10
11 private:
12     std::atomic<std::shared_ptr<const RuntimeConfig>> current_;
13 };

```

这个设计仍要求配置对象不可变。Shared pointer 保证对象活着, 不保证对象内部字段被多个线程并发修改时安全。若配置需要更新, 构造新对象并发布新版本; 不要在旧对象上原地修改。

生命周期谱系 生命周期方案是一条谱系。永不释放旧对象最简单, 适合少量配置和教学示例, 代价是内存增长。Shared pointer 把生命周期交给引用计数, 易用但热路径有原子计数成本。Hazard pointer 让读者公开自己正在保护哪个指针, 回收精确但实现复杂。Epoch 回收按批次延迟释放, 读路径轻但回收不精确。RCU 把读侧临界区变成系统级合同, 适合读多写少, 但需要成熟运行时或内核支持。

方案	能力	代价	教学项目取舍
永不释放	最容易证明, 读者无悬垂	内存只增不减	可用于一次性配置
shared pointer	生命周期自动, 接口清楚	引用计数热点成本	推荐默认热更新
hazard pointer	精确保护单个指针	读者登记和扫描复杂	第 12 章深入
epoch	读侧低成本, 批量回收	释放延迟, 长读者阻塞回收	第 12 章深入
RCU	极轻读路径, 成熟工业实践	系统级合同和回收复杂	只借鉴思想

引用计数不是对象内部同步 引用计数只回答“对象什么时候可以释放”, 不回答“对象内部字段能否并发修改”。`std::shared_ptr<const RuntimeConfig>` 是好接口, 因为 `const` 把热更新变成发布新对象; `std::shared_ptr<RuntimeConfig>` 若多个线程拿到后原地修改字段, 仍然需要 `mutex` 或内部原子协议。很多线上 bug 不是对象提前释放, 而是对象活着但内部状态被并发写坏。

最后一次 release 的语义 引用计数还有一个微妙点: 最后一个引用释放时, 析构线程要看到对象初始化和使用期间需要看到的事实。成熟 `shared pointer` 实现已经封装了这些细节; 自研引用计数必须写清 `increment`、`decrement`、最后释放的 `acquire/release` 关系。教学项目不要自研通用引用计数, 把注意力放在“共享所有权能解决生命周期, 但不能解决内部并发语义”。

11.8 Seq-cst、Fence 和硬件差异

`memory_order_seq_cst` 提供一个所有 `seq-cst` 操作参与的单一全局顺序, 让推理更接近直觉。它适合教学、初始实现和确实需要全局顺序的协议。但它不是让并发程序自动正确的开关: 对象回收、关闭协议、多字段不变量和等待者唤醒仍要单独设计。

Fence 是更底层的工具, 把顺序约束和具体原子对象分离。除非实现底层库、移植成熟算法或有明确性能证据, 普通项目应优先使用带内存序的原子读写、`mutex`、`condition variable` 或成熟并发容器。Fence 代码更难审查, 因为读者必须追踪 fence 前后的普通访问和相关原子操作。

x86、ARM 和编译器 x86-64 内存模型相对强, 很多 `acquire/release load/store` 在硬件指令上看起来很轻; ARM 和 POWER 更弱, 可能需要显式屏障或带 `acquire/release` 语义的指令。可移植 C++ 不能因为代码在 x86 上测试通过, 就省略语言层面的同步。编译器也会根据内存序决定能否重排、缓存或消除访问。内存模型是编译器和硬件共同遵守的契约。

测试不能替代证明 内存序 bug 可能只在特定架构、优化级别、核心数和时序下出现。ThreadSanitizer 擅长发现数据竞争, 但不能证明 CAS 线性化、ABA 回收或最弱内存序正确。压力测试能增加信心, 不能穷尽交错。高质量原子代码必须有协议说明、锁版 `reference`、反例测试和架构说明。

最弱内存序不是目标 很多教材和文章喜欢追求“最弱正确内存序”。工业代码的目标不是炫耀最弱, 而是让协议可审查、性能足够好、跨平台稳定。若 `acquire/release` 与 `relaxed` 的性能差异不可测, 选择更清楚的 `acquire/release` 往往更合适。若某段代码处在每秒数十亿次热路径, 才值得用 `benchmark` 和模型证明进一步放松。每次放松都应留下原因、反例和回退方案。

选择	适合情况	审查要求
<code>mutex</code>	多字段不变量、复杂等待、生命周期复杂	不变量、谓词、锁边界清楚
<code>release/acquire</code> <code>relaxed</code>	单一发布点、槽位状态、任务终态 纯指标、单对象计数、不会发布数据	写明发布字段和读取者 写明不参与协议判断
<code>seq-cst</code>	教学初版、全局顺序要求、调试收敛	说明是否可放松
<code>fence</code>	底层库、成熟算法移植、硬件适配	需要强审查和专门测试

11.9 工业设计参照

工程案例

Linux atomic、refcount 和 RCU。Linux 区分普通 atomic 操作、引用计数安全封装和 RCU 读侧临界区。它购买的能力是低成本状态变化和极轻读路径；代价是严格的生命周期合同、架构相关屏障和大量代码审查规则。教学项目应学习分类思想：计数、状态、发布和回收不能混为一谈。

C++ 标准库。`std::atomic` 给出跨平台语言模型，`std::shared_ptr` 的原子操作给热更新提供较稳妥的生命周期方案，`atomic::wait` 为单原子状态等待提供直接工具。代价是底层细节隐藏，性能需要测量。项目早期应优先使用标准库表达正确语义，再决定是否需要更底层方案。

Folly、Abseil 和高性能基础库。这些库通常把原子封装在更高层类型中，例如原子 shared pointer、RCU 域、分片计数器、hazard pointer 或 executor 状态。它们购买的是可复用协议和集中审查；代价是引入库依赖和学习成本。Compute Systems Engine 不应照搬全部实现，但应照搬“把原子协议封装并写明合同”的做法。

数据库和流处理系统中的版本发布。DuckDB、ClickHouse、RocksDB、Flink 等系统大量使用不可变快照、版本号、manifest、epoch 或 checkpoint 来避免读者看到半更新状态。单机原子发布和分布式 manifest 发布不是同一个机制，但思想相通：先准备完整新事实，再用一个清楚提交点发布。

浏览器、游戏引擎和交易系统的热配置。这些系统常把配置、路由表、策略表或资源索引做成不可变版本。写者构造新版本，验证完整后一次发布；读者拿 snapshot，使用期间不被修改。它购买的是读路径稳定和故障回滚简单；代价是内存瞬时增加、版本回收和写路径复制成本。Compute Systems Engine 的 topology、runtime config 和 partition map 都应优先使用这种模式。

日志系统和 telemetry。高性能日志计数通常不要求每次读取都是一致快照，而是用 per-thread buffer、分片计数、批量 flush 和后台汇总。它购买的是写入热路径低成本；代价是读取滞后和近似。项目报告要区分 telemetry 指标和控制面状态，不能用近似指标驱动关闭、提交或资源释放。

11.10 项目落地：AtomicProtocolNote

本章给 Compute Systems Engine 留下 **AtomicProtocolNote**。它可以是设计文档、代码旁注释或审查表，但每个 atomic 都要能填写。若填写不出来，默认改回 mutex 或把状态机重构得更窄。

字段	要回答的问题	示例
变量用途	指标、发布、状态、索引、引用、等待	<code>completed_tasks</code> 是指标
写者/读者	谁写，谁读，是否单写者	worker 写，driver 汇总
发布数据	是否同步普通字段，字段有哪些	<code>state=Ready</code> 发布 <code>result</code>
内存序	每个 load/store/CAS 的理由	relaxed 不发布数据
线性化点	操作何时对外生效	CAS Pending->Running 成功
失败路径	CAS 失败、关闭、取消如何处理	expected 更新后分类返回
生命周期	指针或结果对象如何保持存活	shared pointer snapshot
等待策略	忙等、atomic wait、条件变量还是任务图	长等待不用忙等

三组必做实验 第一，指标实验：比较 mutex counter、atomic relaxed counter 和 sharded counter，记录吞吐、cache 热点和汇总成本。第二，发布实验：实现一次性配置发布和 shared pointer 热更新，写出 release/acquire 对应字段和生命周期边界。第三，CAS 实验：实现 `try_set_max` 或 `TaskCompletion` 状态机，记录 CAS 失败次数，证明循环内没有不可重复副作用。

反例实验必须保留 高质量实验要保留错误版本。至少保留 relaxed ready 发布配置、CAS 循环内写不可重复日志、多 atomic 读取伪一致快照、裸原子指针热更新后释放旧对象四个反例。报告不是只写“修复后通过”，而要写清：错误版本允许什么坏观察，为什么测试可能不稳定复现，修复版本用哪条同步边或生命周期协议消除了坏观察。

从锁版迁移到原子版 迁移路线要保守。先用 mutex 写 reference，测试正确性和边界；再把纯指标降为 relaxed；再实现边界清楚的 SPSC 槽位状态；最后才考虑 MPSC、MPMC 或无锁结构。每次迁移都要写 **AtomicProtocolNote**。不要从“这里有锁”直接跳到“这里用 CAS”，因为锁可能保护的是一组你还没列出的不变量。

实验：任务完成对象。实现一个小型 **TaskCompletion**：状态为 Pending、Running、Succeeded、Failed、Canceled；worker 用 CAS 从 Pending 转 Running，完成时 release 发布结果或错误；等待者 acquire 读取终态后读取结果。要求写出状态转换表、内存序理由、异常路径、取消语义、等待策略和测试矩阵。对照版本用 mutex/condition variable 实现。

源码阅读任务

本章源码阅读按原子变量用途分类。

1. Linux：阅读 atomic/refcount/RCU 相关文档或代码，区分普通计数、引用计数和 RCU 生命周期合同。
2. C++ 标准库或 libc++：观察 `std::shared_ptr`、`atomic<shared_ptr>`、`condition_variable` 与底层同步的边界。
3. Folly 或 Abseil：找一个原子封装类型，说明它怎样把裸 atomic 协议变成可审查 API。
4. 一个数据库或流处理系统：找版本化配置、manifest、checkpoint 或快照发布路径，说明它如何避免读者看到半更新状态。

报告必须从一个具体协议变量开始，而不是泛泛介绍“项目用了 atomic”。

验收与评分标准

本章满分 100 分：

1. 原子分类，15 分：项目中每个 atomic 都归类为指标、发布、状态、索引、引用或等待。
2. 发布协议，20 分：能用 release/acquire 正确发布普通字段，并写出 happens-before 与生命周期边界。
3. relaxed 边界，15 分：能说明 relaxed 变量不发布其他数据，能解释多指标快照不一致。
4. CAS 状态机，20 分：状态转换、成功点、失败内存序、**expected** 更新和副作用边界完整。
5. 生命周期，10 分：原子指针、shared pointer、hazard/epoch/RCU 的适用边界说清楚。
6. 测试与 reference，10 分：有锁版 reference、压力测试、TSan 或等价数据竞争检查，以及未验证架构说明。
7. 工业案例迁移，10 分：至少分析两个成熟系统的约束、机制、能力、代价和项目取舍。

11.11 本章小结

原子操作不是“更快的锁”，而是一组更窄、更需要协议说明的同步工具。Relaxed 适合只看自身数值的指标；release/acquire 适合发布普通数据；CAS 适合定义单点责任转移；seq-cst 适合初始推理或确实需要全局顺序的场景；原子指针必须配生命周期协议。多个 atomic 不会自动组成一致快照，atomic flag 不会自动发布对象，CAS 成功也不会自动处理 ABA 和回收。

本章真正留下的是 **AtomicProtocolNote**。第 12 章的无锁数据结构会把这些小协议组合成 SPSC ring、Treiber stack、ABA 防护和内存回收；第 14 章的任务运行时会把原子状态机用于 ready count 和 completion；第 18 章的单机提交状态会迁移到分布式 manifest 和 attempt commit。

11.12 本章习题

习题与作业

1. 给出一个 relaxed **ready** flag 发布配置的错误版本。说明为什么它没有数据竞争但仍缺少发布关系，并改成 release/acquire。
2. 为 **WorkerSnapshot** 中的计数字段设计 relaxed 或分片计数方案。说明哪些读数可以近似，哪些不能参与协议判断。
3. 实现 **try_set_max**，记录 CAS 失败次数。把日志写在 CAS 循环内和成功后各实现一次，解释语义差异。
4. 设计一个 **TaskCompletion** 状态机，用 atomic enum 表达 Pending、Running、Succeeded、Failed、Canceled。写出每个转换的线性化点和内存序。
5. 比较裸原子指针、**shared_ptr** 热更新、epoch 和 RCU 四种配置发布生命周期方案，说明各自的成本和适用场景。
6. 阅读一个工业原子封装或 RCU 实现，填写 **AtomicProtocolNote**：变量用途、写者读者、发布数据、内存序、线性化点、失败路径和生命周期。

无锁数据结构

先看一个 Treiber stack 事故。线程 T1 读取 `head` 指向节点 A，还没来得及 CAS；线程 T2 弹出 A，释放它，又从对象池分配了一个新节点，地址恰好仍是 A；T1 恢复后看到 `head` 的位模式仍等于 A，于是 CAS 成功，把一个生命周期已经变化的节点当成旧节点使用。这里没有 `mutex` 阻塞，也没有普通数据竞争，`atomic` 和 CAS 都用上了，但程序仍然错了。CAS 比较的是值，不理解对象的历史。

本章的目标不是把“无锁”塑造成更高级的写法，而是把它还原成工程合同：进展保证是什么，线性化点在哪里，内存序发布了什么，失败重试有什么成本，节点什么时候能释放，ABA 如何防护，关闭和等待由谁处理。无锁数据结构的难点通常不在少写一把锁，而在完整生命周期协议。

学习目标

学完本章后，读者应能完成八件事：

1. 区分 blocking、obstruction-free、lock-free 和 wait-free，并说明它们不等于低尾延迟。
2. 为并发结构指出线性化点，包括成功路径、满/空失败路径和关闭路径。
3. 推导 SPSC ring buffer 的所有权、不变量、release/acquire 发布和测试边界。
4. 说明 SPSC、MPSC、SPMC、MPMC 是不同问题，不能把单写者算法冒充多写者算法。
5. 用 Treiber stack 的 ABA 事故解释为什么地址相同不代表对象身份相同。
6. 比较 tagged pointer、hazard pointer、epoch reclamation、RCU 和 shared ownership 的成本与适用场景。
7. 设计无锁结构测试矩阵，覆盖唯一 ID、容量边界、随机交错、CAS 失败、节点复用和回收滞留。
8. 在 Compute Systems Engine 中填写 `LockFreeLifecycleReport`，决定何时保留 `mutex`，何时采用 SPSC，何时使用成熟库。

12.1 先问是否需要无锁

Blocking 结构可能因为一个线程持锁后暂停而阻塞其他线程。Lock-free 算法保证系统整体持续前进，但单个线程可能反复失败。Wait-free 算法保证每个线程在有限步内完成，通常实现代价更高。Obstruction-free 只在无竞争时保证进展。这些术语不是装饰，它们定义的是竞争和失败时的承诺。

无锁不是默认优化按钮 如果临界区短、竞争不高、状态复杂或需要阻塞等待，`mutex` 加 `condition variable` 往往更清楚、更省 CPU，也更容易维护。无锁结构适合边界窄、热点明确、生产者消费者模型稳定、容量和生命周期可证明的路径。高竞争 CAS 循环可能烧满 CPU，产生 cache line 迁移和尾延迟；`mutex` 在严重竞争时让线程睡眠，反而更节能。

工程决策要从 profile 和语义出发。队列操作只占端到端 2

进展保证要写具体 “没有 `mutex`”不等于 lock-free。若算法等待某个槽位从 Writing 变成 Ready，而写该槽位的线程崩溃后无人修复，其他线程可能一直等。若 push 每次都调用可能阻塞的通用 allocator，整体也不能简单宣称 lock-free。进展保证要写清依赖条件：是否允许动态分配，是否允许线程取消，是否会在队列满时返回失败，是否把阻塞等待放在结构外层。

进展保证和尾延迟是两件事 Lock-free 保证的是系统整体有线程能前进，不保证某个请求低延迟。一个线程可能在 CAS 循环里失败很多次，业务请求尾延迟仍然很差。Wait-free 保证每个线程有限步完成，但常常需要更多元数据、更多内存和更复杂的证明。Blocking `mutex` 在竞争严重时让线程睡眠，尾延迟未必总比 CAS 自旋差。工程报告不能只写“lock-free 所以快”，必须记录 CAS 失败次数、重试时间、CPU 占用、满/空返回次数和 p99/p999 延迟。

进展术语	保证	工程解释
blocking	持锁线程停住可能阻塞别人	容易证明，适合复杂状态和等待
obstruction-free	无竞争时能完成	需要外部退避或仲裁，单独价值有限
lock-free	总有某个线程能前进	单个线程可能饥饿，尾延迟仍需测量
wait-free	每个线程有限步完成	证明和实现成本最高，接口通常更窄

先定义失败语义 无锁结构常把阻塞变成失败返回。队列满时 `try_push` 返回 `false`，队列空时 `try_pop` 返回空，这不是异常，而是接口语义。调用者必须决定：重试、退避、切换队列、阻塞在外层 Channel、还是向上游施加背压。若上层没有策略，所谓 non-blocking 只会变成忙等烧 CPU。

Listing 12.1 try 接口必须配套外层策略

```

1 bool submit_with_backoff(SpscIntQueue& queue, std::int32_t value) {
2     for (std::int32_t attempt = 0; attempt < 8; ++attempt) {
3         if (queue.try_push(value)) {
4             return true;
5         }
6         std::this_thread::yield();
7     }
8     return false;
9 }

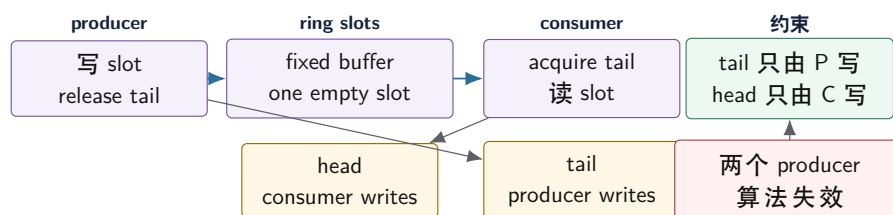
```

这段代码不是推荐最终实现，而是提醒读者：`try_push` 的 `false` 是系统设计的一部分。生产系统可能用 semaphore 睡眠、事件循环注册可写、或者把失败转成背压指标。不能在热循环里无限 `while(!try_push()){}`，否则下游慢会变成上游空转。

12.2 SPSC Ring: 最合适的起点

单生产者单消费者队列是学习无锁的最佳起点，因为所有权非常清楚：只有生产者写 `tail` 和待发布槽位，只有消费者写 `head` 和待释放槽位。生产者读取 `head` 判断空间，消费者读取 `tail` 判断数据。每个索引只有一个写者，这比多个线程 CAS 同一个端点简单得多。

SPSC ring: 单写者约束让协议变简单



SPSC 的性能来自角色限制，不来自算法能抵抗任意误用。类型名和接口合同必须写清 Single Producer / Single Consumer。

图示：本图要证明的命题是：SPSC 简单的根本原因是单写者所有权。

不变量和线性化点 常见 SPSC ring 保留一个空槽区分满和空。若 `head==tail`，队列空；若 `next(tail)==head`，队列满。生产者写入元素后 `release` 发布新 `tail`，这是 `try_push` 对消费者可见的线性化点。消费者 `acquire` 读取 `tail` 后才能读槽位，读取元素后 `release` 发布新 `head`，让生产者知道槽位可复用。

Listing 12.2 SPSC ring buffer 的核心实现

```

1 class SpscIntQueue {
2 public:
3     explicit SpscIntQueue(std::uint64_t capacity)
4         : buffer_(static_cast<std::size_t>(capacity + 1), 0) {}
5
6     bool try_push(std::int32_t value) {
7         const std::uint64_t tail =
8             this->tail_.load(std::memory_order_relaxed);
9         const std::uint64_t next_tail = this->next(tail);
10        const std::uint64_t head =
11            this->head_.load(std::memory_order_acquire);
12        if (next_tail == head) {
13            return false;
14        }
15        this->buffer_[static_cast<std::size_t>(tail)] = value;
16        this->tail_.store(next_tail, std::memory_order_release);
17        return true;
18    }
19
20    std::optional<std::int32_t> try_pop() {
21        const std::uint64_t head =
22            this->head_.load(std::memory_order_relaxed);
23        const std::uint64_t tail =
24            this->tail_.load(std::memory_order_acquire);
25        if (head == tail) {
26            return std::nullopt;
27        }
28        const std::int32_t value =
29            this->buffer_[static_cast<std::size_t>(head)];
30        this->head_.store(this->next(head), std::memory_order_release);
31        return value;
32    }
33
34 private:
35     std::uint64_t next(std::uint64_t index) const {
36         const std::uint64_t size =
37             static_cast<std::uint64_t>(this->buffer_.size());
38         return (index + 1) % size;
39     }
40
41     std::vector<std::int32_t> buffer_;
42     alignas(64) std::atomic<std::uint64_t> head_{0};
43     alignas(64) std::atomic<std::uint64_t> tail_{0};
44 };

```

这段代码只适用于一个生产者和一个消费者。两个生产者同时调用 `try_push` 会同时写 `tail` 和同一槽位；两个消费者同时调用 `try_pop` 会同时写 `head`。无锁结构的安全来自约束被遵守，不能靠模糊命名让调用者猜。

关闭放在外层 Channel 底层 SPSC ring 可以只提供非阻塞 `try_push/try_pop`。若系统需要阻塞等待、关闭、drain、取消和超时，更好的分层是外层 **Channel** 负责生命周期，底层 ring 负责快速交换。关闭不是一个可以随便塞进 ring 的 `bool`，它要唤醒等待者、拒绝新写、排空旧数据，

并与线程池 shutdown 协同。第 10 章的 `QueueContract` 仍然适用。

测试 SPSC SPSC 测试要覆盖容量 1、2、非二次幂、环绕多轮、生产者快消费者慢、消费者快生产者慢、生产者提前结束、消费者 drain。生产者生成单调序号，消费者检查严格递增和不丢不重。还要故意让两个生产者误用，确认文档和 debug 检查能暴露错误。性能报告记录吞吐、空/满返回次数、CPU 占用和 cache line 对齐影响。

容量、取模和缓存形状 SPSC ring 可以使用任意容量配合取模，也可以使用二次幂容量配合 bit mask。二次幂版本更快，但接口要明确实际可用容量和内部数组大小；保留一个空槽时，内部大小为 $\text{capacity} + 1$ ，二次幂 mask 设计则常把 sequence 或可用容量重新定义。教学版优先使用清楚的 `next(index)`，性能版才在测量后改成 mask。优化前要先证明不丢、不重、环绕正确。

false sharing 和索引缓存行 SPSC 的 `head` 和 `tail` 会被两个核心频繁读写。若它们落在同一 cache line，生产者写 `tail` 会让消费者所在核心的同一条线失效，消费者写 `head` 也会反向影响生产者。把两个索引分开对齐能减少 false sharing，但会增加对象大小。报告要测量对齐前后吞吐和 CPU 占用，不要把 `alignas(64)` 当成无条件正确。

Listing 12.3 索引对齐表达的是缓存所有权边界

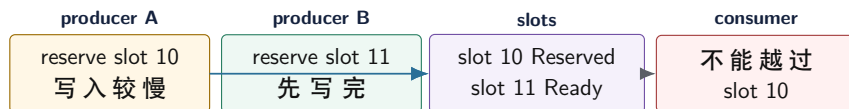
```
1 struct alignas(64) ProducerIndex {
2     std::atomic<std::uint64_t> tail{0};
3 };
4
5 struct alignas(64) ConsumerIndex {
6     std::atomic<std::uint64_t> head{0};
7 };
```

12.3 SPSC、MPSC、MPMC 不能混讲

队列难度由写者数量决定。SPSC 每端单写者。MPSC 有多个生产者争用 `tail`，消费者独占 `head`。SPMC 生产者独占 `tail`，多个消费者争用 `head`。MPMC 两端都有多个写者，通常需要每槽位序号、CAS 预留、提交状态和复杂失败路径。不要把 SPSC 加一个 CAS 就宣传成 MPMC。

MPSC 的预留和提交 MPSC 中，多个生产者可以用 CAS 或 `fetch_add` 预留槽位，但预留不等于数据已可读。生产者 A 可能预留了较早槽位却还没写完，生产者 B 预留了较后槽位并写完。消费者不能越过 A 读取 B，除非算法允许乱序并写清语义。因此 MPSC 常需要槽位状态：Reserved、Ready、Consumed，或类似序号。tail 表示预留进度，slot state 表示数据发布进度。

MPSC：预留顺序不等于提交顺序



tail 只说明槽位已被预留。消费者能否读取，要看每个槽位自己的提交状态或 sequence。

图示：本图要证明的命题是：多生产者队列必须区分预留进度和可读提交进度。

MPMC 的槽位序号模型 经典有界 MPMC 队列常让每个槽位维护 sequence。生产者根据 `tail` 找槽位，看到 sequence 表示空闲后 CAS 预留，写数据，再 release 更新 sequence 表示可读。消费者根据 `head` 找槽位，看到 sequence 表示可读后 CAS 预留读取，读数据，再更新 sequence 表示下一轮空闲。Sequence 用于区分同一数组位置的不同轮次，避免旧 Empty/Ready 被误认。

这个模型很强，但教学项目不应轻率自研生产级 MPMC。关闭、阻塞等待、动态分配、异常、

元素析构、CAS 失败风暴和性能退避都会进来。若确有 MPMC 需求，优先评估成熟库；若只是线程池全局队列，mutex bounded queue 或分片队列加 work stealing 往往更稳。

有界 MPMC 的槽位状态账本 MPMC 队列至少有三层状态：全局 **tail** 用于生产者抢预留，全局 **head** 用于消费者抢读取，每个槽位的 **sequence** 用于说明该轮次是空、可读还是已消费。全局索引只解决“谁拿到一个位置”，槽位 **sequence** 才解决“这个位置当前是哪一轮事实”。缺少 **sequence**，数组位置环绕后，旧的 Ready/Empty 状态会被误认为新一轮状态。

状态字段	解决的问题	典型错误
tail	多生产者预留哪个槽位	预留后未写完就让消费者读
head	多消费者竞争哪个槽位	两个消费者读同一元素
slot.sequence	区分环绕轮次和可读性	旧 Ready 被当成新 Ready
slot.storage	存放元素生命周期	构造失败后槽位状态无法恢复
外层 close	等待、取消、drain、阻塞策略	把关闭塞进每个槽位导致协议爆炸

成熟库也需要外层合同 成熟 MPMC 队列通常只承诺队列操作，不承诺你的业务关闭语义。库可能支持 non-blocking **try_enqueue**，但不知道你的任务是否可取消；可能支持 blocking pop，但不知道 shutdown 时是否 drain；可能要求元素 noexcept move，或者在内部长期持有内存块。引入库前要把库合同翻译成第 10 章的 **QueueContract**，否则 benchmark 再好也可能不适合项目。

12.4 Treiber 栈：短代码背后的生命周期

Treiber stack 的 push/pop 看起来短，正好适合作为反面教材：如果只展示 CAS，不讲回收，就是不完整。push 分配节点，把 **node->next** 指向旧 **head**，CAS **head** 为新节点。pop 读取旧 **head** 和 **next**，CAS **head** 为 **next**。线性化点通常是 CAS 成功，但 pop 的节点何时能释放，仍然没有答案。

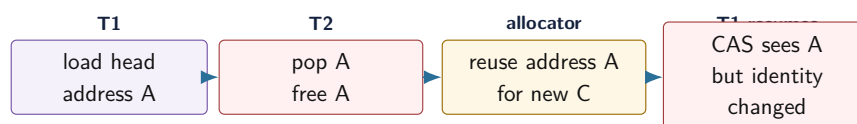
Listing 12.4 Treiber 栈的教学骨架，不包含回收协议

```
1 struct StackNode {
2     std::int32_t value = 0;
3     StackNode* next = nullptr;
4 };
5
6 class LockFreeStack {
7 public:
8     void push(StackNode* node) {
9         StackNode* observed = this->head_.load(std::memory_order_relaxed);
10        do {
11            node->next = observed;
12        } while (!this->head_.compare_exchange_weak(
13            observed,
14            node,
15            std::memory_order_release,
16            std::memory_order_relaxed));
17    }
18
19 private:
20     std::atomic<StackNode*> head_{nullptr};
21 };
```


这段代码不能作为完整栈。它没有 pop，更没有说明节点所有权。节点由调用者拥有、栈拥有、对象池拥有，还是回收系统拥有？pop 返回节点后，其他线程是否还可能持有旧指针？这些问题决定结构能否用于生产。

ABA 时间线 ABA 可以这样发生：T1 读取 `head=A`，并读取 `A->next=B`，随后暂停。T2 pop A，又 pop B，释放 A；对象池把地址 A 分给新节点 C，并 push 到栈顶。T1 恢复后看到 `head` 的地址仍是 A，CAS 成功，把 `head` 改成旧的 B。地址相同，生命周期已经不同，结构被破坏。

ABA：地址相同不代表对象身份相同



CAS 只比较位模式。若旧指针仍可能被线程持有，释放和复用地址就会制造 ABA。

图示：本图要证明的命题是：无锁节点从结构中摘除后，不能立即等同于无人访问。

Tagged pointer 和 generation 带标签指针把指针或索引与 generation 一起比较。地址 A 回到 `head` 时，generation 已变化，CAS 不会误认为状态没变。教学中更推荐用数组索引加 generation，而不是偷指针低位，因为后者依赖对齐、地址空间和平台 ABI。Generation 思想也适用于对象池、句柄系统和分布式 epoch：位置相同，不代表身份相同。

Listing 12.5 标签句柄的语义形状

```

1 struct TaggedIndex {
2     std::uint32_t index = 0;
3     std::uint32_t generation = 0;
4 };
  
```

对象池句柄比裸指针更适合教学 教学项目可以用固定数组对象池和 `TaggedIndex` 句柄模拟节点身份。数组 index 说明位置，generation 说明该位置当前是哪一代对象。释放槽位时 generation 增加，旧句柄自然失效。这个模型比裸指针更容易测试 ABA，因为读者能打印 index/generation，也能故意把对象池缩小到 2 个槽位快速复用位置。

Listing 12.6 对象池句柄检查把身份问题显式化

```

1 struct PoolHandle {
2     std::uint32_t index = 0;
3     std::uint32_t generation = 0;
4 };
5
6 struct PoolSlot {
7     std::uint32_t generation = 0;
8     StackNode node{};
9 };
10
11 bool is_live(const PoolSlot& slot, PoolHandle handle) {
12     return slot.generation == handle.generation;
13 }
  
```

真正生产级无锁栈仍需完整回收协议；tagged handle 只是让“同一位置不同身份”变得可见。

若 generation 位数太小，长时间运行后可能溢出；若旧句柄能跨很久保存，溢出后仍可能 ABA。因此报告要写 generation 位数、最大复用速率和是否接受溢出风险。

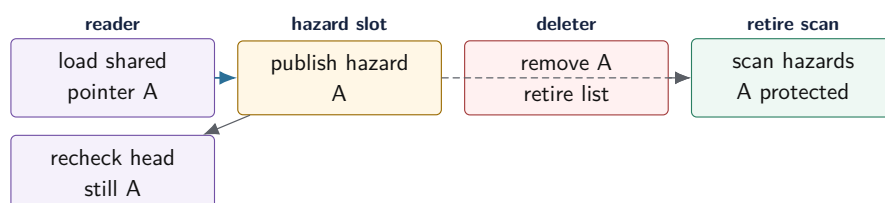
12.5 回收协议：Hazard、Epoch、RCU

无锁链式结构必须把“从结构中摘除”和“释放内存”分开。摘除说明新的入口路径到不了该节点，不说明旧操作没有持有指针。C++ 没有自动垃圾回收，必须显式证明没有线程仍可能解引用旧节点。

Hazard pointer：先声明，再解引用 Hazard pointer 的流程是：线程读取共享指针；把该指针发布到自己的 hazard 槽位；重新读取共享指针确认仍然相同；确认后才能解引用。删除者不能直接释放节点，而是放入 retired 列表；当列表足够大时，扫描所有 hazard 槽位，释放没有被任何线程声明保护的节点。

它的优点是精确，某个节点没人保护就能释放；代价是读路径要发布 hazard，释放路径要扫描所有线程槽位。线程退出时必须清空槽位，否则 retired 节点可能永远不能释放。教学项目可以固定 worker 数简化注册，但文档必须说明假设。

Hazard pointer：保护声明必须早于解引用



删除者只把节点移入 retired 列表。只有扫描不到任何 hazard 槽位保护该节点时，才能真正释放。

图示：本图要证明的命题是：无锁读者必须先公开自己可能解引用的节点，删除者才能安全延迟释放。

Epoch：批量延迟回收 Epoch 回收把时间分成阶段。线程进入无锁操作时记录当前 epoch，退出时离开。删除节点时记录退休 epoch。只有当所有线程都离开旧 epoch 后，旧 epoch 中退休的节点才能释放。它把单节点精确保护换成批量延迟回收。

Epoch 的读路径通常更轻，但害怕暂停线程。若某个线程进入 epoch 后被抢占、阻塞 I/O、执行未知回调或长时间不退出，旧节点无法释放，retired 列表和内存峰值会增长。Epoch 临界区必须短，不应包含阻塞等待和用户回调。这个约束要写进报告。

回收协议也要有指标 回收失败不会立刻表现为错误结果，常常表现为内存慢慢上涨。实验必须记录 retired 节点数量、最大退休年龄、epoch 推进次数、阻塞回收的线程 id、hazard 扫描耗时和实际释放数量。没有这些指标，epoch 泄漏会被误判为“数据结构正常但内存有点高”。

RCU：读多写少的系统合同 RCU 可以看成读多写少的发布和回收模型。读者进入 read-side 临界区，读取当前版本；写者复制新版本，修改后发布新指针，然后等待所有旧读者离开 grace period，再释放旧版本。它适合路由表、配置快照、只读索引、分片元数据。代价是写路径复制、旧版本延迟回收和系统级读侧合同。

Compute Systems Engine 不需要自研完整 RCU，但应借鉴思想：driver 发布新 route table 或 config snapshot 时，worker 读取不可变版本；旧版本在没有读者后释放。若更新频繁、对象很大或读者可能阻塞很久，RCU 思想就不一定合适。

回收方案	适合场景	主要成本	项目建议
不释放/统一释放	教学、一次性对象、短进程	内存增长	可作 reference
Tagged handle	对象池、槽位复用、ABA 检测	版本溢出、句柄检查	推荐理解身份
Hazard pointer	指针访问复杂，回收要较精确	hazard 发布和扫描	第 12 章实验选做
Epoch	操作短、线程固定、读侧要轻	暂停线程阻塞回收	可做小模型
RCU	读多写少不可变快照	写路径复制和 grace period	借鉴设计

12.6 无锁和等待、关闭、异常

很多系统需要无锁 fast path 和可阻塞 slow path。SPSC ring 的 `try_push` 满时返回 false；上层可以用 semaphore、condition variable、eventfd 或运行时任务图等待空间。底层无锁结构负责快速状态转换，上层 `Channel` 负责 close、drain、cancel、timeout 和唤醒。把所有语义塞进一个“万能无锁队列”，通常会把证明复杂度推高到不可维护。

异常和元素类型 无锁热路径最好不抛异常。若生产者 CAS 预留槽位后，元素构造抛异常，槽位状态如何回滚？消费者如何知道槽位不可读？因此教学代码使用 `std::int32_t`、指针或预构造对象，不是偷懒，而是隔离同步机制。泛型无锁容器必须写明元素要求，例如 `noexcept move`、外部构造好对象、析构无副作用，或使用复杂槽位生命周期管理。

动态分配会改变进展保证 无锁算法内部如果调用可能阻塞的通用 allocator，就不能简单宣称整个操作 lock-free。分配器可能加锁、系统调用或在内存压力下阻塞。固定数组、有界对象池、预分配节点和回收批处理，是无锁结构常见配套。性能报告必须包含分配和回收成本，否则只测了 CAS 的一半。

退避策略是协议的一部分 CAS 失败后立刻重试，在低竞争下很快；在高竞争下可能让所有线程持续打同一条 cache line。退避策略包括短暂停顿、`yield`、指数退避、切换本地队列、批量提交或回到 blocking slow path。退避不是补丁，而是负载策略。报告要记录退避次数和平均重试轮数，说明它降低了 CAS 风暴还是只是掩盖吞吐问题。

Listing 12.7 CAS 失败计数和退避让竞争可观察

```

1 bool try_transition(std::atomic<std::int32_t>& state,
2                     std::int32_t from,
3                     std::int32_t to,
4                     std::atomic<std::uint64_t>& cas_failures) {
5     std::int32_t expected = from;
6     if (state.compare_exchange_strong(expected,
7                                       to,
8                                       std::memory_order_acq_rel,
9                                       std::memory_order_acquire)) {
10         return true;
11     }
12     cas_failures.fetch_add(1, std::memory_order_relaxed);
13     return false;
14 }
```

12.7 工业设计参照

工程案例

Linux RCU 和 ring buffer。Linux RCU 把读侧临界区、发布新版本和 grace period 回收分开，购买极轻读路径；代价是系统级约束和复杂调试。内核 ring buffer 也围绕 per-cpu、提交点和可观测性设计。可迁移原则是：读写角色、提交点和回收时机必须分开写清。

Chase-Lev work stealing deque。Owner 从 bottom push/pop, stealer 从 top steal; 大多数路径单写者，只有最后一个元素需要 CAS 仲裁。它购买的是本地性和低竞争窃取；代价是 top/bottom、扩容、内存序和边界竞争复杂。第 14 章应先用 mutex deque 验证策略，再决定是否需要无锁 deque。

Moodycamel、Folly、Boost.Lockfree 等用户态库。成熟库通常明确支持 SPSC、MPSC、MPMC、有界/无界、阻塞/非阻塞、元素要求和内存回收策略。引入前要读接口合同，不要只看 benchmark。若库没有项目需要的 close/drain 语义，就要外层封装 QueueContract。

LMAX Disruptor 和序号环。Disruptor 系列设计把序号、槽位、发布和消费者进度拆开，适合低延迟流水线。它购买的是预分配、顺序访问和清楚的发布序号；代价是模型更专用，消费者拓扑和等待策略要显式设计。可迁移原则是：高性能 ring 的核心不是“数组很快”，而是每个序号的所有权、发布和消费进度都有账本。

io_uring completion queue。io_uring 的提交队列和完成队列把生产者消费者边界做成共享 ring，但业务仍要处理 request id、buffer 生命周期、取消和迟到 completion。它说明无锁或低锁 ring 只能交付事件，不能替你证明业务提交语义。第 15 章的 I/O 管线会复用这个边界。

JVM/Go 等有 GC 的运行时。垃圾回收能缓解节点回收和 ABA 的一部分风险，因为对象不会在仍可达时释放；但 CAS 线性化点、状态机、竞争失败和队列语义仍然存在。C++ 教材必须把回收显式讲出，因为运行时不会替你保底。

12.8 项目落地：LockFreeLifecycleReport

Compute Systems Engine 在本章不应把所有队列改成无锁。更好的目标是建立 LockFreeLifecycleReport：对每个候选无锁结构，写明使用场景、生产者消费者数量、容量、线性化点、进展保证、内存序、ABA 防护、回收策略、关闭外置方式、测试矩阵、性能对照和回退方案。

报告字段	要回答的问题	示例
模型	SPSC、MPSC、SPMC、MPMC、stack、deque	reader -> parser 是 SPSC
容量	有界还是无界，满时如何处理	满时返回 false, Channel 背压
线性化点	成功和失败操作何时生效	push 发布 tail
进展保证	blocking、lock-free、wait-free, 依赖什么	SPSC try 操作无阻塞分配
内存序	哪些 release/acquire 发布数据	tail 发布槽位内容
回收	固定数组、统一释放、hazard、epoch、RCU	SPSC 固定数组无需节点回收
关闭	close/drain/cancel 在哪一层实现	外层 QueueContract
指标	CAS 失败、满空次数、retired 长度、尾延迟	per-worker 汇总
回退	锁版 reference 是否保留	mutex bounded queue

推荐项目路线 第一步，保留 mutex bounded queue 作为 reference 和默认实现。第二步，在一对一管线中实现 SPSC ring，用严格测试证明不丢不重。第三步，若出现多生产者热点，优先尝试分片 SPSC 或每 worker 队列；只有 profile 证明需要时，再评估成熟 MPSC/MPMC。第四步，无锁

栈只作为 ABA 和回收教学模型，不进入核心路径。第五步，所有无锁实验必须输出 CAS 失败、满空次数、内存峰值和关闭时间。

选型矩阵 项目不应该问“要不要无锁”，而应问“哪个边界需要哪种并发结构”。单个生产者到单个消费者的高频数据流，SPSC ring 是合理候选；多个 worker 提交到一个低频控制队列，mutex queue 更稳；工作窃取 deque 适合 owner 本地 push/pop 很多、steal 较少的任务运行时；分布式消息模拟通常更需要可追踪状态和故障注入，而不是极限无锁。

场景	首选结构	理由
reader -> parser 一对一 一流	SPSC ring + 外层 Channel	角色固定，热路径清楚
全局任务提交队列	mutex bounded queue	关闭、异常、等待语义更重要
每 worker 本地任务	deque, 先锁版再优化	本地性比全局无锁更关键
多生产者日志缓冲	分片 buffer 或成熟 MPSC	避免所有线程争一个 tail
配置/路由快照	atomic shared pointer 或 RCU 思想	读多写少，不可变版本更清楚
教学 ABA 模型	小对象池 + tagged handle	容易复现和解释身份变化

实验：SPSC ring 与 mutex queue 对照。实现 `SpscIntQueue` 和锁版 `BoundedQueue<int32_t>`。用同一 producer/consumer 输入生成 0 到 N-1 的序号，验证不丢、不重、严格递增。扫描容量 1、2、3、64、非二次幂容量，记录吞吐、满/空返回次数、CPU 占用和尾延迟。再故意用两个 producer 调用 SPSC，说明接口约束被破坏后为什么不能保证正确。

实验：ABA 与回收可视化。实现一个只有 2 到 4 个槽位的小对象池，每个槽位带 generation。构造 Treiber stack 的教学模型，让一个线程暂停在读取 head 后，另一个线程 pop、free、reuse 同一 index，再让第一个线程恢复。报告分别展示裸 index、tagged index、统一释放、hazard pointer 小模型和 epoch 小模型的行为差异。验收重点不是吞吐，而是能用日志解释每个节点的身份变化和释放时机。

测试矩阵 无锁测试必须比锁版更严。测试容量一、容量二、非二次幂、wrap around、多轮循环、随机 `yield`、生产者快、消费者快、长时间压力。Treiber/节点结构测试要使用小对象池快速复用地址，制造 ABA 条件；开启 AddressSanitizer/ThreadSanitizer 时要理解工具限制；记录 retired 列表长度，防止回收静默失控。

源码阅读任务

本章源码阅读以“线性化点和回收”为主线。

1. Linux RCU：阅读 RCU 文档或 `include/linux/rcupdate.h`，说明读侧临界区、grace period 和回收如何分工。
2. Linux ring buffer：阅读 `kernel/trace/ring_buffer.c` 的高层设计，关注 per-cpu、提交点和 reader/writer 边界。
3. Chase-Lev deque 或 oneTBB work stealing：找 owner 路径和 thief 路径，说明最后一个元素竞争如何线性化。
4. 成熟用户态队列库：填写支持模型、有界/无界、关闭语义、元素要求、回收策略和测试状态。

阅读报告要说明哪些思想适合 Compute Systems Engine，哪些复杂度暂时拒绝。

验收与评分标准

本章满分 100 分：

1. 适用范围，15 分：明确 SPSC/MPSC/MPMC 或栈/deque 模型，不用模糊“FastQueue”命名。

2. 线性化点, 20 分: 成功、满、空、关闭或失败路径都有可解释的生效点。
3. 内存序和协议, 15 分: release/acquire 与发布的数据对应, relaxed 只用于安全指标或单写者本地读取。
4. 生命周期与回收, 20 分: ABA、tagged handle、hazard、epoch、RCU 或固定数组策略讲清, 不能直接 delete 未证明安全的节点。
5. 测试矩阵, 15 分: 唯一 ID、不丢不重、容量边界、随机交错、节点复用、sanitizer 和长期压力覆盖。
6. 性能证据, 10 分: 有锁版对照、CAS 失败或满空次数、内存峰值和尾延迟记录。
7. 工业案例迁移, 5 分: 至少分析一个成熟实现的约束、机制、能力、代价和项目取舍。

12.9 本章小结

无锁结构的本质不是“没有锁”，而是用更细的原子协议维护并发对象。SPSC ring 依靠单写者所有权和 release/acquire 发布；MPSC/MPMC 需要预留、提交和槽位序号；Treiber 栈展示 CAS 线性化点，也暴露 ABA 和回收难题；hazard pointer、epoch 和 RCU 都是在回答“旧节点何时安全释放”。Lock-free 保证系统整体进展，不保证每个线程低延迟；无锁代码必须可证明、可测试、可回退。

第三部分到这里形成闭环：第 9 章把线程和 worker 状态变得可见；第 10 章用锁和条件变量保护不变量与等待谓词；第 11 章把窄状态迁移成可审查的原子协议；本章告诉读者，若继续走向无锁，就必须把线性化点和生命周期一起带上。第 14 章的任务运行时与工作窃取，会复用这些边界。

12.10 本章习题

习题与作业

1. 为 `SpscIntQueue` 写不变量表：容量、head、tail、满/空判断、发布顺序和单写者约束。指出 `try_push` 和 `try_pop` 的线性化点。
2. 用两个 producer 误用 SPSC ring，说明它为什么不再正确。不要只说“文档禁止”，要指出哪个字段变成多写者。
3. 画出 Treiber stack 的 ABA 时间线，并分别说明 tagged pointer、hazard pointer 和 epoch 如何缓解不同部分的问题。
4. 设计一个对象池句柄 `TaggedIndex`，包含 index 和 generation。说明 generation 何时增加，旧句柄如何失效。
5. 比较 hazard pointer 和 epoch：读路径成本、释放延迟、暂停线程影响、线程退出处理、适用场景。
6. 填写一个 `LockFreeLifecycleReport`，决定 Compute Systems Engine 的某条一对一管线是否值得从 mutex queue 替换成 SPSC ring。

第零部分

并行算法与运行时

第 13 章

归约、扫描与分区

先看一个并行 filter 的常见事故。多个线程扫描输入，遇到满足条件的元素就把它 **push_back** 到同一个输出 vector。加锁后结果正确但很慢；改成原子递增下标后，输出位置唯一了，顺序却变成线程调度顺序；改成每个线程一个本地 vector 后，最后拼接又要说明按什么顺序拼接，失败后哪些元素有效也说不清。问题不在于没有找到“更快的容器”，而在于并行程序没有先回答：每个输出元素到底写到哪里。

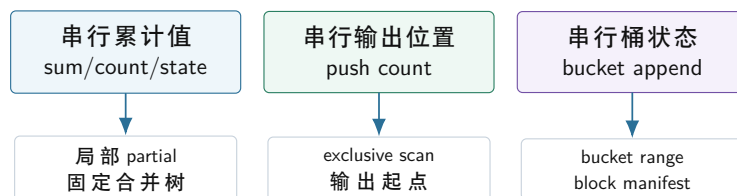
本章从这个问题进入三个基础模式：**归约** (reduce)、**扫描** (scan) 和 **分区** (partition)。Reduce 把全局状态拆成局部 partial 再合并；scan 把“前面已经输出多少”变成可计算的前缀位置；partition 把多个输出目的地的写入区间提前分配好。它们不是算法题里的孤立名词，而是数据库执行器、日志计算、图处理、排序、shuffle、checkpoint 和分布式恢复共同使用的系统骨架。

学习目标

学完本章后，读者应能完成六件事：

1. 从串行 reference 推导并行切分方式，而不是先开线程再补正确性。
2. 为 reduce 写出 identity、合并函数、确定性、浮点误差和 partial 所有权。
3. 解释 scan 为什么能把稳定 filter 的共享 **push_back** 改成无冲突写区间。
4. 为多路 partition 推导二维计数、前缀偏移、写出区间和 manifest。
5. 判断静态切分、动态调度、热点隔离、稳定输出和 cutoff 的取舍。
6. 把单机 partition 的 block 身份迁移到分布式 shuffle 和失败重做。

从串行隐含状态到并行显式结构



并行算法的关键不是线程 API，而是把串行循环里的隐含状态显式化为 partial、offset 和 manifest。

图示：本图要证明的命题是：reduce、scan、partition 都是在把串行状态改造成可合并、可定位、可恢复的结构。

13.1 从串行语义推导并行结构

并行算法不能从“开几个线程”开始。第一步永远是串行语义：输入中哪些元素参与计算，输出顺序是否重要，错误如何报告，浮点误差如何比较，输出是否可重试。串行 reference 可以慢，但它定义了问题。并行版本的每一次重排、切分和合并，都必须能回到 reference 解释。

Reference 和合同 以 **count_if** 为例，串行 reference 只做一件事：从左到右检查每个元素，满足谓词就加一。

Listing 13.1 串行 count-if reference

```
1 std::int64_t count_at_least(std::span<const std::int32_t> values,
2                             std::int32_t threshold) {
```



```

3  std::int64_t count = 0;
4  for (const std::int32_t value : values) {
5      if (value >= threshold) {
6          ++count;
7      }
8  }
9  return count;
10 }

```

这个 reference 暗含四个事实：每个输入元素被检查一次且只检查一次；谓词没有外部副作用；输出只是一个整数；整数加法是合并规则。并行版本可以把输入切成块，是因为这些事实允许局部计数和最后求和。若问题改成“输出所有满足谓词的元素并保持顺序”，算法结构立刻改变，因为输出位置不再是单个计数器。

半开区间和覆盖证明 数组切分应使用半开区间 `[begin, end)`。好的切分满足三个条件：所有块覆盖整个输入；相邻块不重叠；线程数大于元素数时仍然正确。一个常用公式如下：

Listing 13.2 按块编号计算半开区间

```

1  struct Range {
2      std::size_t begin = 0;
3      std::size_t end = 0;
4  };
5
6  Range block_range(std::size_t total,
7                  std::size_t block_index,
8                  std::size_t block_count) {
9      const std::size_t begin = (total * block_index) / block_count;
10     const std::size_t end = (total * (block_index + 1)) / block_count;
11     return Range{begin, end};
12 }

```

工程实现还要处理 `block_count==0` 和极大输入乘法溢出。教材中先用这个公式，是为了让读者看到覆盖关系：第 `k` 块的 `end` 等于第 `k+1` 块的 `begin`，第一块从 0 开始，最后一块到 `total` 结束。并行正确性的很多证明，都从这个切分证明开始。

ParallelSemanticContract 本章建议在项目中引入一个教学用合同对象，记录并行化前必须回答的问题。

Listing 13.3 ParallelSemanticContract 的教学形态

```

1  enum class OutputOrder {
2      Stable,
3      Unordered,
4      DeterministicByKey
5  };
6
7  struct ParallelSemanticContract {
8      bool has_identity = true;
9      bool associative = true;
10     bool commutative = true;
11     bool allows_floating_error = false;
12     OutputOrder output_order = OutputOrder::Stable;
13     bool retryable_by_chunk = false;
14 };

```

这个结构不是要把所有算法抽象成一个框架，而是把思考顺序固定下来。Reduce 关注 identity 和合并函数；scan 关注输出位置；partition 关注目标桶和 manifest；恢复关注 chunk 是否可重做。若合同写不清，就不应进入线程实现。

13.2 Reduce：把共享状态变成 partial

Reduce 的第一原则是避免高频共享写。多个线程对一个全局原子计数器 `fetch_add`，结果可能正确，但所有 worker 都在争同一条 cache line。线程本地 partial 把高频更新留在本地，阶段末尾再合并。这个思想在 Linux per-cpu counter、数据库 map-side combine、分布式 reduce 和高性能 telemetry 中反复出现。

Identity、结合律和确定性 Reduce 需要 identity 和合并函数。整数求和的 identity 是 0，最大值的 identity 可以是类型最小值，集合并的 identity 是空集合。若 identity 选错，空输入和空块就会错。结合律决定能否改变合并顺序；交换律决定能否重排 partial；确定性决定输出是否每次一致。

浮点求和是最好的反例。数学实数加法满足结合律，但机器浮点加法会舍入。串行左折叠、四累加器、树形合并、按 NUMA 节点分层合并，可能得到不同低位结果。若业务需要逐位可复现，就要固定合并树；若业务允许误差，就要写明绝对误差、相对误差或 ULP 边界。并行化之前必须选择语义。

Partial 的所有权 线程本地 partial 应避免 false sharing。每个 worker 写自己的槽位，最后由合并阶段读取。若 partial 很小，可以用对齐槽；若 partial 是大型 histogram 或 hash map，内存预算和合并策略就成为核心问题。

Listing 13.4 线程本地 partial 的基本形态

```

1 constexpr std::size_t CACHE_LINE_BYTES = 64;
2
3 struct alignas(CACHE_LINE_BYTES) CountPartial {
4     std::int64_t count = 0;
5 };
6
7 void count_worker(std::span<const std::int32_t> values,
8                 std::int32_t threshold,
9                 CountPartial& partial) {
10     std::int64_t local = 0;
11     for (const std::int32_t value : values) {
12         if (value >= threshold) {
13             ++local;
14         }
15     }
16     partial.count = local;
17 }

```

这个 worker 只写自己的 `partial`。正确性来自区间切分和整数加法；性能收益来自消除每个命中元素上的全局同步。代价也要写清：partial 占用额外空间，读取全局结果有延迟，最后合并可能成为新瓶颈。

线性合并和树形合并 线性合并最简单：一个线程从头到尾扫描 partial。worker 数少、partial 小时足够。树形合并把 partial 两两合并，关键路径从 $O(p)$ 降到 $O(\log p)$ ，其中 p 是 partial 数。树形合并的代价是更多阶段、更多临时对象和更复杂调度。

合并方式	适用场景	主要风险
线性合并	partial 很小, worker 数少, 简单优先	单线程尾部, 合并阶段不可并行
固定树形合并	需要可复现, partial 较多	调度自由度降低, 临时对象更多
动态树形合并	吞吐优先, partial 大小不均	浮点或顺序敏感结果可能不稳定
分层合并	NUMA 或分布式场景	层级设计复杂, 需要记录阶段边界

工业参照: per-cpu counter 和 map-side combine Linux per-cpu counter 的核心思想, 是高频写本 CPU 的局部状态, 低频读取时再汇总。它牺牲了读取的即时一致性, 换取写路径低争用。分布式计算中的 map-side combine 也是同一个思想: map worker 先本地聚合, 把网络传输从“每条记录”降低到“每个本地 key 的 partial”。优秀设计不在名字, 而在取舍: 写快了, 读或合并更复杂; 状态分散了, 观测和恢复要更清楚。

浮点归约的数值合同 整数计数让 reduce 看起来简单, 但很多工程计算是浮点。浮点归约不能只写“加法满足结合律”。有限精度下, 合并顺序改变会改变舍入路径; FMA、fast-math、SIMD 水平合并和多线程树形合并都会影响结果。正确教材必须让读者在性能之前先选择数值合同。

合同	适用场景	代价
逐位一致	审计、回归测试、可复现构建	固定合并树, 调度自由度下降
误差范围	科学计算、统计、机器学习指标	需要误差测试和边界输入
最快近似	临时监控、趋势估计	结果可能随线程数和机器变化
补偿求和	高精度求和、抵消严重输入	计算更多, 代码复杂

实验应构造四类输入: 全正均匀数、极大极小混合、正负抵消、包含 NaN 或 Inf 的边界。比较串行左折叠、固定树形、动态树形、Kahan 或 pairwise sum。报告要同时给误差、耗时、重复运行一致性和编译选项。若只给“并行版快了”, 没有说明误差, 结论不完整。

Top-k 也是 reduce Top-k 合并可以看作 reduce: 每个块产生局部候选集合, 合并函数把两个候选集合合成新的前 k。它要求稳定 tie-break 和候选覆盖。若局部阶段提前丢掉全局仍需要的 key, 后续 reduce 无法恢复。这个例子提醒读者: reduce 的合并函数不一定是加法, 也可能是集合、堆、错误列表、统计摘要或 sketch。每一种都要写明 identity、合并顺序和信息丢弃边界。

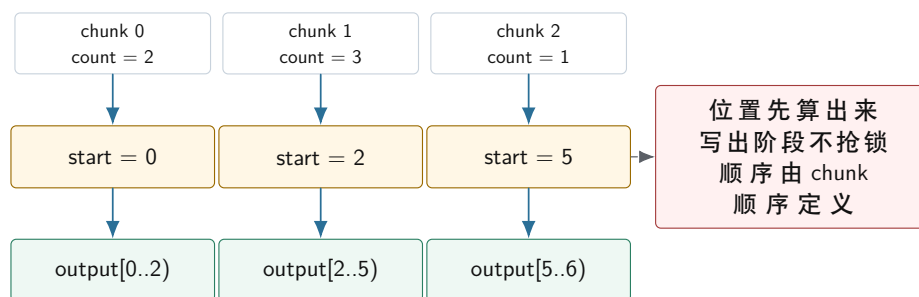
13.3 Scan: 把未知输出数量变成确定位置

Scan 也叫 prefix sum。算法题里它常被写成“计算前缀和”, 但系统工程里更重要的作用是分配输出位置。稳定 filter、stream compaction、radix partition、稀疏矩阵构建、shuffle 文件 offset, 都在回答同一个问题: 每个并行任务要写到哪里。

从 push_back 推导 scan 串行 filter 中, push_back 隐含维护了一个状态: 前面已经输出了多少元素。并行后, 这个状态不能由所有线程抢。稳定 filter 的三阶段推导如下:

1. Count: 每个 chunk 只统计自己会输出多少元素, 不写全局输出。
2. Exclusive scan: 对 chunk counts 做前缀和, 得到每个 chunk 的输出起点。
3. Write: 每个 chunk 重新扫描输入, 把保留元素写入自己的连续区间。

Stable filter: scan 把共享 push 改成独占区间



图示：本图要证明的命题是：scan 的系统价值是把“不知道输出到哪里”改造成“每个 chunk 拥有一个确定输出区间”。

Inclusive 和 exclusive Inclusive scan 的第 i 个输出包含第 i 个输入；exclusive scan 的第 i 个输出只包含它之前的输入。分配输出起点通常使用 exclusive scan。API 名称必须写清楚，不要只叫 scan。

输入	输出	语义
[3,1,4] inclusive	[3,4,8]	每个位置包含自身
[3,1,4] exclusive	[0,3,4]	每个位置是之前元素总量

Scan 的成本 并行 scan 通常比串行 scan 做更多工作。局部扫描读输入写输出，扫描 block sums，回填 offset 还可能再读写输出。小输入上串行更快；大输入上并行才可能胜出。报告必须记录输入规模、block 数、临时数组大小和阶段耗时。Scan 是强工具，但不是免费工具。

错误边界 Count 阶段和 write 阶段必须使用同一谓词版本。如果谓词依赖外部状态，count 得到的数量和 write 实际写出的数量可能不一致。工程实现应在 plan 或 manifest 中记录 predicate 版本、输入范围、expected count 和 checksum。数学上的 scan 正确，不代表系统里的 scan 一定正确。

块内位置和块间位置 前面讲的是 chunk 级输出起点。若要让每个元素都知道自己的精确输出位置，还需要块内位置。常见做法是把谓词结果变成 0/1 mask，然后对块内 mask 做 exclusive scan。元素 i 的全局输出位置等于块起点加块内 rank。这个结构同时解释了 SIMD filter、GPU compaction 和数据库 selection vector 的位置生成。

Listing 13.5 用 mask rank 表达元素输出位置

```

1 struct SelectedPosition {
2     std::size_t input_index = 0;
3     std::size_t output_index = 0;
4 };
5
6 SelectedPosition make_selected_position(std::size_t input_index,
7                                       std::size_t block_output_begin,
8                                       std::size_t rank_in_block) {
9     return SelectedPosition{input_index, block_output_begin + rank_in_block};
10 }
  
```

这个示例故意只表达位置计算，不把谓词、scan 和写出混在一起。教学项目中可以把 mask、rank、output index 分别输出到报告，帮助读者定位错误：是谓词错、块内 scan 错、块间 offset 错，还是写出错。

Selection vector 和材料化输出 Filter 后不一定立刻复制元素。若记录很大，可以输出 selection vector，只保存通过元素的下标；后续算子根据下标访问原始列。选择率低时，这能减少数据移

动；选择率高时，下标本身也占带宽，后续间接访问可能破坏局部性。若后续要顺序扫描大量字段，材料化输出可能更好。工业执行器经常在 selection vector、bitmap 和 materialized batch 之间做取舍。

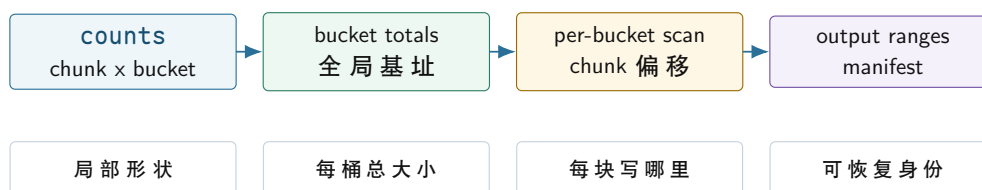
因此 scan 的结果可以服务多种输出形态：直接写紧凑数组、生成 selection vector、生成 bitmap rank、生成 block manifest。优秀实现不会把 scan 只绑定到一种容器，而是把“位置分配”作为可复用机制。

13.4 Partition：多目的地的 scan

Partition 把元素按 key、bucket 或目标 worker 分到多个输出区域。Filter 可以看成二路 partition：保留和不保留；多路 partition 则是每个 bucket 都要分配输出区间。它是单机并行算法和分布式 shuffle 之间最重要的桥。

二维计数表 多路 partition 的第一步是局部直方图：每个 chunk 统计每个 bucket 有多少元素，形成 `counts[chunk][bucket]`。第二步对每个 bucket 沿 chunk 维度做 prefix sum，得到每个 chunk 在该 bucket 中的局部起点；同时计算每个 bucket 在全局输出中的基址。两者相加，才是最终写入位置。

多路 partition 的二维前缀



图示：本图要证明的命题是：partition 是对每个 bucket 分别做 scan，manifest 是这些输出区间的工程化身份。

Manifest 是算法结果的一部分 单机 partition 可以只返回一个重排后的数组，但工程系统还需要知道每个 bucket 的范围、记录数、字节数、checksum 和生成 attempt。这些信息就是 shuffle manifest 的雏形。

Listing 13.6 PartitionManifestEntry 的教学形态

```

1 struct PartitionManifestEntry {
2     std::int32_t partition_id = 0;
3     std::uint64_t offset = 0;
4     std::uint64_t records = 0;
5     std::uint64_t bytes = 0;
6     std::uint64_t checksum = 0;
7 };
  
```

Manifest 让失败恢复和审计成为可能。某个 chunk 写出失败时，系统知道哪些输出区间无效；重复执行同一个 chunk 时，系统知道它应该写回同一段；reduce 读取时，系统知道哪些 block 已提交。没有 manifest，partition 只是一次内存重排；有了 manifest，它就能成为分布式 shuffle 的可靠边界。

稳定和非稳定 partition 稳定 partition 保持同一 bucket 内元素的原始相对顺序，适合审计、测试、可复现输出和需要稳定错误报告的场景。非稳定 partition 可以交换元素或按完成顺序写出，可能更快，但后续 stage 不能假设顺序。稳定性必须写进接口，不应由实现偶然决定。

热点 bucket Partition 解决写入位置，不自动解决负载均衡。若 70% 记录落到同一个 bucket，下游 reduce 仍会长尾。工业系统会用局部 combine、热点 key 拆分、两级 hash、范围分区或采样来缓解。报告必须记录每个 bucket 大小和最大 bucket 占比，否则无法解释后续慢在哪里。

Radix partition: histogram、scan、scatter 的组合 Radix partition 按 key 的若干 bit 分桶，常用于整数 key、哈希 join、radix sort 和数据库执行器。它的核心步骤正好是本章三件事的组合：先做局部 histogram，得到每个桶数量；再 scan 得到桶偏移；最后 scatter 到目标区间。每轮处理多少 bit 是重要参数：bit 多，桶多，counts 矩阵大，cache 压力高；bit 少，轮数多，读写次数增加。

这说明工业实现里的“参数调优”不是玄学。Radix bits 要根据 cache、TLB、输入规模、key 宽度和线程数测量。教材项目可以从 8 bit 一轮和 4 bit 一轮开始比较，记录 counts 矩阵大小、临时输出字节、总读写轮数和吞吐。读者会看到，算法复杂度相同，常数和内存布局仍能决定性能。

Partition 规则要版本化 分区函数一旦改变，旧输出和新输出不能混用。分布式系统中，如果一部分 worker 使用旧 hash seed 或旧 bucket 数，另一部分使用新规则，同一个 key 会被送到不同 reducer。单机项目也应提前训练这个习惯：manifest 记录 partition version、bucket count、hash seed 或 range boundaries。Driver 发现版本不一致时拒绝合并。

Listing 13.7 PartitionPlan 记录分区规则身份

```
1 struct PartitionPlan {
2     std::uint32_t version = 0;
3     std::uint32_t bucket_count = 0;
4     std::uint64_t hash_seed = 0;
5 };
```

这个结构看起来简单，却能防止很多恢复事故。失败重试、增量执行、缓存复用和分布式 rolling upgrade 都需要知道输出是按哪套规则生成的。没有版本，manifest 只能说明“这里有一段数据”，不能说明“这段数据属于哪套分区语义”。

内存预算 多路 partition 的额外内存来自 counts、offsets、临时输出和 manifest。若 **chunk_count** 很大、**bucket_count** 很大，二维 counts 可能很可观。报告应写出实际字节，而不是只写 $O(chunks * buckets)$ 。例如 counts 使用 `std::uint64_t`，空间就是 **chunk_count*bucket_count*8** 字节；selection vector 若使用 `std::uint32_t`，最坏是 **input_count*4** 字节。

内存预算会改变算法选择。桶数太多时，可以分批处理 bucket；热点明显时，可以先采样识别热点；输出太大时，可以按 batch 写临时 block；稳定性要求低时，可以使用局部 buffer 后合并。高质量工程不是只追求吞吐，而是在时间、空间、稳定性和恢复之间选择。

13.5 工业界设计参照

工业系统中的 reduce、scan 和 partition 很少以教科书名字出现，但它们的原理到处都是。优秀设计的共性，是把局部性、输出身份、失败恢复和确定性放在一起考虑。

Linux per-cpu counter per-cpu counter 把高频写入放到每个 CPU 的局部槽位，读取全局值时再汇总。它的原理就是线程本地 partial。它牺牲了全局读的即时精确性，换取写路径低争用。迁移到本书项目，就是 worker 本地事件计数、任务完成数和 I/O 字节数；但若计数用于协议决策，例如“所有任务是否完成”，就不能使用近似汇总。

数据库 vectorized execution DuckDB、ClickHouse、Velox 等执行器常用 batch 和 selection vector。Filter 先生成 selection vector，而不是立刻复制整行；aggregate 先本地聚合，再合并；hash partition 先统计和分配区间，再把数据送到后续 stage。它们优秀的地方不是“用了向量化”四个字，而是让数据布局、输出身份和 pipeline breaker 都有明确边界。

Spark shuffle 和 MapReduce combine Map-side combine 是 reduce 的网络尺度版本：先本地聚合，再跨网络发送 partial。Shuffle manifest 是 partition 输出身份的工程版本：每个 block 有分区、大小、位置和校验。失败重试时，driver 通过 manifest 决定哪些 block 有效，哪些 attempt 应被丢弃。第 13 章的 partition manifest，正是这个工业设计的单机缩小版。

并行 STL 和 oneTBB 并行算法库把范围切分、任务粒度、异常传播和合并策略封装起来。阅读它们时，不要只看是否开线程，而要看四件事：如何切 chunk，如何保存 partial，如何决定 cutoff，如何处理异常和取消。成熟库通常不会对所有输入强行并行，小输入会走串行或轻量路径。这是工业设计中的重要现实：并行也有固定成本。

源码阅读任务

工业案例阅读任务：选择 Linux per-cpu counter、DuckDB/ClickHouse aggregate、Spark shuffle、oneTBB parallel reduce 或 C++ 并行 STL 中一个实现。报告必须回答：它怎样切分输入；局部状态由谁拥有；合并顺序是否确定；输出身份如何记录；失败或异常如何处理；它和本章哪个实验可以对照。

13.6 确定性、审计和恢复

并行算法常被默认允许“不确定顺序”，但工程项目不能这么粗糙。某些输出只关心集合内容，顺序无所谓；某些输出需要 bitwise reproducible，方便测试、缓存、审计和恢复；某些输出允许浮点误差，但要求误差范围稳定。确定性不是装饰，而是语义合同的一部分。

测试 oracle 由输出语义决定 稳定 filter 可以逐项比较；非稳定输出要排序后比较或按多重集合比较；浮点 reduce 要按误差合同比较；partition 要比较每个 bucket 的内容和范围；manifest 要比较条目顺序、checksum 和版本。很多并行 bug 不是计算错，而是测试 oracle 和语义不匹配。

恢复边界 Scan 产生的 offsets、partition 产生的 ranges、reduce 产生的 partial，都是恢复材料。若 write 阶段失败，可以根据 chunk 的输出区间重写；若 partition block 校验失败，可以只重做对应 chunk 或 bucket；若 dynamic reduce 结果不稳定，则恢复后可能无法 bitwise 对齐。算法结构直接影响恢复成本。

失败注入实验 本章要求实现一个故障注入：count 和 scan 完成后，让某个 chunk 在 write 阶段失败。恢复程序应能知道该 chunk 的输出区间无效，其他 chunk 是否可保留由 manifest 决定。若输出由共享 `push_back` 产生，恢复程序很难定位边界。这个实验能让读者看到 scan 的系统价值，而不仅是性能价值。

13.7 深度案例：把共享 push 改造成可恢复 stage

现在把本章主线放进一个完整案例。输入是一批日志记录，目标是保留所有错误记录，并按错误码分区，供后续 reduce 统计错误类型。串行程序可以边扫描 `push_back` 到不同错误码 vector；并行后如果照搬这个结构，就会遇到锁、顺序、扩容和恢复问题。高质量设计要把它拆成四个 stage：count、scan、write、commit。

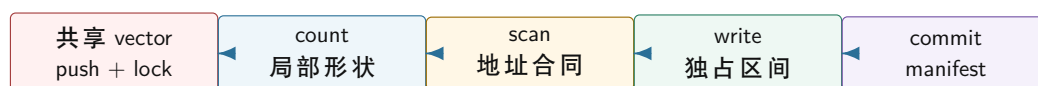
Stage 1: count 只收集形状 每个 chunk 扫描输入，统计每个错误码会输出多少条记录。它不写最终输出，只产生 `counts[chunk][error_code]`。这个阶段容易重做，也容易校验。若 count 阶段已经看到非法格式，可以把错误写入 chunk-local error buffer，而不是立刻提交全局错误报告。

Stage 2: scan 生成地址合同 对每个 error code 的 chunk counts 做 exclusive scan，得到每个 chunk 在该 error code 输出区域里的起点。这个结果就是地址合同：chunk 只能写自己的范围，不能越界，也不需要锁。若 write 阶段写出的数量和 count 不一致，说明谓词版本、输入或错误处理发生了变化，不能提交。

Stage 3: write 写入独占范围 每个 chunk 重新扫描输入，把错误记录写到分配给自己的区间。为了保持稳定顺序，chunk 按输入范围顺序分配区间，chunk 内按原始顺序写。若业务不要求稳定顺序，可以选择更快的非稳定版本，但 manifest 必须标注输出顺序语义。

Stage 4: commit 提交 manifest write 完成后，系统为每个 error code 输出一个或多个 manifest 条目：错误码、offset、records、bytes、checksum、chunk id、attempt。只有 commit 成功的条目才被后续 stage 读取。若 chunk 失败，未提交条目被丢弃或标记失败；重试 chunk 时写回同一语义范围或新的 attempt 临时范围，再由 commit 选择有效版本。

共享 push 到可恢复 stage 的演化



重构后的 stage 不只是更快，还让输出位置、顺序、校验和失败重做都有明确边界。

图示：本图要证明的命题是：scan 把并发写输出的性能问题，同时转化成可恢复的提交协议问题。

为什么这是工业级思路 这个案例的四阶段结构和数据库执行器、shuffle 写出、日志处理管线非常接近。工业系统常把正常路径和恢复路径一起设计：正常路径要少锁、顺序写、批量处理；恢复路径要知道哪个输入产生哪个输出，重复执行是否幂等，哪些输出已经提交。共享 **push_back** 的最大问题不是慢，而是它没有留下这些边界。

13.8 项目落地

Compute Systems Engine 在本章应新增一个小型并行算法套件。它不需要一次写成通用框架，但必须覆盖 reduce、scan 和 partition 的最小可靠切片。

模块	最小能力	必交证据
parallel count-if	串行 reference、global atomic 反例、partial 版本	正确性、线程扩展、共享写解释
stable filter	count、exclusive scan、write 三阶段	输出顺序、chunk counts、offsets、故障注入
multiway partition	二维 counts、per-bucket scan、manifest	每桶大小、最大倾斜、checksum、稳定性
deterministic reduce	固定合并树、误差合同	浮点误差、重复运行一致性、性能代价

输入族 输入不能只有随机均匀数组。必须覆盖空输入、一个元素、刚好一个 chunk、最后 chunk 不满、全通过、全不通过、低选择率、高选择率、单热点 bucket、Zipf 倾斜、浮点极大极小混合和失败 chunk。压力输入要攻击合同边界，而不是只扩大规模。

Cutoff 并行算法不是输入越小越应该并行。任务提交、队列调度、scan、partial、合并都有固定成本。项目应测量串行和并行交叉点，形成 cutoff。小输入走串行，大输入走并行；中等输入可能单线程 SIMD 更合适。Cutoff 不是永恒常数，而是随机器、编译器和输入分布变化的配置。

报告模板 本章实验报告固定写八段：串行语义；并行合同；切分和合并策略；输入族；阶段账本；正确性 oracle；性能结果；未验证边界。阶段账本至少包含 input records、output records、chunk count、temporary bytes、partial bytes、max bucket、checksum 和 elapsed time。

实验	必测输入	必交结论
parallel count-if	空输入、全命中、全不命中、随机命中、小输入、大输入	全局 atomic 为什么慢，partial 何时值得，cutoff 在哪里
浮点 reduce	极大极小混合、正负抵消、NaN/Inf、重复运行	误差合同、固定树形代价、fast 模式边界
stable filter	0%、1%、50%、100% 选择率，最后 chunk 不满	scan 如何分配位置，稳定顺序成本，故障注入结果
multiway partition	均匀 key、单热点、Zipf 倾斜、bucket 数变化	counts 矩阵大小、最大 bucket、manifest 是否足够恢复
radix partition	4 bit、8 bit、不同 key 宽度和输入规模	桶数、轮数、cache 压力和吞吐之间的取舍

阶段账本 每个实验都要输出阶段账本，而不是只输出总耗时。建议字段如下：

Listing 13.8 并行算法阶段账本字段

```
1 stage,input_records,output_records,chunks,buckets,temporary_bytes,
2 partial_bytes,max_bucket,checksum,elapsed_ns,mode
```

这些字段帮助读者解释现象。若并行 filter 在 1% 选择率下不快，可能是 count 和 scan 固定成本主导；若 partition 在单热点输入下慢，**max_bucket** 会暴露下游长尾；若 radix partition 的 8 bit 版本变慢，**buckets** 和 **temporary_bytes** 能说明 cache 压力。没有阶段账本，性能解释很容易变成猜测。

实验：把共享 **push_back** filter 改造成 stable filter。实现串行 reference、加锁 push 反例、每线程本地 buffer 版本、count + scan + write 版本。比较输出顺序、锁等待或替代指标、临时内存、不同选择率下的耗时，并加入一个 write 阶段故障注入。

验收与评分标准

本章大作业按 100 分评分。正确性 30 分：reference、oracle、边界输入和重复运行一致。原理推导 20 分：能从串行语义推导 partial、scan offsets 和 partition ranges。实验质量 20 分：覆盖选择率、热点、浮点误差、cutoff 和故障注入。工业迁移 15 分：能对照 per-cpu、vectorized execution、shuffle 或并行库设计讲清取舍。工程表达 15 分：代码接口位宽明确，manifest 和阶段报告可复查。

13.9 复查清单

一、**Reduce 是否写清合并语义** 检查 identity、结合性、交换性、确定性、浮点误差和 partial 所有权。若这些没有写清，就不能称为可靠并行 reduce。

二、**Scan 是否解释输出位置** 检查读者是否能从共享 **push_back** 推导出 count、exclusive scan、write 三阶段。若只会背 prefix sum 公式，说明还没学到系统意义。

三、**Partition 是否留下 manifest** 检查每个 bucket 的范围、记录数、字节数和 checksum 是否可见。没有 manifest，后续 shuffle 和恢复会断链。

四、**是否区分稳定和非稳定输出** 稳定性影响测试、审计、缓存和幂等。接口和报告必须说明是否保持顺序，以及为此付出了什么成本。

五、**是否记录资源预算** Partial、counts、offsets、本地 buffer、manifest 都占内存。报告要写实际字节，不只写复杂度。

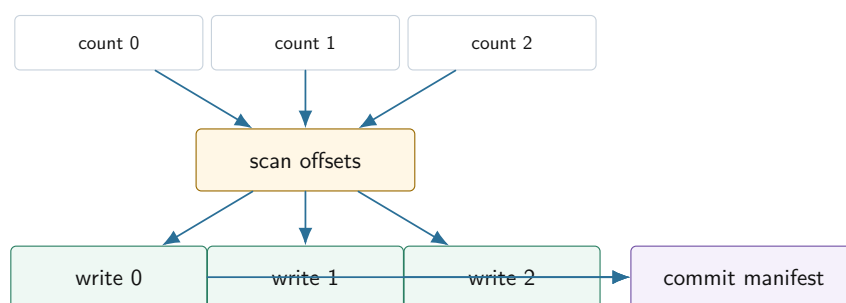
六、**是否连接下一章运行时** Reduce、scan 和 partition 都形成任务依赖。局部任务完成后才能 scan，scan 完成后才能 write，所有 partition block 提交后才能 reduce。下一章任务运行时要把这些依赖显式调度，而不是让 worker 阻塞等待。

13.10 本章和任务运行时的连接

Reduce、scan 和 partition 一旦写成 stage，就自然产生任务图。Count tasks 可以并行；scan offsets 必须等所有 count 完成；write tasks 必须等 offsets 完成；reduce tasks 必须等对应 partition block 提交。若用普通线程池，最容易写成 worker 内部阻塞等待下一阶段，造成线程池耗尽或低效 barrier。下一章的任务运行时要解决的，正是如何把这些依赖显式化，让 worker 只执行 ready task，而不是占着线程等待。

从算法阶段到任务图节点 一个 stable filter 可以拆成三类节点：count node、scan node、write node。Count nodes 只依赖输入 chunk；scan node 依赖所有 count nodes；write nodes 依赖 scan node 和对应 input chunk。Partition 多一个 bucket 维度；reduce 多一个合并树维度。这样算法结构直接变成运行时调度结构。

Stable filter 的任务依赖



图示：本图要证明的命题是：并行算法的阶段依赖应该交给任务运行时调度，而不是让 worker 线程阻塞等待。

运行时要保存哪些指标 为了让下一章能分析任务运行时，本章的实验应该输出每个 task 的 submit、ready、start、finish 时间，以及输入记录数、输出记录数和错误数。这样第 14 章就能解释 worker 空闲、队列长尾、barrier 等待和任务粒度。若第 13 章只输出总耗时，第 14 章就没有诊断材料。

从 barrier 到依赖调度 最简单实现是在每个阶段之间放 barrier：所有 count 完成后主线程做 scan，再启动所有 write。这个版本容易理解，适合教学起点。更进一步，运行时可以把依赖显式化：当最后一个 count 完成时，scan node 变 ready；scan 完成后，所有 write nodes 变 ready；write 完成后，commit node 变 ready。这种结构减少不必要等待，也为 work stealing、故障重试和背压打基础。

13.11 本章习题

习题与作业

习题一：为 parallel count-if 写完整正确性证明。证明每个输入元素被统计一次且只统计一次，partial 合并结果等于串行 reference。

习题二：设计 inclusive scan 和 exclusive scan 的 API。给出输入 `[3,1,4]` 的两个输出，并说明哪个更适合分配输出偏移。

习题三：为 stable filter 写三阶段伪代码。要求说明每阶段读写哪些数组、是否并行、是否有共享写、需要哪些临时空间。

习题四：设计一个倾斜 key 的 partition 测试。要求解释为什么写入区间没有冲突，但下游 reduce 仍可能负载不均。

习题五：比较线性合并和树形合并。给出 partial 数为 8 时的合并步骤，并说明什么时候树形合并不值得。

习题六：阅读一个工业案例，例如 per-cpu counter、vectorized aggregate 或 Spark shuffle manifest。写出它的真实约束、核心原理、取舍、和本章实验的对应关系。

任务运行时与工作窃取

先看一个很小、但足够刺痛工程师的事故：线程池有四个 worker，四个父任务都已经开始执行。每个父任务提交一个子任务后立即调用 `future.get()`。子任务已经排进同一个线程池的队列，队列里有可运行工作；可是四个 worker 都被父任务占住，正在等待这些还没有机会运行的子任务。程序没有数据竞争，锁也没有形成循环等待，CPU 甚至可能还在忙等，但作业不再前进。

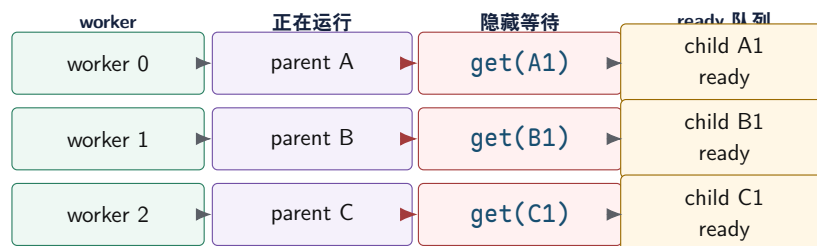
这个事故说明：线程池只知道“有函数就找线程执行”，并不自动理解任务之间的依赖。真正的任务运行时要把依赖、就绪、等待、取消、异常和关闭写成显式状态，让 worker 只执行已经 ready 的工作，而不是把宝贵的执行资源拿去等待同一个资源池里的未来工作。工作窃取也不是“更高级的线程池”这个标签，而是在任务已经 ready 之后，解决本地性、负载均衡和长尾的调度策略。

学习目标

学完本章后，读者应能完成七件事：

1. 解释为什么 worker 内部同步等待同池 future 会导致饥饿，并能构造最小复现。
2. 区分 thread、worker、task、future、task graph 和 runtime 的责任边界。
3. 用 ready count 和 continuation 把“等待”从线程调用栈改写成任务图状态。
4. 说明全局队列、本地 deque、work stealing、work helping 和 task group 的取舍。
5. 为取消、异常、关闭和任务结果生命周期写出可审查的状态机。
6. 通过 trace 判断慢在哪里：关键路径、任务粒度、队列等待、窃取成本还是隐藏阻塞。
7. 从 oneTBB、Cilk、OpenMP task、Folly executor、Linux workqueue 等工业设计中抽取可落地原则，而不是只记住项目名字。

Future 饥饿：ready 任务无人执行



队列里有 ready 子任务，但所有 worker 都被父任务的同步等待占住。修复点不是盲目加线程，而是让等待关系进入运行时状态。

图示：本图要证明的命题是：future 饥饿不是锁环，而是执行资源被隐藏等待耗尽。

14.1 从线程池事故到运行时语义

线程池的基本工作很简单：创建一组 worker 线程，外部提交函数对象，worker 从队列中取出并执行。这个模型适合互相独立的任务，例如并行压缩多个文件、处理多个互不依赖的请求、把输入数组切块后分别计算。它的危险在于接口太容易让人误以为“能提交函数”就等于“能表达并行计算”。

任务运行时比线程池多回答几类问题：某个任务依赖哪些前驱，什么时候 ready，结果由谁拥有，worker 能否阻塞等待，失败怎样传播，取消怎样收束，关闭时已入队和未入队任务分别如何处理。只要这些问题没有写清，后续加入 work stealing、priority、future、I/O completion 或分布式

attempt 都会变成难以复现的隐式控制流。

Thread、worker、task 和 runtime Thread 是操作系统调度的执行实体；worker 是运行时长期持有、用于消费任务的线程；task 是可执行工作单位；runtime 是管理 task 状态、ready 队列、worker、结果、取消和关闭协议的系统。把这些词混用，会导致错误设计。

一个 task 不应等同于一个 thread。创建一百万个 task 可以合理，创建一百万个 OS thread 通常不合理。一个 future 也不应等同于一个运行中的线程，它只是“将来会有结果或错误”的共享状态。运行时的核心价值，就是让大量 task 在有限 worker 上推进，并且让等待不必占住 worker。

最小线程池也要有状态机 很多教学线程池只有一个 **closed** 布尔值，这会把“拒绝新任务”“排空旧任务”“取消未开始任务”“worker 已经退出”混在一起。更清楚的模型至少有三个状态：

Listing 14.1 运行时生命周期的最小枚举

```
1 enum class RuntimeState : std::int32_t {
2     Open,
3     Draining,
4     Stopped,
5 };
6
7 enum class SubmitResult : std::int32_t {
8     Accepted,
9     RejectedDraining,
10    RejectedStopped,
11    QueueFull,
12 };
```

Open 允许提交新任务。**Draining** 拒绝新任务，但继续执行已经接纳的任务。**Stopped** 表示 worker 已退出，所有等待者都必须被唤醒。若支持立即取消，还要把 **shutdownrain** 和 **shutdowncancel** 分成两条路径。状态机不是形式主义，它直接决定条件变量谓词、析构安全、future 等待者能看到什么结果。

Future 的本质是共享结果状态 Future 的重要部分不是 **get()** 这个阻塞调用，而是 shared state。任务函数运行在 worker 栈上，提交者可能在另一个线程等待结果；值、异常、取消状态、完成通知和 continuation 必须放在一个拥有明确生命周期的对象里。若异常只打印到 worker 日志，调用者无法知道任务失败；若取消只设置标志但不唤醒等待者，系统会悬挂；若结果保存在已销毁的 lambda 捕获里，就会出现悬垂引用。

Listing 14.2 Future 共享状态需要表达的语义

```
1 enum class CompletionState : std::int32_t {
2     Pending,
3     Succeeded,
4     Failed,
5     Canceled,
6 };
7
8 struct CompletionRecord {
9     std::uint64_t task_id = 0;
10    CompletionState state = CompletionState::Pending;
11    std::uint32_t error_code = 0;
12    std::uint64_t result_bytes = 0;
13 };
```

这段代码不是完整 future 实现，而是在固定语义。一个可靠运行时必须能区分成功、失败和取

消；也必须能让等待者在所有终态下醒来。生产实现还要保存异常对象、结果存储、等待队列、引用计数和内存同步关系。

worker 内等待为什么危险 外部线程等待 future 通常只是让调用者停住；worker 等待同一个运行时里的 future 则可能消耗执行资源。若被等待的任务还需要 worker 执行，等待者就挡住了被等待者。这不是“future 有问题”，而是“等待发生在错误的位置”。

可选修复有三类。第一，禁止 worker 内部同步等待同池任务，把后续逻辑注册成 continuation。第二，允许 work helping：worker 等待某个 task group 时，主动执行同一组的 ready 任务。第三，把可能阻塞的任务隔离到专用执行域，例如 I/O blocking pool，不占用 compute worker。三者都可以正确，但语义和复杂度不同，必须写进接口。

Continuation 的直觉 Continuation 把“等 B 完成后继续做 A 的后半段”变成一个新的 task。B 完成时，运行时更新 shared state，递减后继的依赖计数，计数归零后把 continuation 放入 ready 队列。这样等待不占线程，worker 可以去执行其他 ready task。

Continuation 也有代价。控制流不如同步代码直观，异常和取消需要沿依赖边传播，深层 continuation 可能造成调试困难。高质量运行时不是盲目禁止同步，而是明确区分：外部同步等待可以简化调用者；worker 内同步等待必须被运行时理解，或者被接口禁止。

14.2 任务图：把等待变成状态

第 13 章的 reduce、scan、partition 已经把算法拆成阶段。count 必须先完成，scan 才能计算 offsets；scan 完成后，write 才能写入独占区间；所有 write 完成后，manifest 才能提交。这种结构天然任务是任务图。若运行时只提供 submit，程序很容易写成 barrier 或 worker 内等待；若运行时支持任务图，依赖就能成为调度事实。

ready count 任务图的节点表示可执行工作，边表示依赖。每个节点记录还有多少前驱未完成。初始计数为零的节点进入 ready 队列；某个节点完成后，遍历后继并递减它们的计数；减到零的那个线程负责把后继入队。这个计数通常叫 ready count 或 remaining predecessors。

Listing 14.3 任务图节点的核心调度字段

```
1 enum class TaskState : std::int32_t {
2     Waiting,
3     Ready,
4     Running,
5     Succeeded,
6     Failed,
7     Canceled,
8 };
9
10 struct TaskNode {
11     std::uint64_t task_id = 0;
12     std::uint32_t first_successor = 0;
13     std::uint32_t successor_count = 0;
14     std::atomic<std::uint32_t> remaining_predecessors{0};
15     std::atomic<TaskState> state{TaskState::Waiting};
16 };
```

这里直接把状态写成枚举，是为了让教学代码先表达清楚生命周期。大型运行时可能把状态压缩成整数编码，或者把热调度字段和冷诊断字段分开存放；但无论编码方式怎样，状态都必须是程序可判断的事实，而不是日志字符串。

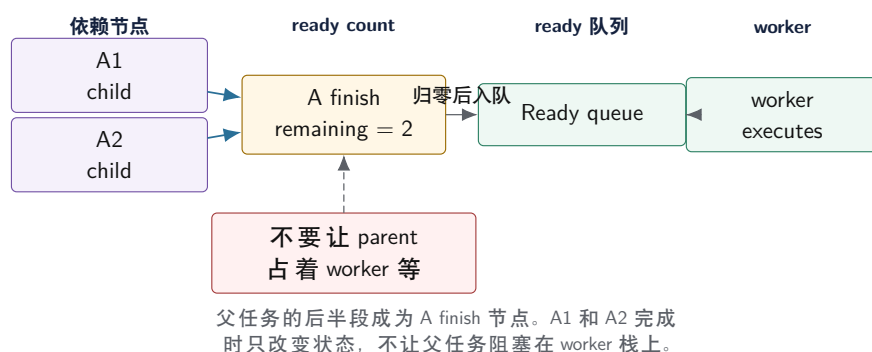
任务状态不是日志字符串 状态要能被程序判断，而不是只写在日志里。一个任务从 **Waiting** 进入 **Ready**，必须有唯一线性化点；从 **Running** 进入 **Succeeded** 或 **Failed**，必须释放结果并唤醒后继；从 **Ready** 进入 **Canceled**，worker 取出时必须能跳过它并完成计数。若状态只是“打印了一行 ready”，运行时无法防止重复入队、重复完成和取消后仍执行。

状态	进入条件	允许的后续状态
Waiting	依赖尚未满足	Ready、Canceled、Failed
Ready	ready count 归零，任务已入队	Running、Canceled
Running	worker 成功取得任务并开始执行	Succeeded、Failed、Canceled
Succeeded	用户函数正常完成，结果可见	终态
Failed	用户函数或运行时错误不可忽略	终态，触发后继策略
Canceled	上游失败、用户取消或 shutdown cancel	终态

完成路径是运行时最关键的代码 任务完成时，运行时要保存结果或异常、转换任务状态、更新 trace、递减后继 ready count、把变 ready 的任务入队、通知等待者，并在失败时触发取消传播。这些动作应集中在一个完成函数中，不能让 worker loop、future、取消逻辑各自修改状态。状态转换分散，是任务运行时最常见的错误来源。

完成路径还要避免在锁内执行未知用户代码。若一个 continuation 是用户函数，完成线程不应在持有 shared state 锁时直接调用它；应先收集需要唤醒或入队的任务，释放锁，再发布到 ready 队列。这个原则会在 I/O completion 和分布式回调中反复出现。

任务图把等待移出 worker 调用栈



图示：本图要证明的命题是：任务图不是画法，而是把等待关系变成 ready count 和后继入队规则。

结果所有权和任务所有权要分开 任务对象完成了，不表示结果对象可以释放。future 可能还没被调用者读取，trace 可能还要汇总，后继任务可能只需要错误摘要而不是完整任务对象。高性能运行时通常把热调度节点、用户结果、诊断记录和队列节点分开管理。教学实现可以简单一些，但要让读者知道为什么“任务执行完就 delete”在 future 和 continuation 存在时非常危险。

关键路径决定任务图上限 任务图性能由总工作量和关键路径共同决定。总工作量是所有任务耗时之和；关键路径是依赖图中必须串行经过的最长路径。线程数再多，也不能突破关键路径。若实际时间远高于关键路径，说明调度、同步、I/O 或负载倾斜有问题；若实际时间接近关键路径，继续增加 worker 收益有限，应优化依赖结构或关键任务本身。

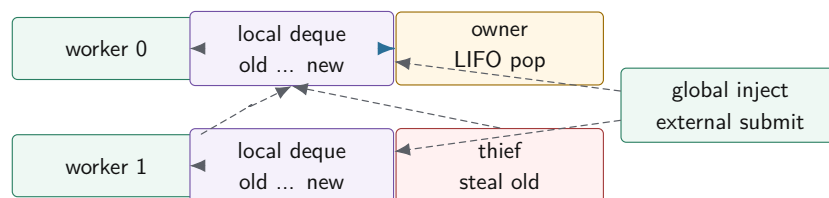
本章实验必须输出 trace，而不是只输出总耗时。Trace 至少要能估计每个任务的 ready 等待时间、运行时间和前驱关系。没有这些数据，读者无法判断慢在关键路径、队列等待、窃取失败还是任务粒度。

14.3 Work Stealing: 本地性和负载均衡的折中

全局队列是最好的第一版 reference。所有任务进入同一队列，worker 从同一队列取任务。它容易证明关闭语义，也容易测试异常和未完成计数。但全局队列有两个明显问题：所有 worker 竞争同一把锁或同一组原子位置；本地生成的子任务可能被任意 worker 拿走，破坏缓存局部性。

Work stealing 的思路是每个 worker 拥有本地 deque。本地 worker 高频 push/pop 自己的一端；空闲 worker 低频从其他 worker 的另一端 steal。这样把常见路径留在本地，把少见路径用于负载均衡。这个不对称所有权，是工作窃取的核心。

本地 deque 和窃取方向



本地 LIFO 保留刚生成任务的 cache 热度；空闲 worker 从另一端偷较老任务，通常能拿到更粗的工作块。

图示：本图要证明的命题是：work stealing 的性能来自所有权不对称，而不是来自“多几个队列”这个表象。

owner LIFO, thief FIFO Owner 从本地端按 LIFO 执行新任务，常能保持递归分治或局部数据的 cache 热度。Thief 从另一端偷较老任务，倾向拿到较大的工作块，减少偷取频率。这个策略并不是绝对真理，但它解释了很多经典运行时的设计：常见路径低开销、本地性好；不均衡时再通过窃取把远处工作拉过来。

Chase-Lev deque 的直觉边界 Chase-Lev deque 是经典工作窃取队列。它利用单 owner 的事实简化本地 push/pop，让多个 thief 通过另一个端点竞争 steal。最难的地方通常不是“数组怎么放任务”，而是边界状态：deque 只剩最后一个元素时，owner pop 和 thief steal 可能竞争同一个任务；扩容和节点回收还要保证被偷走的任务不会被 owner 同时释放。

教学项目不应一开始就实现无锁 Chase-Lev。更稳的路线是先实现全局队列 reference，再用 mutex 保护每个 worker deque 验证调度策略，最后在 trace 能证明 deque 同步成为瓶颈时再研究无锁队列。这样能把“调度策略是否有效”和“无锁协议是否正确”分开。

任务粒度决定调度是否值得 任务太小，提交、入队、出队、唤醒、trace 和 ready count 成本会吃掉计算；任务太大，最后少数长任务拖住整个 stage，work stealing 也救不了已经不可拆的长尾。一个实用经验是：任务数量要明显多于 worker 数，每个任务又要有足够计算量，使调度固定成本只占小比例。

任务粒度还要和 cache、NUMA 和取消检查点一起选择。按 cache 友好的块切分，有利于顺序访问；按过小粒度切分，有利于负载均衡但增加调度成本；长任务如果没有检查点，取消响应会很慢。项目报告应扫描 block size，而不是凭感觉写一个常数。

work helping 和 work stealing 不同 Work stealing 发生在 worker 空闲时，它去别人的队列里找 ready 任务。Work helping 发生在 worker 即将等待某个 task group 时，它不睡眠，而是帮助执行能推进该等待条件的 ready 任务。前者解决负载不均，后者缓解 worker 内等待导致的资源浪费。二者可以组合，但语义不同。

Work helping 的风险是重入。等待路径执行了另一个任务，另一个任务可能再等待更多任务，异常传播和栈增长都会复杂。教学项目可以先禁止 worker 内等待，再把 helping 作为扩展。若实现 helping，至少要限制在同一 task group 内，并用 trace 标记帮助执行事件。

work-first 和 help-first 递归分治调度常讨论 work-first 和 help-first。Work-first 倾向让当前 worker 继续沿着串行执行路径推进，把需要别人帮忙的 continuation 暴露给 thief；help-first 倾向先把子任务交给别人，当前 worker 继续后续逻辑。二者影响关键路径开销、cache 局部性、任务数量和窃取频率。

本书项目不要求复刻 Cilk 理论模型，但要让读者知道：spawn 后谁继续执行、谁进入 deque，不是无关细节。它决定本地性、关键路径和调度开销。报告中如果只写“使用 work stealing”，没有说明任务生成策略，结论不完整。

NUMA、本地性和公平 在多 NUMA 节点机器上，随机偷取可能把访问某个内存节点的数据

搬到远端执行，降低吞吐。更稳的策略是先在同 NUMA 节点内偷取，空闲时间超过阈值后再跨节点偷取。对于小计算任务，跨节点代价可能不明显；对于大数组块、hash table 或 page cache 相关任务，远端访问会成为主要成本。

公平也不是免费。严格公平可能破坏本地性；强优先级可能让低优先级任务长期饥饿。控制面任务，例如取消传播、manifest commit、错误汇总，通常很短但影响关键路径，可能需要高优先级或独立队列。运行时设计是在吞吐、尾延迟、本地性、公平和实现复杂度之间取舍。

14.4 取消、异常、关闭和可见性

高质量运行时的难点往往不在成功路径，而在任务失败、用户取消、关闭过程和结果可见性。一个只在正常输入下跑得快的线程池，不能作为教材的终点。真正可用的运行时必须让每个任务进入终态，让每个等待者醒来，让每个错误有归属，让每个输出有提交边界。

取消是协作式状态转换 取消不是强杀线程。强行终止 C++ 线程会破坏 RAII、锁状态、对象不变量和外部资源。运行时通常设置 cancel token，任务在安全点检查，例如处理完一个 chunk、完成一次 I/O、写出一个 block、进入下一轮循环之前。Waiting 或 Ready 任务可以直接标记 **Canceled**；Running 任务只能请求停止；Succeeded 或 Failed 任务已经是终态，不能再取消。

取消传播要沿任务图发生。若前驱失败，依赖它的后继可能永远不会 ready。运行时必须根据策略把后继取消、跳过、等待重试或转入补偿任务。这个策略不能由每个 worker 临时决定，应由 task graph、stage 或 driver 统一定义。

异常不能停在 worker 线程 用户任务抛异常后，worker 不能静默退出，更不能让未完成任务计数泄漏。教学版至少要捕获逃逸异常、记录 failed、减少未完成计数、唤醒等待者。future 版要把异常保存在 shared state 中，让外部 **get()** 观察。任务图版还要决定下游如何处理：取消、重试、继续使用部分结果，还是让整个 job 失败。

Listing 14.4 错误摘要比日志字符串更适合运行时

```
1 enum class TaskErrorKind : std::int32_t {
2     None,
3     UserException,
4     Canceled,
5     RejectedByShutdown,
6     ResourceLimit,
7     InternalRuntimeError,
8 };
9
10 struct TaskErrorSummary {
11     std::uint64_t task_id = 0;
12     std::uint32_t attempt = 0;
13     TaskErrorKind kind = TaskErrorKind::None;
14     std::uint32_t error_code = 0;
15 };
```

错误分类影响上层决策。用户异常可能让 stage 失败；取消是预期控制流；资源不足可能触发背压；关闭拒绝说明提交时机错误；内部运行时错误可能要求停止整个 runtime。把它们都写成 **failed**，driver 就无法做正确恢复。

Drain shutdown 和 cancel shutdown **shutdownrain** 表示停止接收新任务，但已经接纳的任务继续完成。它适合作业正常结束。**shutdowncancel** 表示未开始任务进入取消，运行中任务协作检查 token，等待者收到取消或失败结果。两者不能混成一个 **shutdown()**，否则调用者不知道结果是否应该完成。

关闭期间的 **submit** 要有线性化点。若 **submit** 先观察到 **Open** 并成功入队，任务必须被执行或取消并通知；若 **shutdown** 先把状态改成 **Draining**，后续提交必须失败。偶发“submit 返回成功但任务丢失”的运行时，是关闭协议没有定义清楚。

未完成计数的语义 `wait_idle` 不能只看队列是否为空，因为 worker 可能已经取出任务正在执行。它也不能只看 worker 是否空闲，因为提交者可能正准备入队。更可靠的做法是维护未终态任务计数：提交成功加一，任务进入 Succeeded、Failed 或 Canceled 终态时减一，计数归零时唤醒等待者。异常路径、取消路径和拒绝路径都必须审查计数是否平衡。

内存可见性 使用 mutex 和 condition variable 的第一版，发布关系通常由锁建立：提交者先写完任务对象，再在锁内入队并通知；worker 在锁内取出任务后能看到字段。无锁 ready 队列和工作窃取 deque 会把这个问题暴露出来：生产者必须先构造任务，再用 release 语义发布；消费者必须用 acquire 语义获取，才能看到任务字段。

结果发布也是同样道理。worker 写入结果后把状态设为完成，等待者读取完成状态后必须能看到结果。标准库 future/promise 封装了这些同步；自研 shared state 就要自己建立 happens-before。运行时不是只管调度，它还管结果何时对其他线程可见。

低开销观测 Trace 和 counters 是运行时的眼睛，但观测本身不能成为瓶颈。每个任务完成时更新同一个全局原子计数器，在小任务高频场景下会形成共享写热点。更好的做法是 worker 写自己的 `WorkerSnapshot` 或 per-worker buffer，周期性汇总。协议状态必须精确，例如未完成任务计数；实验指标可以延迟汇总，例如平均任务耗时和空闲时间。

Listing 14.5 运行时 trace 和 worker 指标的教学形态

```

1 struct TaskStateRecord {
2     std::uint64_t task_id = 0;
3     std::uint32_t attempt = 0;
4     std::uint32_t worker_id = 0;
5     std::uint32_t remaining_dependencies = 0;
6     std::uint64_t submit_ns = 0;
7     std::uint64_t ready_ns = 0;
8     std::uint64_t start_ns = 0;
9     std::uint64_t finish_ns = 0;
10    TaskErrorKind error_kind = TaskErrorKind::None;
11 };
12
13 struct WorkerSnapshot {
14     std::uint32_t worker_id = 0;
15     std::uint64_t executed_tasks = 0;
16     std::uint64_t local_pops = 0;
17     std::uint64_t steal_attempts = 0;
18     std::uint64_t steal_successes = 0;
19     std::uint64_t idle_ns = 0;
20     std::uint64_t running_ns = 0;
21 };

```

这些字段会在分布式章节继续扩展成 task attempt、executor、request id、epoch 和 shuffle block。单机运行时先把身份和状态打稳，后续扩展才不会换一套概念。

14.5 工业界优秀设计案例

工业系统的价值不在于名字，而在于它们在约束下做出的选择。读源码或论文时，报告应回答五个问题：它面对什么负载和失败约束；核心机制是什么；机制购买了什么能力；付出了什么成本；本书项目应该采用、简化还是拒绝哪些部分。

工程案例

Cilk 和 work-first 调度 Cilk 系列运行时把 fork-join 并行的常见路径做得很轻，窃取发生在空闲 worker 上。它的核心原则不是“有一个 deque”，而是把大部分调度开销

放到被偷取的少数路径上，让普通串行路径接近原程序执行。这个思想适合递归分治和细粒度并行，但要求任务函数遵守比较清晰的 fork-join 结构。

本书项目可以借鉴它的两个原则：一是本地执行路径要便宜，二是窃取应该暴露较粗工作而不是极小任务。不必一开始实现 Cilk 的完整 continuation 模型，也不必把所有算法都改成递归 spawn。对 stable filter、partition 和 reduce 来说，显式 task table 更利于 trace、取消和恢复。

工程案例

oneTBB 和任务粒度 oneTBB 的工程价值在于把任务划分、worker 调度、work stealing 和可组合并行算法组织在一起。它不会神奇消除任务粒度问题：parallel_for 或 flow graph 的性能仍取决于 range 切分、body 成本、数据局部性和阻塞行为。优秀设计提供机制和默认策略，但无法替业务选择语义。

本书项目借鉴 oneTBB 的方式是：先写 range、blocked range、grain size 和 trace，再决定是否 work stealing。报告要说明任务体是否足够重、是否访问连续内存、是否会阻塞 I/O。若任务只是几十条指令，换成成熟运行时也不会自动变快。

工程案例

OpenMP task 和依赖表达 OpenMP task 让程序员声明任务及其依赖，运行时在依赖满足后调度。这说明工业界也不会把所有并行性都塞进线程数量。依赖表达可以提高可读性和调度自由度，但也会引入运行时开销、调试复杂度和数据依赖声明错误的风险。本书项目不需要复制 OpenMP 语法，但要学习它的抽象：task 不是立即执行的函数调用，而是带依赖和生命周期的工作记录。对 count、scan、write、commit 来说，依赖边比 worker 内等待更清楚。

工程案例

Folly executor 和执行域 Folly 等工程库强调 executor 抽象：不同任务应进入合适的执行域。CPU-bound 任务、I/O completion、定时器、后台维护、优先级队列，资源特征不同，不能全部塞进一个 FIFO 池。这个思想对第二册非常重要，因为第 15 章会把 CPU 任务接到 I/O completion 和背压。

本书项目可以先定义 compute domain 和 blocking I/O domain。Compute worker 只执行 ready 的 CPU 任务；I/O 任务或慢等待进入独立执行域；完成事件再把后继任务放回任务图。这样慢磁盘、慢网络或模拟 I/O 不会占满 compute worker。

工程案例

Linux workqueue Linux workqueue 把“稍后执行的工作项”和“执行它的 worker pool”分开，并处理并发限制、阻塞、救援线程和 CPU 亲和性等工程问题。用户态不应照抄内核实现，但可以学习它的状态分离：工作项有身份和状态，worker pool 有资源和策略，阻塞不能让整个系统无法前进。

源码阅读时不要试图一次读完整个 workqueue。围绕一个问题进入：工作如何入队，worker 如何被唤醒，work item 如何避免重复排队，阻塞路径为什么需要救援机制。把这些问题和本章的 task state、ready queue、shutdown、trace 对照，收获会比背函数名更大。

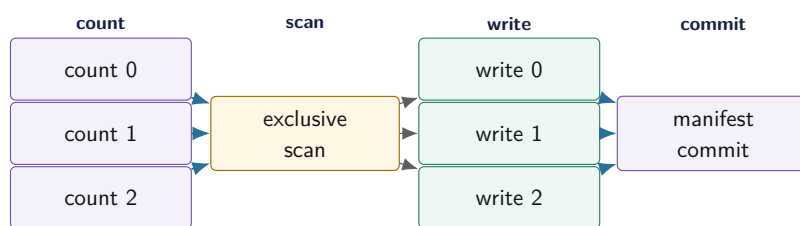
数据库执行器和分布式调度的同构性 DuckDB、ClickHouse、Spark 这类系统表面差异很大，但都在处理类似对象：可执行 fragment、依赖、worker、队列、partition、manifest、失败和重试。数据库 vectorized execution 关注批和局部性；Spark DAG scheduler 关注 stage 依赖和 task attempt；本地运行时关注 worker 和 deque。它们的共同原则是：先把工作身份和依赖写清，再谈调度策略。

14.6 深度案例：stable filter 如何进入任务运行时

第 13 章已经给出 stable filter 的算法结构：count、scan、write、commit。现在把它放入任务运行时。朴素实现会在阶段之间使用 barrier：提交所有 count，主线程等待；做 scan；再提交 write；最后等待 commit。这个版本简单、可靠，是第一版 reference。但它也暴露问题：barrier 处可能让 worker 空闲；阶段等待由主线程控制，trace 不足；失败和取消只能粗粒度处理。

任务图版本把每个阶段写成节点和边。所有 count block 初始 ready；最后一个 count 完成后，scan node ready；scan 完成后，所有 write block ready；write 完成后递减 commit 的计数；commit 发布 manifest。worker 不需要阻塞等待下一阶段，运行时能看到每个节点处于 Waiting、Ready、Running 还是终态。

Stable filter 的运行时 DAG



DAG 让运行时知道谁在等谁。失败、取消、重试和 trace 都沿节点身份传播，而不是散落在 barrier 和 worker 调用栈里。

图示：本图要证明的命题是：第 13 章的算法阶段一旦进入工程系统，就应该变成任务图节点和提交边界。

barrier reference 仍然要保留 任务图不是为了否定 barrier。Barrier 版本简单，适合作为 reference 和教学起点。它帮助验证输出、manifest、错误处理和基础性能。任务图版本必须和 barrier reference 在语义上对齐：同样输入、同样谓词、同样稳定顺序、同样 manifest。没有 reference，任务图变快或变慢都不好解释。

失败路径 若某个 count 失败，scan 不应永远 waiting。运行时应把 scan 和后续 write、commit 标记为 canceled 或让 driver 发起重试。若某个 write 失败，commit 不能发布 manifest。若 write 已经产生临时输出，commit 前崩溃恢复时必须能清理未提交文件。这正是任务运行时和第 15 章 I/O 提交协议的连接点。

长尾和 work stealing 如果一个 write block 因输入倾斜变得很慢，work stealing 只能帮助尚未开始或可拆分的工作。已经运行且不可拆的长任务，偷不走。解决方式可能是更细粒度切分、热点 block 特殊处理、动态拆分剩余范围，或在下一轮根据历史统计调整块大小。Work stealing 不是长尾万能药，它需要算法暴露可偷取的剩余工作。

关键路径报告 Stable filter DAG 的关键路径大致是最长 count、scan、最长 write、commit 之和。若 trace 显示端到端时间远高于这个估计，检查 ready 队列等待、worker 空闲、窃取失败和 I/O 等待。若接近关键路径，则优化方向是加速 scan、拆分长 write、批量 commit 或改变算法依赖，而不是继续加 worker。

14.7 项目落地

Compute Systems Engine 的任务运行时不要一次写成完整工业框架。正确路线是小步推进，每一步都有 reference、测试和指标。

第一步：全局队列 reference 实现固定 worker 数、全局 FIFO 队列、submit、wait_idle、shutdownrain、异常捕获和未完成计数。测试包括：提交 N 个任务全部执行一次；任务抛异常不会杀死 worker；wait_idle 在任务终态后返回；shutdown 后 submit 失败；析构不会遗留 joinable thread；多生产者提交不破坏队列。

第二步：任务图和 trace 加入 task id、ready count、successor 列表、TaskStateRecord 和错

误摘要。先用全局 ready 队列执行任务图。用第 13 章 stable filter 的 count、scan、write、commit 作为 workload。测试并发前驱同时完成时，后继只入队一次；前驱失败时，后继不会永远 waiting；空图和单节点图都能正常终止。

第三步：本地 deque 和工作窃取 每个 worker 增加 mutex deque。外部提交进入 global inject queue；worker 产生的本地后继可以进入自己的 deque；空闲 worker 先取全局，再随机或轮询 steal。记录 local pops、steal attempts、steal successes、empty steals、worker idle time 和 stolen task duration。先验证策略收益，再考虑无锁 deque。

第四步：执行域和背压接口 区分 compute domain 和 blocking 或 I/O domain。CPU 任务不要阻塞等磁盘或网络；I/O 完成事件通过 CompletionRecord 重新进入任务图。队列容量要有返回值，不能无限堆积。这个步骤和第 15 章衔接。

Listing 14.6 项目中 TaskRuntimePlan 的接口草图

```
1 enum class QueuePolicy : std::int32_t {
2     GlobalOnly,
3     LocalDeque,
4     WorkStealing,
5 };
6
7 struct TaskRuntimePlan {
8     std::uint32_t worker_count = 0;
9     QueuePolicy queue_policy = QueuePolicy::GlobalOnly;
10    bool enable_task_graph = false;
11    bool enable_trace = true;
12    bool enable_cancellation = true;
13 };
```

这个结构不是为了把运行时配置化成复杂产品，而是为了让实验能逐项打开机制。全局队列、任务图、work stealing、trace、cancellation 每个复杂度都要有购买理由。若某机制说不出修复哪个反例，就不应该进入最小项目。

实验矩阵

实验	输入形状	要解释的现象
future 饥饿复现	worker 数等于父任务数，父任务等待同池子任务	ready 队列有工作但 worker 被 wait 占住
barrier 对比 DAG	stable filter 的 count、scan、write、commit	worker 空闲、关键路径、失败传播
任务粒度扫描 work stealing	从很小 chunk 到很大 chunk 倾斜分治任务、热点 block、均匀任务	调度开销、长尾、cache 局部性 steal 是否减少空闲，是否破坏本地性
取消传播	上游 count 或 write 注入失败	后继是否进入终态，等待者是否醒来
关闭协议	submit 与 shutdown 并发	submit 返回值、未完成计数、worker join

Trace 摘要命令 项目应提供类似 `enginetracesummarize` 的输出。最低字段包括任务总数、终态分布、最长 ready 等待、最长 running、估计关键路径、worker idle 比例、steal 成功率、取消传播时间和首个错误任务。摘要可以是 key-value 文本，不要求 GUI。关键是让读者能从证据解释运行时，而不是凭感觉判断。

Listing 14.7 任务运行时 trace 摘要示例


```

tasks.total=1536
tasks.succeeded=1536
critical_path_ns=18400000
makespan_ns=23700000
worker.idle_ratio.p95=0.18
steal.attempts=421
steal.successes=73
task.ready_wait_ns.max=2600000
task.running_ns.max=9100000

```

若 makespan 远大于 critical path, 继续查 ready wait、idle ratio、steal 失败和阻塞事件。若 steal attempts 很多但 successes 很少, victim 选择或任务分布有问题。若 success 很多但吞吐不升, 任务可能太小或跨 NUMA 破坏了局部性。

实验: 先实现全局队列线程池 reference, 再实现任务图版 stable filter。禁止在 worker 内调用阻塞 `future.get()` 等待同池任务。报告必须包含 future 饥饿反例、DAG trace、关键路径估计、取消传播测试和 shutdown 测试。

源码阅读任务

源码阅读任选一条窄路径, 不要泛读。

1. 阅读 oneTBB 或其他成熟任务运行时的 worker loop、task enqueue、work stealing 触发条件和 shutdown。报告要区分源码直接事实、实验测得事实和推断解释。
2. 阅读 OpenMP runtime 的 task dependency 或 task team 相关路径。报告重点是依赖如何进入 ready 状态, 而不是罗列 API 名称。
3. 阅读 Linux `kernel/workqueue.c` 和 `include/linux/workqueue.h` 中 work item 入队、worker 唤醒和救援机制相关路径。报告要回答“阻塞或排队过多时系统如何避免停住”。
4. 阅读 Folly executor 或类似库的 executor、priority、blocking 和 keep-alive 设计。报告要说明执行域、生命周期和关闭协议。

验收与评分标准

本章项目评分建议:

1. 正确性 30 分: reference 存在, 任务只执行一次, 依赖顺序正确, 异常、取消、关闭都进入终态。
2. 可观测性 20 分: TaskStateRecord、WorkerSnapshot、关键路径估计和 trace 摘要完整。
3. 运行时设计 20 分: 禁止或正确处理 worker 内等待, ready count 线性化点清楚, shutdown drain/cancel 语义明确。
4. 性能分析 15 分: 任务粒度、队列等待、work stealing、本地性和关键路径都有实验解释。
5. 工业判断 15 分: 能说明从工业系统借鉴了什么、没有借鉴什么、为什么当前项目不需要更复杂机制。

14.8 复查清单

一、**是否从事故推出机制** 本章所有机制都应能回到 future 饥饿或任务图等待事故。若某段话只是在介绍术语, 不能解释哪个失败现象, 就应该改成具体反例。

二、**是否区分等待和 ready** Waiting 表示依赖未满足, Ready 表示可以执行。worker 只应长期执行 Ready 任务。若代码让 worker 睡在 Waiting 条件上, 必须有 helping、隔离执行域或明确禁

止规则。

三、是否保留 reference 全局队列和 barrier 版本不是落后版本，而是语义 reference。work stealing 和 DAG 优化必须能和 reference 对齐输出、错误和 manifest。

四、是否有失败路径 只测试成功路径的运行时不合格。至少要测试用户任务异常、上游失败导致后继取消、shutdown 并发 submit、future 饥饿反例和 worker join。

五、是否解释复杂度购买理由 本地 deque 购买局部性和低争用；stealing 购买负载均衡；task graph 购买显式依赖；continuation 购买非阻塞等待；trace 购买诊断能力；cancellation 购买资源收束。若购买理由不成立，就不应增加机制。

六、是否连接下一章 本章把 CPU 任务的等待和完成写成状态机。下一章 I/O 会把设备等待也写成提交和 completion。TaskStateRecord、CompletionRecord、背压和 manifest 提交边界应自然衔接。

14.9 本章习题

习题与作业

习题一：构造一个两个 worker 的 future 饥饿复现。写出事件序列、ready 队列状态和 worker 状态，再把它改成 continuation 版本。

习题二：为 `RuntimeState` 写状态转移表。说明 `submit`、`wait_idle`、`shutdownrain`、`shutdowncancel` 在每个状态下的行为。

习题三：给 stable filter 的 count、scan、write、commit 画任务图，标出每个节点的 ready count。说明某个 count 失败时后继如何进入终态。

习题四：设计一个 work stealing 实验。先用 mutex deque，不使用无锁 deque。比较全局队列、本地 deque 无窃取、本地 deque 加窃取三种版本。

习题五：解释 owner LIFO、thief FIFO 的本地性理由，并构造一个它表现不好的反例。

习题六：设计 `TaskStateRecord` 和 `WorkerSnapshot` 的低开销采集方案。说明哪些字段必须精确，哪些字段可以采样或延迟汇总。

习题七：阅读一个工业运行时的源码片段，按“约束、核心机制、购买能力、成本、迁移到本项目的方式”写一页报告。

I/O、异步与背压

先看一个写出队列事故。解析和计算阶段很快，reduce worker 很快生成大量 `OutputBlock`；磁盘偶尔变慢，写出线程跟不上。第一版用无界队列，短时间 benchmark 很漂亮，因为计算线程从不等待；几分钟后内存持续上涨，尾延迟越来越长，最终进程被系统杀掉。第二版改成异步写，`submit_write` 返回成功后，上游立刻复用 buffer；几毫秒后 I/O 线程才真正写文件，落盘内容偶尔变成下一批数据。第三版又修了 buffer 生命周期，却把 `submit` 成功当作业务成功，manifest 过早发布，短写和失败 completion 被日志吞掉。

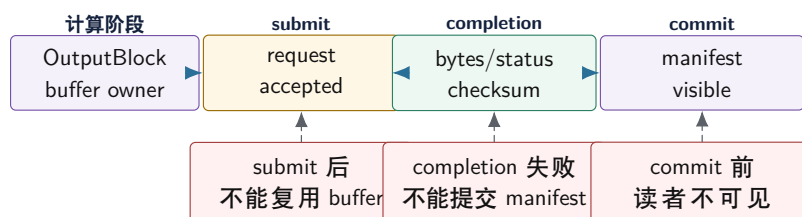
这三个事故逼出本章主线：I/O 不是“换一个系统调用”这么简单，而是必须分清 **提交** (submit)、**完成** (completion)、**提交业务结果** (commit)。提交只表示请求被接纳；完成才说明底层读写有结果；业务提交才说明外部读者可以相信这个输出。异步把等待从调用线程上移走，同时也把所有权、短读短写、错误、取消、背压、可靠写入和崩溃恢复全部暴露出来。

学习目标

学完本章后，读者应能完成七件事：

1. 区分阻塞、非阻塞、异步、ready 事件和 completion 事件的语义差别。
2. 解释为什么 `submit` 成功不等于 I/O 完成，更不等于业务结果提交。
3. 设计 buffer 生命周期，保证异步请求完成前不会被复用、释放或重复归还。
4. 正确处理短读、短写、迟到 completion、取消、超时、关闭和错误分类。
5. 设计有界队列、高低水位线、in-flight bytes 和背压传播链。
6. 用临时文件、checksum、fsync、rename、manifest 和崩溃点矩阵说明可靠写入。
7. 把本章的 CompletionRecord、deadline、attempt、manifest 和背压连接到第 16 至 18 章的分布式 reply、shuffle 和 checkpoint。

提交、完成和业务提交不是同一件事



异步 I/O 的正确性边界不是函数返回，而是请求状态
机：accepted、in-flight、completed、committed 必须分开记录。

图示：本图要证明的命题是：异步化后，提交路径、完成路径和业务提交路径必须由同一个请求身份串起来。

15.1 从阻塞 I/O 到完成事件

阻塞 I/O 的语义最接近普通函数调用：调用线程发起 `read` 或 `write`，线程等待，返回值告诉本次调用处理了多少字节或出了什么错误。它适合配置读取、低并发工具、简单批处理和语义优先的 reference。它的问题是等待设备时占住线程。若计算 worker 在 `fsync`、网络发送或慢磁盘上等待，任务运行时再精巧也无法让这些 worker 执行 ready 的 CPU 任务。

异步 I/O 的目标不是让设备变快，而是把等待从执行线程上移走。提交请求后，CPU 可以继续处理其他工作；完成事件稍后返回，告诉系统这次请求成功、失败、短写、取消还是超时。这个分离

提升并发度，也引入新的状态机。任何异步设计都要先回答：请求提交后谁拥有 buffer，completion 由谁消费，失败如何传播，取消后迟到 completion 怎么处理。

阻塞、非阻塞和异步 阻塞 I/O 调用可能让当前线程睡眠，直到有结果。非阻塞 I/O 调用立即返回，若当前不能读写，返回 `EAGAIN` 或类似状态；应用需要等待 ready 事件后再试。异步 I/O 提交请求，完成队列稍后返回结果。三者不是“低级到高级”的线性关系，而是控制流和资源模型不同。

epoll 这类 ready 机制告诉你 fd 可能可读或可写，不告诉你业务消息已经完整。Socket 可读可能只有半个 header；可写也不保证整个响应一次写完。**io_uring** 更接近提交队列和完成队列模型，但仍然要处理 buffer 生命周期、短读短写、错误码、取消和完成顺序。教材要避免把具体 API 神化，核心是状态机。

短读短写是正常路径 `read` 和 `write` 返回的字节数可能小于请求字节数。网络、管道、非阻塞 fd、信号中断、设备队列和某些文件系统路径都可能出现 partial completion。正确代码必须维护 `offset` 和 `remaining`，直到完成、出错、超时或进入等待。把一次返回当作完整消息，是 I/O 程序最常见的边界错误。

Listing 15.1 同步可靠写循环的教学形态

```
1 enum class IoStepResult : std::int32_t {
2     Complete,
3     RetryLater,
4     Failed,
5 };
6
7 struct WriteProgress {
8     std::uint64_t bytes_total = 0;
9     std::uint64_t bytes_done = 0;
10 };
11
12 bool is_write_complete(const WriteProgress& progress) {
13     return progress.bytes_done == progress.bytes_total;
14 }
```

这个片段没有包装系统调用，而是先固定语义：写出路径必须有进度对象。同步版本在循环里推进进度；异步版本在 completion handler 里推进进度。异步化不改变短写语义，只是把“下一步继续写”从调用栈搬到请求状态表。

CompletionRecord 本章给项目留下 `CompletionRecord`。它是 I/O completion 的结构化事实，作用类似第 14 章的 `TaskStateRecord`。业务层只能根据 completion 改变提交状态，不能在 submit 后提前宣布成功。

Listing 15.2 CompletionRecord 的教学形态

```
1 enum class IoStatus : std::int32_t {
2     Succeeded,
3     Partial,
4     RetryableError,
5     FatalError,
6     Canceled,
7     TimedOut,
8 };
9
10 struct CompletionRecord {
11     std::uint64_t request_id = 0;
```



```

12  std::uint64_t block_id = 0;
13  std::uint32_t attempt = 0;
14  IoStatus status = IoStatus::Succeeded;
15  std::uint64_t bytes_requested = 0;
16  std::uint64_t bytes_transferred = 0;
17  std::uint64_t checksum = 0;
18  std::uint64_t submit_ns = 0;
19  std::uint64_t complete_ns = 0;
20  std::uint32_t error_code = 0;
21  };

```

request_id 区分一次 I/O 请求，**block_id** 区分业务输出，**attempt** 区分重试或迟到完成。若 completion 不带这些身份，完成顺序一乱，driver 就无法判断哪个结果仍然有效。第 16 章的 RPC reply 会复用同样思想，只是 completion 从本机设备事件变成网络响应。

迟到 completion 请求超时后，底层操作可能仍然完成。上层“不等了”不等于设备“不会再回来”。Completion 到达时，状态机要先检查 request 是否仍然有效。若新 attempt 已经提交，旧 completion 只能释放资源、记录 late completion 或清理临时文件，不能重复提交业务结果。

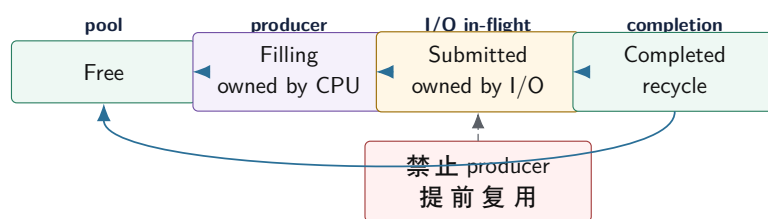
这就是为什么写出路径不能直接写最终文件。旧 attempt 即使迟到成功，也应只写自己的临时文件；最终 manifest 由 driver 决定哪个 attempt 生效。没有临时文件和 manifest，迟到 completion 就可能覆盖新结果。

15.2 所有权、Buffer 池和取消

异步 I/O 最常见的正确性 bug 不是系统调用用错，而是 buffer 生命周期用错。同步调用返回后，调用者通常可以复用 buffer；异步提交后，设备、内核、I/O 线程或发送队列可能仍在读取这个 buffer。若上游提前复用，磁盘或网络看到的就是后续数据。高性能系统常用 buffer pool 减少分配，但 buffer pool 只有配套状态机才安全。

所有权转移 Producer 从 pool 取出 free buffer，填充数据后提交给 writer。提交成功后，buffer 所有权转移给 I/O 子系统；producer 不能再写。Completion 处理完以后，buffer 才能进入 recycling 或 free。取消、失败、短写都必须走 completion 或 reap 路径，不能绕开状态机。

Buffer 生命周期保护异步写



buffer pool 本身不是优化魔法。只有每个状态都有唯一所有者，pool 才不会产生悬垂、提前复用和双重归还。

图示：本图要证明的命题是：异步请求提交后，buffer 生命周期必须由 completion 决定，而不能由 submit 返回值决定。

Buffer 池也是背压 Buffer 池大小决定最多 in-flight 字节。若每个 buffer 1 MiB，允许 512 个 in-flight，就已经占用 512 MiB。池耗尽时，上游必须等待、降低读取速率、减小 batch 或返回 backpressure。这比只限制队列条目数更直接，因为少量超大 block 也会击穿内存预算。

项目应同时记录 item count 和 byte count。队列里 100 个 4 KiB 请求和 100 个 64 MiB 请求不是同一种压力。第 18 章 shuffle 的 in-flight bytes 限制，正是从这里来的。

取消是尽力而为 取消请求后，底层操作可能已经完成、正在内核或设备中、还在队列里、或

者尚未提交。取消只能改变上层接受结果的策略，不能保证物理写入没有发生。状态机至少要区分 **Canceled** 和 **Reaped**：前者表示用户不再接受业务结果，后者表示底层完成事件已经被消费、资源可以释放。

若取消后直接销毁上下文，迟到 completion 就可能访问已释放对象。正确方式是让 request context 活到 reaped；completion 到达时检查状态，若已取消，则释放 buffer、清理临时文件、记录 late completion，不提交 manifest。

关闭时 drain 还是 cancel I/O 子系统关闭有两种常见语义。Drain 表示拒绝新请求，但继续处理已经接受的请求，直到它们完成或失败；cancel 表示未开始请求取消，in-flight 请求尽力取消或等待 completion 后清理。日志调试数据可能允许 cancel，checkpoint、manifest、必须完成的 shuffle block 通常需要 drain 或明确失败。

关闭不是析构时设置一个布尔值。它要停止接收新请求、唤醒等待者、处理队列中请求、消费 completion queue、归还 buffer、报告未完成请求并 join 线程。若 completion queue 没有 drain，底层稍后完成可能访问已经销毁的上下文。

WriteRequest 和 completion loop 异步 I/O 的中心不是 submit，而是 completion loop。Submit 只是把请求交出去；completion loop 接收完成事件，更新请求状态，推进短写 offset，归还 buffer，触发后续 task，汇总错误。若 completion loop 设计不清，异步代码就会散成回调、锁和共享变量，最后很难证明同一个请求只完成一次。

Listing 15.3 WriteRequest 的教学状态

```
1 enum class WriteRequestState : std::int32_t {
2     Created,
3     Queued,
4     Submitted,
5     PartialWritten,
6     Completed,
7     Failed,
8     Canceled,
9     Reaped,
10 };
11
12 struct WriteRequest {
13     std::uint64_t request_id = 0;
14     std::uint64_t block_id = 0;
15     std::uint32_t attempt = 0;
16     std::uint64_t buffer_id = 0;
17     std::uint64_t bytes_total = 0;
18     std::uint64_t bytes_done = 0;
19     WriteRequestState state = WriteRequestState::Created;
20 };
```

Completion loop 收到事件后，第一步不是执行业务回调，而是查找 **request_id**，验证 attempt，检查当前状态。若状态已经是 **Canceled**，completion 只能清理底层资源并进入 **Reaped**；若是 **Submitted** 且只写了一部分，就更新 **bytes_done** 并提交剩余部分；若全部完成且 checksum 通过，才触发 commit task。这个顺序能防止重复回调、迟到完成污染结果和失败路径泄漏 buffer。

完成回调不要做重 CPU 工作 Completion loop 线程不应执行重 CPU 计算，例如解析大文件、压缩、排序或大规模 checksum 聚合。它可以做必要的状态验证和轻量 checksum 检查，然后把后续 CPU work 放回任务运行时。否则底层 I/O 已经完成，completion queue 却因为回调线程忙于计算而积压，表现为“设备不慢，但用户态迟迟看不到完成”。

这个边界和第 14 章一致：worker 只执行 ready 的 CPU 任务，I/O completion 只把等待完成的事实带回状态表。若 completion handler 直接提交 manifest、执行 reduce 或递归触发大量回调，系统会把控制面和数据面混在一起，调试难度急剧上升。

15.3 背压：容量合同和传播链

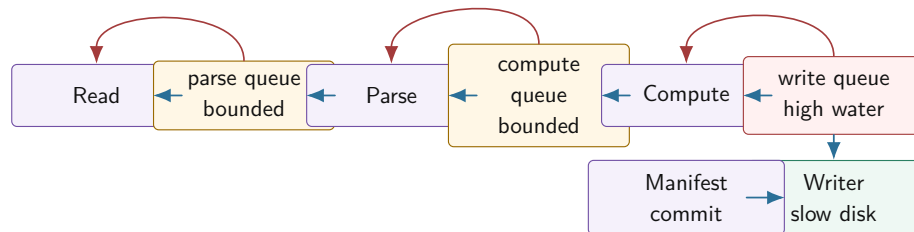
背压是系统告诉上游“下游处理不过来”的机制。无界队列把这个事实藏进内存，直到 OOM、尾延迟爆炸或恢复窗口无法清理。有界队列把容量变成合同：队列满时，生产者必须等待、失败、降级、丢弃低优先级数据或把压力继续传回源头。背压不是性能失败，而是系统保护自己。

队列长度不是唯一指标 队列长度、队列等待时间、入队速率、出队速率、in-flight bytes、completion 延迟和错误率要一起看。长度高但短暂，可能只是 burst；长度持续增长，说明生产速度长期超过消费速度；长度不高但等待时间长，可能是每个请求很大；completion 延迟高，可能是设备慢；completed 到 committed 时间高，可能是提交路径慢。

I/O 报告应拆分延迟：queued 到 submitted, submitted 到 completed, completed 到 committed。只看端到端总时间，无法判断背压该加在 reader、compute、writer 还是 manifest。

高低水位线 单一容量会让系统在“满”和“不满”之间频繁抖动。高低水位线更稳定：超过高水位时上游暂停或减速；低于低水位时恢复。高水位不能太接近容量，否则反馈太晚；低水位不能太接近高水位，否则频繁开关。水位线要用实验调，不是随手写一个数字。

背压必须穿过整条流水线



写盘慢时，压力应从 write queue 传回 compute、parse、read。若中间任何一段是无界队列，背压链断开，内存就在那里堆积。

图示：本图要证明的命题是：背压是整条数据流的协议，不是某一个队列的局部属性。

策略有业务语义 队列满时怎么处理，取决于数据价值。准确计算的 shuffle block 不能静默丢弃；调试日志可以在过载时丢弃并增加 drop counter；交互请求可能返回 busy 或 deadline exceeded；离线批处理可以等待更久。一个 bool 返回值不够，接口应区分 accepted、would block、timed out、closed、canceled 和 failed。

Listing 15.4 背压提交结果的教学枚举

```

1 enum class BackpressureResult : std::int32_t {
2     Accepted,
3     WouldBlock,
4     TimedOut,
5     Closed,
6     DroppedByPolicy,
7 };
8
9 struct QueueCapacity {
10     std::uint64_t max_items = 0;
11     std::uint64_t max_bytes = 0;
12     std::uint64_t high_water_bytes = 0;
13     std::uint64_t low_water_bytes = 0;
14 };
  
```

策略要可观测。若丢弃低优先级日志，必须记录丢弃数量、优先级和原因；若阻塞生产者，必须记录阻塞时间；若返回 busy，必须记录上游是否尊重了 busy。静默背压和静默丢弃都会让系统不可诊断。

deadline 和 timeout Timeout 是某个操作愿意等待多久；deadline 是整个请求最晚可以完成的时间点。流水线更适合传 deadline，因为请求经过 read、parse、compute、write、commit 多个阶段，每个阶段都应知道剩余预算。若每段都重新获得完整 timeout，端到端延迟可能无限膨胀。

Deadline 也约束重试。剩余 5 ms 的请求，不应启动预期 50 ms 的 fsync 或 RPC 重试。第 16 章进入分布式后，deadline 还会遇到时钟和跨节点传播问题；本章先用单调时钟训练“剩余预算”这个思维。

延迟分解比总耗时更有解释力 I/O pipeline 的端到端时间应拆成阶段：read service、parse queue wait、parse service、compute queue wait、compute service、write queue wait、write service、commit wait。若总耗时上升但 write service 没变，可能是队列等待增加；若 write service 上升，可能是设备、fsync 或文件系统路径变慢；若 commit wait 上升，可能是 group commit 等批或 manifest 锁。

观察	可能原因	下一步检查
queue wait 高，service 低	下游并发不足或背压过早	worker 数、队列容量、水位线
service 高，queue wait 低	设备或同步策略慢	fsync、batch、page cache、模拟器延迟
in-flight bytes 高	大 block 或 completion 慢	block size、buffer pool、completion loop
late completion 多	deadline 太紧或重试过快	deadline 预算、attempt 去重、清理路径
orphan temp files 多	commit 前失败或恢复清理弱	崩溃点矩阵、manifest 扫描规则

这张表的价值在于训练判读口径。背压生效时，吞吐可能下降，但内存稳定、错误不增加、队列等待有上界；背压失败时，短期吞吐可能更好，但 in-flight bytes 和队列等待持续上涨。高质量报告必须解释这些曲线，而不是只宣布“异步版更快”。

15.4 可靠写入：从临时文件到 Manifest

写文件成功不等于结果可靠。普通 **write** 可能只是把数据放进 page cache；rename 可能只更新目录项；manifest 可能已写但未持久化；进程可能在任意步骤崩溃。可靠写入的目标不是让所有中间状态都消失，而是让每个中间状态都能被恢复程序解释。

先写临时文件 输出不应直接覆盖最终文件。Writer 先写临时路径，临时名包含 stage、partition、task、attempt 和随机后缀。数据完整、checksum 通过、必要同步完成后，再通过 rename 或 manifest commit 让它成为可见结果。未提交临时文件不是有效输出，reduce 不应扫描临时目录取数据。

崩溃点矩阵 可靠 I/O 不是一句“使用 fsync”，而是一张崩溃点矩阵。每一步都要说明崩溃后恢复怎么处理。

崩溃位置	恢复时看到什么	恢复动作
创建临时文件前	没有候选输出	重新执行 task 或保持未完成
写临时文件中	可能有残缺临时文件	校验失败，删除或保留诊断
写完但未校验	文件存在但身份未确认	重新校验，失败则重做
校验通过但未提交 manifest	候选文件存在，外部不可见	按策略补提交或清理孤儿
manifest 已提交	权威记录引用该 block	接受为有效输出
清理旧文件前	旧临时文件仍在	根据 manifest 再次清理

项目可以先承诺进程崩溃恢复，再把掉电持久化作为扩展。若要求掉电后恢复，需要考虑文件 fsync、目录 fsync、manifest fsync、文件系统和设备缓存语义。教材不能保证所有平台细节一致，但必须让读者知道可靠性承诺有层级。

fsync、rename 和 manifest 的边界 常见本地文件模式是：写临时文件，确保内容完整，必要时 fsync 文件，rename 到最终名，必要时 fsync 父目录，最后发布 manifest 或把 manifest 作为唯一权威入口。不同文件系统、对象存储、NFS 和移动平台语义会变化，所以项目报告必须写清文件系统、测试方式和未验证边界。

Manifest 的意义是把“哪些输出有效”集中成权威事实。外部读者只信 manifest，不扫描目录。这样崩溃后，孤儿临时文件可清理，重复 attempt 可去重，迟到 completion 不会直接污染最终结果。第 18 章的 shuffle manifest 会直接复用这个模型。

批处理和 group commit 每个 block 都 fsync，可靠但慢；多个 block 共享一次持久化，吞吐高但每个请求等待更久，崩溃后可能重做更多未提交工作。数量阈值和时间阈值要一起设置：达到 N 个 block 或等待 T 时间就提交。只按数量低流量时延迟高；只按时间高流量时批太小。

策略	适用场景	风险
每 block 同步	审计强、数据少、语义优先	吞吐低，尾延迟高
按数量批量提交	高吞吐离线写出	低流量等待时间长
按时间窗口提交	延迟预算明确	高流量批次可能偏小
数量或时间触发	常见工程折中	需要调参和监控

可靠写入也是性能取舍 可靠写入会增加系统调用、flush、metadata 和恢复扫描成本。它不是无条件越强越好。离线中间结果可能允许重算，最终 checkpoint 需要更强提交；调试日志可以丢，manifest 不可以丢。教材级设计要把可靠性承诺、重做成本和性能成本放在同一个表里比较。

Page cache 会误导 benchmark 文件 I/O benchmark 很容易被 page cache 误导。第一次读大文件可能来自设备，第二次读同一文件可能来自内存；写文件返回快，可能只是进入 page cache，真正 writeback 稍后发生。若报告没有说明冷 cache、热 cache、是否 fsync、文件大小是否超过内存、是否使用 tmpfs，结果就无法解释。

本章实验应区分三类目标。研究背压时，可以使用模拟 I/O 稳定控制延迟和错误；研究 page cache 时，要明确预热、文件大小和系统限制；研究可靠提交时，要关注 fsync、rename、manifest 和崩溃点。把三类目标混在同一张性能图里，读者会误把缓存效果当作 I/O 设计效果。

Page cache 还会影响内存账本。大量未落盘 dirty page 可能不计入进程 RSS，却消耗系统内存并触发回写压力。报告可以记录进程内存、系统可用内存、输出目录大小和 completion 延迟。即使指标粗略，也比完全忽略系统缓存更诚实。

15.5 工业界优秀设计案例

工业 I/O 设计的共同点，是把等待、完成、容量和提交边界写成可审查状态。读源码时不要只列 API 名称，要说明它购买了什么能力、付出什么成本、适合什么负载。

工程案例

Linux page cache 和 writeback 普通文件写入常先进入 page cache，dirty page 之后由 writeback 写到设备。于是 write 返回快，不代表掉电后可恢复。fsync 的价值是推动数据和相关元数据到更强持久化边界。这个设计购买了吞吐和合并写入，也带来 benchmark 误判：只测 write 返回时间，可能完全测的是内存路径。

项目报告应区分应用层背压实验、page cache 实验和可靠提交实验。研究背压时可以用模拟 I/O；研究可靠性时要关注 fsync 和崩溃点；研究真实设备吞吐时要记录 cache 热冷、文件大小、同步策略和系统权限。

工程案例

epoll 的 ready 语义 `epoll` 通知 fd ready, 不通知业务消息完整。它购买的是少量线程等待大量 fd 的能力, 不替应用处理 framing、短读、短写和业务队列。边沿触发减少重复通知, 但要求应用读写到 `EAGAIN`, 否则容易遗漏机会。

本书项目不必实现复杂网络 reactor, 但要学习 ready 语义的边界: 事件循环线程不能做重 CPU 工作; 业务消息要有状态机; 完成后要把 CPU work 交给任务运行时; 背压要通过停止读取、减少发送或应用层 busy 传出去。

工程案例

io_uring 的 SQ/CQ 模型 `io_uring` 把提交队列和完成队列显式化。用户态提交请求, 内核推进, 完成队列返回结果。它购买了更低系统调用开销和更丰富的异步操作, 但没有取消设备延迟、错误、容量和生命周期问题。Buffer 注册和零拷贝还会让 buffer 生命周期更严格。

项目可以先用 I/O worker 模拟异步, 训练 CompletionRecord、buffer ownership、back-pressure 和 reliable commit; 之后再把底层替换成 `io_uring`。这样 API 变化不会改变教材主线。

工程案例

Kafka、RocksDB 和 group commit Kafka、RocksDB 等系统都大量使用批处理、顺序写、后台 flush、日志或 manifest 记录提交事实。它们的设计说明: 高吞吐存储系统不是每条记录都立即同步到最终结构, 而是把数据面和控制面分开, 用日志、manifest、sequence number、flush 和 compaction 管理可见性。

本书项目借鉴的是原则: 大数据块可以提前流式写入, 真正需要强一致提交的是小的 manifest 或 commit record; 后台清理和恢复扫描必须以 commit record 为权威, 不以目录扫描为权威。

工程案例

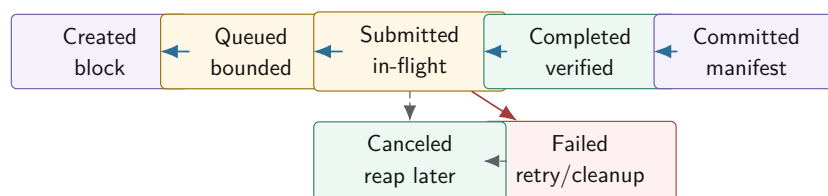
TCP 背压和应用层背压 TCP 接收端慢会让接收窗口缩小, 本端发送最终阻塞或返回 `EAGAIN`。这是字节流层面的背压。应用层仍要有自己的背压, 因为 TCP 不知道业务优先级、deadline、幂等、内存预算和队列语义。Reduce 端忙时, 应用可以返回 busy 或降低 shuffle 拉取窗口; 只依赖 TCP buffer, 反馈太晚且语义太粗。

第 18 章分布式 shuffle 会把这里的本机 write queue 扩展成远端 in-flight bytes、per-worker window 和 busy reply。概念相同, 尺度变大。

15.6 深度案例: 异步写出 Shuffle Block

把第 13 章的 partition manifest、第 14 章的任务运行时和本章 I/O 放到一起, 得到一个可恢复写出管线。Compute task 生成 `OutputBlock`, I/O stage 写临时文件, completion 成功后触发 commit task, commit task 更新 manifest。Reduce 只读取 manifest 中已提交的 block。这个结构看起来比“直接写文件”复杂, 但它能解释短写、失败、迟到 completion、重复 attempt、崩溃恢复和背压。

异步写出管线的状态机



Created 到 Submitted 是资源所有权路径；Completed 到 Committed 是业务可见性路径。失败和取消都必须释放 buffer 和容量预算。

图示：本图要证明的命题是：异步写出必须同时管理资源状态和业务提交状态，不能用一个成功布尔值替代。

第一版：同步 reliable write reference 先写同步 reference：创建临时文件，可靠写循环推进 offset，完成后计算 checksum，执行项目承诺的同步策略，然后返回候选 block。这个版本可能慢，但它固定短写、错误和文件状态语义。异步版本必须和它输出同样的 manifest 结果。

第二版：I/O worker 和 completion queue 把 reliable write 放到 I/O worker，计算线程提交 `WriteRequest` 后继续处理 CPU work。Completion queue 返回 `CompletionRecord`。Completion loop 只更新状态、归还 buffer、触发后续 task，不在 completion 线程里做重 CPU 计算。若 completion loop 自己被 CPU 工作拖慢，已经完成的 I/O 也会迟迟不被业务层看到。

第三版：背压闭环 为 write queue 和 buffer pool 设置 item、byte、高低水位。高水位时，reduce 暂停生成新 block 或降低 batch；reduce 队列满时，partition 减速；partition 减速后，map 输出缓冲减少；最终 reader 停止读取更多输入。报告要画出这条背压传播链，不能只说“队列满了会等”。

第四版：迟到 completion 和 attempt 请求超时后，driver 可以创建新 attempt。旧 attempt 后来完成，只能写自己的临时文件并进入 late/reaped 状态。Commit task 检查当前 task attempt 和 manifest，只有获胜 attempt 能提交。这个结构和第 16 章“成功响应丢失后重试”的问题完全同构。

15.7 项目落地

Compute Systems Engine 本章应实现一个可控 I/O 模拟器和一个有界异步写出管线。不要依赖真实磁盘偶然变慢来触发问题。模拟器参数包括生产速率、消费延迟、短写概率、错误率、队列容量、buffer 大小、batch 阈值、deadline 和重试预算。故障应可复现，报告才可复查。

项目接口

Listing 15.5 I/O 管线计划的教学形态

```

1 enum class FsyncPolicy : std::int32_t {
2     None,
3     PerBlock,
4     GroupCommit,
5 };
6
7 struct IoPipelinePlan {
8     QueueCapacity write_queue_capacity;
9     FsyncPolicy fsync_policy = FsyncPolicy::None;
10    std::uint64_t batch_bytes = 0;
11    std::uint64_t batch_timeout_ns = 0;
12    std::uint32_t max_retries = 0;
13    bool enable_fault_injection = true;
14 };
  
```

这个计划对象用于实验，不是要把 I/O 做成配置大杂烩。每个字段都对应一个判断：容量如何保护内存，fsync 策略承诺什么可靠性，batch 如何折中吞吐和尾延迟，重试预算如何避免风暴，故障注入如何证明失败路径。

实验矩阵

实验	输入形状	要解释的现象
无界队列反例	生产速度大于写出速度	内存增长、尾延迟增长、压力被隐藏
有界队列背压 短写循环	item 和 bytes 双容量 每次只写部分字节	吞吐、等待时间、内存上界 offset 是否正确推进, checksum 是否匹配
迟到 completion 崩溃点矩阵	超时后旧 attempt 完成 写临时文件到 manifest 各点中断	是否重复提交, buffer 是否释放 恢复是否只接受 committed 输出
batch 和 fsync	per-block、group commit、no fsync	吞吐、p99、重做成本、可靠性承诺

IoPipelineReport 每次实验输出机器可读摘要。最低字段包括：输入规模、block 数、bytes、submit count、completion count、retry count、dropped count、late completion count、queue max items、queue max bytes、writer service time、queue wait time、commit wait time、manifest entries、orphan temp files。

Listing 15.6 I/O 管线报告摘要示例

```
blocks.submitted=2048
blocks.committed=2048
bytes.in_flight.max=268435456
write_queue.items.max=128
completion.late=3
write.short_count=91
retry.count=7
manifest.entries=2048
temp.orphans.cleaned=3
```

报告解释要有判断：无界队列吞吐高但内存上涨，是压力被隐藏；有界队列吞吐下降但内存稳定，是背压生效；late completion 被 reaped 且未提交，是状态机正确；**short_count** 高但 checksum 正确，是短写循环正确。

源码阅读任务

源码阅读任务

源码阅读任选一条窄路径：

1. Linux 普通文件路径：从 `fs/read_write.c`、`mm/filemap.c` 和 `fs/sync.c` 选择一个问题，解释 `write`、page cache、dirty page 和 `fsync` 的关系。
2. Linux ready 事件：阅读 `fs/eventpoll.c` 的一个小路径，解释 ready 通知为什么不等于业务消息完整。
3. `io_uring`：围绕提交队列、完成队列、request id、取消和 completion 顺序写报告，不要求实现完整 runtime。
4. Kafka、RocksDB 或类似系统：阅读日志写入、manifest、flush、group commit 或 compaction 的一条路径，说明它如何分离数据面和提交事实。

报告必须区分源码直接事实、实验测得事实和推断解释。不要泛写“Linux I/O 很复杂”，要回答本章问题：请求何时完成，buffer 何时可复用，结果何时可见，过载如何反馈。

实验：实现一个慢 writer 模拟器。先用无界队列观察内存和尾延迟，再改成有界队列和 高低水位线。最后加入短写、迟到 completion 和崩溃点矩阵。验收不是“程序跑完”，而是 manifest、checksum、buffer pool、queue capacity 和 completion 状态都一致。

验收与评分标准

本章项目评分建议：

1. 正确性 30 分：短写处理、buffer 生命周期、completion 去重、取消和关闭路径正确。
2. 可靠提交 20 分：临时文件、checksum、manifest、崩溃点矩阵和恢复规则清楚。
3. 背压设计 20 分：队列按 item 和 byte 限制，高低水位线能传回上游。
4. 可观测性 15 分：CompletionRecord、IoPipelineReport、队列等待和服务时间完整。
5. 工业判断 15 分：能说明 fsync、batch、page cache、epoll/io_uring、group commit 的适用边界和成本。

15.8 复查清单

一、是否分清 **submit**、**complete**、**commit** Submit 成功只表示请求被接受；completion 成功只表示底层 I/O 有结果；commit 才表示业务结果可见。任何把三者混成一个布尔值的设计都不合格。

二、**buffer 生命周期是否有唯一所有者** Filling、submitted、completed、free 每个状态都要有唯一所有者。失败、取消、迟到 completion 都要归还或 retire buffer，不能泄漏或双重归还。

三、**背压是否传到源头** write queue 高水位必须影响 compute、parse 或 read。若中间某段仍无限生产，背压链断了。

四、**可靠写入是否有崩溃点矩阵** 只写“使用临时文件和 fsync”不够。每个崩溃位置都要有恢复动作，manifest 必须是权威入口。

五、**错误是否结构化分类** Retryable、fatal、canceled、timed out、corrupt、backpressure 不应都变成字符串日志。driver 要能根据错误类型做不同决策。

六、**是否连接分布式章节** CompletionRecord 对应 RPC reply；attempt 对应远端 task attempt；manifest 对应 checkpoint 和 shuffle 元数据；in-flight bytes 对应分布式流控窗口。若这些对象不能复用，说明边界还不稳。

15.9 本章习题

习题与作业

习题一：设计一个异步写请求状态机，至少包含 Created、Queued、Submitted、Partial-Written、Completed、Failed、Canceled、Committed、Reaped。说明每个状态允许哪些转移。

习题二：给一个使用 buffer pool 的异步写出模块画所有权图。说明 submit 成功、submit 失败、completion 成功、completion 失败、取消和关闭时 buffer 如何归还。

习题三：写一个短写处理流程。要求维护 offset、remaining、可重试错误和 fatal 错误，并说明异步版本如何把循环拆成 completion 事件。

习题四：设计高低水位线实验。比较无界队列、有界队列、有界队列加水位线三种方案的内存峰值、吞吐和 p99 延迟。

习题五：为可靠写入写崩溃点矩阵。状态包括写临时文件前、写中、校验后、rename 后、manifest 前、manifest 后。每个状态写恢复动作。

习题六：解释 **epoll** ready 和 **io_uring** completion 的差别。为什么 ready 不等于业务消息完整，completion 也不等于业务提交？

习题七：阅读一个工业 I/O 或存储系统片段，按“约束、核心机制、购买能力、成本、迁

移到本项目的方式”写报告。

第零部分

分布式计算：把单机原则扩展到集群

第 16 章

分布式计算模型

分布式系统不应该从术语表开始。把 RPC、CAP、租约、逻辑时钟、幂等、背压按字典顺序列出来，读者很快会背几个名字，却仍然不知道程序为什么会错。本章从一个会在小样例里通过、在故障注入里立刻出错的代码开始：Driver 把 task 7 发给 worker，worker 已经把结果写到临时 block，成功响应在返回途中丢失。Driver 等不到响应，按本地 timeout 重新派发同一个 task。几秒后，旧响应和新响应先后到达，两个物理执行都声称自己完成了 task 7。

这不是“网络不可靠”这种空话，而是第 15 章 completion 模型在网络边界上的放大。提交请求不等于远端执行，远端执行不等于响应到达，响应到达不等于业务提交。调用一旦跨过线程、进程或机器边界，原来由调用栈、共享内存和同步返回值提供的保证，就必须由消息身份、attempt、deadline、epoch、状态表、提交表、trace 和 manifest 重新写出来。

Listing 16.1 反例：把远端调用当成本地函数

```
1 enum class RemoteStatus : std::uint16_t {
2     Ok,
3     Timeout,
4     Fatal,
5 };
6
7 struct RemoteReply {
8     RemoteStatus status = RemoteStatus::Fatal;
9     std::uint32_t task_id = 0;
10    std::uint64_t block_id = 0;
11    std::uint64_t checksum = 0;
12 };
13
14 bool run_task_once(std::uint32_t task_id) {
15     const RemoteReply reply = run_remote(task_id);
16     if (reply.status == RemoteStatus::Timeout) {
17         mark_failed(task_id);
18         return false;
19     }
20     if (reply.status != RemoteStatus::Ok) {
21         mark_failed(task_id);
22         return false;
23     }
24     append_manifest(reply.task_id, reply.block_id, reply.checksum);
25     return true;
26 }
```

这段代码至少有四个漏洞。第一，timeout 是本地等待结束，不是远端失败事实。第二，重试不是撤销第一次执行后再试一次，而是制造新的物理 attempt。第三，Ok 只说明某个远端执行声称成功，不说明它属于当前 attempt、当前 Driver epoch、当前 deadline，也不说明它有权进入 manifest。第四，manifest 写入前后崩溃的恢复语义完全不同，内存状态不能成为权威事实。

学习目标

学完本章后，读者应能完成九件事：

1. 解释为什么远端 timeout 只能说明 unknown，不能直接说明 failed。
2. 区分 logical task、physical attempt、request、reply、epoch、deadline 和 commit。
3. 设计消息信封，让响应携带足够证据进入状态机，而不是直接改业务结果。
4. 设计协议版本和兼容规则，知道哪些字段变化可以滚动升级，哪些变化必须拒绝。
5. 用状态表和提交表处理重试、迟到响应、重复响应、Driver 重启和旧 epoch。
6. 说明 exactly-once 的工程边界：执行可以重复，外部可见提交只能唯一。
7. 把 deadline、重试预算、busy/backpressure 写进协议，而不是藏在 sleep 和日志里。
8. 用事件日志重放响应丢失、乱序、重复和重启事故，形成确定性调试入口。
9. 用本机 MessageBus 和故障矩阵复现分布式错误，再迁移到多进程和真实网络。

16.1 从响应丢失开始

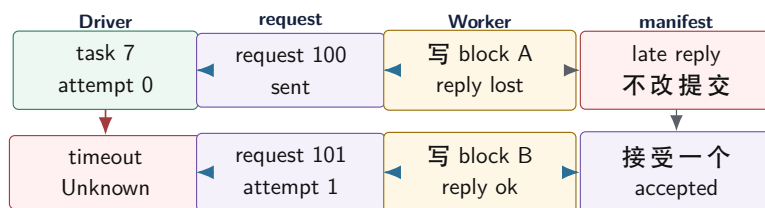
正常路径很诱人。Driver 取出 split 7，调用 `run_remote(split)`；worker 读取输入、统计 key、写出 block，再返回路径和 checksum；Driver 追加 manifest，reduce 后续读取这个 block。小样例能跑，日志也干净。很多坏设计都在正常路径里显得合理，因为还没有事件打破本地调用直觉。

现在只改变一件事：worker 业务代码全部成功，成功响应在返回途中丢失。Driver 看到 timeout，于是把 task 标记为 failed。这里错误已经发生。请求可能根本没有到达；也可能到达 worker 队列但尚未执行；可能执行完成但响应丢失；可能正在发送响应；也可能 worker 崩溃。timeout 把这些物理情况都压成一个本地事实：“我没有在规定时间内看到结果”。它不能证明远端失败。

再让 Driver 重试。第一次执行可能已经写出了 block A，第二次执行又写出 block B。若程序让 reduce 扫描输出目录，就会把同一个 split 统计两次；若 `append_manifest` 无条件追加，也会制造重复提交。提交表不是凭空出现的概念，它是从“执行可以重复，但外部结果只能生效一次”这个缺口里长出来的。

最后让旧响应迟到。响应里只有 task id、block id 和 checksum，Driver 无法判断它属于第几次物理执行，无法判断它是否来自重启前的 Driver，无法判断它是否已经过了 deadline，也无法判断它是否重复。于是 request id、attempt、epoch、deadline 和 trace 不是装饰字段，而是响应进入状态机前必须携带的证据。缺一个字段，就会有一个故障位置只能靠猜。

响应丢失后，执行可以重复，提交只能唯一



Driver 可以创建新的 attempt，但 manifest 只能接受一个 attempt。旧响应迟到只能进入 trace 或重复指标，不能重开终态。

图示：本图要证明的命题是：分布式 exactly-once 的可落地边界不是“只执行一次”，而是“外部可见提交只生效一次”。

本地调用直觉为什么失效 本地函数的失败边界比较窄：没调用、抛异常、返回错误，通常能在同一进程里用栈和断点观察。远端调用被拆到多个位置：客户端序列化、发送队列、网络、服务端接收队列、业务线程、输出写入、响应队列、客户端 completion。每一段都可能排队，也都可能在“对方已经做了事”以后才失败。

RPC 库可以隐藏连接池、编码、线程和重试样板，但不能消除不确定性。调用者必须明确写出：

请求身份是什么，timeout 后进入什么状态，是否允许重试，重复请求如何处理，迟到响应如何处理，远端输出如何提交，取消是否影响远端提交。没有这些答案，程序只是把分布式错误藏在库调用后面。

Timeout 是 Unknown 把 timeout 写成 failure，是分布式代码的第一个常见错误。Timeout 后可以重试、查询、降级或放弃，但不能凭本地 timer 推断远端事实。更稳的状态名应该是 **TimedOut** 或 **UnknownRemoteState**，而不是 **Failed**。Failed 应表示系统已经根据协议决定这个逻辑任务不能再自动成功。

Deadline 和 timeout 也要区分。Timeout 是某一层本地等待多久；deadline 是这次逻辑请求最晚还能被接受到什么时间。请求迟到但还在 deadline 内，可能仍可接收；请求已经过 deadline，即使远端稍后成功，也不能直接提交业务结果。第 15 章 I/O 迟到 completion 的状态机，在这里变成远端 reply 的状态机。

16.2 消息身份和协议边界

本地返回值靠调用栈归属，远端响应靠字段归属。消息必须携带足够证据，让接收方能判断它属于哪个逻辑任务、哪个物理 attempt、哪个 request、哪个 Driver epoch、哪个 deadline 和哪个 trace。字段看起来多，但每个字段都对应一个故障问题。

消息信封 消息信封不是把业务字段包一层好看外壳，而是协议边界的证据集合。教学版可以这样写：

Listing 16.2 消息信封的教学形态

```

1 constexpr std::uint16_t DISTRIBUTED_PROTOCOL_VERSION = 1;
2
3 enum class MessageKind : std::uint16_t {
4     TaskRequest,
5     TaskReply,
6     Busy,
7     Cancel,
8 };
9
10 enum class RpcStatus : std::uint16_t {
11     Ok,
12     Retryable,
13     Busy,
14     StaleEpoch,
15     DeadlineExpired,
16     InvalidRequest,
17     Fatal,
18 };
19
20 struct MessageEnvelope {
21     std::uint16_t protocol_version = DISTRIBUTED_PROTOCOL_VERSION;
22     MessageKind kind = MessageKind::TaskRequest;
23     RpcStatus status = RpcStatus::Ok;
24     std::uint32_t job_id = 0;
25     std::uint32_t stage_id = 0;
26     std::uint32_t task_id = 0;
27     std::uint32_t attempt = 0;
28     std::uint64_t request_id = 0;
29     std::uint64_t trace_id = 0;
30     std::uint32_t driver_epoch = 0;
31     std::uint64_t deadline_ns = 0;

```

```

32  std::uint32_t payload_bytes = 0;
33  std::uint64_t payload_checksum = 0;
34  };

```

这个结构不是 wire format。真实网络协议要明确大小端、变长字段、版本兼容、长度限制、校验位置和错误处理。教学项目可以在本机 MessageBus 中复制这个结构；一旦进入 socket、文件或跨版本存储，就不能把 C++ 对象内存直接发送出去。

字段对应的失败问题 没有 `task_id`，响应不知道属于哪个逻辑工作单元；没有 `attempt`，旧 `attempt` 和当前 `attempt` 无法区分；没有 `request_id`，重复消息和迟到 `completion` 无法去重；没有 `driver_epoch`，Driver 重启或控制面切换后的旧响应可能污染新状态；没有 `deadline_ns`，过期请求仍可能被错误提交；没有 `trace_id`，跨组件日志无法串联。

字段	解决的问题	缺失后果
<code>task id</code>	逻辑工作单元身份	结果无法归属
<code>attempt</code>	物理执行身份	旧执行覆盖新执行
<code>request id</code>	一次消息交互身份	重复响应无法去重
<code>epoch</code>	控制面新旧身份	重启前事件污染新状态
<code>deadline</code>	结果是否还可接受	过期结果被提交
<code>trace id</code>	跨组件因果链	事故无法复盘

控制消息和数据消息分开 控制消息携带 `task`、`attempt`、`deadline`、状态和提交决定；数据消息携带 `payload` 或 `block` 引用。把大数据直接塞进控制消息，会让重试、日志、`trace` 和持久化都变重。高性能系统通常让数据面走批处理、分片和背压，控制面只传身份、校验和提交事实。本书项目中，worker 可以写临时 `shuffle block`，`reply` 只返回 `block id`、`bytes` 和 `checksum`；`manifest` 再决定是否接受。

部分读写和消息边界 进入多进程后，`send` 和 `recv` 不会按消息对象自然对齐。一次 `recv` 可能只有半个 header，一次 `send` 可能只写出部分字节。第 15 章短读短写在这里再次出现：协议要有 framing，例如固定 header 加 `payload length`；读取方先收完整 header，校验长度，再收完整 `payload`。若项目从 C++ 结构体复制直接跳到 socket，很快会在部分读写、大小端、对齐和版本上出错。

16.3 协议版本、兼容和滚动升级

协议版本不是为了在 header 里放一个好看的数字。它回答一个很具体的问题：当新旧 Driver、新旧 Worker、新旧 `manifest` 同时存在时，接收方如何知道这条消息能不能安全解释。没有版本规则，升级就会退化成“全系统同时停机换代码”。这在教学项目里可以暂时忍受，在真实系统里通常不可接受。

兼容性是语义承诺，不是字段数量 新增字段不一定破坏兼容，前提是旧接收方忽略它以后仍然安全，新接收方给缺失字段一个保守默认值。例如新版本给 `reply` 增加 `worker_load_score`，旧 Driver 不看它，最多失去调度优化；新 Driver 收到旧 Worker 的 `reply`，把 `load score` 当成 `unknown`，也不会重复提交。相反，改变旧字段含义通常破坏兼容。若 `deadline_ns` 过去表示绝对时间，新版本改成相对时长，即使字段名和类型不变，也必须视为协议破坏。

未知 `enum` 值也不能随便 `cast` 后继续执行。收到未知 `RpcStatus` 时，安全策略通常是拒绝提交、记录 `trace`、按 `retryable` 或 `invalid` 分类进入状态机，而不是把它当 `Ok`。分布式协议里的默认值必须保守：宁可拒绝一个需要人工观察的结果，也不能把不理解的消息提交为业务事实。

Listing 16.3 协议兼容判断的教学形态

```

1  constexpr std::uint16_t DISTRIBUTED_MIN_PROTOCOL_VERSION = 1;
2  constexpr std::uint16_t DISTRIBUTED_MAX_PROTOCOL_VERSION = 2;
3
4  enum class ProtocolDecision : std::int32_t {

```

```

5   Accept,
6   RejectTooOld,
7   RejectTooNew,
8   RejectUnsupportedFeature,
9   };
10
11 struct ProtocolHeader {
12     std::uint16_t protocol_version = DISTRIBUTED_MIN_PROTOCOL_VERSION;
13     std::uint32_t feature_bits = 0;
14     std::uint32_t header_bytes = 0;
15     std::uint32_t payload_bytes = 0;
16 };
17
18 ProtocolDecision classify_protocol(const ProtocolHeader& header,
19                                   std::uint32_t supported_features) {
20     if (header.protocol_version < DISTRIBUTED_MIN_PROTOCOL_VERSION) {
21         return ProtocolDecision::RejectTooOld;
22     }
23     if (header.protocol_version > DISTRIBUTED_MAX_PROTOCOL_VERSION) {
24         return ProtocolDecision::RejectTooNew;
25     }
26     if ((header.feature_bits & ~supported_features) != 0) {
27         return ProtocolDecision::RejectUnsupportedFeature;
28     }
29     return ProtocolDecision::Accept;
30 }

```

这段代码仍然是教学形态。真实系统还要限制 header 长度、payload 长度、字段重复、压缩标记、认证信息和校验范围。核心不在代码长度，而在纪律：接收方先判断自己是否理解协议，再让消息进入业务状态机。

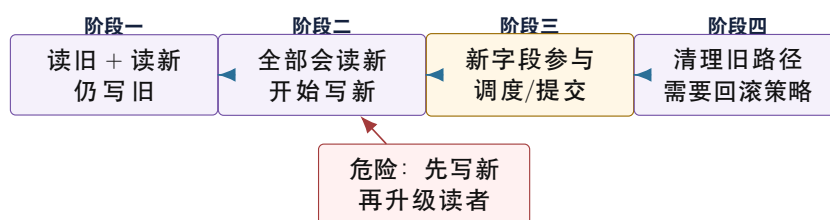
消息兼容和持久化兼容不同 短生命周期消息可以用更宽松的兼容策略：未知优化字段可以忽略，未知状态可以拒绝并重试。但 manifest、checkpoint、commit log 是恢复入口，兼容规则必须更严格。一个新版本写出的 checkpoint，旧版本如果读不懂，不能假装读懂；一个旧版本 manifest 缺少新版本需要的字段，新版本必须有明确迁移逻辑。否则系统在升级当天能跑，第一次崩溃恢复时才暴露数据语义不一致。

变化类型	一般是否可滚动升级	必要条件
新增可选观测字段	通常可以	旧端忽略仍安全，新端有保守默认值
新增可选调度字段	通常可以	不影响提交、恢复和幂等判断
新增必需身份字段	通常不可以直接升级	先部署能读旧格式的版本，再切写新格式
改变字段含义	不可以当兼容变化	必须新版本号、新解析路径和迁移策略
删除持久化字段	高风险	证明恢复不再依赖，且旧版本不会回滚读取
扩展 enum	取决于默认策略	未知值必须拒绝或保守降级，不能当成功

滚动升级要分阶段 工业系统常用的升级纪律是：第一阶段让所有节点先学会读新旧两种格式，但仍写旧格式；第二阶段确认集群里没有旧读者，再开始写新字段；第三阶段让新功能真正参与调

度或提交；最后才考虑清理旧兼容路径。这里的关键不是流程繁琐，而是避免“一个节点写了别的节点读不懂的事实”。

滚动升级先扩展读取能力，再改变写入语义



滚动升级的核心顺序是先扩大解释能力，再改变外部事实的写入语义。否则故障恢复会读到自己不理解的 manifest 或 checkpoint。

图示：本图要证明的命题是：协议版本的价值在于约束升级顺序，而不是记录构建号。

本章项目的版本边界 Compute Systems Engine 可以采用一个简化规则：每条 MessageBus 消息携带 `protocol_version` 和 `feature_bits`；manifest 文件携带 `manifest_version`；状态恢复只接受自己明确支持的 manifest 版本；未知消息 feature 进入 rejected trace，不得提交。这样项目不会被复杂兼容系统拖慢，但读者已经学到真实系统最重要的升级纪律。

16.4 状态表和响应处理

调用栈消失以后，状态表就是新的事实中心。Driver 日志说 timeout，worker 日志说 done，MessageBus 日志说 drop reply，manifest 日志说 commit attempt 1。若没有统一状态表，这些局部事实互相矛盾。状态表把逻辑 task、物理 attempt、request、reply、deadline、epoch 和 commit 连接起来。

逻辑任务和物理 attempt Task 7 是逻辑工作单元，attempt 0 和 attempt 1 是两次物理执行，request 100 和 request 101 是两次消息交互，trace 9001 是跨组件观测链。它们不能混成一个 id。混在一起会产生两类错误：重试时换 request id 导致服务端无法去重；迟到响应只按 task id 匹配导致旧 attempt 覆盖新 attempt。

Listing 16.4 Driver 侧任务状态记录

```

1 enum class DistributedTaskState : std::int32_t {
2     Pending,
3     Sent,
4     Running,
5     TimedOut,
6     Retrying,
7     Committed,
8     Failed,
9     Abandoned,
10 };
11
12 struct DistributedTaskRecord {
13     std::uint32_t job_id = 0;
14     std::uint32_t stage_id = 0;
15     std::uint32_t task_id = 0;
16     std::uint32_t current_attempt = 0;
17     std::uint64_t active_request_id = 0;
18     std::uint32_t driver_epoch = 0;
  
```

```

19 std::uint64_t deadline_ns = 0;
20 DistributedTaskState state = DistributedTaskState::Pending;
21 std::uint32_t retry_count = 0;
22 std::uint32_t late_reply_count = 0;
23 std::uint32_t stale_reply_count = 0;
24 };

```

终态要明确。Committed 表示外部结果已经生效；Failed 表示系统决定不再接受自动成功；Abandoned 表示上游不再等待但远端事件仍可能迟到。终态以后，旧响应不能重新打开状态，除非进入人工恢复流程且带新的 epoch。很多分布式 bug 都来自“失败以后又成功”“取消以后又回调”“重启以后旧 leader 又写入”这类终态不清。

响应处理顺序 收到响应后，Driver 不应直接写 manifest。它要先验证协议版本和 checksum，防止损坏消息进入业务状态；再验证 epoch，拒绝旧 Driver 或旧控制面产生的响应；然后验证 task 和 request；接着比较 attempt；最后查询提交表，判断结果是否仍能提交。

Listing 16.5 响应进入状态机前的分类

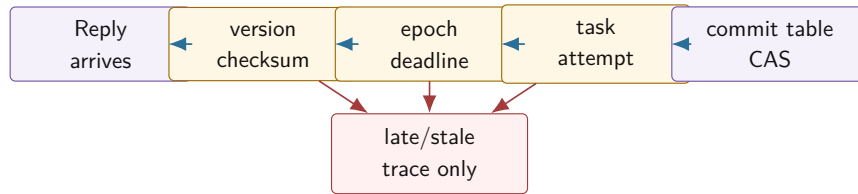
```

1 enum class ReplyDecision : std::int32_t {
2     AcceptForCommit,
3     DuplicateOfCommitted,
4     LateAttempt,
5     StaleEpoch,
6     DeadlineExpired,
7     CorruptMessage,
8     UnknownRequest,
9 };
10
11 ReplyDecision classify_reply(const MessageEnvelope& reply,
12                             const DistributedTaskRecord& task,
13                             std::uint64_t now_ns) {
14     if (reply.payload_checksum == 0) {
15         return ReplyDecision::CorruptMessage;
16     }
17     if (reply.driver_epoch != task.driver_epoch) {
18         return ReplyDecision::StaleEpoch;
19     }
20     if (reply.task_id != task.task_id ||
21         reply.request_id != task.active_request_id) {
22         return ReplyDecision::UnknownRequest;
23     }
24     if (now_ns > task.deadline_ns) {
25         return ReplyDecision::DeadlineExpired;
26     }
27     if (reply.attempt != task.current_attempt) {
28         return ReplyDecision::LateAttempt;
29     }
30     if (task.state == DistributedTaskState::Committed) {
31         return ReplyDecision::DuplicateOfCommitted;
32     }
33     return ReplyDecision::AcceptForCommit;
34 }

```

真实代码还要查询持久化提交表，不能只看内存里的 `DistributedTaskRecord`。但这个流程展示核心纪律：成功响应只是一个事件。事件进入状态机以后，状态机根据当前事实决定它是否还有权改变结果。

响应处理先验证身份，再尝试提交



响应不能越过状态表直接写 manifest。任何身份不匹配都只能成为可观测事件，不能改变外部结果。

图示：本图要证明的命题是：分布式成功路径也必须先做身份验证，最后才尝试提交。

Trace 替代跨进程调用栈 本地函数用调用栈复盘因果，分布式系统用 trace 复盘因果。每个事件至少写 trace id、request id、attempt、component、state、time 和 reason。响应丢失事故的 trace 应能看到 send、deliver、execute、write temp block、drop reply、timeout、retry、commit、late reply。

Trace 不是为了日志好看。只有 Driver 日志，无法知道 worker 是否写过输出；只有 worker 日志，无法知道 Driver 是否已经放弃；只有平均延迟，无法知道慢在网络、队列、执行还是提交。Trace 把局部事实按身份连接起来，变成可审查事件序列。

16.5 重放日志和确定性调试

Trace 告诉我们发生过什么，replay log 则让我们把状态机输入重新喂一遍。这两个东西容易混淆。Trace 可以有采样、聚合和面向人的文字；replay log 必须是机器可读、顺序明确、足以重建状态机决策的输入序列。响应丢失、迟到回复、乱序消息、Driver 重启这些错误，不应该靠“多跑几次也许复现”，而应该变成可重放测试。

记录边界事件，不记录内部猜测 重放日志应该记录状态机边界输入：消息发送、消息投递、消息丢弃、timer 触发、worker 完成、commit 尝试、Driver crash、recovery start。不要记录“我觉得 worker 可能失败了”这种推断，也不要每个循环变量都写进去。重放的目标是：同一份输入事件序列，在同一份初始 checkpoint 上，必须产生同样的 commit 表、manifest 和拒绝计数。

Listing 16.6 可重放事件的教学形态

```
1 enum class DistributedEventKind : std::int32_t {
2     MessageSent,
3     MessageDelivered,
4     MessageDropped,
5     TimerFired,
6     WorkerCompleted,
7     CommitAttempted,
8     DriverCrashed,
9     RecoveryStarted,
10 };
11
12 struct DistributedReplayEvent {
13     std::uint64_t event_id = 0;
14     DistributedEventKind kind = DistributedEventKind::MessageSent;
```

```

15  std::uint64_t trace_id = 0;
16  std::uint64_t request_id = 0;
17  std::uint32_t task_id = 0;
18  std::uint32_t attempt = 0;
19  std::uint32_t driver_epoch = 0;
20  std::uint64_t logical_time_ns = 0;
21  };

```

这里用 `logical_time_ns`，不是为了伪装成真实时钟，而是为了给 timer 和 deadline 一个可重放的时序。测试时钟由 harness 推进，业务代码不能直接读系统墙钟。否则同一份日志今天能复现，明天因为调度抖动又不能复现。

重放要比较外部事实 重放完成后，不应该只比较日志行数。真正的断言是外部事实：每个 logical task 是否最多一个 commit，manifest checksum 是否一致，late reply 计数是否一致，orphan block 清理是否一致，Busy 后 in-flight 是否下降。若重放只验证“程序没有崩溃”，它仍然可能默默重复提交。

事故	Replay 输入	必须比较的结果
成功响应丢失	deliver request、worker done、drop reply、timer fire	retry 创建新 attempt，commit 仍唯一
旧响应迟到	attempt 1 commit 后再 deliver attempt 0 reply	late reply 增加，manifest 不变
Driver 重启	commit 前后分别 crash，再 recovery	commit 前不承认临时文件，commit 后必须承认
协议不兼容	deliver unsupported feature reply	rejected trace 增加，不能提交
Busy 风暴	连续 busy 和 timer fire	in-flight 降低，retry 受预算约束

Snapshot 缩短重放，不改变语义 长时间运行的系统不能每次从事件零开始重放。可以周期性写 snapshot：状态表、提交表、manifest offset、worker backpressure 摘要和最后 event id。恢复测试从 snapshot 开始读后续事件。Snapshot 只是加速，不是新的事实来源；若 snapshot 和 manifest 冲突，恢复仍要按本章前面的权威记录规则处理。

确定性调试不是掩盖并发问题 重放日志能把协议状态机变成确定性测试，但不能自动证明数据竞争不存在。MessageBus、状态表和提交表内部仍要遵守线程安全规则。教学项目最简单的做法是让状态机单线程消费事件，worker 线程只产生 completion 事件；等协议正确后，再讨论多线程执行器如何提高吞吐。先把语义做确定，再优化并发度，这比一开始把所有东西放进共享队列更容易学清楚。

16.6 幂等提交和 Manifest

Exactly once 不能解释成“远端函数只执行一次”。在真实故障下，执行可能发生零次、一次或多次。工程上更可实现的目标是：外部可见提交只生效一次。Worker 可以重复算，临时文件可以重复写，响应可以重复到达，但最终 manifest 只能接受一个结果。

提交表保存结果，而不只是保存见过的 ID 很多错误实现只保存一个 set：见过某个 request id 就拒绝重复。这不够。若第一次请求已经完成但响应丢失，客户端重试时需要得到同一语义结果；只知道“见过”不能告诉客户端上次结果是什么。提交表应保存 accepted attempt、block id、checksum、record count、bytes、commit state 和错误分类。

Listing 16.7 提交表记录外部可见事实

```

1  enum class CommitState : std::int32_t {
2      Empty,

```



```

3   Preparing,
4   Committed,
5   RejectedDuplicate,
6   Failed,
7 };
8
9 struct CommitRecord {
10  std::uint32_t job_id = 0;
11  std::uint32_t stage_id = 0;
12  std::uint32_t task_id = 0;
13  std::uint32_t accepted_attempt = 0;
14  std::uint64_t block_id = 0;
15  std::uint64_t checksum = 0;
16  std::uint64_t record_count = 0;
17  CommitState state = CommitState::Empty;
18 };

```

提交表要和 manifest 发布在同一边界。若先记录 committed，再写 manifest 失败，恢复时会看到状态冲突；若先写 manifest，再状态表没记录，重试可能重复提交。教学项目可以把二者简化成同一个 manifest 文件或一个事务性 append；真实系统会用日志、数据库事务、WAL 或共识日志保证这个边界。

重复请求和重复响应不同 重复请求发生在客户端重试，服务端可能要返回旧结果或拒绝旧 attempt。重复响应发生在网络重传、客户端重启或旧完成迟到，Driver 要根据状态表决定是否接受。两者都叫 duplicate，但处理位置不同。服务端侧等表保护执行和副作用，Driver 侧提交表保护外部可见结果。

临时输出不是提交 Worker 写出临时 block，不表示 reduce 可以读取。Reduce 只读 manifest 中 accepted 的 block。临时目录可能有多个 attempt 输出、半写文件、过期文件和诊断文件。恢复时以 manifest 为权威，扫描临时目录只是清理或补偿。这个模型直接承接第 15 章可靠写入。

16.7 Deadline、重试预算和背压

不受控制的重试会放大故障。一个 worker 慢，Driver 超时重试；重试增加队列压力，worker 更慢；更多请求超时，更多重试进入系统，最后形成重试风暴。分布式系统必须把 deadline、重试预算和 backpressure 写进协议，而不是藏在 sleep 和日志里。

重试预算按逻辑请求计算 预算属于 logical task，而不是某一次 socket 调用。task 7 attempt 0 超时后，attempt 1 不能重新获得完整无限预算。预算至少包括最大 attempt 数、总 deadline、退避策略和是否允许 speculation。若剩余时间不足以完成一次合理执行，就应失败或降级，而不是启动注定迟到的 attempt。

Busy 是控制信号，不是普通失败 Worker 队列满、磁盘慢、内存高水位或 shuffle 写出阻塞时，应该返回 Busy 或 backpressure，而不是伪装成 Fatal。Busy 表示“我现在不能接更多工作，稍后或换节点”。Driver 对 Busy 的处理应是退避、减少该 worker in-flight、调整调度，而不是立即把 task 标成失败或疯狂重试。

Listing 16.8 节点背压的协议摘要

```

1 struct WorkerBackpressure {
2  std::uint32_t worker_id = 0;
3  std::uint32_t route_epoch = 0;
4  std::uint64_t inflight_tasks = 0;
5  std::uint64_t inflight_bytes = 0;
6  std::uint64_t retry_after_ns = 0;
7  bool accepts_new_tasks = true;

```

```
8 };
```

这和第 15 章 write queue 的高水位线同构。单机里写盘慢导致 write queue high water；集群里某个 worker 慢导致 per-worker in-flight high water。尺度变了，控制原则没变。

慢节点不是失败节点 慢节点要分类型处理。CPU 慢、磁盘慢、网络慢、队列积压、GC 或系统暂停，对应不同动作。盲目把慢节点标为 failed，会触发不必要重试和数据迁移；完全不处理慢节点，会拖长尾。Speculation 可以缓解长尾，但必须和幂等提交绑定：多个 attempt 可以同时执行，manifest 只能接受一个。

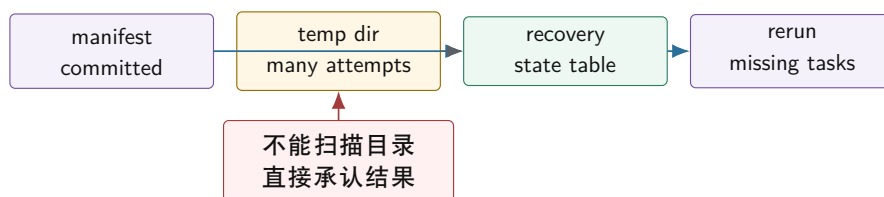
16.8 Epoch、恢复和旧事件隔离

Driver 重启以后，内存里的 Running、TimedOut、Retrying 都消失了，但远端 worker、临时 block、迟到响应和旧连接可能仍然存在。Epoch 用来隔离新旧控制面。新的 Driver epoch 启动后，旧 epoch 的响应、提交请求和心跳不能直接改变新状态。

Epoch 不是时间戳 Epoch 是控制面版本，不是墙钟时间。Driver 每次重启、leader 切换或恢复状态时获得新的 epoch。消息携带 epoch，接收方比较自己的当前 epoch。旧 epoch 消息可以记录为 stale，但不能提交结果。若系统没有 epoch，重启前的迟到响应可能在重启后污染新的任务表。

恢复时谁是权威事实 恢复不能从临时目录猜结果。权威事实应该是 manifest、提交表、checkpoint 或日志。恢复流程先读取权威记录，确认哪些 task 已提交；再扫描临时输出，清理不在 manifest 的孤儿文件；最后为未提交 task 重新创建 attempt。若先扫描目录再补 manifest，就会把半写文件或旧 attempt 当成结果。

恢复时 manifest 是权威入口



恢复先读权威提交记录，再用临时目录做清理和补偿。临时文件存在不代表业务结果已提交。

图示：本图要证明的命题是：崩溃恢复必须以提交事实为入口，而不是以残留物理文件为入口。

Driver commit 前后崩溃 commit 前崩溃：worker 输出可能存在，但没有权威记录，恢复时不能自动接纳。commit 后崩溃：manifest 已经接受结果，恢复时必须承认，即使内存状态丢失。这个边界和第 15 章写 manifest 前后崩溃完全一致，只是现在旧响应和远端 worker 也会迟到。

16.9 本机 MessageBus：可控故障先于真实网络

真实网络故障很多，直接上多机器会让读者同时面对 socket、序列化、线程、日志和协议 bug。更好的路线是先实现本机 MessageBus：显式消息队列、可配置延迟、丢包、乱序、重复、Busy、队列容量和 timer。它不是真实网络，但能稳定复现分布式协议错误。

故障规则是核心功能 MessageBus 不是一个简单队列，而是故障注入器。每条消息可以按规则 drop、delay、duplicate、reorder。Timer 驱动状态表，而不是业务线程 sleep。Inbox 有界，队列满时返回 Busy 或 backpressure。只要这些规则足够明确，读者就能复现响应丢失、迟到 reply、重复 reply 和 Driver 重启。

Listing 16.9 MessageBus 故障规则的教学形态

```
1 enum class FaultAction : std::int32_t {
```

```

2   Deliver,
3   Drop,
4   Delay,
5   Duplicate,
6   Reorder,
7 };
8
9 struct MessageFaultRule {
10     std::uint64_t rule_id = 0;
11     FaultAction action = FaultAction::Deliver;
12     std::uint64_t delay_ns = 0;
13     std::uint32_t match_task_id = 0;
14 };

```

多进程会暴露新的边界 本机 MessageBus 仍然会掩盖一部分问题：Driver 和 Worker 可能不小心共享同一个全局变量，配置更新看起来同时生效，C++ 指针在日志里似乎能代表对象身份，队列里的消息也像完整对象一样一次到达。因此本机阶段要主动约束自己：消息只通过 envelope 和 payload 传递，状态只通过显式事件改变，测试禁止 Worker 直接读 Driver 的状态表。这样迁移到多进程时，协议语义不会重写。

Linux 观察只服务协议问题 ss 可以看连接状态，但不能证明业务请求安全；tcpdump 可以看包，但 TCP 字节流不等于应用消息边界；strace 可以看系统调用，但不能替代状态表；tcnetem 可以制造延迟和丢包，但测试通过仍要看 manifest 和提交表。工具是证据来源，不是协议设计本身。

16.10 从本机 MessageBus 到多进程

多进程迁移不是“把队列换成 socket”这么简单。它会把本机模拟阶段被隐藏的边界一次性暴露出来：地址空间隔离、进程生命周期、字节流 framing、配置一致性、部分读写、连接关闭、权限和临时文件清理。若前面章节的协议身份、状态表和 manifest 做得扎实，多进程只是更真实的传输层；若前面靠共享内存和对象引用偷懒，多进程会迫使你重写设计。

地址空间隔离会消灭隐式依赖 本机版本里，一个 TaskRequest 也许偷偷带着指向输入 buffer 的指针；一个 worker 也许能调用全局单例读取最新配置；一个测试也许直接检查 Driver 内存对象。多进程以后这些路径全断了。跨进程只能传值、文件描述符、路径、block id 或网络地址，不能传对象身份。读者要把这当成好事：它迫使协议说清楚什么是事实，什么只是进程内部实现细节。

本机阶段容易偷懒的地方	多进程暴露的问题	正确设计动作
共享全局配置	节点看到的配置版本不同	配置带 epoch/version，消息记录使用版本
传递对象指针	地址空间不同，指针无意义	传递稳定 id、offset、path 或 serialized value
队列保证完整对象	socket 是字节流	定义 frame header、长度、校验和解析状态机
线程退出即清理	进程 crash 留下临时文件	以 manifest 恢复，清理 orphan block
连接关闭当失败	关闭只说明传输断开	task 状态进入 unknown，再由 deadline/查询/重试处理

Framing 是协议的一部分 TCP 提供有序字节流，不提供应用消息边界。读取方必须先读 frame header，再根据长度读取 payload；若长度超过上限，立即拒绝；若 checksum 不匹配，进入 corrupt message；若连接中途关闭，当前 frame 是 incomplete transport event，不是业务 Fatal。这个状态机和第 15 章短读短写完全一致，只是现在读写对象从文件变成网络连接。

```

1 constexpr std::uint32_t DISTRIBUTED_FRAME_MAGIC = 0x44535431U;
2 constexpr std::uint32_t DISTRIBUTED_MAX_FRAME_BYTES = 16777216U;
3
4 struct NetworkFrameHeader {
5     std::uint32_t magic = DISTRIBUTED_FRAME_MAGIC;
6     std::uint16_t protocol_version = DISTRIBUTED_PROTOCOL_VERSION;
7     std::uint16_t header_bytes = 0;
8     std::uint32_t payload_bytes = 0;
9     std::uint64_t payload_checksum = 0;
10 };

```

这仍然不是让读者直接 `send(&header, sizeof(header))`。真实 wire format 要逐字段编码，固定大小端，明确 padding 不进入网络。这里的结构只是把 frame 需要的概念列出来：magic 识别协议，version 选择解析规则，header bytes 支持扩展，payload bytes 定义边界，checksum 防止损坏 payload 进入业务状态。

连接事件不是业务事件 连接建立、连接关闭、读超时、写失败，都属于 transport event。它们会影响消息是否可达，却不直接证明 task 成功或失败。连接关闭时，Driver 最多知道“这条连接上还有未完成 request 的状态未知”。正确动作是把相关 request 标成 transport unknown，等待 deadline、查询提交表、重试或进入恢复流程。把 connection reset 直接映射成 task failed，会重新犯 timeout 当 failure 的错误。

最小多进程实验矩阵 本章不要求做完整 RPC 框架，但至少应设计四类迁移实验。第一，partial frame：发送方多次写 header 和 payload，接收方必须正确拼帧。第二，worker crash after temp write：worker 写完临时 block 后崩溃，Driver 不得扫描目录承认结果。第三，driver restart with old reply：旧进程退出后，旧响应或旧连接事件不得污染新 epoch。第四，version mismatch：新 Worker 带未知 feature，旧 Driver 必须拒绝提交并留下 trace。

多进程实验	故障输入	验收重点
partial frame	header/payload 分片到达	frame parser 不越界，不误提交
connection close	reply 发送前连接关闭	request 进入 unknown，不直接 failed
worker crash	temp block 已写，reply 未发	恢复不扫描临时目录承认结果
old epoch reply	Driver 重启后旧 reply 到达	stale epoch 计数增加，manifest 不变
version mismatch	未知 feature 或过高版本	协议拒绝，trace 可解释原因

16.11 工业设计参照

读工业系统要读约束，不只读名字 优秀工业设计不是把某个系统名字搬进教材。读者要问五个问题：它面对的故障和规模约束是什么；它用什么身份、日志或状态机把不确定性压住；它购买了什么能力；它付出了什么复杂度；本书项目应该采用、简化还是暂时拒绝哪一部分。下面几个案例都按这个方式阅读。

工程案例

gRPC deadline 和 status gRPC 等 RPC 系统把 deadline、status、metadata 和 cancellation 放进调用模型，解决的是跨服务调用的传输边界：调用不能无限等，错误要有分类，取消要能传播，metadata 要能携带认证、trace 或路由信息。它的核心机制是把 deadline 随请求向下游传播，并把状态码作为协议的一部分返回，而不是让每个调用点随手写一个 timeout。

代价是：框架只能告诉你调用层发生了什么，不能替业务决定远端副作用是否已经发生。Deadline exceeded 不等于远端没有执行，cancelled 也不一定撤销远端已经完成的写入。本书项目应该借鉴 deadline/status/metadata 的显式化，但不能借用“RPC 返回失败所以 task failed”的错误直觉。提交表、attempt 和 manifest 仍由业务协议负责。

工程案例

Spark task attempt 和 speculative execution Spark 这类计算引擎必须处理慢节点、失败节点、Executor 丢失和长尾任务。它允许同一个 logical task 出现多个 physical attempt，甚至用 speculative execution 让另一个节点并行跑同一份工作。核心机制是把 task id、stage attempt、task attempt 和输出提交分开，使调度器能多次执行，但下游只承认被提交协议接受的结果。

代价是额外计算、更多临时输出、attempt 清理和更复杂的指标解释。Speculation 不是免费加速器，它会占用集群资源，也会放大非幂等输出的风险。本书项目可以采用 task/attempt/manifest 的身份模型，暂时简化掉复杂的资源调度策略；只有当 late reply 和 duplicate commit 已经被测试覆盖后，才适合加入 speculation。

工程案例

MapReduce output commit MapReduce 的 output commit 设计面对一个现实约束：task 可能重试，worker 可能崩溃，输出文件系统可能只看到物理文件，不能理解哪个 attempt 才是语义结果。核心机制是让 worker 写 attempt 私有的临时位置，再由 committer 决定哪些输出进入最终可见目录或 manifest。这样重复执行不会自动变成重复业务结果。

代价是提交协议依赖底层存储能力。某些文件系统的 rename 接近原子操作，某些对象存储的目录语义和一致性模型更弱，committer 就要更谨慎。本书项目不要假设“文件出现在目录里就是提交”，而要让 manifest 成为唯一读取入口；临时目录只用于清理、诊断和恢复补偿。

工程案例

Kafka producer id 和 sequence number Kafka 的幂等生产路径面对的约束是高吞吐追加日志和不可避免的网络重试。Producer 发送记录后可能没收到 ack，于是重试；Broker 必须分辨这是新记录还是重复记录。核心机制是 producer id、producer epoch 和 per-partition sequence number，Broker 用这些身份信息识别重复、乱序和旧 producer 事件。

代价是协议状态更重，Broker 要维护生产者状态，异常恢复时要处理 epoch 变化和序列约束。它给本书项目的启发不是照搬 Kafka 事务，而是理解“幂等必须落在身份和日志上”。我们的 request id、attempt、driver epoch 和 commit table，是教学版的同类机制。

工程案例

Kubernetes controller 的 reconcile 思路 Kubernetes 控制器处理的是另一类分布式问题：观察到的事件可能丢失、重复、乱序，控制器进程也可能重启。它的核心思路不是相信每个 watch 事件都完整可靠，而是不断读取当前期望状态和实际状态，执行幂等 reconcile。对象的 uid、resource version、generation 和 status，把“我看到了什么事件”和“系统现在应该变成什么状态”分开。

这个案例对本章很有价值：事件是提示，不是最终事实；状态机要能重复运行而不制造重复副作用。代价是收敛式控制通常不是最低延迟路径，也需要处理冲突和版本检查。本书项目可以借鉴 reconcile 的幂等思想，用 manifest/commit table 作为当前事实，不要让单条 reply 直接支配最终状态。

16.12 项目落地

Compute Systems Engine 的第 16 章项目不要求立刻写真实集群。目标是把单机 completion 模型扩展成本机分布式模拟：Driver、Worker、MessageBus、CommitTable、Manifest 和 Trace 全部显式化。

实现路线 第一版，把远程调用改成显式消息：Driver 发送 `TaskRequest`，Worker 返回 `TaskReply`。第二版，加入协议版本和 feature 检查，但只支持一个版本。第三版，加入状态表和提交表，但先不重试。第四版，加入故障注入，只开一个故障：成功响应丢失。第五版，加入 retry、attempt、deadline 和 late reply 处理。第六版，加入 Busy 和 per-worker backpressure。第七版，把状态表和 manifest 写成可恢复文件。第八版，加入 replay log。第九版，把 MessageBus 的一个路径迁移到多进程 frame parser。

实验矩阵

实验	故障输入	验收重点
响应丢失	Worker 成功, reply drop	timeout 是 unknown, retry 创建新 attempt
旧响应迟到 Driver 重启 Busy 响应 重复请求 乱序消息	attempt 0 晚于 attempt 1 到达 commit 前后分别崩溃 worker inbox 高水位 同一 request 重放 reply 顺序打乱	manifest 只接受一个 attempt 恢复以 manifest 为权威 Driver 退避并减少 in-flight 服务端不重复提交副作用 状态表按身份处理，不按到达顺序处理
协议不兼容 事件重放	未知 feature 或过高版本 固定 replay log 输入	拒绝提交并留下可解释 trace commit 表、manifest 和计数完全一致
多进程分帧	header/payload 部分到达	frame parser 正确拼帧并保护长度上限

DistributedRunReport 项目报告至少输出：task 总数、attempt 总数、request 总数、timeout 数、retry 数、late reply 数、stale epoch 数、protocol rejected 数、corrupt frame 数、busy 数、committed 数、duplicate rejected 数、orphan block 清理数、replay event 数、每个 worker in-flight 峰值、每个队列高水位次数。没有这些指标，分布式协议只能靠猜。

Listing 16.11 分布式模拟报告摘要示例

```
tasks.total=128
attempts.total=137
reply.late=5
reply.stale_epoch=2
protocol.rejected=1
frame.corrupt=0
commit.accepted=128
commit.duplicate_rejected=9
worker.busy=14
replay.events=402
orphan_blocks.cleaned=9
```

源码阅读任务

源码阅读任选一条窄路径：

1. 阅读一个 RPC 框架的 deadline、status 和 cancellation 文档或源码路径，说明框架解决了什么、没有替业务解决什么。

2. 阅读 Spark、MapReduce 或类似系统的 task attempt / output commit 设计，重点看重复 attempt 如何避免重复输出。
3. 阅读 Kafka 或类似消息系统的 producer epoch、sequence number 或幂等生产路径，说明它如何处理重试和重复。
4. 阅读 Kubernetes controller 或类似控制系统的 reconcile 路径，说明它如何处理重复、乱序和进程重启。
5. 阅读 Linux 网络观察工具的输出，说明 `ss`、`tcpdump`、`strace` 能证明什么，不能证明什么。

验收与评分标准

本章项目评分建议：

1. 正确性 30 分：timeout、retry、late reply、duplicate reply、Driver 重启都不会重复提交。
2. 协议设计 20 分：消息身份、attempt、request、epoch、deadline、status 字段完整且语义清楚。
3. 版本和恢复 20 分：版本不兼容会被拒绝，manifest 和提交表是权威事实，commit 前后崩溃行为明确。
4. 背压、重放和多进程 15 分：Busy、deadline、retry budget、replay log、frame parser 有指标和测试。
5. 工业判断 15 分：能说明从 RPC、Spark、MapReduce、Kafka、Kubernetes 等系统借鉴了什么，哪些复杂度暂不引入。

16.13 复查清单

一、Timeout 是否仍被当成失败 若 timeout 分支直接 `mark_failed`，没有 unknown、retry budget 和 attempt 状态，本章核心没有学到。

二、响应是否先验证身份 成功响应必须先检查 task、attempt、request、epoch、deadline 和 checksum，再尝试提交。不能直接写 manifest。

三、协议版本是否真的生效 未知 feature、过高版本、过低版本、未知成功状态都不能静默进入提交路径。持久化 manifest 的兼容规则必须比临时消息更严格。

四、提交是否唯一 多个 attempt 可以执行，manifest 只能接受一个。重复响应和迟到响应必须可观测，但不能改变终态。

五、恢复是否以权威记录为入口 恢复先读 manifest 或提交表，再清理临时输出。不能扫描目录直接承认结果。

六、背压是否写进协议 Worker 忙时应返回 Busy 或 backpressure，Driver 应退避并调整 in-flight。不能把 Busy 当 Fatal，也不能立即疯狂重试。

七、重放是否比较外部事实 Replay 测试要比较 commit table、manifest、计数和 orphan 清理结果。只比较日志行数或程序退出码不够。

八、多进程是否只替换传输层 迁移到 socket 后，状态机语义不应重写。连接关闭、partial frame 和进程崩溃都必须进入已有 unknown、deadline、recovery 和 manifest 模型。

九、是否为第 17 章留下边界 第 16 章解决单个 task attempt 的消息、重试和提交。第 17 章会把问题扩展到分片归属、复制日志和共识。不要在第 16 章里提前把所有控制面问题塞进一个“大 leader”。

16.14 本章习题

习题与作业

习题一：为响应丢失事故写事件表。事件至少包括 send、execute、write temp block、reply drop、timeout、retry、commit、late reply。标出每个事件能证明什么，不能证明什么。

习题二：设计 `MessageEnvelope`。解释每个字段对应哪个故障问题。再说明为什么这个结构不能直接 `memcpy` 到 socket。

习题三：设计协议版本规则。给出新增可选字段、新增必需字段、改变字段含义、扩展 enum、删除持久化字段五种变化的兼容判断。

习题四：写一个 `classify_reply` 的测试矩阵，覆盖 stale epoch、late attempt、unknown request、deadline expired、duplicate committed 和 accept for commit。

习题五：设计提交表。要求同一个 logical task 的多个 attempt 只能有一个进入 Committed。说明 commit 前崩溃和 commit 后崩溃的恢复动作。

习题六：设计 MessageBus 故障注入规则。至少支持 drop、delay、duplicate、reorder 和 Busy。说明如何保证实验可复现。

习题七：写一份 replay log，让它稳定复现“attempt 0 成功但 reply 丢失，attempt 1 提交，attempt 0 reply 迟到”的事故。给出重放后的断言。

习题八：比较 timeout、deadline 和 retry budget。构造一个没有预算的重试风暴，再给出修复方案。

习题九：实现一个最小 frame parser 测试。要求 header 和 payload 可以分片到达，超过最大长度必须拒绝，checksum 错误不能进入业务状态机。

习题十：阅读一个工业系统的 task attempt、幂等消息或 reconcile 设计，按“约束、核心机制、购买能力、成本、迁移到本项目的方式”写报告。

分片、复制与共识

第 16 章只跟踪一个 task attempt：请求可能丢、响应可能迟到、提交必须唯一。本章把问题扩大到一段状态的归属。日志统计系统按 key 做 reduce，`event=login` 突然成为热点，partition 12 的队列持续上涨。Driver 决定把 partition 12 从 worker A 迁到 worker B，并把这个热 key 拆成两个子分片。迁移过程中，旧客户端仍按旧路由把写发给 A，新客户端已经按新路由把写发给 B；A 网络短暂隔离后恢复，还带着旧 owner 身份继续接受写；B 只追到一半增量就开始服务读。最后，两个 worker 都认为自己有权提交同一段 key range 的结果。

这个事故比术语表更能说明本章。为什么需要分片？因为单个 owner 扛不住所有 key，也不能让热点 key 把整个 reduce 阶段拖成长尾。为什么需要路由版本？因为客户端、worker 和 Driver 在迁移期间会同时看到新旧归属事实。为什么需要复制？因为只有一个 owner 时，owner 慢或死就让这段状态不可用。为什么需要共识或等价的强协调？因为“partition 12 现在归谁、哪个 manifest generation 已提交”这类控制面事实不能靠两个节点各自宣布。为什么需要 fencing？因为旧 owner 即使还活着，也必须失去写入权。

本章的主线是状态归属。分片先回答“状态放在哪里、请求送给谁”；复制回答“owner 故障后谁有足够状态接管”；共识回答“多个控制面可能同时行动时，哪条顺序才算真”。三者不是并列名词，而是同一个工程问题逐步升级后的三个回答：容量不够时拆开，单点不可靠时复制，多个副本或多个控制面可能争权时建立唯一提交顺序。

学习目标

学完本章后，读者应能完成八件事：

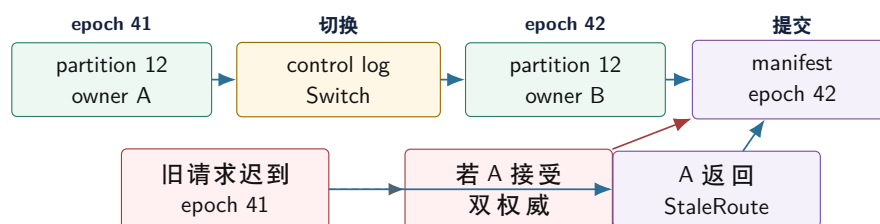
1. 从旧 owner 迟到写入事故解释分片、路由 epoch、fencing 和提交权。
2. 比较 hash 分片、range 分片、虚拟分片和热点 key 拆分的适用条件。
3. 设计可恢复的再分片状态机，而不是把再分片写成一次后台拷贝。
4. 说明复制提交点如何影响写成功、读新鲜度、故障恢复和尾延迟。
5. 用 quorum 的交集解释为什么多数确认能保留已提交事实，也能说出它不能自动解决的问题。
6. 区分 stale read、monotonic read、linearizable read，并把读语义写进接口。
7. 解释 leader term、log index、commit index 和 snapshot 的工程意义。
8. 在 Compute Systems Engine 中落地 partition assignment、route epoch、replica progress、control log 和实验报告。

17.1 旧 Owner 事故：状态归属先于算法

先不讲 Raft，也不讲 CAP。只看一条迟到写。Epoch 41 时，partition 12 归 worker A；Driver 发布 epoch 42 后，partition 12 迁到 worker B；一个 epoch 41 的写请求因为网络延迟，在 epoch 42 生效后才到达 A。若 A 只按 partition id 判断“我曾经负责过它”，它会接受旧写；B 同时接受新写，系统就出现两个权威。

这就是分布式状态的第一条纪律：同一个名字不代表同一个权利。Partition id 相同，不代表 owner 仍然有效；worker 还活着，不代表它还有提交权；文件存在，不代表它已经进入 manifest；leader 能响应，不代表它仍处于当前 term。每个能改变外部状态的请求，都必须携带 ownership epoch 或等价的 fencing token。

旧 owner 迟到写入：没有 epoch 就会出现两个权威



Epoch 的作用不是刷新缓存，而是把“曾经拥有”和“现在有权”分开。旧请求可以被记录、转发或拒绝，但不能悄悄提交。

图示：本图要证明的命题是：分片正确性的第一层不是哈希函数，而是带版本的所有权。

状态归属包含五个问题 一个 partition 的归属至少要回答五件事：谁是当前 owner，哪个 epoch 证明它是当前 owner，当前处于 serving 还是 migrating，哪些副本拥有足够新状态，哪个控制日志位置已经提交。只知道 key 经过 hash 落到 partition 12，不足以处理故障、迁移和恢复。

旧消息是正常输入 旧 epoch 请求、旧 owner 响应、旧 leader 心跳、旧 manifest 写入，都不是“理论上不会发生”的异常。网络延迟、进程暂停、重试、恢复和连接复用都会制造旧事件。高质量系统不是祈祷旧事件不出现，而是让每条旧事件进入状态机后只能产生受控结果：拒绝、转发、记录、清理或人工介入。

Listing 17.1 分片请求必须携带所有权证据

```

1 enum class PartitionRequestKind : std::uint16_t {
2     Read,
3     Write,
4     CommitOutput,
5 };
6
7 struct PartitionRequest {
8     std::uint64_t request_id = 0;
9     PartitionRequestKind kind = PartitionRequestKind::Write;
10    std::uint32_t partition_id = 0;
11    std::uint64_t route_epoch = 0;
12    std::uint64_t leader_term = 0;
13    std::uint32_t owner_worker = 0;
14    std::uint32_t attempt = 0;
15    std::uint64_t input_block_id = 0;
16    std::uint64_t input_checksum = 0;
17 };
  
```

这个结构仍然是教学形态。真实系统还要携带认证、payload、长度、校验和版本兼容字段。本章关心的是控制身份：partition、epoch、term、owner 和 request id 必须进入提交路径。若接口只提供 `commit(partition_id, block_id)`，调用者很容易绕过所有权检查。

17.2 分片：集群尺度的数据布局

分片可以看成单机数据布局在集群尺度上的版本。单机上，数组怎样落到 cache line、页和 NUMA 节点决定访问成本；集群里，key、任务、shuffle block 和副本怎样落到 worker 决定网络成本、负载均衡、故障半径和扩容方式。分片不是取模公式，而是访问模式、负载分布和恢复边界的共同设计。

Hash、Range 和虚拟分片 Hash 分片把 key 打散，适合点查、均匀 key 和简单扩容，但范围

扫描要访问多个分片，也不能自动拆开单个热点 key。Range 分片保留顺序，适合范围查询和排序，但递增 id、时间戳或热门区间会把写入压到某个范围。虚拟分片把逻辑分片数量做得比物理 worker 多，便于迁移、限流和负载均衡；它类似单机运行时把任务块数量做得多于线程数，让调度器有调整空间。

策略	适合场景	主要风险	项目取舍
Hash 分片	均匀 key、点查、普通 shuffle	热点 key 仍集中，范围扫描差	默认实现
Range 分片	排序、范围查询、时间窗口扫描	递增写入和热门区间倾斜	作为对照实验
虚拟分片 热点 key 拆分	动态扩缩容、迁移、限流 单个 key 压垮 reduce	元数据和调度更复杂 需要二级合并和版本记录	推荐加入 必做实验

热点 key 不能只靠加节点解决 若 `event=login` 占大多数记录，普通 hash 会把它稳定地发到同一个 reduce partition。增加 worker 数量只会让冷 key 更分散，热点仍然长尾。常见处理包括 map-side combine、热 key 拆成多个子 key、二级 reduce、缓存、限流和业务异步化。每种方法都要回答两个问题：最终输出是否仍是原 key 的精确结果，监控是否还能把子 key 聚回业务 key。

Listing 17.2 热点 key 拆分的执行身份和业务身份要分开

```
normal key:
    reduce_partition = hash(key) % reduce_count

hot key:
    local_shard = hash(input_split, local_counter) % hot_shard_count
    reduce_partition = hot_route[key][local_shard]
    final output key = key
```

子 key 是执行层身份，不是业务层 key。Manifest 要记录 hot route version、子分片范围和最终合并规则。否则作业中途改变热点策略，会让同一个业务 key 的 partial 按两套规则落盘，后续 reduce 无法解释。

分片函数必须有版本 分片函数、key 规范化、hash seed、热点列表和二级合并规则都属于协议。作业进行中改变这些规则，不带版本就会让相同 key 前后落到不同位置。Shuffle block、manifest entry 和 trace 都应记录 partition version。Reduce 端要么只合并同一版本的数据，要么执行明确的迁移转换，不能把不同规则下的 block 混在一起。

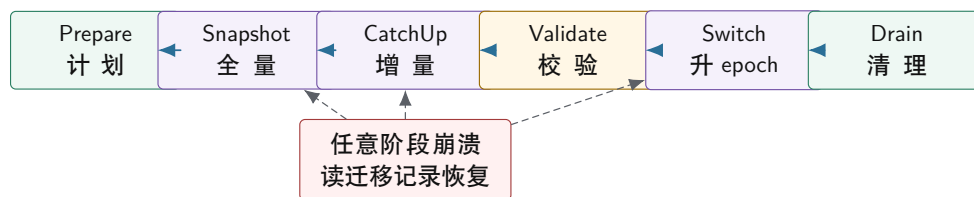
17.3 路由 Epoch 和再分片状态机

再分片最容易被误讲成“把数据从 A 拷到 B”。真正危险的是迁移期间的新写、旧写、迟到响应和崩溃恢复。历史数据复制只是第一步；复制期间旧 owner 仍可能接收写，新 owner 可能还没有追平，客户端路由表可能过期，Driver 可能重启。一个可靠再分片必须写成状态机。

冻结、追赶和双写 冻结写入最容易证明：旧 owner 停止写，复制完成后切 epoch，新 owner 服务。代价是不可用窗口。日志追赶减少停顿：旧 owner 生成快照后继续写 WAL，新 owner 加载快照并追增量，追到切换点再发布新 epoch。代价是需要可靠日志、增量重放和追赶速度控制。双写看起来可用性最高：旧 owner 和新 owner 都写成功才返回。代价是一边成功一边失败、重试重复写、读路径选择和恢复依据都变复杂。

教学项目应先实现冻结或日志追赶，再把双写作为反例分析。若无法回答 request id 如何去重、部分成功如何补偿、旧响应如何处理、恢复时以哪边日志为准，就不要把双写当成“平滑迁移”的简单方案。

再分片是可恢复状态机，不是一次拷贝



每个阶段都要有持久化迁移记录。恢复程序根据阶段继续复制、继续追增量、确认 switch 是否已提交，或清理旧 owner。

图示：本图要证明的命题是：再分片的正确性来自阶段和恢复记录，而不是后台 copy 的成功日志。

Listing 17.3 分区归属记录的教学形态

```

1 enum class OwnershipState : std::uint8_t {
2     Serving,
3     PreparingMove,
4     CopyingSnapshot,
5     CatchingUp,
6     Switching,
7     Draining,
8     Retired,
9 };
10
11 struct OwnershipRecord {
12     std::uint32_t partition_id = 0;
13     std::uint64_t route_epoch = 0;
14     std::uint64_t leader_term = 0;
15     std::uint32_t owner_worker = 0;
16     std::uint32_t target_worker = 0;
17     std::uint64_t snapshot_version = 0;
18     std::uint64_t catchup_log_index = 0;
19     OwnershipState state = OwnershipState::Serving;
20 };
  
```

提交接口必须检查当前权利 让非法提交难以写出来，是 C++ 接口设计的一部分。提交函数应要求传入 request 和当前 ownership record，并在函数内部检查 partition、epoch、term、owner 和状态，而不是把这些检查散落在调用方。

Listing 17.4 提交前验证所有权和任期

```

1 enum class PartitionCommitDecision : std::int32_t {
2     Accept,
3     StaleRoute,
4     StaleTerm,
5     WrongOwner,
6     NotServing,
7 };
8
9 PartitionCommitDecision can_commit_partition_output(
  
```



```

10     const PartitionRequest& request,
11     const OwnershipRecord& owner,
12     std::uint64_t current_term) {
13     if (request.partition_id != owner.partition_id ||
14         request.route_epoch != owner.route_epoch) {
15         return PartitionCommitDecision::StaleRoute;
16     }
17     if (request.leader_term != current_term ||
18         owner.leader_term != current_term) {
19         return PartitionCommitDecision::StaleTerm;
20     }
21     if (request.owner_worker != owner.owner_worker) {
22         return PartitionCommitDecision::WrongOwner;
23     }
24     if (owner.state != OwnershipState::Serving) {
25         return PartitionCommitDecision::NotServing;
26     }
27     return PartitionCommitDecision::Accept;
28 }

```

这个函数不是完整分布式存储，只展示边界：提交不是“结果文件写完了”，而是“当前权利允许这个结果进入外部事实”。未来若把控制面换成真实 Raft，`leader_term` 的来源会改变，但提交前验证权利的接口形状仍然成立。

17.4 复制：写成功到底证明什么

只有一个 owner 时，owner 死了就无法继续服务。复制的动机很朴素：让其他副本拥有足够状态，在 owner 故障后可以接管。但复制不是“多写几份文件”。要问客户端看到成功时，哪些副本已经保存了这个事实；读请求从哪里读，允许多旧；故障恢复时哪个副本有资格成为新 owner；落后副本怎样补齐；删除怎样传播。

提交点决定恢复语义 同一条写，在不同提交点下含义完全不同。Leader 内存返回，延迟最低，但 leader 崩溃可能丢已承诺结果；本地 WAL 返回，单节点崩溃恢复更好，但 leader 永久丢失时其他副本可能不知道；多数副本持久化后返回，故障恢复更强，但写延迟和尾部成本上升；所有副本确认最保守，但最慢副本决定可用性。

提交点	购买的能力	主要成本	适合对象
Leader 内存	极低延迟	崩溃易丢已承诺结果	临时指标
Leader WAL	单机恢复清楚	leader 永久丢失仍危险	可重试任务状态
多数副本 WAL	已提交事实更可恢复	跨节点 RTT 和尾延迟	manifest、路由元数据
所有副本确认	最强副本一致	可用性受最慢副本限制	少数关键审计状态

WAL 是复制和 I/O 的连接点 Write-ahead log 的核心不是文件格式，而是状态改变先进入可恢复记录，再对外承诺。第 15 章讲过 write、completion、commit 不能混淆；这里同样成立。Leader 追加日志不等于 follower 持久化，多数确认不等于状态机已经应用，状态机应用不等于 snapshot 已经安全截断。要把 append、persist、commit、apply、snapshot 分开。

Listing 17.5 副本进度摘要

```

1 struct ReplicaProgress {

```

```

2  std::uint32_t worker_id = 0;
3  std::uint32_t failure_domain = 0;
4  std::uint64_t last_appended_index = 0;
5  std::uint64_t last_persisted_index = 0;
6  std::uint64_t last_committed_index = 0;
7  std::uint64_t last_applied_index = 0;
8  bool voting_member = true;
9  };

```

这些指标必须按 partition 或 replica group 维度记录。全局平均复制滞后小，不代表热点 partition 健康。一个关键 partition 的 follower 落后，故障时会直接影响可接管性。

复制和备份不同 复制解决在线可用性：某个副本不可用时，系统还能服务。备份解决历史恢复：误删、程序 bug、坏版本写入或运维误操作后，系统能回到过去某个正确版本。复制会快速传播错误删除，不能替代备份。备份要有快照时间点、保留策略、隔离权限、校验和恢复演练。Compute Systems Engine 的 checkpoint 也应如此：只写文件不算完成，必须能恢复并验证 input offset、manifest、去重表和输出 checksum。

17.5 Quorum 和读一致性

Quorum 模型常用 N 表示副本数， W 表示写入需要确认的副本数， R 表示读取需要查询的副本数。若 $R + W > N$ ，读集合和写集合至少有交集。这个交集能帮助读请求发现已提交版本，也能帮助新 leader 找到共同事实。但公式本身不是数据库。系统仍要处理版本比较、并发写、删除 tombstone、慢副本、读修复、反熵和冲突解决。

用恢复案例理解多数派 三副本 A、B、C，写版本 10。若 $W=2$ ，A 和 B 确认后返回成功，A 随后崩溃，B 和 C 中至少 B 知道版本 10。新 leader 从多数派中选出时，有机会看到已提交版本。若只有 A 写完就返回成功，A 崩溃后 B 和 C 都停在版本 9，系统没有证据证明版本 10 曾被承诺。多数派的意义不是公式漂亮，而是故障后保留共同事实。

读路径也有合同 写路径认真复制，读路径随便读 follower，用户仍可能看到旧状态。读语义要写进接口，而不是藏在实现里。

读语义	能保证什么	成本和适用场景
Best effort stale read	可能返回落后副本状态	延迟低，适合监控预览、搜索、缓存
Monotonic read	同一客户端不读到低于已见版本	需要客户端携带版本，适合用户会话
Read-your-writes	写后自己能读到新值	需要会话版本或 leader 路径
Linearizable read	读开始前已提交写必须可见	延迟高，适合路由、权限、manifest

版本号让旧读可解释 响应中带上 route epoch、leader term、log index、data version 或 replica lag，能让调用方知道自己读到哪个历史点。没有版本号，读到旧值只能猜。控制面读通常需要强读或等价保证；观测型读可以陈旧，但要诚实标注滞后边界。若调度器用旧 route 读决定 owner，会把请求发错；若监控看板读旧统计，只要标注版本，业务风险低得多。

读修复、反熵和 Merkle tree 副本不一致需要修复。读修复在读请求发现旧副本时顺手修复，热数据收敛快，但会增加读路径成本。反熵在后台比较副本并修复冷数据，前台稳定但收敛慢。大规模 key-value 系统常用 Merkle tree 之类摘要结构先比较范围哈希，再下钻到差异 key。教学项目不必实现完整 Merkle tree，但可以用 manifest checksum 学同一个原则：先比摘要，再决定是否重读或修复。

17.6 副本放置、成员变更和删除

复制数量不等于可靠性。三个副本如果在同一台机器、同一机架、同一电源域或同一个对象存储故障域里，某个物理故障可能同时影响它们。工业系统讨论副本时一定会讨论 failure domain：磁盘、机器、机架、可用区、网络交换机、维护批次和软件版本。副本放置不是运维细节，而是复制协议购买可靠性的前提。

故障域进入放置策略 本机教学项目也可以模拟故障域。给 worker 加 `failure_domain`，要求同一个 replica group 的 voting members 尽量分布在不同 domain。然后注入一个 domain 故障，看是否仍有 quorum。这个实验能让读者明白：N=3 只是副本数，真正的问题是这三个副本是否会被同一个故障同时打掉。

成员变更不能随手改副本列表 复制组扩容、缩容、替换机器或迁移副本时，quorum 的定义会变化。成员变更比普通写入更危险，因为旧配置和新配置如果没有安全交叠，可能出现两个多数派，各自认为自己能提交。成熟共识协议会把成员变更写入日志，用联合共识或分阶段切换保证交叠。教学项目可以简化为两阶段：先把新副本加入为 non-voting member 并追平日志，再在控制日志中提交配置切换，最后移除旧副本。不能直接把 vector 里的 worker id 改掉。

删除需要 tombstone 和安全点 复制系统中的删除不能简单等同于 `erase`。若某个副本离线时错过删除，恢复后可能把旧值重新传播回来。Tombstone 用删除标记表示“这个 key 已被删除”，让落后副本也能学习删除事实。Tombstone 需要保留到系统确信所有相关副本、日志和 snapshot 都越过删除点；保留太短会旧值复活，保留太长会增加空间和查询成本。Compute Systems Engine 里的旧 shuffle block 清理也同构：只有当 manifest generation、checkpoint 和恢复策略都不再需要某个 block 时，物理删除才安全。

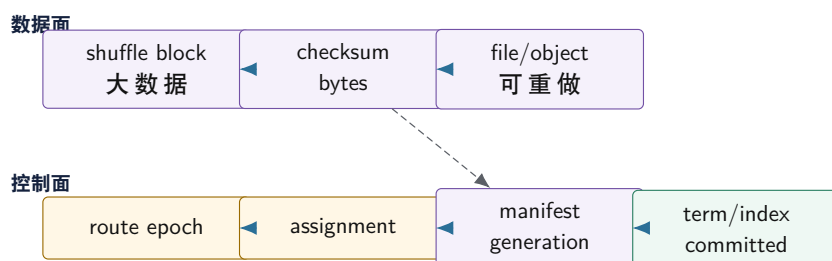
再平衡要受背压约束 副本迁移和再平衡会消耗网络、磁盘和 CPU。后台迁移如果不受限制，会和前台请求争资源，导致更多 timeout 和重试。工业系统通常限制同时迁移的 partition 数、每个 worker 的迁移带宽、每个 replica group 的落后上限，并在高水位时暂停迁移。这里又回到第 15 章和第 16 章：背压不是局部队列技巧，而是跨 I/O、运行时和分布式控制面的共同纪律。

17.7 Leader、Term 和共识边界

复制还没有解决控制面分裂。网络分区时，旧 Driver A 认为自己仍是 leader，新 Driver B 在另一侧也被选出。A 发布 epoch 42a，B 发布 epoch 42b，两个 manifest 都声称自己有效。Worker 如果只要听到 Driver 命令就执行，最终 reduce 会看到两个互斥真相。这就是共识边界：系统需要让少量关键控制面事实进入一条唯一顺序。

共识解决什么 Leader 决定谁能提出控制面写入；term 区分新旧领导权；log index 给控制操作排序；commit index 表示哪些操作已经被足够副本接受并可对外生效；apply 表示状态机已经执行已提交日志。共识的价值不是让操作更快，而是在故障、延迟和旧消息中保持一条可恢复历史。

共识日志只承载控制面事实，数据面走高吞吐路径



大数据块不必全部进入共识日志；决定哪些块有效的元数据必须进入强一致控制面。控制面太大，吞吐会垮；控制面太弱，结果会分叉。

图示：本图要证明的命题是：共识应用在“决定哪些数据有效”的小状态上，而不是替代数据面吞吐设计。

共识不解决什么 共识不自动解决业务幂等、热点 key、数据布局、磁盘损坏、查询优化、外部 I/O 重放和读路径选择。请求去重表仍要进入状态机，输出文件仍要校验，重试预算仍要限制，热点 key 仍要拆，snapshot 仍要验证。把所有问题推给共识，是分布式教材常见的偷懒。

Listing 17.6 控制日志条目的教学形态

```
1 enum class ControlOperationKind : std::uint16_t {
2     UpdateRouteEpoch,
3     ChangePartitionOwner,
4     CommitManifestEntry,
5     StartMigration,
6     FinishMigration,
7 };
8
9 struct ControlLogEntry {
10     std::uint64_t log_index = 0;
11     std::uint64_t leader_term = 0;
12     ControlOperationKind kind = ControlOperationKind::UpdateRouteEpoch;
13     std::uint64_t request_id = 0;
14     std::uint32_t partition_id = 0;
15     std::uint64_t route_epoch = 0;
16     std::uint64_t payload_checksum = 0;
17 };
```

Snapshot 和日志截断 控制日志无限增长不可接受。Snapshot 保存某个 log index 的状态机快照，恢复时先加载 snapshot，再回放之后的日志。Snapshot 必须记录 last included index 和 term，不能只写一个状态文件。截断旧日志前，要确认需要恢复的节点已经拥有 snapshot 或不再需要旧日志。计算作业也有同构问题：stage checkpoint 成功后，哪些 shuffle block 可以删除，哪些还要保留给回滚或重试。

状态机必须确定 共识只决定日志顺序，状态机应用日志必须确定。若应用日志时读取本地时间、随机数、目录扫描顺序或不稳定的哈希遍历，不同副本会得到不同状态。C++ 实现中，生成 manifest 或 snapshot 时不要依赖 `std::unordered_map` 的遍历顺序；要按 key、partition id 或 log index 排序。外部 I/O 也不能在日志重放时无条件重复执行，必须通过提交表和幂等协议保护。

17.8 工业设计参照

工业案例的阅读口径 读工业系统不是为了背系统名，而是读它面对的约束、核心机制、购买的能力、付出的成本，以及本书项目应采用、简化或拒绝哪一部分。

工程案例

Kafka partition、leader 和 ISR Kafka 把 topic 拆成 partition，每个 partition 有 leader，followers 复制日志。它面对的是高吞吐追加、broker 故障、生产者重试和消费者按 offset 读取。核心机制是 partition leader 定义写入口，日志 offset 定义顺序，ISR 表示足够跟上的副本集合，producer id 和 sequence number 处理重试重复。

代价是 partition 数量、leader 分布、ISR 抖动、跨机复制和消费者 rebalancing 都会影响尾延迟和可用性。本书项目可以借鉴 partition leader、offset、producer request id 和 manifest offset 的思想，但不应把每条 shuffle 记录都塞进强控制日志。数据面批量写 block，控制面提交 block 引用即可。

工程案例

Dynamo/Cassandra 风格的一致性哈希和 quorum 这类系统面对节点频繁增减、低延迟读写和跨副本不一致。核心机制包括一致性哈希或 token range、N/R/W quorum、hinted handoff、read repair、反熵和版本冲突处理。它们说明 quorum 不是单独公式，而是一整套补副本、修冲突和处理删除的机制。

代价是读写可能看到多个版本，删除需要 tombstone，后台修复消耗 I/O，最后写入胜出可能因为时钟问题丢语义。本书项目可以借鉴虚拟分片、读写 quorum 的实验和后台校验；暂时拒绝复杂冲突模型，因为计算引擎的 manifest 和 route 元数据应尽量保持单 leader 强提交。

工程案例

etcd/Raft 控制面 etcd 用 Raft 维护小而关键的 key-value 控制状态，常被 Kubernetes 等系统用于配置、租约和集群元数据。它面对的是控制面必须一致、节点可能失败、旧 leader 可能恢复、客户端 watch 可能断开。核心机制是 leader、term、log index、commit index、snapshot 和 watch revision。

代价是写入要走 leader 和多数派，吞吐不应和数据面混在一起。它给本书项目的启发非常直接：route epoch、partition assignment、manifest generation 属于控制面，可以用简化日志模拟；shuffle block 和大输出属于数据面，不要进入共识热路径。

工程案例

HDFS NameNode 和 block replication HDFS 把文件切成 block，DataNode 存储副本，NameNode 维护命名空间和 block 元数据。它面对的是大文件吞吐、DataNode 故障和副本放置。核心机制是控制面集中管理 block location 和副本健康，数据面负责大块传输；副本放置考虑机架等故障域。

代价是 NameNode 元数据成为关键控制面，需要 checkpoint、journal 和高可用机制。Compute Systems Engine 可以借鉴“控制面记录 block 引用和 checksum，数据面传输大 block”的分层；同时要记住复制数量不等于备份，误删和坏写会被复制。

工程案例

Spanner 的强一致读写和时间假设 Spanner 这类系统把复制、事务和时间边界结合起来，面对跨数据中心一致性和高可用读写。它的核心思想之一是把提交时间、复制日志和读时间戳变成协议对象，让读写能在明确时间边界下解释。

代价是系统复杂度、时钟基础设施和跨地域延迟都很高。本书项目不应照搬 TrueTime 或跨地域事务，但应学习一个原则：读语义要明确，时间假设要写出来，租约和时间戳不能被当作无限可靠的魔法。

17.9 贯穿审查：只跟踪一个 Key 的一生

分片、复制和共识一旦讲得太大，就容易变成名词列表。一个更有效的审查方法，是只跟踪一个 key：event=login。它最初属于 partition 12，owner 是 worker A，route epoch 是 41。随着流量增加，它被识别为热点，被拆成两个执行子分片，迁移到 worker B 和 worker C。期间 A 仍在处理旧请求，B 正在加载快照，C 正在追增量，Driver 又经历一次 leader term 切换。只要把这个 key 的所有事件写清，本章的大部分概念都会自然出现。

进入分片函数 记录被解析后，系统根据 key 规范化规则和 partition function version 把 login 映射到 partition 12。这里至少有三个事实：key 的字节表示、hash 或 range 规则、partition version。若两个 worker 对大小写、编码或 hash seed 理解不同，同一个业务 key 会进入不同 partition。分片函数不是数学背景，而是协议的一部分。

成为热点 全局吞吐正常时，partition 12 仍可能长尾，因为大多数 login 都打到它。报告不能只看平均每个 partition 多少记录，要看最大 partition、top-k key 占比、reduce p99、队列水位和

二级合并成本。热点 key 逼迫系统把执行身份和业务身份分开：子 key 可以用于并行，最终输出仍必须聚回 **login**。

迁移和复制 Epoch 41 时 A 有权提交 partition 12；Epoch 42 发布后，B 和 C 接管子分片。A 可以清理旧状态，可以转发旧请求，可以返回 StaleRoute，但不能在 epoch 42 下提交新结果。复制时还要问：A 的版本 100 是否已被多数确认，B 或 C 是否追到 100，哪个副本能在 A 崩溃后接管。跟踪一个 key 的版本，比背 quorum 公式更能说明哪个历史被承诺过。

读取和删除 迁移后，监控看板可以读带版本的 stale 统计，调度器不能用 stale route 决定 owner，manifest commit 必须读强一致控制面。旧子分片和旧 block 也不能立刻物理删除：旧 attempt 响应可能迟到，落后副本可能还要补日志，恢复程序可能需要解释旧文件。清理要等 manifest generation、checkpoint 和 tombstone 安全点都越过旧状态。

审计材料 最终 run report 应能按 key 查询：**login** 在 epoch 41 属于 partition 12，热点检测何时触发，拆分规则版本是多少，快照 checksum 是多少，增量日志范围是什么，Switch 的 log index 是多少，epoch 42 发布后旧请求拒绝了多少，最终 manifest 引用哪些子分片。这样的审计材料不是装饰，而是出现错账、重复提交或长尾时证明系统可信的证据。

17.10 项目落地

Compute Systems Engine 在本章不要求写完整工业级共识。目标是实现一个最小但语义清楚的分片复制模拟：Driver 维护 partition assignment、route epoch、migration state、replica progress、control log 和 manifest generation；Worker 处理数据面 block；MessageBus 注入旧路由请求、旧 leader commit、慢副本和崩溃恢复。

实现路线 第一版，给 shuffle partition 增加 **partition_id**、**partition_version** 和 **route_epoch**。第二版，实现 partition assignment 表，所有提交都走 **can_commit_partition_output**。第三版，加入热点 key 生成器，比较普通 hash、map-side combine、热 key 拆分和二级 reduce。第四版，实现冻结式再分片状态机。第五版，加入三副本元数据日志的简化多数提交。第六版，加入读模式：stale read、monotonic read、leader read。第七版，加入 snapshot 和恢复演练。

实验矩阵

实验	故障或压力输入	验收重点
热点 key	一个 key 占大多数记录	普通 hash 长尾，拆分后二级合并正确
旧路由写	epoch 41 写在 epoch 42 后到达	旧 owner 返回 StaleRoute，manifest 不变
迁移崩溃	Prepare/CatchUp/Switch 各阶段崩溃	恢复按迁移记录继续或回滚
落后副本接管	版本只在单副本上存在	未达提交点的版本不能被承诺
多数提交	三副本中一个慢、一个故障	多数可用时提交，少于多数时拒绝
旧 leader 写	新 term 后旧 Driver 尝试提交	StaleTerm 增加，manifest generation 不分叉
读一致性	follower 落后两个版本	stale/monotonic/leader read 行为可解释
snapshot 恢复	日志截断后重启	snapshot 加后续日志恢复同一状态

PartitionRunReport 报告至少输出：partition 总数、route epoch、迁移次数、stale route 数、stale term 数、hot key top-k、最大 partition 负载、二级 reduce 成本、replica lag 最大值、quorum commit 数、quorum reject 数、manifest generation、snapshot index、orphan block 清理数。没有这些指标，分片复制问题只能靠猜。

Listing 17.7 分片复制报告摘要示例

```

partitions.total=64
route.epoch=42
migration.completed=1
route.stale_rejected=17
term.stale_rejected=2
hot_key.max_share=0.61
quorum.commit=128
quorum.reject=3
replica.max_lag=2
manifest.generation=42
snapshot.last_included_index=350

```

实验：只跟踪一个 key 的一生。让 `event=login` 从普通 key 变成热点，触发拆分和迁移；在迁移切换后投递一条旧 epoch 写；让一个副本落后；再让旧 Driver 尝试提交旧 term manifest。报告要能按 trace id 列出 route epoch、owner、replica version、commit index 和最终 manifest。

源码阅读任务

源码阅读任选一条窄路径：

1. 阅读 Kafka partition leader、replica lag、producer sequence 或 consumer offset 的一条路径，说明 offset 和 leader 如何定义顺序。
2. 阅读 etcd/Raft 的 log entry、commit index、snapshot 或 watch revision 路径，说明控制面状态如何被排序和恢复。
3. 阅读 Cassandra/Dynamo 类系统的 token range、quorum、read repair 或 tombstone 设计，说明 quorum 之外还需要哪些修复机制。
4. 阅读 HDFS 或类似系统的 block location、replication、checkpoint 或 journal 路径，说明控制面和数据面如何分离。

报告必须按“约束、核心机制、购买能力、成本、迁移到本项目的方式”组织，不能只罗列名词。

验收与评分标准

本章项目评分建议：

1. 状态归属 25 分：partition、owner、route epoch、leader term、state 都进入提交路径，旧 owner 不能提交。
2. 再分片 20 分：迁移状态机可恢复，旧路由、切换崩溃、重复请求和清理都有测试。
3. 复制和读语义 20 分：提交点、replica lag、quorum、stale/monotonic/leader read 行为清楚。
4. 热点处理 15 分：能构造倾斜输入，比较普通 hash、combine、热 key 拆分和二级合并。
5. 工业判断 20 分：能说明 Kafka、Dynamo/Cassandra、etcd/Raft、HDFS、Spanner 等设计中采用、简化和拒绝的部分。

17.11 复查清单

一、是否先问状态归属 遇到 key、partition、manifest、checkpoint 或 replica，先问当前 owner 是谁，epoch 是多少，谁有权提交。若只看到 hash 或文件路径，还没有进入本章核心。

二、旧事件是否有明确处理 旧 route 请求、旧 leader commit、旧副本响应、旧 snapshot 都要进入状态机。不能静默接受，也不能静默丢弃。

三、再分片是否可恢复 Prepare、Snapshot、CatchUp、Validate、Switch、Drain 每个阶段崩溃后，都要知道恢复动作。若恢复只能“重新跑脚本”，就不可靠。

四、复制是否定义提交点 客户端看到写成功时，哪些副本已经内存保存、WAL 持久化、多数确认、状态机应用，要写清楚。没有提交点，就无法判断故障后哪个版本可信。

五、读接口是否声明一致性 控制面读、调度读、监控读不应混用一个含糊 API。读结果要带版本或滞后，调用方要选择 stale、monotonic 或 strong。

六、共识边界是否足够小 Route epoch、assignment、manifest generation、commit log 属于强控制面；shuffle block、临时 partial、可重算数据属于数据面。把边界画错，系统要么慢，要么不可信。

七、是否为第 18 章留下接口 第 18 章会把 map、shuffle、reduce、runtime、I/O 和分布式控制面合成完整计算引擎。本章留下的 **OwnershipRecord**、**PartitionRequest**、**ControlLogEntry** 和 **PartitionRunReport** 应能被直接复用。

17.12 本章习题

习题与作业

习题一：为一个日志统计系统选择分片策略。分别讨论点查、范围查询、递增时间戳写入和热点 key 下 hash、range、虚拟分片的表现。

习题二：构造旧 owner 迟到写入时间线。要求写出 route epoch、owner、请求到达顺序、服务端决策和 manifest 变化。

习题三：设计再分片状态机。至少包含 Prepare、Snapshot、CatchUp、Validate、Switch、Drain。说明每个状态崩溃后的恢复动作。

习题四：比较冻结、日志追赶、双写三种迁移策略。必须说明部分成功、旧请求、读路径和恢复成本。

习题五：设计热点 key 拆分方案。说明如何检测热点、如何生成子 key、如何二级合并、如何保持 manifest 幂等。

习题六：比较三种写提交点：leader 内存返回、本地 WAL 返回、多数副本 WAL 返回。分别说明延迟、数据丢失风险和读路径影响。

习题七：用三副本 A、B、C 手算 quorum 恢复。版本 10 分别只在 A、在 A/B、在 B/C 三种情况下，判断 A 崩溃后能否承诺版本 10。

习题八：设计读一致性接口。至少支持 stale read、monotonic read 和 leader read，并说明响应中应返回哪些版本字段。

习题九：解释 leader term、log index、commit index、apply index、snapshot index 的区别。每个概念都要对应一个故障或恢复问题。

习题十：阅读一个工业系统的分片、复制或控制面日志设计，按“约束、核心机制、购买能力、成本、迁移到本项目的方式”写报告。

第 18 章

分布式计算工程实践

第二卷最后一章不再新增一个孤立主题，而是把前面所有机制收束成一个可运行、可测量、可故障注入、可恢复的教学系统：Compute Systems Engine。它处理一批日志记录，按 key 聚合计数，经过 map、combine、shuffle、reduce 和 manifest commit，最后输出确定结果。目标不是替代 Spark、Flink 或数据库执行器，而是在一台机器上把现代计算系统的关键边界写清：数据怎样进入热路径，任务怎样并行，I/O 怎样可靠完成，远端消息怎样提交，失败后谁是权威事实。

先看最终项目要能复现的一次完整事故。输入有三个 split，其中一个 split 含非法字段，一个 key 是热点；worker 1 写出 task 0 后，Driver 在 manifest 提交后崩溃；worker 2 的 task 1 第一次写出遇到短写；task 2 的成功响应被 MessageBus 丢弃，Driver 超时重试，旧响应随后迟到。正常路径里这只是一个日志统计作业，事故路径里它同时考验解析错误、I/O 可靠写、任务状态、重试、幂等提交、热点 key、trace 和恢复。

本章的判断标准很简单：读者不能只让程序跑通，还要能解释程序为什么可信。解释必须落到具体对象：InputSplit、TaskId、AttemptId、ShuffleBlock、ManifestEntry、MessageEnvelope、Checkpoint 和 RunReport。如果一个结果出错，报告应能把它缩小到解析、分区、写出、提交或恢复中的某个边界，而不是让人翻日志猜。

学习目标

学完本章后，读者应能完成八件事：

1. 把第二卷的硬件、布局、并发、运行时、I/O 和分布式机制组合成一条端到端计算流水线。
2. 为日志统计项目定义串行 reference、输入身份、输出语义和错误合同。
3. 设计 map-side combine、shuffle manifest、reduce 去重和热点 key 拆分的工程边界。
4. 区分执行事实、存储事实和提交事实，解释为什么 manifest 是恢复权威。
5. 设计 task attempt、request id、deadline、retry budget 和 checkpoint，使响应丢失和 Driver 重启不会重复提交。
6. 输出能审查的 RunReport，包含 correctness、performance、resources、faults、recovery 和 unverified boundaries。
7. 按里程碑交付最终项目，而不是一次性堆出一个不可调试的大框架。
8. 说明本章项目如何为第三卷 CPU 推理引擎留下模型文件、tensor buffer、算子任务和服务化请求的系统接口。

18.1 最终系统的第一条主线：Reference 先行

任何分布式计算项目都要先有可验证 reference。Reference 可以慢，但必须定义语义：输入格式是什么，坏行如何处理，key 如何规范化，计数类型是否溢出，输出顺序是否稳定，错误摘要如何记录。没有 reference，后面的线程池、shuffle、重试、checkpoint 和热点优化都没有正确性锚点。

日志记录和输出语义 教学项目可以从固定字段日志开始：timestamp、user id、event type 和 value。Reference 单线程读取全部输入，跳过或记录坏行，按 event type 累加 value，最后按 event type 排序输出。优化版本可以改变布局、并行方式、shuffle 方式和写出方式，但不能改变这份语义。

Listing 18.1 Reference 使用的稳定数据合同

```
1 struct LogRecord {
2     std::uint64_t timestamp = 0;
3     std::uint64_t user_id = 0;
```

```

4  std::uint32_t event_type = 0;
5  std::int64_t value = 0;
6  };
7
8  struct EventCount {
9      std::uint32_t event_type = 0;
10     std::int64_t total = 0;
11 };

```

这段结构体是内存语义，不等于文件格式。文件格式可以是文本、二进制或列式块，但解析后必须进入稳定数据合同。若未来加入 schema 版本、字符串 key 或变长字段，也要先扩展 reference，再扩展优化路径。

输入身份 输入 split 不能只是“第几个任务”。它要能定位字节范围和恢复边界。文本文件按字节切分时，非零 begin 要跳到下一行起点；end 可以读到当前行结束；错误记录要能回到 file id、offset 和 record id。若每次运行都无法证明处理的是同一批字节，benchmark 和恢复报告都不可信。

Listing 18.2 输入分片元数据

```

1 struct InputSplit {
2     std::uint64_t split_id = 0;
3     std::uint64_t file_id = 0;
4     std::uint64_t begin_offset = 0;
5     std::uint64_t end_offset = 0;
6     std::uint64_t input_checksum = 0;
7 };

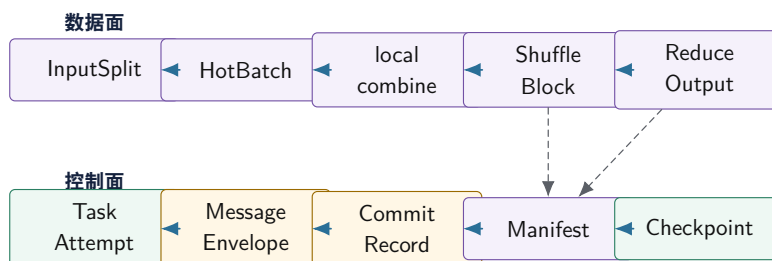
```

`split_id` 是逻辑 task 的基础。Attempt 重试时 `split_id` 不变，物理执行变的是 `attempt`。这个区分贯穿全章：逻辑身份决定语义，物理 attempt 只是实现路径。

18.2 端到端架构：控制面和数据面分开

Compute Systems Engine 的架构可以分成两条路径。数据面搬运和处理大块数据：读取 split、解析 batch、本地 combine、写 shuffle block、reduce 合并。控制面维护小而关键的事实：task 状态、attempt、manifest、checkpoint、route epoch、retry budget 和 recovery audit。控制面决定哪些数据有效，数据面负责高吞吐处理。

最终引擎：数据面搬运大块，控制面定义有效性



数据面可以重做、重发、重读；控制面决定哪个 attempt 的哪个 block 对下游可见。最终系统的恢复能力来自这条边界。

图示：本图要证明的命题是：最终计算引擎不是一个大函数，而是数据面和控制面共同维护的状态机。

模块边界 Parser 接收 `InputSplit`，输出 batch 和错误摘要；它不提交结果。Kernel 接收

热字段 span，输出本地 partial；它不理解重试。Partitioner 接收 partial，输出按 partition 分组的 block；它不写 manifest。I/O 层负责临时文件、buffer 生命周期、短写和 checksum；它不决定业务生效。Driver 维护 task table、retry、commit 和 checkpoint；它不解析每行日志。MessageBus 传递消息和故障事件；它不决定业务成功。

边界清楚不是为了目录好看，而是为了事故时能定位责任。若 Parser 直接写 manifest，解析错误会污染提交语义；若 I/O completion handler 直接提交业务结果，短写和迟到 completion 会绕过状态表；若 Runtime 理解业务 key，任务运行时就无法复用到后续推理引擎。

目录建议 项目可以按责任组织：`engine/model` 放稳定 ID 和消息结构；`engine/reference` 放串行 reference；`engine/input` 放 split 和解析；`engine/kernel` 放 combine、filter、partition；`engine/runtime` 放队列、线程池和任务图；`engine/io` 放可靠写和 block 校验；`engine/driver` 放 task 状态、manifest 和 checkpoint；`engine/fault` 放故障注入；`engine/report` 放 RunReport。

18.3 Map、Combine、Shuffle 和 Reduce

朴素 shuffle 会把每条记录都发给 reduce。若一亿条日志只有一万个 event type，这样会浪费大量网络、序列化和写盘成本。Map-side combine 先在每个 worker 本地按 key 聚合，再把 partial 发给 reduce。发送量从记录数下降到本地出现过的 key 数。这是第 13 章 reduce 原则在网络尺度上的应用。

本地聚合策略 如果 event type 是小范围整数，可以用连续数组，内存访问规则、容易向量化；如果 key 稀疏或是字符串，可以用哈希表，但要承担随机访问、分配和 rehash 成本；如果输入已经按 key 排序，可以顺序合并。项目报告不应只写“用了哈希表”，而要记录 key 基数、最大 bucket、combine ratio、内存峰值和输出 block 数量。

Listing 18.3 小整数 key 的本地 combine

```
1 std::vector<std::int64_t> combine_events(
2     std::span<const LogRecord> records,
3     std::uint32_t event_type_count) {
4     std::vector<std::int64_t> partial(event_type_count, 0);
5     for (const LogRecord& record : records) {
6         if (record.event_type < event_type_count) {
7             partial[record.event_type] += record.value;
8         }
9     }
10    return partial;
11 }
```

真实项目不能静默忽略越界 event type。教学片段只突出 combine 结构；生产版应把非法 key 进入错误摘要，并让 reference 和优化路径共享同一错误合同。

ShuffleBlock 和 Manifest Shuffle 可以先用本机内存队列模拟，也可以写本地文件再由 reduce 拉取。无论哪种方式，都要有 manifest 描述哪些 block 有效。Reduce 不应扫描临时目录，也不应接受 MessageBus 里随机到达的所有 partial。它只读取 manifest 接受的 block。

Listing 18.4 Shuffle block 和 Manifest entry

```
1 struct ShuffleBlock {
2     std::uint64_t block_id = 0;
3     std::uint64_t map_task_id = 0;
4     std::uint32_t attempt = 0;
5     std::uint32_t reduce_partition = 0;
6     std::uint64_t byte_size = 0;
7     std::uint64_t checksum = 0;
```

```

8 };
9
10 struct ManifestEntry {
11     std::uint64_t generation = 0;
12     std::uint64_t logical_task_id = 0;
13     std::uint32_t accepted_attempt = 0;
14     ShuffleBlock block;
15 };

```

Manifest entry 是提交事实。临时文件存在只是存储事实；worker 返回 Ok 只是执行事实；只有 manifest 接受某个 logical task 的某个 attempt，下游才允许读取对应 block。这个规则是最终项目可靠性的中心。

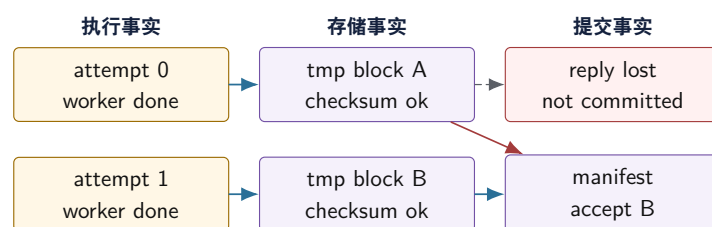
Reduce 端去重 Reduce 端最危险的错误是重复合并同一个 logical map task。Attempt 0 写出后响应丢失，Attempt 1 重试并提交；如果 Reduce 扫描目录，它会把两个 block 都读进去，计数翻倍。正确策略是 reduce 根据 manifest 拉取已接受 attempt，或按 logical task 去重。按 attempt 去重不够，因为不同 attempt 仍可能属于同一个 logical task。

热点 key 如果某个 key 占输入 90%，普通 hash partition 会让一个 reduce partition 长尾。第一层优化是 map-side combine，减少重复传输；第二层是热 key 拆成多个执行子 key，分给多个 reduce；第三层再做二级 reduce，把子 key 合回业务 key。子 key 是执行身份，不是业务身份。Manifest 和 RunReport 要记录 hot route version、subkey count、final merge checksum 和二级合并成本。

18.4 Attempt、提交事实和恢复权威

最终项目必须让读者区分三种事实。执行事实：某个 worker 是否做过某次 attempt。存储事实：某个 block 是否存在且 checksum 正确。提交事实：下游是否允许把这份结果计入最终输出。系统可以有多个执行事实和多个存储事实，但每个 logical task 只能有一个提交事实。

执行事实、存储事实和提交事实不能混淆



Attempt 0 可以真实执行过，block A 也可以真实存在，但它没有进入 manifest。恢复和 reduce 只能相信提交事实。

图示：本图要证明的命题是：最终结果由 manifest 决定，不由目录扫描或 worker 成功响应决定。

状态表 最小状态表可以分四类。Task table 保存逻辑任务：task id、stage id、current attempt、deadline、retry budget、state。Attempt table 保存物理执行：attempt、worker id、start、finish、reply state、candidate block。Block table 保存存储事实：block id、path、bytes、checksum、write state、orphan reason。Manifest table 保存提交事实：logical task、accepted attempt、accepted block、generation。

Listing 18.5 Task 和 attempt 的教学形态

```

1 enum class TaskState : std::int32_t {
2     Pending,

```



```

3   Running,
4   Retrying,
5   Committed,
6   Failed,
7 };
8
9 struct TaskRecord {
10     std::uint64_t task_id = 0;
11     std::uint32_t current_attempt = 0;
12     std::uint64_t deadline_ns = 0;
13     std::uint32_t retry_count = 0;
14     TaskState state = TaskState::Pending;
15 };
16
17 struct AttemptRecord {
18     std::uint64_t task_id = 0;
19     std::uint32_t attempt = 0;
20     std::uint32_t worker_id = 0;
21     std::uint64_t candidate_block_id = 0;
22     bool reply_received = false;
23 };

```

恢复顺序 恢复不是把旧进程内存复活。正确顺序是先读 manifest，确定哪些 logical task 已经外部生效；再读 checkpoint，恢复 job、stage、retry budget 和未完成任务；再扫描临时目录，把未被 manifest 引用的 block 标成 orphan candidate；最后重新调度 pending 或 uncertain task。若恢复程序先扫描目录并承认文件，就会把未提交 attempt 当成结果；若只读 checkpoint 而忽略 manifest，就会丢掉 commit 后崩溃的事实。

18.5 可靠 I/O、Checkpoint 和 Backpressure

最终项目的 I/O 层承接第 15 章：submit、completion 和 commit 必须分开。Worker 写出 block 时，先写临时文件，可靠写循环处理短写，完成后计算 checksum，再向 Driver 提交。Driver 接受后，manifest 才让 block 对 reduce 可见。短写、延迟 completion、取消后 completion、rename 前后崩溃，都应进入测试。

Checkpoint 内容 Checkpoint 保存控制面，不复制所有大数据。至少包含 job id、stage 状态、task table、manifest generation、retry budget、route epoch、输入身份和 idempotency 摘要。Shuffle block 本身可以留在数据面，只要有稳定路径、bytes、checksum 和 manifest 引用。Checkpoint 小而强，block 大而可校验，这就是控制面和数据面分离在最终项目里的落地。

Backpressure 贯穿流水线 读取、解析、map combine、shuffle 写出、reduce 合并和 manifest commit 都可能成为瓶颈。每个阶段都要有有界队列或资源窗口：input queue、in-flight shuffle bytes、buffer pool、reduce partition queue、checkpoint writer。队列满时，上游应等待、降速、合并或返回 Busy，而不是无限分配内存。RunReport 要记录 high watermark、busy count、wait time 和 dropped/late events。

资源预算 最终项目不是某个函数最快，而是在资源预算内稳定完成工作。预算包括线程数、队列容量、buffer pool bytes、临时文件数、in-flight requests、checkpoint frequency 和 retry count。若吞吐提升伴随内存无限增长，不能算成功；若平均延迟下降但 p99 和 recovery cost 爆炸，也要写入报告。

18.6 故障注入和 RunReport

没有故障注入的分布式计算项目只证明 happy path。最终项目应把故障注入当作一等模块，而不是调试脚本。故障规则要有 id、匹配条件、触发次数和 action；所有事故演练都能用固定 seed 或

固定 rule id 重放。

故障矩阵

故障	注入方式	预期行为
坏行	split 中插入非法字段	reference 和优化路径记录同样错误摘要
短写	第一次 write 只写部分字节	继续写剩余字节，未完整不提交 manifest
响应丢失	drop 某次 TaskReply	timeout 进入 unknown, retry 新 attempt, manifest 唯一
重复响应	同一 reply 投递两次	第二次进入 duplicate 或 late 指标
Driver 崩溃	manifest 前后分别终止	前者重做，后者承认已提交事实
Reduce busy	partition queue 高水位	Map 端背压，retry budget 不被放大
热点 key	一个 key 占大多数输入	普通 hash 长尾，拆分后正确二级合并
旧 epoch	route 切换后旧请求到达	StaleRoute，不改变 manifest

RunReport 是最终系统的用户界面 RunReport 不是日志汇总，而是可审查证据。它至少包含输入摘要、reference checksum、engine checksum、错误行数、stage 耗时、queue high watermark、task 数、attempt 数、retry amplification、late reply 数、duplicate reject 数、manifest generation、checkpoint generation、故障规则、恢复动作和未验证边界。

Listing 18.6 RunReport 摘要示例

```
input.records=3000000
input.bad_records=12
reference.checksum=0x51b76290
engine.checksum=0x51b76290
tasks.total=96
attempts.total=101
reply.late=2
commit.duplicate_rejected=3
shuffle.bytes=1843200
queue.reduce.high_watermark=65536
manifest.generation=17
checkpoint.generation=4
fault.rule=drop_task_reply_once
boundary.unverified=real_network_power_loss
```

报告要机器可读，也要人能读。机器可读便于回归比较，人能读便于排查事故。若报告只能说“success”，它不能支持工程判断；若报告能解释每个故障的状态变化，它就是系统的一部分。

18.7 里程碑：每一步都能失败、解释和回退

最终项目不应一次性实现完整框架。每个里程碑只引入一个新边界，同时保留上一版可运行结果和失败测试。这样复杂度有来源，出错时能回退。

一、**串行 reference 和输入身份** 实现输入解析、错误摘要、排序输出和 reference report。测试空文件、单行、坏行、越界 event type、超长行和固定随机输入。这个阶段回答“到底在算什么”。

二、**HotBatch 和本地 combine** 把热字段从原始记录中分离，保留 record id 回溯。实现本

地 combine, 并记录 key 数、combine ratio、内存峰值。这个阶段回答“如何少搬热路径数据, 同时不丢证据”。

三、并行 partition 和任务运行时 用 count-scan-write 替代共享 push, 建立 bucket range 和 partition manifest; 再用线程池或任务图执行 chunk 任务, 记录 ready、running、waiting、done、cancelled。这个阶段回答“多个 worker 如何写同一个逻辑输出而不互相踩”。

四、可靠写和 manifest commit 实现临时 block、短写处理、checksum、manifest append 和恢复扫描。测试写中崩溃、写完未提交崩溃、提交后崩溃。这个阶段回答“文件存在为什么不等于结果有效”。

五、MessageBus、retry 和幂等提交 把本地调用改成消息事件。加入 request id、attempt、deadline、retry budget、Busy 和 late reply。测试响应丢失后只提交一个 attempt。这个阶段回答“远端执行成功为什么不是外部生效”。

六、Checkpoint、热点 key 和两进程边界 加入 checkpoint generation、恢复 audit、热点 key 拆分和可选两进程部署。两进程部署不追求性能, 而是检查序列化、framing、process crash 和共享状态假设。这个阶段回答“系统重启后谁说了算, 以及本机模拟有没有偷共享内存的懒”。

毕业演练: 使用三个小 split、两个 worker 和一个 MessageBus。打开四个故障: 坏行、短写、成功响应丢失、Driver 在 manifest 后重启。要求最终 checksum 等于 reference, manifest 每个 logical task 只有一个 accepted attempt, RunReport 能列出 late reply、duplicate rejected、partial write 和 recovery action。

18.8 一次失败排查日记

最终系统要训练的不仅是设计能力, 还包括排查能力。假设小输入只有一百条记录, reference 输出 key A 为三十七, 优化版输出三十九。差异很小, 最容易被误判成并行顺序、随机噪声或“测试不稳定”。正确做法不是先改代码, 而是沿 RunReport 一层一层收窄。

一、先排除输入和解析 先比较 input checksum、record count、bad record count 和 reference parser 的错误摘要。若这些不同, 问题还在输入身份或 parser 合同; 不要继续查 shuffle。若这些一致, 再比较 HotBatch checksum 和 record id 回溯。若 HotBatch 与 reference 的 key/value 序列一致, 说明冷热布局没有丢字段。排查要先把上游语义排除, 而不是一看到并行版本就怀疑线程。

二、再排除输出位置 查看 count、scan、write 三阶段的 bucket range。若 count 阶段显示 chunk 4 对 bucket 2 有七条, write 阶段写了九条, 就说明 predicate 在两次执行中不一致, 或者 chunk 4 被执行了两次。若 range 正确但最终多了两条, 就继续查 manifest。Scan 和 partition 的中间账本, 是把“结果不一致”缩小到“输出位置或重复提交”的关键证据。

三、最后查提交点 Task table 显示 chunk 4 有两个 attempt。Attempt 0 写出成功, 但 TaskReply 被故障规则丢弃; Attempt 1 后来写出并提交。Manifest 却包含 attempt 0 和 attempt 1 两条 block。根因不是 reduce 算错, 而是 manifest 按 block 文件名接受提交, 没有按 logical task 去重。修复应落在 CommitRecord: 同一个 logical task 只能接受一个 attempt, 旧 attempt 只能进入 LateAttempt 或 Duplicate 分类。

四、把事故固化成回归测试 新增回归测试: `drop_reply_then_retry_keeps_one_manifest_entry`。输入固定, 故障规则固定丢 attempt 0 的 reply。测试断言 parallel checksum 等于 reference, manifest 中 chunk 4 只有一条 accepted entry, late reply count 为一, 临时目录可以存在旧 block。注意最后一条: 旧 block 存在不是错误, 错误是把它当提交事实。这个测试能防止后续重构再次把目录扫描引回 reduce。

五、补报告字段 这次事故还说明旧报告缺字段。RunReport 应增加 task commit summary: task id、accepted attempt、candidate blocks、late replies、duplicate replies、orphan blocks。报告字段不是越多越好, 而是每次事故后补上能缩短排查路径的证据。高质量系统会把排查经验沉淀成报告, 而不是只沉淀在某个人脑子里。

18.9 实现工单样例

最终项目不应以“实现一个分布式计算引擎”这种大工单开始。更稳的做法是逐张工单收敛，每张工单解决一个具体缺口，附带测试和报告字段。

工单一：输入身份 动机：错误报告和恢复无法定位输入。修改：增加 FileId、ChunkId、RecordId 和 input checksum；reference report 输出这些身份。测试：同一文件两次运行 id 稳定，修改文件后 FileId 改变，错误行能回溯 offset。完成标准：没有性能要求，只要求身份稳定。

工单二：共享 push 改为 count-scan-write 动机：共享输出锁竞争，失败后不知道写入范围。修改：filter/partition 先 count，再 exclusive scan，再写独占区间。测试：共享 push 反例、稳定顺序、chunk 写失败。报告字段：chunk count、bucket range、scan checksum。完成标准：结果与 reference 一致，输出区间可审计。

工单三：有界队列关闭合同 动机：取消或错误时消费者可能永远等待。修改：队列返回 Success、Closed、Cancelled；close drain 和 close cancel 分开；关闭时 notify all。测试：消费者等待空队列时关闭，生产者等待满队列时关闭，drain 保留已有元素。报告字段：queue high watermark、closed mode、waiter wake count。

工单四：可靠写处理短写 动机：write 返回部分字节时可能误提交坏 block。修改：WriteRequest 保存 offset 和 remaining；completion 区分 PartialWritten、Completed、Failed。测试：第一次只写部分字节，最终 checksum 正确；fatal 错误时 manifest 不提交。报告字段：partial write count、retry count、block checksum。

工单五：manifest 拒绝重复 attempt 动机：响应丢失后重试会产生两个候选 block。修改：CommitRecord 按 logical task 保存 accepted attempt；重复或迟到 attempt 返回 LateAttempt 或 Duplicate。测试：attempt 0 写出后响应丢失，attempt 1 提交，attempt 0 迟到。报告字段：accepted attempt、late reply count、duplicate rejected。

工单六：恢复从 manifest 重建状态 动机：Driver 崩溃后内存状态丢失，可能重复提交。修改：启动时读取 manifest 和 checkpoint，重建 task table；旧 epoch 响应拒绝。测试：manifest append 前崩溃要重做，append 后崩溃要承认已提交。报告字段：recovered committed tasks、rescheduled tasks、stale replies。

18.10 工业设计参照

工程案例

Spark: stage、shuffle 和 output commit Spark 的核心经验不是“分布式计算可以并行”，而是把宽依赖切成 stage，把 map output 和 shuffle metadata 管起来，把 task attempt 和输出提交分开。它面对 executor 故障、长尾 task、shuffle 文件丢失和 speculative execution。核心机制是 stage boundary、task attempt、map output tracker、shuffle block 和 output commit protocol。代价是 driver 元数据和 shuffle 服务成为关键路径，speculation 也会放大资源消耗。

本书项目应借鉴 stage、attempt、shuffle manifest 和 speculative attempt 的幂等提交边界，但不需要实现 Spark 的完整调度、资源管理和外部 shuffle 服务。

工程案例

Flink: checkpoint barrier 和状态一致性 Flink 这类流式系统面对的是持续输入、算子状态和外部输出的一致恢复。核心机制是 checkpoint barrier、operator state、input offset 和 sink commit 的绑定。它说明 checkpoint 不能只保存“程序运行到哪里”，而要把输入位置、状态和输出提交点放在同一一致边界里。代价是 checkpoint 频率、状态大小和外部 sink 协议会直接影响延迟和吞吐。

本书项目是批处理模拟，但应借鉴这个原则：checkpoint 保存控制面和提交边界，不只保存内存 dump。若只保存 task table，不保存 manifest generation，恢复后仍可能重复输出。

工程案例

DuckDB/ClickHouse: 向量化执行和 pipeline 单机分析型执行器的经验同样适用最终项目。DuckDB、ClickHouse 这类系统强调列式或向量化 batch、pipeline、selection vector、压缩和 operator profiling。它们面对的是 CPU cache、SIMD、分支、内存带宽和复杂查询管线。核心机制是批量处理热字段，让算子间传递结构化 batch，并输出可解释 profile。

本书项目应借鉴 batch、selection vector、operator profile 和 pipeline breaker 的思想。Map-side combine 和 reduce merge 不应只当分布式概念，也要继承单机执行器的热路径纪律。

工程案例

Ray/actor runtime: 任务、对象和故障边界 Ray 等系统把分布式执行抽象成 task、actor 和 object store，面对的是任务依赖、对象传输、worker 故障和调度可观察性。它提醒我们：数据对象和任务状态要有稳定身份，运行时不能只靠调用栈理解依赖，失败后要知道哪些对象可重建、哪些需要重新提交。

本书项目不需要复制 Ray 的 API，但应借鉴对象身份、任务 trace 和故障重建的思想。ShuffleBlock、ManifestEntry 和 TaskRecord 就是教学版的对象和任务账本。

18.11 面向第三卷的接口

第二卷没有直接写 CPU 推理引擎，是为了先把计算系统地基打稳。第三卷会处理模型权重、tensor buffer、算子内核、batch 调度、KV cache、请求 deadline 和流式输出。这些不是全新问题，而是本章接口的延伸。

模型文件对应输入和 manifest 模型文件有 tensor 名称、shape、dtype、量化参数、checksum 和版本。它比日志输入更严格，但同样需要输入身份、可靠读取、manifest 和版本边界。加载权重时，CompletionRecord、checksum 和 manifest 仍然适用。

算子对应 kernel 和 task 矩阵乘、归一化、softmax、采样和量化都需要 scalar reference、误差合同、HotKernelReport、MemoryShape 和 KernelVariant。线程池和任务图会调度 layer、tile、token 或 batch。背压从 shuffle queue 变成 request queue 和 token generation queue。

KV cache 对应生命周期和所有权 KV cache 是跨 token 持续存在的状态，既要高效访问，又要按请求隔离。它会重新考验 BatchView、buffer ownership、ProgressAndReclaim、队列关闭和取消语义。若第二卷项目已经把所有权和状态机写清，第三卷进入 KV cache 时就不会从零开始。

18.12 复查清单

一、**Reference 是否仍然是权威** 所有优化版本、故障版本和分布式模拟都要能回到 reference。若 reference 未定义错误行、输出顺序或溢出语义，先补 reference，不要继续优化。

二、**Manifest 是否是唯一提交入口** Reduce 和恢复都只能读取 manifest 接受的 block。临时目录、worker Ok 响应和 checkpoint 内存状态都不能越过 manifest。

三、**故障是否可复现** 响应丢失、短写、Driver 崩溃、旧 epoch、Reduce busy 和热点 key 都要用固定规则复现。随机压力可以补充，不能替代确定性故障。

四、**报告是否能解释失败** RunReport 应能让读者沿输入、batch、partition、block、manifest、checkpoint 查到差异位置。若失败只能靠翻源码，报告还不合格。

五、**性能结论是否有边界** 性能报告必须带输入、环境、线程数、阶段耗时、资源峰值和未验证条件。移动端或受限 Linux 环境的数字不能被写成通用硬件结论。

六、**第三卷接口是否克制** 可以预留 ModelArtifact、TensorBuffer、KernelTask 和 InferenceRequest 的接口方向，但不要在第二卷展开 AI 算子正文。最终章的任务是收束计算系统，不是提前分散主题。

源码阅读任务

源码和工业阅读任选一条：

1. 阅读 Spark 的 task attempt、shuffle block 或 output commit 相关路径，说明它如何避免重复输出。
2. 阅读 Flink checkpoint 和 sink commit 的一条路径，说明 input offset、operator state 和外部输出如何绑定。
3. 阅读 DuckDB、ClickHouse 或类似执行器的 vectorized pipeline/profile 路径，说明 batch 和 pipeline breaker 如何影响性能证据。
4. 阅读 Linux 的 futex、page cache、epoll 或进程退出路径之一，必须从本章项目现象进入，不允许只列函数名。

报告按“项目现象、源码入口、能解释的事实、不能证明的边界、回到项目的设计结论”组织。

验收与评分标准

最终项目评分建议：

1. 语义完整性 20 分：reference、输入身份、错误合同、输出排序和 checksum 清楚。
2. 工程结构 20 分：driver、worker、runtime、io、manifest、bus、report 边界清楚，核心 ID 稳定。
3. 故障和恢复 25 分：短写、响应丢失、重复 attempt、Driver 重启、旧 epoch、热点 key 都有确定性测试。
4. 性能证据 15 分：阶段耗时、资源高水位、瓶颈迁移和未验证边界写入 RunReport。
5. 工业判断和第三卷衔接 20 分：能说明从 Spark、Flink、向量化执行器、Linux 路径中采用、简化和拒绝的部分，并能把接口迁移到 CPU 推理引擎。

18.13 本章习题

习题与作业

习题一：为日志统计系统写端到端数据流。要求包含 InputSplit、HotBatch、map-side combine、ShuffleBlock、ManifestEntry、ReduceOutput 和 Checkpoint。

习题二：设计一个 reference 合同。说明输入坏行、event type 越界、value 溢出、输出排序和错误摘要如何处理。

习题三：构造响应丢失事故。Attempt 0 写出 block 后 reply 被 drop，Attempt 1 提交，Attempt 0 reply 迟到。写出 task table、block table、manifest table 的状态变化。

习题四：设计 shuffle manifest。说明它记录哪些字段，为什么 reduce 不能扫描临时目录。

习题五：设计 checkpoint 内容清单。要求说明哪些属于控制面，哪些属于可校验数据面，恢复时按什么顺序读取。

习题六：设计热点 key 实验。比较普通 hash、map-side combine、热点拆分和二级 reduce，报告网络字节、reduce p99、最终 checksum 和二级合并成本。

习题七：写一个故障注入表。至少包含坏行、短写、丢响应、重复响应、Driver 崩溃、Reduce busy、旧 epoch。每项写注入方式、预期行为和报告字段。

习题八：为最终项目设计 RunReport。字段必须覆盖 correctness、performance、resources、faults、recovery、unverified boundaries。

习题九：把本章接口迁移到 CPU 推理引擎。说明 ModelArtifact、TensorBuffer、KernelTask、KV cache、InferenceRequest 分别复用哪些第二卷能力。

习题十：写一份最终项目评审报告。每个模块用“输入输出合同、不变量、失败路径、指标、未验证边界”五项说明。

附录 A

实验路线

第二册的实验围绕 Compute Systems Engine 展开。它不是一组互不相干的 microbenchmark，而是一条逐步扩展的工程主线：先建立 reference、测量纪律和阶段报告，再加入数据布局、SIMD、同步、并行算法、运行时、I/O 背压和本机分布式模拟。每个阶段都要保留可运行代码、正确性测试、性能报告和失败输入。

核心思想

本册实验的目标不是追求一次跑出最高吞吐，而是训练读者把系统拆成可验证的合同、可复现的测量和可解释的性能瓶颈。

A.1 统一交付格式

每个实验都必须提交五类材料。第一，环境记录：CPU 型号、逻辑 CPU、物理核心、SMT、NUMA、内存、编译器版本、编译选项、操作系统、是否容器或 Termux。第二，输入记录：规模、分布、随机种子、错误样本、边界样本。第三，正确性记录：reference、checksum、输出规模、不变量和失败样例。第四，性能记录：运行命令、重复次数、原始数据、统计摘要和异常值处理。第五，结论边界：哪些机器、输入和参数下结论成立，哪些尚未验证。

验收与评分标准

实验报告不得只给一张耗时表。完整报告至少包含环境、输入族、命令、reference 对照、原始数据摘要、阶段报告、图表或表格、结论边界和下一步实验。若平台不支持 `perf`、NUMA 或亲和性，报告要写清限制，并用用户态 trace 或简化指标替代。

A.2 统一对象账本

从第一阶段开始，实验就使用同一套对象语言。这样第 18 章最终项目不是突然出现的新框架，而是前面实验逐步长出来的系统。

对象	含义	最早出现的实验
InputSplit	输入文件和字节范围的稳定身份	顺序 reference、输入生成器
HotKernelReport	单核热循环的输入族、反汇编、计数器 and 结论边界	核心流水线、分支预测、SIMD 前置实验
HotBatch	热字段列式批次和 record id 回溯	数据布局、SIMD、解析内核
TaskId/AttemptId	逻辑任务和物理执行尝试	线程池、任务图、故障重试
ThreadPoolPlan	worker 数、任务类别、队列容量和关闭策略	线程调度、阻塞隔离、线程池实验
WorkerSnapshot/ThreadTimeLine	worker 状态、任务时间线和等待点证据	线程调度、任务运行时、性能复盘
QueueContract	容量、谓词、关闭、drain、cancel 和背压语义	锁、条件变量、有界队列、I/O 队列
AtomicProtocolNote	atomic 变量用途、内存序、线性化点和生命周期说明	原子操作、发布协议、任务完成状态
LockFreeLifecycleReport	无锁结构的模型、线性化点、进展保证和回收策略	SPSC ring、ABA、hazard、epoch、RCU
ShuffleBlock	map 输出的可校验数据块	partition、I/O 写出、shuffle
ManifestEntry	外部可见提交事实	I/O commit、reduce 去重、恢复
MessageEnvelope	跨 worker 消息身份和状态	MessageBus、重试、迟到响应
OwnershipRecord	partition owner、epoch 和迁移状态	分片、热点 key、旧 owner 拒绝
Checkpoint	恢复时读取的控制面快照	I/O、分布式恢复、最终项目
RunReport	正确性、性能、故障和边界证据	每个阶段报告、毕业项目

A.3 阶段一：测量基础、热循环报告和顺序 reference

目标是让项目有可信地基。实现顺序事件计数、顺序归约、输入生成器、checksum、阶段报告、benchmark harness 和 **HotKernelReport**。读者要能解释为什么 reference 可以慢，但必须简单、确定、可复查；也要能解释为什么一个热循环报告必须同时保存输入族、编译选项、反汇编、计数器或替代证据，以及结论边界。

实验	训练目标	必交证据
环境记录	建立测量上下文	lscpu 、编译器版本、运行命令
顺序归约	固定语义和 checksum	reference 输出、边界输入、重复运行一致性
热循环报告	把单核瓶颈写成证据包	HotKernelReport 、反汇编摘要、计数器可用性
阶段报告	记录 records in/out 和状态大小	CSV 或文本报告、字段定义、样例输出
坏 benchmark 复盘	识别被优化掉、测错路径和噪声	错误代码、修复代码、反汇编或计时证据

A.4 阶段二：硬件成本和数据布局

目标是把数据放置、访问顺序和缓存/TLB 成本变成可测事实。实验从 AoS/SoA、顺序访问、随机访问、stride、TLB 覆盖和 false sharing 开始，再连接到事件流中的热字段列式布局。

实现 **HotEventBatch** 和 **ColdEventPayload** 两种布局。热路径只扫描 **event_type**、**session_id** 和时间戳列；冷字段保留在单独区域。报告比较对象数组、列式热字段和混合布局的 records/s、cache miss 或可替代指标、临时字节和代码复杂度。

A.5 阶段三：单机高性能内核

目标是把 map、filter、scan、partition、histogram、top-k、join、sort、stencil 和 graph frontier 做成 kernel suite。每个内核都要有 reference、优化版、输入族和报告。

内核	最小实现	压力输入
Stable filter	count、scan、write 三阶段	全通过、全不通过、低选择率、最后块不满
Thread-local histogram	his- 每 worker partial，最后合并	均匀 key、单热点、Zipf、稀疏 key
Top-k	精确聚合后选择，稳定 tie-break	分散强 key、全 tie、候选不足
Join	小表 build，大表 probe 或 sort-merge	重复 key、空匹配、输出爆炸
Graph frontier	CSR、vector frontier、bitmap frontier	规则图、随机图、幂律图

A.6 阶段四：SIMD、ILP 和 Roofline

目标是把单线程内核优化从经验变成模型。读者先写 scalar reference，再尝试自动向量化，阅读优化报告和汇编，最后才决定是否手写 intrinsics 或保留 portable 版本。

必做实验包括多累加器归约、数组 map 融合、阈值过滤、ASCII 字节分类、简单 transpose 和 dot product。每个实验都要回答：这个内核是 compute-bound 还是 memory-bound；SIMD 是否改变字节流量；尾部和错误输入怎样处理；数值重排是否允许。第 2 章的 [HotKernelReport](#) 在这里升级为 SIMD 版本对照：同一输入族下必须同时保留 scalar reference、自动向量化版本、可选手写版本、反汇编摘要和误差或 checksum。

常见误区

SIMD 实验不能只测长度刚好是向量宽度倍数的输入。必须覆盖长度为 0、1、向量宽度减一、向量宽度、向量宽度加一、非对齐起点和大输入。

A.7 阶段五：线程、同步和内存模型

目标是让读者把共享状态正确性放在性能之前。实验从线程池阻塞污染、mutex counter、atomic counter、condition variable queue 和 semaphore 开始，再进入 memory order、false sharing、SPSC ring buffer、MPMC queue、ABA 和内存回收。本阶段的交付物不是“换成更高级同步原语”，而是能写出 [ThreadPoolPlan](#)、[QueueContract](#)、[AtomicProtocolNote](#) 和 [LockFreeLifecycleReport](#)，让每个等待点、发布点和回收点都可审查。

实验	必须证明	常见误区
Mutex counter	互斥保护不变量	把锁成本归因成线程数不足
Atomic counter	原子性不等于扩展性	以为 relaxed 会消除 cache line 争用
Condition queue	等待谓词和关闭语义正确	用 <code>if</code> 代替 <code>while</code> 等待
Memory order	release/acquire 建立发布关系	把 x86 现象写成 C++ 语义
Lock-free queue	线性化点和回收方案清楚	只写 CAS，不处理 ABA 和生命周期

A.8 阶段六：并行算法和任务运行时

目标是从“开线程”进入“组织任务”。实现 reduce、scan、partition、thread-local histogram、任务粒度扫描、线程池、任务图、work stealing 指标和 worker snapshot。

实验必须记录 task submit、enqueue、dequeue、start、finish、block begin、block end 和 worker

state。没有内核 trace 权限时，用户态 trace 是最低要求。报告要能解释队列等待、worker 空闲、长尾任务、阻塞任务和上下文切换之间的关系。第 9 章的 [WorkerSnapshot/ThreadTimeline](#) 在这里升级为任务图 trace：线程池里看到的 blocked worker，要能映射到第 14 章的 ready count、continuation 和 work stealing 决策。

构造一个混合 CPU 任务和模拟阻塞 I/O 的线程池实验。比较单池、分池、增加 worker 和异步模拟四种方案。报告 p95 排队延迟、worker 空闲比例、队列最大长度、上下文切换和输出一致性。

A.9 阶段七：I/O、异步和背压

目标是把计算管线接到有限速率的上下游。实现有界队列、批处理、watermark、限流、背压传播、错误队列和 shutdown protocol。读者要能解释队列满时应该阻塞、丢弃、降级、重试还是反压上游。

事件流项目在本阶段加入 [event_time](#) 和 [ingest_time](#) 的区分。实验要构造乱序、迟到、突发、下游变慢和 checkpoint 失败输入。报告要记录 watermark 推进、迟到数据数量、窗口修正、重试次数和最终输出一致性。

A.10 阶段八：本机分布式计算模拟

目标是在单机多进程或多线程模拟中引入网络、消息、故障和恢复。实现 [MessageBus](#)、[MessageEnvelope](#)、partition、[ShuffleBlock](#)、[ManifestEntry](#)、幂等提交、超时重试、[OwnershipRecord](#)、checkpoint、热点 key 和故障注入。这个阶段不追求完整分布式框架，而是让读者看到共享内存原则怎样扩展到消息系统。

主题	实验要求	证据
Shuffle	分区、 ShuffleBlock 、checksum、manifest	block 数量、大小分布、热点 bucket
重试和幂等	重复请求不能重复提交	request id、attempt、commit log、重复消息测试
Checkpoint	失败后从一致边界恢复	checkpoint generation、manifest generation、恢复前后 checksum
旧 owner	epoch 切换后旧请求到达	OwnershipRecord 、StaleRoute、manifest 不变
热点 key	单个 reducer 被拖慢	key 分布、长尾时间、二级 reduce 对照

A.11 毕业项目

第二册毕业项目是一个可运行的 Compute Systems Engine。最低形态包括：输入生成器、顺序 reference、[InputSplit](#)、[HotKernelReport](#)、[HotBatch](#)、阶段报告、stable filter、thread-local histogram、partition manifest、top-k、[ThreadPoolPlan](#)、[WorkerSnapshot](#)、[ThreadTimeline](#)、[QueueContract](#)、用户态 trace、可靠写、[ManifestEntry](#)、背压策略和本机 shuffle 模拟。高级形态再加入 SIMD 热内核、NUMA 初始化策略、[AtomicProtocolNote](#) 审查表、SPSC/MPMC 队列、[LockFreeLifecycleReport](#)、work stealing、[Checkpoint](#)、[MessageBus](#)、[OwnershipRecord](#)、热点 key 拆分、两进程部署和故障注入。

毕业故障	注入方式	验收重点
坏行	输入 split 中插入非法字段	reference 和优化路径错误摘要一致
短写	写 ShuffleBlock 时只完成部分字节	可靠写补齐, 未完整不提交 manifest
响应丢失	drop 某个 attempt 的成功响应	retry 后 manifest 只接受一个 attempt
Driver 崩溃 旧 owner	manifest 前后分别终止 route epoch 切换后旧请求到达	前者重做, 后者承认已提交事实 返回 StaleRoute, manifest generation 不变
热点 key	一个 key 占大多数输入	普通 hash 长尾, 拆分后二级合并正确

验收与评分标准

毕业项目按 100 分评分。语义完整性 25 分, 要求 reference、输入身份、错误合同、checksum 和失败输入完整; 性能分析 20 分, 要求实验矩阵、原始数据、阶段报告和瓶颈解释; 系统设计 20 分, 要求合同清楚、状态可观察、关闭和错误路径可靠; 恢复和故障注入 20 分, 要求短写、响应丢失、重复 attempt、Driver 崩溃、旧 epoch 和热点 key 都有确定性测试; 源码阅读与工程质量 15 分, 要求至少两组真实源码对照、C++ 接口清楚、位宽明确、测试可重复、报告可复现。

附录 B

源码阅读索引

本附录不是开源项目目录，而是第二册的源码阅读任务表。源码阅读必须从一个可验证问题进入：线程为什么迁移，futex 为什么进入内核，TLB miss 为什么上升，top-k 为什么漏报，shuffle 为什么出现热点。没有问题边界的源码阅读，很容易变成复制函数名和模块名。

核心思想

源码阅读报告要区分三类事实：代码中直接看到的控制流或数据结构，实验中直接测得的现象，以及从二者推导出的解释。不要把推测写成事实。

B.1 阅读方法

每次源码阅读只选择一个窄入口，最多沿两到三条调用路径展开。报告固定回答六个问题：入口文件和函数是什么；它对应本书哪个机制；关键状态对象是什么；正常路径和慢路径如何分开；错误、阻塞或回收边界在哪里；它如何解释本书实验中的一个现象。

验收与评分标准

源码报告评分重点不是读了多少文件，而是问题是否明确、路径是否收敛、图是否能说明状态变化、实验是否能验证解释。大段粘贴源码、泛泛复述项目架构或只列函数名，不计为高质量阅读。

B.2 工业案例报告模板

工业案例必须写成可迁移的设计分析，而不是项目介绍。每份报告固定包含七段：第一段写真实约束，例如吞吐、延迟、内存预算、故障恢复、确定性输出或多租户隔离。第二段写核心原理，例如列式 batch、work stealing、futex slow path、LSM compaction、shuffle manifest、hazard pointer 或 backpressure。第三段写关键数据结构和状态转换。第四段写 fast path 和 slow path 的分界。第五段写设计取舍：它牺牲什么，换来什么。第六段写和本书实验的对应关系。第七段写一个最小复现实验或伪代码接口。

常见误区

工业案例不是权威背书。不能因为 DuckDB、ClickHouse、Spark、Kafka、Redis、RocksDB、oneTBB 或 Folly 使用了某种设计，就直接说它适合所有场景。必须说明该设计的输入假设、硬件环境、并发模型、失败模型和不可迁移之处。

B.3 Linux 内核

主题	建议入口	阅读任务
调度与迁移	kernel/sched/core.c 、 kernel/sched/fair.c	解释 runnable、running、wakeup、migration 和 context switch 怎样对应用户态线程池指标。
亲和性和拓扑	kernel/sched/ 、 /sys/devices/system/cpu/ 相关文档路径	解释 CPU affinity 成功或失败时，用户态实验为什么会变化。
缺页和页表	mm/memory.c 、 mm/mmap.c	把 page fault、mmap、VMA 和 TLB/page walk 实验连接起来。
透明大页和 NUMA	mm/huge_memory.c 、 mm/mempolicy.c	解释 huge page、first touch、NUMA placement 对吞吐曲线的影响。
perf 事件	kernel/events/	解释 perfstat 事件从用户态请求到内核计数器配置的大致路径。
PMU Top-Down	tools/perf/ 、架构事件文档	把 retiring、frontend bound、backend bound 和 bad speculation 映射到 HotKernelReport 的证据边界。
futex	kernel/futex/ 或对应版本路径	解释 mutex/condition variable 从用户态 fast path 进入内核等待的 slow path。
epoll 和 I/O 等待	fs/eventpoll.c	解释 readiness、等待队列和异步 I/O 实验的状态变化。
排序和集合辅助	lib/sort.c 、 bitmap/hashtable 相关入口	对照第 8 章的 sort、bitmap frontier 和集合表示实验。

B.4 C/C++ 运行时和标准库实现

glibc 和 musl 适合阅读 pthread、mutex、condition variable、malloc 和系统调用封装。阅读重点是 fast path 和 slow path 的分界：无竞争时是否只走用户态原子操作，竞争时何时进入 futex，等待谓词和唤醒怎样组织，malloc 怎样处理线程本地缓存和全局 arena。

libstdc++、libc++ 和 MSVC STL 的容器与算法实现适合对照标准库语义。阅读 **std::sort**、**std::stable_sort**、**std::vector** 扩容、并行算法实现时，要把接口合同、异常安全、迭代器失效和临时空间写进报告。不要只看最终用了哪种算法。

B.5 编译器和运行时

LLVM 适合阅读自动向量化、循环优化、OpenMP runtime 和 task runtime。建议入口包括 LoopVectorize、SLP vectorizer、OpenMP runtime 的 task 和 thread 管理路径。阅读目标是解释为什么某个循环不能向量化，为什么某个任务粒度导致 runtime overhead，或者为什么 reduction 需要特殊语义。第 2 章的阅读报告应从 **HotKernelReport** 进入：先给出循环源码、优化报告和反汇编，再说明拒绝向量化或内联失败的直接理由。

oneTBB、LLVM OpenMP runtime 或其他成熟任务运行时适合阅读线程池、任务队列、工作窃取和关闭协议。报告重点是任务状态、队列结构、work stealing 触发条件、阻塞任务处理、异常传播和 shutdown。只画一个 worker 循环不够，要说明任务生命周期。

本书对象	建议阅读方向	报告必须回答
HotKernelReport	Linux perf、PMU Top-Down、LLVM 优化报告、数据库执行器 profile	输入族、二进制形状、计数器可用性和瓶颈分类如何形成一条可复查证据链
ThreadPoolPlan	oneTBB arena、OpenMP runtime、Folly executor、Java ForkJoinPool	worker 数、任务类别、阻塞任务和 shutdown 如何进入接口合同
WorkerSnapshot/ThreadPool/TaskQueue/TaskSwitch	task trace、executor metrics、Spark/Flink task metrics	running、blocked、idle、steal、queue wait 怎样成为可解释指标
QueueContract	pthread condition variable、Linux futex、blocking queue、Kafka/Redis 队列边界	等待谓词、close、drain、cancel 和背压怎样定义

B.6 并发库和无锁结构

Folly、Abseil、Boost.Lockfree、Concurrency Kit 或类似项目适合阅读并发容器、原子封装、hazard pointer、epoch reclamation、RCU 风格回收和高性能队列。阅读时先写清进度保证：wait-free、lock-free、obstruction-free 还是 blocking。然后再找线性化点、ABA 防护、内存序和内存回收边界。

常见误区

无锁源码阅读最常见的错误，是只盯着 CAS 循环。真正难点通常在线性化点、对象生命周期、内存回收和失败重试成本。报告若没有说明被删除节点何时可以释放，就没有读懂无锁结构。

本书对象	建议阅读方向	报告必须回答
AtomicProtocolNote	C++ <code>std::atomic</code> 示例、Folly/Abseil 原子封装、Linux refcount	变量用途、发布数据、内存序、线性化点、失败路径和生命周期是什么
LockFreeLifecycleReport	SPSC/MPMC 队列、Treiber stack、hazard pointer、epoch、RCU	生产者消费者模型、进展保证、ABA 防护、回收策略、关闭外置方式和测试矩阵是什么

B.7 HPC 和 AI 计算库

BLIS、OpenBLAS 和 oneDNN 适合阅读 blocking、packing、microkernel、ISA dispatch、primitive cache 和 shape specialization。第二册只要求读懂高性能内核的工程组织，不要求提前进入第三册的完整 AI 推理引擎。报告应回答：reference 在哪里，packing 怎样改变内存访问，microkernel 的 MR/NR 或 tile 形状怎样服务寄存器和 cache，fallback 怎样选择。若用作第 2 章或第 7 章案例，还要把阅读结果写成 **HotKernelReport** 的字段：输入形状、热循环摘要、指令级并行来源和不可迁移的 ISA 假设。

XNNPACK、llama.cpp CPU backend 或类似项目可以作为第三册前置阅读。第二册阶段只关注线程池、任务切分、数据布局 and 调度，不提前深挖量化算子细节。这样可以把 CPU 系统能力和 AI kernel 能力分开训练。

B.8 数据库、存储和分布式系统

Redis、RocksDB、Kafka、Spark、Flink、etcd/Raft、DuckDB 或 ClickHouse 适合阅读事件循环、后台线程、LSM compaction、分区、复制、shuffle、背压、vectorized execution、checkpoint 和故障恢复。选择项目时要跟章节问题匹配：第 2 章读向量化执行器或解析器 hot path，重点是 batch 如何减少每行分派并改善前端供给；第 8 章读 hash aggregate、sort aggregate 或 top-k；第 15 章读 I/O 和背压；第 16 章读 deadline、request id、幂等和 retry；第 17 章读 partition owner、replica progress、

leader term 或 quorum；第 18 章读 shuffle manifest、checkpoint、output commit 或 RunReport 形态。

章节主题	可选源码方向	最小报告产出
典型内核	DuckDB/ClickHouse aggregate、top-k、sort	一张数据流图、一组合同、一组输入反例
异步与背压	Redis event loop、Kafka producer/consumer、RocksDB compaction	等待点、队列边界、背压传播图
消息和幂等	gRPC deadline/status、Kafka producer id、Spark task attempt	request、attempt、deadline、重复请求处理
分区和复制	Kafka partition、etcd/Raft、Cassandra/RocksDB 相关路径	owner、epoch、replica progress、commit index
分布式 shuffle	Spark shuffle、Flink checkpoint、RocksDB checkpoint	manifest、幂等、checkpoint、失败恢复路径
任务运行时	oneTBB、OpenMP runtime、Folly executor	任务状态机、worker 指标、关闭协议

B.9 第十六至十八章的专项阅读

最后三章要求源码阅读直接服务 Compute Systems Engine 的最终对象。报告不应写“某系统支持容错”这种泛泛描述，而要把对象映射清楚：本书的 `MessageEnvelope` 对应哪个 request metadata，本书的 `ManifestEntry` 对应哪个 output commit 事实，本书的 `OwnershipRecord` 对应哪个 partition owner 或 leader term，本书的 `Checkpoint` 对应哪个恢复边界。

本书对象	建议阅读路径	报告必须回答
<code>MessageEnvelope</code>	gRPC deadline/status、Kafka producer metadata、Spark RPC/task message	timeout 证明什么，request id/attempt 如何进入状态机
<code>ManifestEntry</code>	Spark output commit、MapReduce committer、Flink sink commit	worker 输出何时从候选变成外部可见事实
<code>OwnershipRecord</code>	Kafka partition leader、etcd term/index、Cassandra token range	旧 owner 或旧 leader 为什么不能继续提交
<code>Checkpoint</code>	Flink checkpoint、RocksDB checkpoint、etcd snapshot	snapshot 覆盖到哪个 index/offset，恢复时谁是权威
<code>RunReport</code>	DuckDB/ClickHouse profile、Spark UI/event log、Flink metrics	报告如何帮助定位慢 stage、重复 attempt 或恢复动作
<code>WorkerSnapshot</code>	oneTBB/OpenMP runtime metrics、Spark executor metrics、Linux scheduler trace	worker 空闲、阻塞、运行、窃取和队列等待如何解释吞吐或尾延迟
<code>AtomicProtocol</code>	Linux refcount/RCU、Folly atomics、C++ atomic shared pointer	单机状态发布、引用计数和对象回收如何迁移到分布式提交状态

常见误区

第十六至十八章的源码阅读必须从一个事故进入，例如“响应丢失后两个 attempt 都提交”“旧 owner 在新 epoch 后继续写”“Driver 在 manifest 后崩溃”。如果报告不能把源码路径映射回这个事故，它就没有完成本册要求。

B.10 版本和可复现性

源码阅读必须记录项目、版本、commit、编译选项和阅读日期。现代开源项目变化很快，不能把某个 commit 的实现细节写成永久事实。若本地无法构建项目，可以做静态阅读，但报告必须标注“未运行验证”。若能运行最小实验，应把输入、命令、输出和观察结果附到报告中。

术语表

算术强度 操作数与数据移动字节数的比值，用来判断一个内核更可能受计算吞吐还是内存带宽限制。

Roofline 用计算峰值、内存带宽和算术强度估算性能上界的模型。

SIMD 一条指令处理多个数据 lane 的执行方式，是现代 CPU 单核吞吐的重要来源。

乱序执行 CPU 在保持单线程可观察结果不变的前提下，提前执行已经准备好的指令以隐藏延迟。

分支预测 CPU 在分支结果确定前预测执行路径，预测失败会清空错误路径上的工作。

cache line 缓存一致性和缓存填充的基本粒度，常见大小为 64 字节。

TLB 地址翻译缓存，用于缓存虚拟页到物理页的映射。

NUMA 非统一内存访问结构，不同核心访问不同内存节点的成本不同。

false sharing 多个线程写不同变量，但变量落在同一 cache line 上，导致一致性流量和扩展性下降。

内存序 原子操作对可见性和重排施加的约束，例如 relaxed、acquire、release 和 `memory_order_seq_cst`。

lock-free 一类进度保证，表示系统整体能持续前进，但不保证每个线程有限步完成。

背压 下游处理不过来时向上游施加的流量控制机制。

共识 分布式系统中多个节点在故障下对日志顺序或状态达成一致的协议问题。

索引

manifest, 181

oracle, 180

pipeline breaker, 181

reference, 178, 180

内存级并行度, 49

内核, 178

分区, 242

合同, 179

完成, 265

归约, 242

扫描, 242

提交, 265

提交业务结果, 265

材料化, 181

流水线, 181